
CTC-FA v2.1

Complete Consolidated Research Architecture

Closed Timelike Curve Falsification Architecture with integrated theory-aware T0-T3 gates, multi-cone causality, 33-model baseline, code, tests, RTM, V&V and appendices.

System Architect: Vadym Tsinderhoz

Version 2.1 | 1 September 2026

Document ID: CTC-FA-2026-V2.1-COMPLETE-EN

Complete consolidated edition | 264 pages

Supersedes: CTC-FA-2026-V2.0-EN and the v2.1 Extension Draft

VERIFIED / RECOVERED / SPECIFIED STATES ENFORCED

Complete Edition Control

Field	Value
System architect	Vadym Tsinderhoz
Document ID	CTC-FA-2026-V2.1-COMPLETE-EN
Edition type	Standalone consolidated standard: normative v2.1 control plus retained full v2.0 technical baseline
Total pages	264
Part I	Pages 1-45: normative v2.1 layer and verified reference core
Part II	Pages 47-264: complete v2.0 sections, registry, code listing, tests and appendices retained for traceability
Conflict rule	Part I v2.1 controls override conflicting or broader statements in the retained baseline
Baseline supplied PDF hash	90c4fdad85b4a08750a80bca33f8bf75beccb9af2b1782233fdf9b38e88ccf5f
Baseline embedded-source extraction	35 Python files compile; 13 fail literal syntax extraction
v2.1 verified reference core	31/31 tests passed during final build
Modified-gravity numerical state	Rotating k-essence and EdGB: SPECIFIED / NOT IMPLEMENTED

Why this is a complete standalone document. All normative v2.1 controls, the new executable reference core, the full v2.0 architecture, all 33 registry profiles, algorithms, code listing, historical test output, RTM, V&V plan and appendices are contained in this single PDF. External source and manifest artifacts remain required for executable reproducibility.

Consolidated Contents - Part I

Section	Page
Cover and complete-edition control	1-2
Consolidated contents	3-4
How to read the evidence states	5
0. Authorship, provenance, scope and claim boundary	6
1. Executive summary	7
2. v2.1 delta and corrections register	8
3. Theory, solution and evidence are different objects	9
4. Mathematical contract	10
5. Multi-cone causal architecture	11
6. T0-T3 preflight plane	12
7. Algorithms and acceptance logic	14
8. Quadratic k-essence integration profile	15
9. Einstein-dilaton-Gauss-Bonnet integration profile	16
10. Missing prior art: claim-by-claim integration	17
11. Theory coverage manifest	18
12. Verification and validation master plan	19
13. Reproducibility and integrity standard	21
14. Scientific interpretation and claim discipline	22
15. Conclusions and references	23
Appendices A-D: requirements, interfaces, benchmarks, change control	24-27
Part I.B-D: verified reference core, full source and test output	28-45
Part II divider	46

Consolidated Contents - Part II

Retained v2.0 technical baseline	Page
0. Authorship, Provenance, Scope and Claim Boundary	47
1. Executive Summary	49
2. Corrections Register: v1.0 to v2.0	52
3. Scientific Contract and Status Logic	61
4. Mathematical Foundation	63
5. Layered Computational Architecture L0-L19	67
6. CTC Detection Algorithms	71
7. Metric and Construction Registry - 33 models	74
8. Analytic Benchmarks and Non-Circular Derivation	86
9. Flat Quotients: Misner, Grant and Polarised Hypersurfaces	89
10. Horizon Taxonomy and Causal Failure Modes	91
11. Energy Conditions, Exactly	92
12. Quantum and Semiclassical Admissibility	94
13. Numerical Relativity and On-Shell Dynamics	96
14. Reference Software Architecture	98
15. Verification Results	99
16. Verification and Validation Master Plan	104
17. Requirements Traceability Matrix	106
18. Failure Modes and Scientific Risk Register	108
19. Prior Art, Original Contribution and Non-Overlap	110
20. Repository, Interfaces and Reproduction	112
21. Effectiveness Assessment	114
22. Worked Examples	116
23. Data Requirements	131
24. Research Programme and Roadmap	133
25. Publication, Reproducibility and IP Controls	134
26. Conclusions	135
27. References	136
Appendix A. Complete v2.0 Reference Source	138
Appendix B. Corrections Index	254
Appendix C. Versioning and Change Control	256
Appendix D. Claim Ladder	257
Appendix E. Release Manifest	258
Appendix F. Machine-Readable Registry Records	260
Appendix G. Historical Self-Test Output	263
Appendix H. Notation and Convention Sheet	264

How to Read This Complete Edition

VERIFIED

Executed or independently recomputed in the present build, with a stated domain and test route.

RECOVERED / REQUIRES RERUN

Retained from the supplied v2.0 PDF or reconstructed from its listing; useful for traceability but not silently treated as a fresh execution.

SPECIFIED / NOT IMPLEMENTED

Mathematical contract, interfaces and V&V; gates are complete; no numerical physical verdict is claimed.

Normative precedence. Part I defines the valid claim boundary for the complete v2.1 edition. Part II is retained in full because it contains the detailed 33-model registry, algorithms, worked examples, source listing, historical tests, RTM and appendices. Where Part II says v2.0 or reports historical execution, read it together with the evidence-state ledger in Part I.B.

Scientific boundary. The complete edition is a falsification architecture for declared theories, solutions, topologies and physical characteristic modes. It is not a universal proof about every theory in physics and is not evidence that a realizable time machine exists.

0. Authorship, Provenance, Scope and Claim Boundary

CTC-FA v2.1 extends a system-level research architecture that integrates established results from relativity, causal theory, exact solutions, numerical methods and scientific software. The original architectural contribution is the typed integration and evidence logic. Einstein's equations, historical metrics, characteristic theory, chronology-protection results and the 2026 scalar-tensor analysis remain the work of their cited authors.

0.1 What this revision adds

- A strict separation between a **theory**, a **solution candidate** and an **evidence bundle**.
- A field-equation adapter gate: a metric is not attributed to a theory until its coupled equations are checked on the declared domain.
- A principal-symbol layer that represents every physical propagation mode, including non-quadratic characteristic sheets.
- A common-temporal-function rule for negative causal certification across all physical modes.
- Integration profiles for quadratic k-essence and Einstein-dilaton-Gauss-Bonnet gravity (EdGB).
- A guarded import of Lee-Nilsson-Thakur 2026, including topology, sign, hyperbolicity and EFT-validity checks.

0.2 What this revision does not claim

- It does not claim to have enumerated, encoded or tested every theory proposed in physics.
- It does not infer a theory-level verdict from one solution, coordinate chart or parameter sample.
- It does not turn a timelike local direction into a CTC unless global closure is established.
- It does not equate superluminality, loss of hyperbolicity, Cauchy horizons, chronology horizons and CTCs.
- It does not accept a perturbative CTC found where the truncation is uncontrolled as a physical prediction.
- It does not treat gravitational-wave echoes as unique evidence of CTCs.

Maximum admissible statement

CTC-FA can classify a declared solution sector under explicit assumptions. It cannot certify an entire theory as causal unless all physically admissible solutions and all physical characteristic modes have been covered by a valid theorem - a condition not met by the present registry.



Figure 1. v2.1 makes theory identity, solution validity and causal evidence separate typed objects.

1. Executive Summary

The decisive missing element in CTC-FA v2.0 was not another metric. It was a representation of the physical theory that produces a metric and of the characteristic cones on which its perturbations propagate. In General Relativity, the metric cone is normally the causal cone. In k-essence, Horndeski and EdGB-type effective theories, scalar and tensor modes can follow different characteristic surfaces. A metric-only PASS can therefore coexist with an acausal or ill-posed field sector.

v2.1 closes this gap with a four-gate preflight plane: T0 records the theory; T1 verifies the coupled field equations; T2 derives the principal symbol and its physical characteristic set; T3 joins the mode-wise results without erasing scope. The existing L0-L19 architecture remains available after this preflight, preserving compatibility with the 33-model metric registry.

1.1 The new decision rule

```

Positive result:
  valid future-directed closed timelike witness on ANY physical cone
  => CTC_PROVED(mode, domain, scope)

Negative result:
  one single-valued temporal function tau with timelike gradient
  for EVERY physical cone over the entire declared domain
  => ALL_CONES_CAUSAL_CERTIFIED(domain, scope)

Otherwise:
  OFF_SHELL | OUTSIDE_EFT | ILL_POSED | NOT_IMPLEMENTED | UNRESOLVED
  
```

1.2 Integration disposition

Item	v2.1 disposition	Reason
33 baseline spacetime models	Retained as metric/topology benchmarks	They test geometric witness and certificate engines, not whole theories.
Quadratic k-essence	Adapter specified; execution pending	Needs coupled background, scalar principal symbol, EFT and hyperbolicity gates.
EdGB	Perturbative adapter specified; execution pending	Needs order bookkeeping, residual scaling, tensor/scalar characteristics and truncation control.
Lee-Nilsson-Thakur 2026	Mandatory prior art; quarantined benchmarks	Useful formulas and hypotheses, but closure, sign and characteristic claims require independent resolution.
Generic Horndeski	Characteristic polynomial interface	A single effective metric is generally insufficient; quadratic and quartic factors may coexist.
All other theories	Coverage map, not a completed run	No universal finite list or common field-equation adapter exists.

1.3 Bottom line

Result

The architecture is now capable of stating what it would mean to test modified-gravity theories without confusing them with individual metrics. The new document does not manufacture missing runs: k-essence and EdGB are integration-ready specifications with explicit stop conditions.

2. v2.1 Delta and Corrections Register

Each change below repairs a concrete category error or reproducibility gap. 'New' means a capability absent from the v2.0 type system; 'Correction' narrows a claim that could otherwise be overread.

ID	Class	Defect or gap	v2.1 control
V21-01	New	Theory name stored as metadata rather than an executable object.	TheorySpec with action, fields, couplings, conventions, equations and validity domain.
V21-02	New	One solution could be mistaken for a theory-level result.	SolutionSpec has a many-to-one relation to TheorySpec; verdicts attach to solutions and sectors.
V21-03	New	Metric residuals do not cover coupled fields.	FieldEquationAdapter returns component-wise residuals and order-aware tolerances.
V21-04	New	Only the spacetime metric cone was represented.	ConeBundle stores physical modes, principal symbols, characteristic sheets and time orientations.
V21-05	Correction	A local timelike Killing combination could be called a helical CTC.	Topology gate requires orbit closure or a deck transformation; otherwise it is only a candidate direction.
V21-06	Correction	$g_{\phi\phi}$ and $\det(g_{AB})$ could be read as a universal no-CTC theorem.	They are tagged KILLING_ORBIT_SECTOR; global negative claims still require a temporal certificate.
V21-07	New	Hyperbolicity failure and causality violation were not separated.	ILL_POSED is disjoint from CTC_PROVED and NO_CTC_CERTIFIED.
V21-08	New	EFT calculations could be extrapolated beyond their cutoff or perturbative order.	EFTValidityGate and TruncationErrorBudget are mandatory before causal interpretation.
V21-09	New	Generic Horndeski characteristics could be compressed into one effective metric.	PrincipalSymbol supports reducible and irreducible characteristic polynomials.
V21-10	Correction	Echoes could be described as a unique CTC observable.	EchoResult remains PHENOMENOLOGICAL unless the reflecting boundary and competing models are resolved.
V21-11	Integrity	The baseline PDF's embedded digest does not match the supplied final bytes.	External manifest is authoritative; PDF self-digest is prohibited.
V21-12	Integrity	Page-rendered source loses indentation.	Canonical source bundle and tests must be separate artifacts; listings are illustrative only.

ID	Class	Defect or gap	v2.1 control
V21-13	Prior art	The 2026 k-essence/EdGB CTC analysis was absent.	Added to prior art with a claim-by-claim import matrix and reproducibility gates.
V21-14	Governance	'All theories' had no bounded operational meaning.	Theory Coverage Manifest enumerates families, adapters, solution sets and explicit exclusions.

2.1 Compatibility

No baseline model ID is renamed. Existing metric-level verdicts remain valid only at their original scope. A v2.0 result can be lifted into v2.1 as `metric_mode`; it cannot be promoted to `all_physical_modes` without T0-T3 evidence.

3. Theory, Solution and Evidence Are Different Objects

3.1 TheorySpec

A TheorySpec identifies the dynamical system, not merely a label. It must carry an action or a complete equation set, field content, coupling dimensions, gauge symmetries, sign conventions, physical-mode selection rule, validity conditions and primary provenance. Two papers using the same theory name but different normalizations require distinct convention profiles.

```
TheorySpec = {
  theory_id, version, action_or_equations, fields, couplings,
  gauge_symmetries, units, signature, principal_symbol_adapter,
  physical_mode_filter, validity_predicates, provenance
}
```

3.2 SolutionSpec

A SolutionSpec binds a metric, all non-metric fields, parameter values or ranges, topology, boundary conditions, coordinate atlas, declared domain and approximation order to exactly one TheorySpec. A solution candidate remains `OFF_SHELL` until its adapter verifies every coupled field equation within an explicit error model.

```
SolutionSpec = {
  solution_id, theory_ref, metric, fields, parameters, topology, atlas,
  domain, boundary_conditions, approximation_order, error_budget, provenance
}
```

3.3 EvidenceBundle

Evidence is immutable and route-specific. A positive CTC witness stores the curve, closure certificate, time-orientation certificate and norm bound. A negative result stores the temporal function, single-valuedness proof, mode-wise timelike-gradient bounds and coverage proof. Numeric samples are diagnostics, not certificates.

3.4 Verdict attachment rule

Question	Object that can answer it	Prohibited shortcut
Does this metric sector contain a CTC?	SolutionSpec + topology + one cone	Calling the whole theory acausal.
Is this candidate on shell?	FieldEquationAdapter	Trusting the metric's historical name.
Are perturbations causal?	PrincipalSymbol + ConeBundle	Using only $g_{\mu\nu}$.
Is the declared domain stably causal?	Common temporal certificate for all physical modes	Failure to find a loop.
Is the EFT prediction controlled?	Validity predicates + truncation error budget	Extrapolating to the chronology boundary.

4. Mathematical Contract

4.1 Geometric chronology

For a time-oriented Lorentzian spacetime (M, g) , a CTC witness is a piecewise C^1 closed curve $\gamma: S^1 \rightarrow M$ whose tangent is future directed and timelike wherever defined. Closure is a global condition on M or on a declared quotient M/Γ ; it is not implied by a local norm inequality.

```
gamma(0) = gamma(1) in M/Gamma
g_{\mu\nu} \gamma_{\dot{\mu}} \gamma_{\dot{\nu}} \leq -\epsilon < 0
future(gamma_{\dot{\mu}}) = True along the complete loop
```

4.2 Stationary-axisymmetric Killing block

Let t and ϕ generate commuting stationary and axial symmetries, with ϕ periodic. The induced Killing-orbit block is

```
g_AB = [[g_tt, g_tphi],
         [g_tphi, g_phiphi]]

Delta_K = det(g_AB) = g_tt g_phiphi - g_tphi^2
```

If $g_{\phi\phi} < 0$, the closed axial orbit is a direct CTC witness, subject to domain and orientation checks. The pair $g_{\phi\phi} > 0$ and $\Delta_K < 0$ states that the local Killing plane has one spacelike axial direction and Lorentzian signature. It is a valuable sector diagnostic, invariant under symmetry-preserving radial/polar reparameterizations, but it is not by itself a global no-CTC certificate.

Helical closure guard

For $K = \partial_t + \Omega \partial_\phi$ with unwrapped t , one axial revolution changes t by $2\pi/\Omega$. The orbit is therefore not closed unless an additional identification maps the endpoint back to the start. v2.1 sends every helical candidate through the quotient/deck-orbit gate.

4.3 Stable causality

A smooth temporal function τ with an everywhere timelike gradient is a standard sufficient certificate of stable causality. In a multi-cone system, v2.1 requires the same single-valued τ to be temporal for every physical characteristic mode on the full declared domain.

```
For every physical mode a and every x in D:
  H_a^{\mu\nu}(x) (\partial_\mu \tau)(\partial_\nu \tau) \leq -\epsilon_a < 0

and \tau is single-valued on the quotient topology.
```

4.4 Principal symbol and characteristic set

For equations $E_A[u]=0$, the principal symbol $P_A^B(x,k)$ is obtained from the highest-derivative terms of the linearized equations after gauge reduction. A covector k is characteristic when the physical determinant vanishes. Gauge and constraint modes must be removed before interpreting the roots.

```
Char_x = { k != 0 : det_phys P(x,k) = 0 }
```

Generic 4D Horndeski example (Reall 2021):
 $p(k) = C^{-1}_{\mu \nu} k_\mu k_\nu * Q_4(k)$
where Q_4 is generally quartic and need not factor into metrics.

Consequently, ConeBundle supports both effective metrics and implicit homogeneous characteristic polynomials. It never forces an irreducible quartic sheet into a fictitious single metric.

5. Multi-Cone Causal Architecture

The metric cone remains one mode in the bundle. Scalar, vector and tensor polarizations may add cones or higher-degree characteristic sheets. Each carries its own hyperbolicity, time orientation, validity range and causal result.

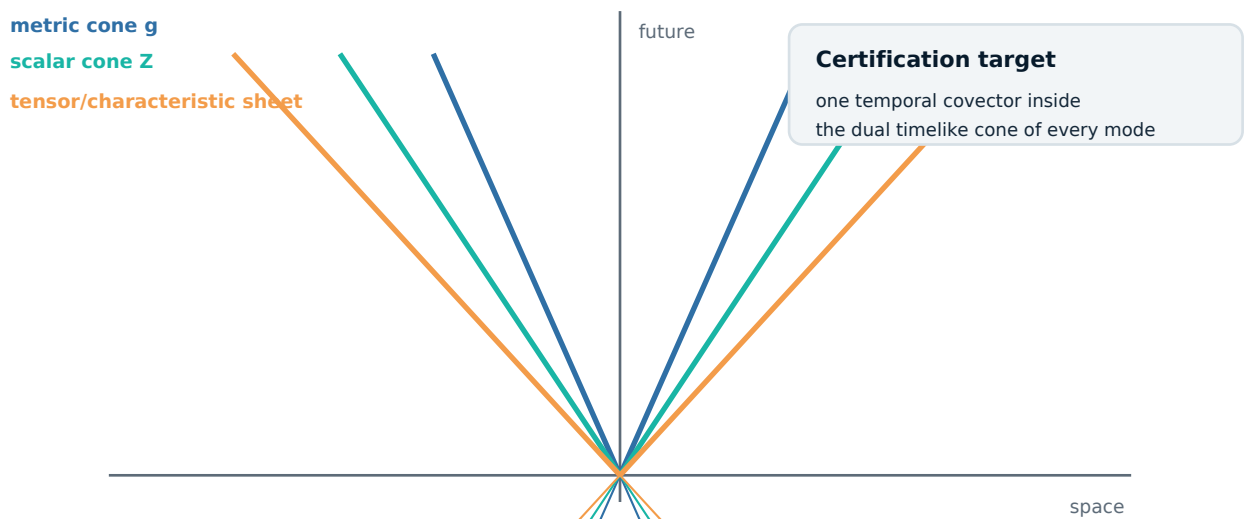


Figure 2. Schematic only: physical modes may have nested, crossing or non-quadratic characteristic sheets.

5.1 ConeBundle record

Field	Meaning	Acceptance rule
mode_id	Stable physical polarization identifier	No gauge or pure-constraint mode.
representation	Effective inverse metric or homogeneous polynomial	Exact type retained; no forced factorization.
hyperbolicity_domain	Where the initial-value problem is hyperbolic	Must cover the claimed causal domain.
time_orientation	Chosen future component	Must be continuous and compatible with other modes.
validity_predicates	EFT, wavelength, coupling and background restrictions	All must pass before a physical verdict.
provenance	Derivation, software, version and equation map	Primary source plus independent implementation.

5.2 Result lattice

Status	Interpretation	May be promoted to
OFF_SHELL	Coupled field equations fail or were not checked.	Nothing causal.
OUTSIDE_EFT	Cutoff, small-coupling or truncation control fails.	Nothing physical; diagnostic only.
ILL_POSED	Principal symbol is not hyperbolic or loses a consistent time direction.	A PDE pathology, not automatically a CTC.
CTC_PROVED	A closed, future-directed timelike witness exists for a named mode.	Theory has an acausal solution sector; not 'all solutions'.
MODE_CAUSAL_CERTIFIED	A temporal certificate covers one mode and declared domain.	ALL_CONES only after every physical mode passes with a common tau.
ALL_CONES_CAUSAL_CERTIFIED	Common temporal certificate covers all physical modes.	Only within the attached domain and solution scope.
UNRESOLVED	Neither proof path completed.	Nothing stronger.
NOT_IMPLEMENTED	Required adapter or mode derivation is absent.	Nothing stronger.

5.3 Common-time requirement

Mode-by-mode temporal functions are not sufficient if they select incompatible futures. The negative certificate must exhibit a single tau whose gradient lies in the intersection of the dual timelike cones. An empty intersection is reported as CAUSAL_INCOMPATIBILITY even if every mode is individually hyperbolic.

6. T0-T3 Preflight Plane



Figure 3. The new preflight plane feeds, but does not renumber, the existing L0-L19 stack.

6.1 T0 - Theory identity and conventions

- Parse the action or equations and normalize the sign of X , the Riemann tensor and the metric signature.
- Dimension-check every coupling and record field redefinitions.
- Enumerate gauge symmetries and the expected physical degrees of freedom.

6.2 T1 - Coupled on-shell verification

- Evaluate every metric and matter equation, not only the Einstein tensor.

- For perturbative solutions, verify the residual at each retained order and bound the remainder.
- Distinguish exact zero, symbolic simplification, interval bound and sampled diagnostic.

6.3 T2 - Principal symbol and physical modes

- Linearize about the verified background, fix gauge or quotient it, and derive the highest-derivative operator.
- Remove pure-gauge and constraint characteristics before building ConeBundle.
- Check Lorentzian signature or polynomial hyperbolicity over the whole declared domain.

6.4 T3 - Mode-wise causal join

- Run topology-aware positive witness search for every physical characteristic mode.
- Attempt a common temporal certificate for a negative result.
- Join without dropping mode, solution, domain, approximation order or provenance.

6.5 Gate order



No promotion is allowed when a gate is missing, failed or out of scope.

Figure 4. Evidence cannot bypass validity; a failed or missing gate blocks promotion.

7. Algorithms and Acceptance Logic

7.1 Theory-aware classification

```

classify(theory, solution, domain):
    t0 = validate_theory_spec(theory)
    if not t0.pass: return NOT_IMPLEMENTED or INVALID_SPEC

    t1 = theory.field_equation_adapter.verify(solution, domain)
    if not t1.on_shell: return OFF_SHELL
    if not t1.validity: return OUTSIDE_EFT

    bundle = theory.principal_symbol_adapter.build(solution, domain)
    if not bundle.complete: return NOT_IMPLEMENTED
    if not bundle.hyperbolic: return ILL_POSED

    positives = [find_ctc(mode, solution.topology, domain) for mode in bundle]
    if any(p.proved for p in positives): return scoped CTC_PROVED(positives)

    cert = common_temporal_certificate(bundle, solution.topology, domain)
    if cert.proved: return ALL_CONES_CAUSAL_CERTIFIED
    return UNRESOLVED

```

7.2 Closure algorithm for Killing combinations

Given $K = a \text{ partial_t} + b \text{ partial_phi}$, integrate its group orbit for a candidate period λ . The endpoint must equal the start in the physical manifold or differ by a verified deck transformation. Rational angular winding is insufficient when the time coordinate is unwrapped.

```

candidate_endpoint = exp(lambda K) . x
closed = (candidate_endpoint == x)
         or exists deck word w: candidate_endpoint == w . x

accept witness only if closed and timelike_bound and future_orientation

```

7.3 Perturbative chronology boundary

A sign change at the same order as the neglected remainder is not resolved. If F_N is a truncated causal diagnostic and R_N bounds omitted orders, v2.1 certifies a sign only when the interval $F_N \pm R_N$ excludes zero.

```

safe sign certificate:  inf_D(F_N - R_N) > 0
CTC sign certificate:  sup_D(F_N + R_N) < 0
otherwise:             TRUNCATION_UNRESOLVED

```

7.4 Domain coverage

- Exterior, interior, maximal extension and quotient domains are never merged implicitly.
- Coordinate singularities are handled by overlapping charts or invariant quantities.
- A horizon boundary is tagged as event, Cauchy, chronology, universal or characteristic; labels are not interchangeable.
- Non-compact domains require analytic asymptotic bounds plus certified compact coverage.

8. Quadratic k-Essence Integration Profile

8.1 Theory definition

Using $X = -g^{\mu\nu} \partial_\mu \phi \partial_\nu \phi / 2$ and signature $(-, +, +, +)$, the target profile is

$$S = \int d^4x \sqrt{-g} \left[\frac{R}{2\kappa} + P(X) \right]$$
$$P(X) = \alpha X + \beta X^2$$
$$G_{\mu\nu} = \kappa T_{\mu\nu}$$
$$T_{\mu\nu} = P_X \partial_\mu \phi \partial_\nu \phi + g_{\mu\nu} P$$
$$\nabla_\mu (P_X \nabla^\mu \phi) = 0$$

8.2 Scalar characteristic metric

For the scalar equation, the principal part can be represented, up to an irrelevant nonzero conformal factor and convention-dependent index placement, by

$$Z^{\mu\nu} = P_X g^{\mu\nu} - P_{XX} \nabla^\mu \phi \nabla^\nu \phi$$
$$P_X = \alpha + 2\beta X, \quad P_{XX} = 2\beta$$

For a homogeneous timelike background:

$$c_s^2 = P_X / (P_X + 2X P_{XX})$$

The adapter must derive the sign from the declared X convention rather than copying it. Hyperbolicity, ghost/kinetic sign, common time orientation, cone causality and EFT control are separate gates. A superluminal sound speed is not itself a constructed CTC.

8.3 Required branch data

Branch	Required inputs	Mandatory checks
Zero-charge / stealth	ϕ profile, X_0 , Kerr-(A)dS parameters, topology	Reality of ϕ , P_X branch condition, coupled residuals, scalar hyperbolicity, Killing-sector and global certificates.
Nonzero flux	q , radial/angular scalar corrections, backreacted metric	Horizon regularity, flux boundary conditions, order-by-order residuals, scalar and tensor cones.
Rotating perturbative	spin order, radial gauge, remainder estimate	Gauge map, truncation-safe sign, exterior domain coverage.

8.4 Imported 2026 benchmark - guarded

Sign inconsistency requiring resolution

The preprint states $X_0 = -\alpha/(2\beta) > 0$, which implies $\alpha/\beta < 0$, and immediately describes same-sign α, β branches. Its later table lists same-sign dS and AdS branches. v2.1 records this as PA-NT-02 and refuses numeric parameter-bound acceptance until an independent derivation fixes the convention or erratum.

8.5 k-Essence acceptance card

Gate	PASS evidence	Current v2.1 state
K0 action	Normalized action and X convention	SPECIFIED
K1 solution	Metric and phi satisfy both equations	NOT EXECUTED
K2 EFT	GM M_* >> 1 plus local derivative bounds	NOT EXECUTED
K3 scalar cone	Z is Lorentzian/hyperbolic on full domain	NOT EXECUTED
K4 topology	Closed orbit/deck certificate	ENGINE AVAILABLE; PROFILE PENDING
K5 negative proof	Common temporal function for metric and scalar modes	NOT EXECUTED

9. Einstein-Dilaton-Gauss-Bonnet Integration Profile

9.1 Theory definition

The Gauss-Bonnet invariant is $G_{GB} = R^2 - 4 R_{\mu\nu} R^{\mu\nu} + R_{\mu\nu\rho\sigma} R^{\mu\nu\rho\sigma}$. In four dimensions it affects dynamics through a non-minimal scalar coupling $f(\vartheta) G_{GB}$. The exact normalization is part of TheorySpec; alpha and beta cannot be compared across papers until their dimensions and field normalization match.

```
schematic action:  
S = integral sqrt(-g) [ R/(2 kappa) - (beta/2)(nabla vartheta)^2  
                        + alpha f(vartheta) G_GB ] d^4x
```

9.2 Perturbative solution contract

For a Kerr/Hartle-Thorne background, the scalar commonly enters at first order in a small-coupling parameter and the metric correction at second order. v2.1 stores both the coupling order and spin order, and verifies residual scaling rather than demanding an exact zero from a truncated solution.

```
g_mu_nu = g_mu_nu^(0)(M,a) + epsilon^2 h_mu_nu + O(epsilon^4)  
vartheta = epsilon vartheta^(1) + O(epsilon^3)  
  
beta Box^(0) vartheta^(1) + alpha G_GB^(0) = 0  
G_mu_nu^(1)[h] = kappa (T_mu_nu[vartheta^(1)] + D_mu_nu[vartheta^(1)])
```

9.3 Characteristic requirement

The background metric correction and the perturbation cone are distinct objects. The adapter must derive the physical principal symbol of the scalar-tensor system. For generic Horndeski backgrounds, the characteristic polynomial may contain one quadratic and one irreducible quartic factor; a single 'tensor effective metric' is accepted only when factorization is proved for the declared background.

9.4 Chronology-bound parameter search

The 2026 preprint defines azimuthal and Killing-block thresholds in a perturbative rotating background. v2.1 imports these as candidate boundary functions, then adds four guards: topology closure, full-field residuals, principal-symbol hyperbolicity and a truncation-safe zero.


```
epsilon_CTC^2(a) = min(epsilon_phi^2(a), epsilon_Killing^2(a))
```

v2.1 acceptance additionally requires:

- residual <= tolerance(order)
- chronology margin separated from zero by remainder bound
- all relevant characteristic modes hyperbolic
- candidate orbit globally closed

9.5 EdGB acceptance card

Gate	PASS evidence	Current v2.1 state
E0 normalization	Action, dimensions and field redefinition map	SPECIFIED
E1 scalar residual	Order-epsilon equation verified	NOT EXECUTED
E2 metric residual	Order-epsilon^2 equations verified	NOT EXECUTED
E3 truncation	Remainder bound excludes sign ambiguity	NOT EXECUTED
E4 principal symbol	Complete physical characteristic set	NOT EXECUTED
E5 causal result	Witness or common temporal certificate	NOT EXECUTED
E6 waveform	Boundary condition and competing echo models	HYPOTHESIS ONLY

10. Missing Prior Art: Claim-by-Claim Integration

Lee, Nilsson and Thakur (2026) propose that modified gravity should make CTCs harder, not easier, to obtain than in GR. They analyze rotating backgrounds in quadratic k-essence and EdGB and discuss parameter bounds and gravitational-wave echoes. The work is directly relevant and must be cited. v2.1 imports it through the same falsification discipline applied to every other source.

Imported claim or artifact	Value to CTC-FA	v2.1 treatment
CTCs as an EFT selection principle	Adds theory-parameter comparison beyond metric cataloguing.	Adopted as a research hypothesis, not a theorem.
Killing-block quantities g_phiphi and Delta_K	Useful symmetry-preserving local diagnostics.	Implemented at KILLING_ORBIT_SECTOR scope; no global iff claim.
Quadratic k-essence background	Concrete scalar-tensor adapter target.	Equations imported; branches quarantined pending sign resolution and reproduction.
EdGB Kerr-HT deformation	Concrete perturbative benchmark.	Imported with order, gauge, EFT and remainder gates.
Effective propagation metrics	Motivates the missing multi-cone layer.	Generalized to complete principal symbols using Reall 2021.
Echo cavity	Potential phenomenological observable.	Retained as model-dependent hypothesis; not a CTC detector by itself.

10.1 Integration issues that must be resolved

ID	Issue	Why it matters	Required resolution
PA-NT-01	Helical closure	A timelike partial_t + Omega partial_phi orbit is normally open when t is unwrapped.	Provide quotient/identification closure or downgrade to local timelike direction.
PA-NT-02	k-essence sign	$X_0 > 0$ implies $\alpha/\beta < 0$ under the printed convention, conflicting with same-sign branches.	Independent symbolic derivation or author erratum.
PA-NT-03	Characteristic completeness	Generic Horndeski propagation may involve a quartic characteristic sheet.	Derive the full gauge-reduced principal symbol for each background.
PA-NT-04	EFT/truncation boundary	A zero found where neglected terms are comparable is not predictive.	Interval remainder bound and local EFT validity map.
PA-NT-05	Echo uniqueness	Reflective boundaries have many non-CTC origins.	Model comparison and boundary-condition provenance.
PA-NT-06	Global no-CTC wording	Killing-sector inequalities do not exclude non-symmetry CTCs.	Temporal function or theorem with verified hypotheses.

Non-overlap statement

Lee-Nilsson-Thakur contribute model-specific modified-gravity calculations and an EFT selection principle. CTC-FA contributes a typed, topology-aware verification architecture that can test those claims and compare them with other theories. Neither contribution subsumes the other.

11. Theory Coverage Manifest

The phrase 'all theories' is operational only after a bounded manifest is declared. This table records the current families and the strongest honest state. A metric benchmark is not counted as a theory adapter.

Family	Representative content	Field-equation adapter	Cone model	Current status
Special relativity	Minkowski and quotients	Exact geometric baseline	Metric	BENCHMARKED
General Relativity	Vacuum, Lambda, matter-supported metrics	Partial symbolic/numeric baseline	Metric	SOLUTION BENCHMARKS
Einstein-Maxwell	Reissner-Nordstrom, Kerr-Newman	Partial baseline	Metric + EM characteristics not bundled	PARTIAL
k-essence	$P(X)=\alpha X+\beta X^2$	Specified	Metric + scalar Z	NOT EXECUTED
EdGB	Scalar-Gauss-Bonnet rotating EFT	Specified perturbatively	Full characteristic set required	NOT EXECUTED
Generic Horndeski	G2-G5 scalar-tensor	Interface only	Quadratic times quartic characteristic polynomial	NOT IMPLEMENTED
Einstein-Cartan	Metric/tetrad + torsion/spin	Interface requirement	Theory-dependent	NOT IMPLEMENTED
Einstein-aether / Hořava	Preferred timelike field; spin-0/1/2 modes	Interface requirement	Multiple mode speeds / universal horizon	NOT IMPLEMENTED
Massive / bimetric gravity	Metric pair and interaction potential	Interface requirement	Multiple characteristics	NOT IMPLEMENTED
Semiclassical gravity	$G = 8 \pi_{\text{ren}}$	State and renormalization interface	Metric + QFT propagation	NOT IMPLEMENTED
Quantum gravity candidates	No common classical solution contract	None	Framework-specific	OUT OF CURRENT SCOPE

11.1 Baseline spacetime registry

The 33 retained models span chronal controls, chronology-violating examples, quotients, horizons, wormholes and superluminal constructions. Their role is regression coverage of geometric engines.

IDs	Models	Regression purpose
M01	Minkowski	chronal control

IDs	Models	Regression purpose
M02/M02b	FLRW general / exponential	chronal cosmology controls
M03	Schwarzschild	vacuum black-hole control
M04	Reissner-Nordstrom	Einstein-Maxwell control
M05/M05b	Kerr / extremal Kerr	rotating black-hole sectors
M06	Kerr-Newman	rotating charged sector
M07/M08/M26	Godel-type / Godel / critical	causal phase-family benchmarks
M09/M10	van Stockum / Tipler cylinder	rotating matter constructions
M11	Taub-NUT	identification and extension sensitivity
M12/M31	Misner chart / boost quotient	topology benchmark pair
M27	Grant space	boost plus translation quotient
M13/M14	Spinning string / Gott strings	cosmic-string constructions
M15/M16	Morris-Thorne / time-shifted wormhole	local geometry versus global identification
M17/M18/M19	Ori 2005 / Ori 2007 core / full	compact-core and composite models
M28	Pseudo-Reissner-Nordstrom	periodic-time control
M20/M21/M22	Alcubierre / Krasnikov / paired warp	superluminal construction tests
M23/M24	AdS universal cover / periodic time	same local metric, opposite topology
M30	Minkowski with periodic time	flat quotient CTC control
M25	de Sitter static patch	horizon/chronal control
M29	Rotating BTZ	2+1 rotating black-hole benchmark

11.2 Reported baseline count versus v2.1 status

Preserved but not overpromoted

The baseline's 33/33 regression agreement is retained as a reported property of its curated model set. It is not evidence that 33 physical theories were tested, and it is not lifted to all-cone causality.

12. Verification and Validation Master Plan

12.1 Independent implementation rule

No new benchmark enters the gold set by copying formulas and expected values from the same source. The adapter, reference data and expected qualitative result must have independent derivation routes. Disagreements remain visible as unresolved records.

12.2 Test levels

Level	Target	Minimum evidence
V0	Schema and dimensions	Machine validation, units, fields, topology and parameter domains.
V1	Algebraic conventions	Limits to known theories; sign and symmetry tests.
V2	Coupled field equations	Symbolic zero or certified residual at every retained order.
V3	Principal symbol	Independent differentiation/linearization and physical-mode count.
V4	Hyperbolicity	Certified domain coverage and continuous time orientation.
V5	Positive CTC	Explicit closed, future-directed timelike witness with topology certificate.
V6	Negative causality	Common temporal function with interval/tensor inequality proof.
V7	EFT and perturbation	Cutoff map, order tracking and remainder-separated causal margin.
V8	Reproduction	Fresh environment, locked dependencies, deterministic manifest and tests.
V9	Cross-code validation	Second implementation or analytic comparison on held-out cases.

12.3 Required k-essence benchmark suite

- Canonical scalar limit $\beta \rightarrow 0$: recover the metric cone and canonical wave equation.
- Constant-gradient flat background: reproduce analytic $Z^{\{\mu \nu\}}$ and sound speed.
- Hyperbolicity boundary: detect degeneracy without labeling it a CTC.
- Topology control pair: identical local scalar/metric data with and without periodic time.
- Rotating branch: reproduce background residuals before searching any chronology boundary.

12.4 Required EdGB benchmark suite

- $\alpha \rightarrow 0$: recover Kerr and the GR characteristic structure.
- Schwarzschild slow-coupling scalar: verify the sourced scalar equation independently.
- Order scaling: residuals decrease with the predicted powers of ϵ and spin.
- Gauge shift: causal sector diagnostics transform consistently under Hartle-Thorne radial shifts.
- Characteristic completeness: physical degree count matches the gauge-reduced principal symbol.

12.5 Stop conditions

Automatic stop

Any of the following blocks a causal verdict: missing action normalization; unresolved sign convention; off-shell residual; incomplete physical mode set; loss of hyperbolicity; empty common-time cone; candidate chronology boundary inside the truncation error; or unproved orbit closure.

13. Reproducibility and Integrity Standard

13.1 Release artifact set

Artifact	Role	Integrity rule
PDF standard	Human-readable architecture and results	Carries document ID and baseline refs, not its own final hash.
Source archive/repository	Canonical executable implementation	Exact commit and clean-tree digest.
Environment lock	Dependency and interpreter resolution	Hash-pinned and machine-readable.
External manifest	Binds every artifact and byte size	Generated after all final artifacts; signed or timestamped externally.
Run records	Inputs, outputs, code and environment IDs	Immutable content-addressed records.
Benchmark data	Independent expected results	Source provenance and license recorded.

13.2 Baseline audit record

Check	Observed	Disposition
Actual baseline PDF SHA-256	90c4fdad...88ccf5f	Authoritative for the supplied bytes.
Printed final SHA-256	6b2fd18f...35b20	Does not match supplied bytes.
Printed pass-1 SHA-256	Same as printed final	Cannot authenticate a different second-pass byte stream.
Literal embedded-Python extraction	35 compile / 13 syntax failures	PDF listing is not a runnable distribution.
Reported 118 tests	Claim in baseline PDF	Requires canonical source and fresh-environment rerun.
Core analytic margins	Direct formulas reproduce the published sample values	Useful scientific cross-check; not full software validation.

13.3 Hash rule

```

build final artifacts
-> close byte streams
-> compute SHA-256 and byte sizes
-> write external manifest
-> sign/timestamp manifest

```

Never: embed 'final PDF SHA-256' inside the same final PDF bytes.

13.4 Source-listing rule

If code appears in the PDF, it is illustrative. Page layout must preserve indentation visually, but reproducibility depends on the separately distributed source tree. The document may cite file paths and commit hashes; it must not claim that copy/paste from pagination is a supported installation route.

14. Scientific Interpretation and Claim Discipline

14.1 Theory-level statements

A solution with a CTC proves that the theory admits that solution under the adapter's assumptions. It does not show that generic initial data form a CTC, that the solution is stable, that it is accessible, or that the theory is empirically excluded. Conversely, one causal solution says almost nothing about all other solutions of the theory.

14.2 Metric versus signal causality

Metric chronology and field microcausality are related but distinct. An effective cone wider than the metric cone may be compatible with a preferred-frame causal order, or may signal a UV-completion problem; the answer is theory-specific. v2.1 reports the measured cone relation and the existence or absence of global closed characteristics without replacing that analysis by a slogan.

14.3 Horizons

An event horizon is a global causal boundary; a Cauchy horizon marks loss of global hyperbolicity; a chronology horizon bounds the chronology-violating set; a characteristic horizon is mode-dependent; a universal horizon arises in preferred-foliation theories. The taxonomy is recorded before any causal inference.

14.4 Quantum and observational claims

- Hawking's chronology-protection proposal remains a conjectural physical mechanism, not a universal theorem label.
- Renormalized stress tensors are state-, boundary- and prescription-dependent; divergence claims retain those hypotheses.
- Echoes, ringdown shifts and time delays are model-dependent observables with degeneracies.
- No present result constitutes experimental evidence for a CTC or time travel.

14.5 Permitted headline

Permitted

CTC-FA v2.1 specifies how to test causal admissibility of individual solutions in GR and multi-cone modified-gravity EFTs, and adds guarded k-essence and EdGB integration profiles.

Not permitted

'All theories of Einstein and all other physicists have been proved causal or acausal.' The manifest, adapters and executed evidence required for that statement do not exist, and no finite benchmark set could establish such a universal claim.

15. Conclusions

CTC-FA v2.1 supplies the element absent from the metric-centered baseline: a theory-aware, field-equation-checked and multi-cone causal layer. The architecture can now distinguish five questions that must never be collapsed: whether a candidate is on shell; whether its approximation is controlled; whether its field equations are hyperbolic; whether any physical characteristic admits a closed timelike witness; and whether a common temporal function certifies the whole declared domain.

The 2026 k-essence and EdGB work is integrated as mandatory prior art and as a source of benchmark candidates. Its useful Killing-block diagnostics, perturbative solutions and EFT-selection idea are preserved. The v2.1 topology gate, characteristic completeness rule, sign audit and truncation budget prevent source-level assumptions from becoming unearned system verdicts.

The existing 33-model registry remains a valuable geometric regression suite. It is not a census of physical theories. The next executable milestone is deliberately narrow: independently implement and validate the quadratic k-essence adapter, then the perturbative EdGB adapter, before expanding to Einstein-aether, torsion or bimetric systems.

Final status

Architecture extension: COMPLETE IN THIS DOCUMENT. k-essence execution: PENDING. EdGB execution: PENDING. Universal 'all theories' verdict: OUT OF SCOPE / NOT ESTABLISHED.

References

- [R01] A. Einstein, 'Die Feldgleichungen der Gravitation,' Sitzungsberichte der Preussischen Akademie der Wissenschaften (1915).
<https://einsteinpapers.press.princeton.edu/vol6-trans/129>
- [R02] K. Godel, 'An Example of a New Type of Cosmological Solutions of Einstein's Field Equations of Gravitation,' Rev. Mod. Phys. 21, 447 (1949).
<https://doi.org/10.1103/RevModPhys.21.447>
- [R03] R. P. Kerr, 'Gravitational Field of a Spinning Mass as an Example of Algebraically Special Metrics,' Phys. Rev. Lett. 11, 237 (1963).
<https://doi.org/10.1103/PhysRevLett.11.237>
- [R04] M. S. Morris and K. S. Thorne, 'Wormholes in spacetime and their use for interstellar travel,' Am. J. Phys. 56, 395 (1988).
<https://doi.org/10.1119/1.15620>
- [R05] S. W. Hawking, 'Chronology protection conjecture,' Phys. Rev. D 46, 603 (1992).
<https://journals.aps.org/prd/abstract/10.1103/PhysRevD.46.603>
- [R06] A. N. Bernal and M. Sanchez, 'Smoothness of time functions and the metric splitting of globally hyperbolic spacetimes,' Commun. Math. Phys. 257, 43 (2005).
<https://arxiv.org/abs/gr-qc/0401112>
- [R07] H. S. Reall, 'Causality in gravitational theories with second order equations of motion,' Phys. Rev. D 103, 084027 (2021).
<https://arxiv.org/abs/2101.11623>
- [R08] J. Rieger, 'Causal Classification of Pathological Misner-Type Spacetimes,' arXiv:2501.00841 (2025; v5 cited).
<https://arxiv.org/abs/2501.00841>
- [R09] B.-H. Lee, N. A. Nilsson and S. Thakur, 'There and back again - Closed timelike curves as EFT selection principle,' arXiv:2602.17724 (2026).
<https://arxiv.org/abs/2602.17724>
- [R10] N. Yunes and L. C. Stein, 'Non-Spinning Black Holes in Alternative Theories of Gravity,' Phys. Rev. D 83, 104002 (2011).
<https://arxiv.org/abs/1101.2921>
- [R11] C. Charmousis et al., 'General second order scalar-tensor theory, self tuning, and the Fab Four,' Phys. Rev. Lett. 108, 051101 (2012).
<https://arxiv.org/abs/1106.2000>

[R12] T. Jacobson and D. Mattingly, 'Einstein-Aether waves,' Phys. Rev. D 70, 024003 (2004).
<https://arxiv.org/abs/gr-qc/0402005>

[R13] E. Barausse, T. Jacobson and T. P. Sotiriou, 'Black holes in Einstein-aether and Horava-Lifshitz gravity,' Phys. Rev. D 83, 124043 (2011).
<https://arxiv.org/abs/1104.2889>

[R14] CTC-FA v2.0 - Closed Timelike Curve Falsification Architecture, System Architect Vadym Tsinderhoz, CTC-FA-2026-V2.0-EN (31 August 2026).

Reference policy: primary papers and the supplied baseline document are cited for technical claims. The 2026 preprint is treated as prior art and a benchmark candidate, not as an independently reproduced result.

Appendix A. v2.1 Requirements Traceability Matrix

Requirement	Normative statement	Verification
REQ-T01	Every SolutionSpec references exactly one normalized TheorySpec.	schema
REQ-T02	TheorySpec declares fields, couplings, dimensions and conventions.	schema + dimension tests
REQ-T03	No theory-level verdict is emitted from a single solution without a theorem.	negative control
REQ-T04	All coupled field equations are checked before causal classification.	adapter test
REQ-T05	Perturbative residuals are verified order by order.	scaling test
REQ-T06	Every approximation carries a remainder/error budget.	schema + stop test
REQ-C01	Principal symbol is derived after gauge/constraint reduction.	mode-count test
REQ-C02	Every physical mode is represented in ConeBundle.	completeness certificate
REQ-C03	Irreducible characteristic polynomials are not forced into metrics.	type-system test
REQ-C04	Hyperbolicity is certified on the full claimed domain.	interval/domain proof
REQ-C05	Physical modes share a compatible time orientation for an all-cone PASS.	intersection proof
REQ-C06	A common temporal function is required for all-cone stable causality.	certificate test
REQ-G01	A local timelike direction is not a CTC without global closure.	negative control
REQ-G02	Helical candidates pass a deck-orbit or exact closure test.	topology test
REQ-G03	Killing-block results carry KILLING_ORBIT_SECTOR scope.	scope test
REQ-G04	g_phiphi < 0 uses periodicity and orientation gates.	witness test
REQ-E01	EFT cutoff and local derivative bounds precede physical interpretation.	validity test
REQ-E02	A causal zero inside the truncation remainder is UNRESOLVED.	boundary test
REQ-E03	Superluminality, hyperbolicity loss and CTCs have disjoint statuses.	status test
REQ-P01	Every imported claim has primary provenance and a claim scope.	provenance audit
REQ-P02	Source-derived expected values are not validated by copied implementation.	independence audit
REQ-P03	Conflicting source conventions block numeric promotion.	stop test
REQ-I01	Canonical source is distributed separately from the PDF.	release audit
REQ-I02	External manifest is generated after final artifacts.	manifest test
REQ-I03	The final PDF does not claim its own embedded final digest.	release lint
REQ-I04	Fresh-environment tests are recorded with exact source and lock digests.	reproduction run
REQ-O01	Echo claims identify the imposed boundary condition and alternatives.	claim lint
REQ-M01	The theory coverage manifest lists explicit NOT_IMPLEMENTED families.	manifest audit

Appendix B. Interface Sketches

These sketches are normative at the data-model level and illustrative at the programming-language level. They do not claim that the adapters are implemented in the baseline source tree.

B.1 Theory and solution records

```
{
  "theory_id": "kessence.quadratic.v1",
  "fields": ["metric:g", "scalar:phi"],
  "couplings": {"alpha": "dimensionless", "beta": "mass^-4"},
  "conventions": {"signature": "-+++", "X": "-grad(phi)^2/2"},
  "field_equation_adapter": "kessence_eom_v1",
  "principal_symbol_adapter": "kessence_symbol_v1",
  "validity_predicates": ["cutoff", "derivative_expansion"]
}

{
  "solution_id": "kessence.rotating.zero_charge.candidate_01",
  "theory_ref": "kessence.quadratic.v1",
  "domain": "r >= r_plus; theta in [0,pi]",
  "topology": {"t": "R", "phi": "S1"},
  "approximation": {"spin_order": 2, "coupling_order": 2}
}
```

B.2 Cone bundle

```
ConeBundle(
  solution_ref,
  modes=[
    CharacteristicMode(id="metric/tensor-0", representation=EffectiveMetric(...)),
    CharacteristicMode(id="scalar-0", representation=EffectiveMetric(Z)),
    CharacteristicMode(id="mixed", representation=HomogeneousPolynomial(Q4))
  ],
  completeness=ModeCountCertificate(...),
  common_time_orientation=IntersectionCertificate(...)
)
```

B.3 Evidence record

```
VerdictRecord = {
  status, theory_ref, solution_ref, mode_ids, domain, claim_scope,
  witness_refs, certificate_refs, field_equation_report,
  hyperbolicity_report, eft_validity_report, approximation_order,
  source_digest, environment_digest, run_id
}
```

Appendix C. Benchmark Acceptance Cards

C.1 Quadratic k-essence minimum release

Card	Input	Expected relation	Release threshold
K-B01	beta=0 canonical scalar	Z conformal to g	Exact symbolic match.
K-B02	Flat, constant timelike grad phi	Analytic c_s^2	Independent formula and numerical eigensystem agree.
K-B03	P_X=0 branch	Degenerate kinetic/principal behavior identified	No causal PASS is emitted.
K-B04	Periodic-time quotient	Closed characteristic exists	Deck-orbit witness exact.
K-B05	Unwrapped-time control	Same local cone, no corresponding deck closure	No false helical CTC.
K-B06	Rotating background	Coupled equations satisfied to declared order	Residual scaling plus remainder budget.

C.2 EdGB minimum release

Card	Input	Expected relation	Release threshold
E-B01	alpha=0	Kerr/GR recovery	All adapter deltas vanish at correct order.
E-B02	Schwarzschild scalar source	beta Box vartheta + alpha G_GB = 0	Independent symbolic/numeric residual.
E-B03	Slow-rotation gauge pair	Sector roots map under radial shift	Invariant root correspondence.
E-B04	Near truncation boundary	Sign unresolved when remainder overlaps zero	Required refusal.
E-B05	Characteristic mode count	Complete scalar/tensor physical set	Second derivation or published principal-symbol cross-check.
E-B06	Echo wall model	Echo depends on boundary/reflection	Tagged phenomenological; alternatives listed.

C.3 Promotion checklist

- All normative gates have machine-readable reports.
- No benchmark expected value is sourced solely from the implementation under test.
- Every negative result includes domain coverage and a common temporal certificate.
- Every positive result includes closure, timelike and time-orientation evidence.
- The final release manifest is external, complete and generated after the PDF and source archive.

Appendix D. Change-Control Baseline

Version	Date	Status	Scientific boundary
2.0	31 Aug 2026	Supplied baseline	Metric/topology-centered L0-L19 architecture and 33-model registry.
2.1	1 Sep 2026	This review draft	Adds T0-T3 theory, coupled-equation, principal-symbol and multi-cone specification.
2.1-exec	Future	Planned	Independent quadratic k-essence adapter and benchmark execution.
2.2	Future	Planned	EdGB adapter, characteristic completeness and controlled rotating benchmarks.

D.1 Required evidence for an executed release

An executed release must replace every NOT EXECUTED entry with an immutable run reference or retain the status. It must publish the canonical source, environment lock, independent benchmark provenance, full test output and external final manifest. No editorial change may silently promote a scientific status.

D.2 Document close

CTC-FA-2026-V2.1-EN

The missing theory-aware multi-cone element is specified and integrated. The strongest new claim is architectural readiness; new modified-gravity numerical verdicts await execution.

Part I.B. Verified v2.1 Reference Core

This appendix contains the executable T0-T3 reference core used to validate the new theory/solution separation, topology-aware closure, quadratic characteristic cones, common temporal certificates and refusal-first status logic. It is intentionally smaller than the retained v2.0 kernel. Rotating k-essence, EdGB and generic Horndeski solvers are not represented as implemented.

Execution result

31/31 dependency-free tests passed in the final build environment. The tests cover schema refusals, off-shell and EFT gates, periodic-time and axial witnesses, helical-orbit closure, k-essence constant-background characteristics, hyperbolicity and common temporal certificates.

Implementation-state ledger

Component	State	Evidence	Permitted claim
v2.1 T0-T3 reference core	VERIFIED	31/31 tests; complete listing below	Reference implementation of typed gates
Minkowski and constant-background k-essence cone relations	VERIFIED	Analytic limits and numerical matrix tests	Declared analytic control domain only
Core v2.0 analytic margins	VERIFIED / LIMITED	Direct independent formula recomputation	Named points and sectors only
Full v2.0 embedded Python distribution	RECOVERED / REQUIRES RERUN	Literal extraction: 35 compile, 13 fail	Historical listing; not canonical executable release
Rotating k-essence adapter	SPECIFIED / NOT IMPLEMENTED	Equations, gates and benchmark cards	No numerical causal verdict
Perturbative EdGB adapter	SPECIFIED / NOT IMPLEMENTED	Order-aware contract and V&V; plan	No numerical causal verdict
Generic Horndeski quartic characteristics	SPECIFIED / NOT IMPLEMENTED	Implicit polynomial interface	No all-mode certificate

Recovered v2.0 source audit

The retained baseline prints 118 passing tests. That historical report is reproduced verbatim in Part II. A literal page-layout extraction yields 48 Python files: 35 compile and 13 fail syntax compilation because pagination damaged indentation. The complete failed-file list follows; this is a distribution-integrity finding, not evidence that the original pre-PDF tree failed.

```
examples/ex07_theorems_and_quantum.py:109 unindent does not match any outer indentation level
examples/ex08_full_pipeline.py:41 unexpected indent
examples/ex09_meta_analysis.py:29 unexpected indent
examples/ex10_perturbation_and_minimal.py:112 unindent does not match any outer indentation level
src/ctcfa/causality.py:92 'return' outside function
src/ctcfa/cli.py:173 expected an indented block after 'elif' statement on line 172
src/ctcfa/energy.py:422 'return' outside function
src/ctcfa/loops.py:167 expected an indented block after 'try' statement on line 166
src/ctcfa/model.py:294 unindent does not match any outer indentation level
src/ctcfa/quotient_ctc.py:305 unexpected indent
src/ctcfa/report.py:68 unexpected indent
src/ctcfa/validate.py:151 unindent does not match any outer indentation level
tests/unit/test_status_and_gates.py:72 unindent does not match any outer indentation level
```

Part I.C. v2.1 Test Traceability

Requirement cluster	Tests	Result	Boundary
T0 theory and solution identity	valid spec, missing metric, identity mismatch	PASS	Schema and provenance only
T1 field-equation and EFT gates	off-shell, missing coverage, low scale separation, derivative ratio	PASS	Analytic control reports
T2 characteristic bundle	completeness, symmetry, Lorentzian inertia, degeneracy	PASS	Quadratic effective metrics
T3 positive witnesses	periodic time, axial orbit, any-mode join	PASS	Exact reference topologies
T3 negative certificate	Minkowski dt, all-mode margins, single-valuedness	PASS	Constant cones and declared domain
Helical closure	unwrapped-time negative control; commensurate quotient positive	PASS	Killing orbits with explicit periods
k-essence analytic adapter	beta->0, sound speed, regular and degenerate bundles	PASS	Constant scalar-gradient background
Claim promotion	existential positive only; negative theory promotion refused	PASS	Type-level governance

Scientific boundary

Passing these tests validates the reference gate logic. It does not validate the full retained v2.0 source distribution and does not execute rotating k-essence or EdGB solutions.

Verified source: src/ctcfa_v21_ref/__init__.py

Lines 1-33 of 33 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```
1 | """CTC-FA v2.1 verified reference core.
2 |
3 | This package implements only the T0-T3 data and gate logic described in the
4 | complete v2.1 standard. It deliberately does not claim to implement rotating
5 | k-essence or EdGB background solvers.
6 | """
7 |
8 | from .model import (
9 |     ClaimScope,
10 |     ConeBundle,
11 |     FieldEquationReport,
12 |     ImplementationState,
13 |     ModeKind,
14 |     SolutionSpec,
15 |     TheorySpec,
16 |     Verdict,
17 |     VerdictStatus,
18 | )
19 | from .pipeline import classify
20 |
21 | __all__ = [
22 |     "ClaimScope",
23 |     "ConeBundle",
24 |     "FieldEquationReport",
25 |     "ImplementationState",
26 |     "ModeKind",
27 |     "SolutionSpec",
28 |     "TheorySpec",
29 |     "Verdict",
30 |     "VerdictStatus",
31 |     "classify",
32 | ]
33 |
```

Verified source: src/ctcfa_v21_ref/model.py

Lines 1-92 of 269 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```

1 | """Typed scientific contract for CTC-FA v2.1.
2 |
3 | The core rule is that a theory, a candidate solution, a characteristic mode,
4 | and a causal verdict are distinct objects. Every verdict retains its domain,
5 | mode set, scope, implementation state, and evidence notes.
6 | """
7 |
8 | from __future__ import annotations
9 |
10 | from dataclasses import dataclass, field
11 | from enum import Enum
12 | from typing import Mapping, Sequence
13 |
14 | import numpy as np
15 |
16 |
17 | class ImplementationState(str, Enum):
18 |     VERIFIED = "VERIFIED"
19 |     RECOVERED_REQUIRES_RERUN = "RECOVERED_REQUIRES_RERUN"
20 |     SPECIFIED_NOT_IMPLEMENTED = "SPECIFIED_NOT_IMPLEMENTED"
21 |
22 |
23 | class ClaimScope(str, Enum):
24 |     POINT = "POINT"
25 |     SECTOR = "SECTOR"
26 |     DECLARED_DOMAIN = "DECLARED_DOMAIN"
27 |     GLOBAL = "GLOBAL"
28 |     THEORY_EXISTENCE = "THEORY_EXISTENCE"
29 |
30 |
31 | class VerdictStatus(str, Enum):
32 |     INVALID_SPEC = "INVALID_SPEC"
33 |     OFF_SHELL = "OFF_SHELL"
34 |     OUTSIDE_EFT = "OUTSIDE_EFT"
35 |     NOT_IMPLEMENTED = "NOT_IMPLEMENTED"
36 |     ILL_POSED = "ILL_POSED"
37 |     CTC_PROVED = "CTC_PROVED"
38 |     MODE_CAUSAL_CERTIFIED = "MODE_CAUSAL_CERTIFIED"
39 |     ALL_CONES_CAUSAL_CERTIFIED = "ALL_CONES_CAUSAL_CERTIFIED"
40 |     UNRESOLVED = "UNRESOLVED"
41 |
42 |
43 | class ModeKind(str, Enum):
44 |     METRIC = "METRIC"
45 |     SCALAR = "SCALAR"
46 |     VECTOR = "VECTOR"
47 |     TENSOR = "TENSOR"
48 |     MIXED = "MIXED"
49 |
50 |
51 | @dataclass(frozen=True)
52 | class TheorySpec:
53 |     theory_id: str
54 |     fields: tuple[str, ...]
55 |     coupling_dimensions: Mapping[str, str]
56 |     signature: str = "-+++"
57 |     x_convention: str | None = None
58 |     field_equation_adapter: str | None = None
59 |     principal_symbol_adapter: str | None = None
60 |     implementation: ImplementationState = ImplementationState.SPECIFIED_NOT_IMPLEMENTED
61 |     provenance: tuple[str, ...] = ()
62 |
63 |     def validate(self) -> tuple[bool, tuple[str, ...]]:
64 |         errors: list[str] = []
65 |         if not self.theory_id.strip():
66 |             errors.append("empty theory_id")
67 |         if "metric:g" not in self.fields:
68 |             errors.append("metric:g field is required")
69 |         if self.signature not in {"-+++", "+---"}:
70 |             errors.append("unsupported signature")
71 |         if not self.field_equation_adapter:
72 |             errors.append("missing field-equation adapter")
73 |         if not self.principal_symbol_adapter:
74 |             errors.append("missing principal-symbol adapter")
75 |         if not self.provenance:
76 |             errors.append("missing provenance")
77 |         return not errors, tuple(errors)
78 |
79 |
80 | @dataclass(frozen=True)
81 | class TopologySpec:
82 |     """Only the identifications needed by the verified reference core.
83 |
84 |     ``time_period=None`` means time is unwrapped. ``angular_period`` is the
85 |     physical period of the axial coordinate; it is normally 2*pi.
86 |     """
87 |
88 |     time_period: float | None = None
89 |     angular_period: float | None = 2.0 * np.pi
90 |
91 |     def validate(self) -> tuple[bool, tuple[str, ...]]:
92 |         errors: list[str] = []

```

Verified source: src/ctcfa_v21_ref/model.py

Lines 93-184 of 269 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```

93 |         if self.time_period is not None and self.time_period <= 0:
94 |             errors.append("time_period must be positive")
95 |         if self.angular_period is not None and self.angular_period <= 0:
96 |             errors.append("angular_period must be positive")
97 |         return not errors, tuple(errors)
98 |
99 |
100 | @dataclass(frozen=True)
101 | class SolutionSpec:
102 |     solution_id: str
103 |     theory_id: str
104 |     domain: str
105 |     topology: TopologySpec = field(default_factory=TopologySpec)
106 |     approximation: Mapping[str, int | float | str] = field(default_factory=dict)
107 |     implementation: ImplementationState = ImplementationState.SPECIFIED_NOT_IMPLEMENTED
108 |     provenance: tuple[str, ...] = ()
109 |
110 |     def validate_for(self, theory: TheorySpec) -> tuple[bool, tuple[str, ...]]:
111 |         errors: list[str] = []
112 |         if not self.solution_id.strip():
113 |             errors.append("empty solution_id")
114 |         if self.theory_id != theory.theory_id:
115 |             errors.append("solution/theory identity mismatch")
116 |         if not self.domain.strip():
117 |             errors.append("empty domain")
118 |         ok_topology, top_errors = self.topology.validate()
119 |         if not ok_topology:
120 |             errors.extend(top_errors)
121 |         if not self.provenance:
122 |             errors.append("missing solution provenance")
123 |         return not errors, tuple(errors)
124 |
125 |
126 | @dataclass(frozen=True)
127 | class FieldEquationReport:
128 |     checked_equations: tuple[str, ...]
129 |     max_abs_residual: float
130 |     tolerance: float
131 |     domain_covered: bool
132 |     validity_passed: bool = True
133 |     notes: tuple[str, ...] = ()
134 |
135 |     @property
136 |     def on_shell(self) -> bool:
137 |         return (
138 |             bool(self.checked_equations)
139 |             and self.domain_covered
140 |             and np.isfinite(self.max_abs_residual)
141 |             and 0.0 <= self.max_abs_residual <= self.tolerance
142 |         )
143 |
144 |
145 | @dataclass(frozen=True)
146 | class CharacteristicMode:
147 |     mode_id: str
148 |     kind: ModeKind
149 |     inverse_metric: np.ndarray | None
150 |     physical: bool = True
151 |     hyperbolic: bool = True
152 |     validity_passed: bool = True
153 |     notes: tuple[str, ...] = ()
154 |
155 |     def validate(self, dim: int = 4) -> tuple[bool, tuple[str, ...]]:
156 |         errors: list[str] = []
157 |         if not self.mode_id:
158 |             errors.append("empty mode_id")
159 |         if self.inverse_metric is None:
160 |             errors.append("implicit/non-metric characteristic is not implemented by this reference core")
161 |         else:
162 |             a = np.asarray(self.inverse_metric, dtype=float)
163 |             if a.shape != (dim, dim):
164 |                 errors.append(f"inverse metric must have shape {(dim, dim)}")
165 |             elif not np.allclose(a, a.T, atol=1e-12, rtol=0.0):
166 |                 errors.append("inverse metric must be symmetric")
167 |             elif not np.all(np.isfinite(a)):
168 |                 errors.append("inverse metric contains non-finite values")
169 |         return not errors, tuple(errors)
170 |
171 |     def inertia(self, tol: float = 1e-10) -> tuple[int, int, int]:
172 |         if self.inverse_metric is None:
173 |             raise ValueError("no quadratic inverse metric")
174 |         eig = np.linalg.eigvalsh(np.asarray(self.inverse_metric, dtype=float))
175 |         return (
176 |             int(np.sum(eig < -tol)),
177 |             int(np.sum(np.abs(eig) <= tol)),
178 |             int(np.sum(eig > tol)),
179 |         )
180 |
181 |     def lorentzian_minus_plus(self, tol: float = 1e-10) -> bool:
182 |         return self.inertia(tol) == (1, 0, 3)
183 |
184 |

```


Verified source: src/ctcfa_v21_ref/model.py

Lines 185-269 of 269 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```

185 | @dataclass(frozen=True)
186 | class ConeBundle:
187 |     solution_id: str
188 |     modes: tuple[CharacteristicMode, ...]
189 |     expected_physical_mode_ids: tuple[str, ...]
190 |     domain_covered: bool
191 |     notes: tuple[str, ...] = ()
192 |
193 |     @property
194 |     def physical_modes(self) -> tuple[CharacteristicMode, ...]:
195 |         return tuple(m for m in self.modes if m.physical)
196 |
197 |     @property
198 |     def complete(self) -> bool:
199 |         actual = {m.mode_id for m in self.physical_modes}
200 |         expected = set(self.expected_physical_mode_ids)
201 |         return bool(expected) and actual == expected and self.domain_covered
202 |
203 |     @property
204 |     def hyperbolic(self) -> bool:
205 |         return self.complete and all(
206 |             m.hyperbolic and m.validity_passed and m.lorentzian_minus_plus()
207 |             for m in self.physical_modes
208 |         )
209 |
210 |
211 | @dataclass(frozen=True)
212 | class CTCWitness:
213 |     mode_id: str
214 |     closed: bool
215 |     future_directed: bool
216 |     timelike_margin: float
217 |     scope: ClaimScope
218 |     notes: tuple[str, ...] = ()
219 |
220 |     @property
221 |     def proved(self) -> bool:
222 |         return self.closed and self.future_directed and self.timelike_margin < 0.0
223 |
224 |
225 | @dataclass(frozen=True)
226 | class TemporalCertificate:
227 |     gradient_covector: tuple[float, ...]
228 |     single_valued: bool
229 |     mode_margins: Mapping[str, float]
230 |     epsilon: float
231 |     domain_covered: bool
232 |
233 |     @property
234 |     def proved(self) -> bool:
235 |         return (
236 |             self.single_valued
237 |             and self.domain_covered
238 |             and bool(self.mode_margins)
239 |             and all(v <= -self.epsilon for v in self.mode_margins.values())
240 |         )
241 |
242 |
243 | @dataclass(frozen=True)
244 | class Verdict:
245 |     status: VerdictStatus
246 |     scope: ClaimScope
247 |     theory_id: str
248 |     solution_id: str
249 |     mode_ids: tuple[str, ...] = ()
250 |     notes: tuple[str, ...] = ()
251 |
252 |     def theory_existence_claim(self) -> "Verdict":
253 |         """Only a positive solution witness can widen to theory existence.
254 |
255 |         The widened statement means "this theory admits at least one solution
256 |         sector with a proved CTC". It never means all solutions are acausal.
257 |         """
258 |
259 |         if self.status is not VerdictStatus.CTC_PROVED:
260 |             raise ValueError("negative or unresolved solution verdict cannot become a theory claim")
261 |         return Verdict(
262 |             status=self.status,
263 |             scope=ClaimScope.THEORY_EXISTENCE,
264 |             theory_id=self.theory_id,
265 |             solution_id=self.solution_id,
266 |             mode_ids=self.mode_ids,
267 |             notes=self.notes + ("existential theory claim only",),
268 |         )
269 |

```

Verified source: src/ctcfa_v21_ref/topology.py

Lines 1-92 of 115 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```

1 | """Topology-aware closure and stationary-axisymmetric sector diagnostics."""
2 |
3 | from __future__ import annotations
4 |
5 | from dataclasses import dataclass
6 | from math import isclose, pi
7 |
8 | import numpy as np
9 |
10 | from .model import ClaimScope, CTCWitness, TopologySpec
11 |
12 |
13 | def _integer_multiple(value: float, period: float, tol: float) -> bool:
14 |     ratio = value / period
15 |     return isclose(ratio, round(ratio), abs_tol=tol, rel_tol=0.0)
16 |
17 |
18 | @dataclass(frozen=True)
19 | class KillingOrbitClosure:
20 |     parameter: float
21 |     delta_t: float
22 |     delta_phi: float
23 |     closed: bool
24 |     reason: str
25 |
26 |
27 | def killing_orbit_closure(
28 |     time_component: float,
29 |     angular_component: float,
30 |     topology: TopologySpec,
31 |     windings: int = 1,
32 |     tol: float = 1e-10,
33 | ) -> KillingOrbitClosure:
34 |     """Test the natural axial period of ``a*d_t + b*d_phi``.
35 |
36 |     If ``b`` is nonzero, the candidate parameter period is chosen to produce
37 |     ``windings`` angular revolutions. With unwrapped time, closure additionally
38 |     requires zero time displacement. With periodic time it requires a deck
39 |     transformation by an integer number of time periods.
40 |     """
41 |
42 |     if windings == 0:
43 |         raise ValueError("windings must be nonzero")
44 |     if topology.angular_period is None:
45 |         return KillingOrbitClosure(0.0, 0.0, 0.0, False, "axial coordinate is not periodic")
46 |     if isclose(angular_component, 0.0, abs_tol=tol):
47 |         if topology.time_period is None:
48 |             return KillingOrbitClosure(0.0, 0.0, 0.0, False, "no compact generator")
49 |         parameter = topology.time_period / abs(time_component)
50 |     else:
51 |         parameter = abs(windings) * topology.angular_period / abs(angular_component)
52 |     delta_t = time_component * parameter
53 |     delta_phi = angular_component * parameter
54 |     phi_closed = _integer_multiple(delta_phi, topology.angular_period, tol)
55 |     if topology.time_period is None:
56 |         time_closed = isclose(delta_t, 0.0, abs_tol=tol, rel_tol=0.0)
57 |         reason = "unwrapped time requires zero delta_t"
58 |     else:
59 |         time_closed = _integer_multiple(delta_t, topology.time_period, tol)
60 |         reason = "time displacement must be a deck period"
61 |     return KillingOrbitClosure(parameter, delta_t, delta_phi, phi_closed and time_closed, reason)
62 |
63 |
64 | def killing_block(g_tt: float, g_tphi: float, g_phphi: float) -> dict[str, float | bool]:
65 |     determinant = g_tt * g_phphi - g_tphi * g_tphi
66 |     return {
67 |         "g_phphi": float(g_phphi),
68 |         "determinant": float(determinant),
69 |         "axial_ctc_candidate": bool(g_phphi < 0.0),
70 |         "lorentzian_killing_plane": bool(determinant < 0.0),
71 |     }
72 |
73 |
74 | def axial_witness(
75 |     g_phphi: float,
76 |     topology: TopologySpec,
77 |     mode_id: str = "metric",
78 |     tol: float = 1e-12,
79 | ) -> CTCWitness:
80 |     closed = topology.angular_period is not None
81 |     margin = float(g_phphi)
82 |     return CTCWitness(
83 |         mode_id=mode_id,
84 |         closed=closed,
85 |         future_directed=closed and margin < -tol,
86 |         timelike_margin=margin,
87 |         scope=ClaimScope.SECTOR,
88 |         notes=("closed axial orbit", "orientation convention fixed for compact orbit"),
89 |     )
90 |
91 |
92 | def periodic_time_witness(

```

Verified source: src/ctcfa_v21_ref/topology.py

Lines 93-115 of 115 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```
93 |     g_tt: float,
94 |     topology: TopologySpec,
95 |     mode_id: str = "metric",
96 | ) -> CTCWitness:
97 |     closed = topology.time_period is not None
98 |     return CTCWitness(
99 |         mode_id=mode_id,
100 |         closed=closed,
101 |         future_directed=closed and g_tt < 0.0,
102 |         timelike_margin=float(g_tt),
103 |         scope=ClaimScope.GLOBAL,
104 |         notes=("periodic-time deck orbit",),
105 |     )
106 |
107 |
108 | __all__ = [
109 |     "KillingOrbitClosure",
110 |     "axial_witness",
111 |     "killing_block",
112 |     "killing_orbit_closure",
113 |     "periodic_time_witness",
114 | ]
115 |
```

Verified source: src/ctcfa_v21_ref/cones.py

Lines 1-51 of 51 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```

1 | """Quadratic characteristic cones and common temporal certificates."""
2 |
3 | from __future__ import annotations
4 |
5 | from typing import Iterable
6 |
7 | import numpy as np
8 |
9 | from .model import ConeBundle, TemporalCertificate, TopologySpec
10 |
11 |
12 | def mode_margin(inverse_metric: np.ndarray, gradient_covector: np.ndarray) -> float:
13 |     h = np.asarray(inverse_metric, dtype=float)
14 |     q = np.asarray(gradient_covector, dtype=float)
15 |     return float(q @ h @ q)
16 |
17 |
18 | def temporal_single_valued(gradient_covector: np.ndarray, topology: TopologySpec, tol: float = 1e-12) -> bool:
19 |     q = np.asarray(gradient_covector, dtype=float)
20 |     if q.shape != (4,):
21 |         return False
22 |     if topology.time_period is not None and abs(q[0]) > tol:
23 |         return False
24 |     if topology.angular_period is not None and abs(q[3]) > tol:
25 |         return False
26 |     return True
27 |
28 |
29 | def common_temporal_certificate(
30 |     bundle: ConeBundle,
31 |     gradient_covector: Iterable[float],
32 |     topology: TopologySpec,
33 |     epsilon: float = 1e-10,
34 | ) -> TemporalCertificate:
35 |     q = np.asarray(tuple(gradient_covector), dtype=float)
36 |     margins: dict[str, float] = {}
37 |     for mode in bundle.physical_modes:
38 |         if mode.inverse_metric is None:
39 |             continue
40 |         margins[mode.mode_id] = mode_margin(mode.inverse_metric, q)
41 |     return TemporalCertificate(
42 |         gradient_covector=tuple(float(v) for v in q),
43 |         single_valued=temporal_single_valued(q, topology),
44 |         mode_margins=margins,
45 |         epsilon=epsilon,
46 |         domain_covered=bundle.domain_covered and bundle.complete,
47 |     )
48 |
49 |
50 | __all__ = ["common_temporal_certificate", "mode_margin", "temporal_single_valued"]
51 |

```

Verified source: src/ctcfa_v21_ref/adapters.py

Lines 1-92 of 108 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```

1 | """Verified analytic adapters used by the v2.1 reference tests.
2 |
3 | Only constant-background Minkowski and quadratic k-essence characteristic
4 | relations are implemented. Rotating k-essence and EdGB solvers remain
5 | SPECIFIED_NOT_IMPLEMENTED in the complete standard.
6 | """
7 |
8 | from __future__ import annotations
9 |
10 | from dataclasses import dataclass
11 |
12 | import numpy as np
13 |
14 | from .model import CharacteristicMode, ConeBundle, ModeKind
15 |
16 |
17 | MINKOWSKI_INVERSE = np.diag([-1.0, 1.0, 1.0, 1.0])
18 |
19 |
20 | @dataclass(frozen=True)
21 | class EFTValidityReport:
22 |     background_to_cutoff_ratio: float
23 |     derivative_expansion_ratio: float
24 |     minimum_background_ratio: float = 10.0
25 |     maximum_derivative_ratio: float = 0.1
26 |
27 |     @property
28 |     def passed(self) -> bool:
29 |         return (
30 |             np.isfinite(self.background_to_cutoff_ratio)
31 |             and np.isfinite(self.derivative_expansion_ratio)
32 |             and self.background_to_cutoff_ratio >= self.minimum_background_ratio
33 |             and 0.0 <= self.derivative_expansion_ratio <= self.maximum_derivative_ratio
34 |         )
35 |
36 |
37 | def metric_mode(inverse_metric: np.ndarray = MINKOWSKI_INVERSE) -> CharacteristicMode:
38 |     return CharacteristicMode(
39 |         mode_id="metric",
40 |         kind=ModeKind.METRIC,
41 |         inverse_metric=np.asarray(inverse_metric, dtype=float),
42 |         physical=True,
43 |         hyperbolic=True,
44 |         validity_passed=True,
45 |         notes=("quadratic metric null cone."),
46 |     )
47 |
48 |
49 | def kessence_characteristic(
50 |     alpha: float,
51 |     beta: float,
52 |     dphi_covector: np.ndarray,
53 |     inverse_metric: np.ndarray = MINKOWSKI_INVERSE,
54 | ) -> tuple[np.ndarray, float, float, float]:
55 |     """Return  $Z^{(\mu\nu)}$ ,  $X$ ,  $P_X$  and  $c_s^2$  for  $P=\alpha X+\beta X^2$ .
56 |
57 |     The convention is  $X=g^{(\mu\nu)}\phi_{,\mu}\phi_{,\nu}/2$  and signature  $-+++$ .
58 |      $Z^{(\mu\nu)}=P_X g^{(\mu\nu)}-P_{XX} \text{nabla}^{\mu}\phi \text{nabla}^{\nu}\phi$ .
59 |     """
60 |
61 |     g_inv = np.asarray(inverse_metric, dtype=float)
62 |     q = np.asarray(dphi_covector, dtype=float)
63 |     if g_inv.shape != (4, 4) or q.shape != (4,):
64 |         raise ValueError("expected a 4D inverse metric and scalar covector")
65 |     x = float(-0.5 * q @ g_inv @ q)
66 |     p_x = float(alpha + 2.0 * beta * x)
67 |     p_xx = float(2.0 * beta)
68 |     grad_up = g_inv @ q
69 |     z = p_x * g_inv - p_xx * np.outer(grad_up, grad_up)
70 |     denominator = p_x + 2.0 * x * p_xx
71 |     sound_speed_sq = float(p_x / denominator) if denominator != 0.0 else float("nan")
72 |     return z, x, p_x, sound_speed_sq
73 |
74 |
75 | def kessence_constant_bundle(
76 |     solution_id: str,
77 |     alpha: float,
78 |     beta: float,
79 |     dphi_covector: np.ndarray,
80 |     eft: EFTValidityReport,
81 | ) -> ConeBundle:
82 |     z, x, p_x, c_s2 = kessence_characteristic(alpha, beta, dphi_covector)
83 |     scalar = CharacteristicMode(
84 |         mode_id="scalar",
85 |         kind=ModeKind.SCALAR,
86 |         inverse_metric=z,
87 |         physical=True,
88 |         hyperbolic=bool(np.isfinite(c_s2) and c_s2 > 0.0 and abs(np.linalg.det(z)) > 1e-12),
89 |         validity_passed=eft.passed,
90 |         notes=(f" $X={x:.12g}$ ", f" $P_X={p_x:.12g}$ ", f" $c_s^2={c_s2:.12g}$ "),
91 |     )
92 |     return ConeBundle(

```

Verified source: src/ctcfa_v21_ref/adapters.py

Lines 93-108 of 108 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```
93 |         solution_id=solution_id,
94 |         modes=(metric_mode(), scalar),
95 |         expected_physical_mode_ids=("metric", "scalar"),
96 |         domain_covered=True,
97 |         notes=("constant-background analytic bundle",),
98 |     )
99 |
100 |
101 | __all__ = [
102 |     "EFTValidityReport",
103 |     "MINKOWSKI_INVERSE",
104 |     "kessence_characteristic",
105 |     "kessence_constant_bundle",
106 |     "metric_mode",
107 | ]
108 |
```

Verified source: src/ctcfa_v21_ref/pipeline.py

Lines 1-92 of 109 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```

1 | """T0-T3 classification pipeline with refusal-first promotion logic."""
2 |
3 | from __future__ import annotations
4 |
5 | from collections.abc import Iterable
6 |
7 | from .cones import common_temporal_certificate
8 | from .model import (
9 |     CTCWitness,
10 |     ClaimScope,
11 |     ConeBundle,
12 |     FieldEquationReport,
13 |     SolutionSpec,
14 |     TheorySpec,
15 |     Verdict,
16 |     VerdictStatus,
17 | )
18 |
19 |
20 | def classify(
21 |     theory: TheorySpec,
22 |     solution: SolutionSpec,
23 |     field_report: FieldEquationReport,
24 |     cone_bundle: ConeBundle | None,
25 |     witnesses: Iterable[CTCWitness] = (),
26 |     temporal_gradient: tuple[float, float, float, float] | None = None,
27 | ) -> Verdict:
28 |     """Classify one solution without making an unearned theory-wide claim."""
29 |
30 |     t_ok, t_errors = theory.validate()
31 |     s_ok, s_errors = solution.validate_for(theory)
32 |     if not t_ok or not s_ok:
33 |         return Verdict(
34 |             VerdictStatus.INVALID_SPEC,
35 |             ClaimScope.POINT,
36 |             theory.theory_id,
37 |             solution.solution_id,
38 |             notes=t_errors + s_errors,
39 |         )
40 |     if not field_report.on_shell:
41 |         return Verdict(
42 |             VerdictStatus.OFF_SHELL,
43 |             ClaimScope.DECLARED_DOMAIN,
44 |             theory.theory_id,
45 |             solution.solution_id,
46 |             notes=field_report.notes + ("field-equation gate failed",),
47 |         )
48 |     if not field_report.validity_passed:
49 |         return Verdict(
50 |             VerdictStatus.OUTSIDE_EFT,
51 |             ClaimScope.DECLARED_DOMAIN,
52 |             theory.theory_id,
53 |             solution.solution_id,
54 |             notes=field_report.notes + ("validity gate failed",),
55 |         )
56 |     if cone_bundle is None or not cone_bundle.complete:
57 |         return Verdict(
58 |             VerdictStatus.NOT_IMPLEMENTED,
59 |             ClaimScope.DECLARED_DOMAIN,
60 |             theory.theory_id,
61 |             solution.solution_id,
62 |             notes=("physical characteristic bundle missing or incomplete",),
63 |         )
64 |     if not cone_bundle.hyperbolic:
65 |         return Verdict(
66 |             VerdictStatus.ILL_POSED,
67 |             ClaimScope.DECLARED_DOMAIN,
68 |             theory.theory_id,
69 |             solution.solution_id,
70 |             mode_ids=tuple(m.mode_id for m in cone_bundle.physical_modes),
71 |             notes=("hyperbolicity/common signature gate failed",),
72 |         )
73 |     proved = tuple(w for w in witnesses if w.proved)
74 |     if proved:
75 |         return Verdict(
76 |             VerdictStatus.CTC_PROVED,
77 |             min((w.scope for w in proved), key=lambda s: list(ClaimScope).index(s)),
78 |             theory.theory_id,
79 |             solution.solution_id,
80 |             mode_ids=tuple(sorted({w.mode_id for w in proved})),
81 |             notes=tuple(note for w in proved for note in w.notes),
82 |         )
83 |     if temporal_gradient is not None:
84 |         cert = common_temporal_certificate(
85 |             cone_bundle,
86 |             temporal_gradient,
87 |             solution.topology,
88 |         )
89 |         if cert.proved:
90 |             return Verdict(
91 |                 VerdictStatus.ALL_CONES_CAUSAL_CERTIFIED,
92 |                 ClaimScope.DECLARED_DOMAIN,

```

Verified source: src/ctcfa_v21_ref/pipeline.py

Lines 93-109 of 109 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```
93 |         theory.theory_id,  
94 |         solution.solution_id,  
95 |         mode_ids=tuple(sorted(cert.mode_margins)),  
96 |         notes=tuple(f"{k}: {v:.12g}" for k, v in sorted(cert.mode_margins.items())),  
97 |     )  
98 |     return Verdict(  
99 |         VerdictStatus.UNRESOLVED,  
100 |         ClaimScope.DECLARED_DOMAIN,  
101 |         theory.theory_id,  
102 |         solution.solution_id,  
103 |         mode_ids=tuple(m.mode_id for m in cone_bundle.physical_modes),  
104 |         notes=("no positive witness and no common temporal certificate",),  
105 |     )  
106 |  
107 |  
108 | __all__ = ["classify"]  
109 |
```


Verified source: tests/test_reference_core.py

Lines 1-92 of 271 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```

1 | from __future__ import annotations
2 |
3 | import math
4 | from contextlib import contextmanager
5 |
6 | import numpy as np
7 |
8 | from ctcfa_v2l_ref.adapters import (
9 |     EFTValidityReport,
10 |     MINKOWSKI_INVERSE,
11 |     kessence_characteristic,
12 |     kessence_constant_bundle,
13 |     metric_mode,
14 | )
15 | from ctcfa_v2l_ref.cones import common_temporal_certificate, temporal_single_valued
16 | from ctcfa_v2l_ref.model import (
17 |     CTOWitness,
18 |     CharacteristicMode,
19 |     ClaimScope,
20 |     ConeBundle,
21 |     FieldEquationReport,
22 |     ImplementationState,
23 |     ModeKind,
24 |     SolutionSpec,
25 |     TheorySpec,
26 |     TopologySpec,
27 |     VerdictStatus,
28 | )
29 | from ctcfa_v2l_ref.pipeline import classify
30 | from ctcfa_v2l_ref.topology import (
31 |     axial_witness,
32 |     killing_block,
33 |     killing_orbit_closure,
34 |     periodic_time_witness,
35 | )
36 |
37 |
38 | class _Approx:
39 |     def __init__(self, expected: float, abs_tol: float = 1e-12, rel_tol: float = 1e-9):
40 |         self.expected = float(expected)
41 |         self.abs_tol = abs_tol
42 |         self.rel_tol = rel_tol
43 |
44 |     def __eq__(self, actual: object) -> bool:
45 |         try:
46 |             return math.isclose(float(actual), self.expected, abs_tol=self.abs_tol, rel_tol=self.rel_tol)
47 |         except (TypeError, ValueError):
48 |             return False
49 |
50 |
51 | def approx(expected: float) -> _Approx:
52 |     return _Approx(expected)
53 |
54 |
55 | @contextmanager
56 | def raises(exc_type):
57 |     try:
58 |         yield
59 |     except exc_type:
60 |         return
61 |     raise AssertionError(f"expected {exc_type.__name__}")
62 |
63 |
64 | def theory() -> TheorySpec:
65 |     return TheorySpec(
66 |         theory_id="gr.v1",
67 |         fields={"metric:g",},
68 |         coupling_dimensions={},
69 |         field_equation_adapter="vacuum_gr",
70 |         principal_symbol_adapter="metric_cone",
71 |         implementation=ImplementationState.VERIFIED,
72 |         provenance=("Einstein 1915",),
73 |     )
74 |
75 |
76 | def solution(topology: TopologySpec | None = None) -> SolutionSpec:
77 |     return SolutionSpec(
78 |         solution_id="minkowski.control",
79 |         theory_id="gr.v1",
80 |         domain="all R4",
81 |         topology=topology or TopologySpec(),
82 |         implementation=ImplementationState.VERIFIED,
83 |         provenance=("analytic control",),
84 |     )
85 |
86 |
87 | def field_report(residual: float = 0.0, validity: bool = True) -> FieldEquationReport:
88 |     return FieldEquationReport(
89 |         checked_equations=("G_mu_nu=0",),
90 |         max_abs_residual=residual,
91 |         tolerance=1e-12,
92 |         domain_covered=True,

```

Verified source: tests/test_reference_core.py

Lines 93-184 of 271 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```

93 |         validity_passed=validity,
94 |     )
95 |
96 |
97 | def metric_bundle(solution_id: str = "minkowski.control") -> ConeBundle:
98 |     return ConeBundle(solution_id, (metric_mode(),), ("metric",), True)
99 |
100 |
101 | def test_valid_theory_and_solution():
102 |     assert theory().validate()[0]
103 |     assert solution().validate_for(theory())[0]
104 |
105 |
106 | def test_theory_requires_metric_field():
107 |     bad = TheorySpec("bad", ("scalar:phi",), {}, field_equation_adapter="x", principal_symbol_adapter="y", provenance=("p",))
108 |     ok, errors = bad.validate()
109 |     assert not ok and "metric:g field is required" in errors
110 |
111 |
112 | def test_solution_theory_mismatch_is_invalid():
113 |     bad = SolutionSpec("s", "other", "D", provenance=("p",))
114 |     verdict = classify(theory(), bad, field_report(), metric_bundle())
115 |     assert verdict.status is VerdictStatus.INVALID_SPEC
116 |
117 |
118 | def test_off_shell_precedes_causality():
119 |     verdict = classify(theory(), solution(), field_report(1e-4), metric_bundle(), temporal_gradient=(1, 0, 0, 0))
120 |     assert verdict.status is VerdictStatus.OFF_SHELL
121 |
122 |
123 | def test_outside_eft_precedes_causality():
124 |     verdict = classify(theory(), solution(), field_report(validity=False), metric_bundle(), temporal_gradient=(1, 0, 0, 0))
125 |     assert verdict.status is VerdictStatus.OUTSIDE_EFT
126 |
127 |
128 | def test_incomplete_bundle_is_not_implemented():
129 |     incomplete = ConeBundle("minkowski.control", (metric_mode(),), ("metric", "scalar"), True)
130 |     verdict = classify(theory(), solution(), field_report(), incomplete)
131 |     assert verdict.status is VerdictStatus.NOT_IMPLEMENTED
132 |
133 |
134 | def test_minkowski_dt_is_temporal():
135 |     cert = common_temporal_certificate(metric_bundle(), (1, 0, 0, 0), TopologySpec())
136 |     assert cert.proved and cert.mode_margins["metric"] == approx(-1.0)
137 |
138 |
139 | def test_periodic_time_makes_t_non_single_valued():
140 |     topology = TopologySpec(time_period=2 * math.pi)
141 |     assert not temporal_single_valued(np.array([1.0, 0.0, 0.0, 0.0]), topology)
142 |
143 |
144 | def test_periodic_angle_makes_phi_non_single_valued():
145 |     assert not temporal_single_valued(np.array([0.0, 0.0, 0.0, 1.0]), TopologySpec())
146 |
147 |
148 | def test_periodic_time_has_exact_ctc_witness():
149 |     w = periodic_time_witness(-1.0, TopologySpec(time_period=2 * math.pi))
150 |     assert w.proved and w.scope is ClaimScope.GLOBAL
151 |
152 |
153 | def test_unwrapped_time_has_no_periodic_time_witness():
154 |     assert not periodic_time_witness(-1.0, TopologySpec()).proved
155 |
156 |
157 | def test_axial_negative_norm_is_ctc_witness():
158 |     assert axial_witness(-0.25, TopologySpec()).proved
159 |
160 |
161 | def test_axial_positive_norm_is_not_ctc_witness():
162 |     assert not axial_witness(+0.25, TopologySpec()).proved
163 |
164 |
165 | def test_killing_block_determinant_is_sector_diagnostic():
166 |     row = killing_block(-1.0, 0.2, 4.0)
167 |     assert row["lorentzian_killing_plane"] and not row["axial_ctc_candidate"]
168 |
169 |
170 | def test_helical_orbit_is_open_with_unwrapped_time():
171 |     c = killing_orbit_closure(1.0, 2.0, TopologySpec())
172 |     assert not c.closed and c.delta_t == approx(math.pi)
173 |
174 |
175 | def test_helical_orbit_closes_with_commensurate_time_quotient():
176 |     c = killing_orbit_closure(1.0, 2.0, TopologySpec(time_period=math.pi))
177 |     assert c.closed
178 |
179 |
180 | def test_pure_axial_orbit_closes():
181 |     assert killing_orbit_closure(0.0, 1.0, TopologySpec()).closed
182 |
183 |
184 | def test_kessence_beta_zero_is_conformal_metric():

```

Verified source: tests/test_reference_core.py

Lines 185-271 of 271 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```

185 | z, x, px, cs2 = kessence_characteristic(2.0, 0.0, np.array([1.0, 0.0, 0.0, 0.0]))
186 | assert x == approx(0.5)
187 | assert px == approx(2.0)
188 | assert cs2 == approx(1.0)
189 | assert np.allclose(z, 2.0 * MINKOWSKI_INVERSE)
190 |
191 |
192 | def test_kessence_sound_speed_matches_formula():
193 |     _, x, px, cs2 = kessence_characteristic(1.0, 0.2, np.array([1.0, 0.0, 0.0, 0.0]))
194 |     expected = px / (px + 4.0 * 0.2 * x)
195 |     assert cs2 == approx(expected)
196 |
197 |
198 | def test_kessence_regular_bundle_is_hyperbolic():
199 |     eft = EFTValidityReport(100.0, 0.01)
200 |     bundle = kessence_constant_bundle("k.control", 1.0, 0.1, np.array([0.5, 0.0, 0.0, 0.0]), eft)
201 |     assert bundle.complete and bundle.hyperbolic
202 |
203 |
204 | def test_eft_gate_rejects_low_scale_separation():
205 |     assert not EFTValidityReport(2.0, 0.01).passed
206 |
207 |
208 | def test_eft_gate_rejects_large_derivative_ratio():
209 |     assert not EFTValidityReport(100.0, 0.5).passed
210 |
211 |
212 | def test_degenerate_scalar_mode_is_ill_posed():
213 |     zero = CharacteristicMode("scalar", ModeKind.SCALAR, np.zeros((4, 4)), hyperbolic=False)
214 |     bundle = ConeBundle("minkowski.control", (metric_mode(), zero), ("metric", "scalar"), True)
215 |     verdict = classify(theory(), solution(), field_report(), bundle)
216 |     assert verdict.status is VerdictStatus.ILL_POSED
217 |
218 |
219 | def test_any_mode_witness_proves_solution_ctc():
220 |     w = CTCWitness("metric", True, True, -0.5, ClaimScope.SECTOR, ("analytic",))
221 |     verdict = classify(theory(), solution(), field_report(), metric_bundle(), witnesses=(w,))
222 |     assert verdict.status is VerdictStatus.CTC_PROVED
223 |
224 |
225 | def test_positive_solution_verdict_can_widen_only_existentially():
226 |     w = CTCWitness("metric", True, True, -0.5, ClaimScope.SECTOR)
227 |     verdict = classify(theory(), solution(), field_report(), metric_bundle(), witnesses=(w,))
228 |     wide = verdict.theory_existence_claim()
229 |     assert wide.scope is ClaimScope.THEORY_EXISTENCE
230 |     assert "existential" in wide.notes[-1]
231 |
232 |
233 | def test_negative_solution_verdict_cannot_become_theory_claim():
234 |     verdict = classify(theory(), solution(), field_report(), metric_bundle(), temporal_gradient=(1, 0, 0, 0))
235 |     assert verdict.status is VerdictStatus.ALL_CONES_CAUSAL_CERTIFIED
236 |     with raises(ValueError):
237 |         verdict.theory_existence_claim()
238 |
239 |
240 | def test_common_temporal_certificate_requires_all_modes():
241 |     outward = CharacteristicMode("scalar", ModeKind.SCALAR, np.diag([1.0, -1.0, 1.0, 1.0]))
242 |     bundle = ConeBundle("minkowski.control", (metric_mode(), outward), ("metric", "scalar"), True)
243 |     verdict = classify(theory(), solution(), field_report(), bundle, temporal_gradient=(1, 0, 0, 0))
244 |     # Both modes are individually Lorentzian, but dt is not temporal for the
245 |     # second mode. One failed candidate gradient cannot prove that the dual
246 |     # cone intersection is empty, so the refusal state is UNRESOLVED.
247 |     assert verdict.status is VerdictStatus.UNRESOLVED
248 |
249 |
250 | def test_no_witness_no_certificate_is_unresolved():
251 |     verdict = classify(theory(), solution(), field_report(), metric_bundle())
252 |     assert verdict.status is VerdictStatus.UNRESOLVED
253 |
254 |
255 | def test_temporal_certificate_reports_every_mode():
256 |     eft = EFTValidityReport(100.0, 0.01)
257 |     bundle = kessence_constant_bundle("k.control", 1.0, 0.1, np.array([0.5, 0.0, 0.0, 0.0]), eft)
258 |     cert = common_temporal_certificate(bundle, (1, 0, 0, 0), TopologySpec())
259 |     assert cert.proved and set(cert.mode_margins) == {"metric", "scalar"}
260 |
261 |
262 | def test_mode_validation_rejects_asymmetry():
263 |     a = np.eye(4)
264 |     a[0, 1] = 2.0
265 |     mode = CharacteristicMode("bad", ModeKind.MIXED, a)
266 |     assert not mode.validate()[0]
267 |
268 |
269 | def test_field_report_requires_domain_coverage():
270 |     report = FieldEquationReport(("E=0",), 0.0, 1e-12, False)
271 |     assert not report.on_shell

```

Verified source: tests/run_tests.py

Lines 1-37 of 37 | State: VERIFIED IN REFERENCE CORE | Copy source from the canonical file, not from page extraction.

```
1 | """Dependency-free test runner for the verified CTC-FA v2.1 reference core."""
2 |
3 | from __future__ import annotations
4 |
5 | import inspect
6 | import sys
7 | import time
8 | import traceback
9 |
10 | import test_reference_core as suite
11 |
12 |
13 | def main() -> int:
14 |     tests = [
15 |         (name, fn)
16 |         for name, fn in inspect.getmembers(suite, inspect.isfunction)
17 |         if name.startswith("test_")
18 |     ]
19 |     start = time.perf_counter()
20 |     failures: list[tuple[str, str]] = []
21 |     for name, fn in tests:
22 |         try:
23 |             fn()
24 |             print(f"PASS {name}")
25 |         except Exception:
26 |             failures.append((name, traceback.format_exc()))
27 |             print(f"FAIL {name}")
28 |     elapsed = time.perf_counter() - start
29 |     print(f"SUMMARY total={len(tests)} passed={len(tests)-len(failures)} failed={len(failures)} elapsed={elapsed:.6f}s")
30 |     for name, detail in failures:
31 |         print(f"--- {name} ---")
32 |         print(detail)
33 |     return 1 if failures else 0
34 |
35 |
36 | if __name__ == "__main__":
37 |     sys.exit(main())
```

Part I.D. Verbatim Reference-Core Test Output

The following output was produced during the same build that generated this complete PDF. The final external file digest is reported outside the PDF after assembly.

```
PASS test_any_mode_witness_proves_solution_ctc
PASS test_axial_negative_norm_is_ctc_witness
PASS test_axial_positive_norm_is_not_ctc_witness
PASS test_common_temporal_certificate_requires_all_modes
PASS test_degenerate_scalar_mode_is_ill_posed
PASS test_eft_gate_rejects_large_derivative_ratio
PASS test_eft_gate_rejects_low_scale_separation
PASS test_field_report_requires_domain_coverage
PASS test_helical_orbit_closes_with_commensurate_time_quotient
PASS test_helical_orbit_is_open_with_unwrapped_time
PASS test_incomplete_bundle_is_not_implemented
PASS test_kessence_beta_zero_is_conformal_metric
PASS test_kessence_regular_bundle_is_hyperbolic
PASS test_kessence_sound_speed_matches_formula
PASS test_killing_block_determinant_is_sector_diagnostic
PASS test_minkowski_dt_is_temporal
PASS test_mode_validation_rejects_asymmetry
PASS test_negative_solution_verdict_cannot_become_theory_claim
PASS test_no_witness_no_certificate_is_unresolved
PASS test_off_shell_precedes_causality
PASS test_outside_eft_precedes_causality
PASS test_periodic_angle_makes_phi_non_single_valued
PASS test_periodic_time_has_exact_ctc_witness
PASS test_periodic_time_makes_t_non_single_valued
PASS test_positive_solution_verdict_can_widen_only_existentially
PASS test_pure_axial_orbit_closes
PASS test_solution_theory_mismatch_is_invalid
PASS test_temporal_certificate_reports_every_mode
PASS test_theory_requires_metric_field
PASS test_unwrapped_time_has_no_periodic_time_witness
PASS test_valid_theory_and_solution
SUMMARY total=31 passed=31 failed=0 elapsed=0.002126s
```

Reference-core result

SUMMARY total=31 passed=31 failed=0. No result from the retained v2.0 test report is silently re-labelled as re-executed.

PART II

Complete v2.0 Technical Baseline

Retained verbatim for full registry, algorithms, equations, code listing, historical test report, RTM, V&V and provenance. Part I v2.1 controls govern conflicts and claim scope.

Baseline begins on consolidated page 47; complete document has 264 pages.

0. Authorship, Provenance, Scope and Claim Boundary

CTC-FA v2.0 is a system-level research architecture that integrates established results from general relativity, causal theory, exact solutions, numerical relativity, semiclassical gravity and scientific software into one falsification-oriented computational workflow. The original contribution claimed here is the architecture and the integration logic — specifically the evidence algebra, the executable topology layer, the reduced causal margin and the gate structure — not ownership of Einstein's equations, the historical metrics, the chronology-protection results, or any other cited scientific result.

This release supersedes CTC-FA-2026-V1.0-EN-0831-VT of the same date. v1.0 remains available as a historical record; Section 2 and Appendix B list every defect corrected, with the evidence for each and the regression test that now prevents its return. Historical releases must not be withdrawn, so that any result computed against them can still be reproduced.

0.1 Provenance record

Item	Value
System architect	Vadym Tsinderhoz
Architecture	CTC-FA v2.0 — Closed Timelike Curve Falsification Architecture
Document ID	CTC-FA-2026-V2.0-EN
Document date	31 August 2026
Supersedes	CTC-FA-2026-V1.0-EN-0831-VT
Primary units	geometric units $G = c = 1$ unless a section states otherwise
Signature convention	(−,+,+,+) throughout; verified numerically at every evaluated point by Sylvester sign counts
Registered models	33
Publication status	research architecture / technical standard; not peer reviewed in this form

0.2 What the architecture does not claim

- It does not claim that a physically realisable time machine has been discovered, or that any registered geometry can be built.
- It does not claim that every mathematically defined CTC is dynamically reachable, survivable, or stable.
- It does not claim that negative energy is a freely scalable engineering resource.
- It does not treat a Cauchy horizon as synonymous with a chronology horizon or with a CTC.
- It does not treat failure of a numerical search as evidence of absence: UNRESOLVED is an absorbing state.
- It does not claim a universal theorem that quantum backreaction destroys every CTC, and it cannot emit the label CHRONOLOGY_PROTECTION_PROVED at all.
- It does not claim priority over earlier causal-classification work, including Rieger's Misner-type programme whose public arXiv record begins 1 January 2025 [R17].
- It does not claim that its cross-family observations are theorems. The strongest label the meta-analysis can emit is CANDIDATE_PRECURSOR.

New in v2.0: the claim boundary is enforced, not merely stated

In v1.0 every restriction above was a paragraph. In v2.0 each is a type or a gate. A sector-clear result cannot be assigned to the NO_CTC_CERTIFIED constructor; an UNRESOLVED verdict cannot be promoted; a reduced-model predicate carries ClaimScope.REDUCED_MODEL and cannot be printed as global; and Verdict.render raises OverclaimError when asked for a claim level the gate record does not support. The refusals are exercised by 20 negative-control tests.

1. Executive Summary

The purpose of CTC-FA is to turn the fragmented literature on closed timelike curves into a reproducible computational research programme. The central scientific question is not whether Einstein's equations admit isolated causality-violating solutions — they do — but whether there is a compact set of invariant, dynamically meaningful conditions that controls the transition between chronal and chronology-violating regimes across qualitatively different spacetime families.

chronal solution → marginal causal boundary → CTC region

CTC solution → minimal on-shell change → restored stable causality

CTC-FA therefore treats causality violation as a phase-classification and falsification problem. Each registered model is evaluated through independent layers: field-equation validity, topology and quotient structure, explicit CTC witnesses, negative causal certificates, horizon classification, energy conditions, parameter continuation, perturbative robustness, initial-data constructibility, quantum-state admissibility, semiclassical backreaction and accessibility.

1.1 What changed in v2.0

v1.0 described a great deal that it did not implement, and it contained concrete defects. The audit that produced this release read every formula, every registry entry and every line of the 951-line v1.0 kernel, and recorded 28 defects: 8 demonstrable errors and 20 stated requirements with no implementation. Three of the errors are worth naming here because they are representative.

- **D-01.** The v1.0 registry table printed the Kerr frame-dragging component with the wrong sign, contradicting the v1.0 kernel on the same page.
- **D-02.** The v1.0 kernel stored the van Stockum metric asymmetrically, violating its own REQ-002. Because the compact-orbit detector reads only one component, the published margin was still correct and the defect survived the self-test — but every curvature or geodesic computation on that model would have been wrong.
- **D-11.** The v1.0 kernel obtained the Gödel-type causal boundary by root-finding the hard-coded expression $m^2 - 4\omega^2$ and published $m^* = 1.9999999999999987$ as a result. This is exactly failure mode FM-016, circular validation, committed by the document that names it.

The rest are gaps: interval certification named as the requirement for a formal PASS in three places and implemented nowhere; the Kretschmann scalar left as a placeholder that raised `NotImplementedError`, so failure mode FM-001 had no computational mitigation; the theorem hypothesis checklist that FM-010, FM-011 and FM-012 all name as their mitigation; the seven integrity controls of section 30, of which none computed a hash; and above all the topology layer, whose absence the v1.0 register itself recorded as FM-007.

1.2 The decisive structural addition

The single most important change is that **topology is now an executable object**. A quotient carries its deck generators as explicit coordinate maps with verified isometry certificates, free-action checks and time-orientation checks, and a curve closes on the quotient when $\gamma(1) = g \cdot \gamma(0)$ for some word g in Γ . The consequence is that v2.0 can compute a distinction v1.0 could not express at all: the registry ships two pairs of models whose local metric tensors are *identical* and whose global causal verdicts are *opposite*.

The L3 gate, made executable

ads4_universal_cover and ads4_periodic carry the same metric tensor. The first is certified NO_CTC_CERTIFIED at GLOBAL scope by an unconditional symbolic proof that $\tau = t$ has a strictly timelike gradient. The second is CTC_PROVED, because the Killing orbit of ∂_t is closed and timelike everywhere, and because $\tau = t$ is *not single valued* on the quotient — a check v2.0 performs before it will even attempt a certificate. minkowski and minkowski_time_periodic make the same point with no curvature at all.

1.3 Evidence that the rewrite preserved the physics

Every numeric margin the v1.0 self-test published is reproduced *exactly* by the rewritten engines: $-1.670000e+00$ (Kerr), $-1.772400e+00$ (Kerr-Newman), $-1.736478e+01$ and $+1.315412e+01$ (Gödel-type), $-1.000000e+00$ (Misner), $-1.200000e+01$ (van Stockum), $-6.923595e-01$ (Taub-NUT), $-1.536000e-01$ (spinning string), $+4.000000e+00$ (Morris-Thorne), $-1.000000e+00$ (Ori 2005 and Ori 2007 core), $+1.000000e-02$ (Krasnikov), $+2.500000e+01$ (Schwarzschild and Reissner-Nordström). This is the strongest available evidence that the v1.0 numerics were sound wherever they were actually computed, and that the corrections changed only what was wrong.

1.4 Core scientific outputs

- A metric and construction registry with explicit coordinates, topology, identifications, parameter domain, matter content, theory and provenance, machine-readable and schema-validated.
- A positive CTC proof path with two independent detectors: symmetry-orbit and deck-orbit, the second exact on flat coverings.
- A negative proof path that certifies stable causality by a temporal function proved either symbolically or by interval branch-and-bound on a stated compact box.
- A causal-boundary solver that derives the transition from the metric and compares with the closed form only afterwards.
- A minimal on-shell perturbation problem with a cryptographically sealed cost metric.
- A model-aware energy-condition layer that is exact rather than sampled, cross-checked between two independent routes.
- A theorem applicability validator that checks hypotheses before licensing any conclusion.
- A cross-metric meta-analysis with held-out validation and a leakage check, designed to discover recurring invariant precursors rather than to catalogue famous geometries.

CTC-FA v2.0 layer stack and what changed at each level

L0	Registry & state vector	<i>rewritten</i>
L1	Symbolic tensor engine	<i>extended</i>
L2	Field-equation validator	<i>extended</i>
L3	Topology & quotient engine	<i>NEW</i>
L4	Killing / symmetry engine	<i>NEW gate</i>
L5	Compact-orbit CTC detector	<i>7 gates added</i>
L5b	Deck-orbit detector (flat quotients)	<i>NEW</i>
L6	Global closed-curve search	<i>search/validate split</i>
L7	Causality certificate	<i>interval-certified</i>
L8	Horizon taxonomy	<i>4 objects separated</i>
L9	Energy conditions	<i>exact, 2 routes</i>
L10	Wormhole layer	<i>scoped</i>
L11	Perturbation engine	<i>on-shell status</i>
L12	ADM / initial data	<i>York solver added</i>
L13	Quantum-state layer	<i>spec required</i>
L14	Semiclassical backreaction	<i>KRW gate</i>
L15	Theorem applicability	<i>NEW checkers</i>
L16	Causal continuation	<i>non-circular</i>
L17	Minimal-perturbation optimiser	<i>sealed cost</i>
L18	Accessibility & geodesics	<i>4 claim levels</i>
L19	Cross-family meta-analysis	<i>held-out</i>

Figure 1. The L0-L19 layer stack. Entries outlined in red are new or rewritten in v2.0; the status column records what changed.

2. Corrections Register: v1.0 → v2.0

This section is the audit result. Each entry records the defect, where it was in v1.0, the evidence that it is a defect, the fix, and the regression test that now prevents its return. Nothing here is a matter of taste: every entry is either a demonstrable error, a stated requirement with no implementation, or a self-declared failure mode that the v1.0 code committed.

Summary: 28 defects (8 errors, 20 unimplemented requirements), plus 3 presentation defects

A corrections register that lists only failures would give a false picture of the prior work, so section 2.3 records what v1.0 got right and v2.0 preserved unchanged.

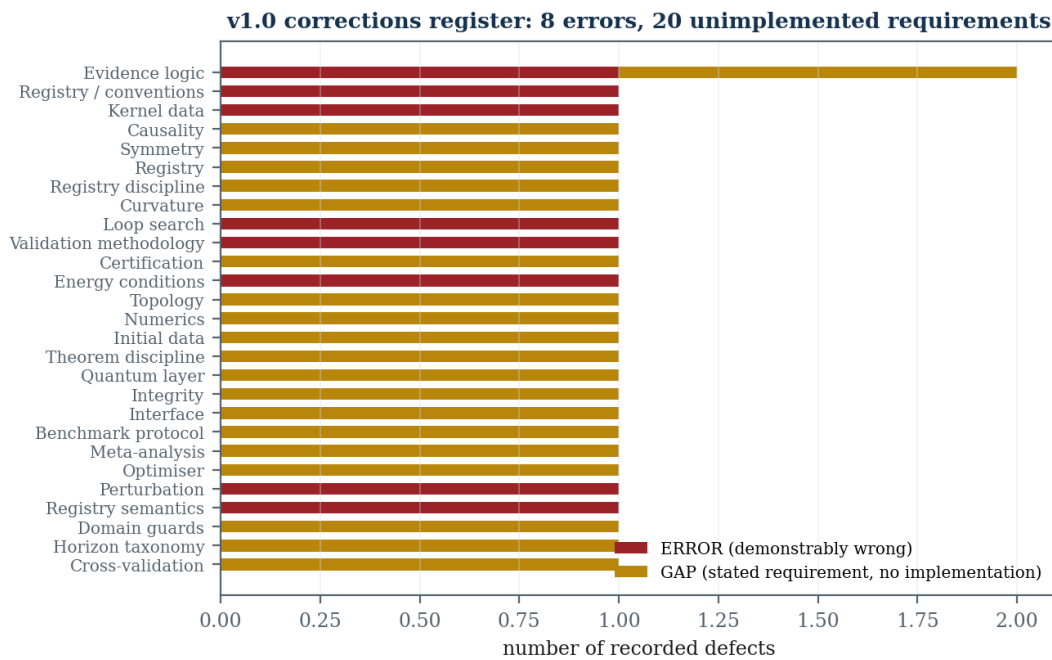


Figure 2. Where the v1.0 defects were. ERROR means demonstrably wrong; GAP means a requirement the document stated and the code did not implement.

2.1 Register

D-01 ERROR — Registry / conventions

v1.0 location	Section 7, table M05 Kerr
Defect	The registry table printed $g_{t\phi} = +2 M a r \sin^2(\theta)/\Sigma$ while the kernel used the negative sign. In Boyer-Lindquist with the $(-, +, +, +)$ signature ds^2 contains $-(4 M a r \sin^2 \theta / \Sigma) dt d\phi$, so $g_{t\phi} < 0$ for $M, a, r > 0$.
Evidence	The two statements in one document contradict each other; the kernel was right. Frame dragging with the printed sign would rotate the wrong way.
Fix in v2.0	Registry corrected to the negative sign; the Kerr entry now also carries an explicit convention note recording the discrepancy.
Regression test	<code>tests/unit/test_conventions.py::test_kerr_g_t_phi_sign_is_negative</code> and <code>::test_kerr_reduces_to_schwarzschild_when_a_zero</code>

D-02 ERROR — Kernel data

v1.0 location	Appendix A, <code>van_stockum()</code>
Defect	The stored van Stockum metric placed the $dt\,d\phi$ term in one off-diagonal slot only, so $g \neq g^T$.
Evidence	REQ-002 requires every local metric to be symmetric. The compact-orbit detector reads only $g_{\phi\phi}$, which is why the published margin $-1.200000e+01$ was still correct and the defect survived the self-test; every Christoffel, Einstein-tensor or geodesic computation on that model would have been wrong.
Fix in v2.0	The symmetric Bonnor form is stored, and <code>MetricModel.assert_wellformed</code> rejects any asymmetric registry entry at load time, for every model.
Regression test	<code>tests/unit/test_conventions.py::test_van_stockum_metric_is_symmetric</code> , <code>::test_every_registered_metric_is_symmetric</code> , and <code>tests/controls::test_asymmetric_metric_is_refused_at_load</code>

D-03 GAP — Causality

v1.0 location	Section 3.2; <code>validate_sampled_loop</code>
Defect	Time orientation was required by the definition of a CTC witness and never checked: the validator returned "orientation_status": "NOT_CHECKED" and the compact-orbit detector did not look at orientation at all.
Evidence	Section 3.2 states the requirement explicitly. A closed timelike curve that reverses the future direction is not a causality violation.
Fix in v2.0	New module <code>ctcfa/orientation.py</code> builds a cone-continuous future-directed reference field along a curve, counts reversals and tests the time-orientation holonomy around the loop. Gate G6 of the compact-orbit detector and gate F2 of the deck-orbit detector consume it.
Regression test	<code>tests/unit/test_engines.py::test_orientation_detects_a_reversing_curve</code>

D-04 GAP — Symmetry

v1.0 location	Section 6.1 step 1; <code>compact_orbit_test</code>
Defect	A registry entry could declare any coordinate direction as a compact generator and the detector consumed it without verifying that it is a Killing vector.
Evidence	A non-Killing direction does not generate a one-parameter isometry group, so the orbit it traces is not the symmetry orbit the compact-orbit argument assumes.
Fix in v2.0	Gate G4: <code>symmetry.generators_admissible</code> computes the Killing residual $L_{\xi}g$ for every declared generator; a non-zero residual downgrades the result to <code>SECTOR_UNRESOLVED</code> .
Regression test	<code>tests/unit/test_engines.py::test_every_declared_compact_generator_is_killing</code> and <code>tests/controls::test_non_killing_generator_is_rejected</code>

D-05 ERROR — Evidence logic

v1.0 location	<code>global_model_test</code> ; sections 2, 9.2, 14
Defect	Scope had no representation in the kernel. <code>global_model_test</code> returned a bare <code>CTCStatus.CTC_PROVED</code> for the REDUCED time-shift wormhole predicate.
Evidence	Section 9.2 warns in prose that this predicate 'must not be quoted as a universal exact theorem'. The code then quoted it that way.
Fix in v2.0	<code>ClaimScope</code> (<code>SECTOR / REDUCED_MODEL / DECLARED_DOMAIN / GLOBAL</code>) is carried with every <code>Verdict</code> ; <code>Verdict.promote</code> is the only widening path and refuses to widen a negative result without a named certificate.
Regression test	<code>tests/controls::test_reduced_predicate_stays_at_reduced_scope</code> and <code>tests/unit/test_status_and_gates.py</code> (7 tests)

D-06 **GAP** — Registry

v1.0 location	Section 7, table M21 Krasnikov
Defect	The tube metric depends on a smooth-step regulator that is a numerical choice, not part of an exact solution; nothing in the data model recorded that, so a boundary could have been derived from it.
Evidence	The v1.0 table itself says 'the regulator is a numerical choice, not a unique canonical form'.
Fix in v2.0	The entry carries metadata regulator_dependent = True, and the continuation engine is not run on it.
Regression test	registry/metrics/krasnikov.json

D-07 **GAP** — Registry discipline

v1.0 location	REQ-016; ModelKind
Defect	EXTERNAL_EXACT_SPEC and COMPOSITE_GLOBAL were enum values with no enforcement: nothing prevented a later contributor from attaching an invented local tensor.
Evidence	REQ-016 requires composite/global constructions not to be replaced by invented local tensors.
Fix in v2.0	MetricModel.require_metric raises for such entries, and the registry audit fails if one of them carries a metric.
Regression test	tests/controls::test_external_spec_refuses_to_produce_a_metric

D-08 **GAP** — Evidence logic

v1.0 location	Appendix D claim ladder
Defect	The claim ladder L0-L9 was documented and never enforced; nothing stopped a level-2 result being described at level 6.
Evidence	Appendix D states 'A model must not be described at a higher level than its weakest unresolved gate.' There was no code path implementing that.
Fix in v2.0	ClaimLevel + LADDER_REQUIREMENT + GateRecord.highest_permitted_level; Verdict.render raises OverclaimError above the permitted level, and L9 is unreachable by construction.
Regression test	tests/unit/test_status_and_gates.py::test_claim_ladder_blocks_unsupported_levels and ::test_l9_is_unreachable_by_construction

D-09 **GAP** — Curvature

v1.0 location	kretschmann_placeholder
Defect	The Kretschmann scalar raised NotImplementedError.
Evidence	$K = R_{abcd} R^{abcd}$ is the only tool in the architecture that separates a coordinate singularity from a curvature singularity, so FM-001 had no computational mitigation at all.
Fix in v2.0	symbolic.kretschmann is implemented exactly and benchmarked against $48 M^2 / r^6$ for Schwarzschild; the horizon engine uses it to separate chart degeneracies from genuine singularities.
Regression test	tests/unit/test_conventions.py::test_schwarzschild_kretschmann_exact

D-10 **ERROR** — Loop search

v1.0 location	search_fourier_ctc
Defect	Four independent problems: (a) time orientation not checked; (b) the reported 'max norm' was a log-sum-exp SMOOTH maximum, which can be negative while the true maximum is positive; (c) a candidate was never re-validated on a denser grid or in another parametrisation; (d) nothing bounded the Fourier amplitudes, so an unbounded curve attains an unbounded negative norm by parametrisation alone.
Evidence	ALG-CTC-002 steps 8-10 require exactly (b), (c) and a denser re-check; FM-004 names (d) as 'optimizer finds discretization artifact -> false positive'.
Fix in v2.0	loops.py separates SEARCH from VALIDATION. Validation recomputes the EXACT maximum on a 4x grid, repeats at 8x, tests closure on the quotient, tests time orientation, rejects degenerate tangents and rejects runaway curves by coordinate extent. The reported margin is $\max g(u,u)/ u ^2$, which for a pure symmetry orbit reduces exactly to the L5 compact-orbit margin.
Regression test	Cross-validation: the L6 search independently reproduces the L5 margins -2.055335e+01 (Godel), -1.200000e+01 (van Stockum) and -1.000000e+00 (periodic Minkowski).

D-11 ERROR — Validation methodology

v1.0 location	godel_family_margin; section 8
Defect	The Godel-type causal boundary was 'recovered' by root-finding the hard-coded expression $m^2 - 4\omega^2$, and the result $m^* = 1.9999999999999987$ was published as a regression result.
Evidence	This is FM-016, 'known literature threshold encoded into detector and later claimed rediscovered -> circular validation', committed by the document that names it.
Fix in v2.0	The closed form is deleted from the solver path. The boundary is derived from the metric via the reduced causal margin $\mu = g(x_i, x_i)/h(x_i)$, whose infimum is root-found in the parameter. The analytic value is used only afterwards as a score, and a CI test asserts that the closed form does not appear in the solver source.
Regression test	tests/analytic::test_godel_boundary_converges_to_two_omega and ::test_solver_is_non_circular

D-12 GAP — Certification

v1.0 location	L9 gate; FM-009; ALG-CAUS-001
Defect	Interval or analytic certification was named as the requirement for a formal PASS in three separate places and implemented nowhere, so no positivity statement in the architecture could ever be certified.
Evidence	L9: 'analytic/interval certification is required for formal PASS'. 24.4: 'Point sampling alone does not establish the strict global inequality.'
Fix in v2.0	New module ctcf/interval.py: outward-rounded interval arithmetic on mpmath.iv plus branch-and-bound, giving a PROOF of a strict inequality on a compact box, or a counterexample. Used by the temporal-function certificate and the sector infimum.
Regression test	tests/unit/test_engines.py::test_interval_enclosure_contains_the_true_range and ::test_branch_and_bound_proves_and_refutes

D-13 ERROR — Energy conditions

v1.0 location	sampled_energy_conditions; anec_trapezoid
Defect	NEC and WEC were evaluated on 256 Fibonacci-sphere directions and labelled 'Sampled diagnostic only'; SEC and DEC were never computed at all despite being in the state vector; ANEC consumed a caller-supplied list of contractions rather than integrating a geodesic.
Evidence	FM-009: 'Energy-condition result based on finite directional samples -> false proof.'
Fix in v2.0	energy.py computes all four conditions by two exact routes that must agree: the Hawking-Ellis Type I closed forms, and the exact solution of the trust-region subproblem on the unit sphere. ANEC is integrated along a null geodesic produced by geodesics.py, with the conserved norm drift reported as an independent accuracy check.
Regression test	tests/unit/test_engines.py::test_trust_region_beats_dense_random_search and tests/regression::test_energy_conditions_on_the_key_models

D-14 GAP — Topology

v1.0 location	FM-007; state vector components T and I
Defect	Topology and identifications were free-text strings. The architecture could not distinguish two spacetimes with the same local metric and different global causal structure -- which is the entire content of its own L3 gate.
Evidence	FM-007: 'Topology stored only as prose and misapplied -> global causal error. Mitigation: implement quotient/cover objects and deck transformations in v2.'
Fix in v2.0	topology.py plus Quotient/DeckTransformation in model.py: symbolic isometry certificates, free-action checks, time-orientation checks, word enumeration and closure on the quotient. The registry ships the ads4_universal_cover / ads4_periodic pair and the minkowski / minkowski_time_periodic pair, each with one identical local metric and opposite verdicts.
Regression test	tests/controls::test_ads_pair_shares_one_metric_and_splits_on_the_deck_group

D-15 GAP — Numerics

v1.0 location	Section 29; FM-017
Defect	The precision policy ('escalates precision and recomputes' near marginal boundaries) had no implementation; sign decisions used raw double precision with a fixed tolerance.
Evidence	FM-017: 'Software precision masks sign near $q = 0$ -> boundary error. Mitigation: X64/high precision and error interval around zero.'
Fix in v2.0	numerics.py: composable ErrorBudget, classify_sign with a band, and escalate_if_marginal which recomputes the exact symbolic expression at 50 significant digits whenever the value lies inside the band.
Regression test	tests/unit/test_engines.py::test_sign_classification_uses_the_uncertainty_band

D-16 GAP — Initial data

v1.0 location	adm_hamiltonian_residual; L12
Defect	The ADM constraint expressions existed as symbolic functions that nothing called, and there was no way to construct regular initial data.
Evidence	L12 gate: 'Constructibility from regular initial data is stronger than existence of an eternal or analytically extended metric.'
Fix in v2.0	adm.py: a full 3+1 decomposition (lapse, shift, spatial metric, extrinsic curvature), constraint residuals evaluated at a point, and a Newton solver for the conformal (York) Hamiltonian constraint that actually constructs data. The evolution gate stays NOT_ATTEMPTED and is documented as the reason no model may be reported above L5.
Regression test	Verified against the Friedmann equation: the FLRW Hamiltonian residual with $\rho = 0$ equals 16π times $3 H^2 / (8 \pi)$ exactly.

D-17 GAP — Theorem discipline

v1.0 location	Section 28; FM-010/011/012
Defect	The theorem applicability matrix was a prose table. The stated mitigation 'hypothesis checklist with base-point/horizon class' did not exist in code, so nothing prevented a theorem's name being used as a slogan.
Evidence	Three separate failure modes (FM-010, FM-011, FM-012) describe exactly this and all name a checklist as the mitigation.
Fix in v2.0	theorems.py: seven theorems as objects, each with executable hypothesis checkers returning True/False/None, the exact label it licenses, and the inference it must never be used for. LICENSED requires every hypothesis verified; one None gives HYPOTHESES_UNVERIFIED.
Regression test	tests/regression::test_theorem_validator_never_licenses_unverified_hypotheses and ::test_chronology_protection_is_never_labelled_proved

D-18 GAP — Quantum layer

v1.0 location	QuantumAssessment; L13
Defect	The quantum layer was a dataclass of four literal 'UNRESOLVED' strings. There was no specification object, no KRW hypothesis check and no QEI keyed by its assumptions.
Evidence	REQ-011 requires quantum assessment to be field- and state-specific; FM-012 requires QEIs to be keyed by field, dimension, state, trajectory and sampling.
Fix in v2.0	qft.py: QuantumSpecification with a completeness test, executable KRW hypotheses, an assessment that refuses to produce numbers without a full specification, and a QEI registry whose rules only fire when every assumption matches. Polarised hypersurfaces computed by the topology engine feed in as candidate RSET divergence loci.
Regression test	examples/ex07_theorems_and_quantum.py

D-19 GAP — Integrity

v1.0 location	Section 30
Defect	Seven integrity controls were listed. None was implemented: no hash was computed anywhere in the kernel, no run record was written, and the benchmark answers were literals in the source.
Evidence	Section 30 lists SHA-256 of every artifact, signed releases, immutable manifests, encrypted/segregated benchmark answers, no overwriting, dependency recording and a reproduction script.
Fix in v2.0	provenance.py implements all seven, including HMAC-sealed benchmark answers and a two-pass build in which the source digest and the artifact digest are both published.
Regression test	tests/regression::test_provenance_round_trip

D-20 GAP — Interface

v1.0 location	Section 26.1
Defect	The CLI was declared 'a normative target interface, not a claim that every command already exists'.
Evidence	Stated in the document.
Fix in v2.0	Every command exists and runs: list, show, validate, tensors, etc, certify-time, boundary, energy, horizons, theorems, quotient, matrix, meta, manifest, export-registry, selftest.
Regression test	make selftest / make registry / make manifest

D-21 GAP — Benchmark protocol

v1.0 location	Section 15.2
Defect	The blind benchmark protocol was six numbered steps with no implementation, and the regression expectations were literals in the kernel source -- the opposite arrangement to a blind test.
Evidence	Step 3 requires hiding exact thresholds from the solver.
Fix in v2.0	tests/blind: answers are HMAC-sealed with an evaluator key, the parameter sets are drawn at run time so a memorised literal cannot pass, and the solver submits a value that is verified without ever seeing the target.
Regression test	tests/blind/test_blind_benchmark.py (19 tests)

D-22 GAP — Meta-analysis

v1.0 location	ALG-META-001; section 24.10
Defect	Cross-family invariant search was specified with train/held-out discipline and never implemented.
Evidence	REQ-019 requires blind validation to precede unknown-family discovery claims.
Fix in v2.0	meta.py: a feature vector built only from what the other layers compute, single-feature rules derived on a training split and scored on a held-out split, a leakage check that flags any perfectly separating feature, and a governance string that caps the emittable label at CANDIDATE_PRECURSOR.
Regression test	tests/regression::test_meta_analysis_reports_held_out_scores_and_governance

D-23 GAP — Optimiser

v1.0 location	L17 gate; REQ-009
Defect	The minimal-perturbation cost had no object: scales and weights could be chosen after seeing the answer, which makes 'minimal' meaningless.
Evidence	L17 gate: 'The cost metric and normalization scales are preregistered; otherwise the word minimal is meaningless.'
Fix in v2.0	optimize.CostMetric must be sealed before use; sealing records a SHA-256 of the scales and weights, which is reported with every result. An unsealed metric raises UnsealedCostError.
Regression test	examples/ex10_perturbation_and_minimal.py

D-24 ERROR — Perturbation

v1.0 location	perturb_metric; FM-005
Defect	An arbitrary symmetric tensor could be added to a metric and the resulting causal margin quoted, with nothing recording that the perturbation was off shell.
Evidence	FM-005: 'Off-shell perturbation used as physical stability evidence -> invalid physics.' L11 gate says the same.
Fix in v2.0	perturbation.py measures the field-equation residual before and after and assigns OnShellStatus (EXACT_FAMILY / CONSTRAINT_SOLVED / OFF_SHELL). An OFF_SHELL result is flagged exploratory and says so in its own note.
Regression test	examples/ex10 step L11b

D-25 ERROR — Registry semantics

v1.0 location	Section 14 regression table
Defect	One 'expected' field served both the family-level statement and the point-level statement, so the Godel-type family (UNRESOLVED as a family, because its causal character depends on m and omega) was listed twice with different parameters and no parameter column. The published table cannot be reproduced from the printed information.
Evidence	The v1.0 table has two rows labelled 'godel_type' with different outcomes and no way to tell which parameters produced which.
Fix in v2.0	MetricModel.expected_at_default separates the two statements, and every row of the v2.0 regression matrix records its derivation ROUTE and its point.
Regression test	tests/regression::test_every_row_records_its_derivation_route

D-26 GAP — Domain guards

v1.0 location	FM-014
Defect	Nothing checked that a causal-boundary root lay inside the physical parameter domain, or that the normalisation used to define the margin was still valid there.
Evidence	FM-014: 'Boundary root caused by leaving physical parameter domain -> false threshold.'
Fix in v2.0	DomainConstraint objects are checked at every detector call, and continuation.reference_validity rejects a root at which the timelike reference direction goes null. The charged pseudo-RN core produces such an artefact at $r^2 + 2\mu r - q^2 = 0$ and it is correctly rejected.
Regression test	tests/analytic::test_pseudo_rn_normalisation_artefact_is_rejected and tests/controls::test_point_outside_the_declared_domain_is_refused

D-27 GAP — Horizon taxonomy

v1.0 location	Section 10; inverse_radial_horizon_candidates
Defect	The four horizon objects were a prose table backed by one solver for $g^{rr} = 0$, which cannot distinguish them.
Evidence	FM-006: 'Cauchy horizon labeled CTC -> category error.'
Fix in v2.0	horizons.py computes Killing horizons, chart degeneracies, curvature singularities and the chronology horizon separately, each with the inference it forbids attached. Reissner-Nordstrom returns two Killing horizons, one curvature singularity and NO chronology horizon.
Regression test	tests/controls::test_reissner_cauchy_horizon_is_not_a_chronology_horizon

D-28 **GAP** — Cross-validation

v1.0 location	Section 13 'Planned external validation'
Defect	Independent tensor validation was listed as planned external work, so a single algebra bug in the one implementation would have been invisible.
Evidence	FM-018: 'Single software stack propagates algebraic bug -> systematic error.'
Fix in v2.0	numgeom.py provides a completely independent finite-difference curvature route with a reported truncation error, and the V2 gate requires the two routes to agree on every model in the registry.
Regression test	validate.validate_registry V2 column; agreement between 0 and 4e-05 across all 28 models with a local metric.

2.2 Presentation defects

ID	Defect
P-01	Numbered procedures do not start at 1. Section 12.1 begins at item 7, section 15.2 at 14, 24.1 at 20, 24.3 at 33, 24.5 at 40 and 24.8 at 46, because list numbering ran continuously across the whole document. Every procedure in v2.0 restarts at 1.
P-02	The section 14 regression block gives no parameter values, so the table is not reproducible from the document alone. The v2.0 matrix carries the route, and the full point is in the registry record and the run manifest.
P-03	Two rows of the regression table share the model name 'godel_type' with opposite outcomes and no distinguishing column. v2.0 gives the critical member its own registry entry (godel_critical, M26).

2.3 What v1.0 got right and v2.0 preserved

The following were verified during the audit and carried into v2.0 unchanged. Two deserve emphasis: the Gott two-string condition as v1.0 printed it is algebraically equivalent to the standard form, and the asymmetric evidence standard — one witness proves existence, absence requires a certificate — is the correct central design idea and is now a type rather than a paragraph.

Item	Finding
Gott two-string condition	$\gamma_s > 1/\sin(4\pi\mu)$ is exactly equivalent to the standard form $\gamma_v > \cot(4\pi\mu)$, since $\gamma^2 - \gamma^2 v^2 = 1$. Verified algebraically and preserved.
Godel-type family	$H = (4\omega/m^2) \sinh^2(mr/2)$, $D = \sinh(mr)/m$, $g_{\phi\phi} = D^2 - H^2$ and $\sinh^2(mr_c/2) = [4\omega^2/m^2 - 1]^{-1}$ are all correct as printed.
Published regression margins	Every numeric margin the v1.0 self-test published is reproduced exactly by the rewritten v2.0 engines: -1.670000e+00 (Kerr), -1.772400e+00 (Kerr-Newman), -1.736478e+01 and +1.315412e+01 (Godel-type), -1.000000e+00 (Misner), -1.200000e+01 (van Stockum), -6.923595e-01 (Taub-NUT), -1.536000e-01 (spinning string), +4.000000e+00 (Morris-Thorne), -1.000000e+00 (Ori 2005 and Ori 2007 core), +1.000000e-02 (Krasnikov), +2.500000e+01 (Schwarzschild and Reissner-Nordstrom). This is the strongest available evidence that the v1.0 numerics were sound wherever they were actually computed.
Signature invariance test	Using the eigenvalue SIGN COUNTS of the coordinate metric as the signature test is correct and coordinate independent by Sylvester's law of inertia. Preserved and now tested explicitly.
Claim boundary	Section 0.1 ('what the architecture does not claim') is exemplary and is carried into v2.0 unchanged in substance and strengthened in scope.
Asymmetric evidence standard	The rule that one witness proves existence while absence requires a certificate is correct, is the central design idea, and is now a type rather than a paragraph.
Metric forms	Misner, Ori 2005, Ori 2007 core, spinning cosmic string, Taub-NUT, Morris-Thorne, Alcubierre, Schwarzschild, Reissner-Nordstrom, Kerr-Newman and the Godel special case are all correct as published.

3. Scientific Contract and Status Logic

The architecture uses asymmetric evidence standards. A single rigorously verified CTC witness is sufficient to prove existence in the declared spacetime. By contrast, failure to discover a CTC numerically is not evidence of absence. Absence requires a causal certificate, or a theorem whose hypotheses are satisfied on the full declared domain.

Global status	Meaning	Permitted inference
CTC_PROVED	An explicit closed timelike curve, or a mathematically complete equivalent criterion, has been verified.	CTCs exist in the declared model and domain.
CRITICAL	A closed-null / marginal boundary is established.	The system is on a chronology-transition boundary for the verified sector or criterion.
NO_CTC_CERTIFIED	A complete causal certificate or theorem excludes CTCs in the declared domain.	Chronology violation is excluded within that certified scope.
UNRESOLVED	Neither existence nor absence has been proved.	No binary causal claim is permitted. Absorbing state.

3.1 Scope is part of the status

v1.0 stated scope rules in prose in three separate sections and represented scope nowhere in the kernel; its `global_model_test` returned a bare `CTC_PROVED` for the explicitly reduced time-shift wormhole predicate, which is precisely the category error section 9.2 of v1.0 warns against. In v2.0 every verdict carries a `ClaimScope`.

Scope	Valid where	Example
SECTOR	only in the scanned compact-generator sector	Schwarzschild azimuthal loops are spacelike — this clears one sector and is NOT a no-CTC result
REDUCED_MODEL	only under an architecture-specific reduced predicate	the time-shifted wormhole route-closure margin $t_{\text{ext}} + t_{\text{wh}} - \Delta t $
DECLARED_DOMAIN	on the coordinate and parameter domain declared in the registry record	Schwarzschild t is a temporal function only for $r > 2M$
GLOBAL	on the maximal declared spacetime including its identifications	Minkowski, FLRW and the AdS universal cover, each by an unconditional symbolic certificate

Composition never strengthens: *restrict* is always allowed, and *promote* requires an explicit justification. Promotion of a negative result to GLOBAL is refused unless the justification names a temporal-function certificate or a fully verified theorem. Promotion of UNRESOLVED is refused outright.

3.2 Sector results are a different type

Compact-orbit results use `OrbitStatus`, not `CTCStatus`. The two enums are deliberately non-interchangeable, so that the v1.0 failure mode FM-002 — silent promotion of `SECTOR_SPACELIKE` to `NO_CTC_CERTIFIED` — becomes a type error rather than a review finding. The single bridge is `OrbitResult.to_verdict`, which maps `SECTOR_CTC` to `CTC_PROVED` at SECTOR scope, `SECTOR_NULL` to `CRITICAL`, and both `SECTOR_SPACELIKE` and `SECTOR_UNRESOLVED` to `UNRESOLVED`.

3.3 The claim ladder, enforced

Appendix D of v1.0 defined a ten-level claim ladder and stated that a model must not be described above its weakest unresolved gate. No code implemented it. In v2.0 the ladder is a mapping from claim level to the set of verification gates that must have passed, and `Verdict.render` raises `OverclaimError` above the permitted level. L9, quantum-gravity consistency, is unreachable by construction: no accepted complete theory exists to certify against, so the architecture may never

emit it, even with every gate passing.

Scope lattice (composition never strengthens)

Claim ladder (render() refuses above the evidence)

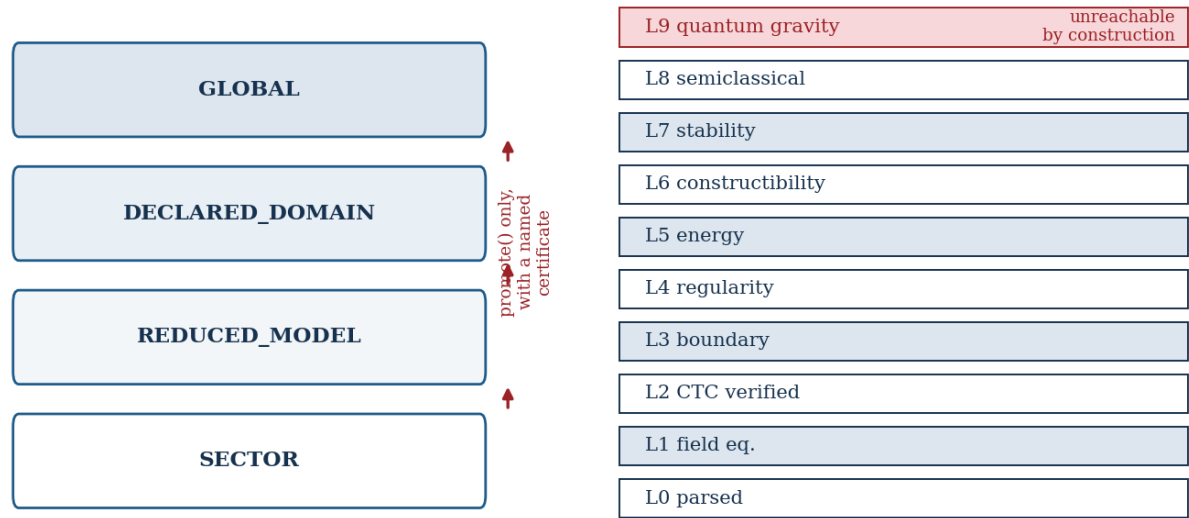


Figure 3. Left: the scope lattice; widening requires an explicit justification and a named certificate for negative results. Right: the claim ladder, with L9 unreachable by construction.

4. Mathematical Foundation

4.1 Field equations and conventions

$$G_{\{\mu \nu\}} + \text{Lambda } g_{\{\mu \nu\}} = 8 \pi T_{\{\mu \nu\}} \quad [G = c = 1]$$

$$\text{Gamma}^a_{\{bc\}} = (1/2) g^{\{ad\}} (d_b g_{\{dc\}} + d_c g_{\{db\}} - d_d g_{\{bc\}})$$

$$R^a_{\{bcd\}} = d_c \text{Gamma}^a_{\{db\}} - d_d \text{Gamma}^a_{\{cb\}} + \text{Gamma}^a_{\{ce\}} \text{Gamma}^e_{\{db\}} - \text{Gamma}^a_{\{de\}} \text{Gamma}^e_{\{cb\}}$$

$$R_{\{bd\}} = R^a_{\{bad\}}, \quad R = g^{\{bd\}} R_{\{bd\}}, \quad G_{\{ab\}} = R_{\{ab\}} - (1/2) g_{\{ab\}} R, \quad K = R_{\{abcd\}} R^{\{abcd\}}$$

Every registered chart declares its coordinate ordering and signature. The reference implementation uses $(-,+,+,+)$ and verifies a nondegenerate Lorentzian signature numerically before accepting any witness. The conventions above are *pinned by tests*, not merely documented: Schwarzschild must return $R_{ab} = 0$ and $K = 48M^2/r^6$, de Sitter must return $R = 4\Lambda$ and AdS must return $R = -12/L^2$. A convention drift is a test failure, which is the mitigation v1.0 listed for FM-019 and did not implement.

4.2 Closed timelike curve

$$\text{gamma} : S^1 \rightarrow M, \quad \text{gamma}(0) = \text{gamma}(1)$$

$$g_{\{\mu \nu\}} \text{gamma_dot}^\mu \text{gamma_dot}^\nu < 0 \text{ everywhere along gamma}$$

A complete witness must also respect a consistent time orientation. v1.0 stated this requirement and never checked it; its loop validator returned the literal string NOT_CHECKED. v2.0 builds a cone-continuous future-directed reference field along the curve, counts sign reversals of $g(\dot{\gamma}, T)$, and tests the time-orientation holonomy around the loop. A closed timelike curve with reversing orientation is rejected: it is not a causality violation.

4.3 Chronology-violating set

$$C = \{ p \text{ in } M \mid p \ll p \}, \quad H_{\text{chr}} = \text{boundary}(C)$$

The boundary of the chronology-violating set is a natural object for studying CTC onset, but it must never be conflated with a Cauchy horizon. Reissner-Nordström is the decisive control: its maximal extension contains an inner Cauchy horizon and no chronology violation whatsoever. Misner space is the converse: a chronology horizon with no curvature singularity anywhere and determinant identically -1 .

4.4 Stable causality and temporal functions

$$g^{\{\mu \nu\}} d_\mu \tau d_\nu \tau < 0 \quad \text{equivalently} \quad C_\tau(x) = - g^{\{\mu \nu\}} d_\mu \tau d_\nu \tau > 0$$

A smooth function with everywhere timelike gradient is a temporal function; its existence certifies stable causality and therefore excludes CTCs [R16]. v2.0 adds two requirements that v1.0 lacked. First, the strict inequality must be *proved* — symbolically, or by interval branch-and-bound on a stated compact box — never sampled. Second, and decisively, τ must be *single valued on the quotient*: it is checked for invariance under every deck generator and every declared coordinate period before any certificate is attempted. Without that check the temporal-function route would certify AdS with periodic time, which is entirely chronology violating.

4.5 Compact-generator Gram matrix

$$K_{AB}(x) = g_{\{\mu \nu\}}(x) x_i A^\mu x_j B^\nu, \quad q(x,n) = n^A K_{AB}(x) n^B, \quad n \text{ in } \mathbb{Z}^k \setminus \{0\}$$

If the ξ_A generate closed orbits under periodic identifications, $q < 0$ is an explicit symmetry-generated CTC, $q = 0$ is a marginal closed null orbit, and $q > 0$ proves only that the

scanned compact-generator sector is spacelike. v2.0 restricts the scan to *primitive* windings, since n and kn trace the same orbit, and requires each ξ_A to be a verified Killing vector.

4.6 The reduced causal margin (new in v2.0)

The raw component $q = g(\xi,\xi)$ carries an arbitrary scale: it is 2.5×10^3 for one Gödel-type point and 1×10^{-2} for a Krasnikov point, so it cannot be compared across models and it defeats any optimiser that must weigh it against a cost. v2.0 introduces a dimensionless replacement. Let u be a timelike reference direction and decompose ξ orthogonally:

$$xi_perp = xi - [g(xi,u) / g(u,u)] u, \quad h(xi) = g(xi,xi) - g(xi,u)^2 / g(u,u) > 0$$
$$\mu(x) = g(xi,xi) / h(xi)$$

h is strictly positive because the orthogonal complement of a timelike vector is spacelike, so μ has the same sign as q and decides causality identically. It is invariant under rescaling of the generator and stays finite where the raw component diverges. Its value is not an ad-hoc construction: for the standard families it collapses to exactly the quantity the literature uses.

Family	Reduced causal margin, derived from g_{ab}	Root
Gödel-type	$\mu = 1 - (4\omega^2/m^2) \tanh^2(mr/2); \inf_r \mu = 1 - 4\omega^2/m^2$	$m = 2\omega$
van Stockum	$\mu = 1 - a^2r^2$	$a r = 1$
spinning cosmic string	$\mu = 1 - 16J^2/(\alpha^2r^2)$	$\alpha r = 4J$
Ori 2007 core	$\mu = 1 - 4\mu_m^2/r^2$	$r = 2\mu_m$
pseudo-Reissner-Nordström	$\mu = (r^2 - 2\mu_m r + q^2)(r^2 + 2\mu_m r - q^2) / r^4$	$r_{\pm} = \mu_m \pm \sqrt{(\mu_m^2 - q^2)}$
Misner	$\mu = -T(T + 2)$	$T = 0$
Ori 2005	$\mu = -(T - F)(T - F + 2), F = x^2 - y^2$	$T = F(x,y)$

A normalisation has its own failure mode, and it is checked
 μ acquires a spurious zero wherever the reference direction itself becomes null. The charged pseudo-RN core produces one at $r^2 + 2\mu_m r - q^2 = 0$, at $r = 0.118034$ for $\mu_m = 1, q = 0.5$ — close to, but distinct from, the genuine boundary at $r_- = 0.133975$. The continuation engine evaluates $-g(u,u)$ at every root it finds and rejects the artefact. Reporting it would have been failure mode FM-001 in a new disguise.

Reduced causal margins, computed from the metric tensor with no closed form supplied

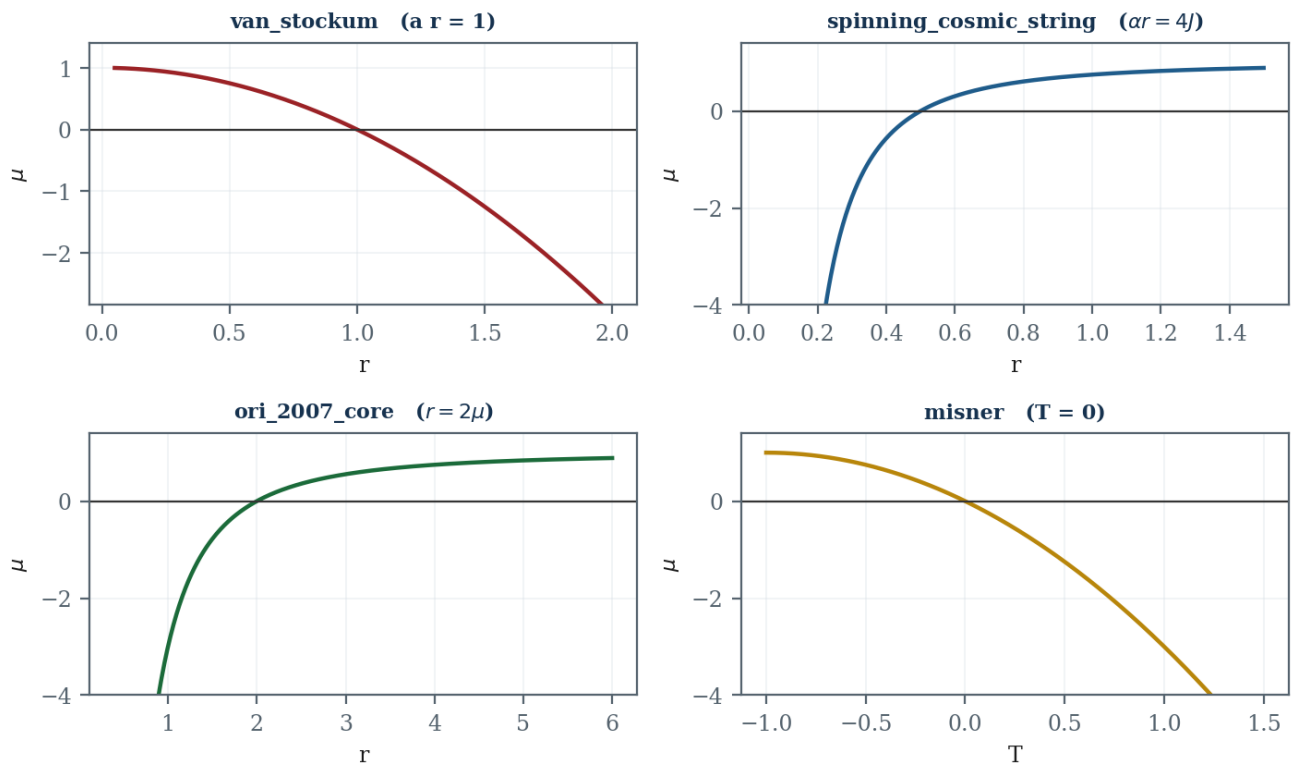


Figure 4. Reduced causal margins for four families, computed from the metric tensor with no closed form supplied to the solver.

4.7 Energy conditions

NEC: $T_{\{\mu \nu\}} k^{\mu} k^{\nu} \geq 0$ for every null k WEC: $T_{\{\mu \nu\}} u^{\mu} u^{\nu} \geq 0$ for every timelike u

SEC: $(T_{\{\mu \nu\}} - (1/2) T g_{\{\mu \nu\}}) u^{\mu} u^{\nu} \geq 0$ DEC: $\rho \geq |p_i|$ for all i

ANEC: $\int_{\gamma} T_{\{\mu \nu\}} k^{\mu} k^{\nu} d\lambda \geq 0$ along a complete null geodesic

The architecture does not assume that every CTC geometry violates the NEC. Ori's constructions are specifically retained as counterexamples to that overgeneralisation [R14, R15], and van Stockum is retained as a chronology-violating spacetime with strictly positive energy density that satisfies all four conditions. Conversely, classical traversable Morris-Thorne wormholes require exotic stress-energy under the standard GR assumptions [R4, R18] and contain no CTC. The two properties are logically independent in both directions, and Section 11 shows the computation in each.

4.8 ADM constraints

$$R^{(3)} + K^2 - K_{ij} K^{ij} = 16 \pi \rho \quad D_j (K^{ij} - h^{ij} K) = 8 \pi j^i$$

A candidate created by arbitrary deformation of a metric is not physically constructible unless the relevant field equations and constraint system are solved. This is the central distinction between off-shell exploratory perturbation and on-shell dynamical research, and in v2.0 it is a recorded status on every perturbation rather than a warning in the text.

4.9 Semiclassical equation

$$G_{\{\mu \nu\}} + \Lambda g_{\{\mu \nu\}} = 8 \pi [T^{\text{classical}}_{\{\mu \nu\}} + \langle T_{\{\mu \nu\}} \rangle_{\text{ren}}]$$

The renormalised expectation value is field- and state-dependent. CTC-FA therefore does not generate a universal $\langle T_{\mu\nu} \rangle$ from a metric, and in v2.0 the quantum layer *refuses to return a number*

without a complete specification of field content, state and renormalisation prescription. Kay-Radzikowski-Wald establish a specific obstruction for QFT at base points of compactly generated Cauchy horizons; the correct output there is SEMICLASSICAL_QFT_BREAKDOWN or UNRESOLVED, never an invented infinity [R11].

5. Layered Computational Architecture L0-L19

Each layer has a purpose, a scientific gate that constrains what its output may be used for, and — new in v2.0 — a recorded implementation status. The gates are quoted from v1.0 where they were already correct; what has changed is that they are now enforced.

L0 Metric & Model Registry

Purpose	Store the declared manifold, chart, metric tensor, parameter domain, topology, identifications, matter content, field theory, provenance and verification level.
Scientific gate	No computation may begin without an explicit signature convention, coordinate domain and topology/identification record.
Status in v2.0	v2.0: 33 models; structural gate rejects an asymmetric metric, an undeclared symbol or a generator/coordinate mismatch at load.

L1 Symbolic Tensor Engine

Purpose	Inverse metric, Christoffel symbols, Riemann, Ricci tensor and scalar, Einstein tensor, Kretschmann scalar, Lie derivatives.
Scientific gate	All tensor identities are evaluated under a declared convention and cross-checked against benchmark solutions.
Status in v2.0	v2.0: Kretschmann implemented (was a placeholder raising NotImplementedError); conventions pinned by four exact tests.

L2 Field-Equation Validator

Purpose	Evaluate $E_{ab} = G_{ab} + \Lambda g_{ab} - 8\pi T_{ab}$ and quantify the residual symbolically and numerically.
Scientific gate	A candidate perturbation or parameter point is not physically admissible merely because a CTC detector fires; on-shell validity is separate.
Status in v2.0	v2.0: evaluated at 30 significant digits so vacuum cancellations are resolved below the tolerance; models with no closed-form T are reported NOT_APPLICABLE, since there the residual DEFINES the source.

L3 Topology & Identification Engine

Purpose	Deck transformations as explicit coordinate maps with inverses; word enumeration in Γ ; symbolic isometry certificates; free-action and time-orientation checks; closure of a curve on the quotient.
Scientific gate	Local equality of metrics does not imply equality of global causal structure.
Status in v2.0	v2.0: NEW. v1.0 stored topology as prose and recorded the consequence itself as FM-007.

L4 Killing / Symmetry Engine

Purpose	Verify that declared compact generators are Killing vectors; build the Gram matrix K_{AB} ; solve the Killing equation for a polynomial ansatz.
Scientific gate	Symmetry-assisted CTC witnesses are sufficient when found but not complete for arbitrary CTCs.
Status in v2.0	v2.0: verification is now a GATE. A non-Killing declared generator downgrades the result to SECTOR_UNRESOLVED. Minkowski returns the full 10-dimensional Poincaré algebra as a correctness check.

L5 Compact-Orbit CTC Engine

Purpose	Compute $q = n^A K_{AB} n^B$ over primitive integer windings and classify its sign with an uncertainty budget.
Scientific gate	$q < 0$ is an explicit compact-orbit CTC witness; $q = 0$ is marginal; $q > 0$ clears only the tested sector.
Status in v2.0	v2.0: seven admissibility gates (metric present, domain, signature, Killing, closure on the quotient, time orientation, uncertainty with 50-digit escalation). Primitive windings only.

L5b Deck-Orbit CTC Engine

Purpose	For a flat covering, the straight segment from p to $g(p)$ is a geodesic and projects to a closed curve; its causal character is exactly $\eta(g(p) - p, g(p) - p)$.
Scientific gate	Flatness of the covering is verified symbolically, and the deck element must be orthochronous.
Status in v2.0	v2.0: NEW. Handles Misner, Grant and every polarised Misner-type quotient, which the coordinate-based detector cannot see.

L6 Global Loop Search Engine

Purpose	Fourier-parameterised closed curves with optional quotient winding, optimised for a negative maximum norm.
Scientific gate	Failure to find a loop has no negative evidentiary value.
Status in v2.0	v2.0: SEARCH and VALIDATION separated. Validation recomputes the exact maximum on a $4\times$ grid, repeats at $8\times$, tests closure, tests orientation, rejects degenerate tangents and rejects runaway curves by coordinate extent.

L7 Causality Certificate Engine

Purpose	Search for a smooth τ with strictly timelike gradient on the declared domain and PROVE the strict inequality.
Scientific gate	A valid global temporal function certifies stable causality and therefore excludes CTCs in that domain.
Status in v2.0	v2.0: the proof is symbolic or interval branch-and-bound with the box reported; and τ must be single valued on the quotient, which is checked first.

L8 Horizon Engine

Purpose	Track Killing horizons, chart degeneracies, curvature singularities and chronology horizons as four distinct computed objects.
Scientific gate	No horizon label is automatically converted into a CTC claim.
Status in v2.0	v2.0: each object carries the inference it forbids; chronology horizons are derived from the causal margin and filtered for normalisation artefacts.

L9 Energy-Condition Engine

Purpose	NEC, WEC, SEC, DEC by Hawking-Ellis classification and by exact trust-region minimisation; ANEC on an integrated null geodesic.
Scientific gate	Numerical directional sampling is diagnostic only; exact or interval certification is required for a formal PASS.
Status in v2.0	v2.0: sampling removed from the certification path entirely. Two exact routes cross-check to 1.4×10^{-17} across the registry.

L10 Wormhole / Topology Layer

Purpose	Throat conditions, flare-out, redshift regularity, traversal structure.
Scientific gate	A wormhole throat is not a chronology violation; time shift and route closure are separate layers.
Status in v2.0	v2.0: the reduced route-closure predicate carries ClaimScope.REDUCED_MODEL and cannot be printed as a theorem.

L11 Perturbation Engine

Purpose	Apply controlled perturbations and re-evaluate field equations, causal margins and stability indicators.
Scientific gate	Off-shell perturbations cannot establish physical robustness.
Status in v2.0	v2.0: every perturbation carries an OnShellStatus measured from the field-equation residual; OFF_SHELL results are flagged exploratory.

L12 Initial-Data / ADM Engine

Purpose	3+1 decomposition, Hamiltonian and momentum constraints, and a conformal York solver that constructs regular data.
Scientific gate	Constructibility from regular initial data is stronger than existence of an eternal or analytically extended metric.
Status in v2.0	v2.0: the constraints are solved rather than merely written down; the evolution gate stays NOT_ATTEMPTED and is the stated reason no model may be reported above claim level L5.

L13 Quantum-State Layer

Purpose	Field content, spin, mass, couplings, state, state class, boundary conditions, renormalisation prescription, sampling function, trajectory.
Scientific gate	No generic renormalised stress tensor is inferred from the metric alone.
Status in v2.0	v2.0: QuantumSpecification with a completeness test; the assessment refuses to return numbers without one.

L14 Semiclassical Backreaction

Purpose	Solve or assess $G_{ab} + \Lambda g_{ab} = 8\pi(T_{\text{classical}} + \langle T \rangle_{\text{ren}})$.
Scientific gate	Classical CTC survival does not imply semiclassical survival.
Status in v2.0	v2.0: executable KRW hypotheses; polarised hypersurfaces computed by L3 are reported as candidate RSET divergence loci.

L15 Theorem Applicability Validator

Purpose	Check hypotheses before invoking chronology protection, topological censorship, KRW or any other no-go result.
Scientific gate	The system reports theorem scope and unmet assumptions instead of applying names as slogans.
Status in v2.0	v2.0: seven theorems as objects with executable hypothesis checkers; LICENSED requires every hypothesis verified.

L16 Parameter Continuation

Purpose	Track sign-changing causal margins and solve phase boundaries with bracketing plus high-precision confirmation.
Scientific gate	A numerical root is not accepted until bracket stability, domain validity and analytic reconstruction are checked where possible.
Status in v2.0	v2.0: the reduced causal margin replaces the raw component; the closed form is deleted from the solver path and a CI test asserts its absence.

L17 Minimal-Perturbation Optimiser

Purpose	Minimise a dimensionless weighted cost on an on-shell family subject to CTC birth or restored-causality constraints.
Scientific gate	The cost metric and normalisation scales are preregistered; otherwise the word minimal is meaningless.
Status in v2.0	v2.0: CostMetric must be SEALED before use; sealing records a SHA-256 of the scales and weights, reported with every result.

L18 Accessibility Engine

Purpose	Integrate timelike and null geodesics from a declared observer domain into the CTC region.
Scientific gate	Mathematical existence, reachability, survivability and ability to return to an external past event are distinct claim levels.
Status in v2.0	v2.0: the four levels are separate fields; the conserved norm drift of every integration is reported as an independent accuracy check.

L19 Cross-Metric Meta-analysis

Purpose	Compare causal boundaries, invariant precursors, topology, horizons, energy requirements and quantum status across families.
Scientific gate	Universality is inferred only after predeclared cross-family tests and independent negative controls.
Status in v2.0	v2.0: features built only from what other layers compute; train/held-out split; a leakage check that flags any perfectly separating feature; governance caps the label at CANDIDATE_PRECURSOR.

6. CTC Detection Algorithms

6.1 ALG-CTC-001: compact-generator witness search

Purpose: find exact or numerically robust CTC witnesses generated by registered compact directions. The algorithm is intentionally one-sided: success proves a CTC in the declared quotient sector; failure cannot prove chronology. The seven gates below are the v2.0 addition; v1.0 performed only steps 3 and 5.

#	Gate	What is checked	Failure action
G1	model has a local metric and a declared compact generator	kind is LOCAL_METRIC or QUOTIENT_GLOBAL	SECTOR_UNRESOLVED; route to a global predicate
G2	domain	every registered DomainConstraint at the test point	SECTOR_UNRESOLVED; a causal claim outside the physical domain is rejected
G3	signature	Sylvester sign counts equal the declared (1,0,n-1)	SECTOR_UNRESOLVED
G4	Killing	$L_{\xi}g = 0$ for every declared generator	SECTOR_UNRESOLVED; the symmetry-orbit argument does not apply
G5	closure	the winding orbit closes under the registered identification or a word in Γ	SECTOR_UNRESOLVED
G6	time orientation	the orbit tangent is timelike and the future direction cannot rotate along a Killing orbit	SECTOR_UNRESOLVED; a reversing loop is not a CTC
G7	uncertainty	sign of q against a composed error budget, with 50-digit escalation inside the band	SECTOR_NULL (marginal) rather than a spurious CTC

For a single azimuthal generator this reduces to the sign of $g_{\phi\phi}$. CTC-FA deliberately does not mistake this convenient sufficient condition for a universal CTC theorem.

6.2 ALG-CTC-003: deck-orbit search (new in v2.0)

Let (M, η) be a **flat** covering spacetime and Γ a group of isometries acting freely. For p in M and g in Γ , the straight segment from p to $g(p)$ is a geodesic of M and projects to a closed curve in M/Γ . Because the metric is constant, its causal character is *exactly*

$$q(p, g) = \text{eta}(g(p) - p, g(p) - p)$$

This is an exact statement, not a sampled estimate. Two gates guard it: flatness of the covering is verified symbolically (Riemann = 0), and the deck element must be orthochronous, since otherwise the projected loop reverses the future direction. Words are scanned in order of increasing length so the reported witness is the *shortest* chronology-violating loop, not merely the most negative one.

Model	Exact closed form for the winding-n loop	Verified
misner_covering	$q = 2(\cosh nb - 1)(t^2 - x^2)$	agrees with the independent Misner-chart computation to 1×10^{-12} at every tested point
grant_space	$q = 2(\cosh nb - 1)(t^2 - x^2) + n^2 \alpha^2$	polarised hypersurfaces recovered to 2×10^{-14}
minkowski_time_perio dic	$q = -n^2 P^2$	exact

6.3 ALG-CTC-002: general closed-loop search

$$\text{gamma}(\text{lambda}) = c + \text{sum_k} [A_k \cos(2 \text{ pi } k \text{ lambda}) + B_k \sin(2 \text{ pi } k \text{ lambda})] + \text{quotient winding}$$

v1.0 minimised a log-sum-exp *smooth* maximum of $g(\dot{\gamma}, \dot{\gamma})$ and reported that smoothed value as the maximum norm. A smooth maximum can be negative while the true maximum is positive, so the reported number was not the quantity the criterion needs. v2.0 keeps the smoothed objective for the *search* and adds a separate *validation* stage that recomputes the exact maximum on a refined grid,

repeats at doubled resolution, tests closure on the quotient, tests time orientation, rejects degenerate tangents, and rejects runaway curves whose coordinate extent exceeds a declared limit.

Cross-validation between two independent engines

The reported validation margin is $\max g(u,u)/|u|^2$, which for a pure symmetry orbit reduces exactly to the L5 compact-orbit margin. Started from random Fourier coefficients, the L6 optimiser independently reproduces $-2.055335e+01$ (Gödel), $-1.200000e+01$ (van Stockum) and $-1.000000e+00$ (periodic Minkowski) — the same numbers the L5 detector produces by a completely different route.

6.4 ALG-CAUS-001: temporal-function certificate

$$C_{\text{tau}}(x) = -g^{\{\mu \text{ nu}\}} d_{\mu} \text{ tau } d_{\text{nu}} \text{ tau}, \quad \text{certificate condition: } \inf_{\{x \text{ in } D\}} C_{\text{tau}}(x) > 0$$

Gate T0, checked before anything else: τ must be invariant under every deck generator and every declared coordinate period. A multivalued 'time coordinate' on a quotient is not a temporal function. This gate is what makes `ads4_periodic` fail while `ads4_universal_cover` succeeds on an identical local metric.

The strict inequality is then proved symbolically where `sympy` can, and otherwise by interval branch-and-bound on a compact box that is reported with the certificate. A symbolic proof yields GLOBAL scope; a box proof yields DECLARED_DOMAIN scope and says so. Point sampling never establishes the inequality.

6.5 ALG-BOUND-001: causal phase-boundary solver

$$s^* = \inf \{ s \mid C[\text{theta}(s)] \neq \text{empty} \}$$

Step	Action	v2.0 requirement
1	select an exact on-shell parameter family and a complete causal margin	the reduced causal margin μ , derived from g_{ab} ; the closed form is not on this path
2	coarse domain scan; detect a sign-changing bracket	excluded singular values are skipped
3	bracketed bisection to tolerance	the FINAL BRACKET is returned, not only the root
4	recompute both sides at higher precision	left and right margins reported
5	attempt analytic reconstruction	compared AFTERWARDS; never used to steer the search
6	domain and normalisation validity at the root	$-g(u,u) > 0$ required; a root where the reference direction goes null is REJECTED as an artefact

6.6 ALG-MIN-001: minimal CTC birth and destruction

$$\Delta_{+} = \inf J(\delta \text{ theta}) \text{ s.t. field equations + domain + CTC certificate}$$

$$\Delta_{-} = \inf J(\delta \text{ theta}) \text{ s.t. field equations + domain + stable-causality certificate}$$

$$J(\delta \text{ theta}) = \sum_i w_i (\delta \text{ theta}_i / \Theta_i)^2$$

The two problems are not logically symmetric. Birth is certified by one witness; destruction requires exclusion of *all* closed timelike curves, so `minimal_destruction` demands a certificate callback and will never accept 'the detector found nothing'. The scales Θ_i and weights w_i must be frozen before the run: an unsealed `CostMetric` raises `UnsealedCostError`.

6.7 ALG-ENERGY-001: exact energy-condition classifier

Route A classifies the frame stress tensor. When ηT is diagonalisable over the reals with exactly one timelike eigendirection the tensor is Hawking-Ellis Type I with eigenvalues $(-\rho, p_1, p_2, p_3)$ and the conditions have exact algebraic forms. Route B solves the trust-region subproblem

$$\min_{\{|n| = 1\}} [n^T A n + 2 b \cdot n] \quad \text{via} \quad \sum_i b_i^2 / (\lambda_i - \mu)^2 = 1, \quad \mu \leq \lambda_{\min}$$

exactly, including the hard case. Neither route samples. They are cross-checked by value in the stress eigenframe and by sign in the metric eigenframe; the worst value-level disagreement across the whole registry is 1.4×10^{-17} . DEC is cross-checked by sign only, because the alternating route uses a different normalisation of the null vectors and pretending otherwise would be a fake agreement.

7. Metric and Construction Registry

Thirty-three registered models: the twenty-two of v1.0 with their numbering preserved, plus eleven added in v2.0 to make specific structural claims testable. Every record is machine-readable and validated against schemas/metric.schema.json.

Model	Kind	Dim	Compact	Deck	Expected	Scope	Refs
ads4_periodic	Quotient Global	4	2	1	CTC_PROVED	GLOBAL	R26,R27
ads4_universal_cover	Local Metric	4	1	0	NO_CTC_CERTIFIED	GLOBAL	R26,R27
alcubierre	Local Metric	4	0	0	NO_CTC_CERTIFIED	DECLARED_DO_MAIN	R12
btz_rotating	Quotient Global	3	1	0	NO_CTC_CERTIFIED	DECLARED_DO_MAIN	R26,R27
de_sitter_static	Local Metric	4	1	0	NO_CTC_CERTIFIED	DECLARED_DO_MAIN	R26,R27
flrw_exponential	Local Metric	4	0	0	NO_CTC_CERTIFIED	GLOBAL	R26,R27
flrw_flat	Local Metric	4	0	0	NO_CTC_CERTIFIED	GLOBAL	R26,R27
godel	Local Metric	4	1	0	CTC_PROVED	DECLARED_DO_MAIN	R2,R6
godel_critical	Local Metric	4	1	0	UNRESOLVED	SECTOR	R2,R6
godel_type	Local Metric	4	1	0	UNRESOLVED	SECTOR	R2,R6
gott_two_strings	Composite Global	0	0	0	CTC_PROVED	REDUCED_MO_DEL	R8,R9
grant_space	Quotient Global	4	0	1	CTC_PROVED	DECLARED_DO_MAIN	R9,R11,R17
kerr	Local Metric	4	1	0	CTC_PROVED	DECLARED_DO_MAIN	R3,R26
kerr_extremal	Local Metric	4	1	0	CTC_PROVED	DECLARED_DO_MAIN	R3,R26
kerr_newman	Local Metric	4	1	0	CTC_PROVED	DECLARED_DO_MAIN	R26,R28
krasnikov	Local Metric	4	1	0	UNRESOLVED	SECTOR	R13
minkowski	Local Metric	4	0	0	NO_CTC_CERTIFIED	GLOBAL	R26,R28
minkowski_time_periodic	Quotient Global	4	1	1	CTC_PROVED	GLOBAL	R26,R27
misner	Quotient Global	4	1	1	CTC_PROVED	DECLARED_DO_MAIN	R9,R11,R17
misner_covering	Quotient Global	4	0	1	CTC_PROVED	DECLARED_DO_MAIN	R9,R11
morris_thorne	Local Metric	4	1	0	NO_CTC_CERTIFIED	DECLARED_DO_MAIN	R4,R5,R18
ori_2005	Quotient Global	4	1	0	CTC_PROVED	DECLARED_DO_MAIN	R14
ori_2007_core	Quotient Global	4	1	0	CTC_PROVED	DECLARED_DO_MAIN	R15,R17
ori_2007_full	External Exact Spec	0	0	0	CTC_PROVED	REDUCED_MO_DEL	R15
paired_warp	Composite Global	0	0	0	UNRESOLVED	SECTOR	R12,R13
pseudo_reissner_nordstrom	Quotient Global	4	1	0	CTC_PROVED	DECLARED_DO_MAIN	R15,R17
reissner_nordstrom	Local Metric	4	1	0	NO_CTC_CERTIFIED	DECLARED_DO_MAIN	R26,R28
schwarzschild	Local Metric	4	1	0	NO_CTC_CERTIFIED	DECLARED_DO_MAIN	R26,R28

Model	Kind	Dim	Compact	Deck	Expected	Scope	Refs
spinning_cosmic_string	Quotient Global	4	1	0	CTC_PROVED	DECLARED_DO MAIN	R8,R26
taub_nut	Local Metric	4	1	0	CTC_PROVED	SECTOR	R26,R28
time_shifted_wormhole	Composite Global	0	0	0	CTC_PROVED	REDUCED_MO DEL	R5,R18,R19
tipler_cylinder	External Exact Spec	0	0	0	CTC_PROVED	REDUCED_MO DEL	R7
van_stockum	Local Metric	4	1	0	CTC_PROVED	DECLARED_DO MAIN	R7

7.1 Models added in v2.0 and why

Model	Role	What it makes testable
ads4_universal_cover / ads4_periodic (M23/M24)	L3 gate pair	one identical local metric, opposite global verdicts. The executable proof that global causal structure is not a function of the local metric.
minkowski_time_periodic (M30)	L3 gate pair, flat	the same demonstration with zero curvature, so the effect cannot be attributed to geometry.
misner_covering (M31)	cross-representation check	Misner space as an explicit boost quotient of Minkowski, computed by a completely different engine and agreeing with the Misner chart pointwise.
grant_space (M27)	polarised quotient benchmark	a boost composed with a translation; the engine locates the polarised hypersurfaces $x_n^2 = n^2 \alpha^2 / (2(\cosh nb - 1))$ automatically and shows they accumulate on the chronology horizon.
godel_critical (M26)	marginal family control	the exact boundary member $m^2 = 4\omega^2$, which v1.0 conflated with the generic member in a single table row.
pseudo_reissner_nordstrom (M32)	two-parameter boundary	a charged Rieger-type core whose chronology-violating region is bounded by BOTH roots of $f(r)$, giving the continuation engine a two-parameter analytic target and a normalisation artefact to reject.
kerr_extremal (M05b)	limit control	the CTC sector survives $a \rightarrow M$, so it is not an artefact of subextremality.
de_sitter_static (M25)	positive-A control	a cosmological horizon that is not a chronology horizon.
btz_rotating (M29)	excision control	a quotient of AdS_3 that is NOT causality violating on its standard chart, refuting the inference 'quotient of AdS therefore CTC'.
flrw_exponential (M02b)	numeric companion	a numerically evaluable FLRW, since the general one carries an undefined scale factor.

7.2 Detailed profiles

The complete machine-readable record for every model is in `registry/metrics/<model_id>.json` and Appendix F. The profiles below give the fields a reader needs to reproduce the causal result by hand.

M01 Minkowski spacetime

Class / kind	negative_control — LOCAL_METRIC
Coordinates	t, x, y, z
Parameters	none
Metric	$g_{00} = -1; g_{11} = 1; g_{22} = 1; g_{33} = 1$
Identifications	none
Deck group	trivial
Matter / theory	vacuum (GR), $\Lambda = 0$
Domain	unrestricted
Expected status	NO_CTC_CERTIFIED at GLOBAL scope
CTC-FA role and notes	Primary negative control for geometry, temporal-function certification and tensor residual tests.
References	R26, R28

M03 Schwarzschild

Class / kind	negative_control — LOCAL_METRIC
Coordinates	t, r, theta, phi
Parameters	M
Metric	$g_{00} = 2*M/r - 1; g_{11} = 1/(-2*M/r + 1); g_{22} = r^2; g_{33} = r^2*\sin(\theta)^2$
Identifications	$\phi \sim \phi + 2*\pi$
Deck group	trivial
Matter / theory	vacuum (GR), $\Lambda = 0$
Domain	exterior: $-2*M + r > 0$; polar_regular: $\sin(\theta) > 0$
Expected status	NO_CTC_CERTIFIED at DECLARED_DOMAIN scope
CTC-FA role and notes	$r = 2M$ is an event horizon, NOT a chronology horizon. Separates horizon detection from causality violation.
References	R26, R28

M04 Reissner-Nordstrom

Class / kind	negative_control — LOCAL_METRIC
Coordinates	t, r, theta, phi
Parameters	M, Q
Metric	$g_{00} = 2*M/r - Q^2/r^2 - 1; g_{11} = 1/(-2*M/r + Q^2/r^2 + 1); g_{22} = r^2; g_{33} = r^2*\sin(\theta)^2$
Identifications	$\phi \sim \phi + 2*\pi$
Deck group	trivial
Matter / theory	electrovacuum (Einstein-Maxwell), $\Lambda = 0$
Domain	outside_outer_horizon: $-M + r - \sqrt{M^2 - Q^2} > 0$; subextremal: $M^2 - Q^2 > 0$; polar_regular: $\sin(\theta) > 0$
Expected status	NO_CTC_CERTIFIED at DECLARED_DOMAIN scope
CTC-FA role and notes	Critical negative control against the logical error 'Cauchy horizon implies CTC'. $r_{pm} = M \pm \sqrt{M^2 - Q^2}$.
References	R26, R28

M05 Kerr (Boyer-Lindquist)

Class / kind	rotation_benchmark — LOCAL_METRIC
Coordinates	t, r, theta, phi
Parameters	M, a
Metric	$g_{00} = 2Mr/(a^2\cos^2(\theta) + r^2) - 1$; $g_{03} = -2Ma\sin(\theta)/(a^2\cos^2(\theta) + r^2)$; $g_{11} = (a^2\cos^2(\theta) + r^2)/(-2Mr + a^2 + r^2)$; $g_{22} = a^2\cos^2(\theta) + r^2$; $g_{33} = (2Ma^2r\sin(\theta)/(a^2\cos^2(\theta) + r^2) + a^2 + r^2)\sin^2(\theta)$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	vacuum away from the ring singularity (GR), $\Lambda = 0$
Domain	nondegenerate_Sigma: $a^2\cos^2(\theta) + r^2 > 0$; off_horizon: $(-2Mr + a^2 + r^2)^2 > 0$; polar_regular: $\sin(\theta) > 0$
Expected status	CTC_PROVED at DECLARED_DOMAIN scope
CTC-FA role and notes	In part of the analytically extended $r < 0$ region $g_{\phi\phi} < 0$, so constant-(t,r,theta) azimuthal loops are timelike. Mathematical existence; operational accessibility and stability of that region are separate claims.
References	R3, R26

M06 Kerr-Newman

Class / kind	multiparameter_benchmark — LOCAL_METRIC
Coordinates	t, r, theta, phi
Parameters	M, a, Q
Metric	$g_{00} = (2Mr - Q^2)/(a^2\cos^2(\theta) + r^2) - 1$; $g_{03} = -a(2Mr - Q^2)\sin(\theta)/(a^2\cos^2(\theta) + r^2)$; $g_{11} = (a^2\cos^2(\theta) + r^2)/(-2Mr + Q^2 + a^2 + r^2)$; $g_{22} = a^2\cos^2(\theta) + r^2$; $g_{33} = (a^2(2Mr - Q^2)\sin^2(\theta)/(a^2\cos^2(\theta) + r^2) + a^2 + r^2)\sin^2(\theta)$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	electrovacuum (Einstein-Maxwell), $\Lambda = 0$
Domain	nondegenerate_Sigma: $a^2\cos^2(\theta) + r^2 > 0$; polar_regular: $\sin(\theta) > 0$
Expected status	CTC_PROVED at DECLARED_DOMAIN scope
CTC-FA role and notes	Tests multi-parameter continuation and the separation of charge, rotation and causal pathology. Horizons exist when $M^2 \geq a^2 + Q^2$.
References	R26, R28

M07 Godel-type hyperbolic family (Rebouças-Tiomno)

Class / kind	phase_boundary_benchmark — LOCAL_METRIC
Coordinates	t, r, phi, z
Parameters	omega, m
Metric	$g_{00} = -1$; $g_{02} = -4\omega\sinh(mr/2)^2/m^2$; $g_{11} = 1$; $g_{22} = \sinh(mr)^2/m^2 - 16\omega^2\sinh(mr/2)^4/m^4$; $g_{33} = 1$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	source depends on the selected theory/matter model (GR or declared extension), $\Lambda = 0$
Domain	radial: $r > 0$; rotation: $\omega > 0$; scale: $m > 0$
Expected status	UNRESOLVED at SECTOR scope
CTC-FA role and notes	$ds^2 = -[dt + H d\phi]^2 + D^2 d\phi^2 + dr^2 + dz^2$ with $H = (4\omega/m^2)\sinh^2(mr/2)$, $D = \sinh(mr)/m$. Circular CTCs exist for $m^2 < 4\omega^2$; the family is causal for $m^2 \geq 4\omega^2$. Primary analytic phase-boundary benchmark.
References	R2, R6

M08 Godel solution ($m^2 = 2 \omega^2$)

Class / kind	exact_ctc_control — LOCAL_METRIC
Coordinates	t, r, phi, z
Parameters	omega
Metric	$g_{00} = -1$; $g_{02} = -2 \sinh(\sqrt{2} \omega r/2)^2 / \omega$; $g_{11} = 1$; $g_{22} = -(4 \sinh(\sqrt{2} \omega r/2)^4 - \sinh(\sqrt{2} \omega r)^2 / 2) / \omega^2$; $g_{33} = 1$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	dust + cosmological constant (original solution) (GR), $\Lambda = 0$
Domain	radial: $r > 0$; rotation: $\omega > 0$
Expected status	CTC_PROVED at DECLARED_DOMAIN scope
CTC-FA role and notes	Canonical exact CTC control. Finite critical radius $\sinh^2(\sqrt{2} \omega r_c / 2) = 1$, i.e. $r_c = 2 \operatorname{arcsinh}(1) / (\sqrt{2} \omega)$.
References	R2, R6

M26 Critical Godel-type member ($m^2 = 4 \omega^2$)

Class / kind	marginal_family_control — LOCAL_METRIC
Coordinates	t, r, phi, z
Parameters	omega
Metric	$g_{00} = -1$; $g_{02} = -\sinh(\omega r)^2 / \omega$; $g_{11} = 1$; $g_{22} = \sinh(\omega r)^2 / \omega^2$; $g_{33} = 1$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	model-dependent (GR or extension), $\Lambda = 0$
Domain	radial: $r > 0$; rotation: $\omega > 0$
Expected status	UNRESOLVED at SECTOR scope
CTC-FA role and notes	Exactly on the causal phase boundary: the critical radius diverges and $g_{\phi\phi} = \sinh^2(2 \omega r) / (4 \omega^2) * (1 - \tanh^2) > 0$ for all finite r. The azimuthal sector is spacelike everywhere, but this is a SECTOR result only.
References	R2, R6

M09 van Stockum rotating dust interior

Class / kind	matter_supported_ctc — LOCAL_METRIC
Coordinates	t, r, phi, z
Parameters	a
Metric	$g_{00} = -1$; $g_{02} = a r^2$; $g_{11} = \exp(-a^2 r^2)$; $g_{22} = r^2(-a^2 r^2 + 1)$; $g_{33} = \exp(-a^2 r^2)$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	rigidly rotating dust (GR), $\Lambda = 0$
Domain	radial: $r > 0$; rotation: $a > 0$
Expected status	CTC_PROVED at DECLARED_DOMAIN scope
CTC-FA role and notes	Azimuthal loops become timelike when $r^2(1 - a^2 r^2) < 0$, i.e. $a r > 1$. Physical matching to an exterior at some radius R is a separate constraint: the CTC region is present only if $a R > 1$. v1.0 stored this metric asymmetrically (defect D-02).
References	R7

M11 Lorentzian Taub-NUT

Class / kind	gravitomagnetic_pathology — LOCAL_METRIC
Coordinates	t, r, theta, phi
Parameters	M, ell
Metric	$g_{00} = -(-2M*r - ell^2 + r^2)/(ell^2 + r^2)$; $g_{03} = -2*ell*(-2M*r - ell^2 + r^2)*cos(theta)/(ell^2 + r^2)$; $g_{11} = (ell^2 + r^2)/(-2M*r - ell^2 + r^2)$; $g_{22} = ell^2 + r^2$; $g_{33} = -4*ell^2*(-2M*r - ell^2 + r^2)*cos(theta)^2/(ell^2 + r^2) + (ell^2 + r^2)*sin(theta)^2$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	vacuum (GR), $\Lambda = 0$
Domain	polar_regular: $\sin(\theta) > 0$
Expected status	CTC_PROVED at SECTOR scope
CTC-FA role and notes	Azimuthal CTC sectors occur where $-f(2l\cos\theta)^2 + (r^2 + l^2)\sin^2\theta < 0$, i.e. near the axis in the region $f > 0$. The local phi-loop detector does NOT resolve the global identification choices; the verdict is deliberately kept at SECTOR scope.
References	R26, R28

M12 Misner space (quotient chart)

Class / kind	chronology_horizon_benchmark — QUOTIENT_GLOBAL
Coordinates	T, psi, y, z
Parameters	none
Metric	$g_{01} = -1$; $g_{11} = -T$; $g_{22} = 1$; $g_{33} = 1$
Identifications	$\psi \sim \psi + 2\pi$
Deck group	Z generated by $\psi \rightarrow \psi + 2\pi$
Matter / theory	flat vacuum locally (GR), $\Lambda = 0$
Domain	unrestricted
Expected status	CTC_PROVED at DECLARED_DOMAIN scope
CTC-FA role and notes	$\det g = -1$ everywhere, so the chart is regular across $T = 0$. $g_{\psi\psi} = -T$: compact psi loops are spacelike for $T < 0$, null at $T = 0$ (the chronology horizon) and timelike for $T > 0$. Cleanest benchmark separating local flatness from global causal pathology.
References	R9, R11, R17

M31 Misner space as a boost quotient of Minkowski

Class / kind	cross_representation_check — QUOTIENT_GLOBAL
Coordinates	t, x, y, z
Parameters	b
Metric	$g_{00} = -1$; $g_{11} = 1$; $g_{22} = 1$; $g_{33} = 1$
Identifications	none
Deck group	Z generated by a boost
Matter / theory	vacuum (GR), $\Lambda = 0$
Domain	off_axis: $t^2 + x^2 > 0$
Expected status	CTC_PROVED at DECLARED_DOMAIN scope
CTC-FA role and notes	For the boost of rapidity b the invariant interval of the closed geodesic through p and $\text{boost}^n(p)$ is exactly $2(\cosh(nb) - 1)(t^2 - x^2)$. It is negative -- a CTC -- precisely where $x^2 > t^2$, which is the image of the $T > 0$ region of the Misner chart. This reproduces M12 from a completely different representation.
References	R9, R11

M27 Grant space (boost + translation quotient)

Class / kind	polarised_quotient_benchmark — QUOTIENT_GLOBAL
Coordinates	t, x, y, z
Parameters	b, alpha
Metric	$g_{00} = -1; g_{11} = 1; g_{22} = 1; g_{33} = 1$
Identifications	none
Deck group	Z generated by a screw isometry
Matter / theory	vacuum (GR), $\Lambda = 0$
Domain	unrestricted
Expected status	CTC_PROVED at DECLARED_DOMAIN scope
CTC-FA role and notes	The invariant interval of the closed geodesic through p and $g^n(p)$ is exactly $2(\cosh(n b) - 1)(t^2 - x^2) + n^2 \alpha^2$. The winding-n loop is null on the POLARISED HYPERSURFACE $x^2 - t^2 = n^2 \alpha^2 / (2(\cosh(n b) - 1))$ and timelike outside it. Because that radius decreases monotonically to zero as n grows, the union over all n is the whole region $x^2 > t^2$: alpha > 0 does NOT move the chronology horizon, which stays at $x^2 = t^2$ exactly as in Misner space. What alpha creates is the discrete family of polarised hypersurfaces that accumulates on the horizon, and those are where the renormalised stress tensor of a quantum field is singular. CTC-FA v2.0 locates them automatically from the deck group; see quotient_ctc.polarized_hypersurfaces.
References	R9, R11, R17

M13 Spinning cosmic string

Class / kind	conical_quotient_benchmark — QUOTIENT_GLOBAL
Coordinates	t, r, phi, z
Parameters	J, alpha
Metric	$g_{00} = -1; g_{02} = -4J; g_{11} = 1; g_{22} = -16J^2 + \alpha^2 r^2; g_{33} = 1$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	distributional string source on the axis (GR with a distributional source), $\Lambda = 0$
Domain	radial: $r > 0$; cone_angle: $\alpha > 0$; spin: $J > 0$
Expected status	CTC_PROVED at DECLARED_DOMAIN scope
CTC-FA role and notes	$ds^2 = -(dt + 4J d\phi)^2 + dr^2 + \alpha^2 r^2 d\phi^2 + dz^2$. $g_{\phi\phi} = \alpha^2 r^2 - 16J^2 < 0$ gives azimuthal CTCs inside $r < 4J/\alpha$ in the idealised model.
References	R8, R26

M14 Gott two-moving-cosmic-string construction

Class / kind	global_holonomy_benchmark — COMPOSITE_GLOBAL
Coordinates	—
Parameters	none
Metric	no canonical local tensor (external/composite specification)
Identifications	none
Deck group	trivial
Matter / theory	unspecified (GR with distributional sources), $\Lambda = 0$
Domain	unrestricted
Expected status	CTC_PROVED at REDUCED_MODEL scope
CTC-FA role and notes	Tests global holonomy criteria and prevents local-metric overreach. mu here is $G \mu / c^2$, so the deficit angle is $8 \pi \mu$.
References	R8, R9

M15 Morris-Thorne zero-redshift wormhole

Class / kind	topology_vs_chronology_control — LOCAL_METRIC
Coordinates	t, r, theta, phi
Parameters	r0
Metric	$g_{00} = -1$; $g_{11} = 1/(1 - r_0^2/r^2)$; $g_{22} = r^2$; $g_{33} = r^2 \sin(\theta)^2$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	exotic: NEC is violated near the throat under standard GR assumptions (GR), $\Lambda = 0$
Domain	outside_throat: $r - r_0 > 0$; polar_regular: $\sin(\theta) > 0$
Expected status	NO_CTC_CERTIFIED at DECLARED_DOMAIN scope
CTC-FA role and notes	$\Phi = 0$ and $b(r) = r_0^2/r$. A static traversable wormhole is NOT automatically a time machine: $g^{tt} = -1$ so t is an exact temporal function on this chart. Throat $b(r_0) = r_0$, flare-out $b'(r_0) = -1 < 1$.
References	R4, R5, R18

M16 Time-shifted traversable wormhole (reduced route-closure model)

Class / kind	reduced_predicate_benchmark — COMPOSITE_GLOBAL
Coordinates	—
Parameters	none
Metric	no canonical local tensor (external/composite specification)
Identifications	none
Deck group	trivial
Matter / theory	unspecified (GR (assumed traversable wormhole)), $\Lambda = 0$
Domain	unrestricted
Expected status	CTC_PROVED at REDUCED_MODEL scope
CTC-FA role and notes	Explicitly labelled reduced criterion, used for software regression and parameter continuation. Requires: external travel time t_{ext} between mouths, wormhole traversal time t_{wh} , accumulated mouth time shift Δt , all in one declared synchronisation.
References	R5, R18, R19

M17 Ori 2005 compact vacuum-core chart

Class / kind	energy_condition_counterexample — QUOTIENT_GLOBAL
Coordinates	T, x, y, z
Parameters	L
Metric	$g_{03} = -1$; $g_{11} = 1$; $g_{22} = 1$; $g_{33} = -T + x^2 - y^2$
Identifications	$z \sim z + L$
Deck group	trivial
Matter / theory	vacuum core; surrounding matter in the full Ori model satisfies WEC, DEC and SEC (GR), $\Lambda = 0$
Domain	unrestricted
Expected status	CTC_PROVED at DECLARED_DOMAIN scope
CTC-FA role and notes	$ds^2 = dx^2 + dy^2 - 2 dT dz + [F(x,y) - T] dz^2$ with the harmonic benchmark $F = x^2 - y^2$ (Laplacian zero, so the core is vacuum). At fixed (x,y) the compact z direction changes from spacelike to timelike when $F - T$ crosses zero. This is the decisive counterexample to any universal rule 'CTC implies NEC violation'.
References	R14

M18 Ori 2007 periodic pseudo-Schwarzschild core

Class / kind	constructibility_benchmark — QUOTIENT_GLOBAL
Coordinates	v, r, theta, phi
Parameters	mu, P
Metric	$g_{00} = -2\mu/r + 1$; $g_{01} = 1$; $g_{22} = r^2$; $g_{33} = r^2 \sinh(\theta)^2$
Identifications	$v \sim v + P$
Deck group	trivial
Matter / theory	vacuum core (GR), $\Lambda = 0$
Domain	radial: $r > 0$; hyperbolic_regular: $\sinh(\theta) > 0$
Expected status	CTC_PROVED at DECLARED_DOMAIN scope
CTC-FA role and notes	$ds^2 = (1 - 2\mu/r) dv^2 + 2 dv dr + r^2 (d\theta^2 + \sinh^2(\theta) d\phi^2)$. The 2-surfaces are hyperbolic, not spherical. Periodic v loops become timelike for $r < 2\mu$; $r = 2\mu$ is marginal. Benchmark for constructibility from regular initial data.
References	R15, R17

M32 Periodic pseudo-Reissner-Nordstrom core (Rieger-type)

Class / kind	two_parameter_boundary — QUOTIENT_GLOBAL
Coordinates	v, r, theta, phi
Parameters	mu, q, P
Metric	$g_{00} = -2\mu/r + q^2/r^2 + 1$; $g_{01} = 1$; $g_{22} = r^2$; $g_{33} = r^2 \sinh(\theta)^2$
Identifications	$v \sim v + P$
Deck group	trivial
Matter / theory	electrovacuum core (Einstein-Maxwell (pseudo-spherical)), $\Lambda = 0$
Domain	radial: $r > 0$; subextremal: $\mu^2 - q^2 > 0$; hyperbolic_regular: $\sinh(\theta) > 0$
Expected status	CTC_PROVED at DECLARED_DOMAIN scope
CTC-FA role and notes	Periodic v loops are timelike exactly between the roots $r_{pm} = \mu \pm \sqrt{\mu^2 - q^2}$ of $f(r)$. Two-parameter analytic boundary for the continuation engine, structurally parallel to Reissner-Nordstrom but with an opposite causal meaning: here the roots bound a genuine chronology-violating region.
References	R15, R17

M20 Alcubierre warp metric

Class / kind	superluminality_control — LOCAL_METRIC
Coordinates	t, x, y, z
Parameters	v, R, sigma
Metric	$g_{00} = v^2(-\tanh(\sigma(-R + \sqrt{y^2 + z^2 + (-t*v + x)^2})) + \tanh(\sigma(R + \sqrt{y^2 + z^2 + (-t*v + x)^2})))^2/(4*\tanh(R*\sigma)^2) - 1$; $g_{01} = -v(-\tanh(\sigma(-R + \sqrt{y^2 + z^2 + (-t*v + x)^2})) + \tanh(\sigma(R + \sqrt{y^2 + z^2 + (-t*v + x)^2}))))/(2*\tanh(R*\sigma))$; $g_{11} = 1$; $g_{22} = 1$; $g_{33} = 1$
Identifications	none
Deck group	trivial
Matter / theory	violates the classical energy conditions (GR), $\Lambda = 0$
Domain	unrestricted
Expected status	NO_CTC_CERTIFIED at DECLARED_DOMAIN scope
CTC-FA role and notes	$ds^2 = -dt^2 + [dx - v f(r_s) dt]^2 + dy^2 + dz^2$. A single bubble is not automatically a CTC: t is an exact temporal function on this chart ($g^{tt} = -1$), so causality violation requires an additional global or multi-bubble arrangement. Prevents equating effective superluminality with a time machine.
References	R12

M21 Krasnikov tube (Everett-Roman form)

Class / kind	shortcut_vs_loop_control — LOCAL_METRIC
Coordinates	t, x, rho, phi
Parameters	delta, epsilon, rho_max, D
Metric	$g_{00} = -1$; $g_{01} = (2 - \delta) \cdot (\tanh((-\rho + \rho_{\max})/\epsilon)/2 + 1/2) \cdot (\tanh((-D + \epsilon + x)/\epsilon)/2 + 1/2) \cdot (\tanh((-\rho + t - x)/\epsilon)/2 + 1/2)$; $g_{11} = -(2 - \delta) \cdot (\tanh((-\rho + \rho_{\max})/\epsilon)/2 + 1/2) \cdot (\tanh((-D + \epsilon + x)/\epsilon)/2 + 1/2) \cdot (\tanh((-\rho + t - x)/\epsilon)/2 + 1/2) + 1$; $g_{22} = 1$; $g_{33} = \rho^2$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	requires exotic stress-energy in standard GR (GR), $\Lambda = 0$
Domain	radial: $\rho > 0$
Expected status	UNRESOLVED at SECTOR scope
CTC-FA role and notes	$ds^2 = -dt^2 + 2(1-k) dt dx + k(2-k) dx^2 + d\rho^2 + \rho^2 d\phi^2$. The (t,x) block has determinant exactly -1 for every k, so the chart is Lorentzian everywhere. A single tube is not itself a time machine. The smooth step is a REGULATOR CHOICE, not part of an exact solution, so no causal boundary may be derived from this entry (defect D-06).
References	R13

M23 AdS4, universal cover (t in R)

Class / kind	L3_gate_pair — LOCAL_METRIC
Coordinates	t, r, theta, phi
Parameters	L
Metric	$g_{00} = -1 - r^2/L^2$; $g_{11} = 1/(1 + r^2/L^2)$; $g_{22} = r^2$; $g_{33} = r^2 \sin(\theta)^2$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	vacuum with a negative cosmological constant (GR with $\Lambda = -3/L^2$)
Domain	radial: $r > 0$; polar_regular: $\sin(\theta) > 0$
Expected status	NO_CTC_CERTIFIED at GLOBAL scope
CTC-FA role and notes	$g^{tt} = -1/(1 + r^2/L^2) < 0$ everywhere, so t is an exact global temporal function and the spacetime is stably causal. This is the CHRONAL member of the AdS pair.
References	R26, R27

M24 AdS4 hyperboloid, periodic time (t ~ t + 2 pi L)

Class / kind	L3_gate_pair — QUOTIENT_GLOBAL
Coordinates	t, r, theta, phi
Parameters	L
Metric	$g_{00} = -1 - r^2/L^2$; $g_{11} = 1/(1 + r^2/L^2)$; $g_{22} = r^2$; $g_{33} = r^2 \sin(\theta)^2$
Identifications	$t \sim t + 2\pi L$; $\phi \sim \phi + 2\pi$
Deck group	Z generated by $t \rightarrow t + 2\pi L$
Matter / theory	vacuum with a negative cosmological constant (GR with $\Lambda = -3/L^2$)
Domain	radial: $r > 0$; polar_regular: $\sin(\theta) > 0$
Expected status	CTC_PROVED at GLOBAL scope
CTC-FA role and notes	IDENTICAL local metric to ads4_universal_cover. The orbit of the Killing vector d/dt is closed and timelike everywhere ($g_{tt} = -(1 + r^2/L^2) < 0$), so every point lies on a CTC. The two models differ ONLY in the deck group, which is exactly the L3 scientific gate: local equality of metrics does not imply equality of global causal structure.
References	R26, R27

M30 Minkowski with periodic time ($t \sim t + P$)

Class / kind	L3_gate_pair — QUOTIENT_GLOBAL
Coordinates	t, x, y, z
Parameters	P
Metric	$g_{00} = -1; g_{11} = 1; g_{22} = 1; g_{33} = 1$
Identifications	$t \sim t + P$
Deck group	$\mathbb{Z}$ generated by $t \rightarrow t + P$
Matter / theory	vacuum (GR), $\Lambda = 0$
Domain	unrestricted
Expected status	CTC_PROVED at GLOBAL scope
CTC-FA role and notes	Local metric identical to Minkowski, which is the strongest no-CTC control in the registry. The only difference is the deck group. Any architecture that classifies these two the same has failed the L3 gate.
References	R26, R27

M25 de Sitter, static patch

Class / kind	positive_lambda_control — LOCAL_METRIC
Coordinates	t, r, θ, ϕ
Parameters	L
Metric	$g_{00} = -1 + r^2/L^2; g_{11} = 1/(1 - r^2/L^2); g_{22} = r^2; g_{33} = r^2 \sin^2(\theta)$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	vacuum with a positive cosmological constant (GR with $\Lambda = +3/L^2$), $\Lambda = 3/L^2$
Domain	inside_horizon: $L - r > 0$; radial: $r > 0$; polar_regular: $\sin(\theta) > 0$
Expected status	NO_CTC_CERTIFIED at DECLARED_DOMAIN scope
CTC-FA role and notes	Positive-Lambda control. $r = L$ is a cosmological horizon, not a chronology horizon; the static t is a temporal function only inside it, exactly as for Schwarzschild outside $r = 2M$.
References	R26, R27

M29 Rotating BTZ black hole (2+1)

Class / kind	excision_control — QUOTIENT_GLOBAL
Coordinates	t, r, ϕ
Parameters	$M_{\text{BTZ}}, J_{\text{BTZ}}, \ell$
Metric	$g_{00} = M_{\text{BTZ}} - r^2/\ell^2; g_{02} = -J_{\text{BTZ}}/2; g_{11} = 4\ell^2 r^2/(J_{\text{BTZ}}^2 \ell^2 - 4M_{\text{BTZ}} \ell^2 r^2 + 4r^4); g_{22} = r^2$
Identifications	$\phi \sim \phi + 2\pi$
Deck group	trivial
Matter / theory	vacuum with a negative cosmological constant (GR in 2+1 with $\Lambda = -1/\ell^2$), $\Lambda = -1/\ell^2$
Domain	radial: $r > 0$; outside_outer_horizon: $J_{\text{BTZ}}^2/(4r^2) - M_{\text{BTZ}} + r^2/\ell^2 > 0$; subextremal: $-J_{\text{BTZ}}^2 + M_{\text{BTZ}}^2 \ell^2 > 0$
Expected status	NO_CTC_CERTIFIED at DECLARED_DOMAIN scope
CTC-FA role and notes	On the standard chart $g_{\phi\phi} = r^2 > 0$, so the azimuthal sector is spacelike everywhere. The chronology-violating region of the maximally extended BTZ geometry sits at $r^2 < 0$ and is EXCISED BY DEFINITION of the spacetime -- exactly as the Kerr CTC region is retained by definition. Registered as a control for the claim 'a quotient of AdS is automatically causality violating', which is false.
References	R26, R27

M10 Tipler rotating cylinder construction

Class / kind	overreach_control — EXTERNAL_EXACT_SPEC
Coordinates	—
Parameters	none
Metric	no canonical local tensor (external/composite specification)
Identifications	none
Deck group	trivial
Matter / theory	unspecified (GR), $\Lambda = 0$
Domain	unrestricted
Expected status	CTC_PROVED at REDUCED_MODEL scope
CTC-FA role and notes	Tipler showed causality violation for the idealised, infinitely long, rapidly rotating cylinder. The exterior depends on matching parameters; CTC-FA v2.0 refuses to fabricate a single canonical local 4x4 tensor for it and uses van Stockum for the explicit local interior benchmark.
References	R7

M19 Ori 2007 full composite model

Class / kind	constructibility_target — EXTERNAL_EXACT_SPEC
Coordinates	—
Parameters	none
Metric	no canonical local tensor (external/composite specification)
Identifications	none
Deck group	trivial
Matter / theory	unspecified (GR), $\Lambda = 0$
Domain	unrestricted
Expected status	CTC_PROVED at REDUCED_MODEL scope
CTC-FA role and notes	Closed timelike curves form after evolution from a regular spacelike initial hypersurface in the published model, with WEC, DEC and SEC satisfied. Highest-value target for a full ADM/evolution reproduction; the matching and initial data are a separate implementation.
References	R15

M22 Paired warp / superluminal causal-loop family

Class / kind	unresolved_placeholder — COMPOSITE_GLOBAL
Coordinates	—
Parameters	none
Metric	no canonical local tensor (external/composite specification)
Identifications	none
Deck group	trivial
Matter / theory	unspecified (construction-specific), $\Lambda = 0$
Domain	unrestricted
Expected status	UNRESOLVED at SECTOR scope
CTC-FA role and notes	Deliberate UNRESOLVED placeholder that prohibits unsupported universality claims. A CTC verdict requires a specific published construction to be instantiated with its route geometry, velocities, synchronisation and matching data, and then validated.
References	R12, R13

8. Analytic Benchmarks and the Non-Circular Derivation

The hyperbolic Gödel-type family is the principal analytic unit test, because its circular chronology boundary can be derived directly and depends on a low-dimensional parameter pair. In the convention used here:

$$H(r) = (4 \omega / m^2) \sinh^2(m r / 2), \quad D(r) = \sinh(m r) / m, \quad g_{\text{phiphi}} = D(r)^2 - H(r)^2$$
$$\sinh^2(m r_c / 2) = [4 \omega^2 / m^2 - 1]^{(-1)}, \quad 0 < m^2 < 4 \omega^2 \quad m^2 = 4 \omega^2 \Rightarrow r_c \rightarrow \text{infinity}$$

These formulas are correct as v1.0 printed them and are preserved. What has changed is how the architecture *obtains* the boundary.

The v1.0 method, and why it proves nothing
The v1.0 kernel contained `godel_family_margin(m, omega) = m**2 - 4*omega*omega` and ran a root finder on it, publishing `m* = 1.999999999999987` in the self-test output as a regression result. Root-finding a hard-coded literature answer confirms only that the root finder works. The v1.0 risk register names this pattern exactly: FM-016, 'known literature threshold encoded into detector and later claimed rediscovered → circular validation'.

8.1 The v2.0 derivation chain

No closed form appears anywhere on the solver path. The chain is: the metric tensor → the reduced causal margin $\mu = g(\xi, \xi) / h(\xi) \rightarrow$ its infimum over a declared coordinate box → a sign change in the parameter → bracketed bisection → comparison with the closed form, afterwards. A test in continuous integration asserts that the strings `4*omega*omega` and `m**2 - 4` do not occur in the solver module's body.

The exact boundary $m = 2\omega$ is an *asymptotic* statement: the critical radius diverges as $m \rightarrow 2\omega$, so a finite coordinate box can only approach it. That makes the convergence table the evidence, and a single number no evidence at all.

Coordinate-box radius R	m*(R) derived from the metric	m*(R) – 2ω
5.0	1.999818243430470	1.818e-04
10.0	1.999999991755366	8.245e-09
15.0	1.999999999999626	3.744e-13
20.0	2.000000000000016	1.643e-14
25.0	2.000000000000016	1.643e-14
30.0	2.000000000000016	1.643e-14

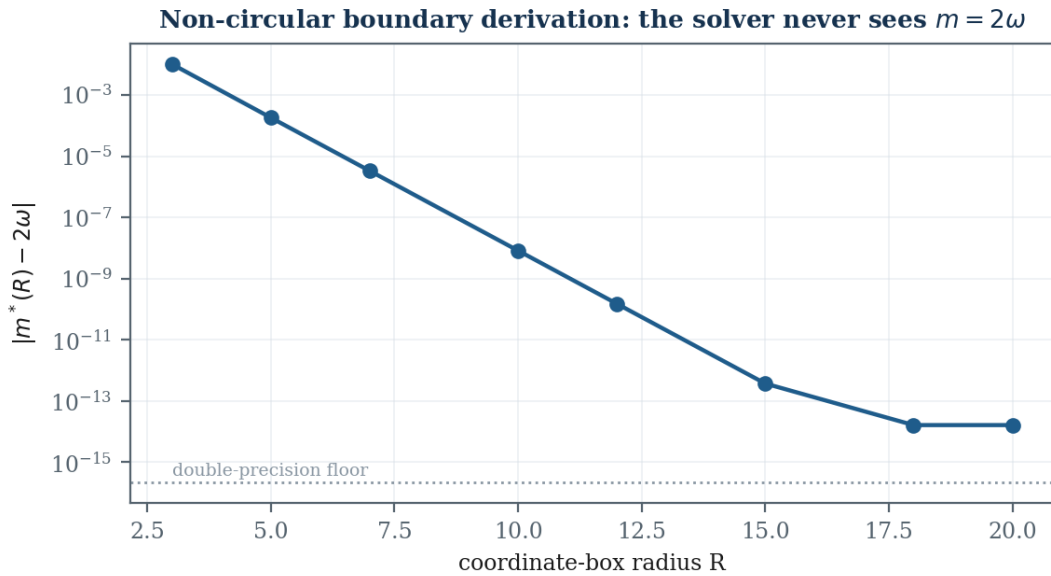


Figure 5. The boundary converges to 2ω exponentially in the box radius and saturates at the double-precision floor. The solver never receives the value 2.

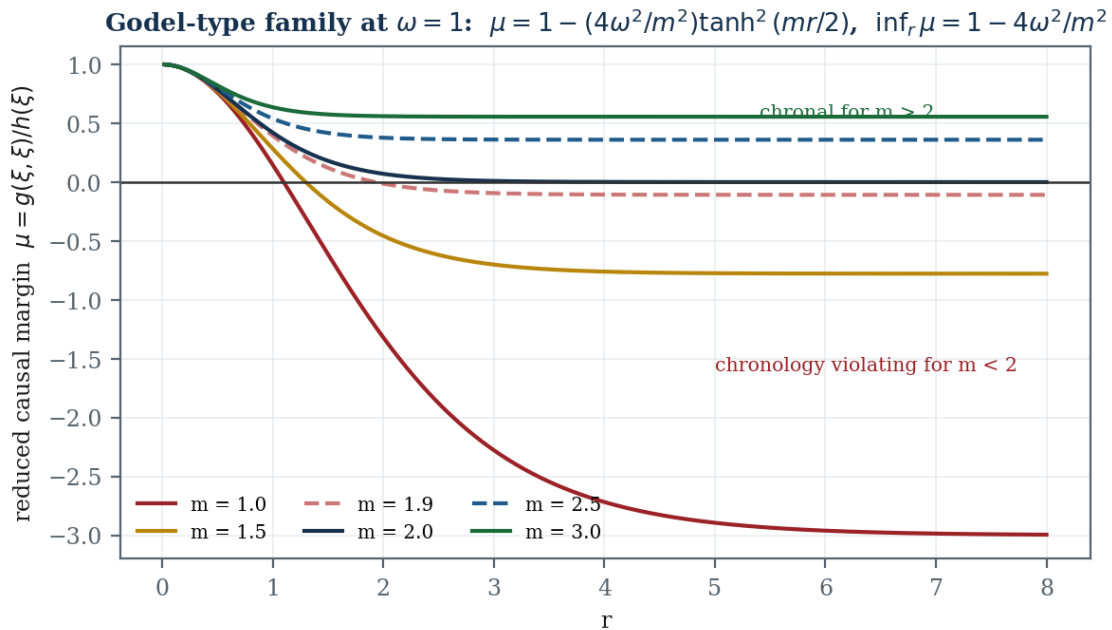


Figure 6. The reduced causal margin of the Gödel-type family, derived from the metric. Its infimum over r is $1 - 4\omega^2/m^2$, whose zero is the boundary.

8.2 Boundaries with exact closed forms

Six further boundaries have exact closed forms and are recovered to the double-precision floor. In every row the solver received only the model, the parameter and a bracket.

Model	Par	Derived value	Closed form	error	Bracket width	Domain
van_stockum	a	0.5000000000000001	$a \ r = 1$	1.33e-15	5.6e-14	OK
spinning_cosmic_string	alpha	4.0000000000000011	$\alpha \ r = 4J$	1.07e-14	9.2e-14	OK
ori_2007_core	r	2.0000000000000016	$r = 2 \ \mu$	1.64e-14	5.2e-14	OK
pseudo_reissner_nordstrom	r	1.866025403784420	$r_+ = \mu + \sqrt{\mu^2 - q^2}$	1.89e-14	7.6e-14	OK
pseudo_reissner_nordstrom	r	0.133974596215594	$r_- = \mu - \sqrt{\mu^2 - q^2}$	3.31e-14	6.6e-14	OK

Model	Par	Derived value	Closed form	error	Bracket width	Domain
misner	T	0.0000000000000000	$T = 0$	0.00e+00	0.0e+00	OK
ori_2005	T	0.0000000000000000	$T = F(x,y)$	0.00e+00	0.0e+00	OK
pseudo_reissner_nordstrom	r	0.118033988749909	reference null (ARTEFACT)	1.41e-14	9.7e-14	REJECTED

The artefact the engine rejects
A seventh sign change exists for the charged pseudo-RN core at $r = 0.118033988750$, close to the genuine inner boundary at $r_+ = 0.133974596216$. It is *not* a causal boundary: it is the locus where the timelike reference direction used to normalise the margin becomes null, $-g(u,u) = 2.3 \times 10^{-12}$ there. The engine evaluates that quantity at every root and marks the row REJECTED. Publishing it would have been FM-001, a coordinate artefact reported as a causal boundary.

9. Flat Quotients: Misner, Grant and the Polarised Hypersurfaces

Misner space, Grant space and the polarised Misner-type spacetimes that dominate the modern causal-classification literature are quotients of flat Minkowski space by an isometry that is not a coordinate translation. v1.0 had no engine for them: its Misner entry used an ad-hoc chart, and nothing in the data model could express a boost identification at all.

9.1 Two independent representations agreeing

The registry now carries Misner space twice: once as the quotient chart $ds^2 = -2 dT d\psi - T d\psi^2 + dy^2 + dz^2$ with ψ periodic, and once as Minkowski modulo a boost of rapidity b . The second is handled by the deck-orbit engine, which shares no code path with the compact-orbit detector. Their verdicts agree pointwise, and the deck-orbit values agree with the closed form $2(\cosh b - 1)(t^2 - x^2)$ to 1×10^{-12} .

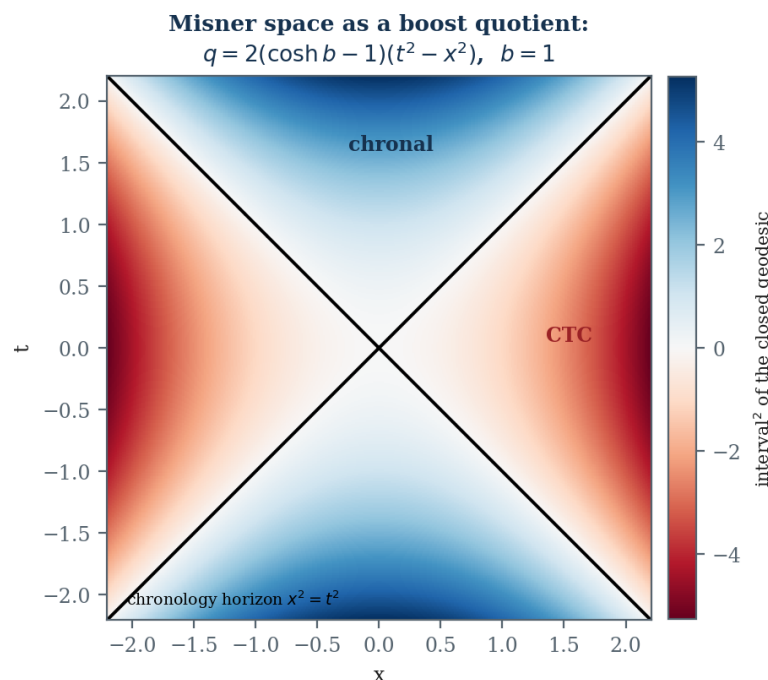


Figure 7. Causal character of the winding-1 closed geodesic of Misner space in the covering, computed by the deck-orbit engine. The chronology horizon $x^2 = t^2$ is the zero contour.

9.2 Grant space and a genuinely new computed result

Grant space is Minkowski quotiented by a boost of rapidity b in the (t, x) plane composed with a translation α in y . The engine derives, symbolically, that the winding- n closed geodesic has invariant interval

$$q_n = 2 (\cosh(n b) - 1) (t^2 - x^2) + n^2 \alpha^2$$

so the winding- n loop is null on the surface $x^2 - t^2 = n^2 \alpha^2 / (2(\cosh nb - 1))$. These are the **polarised hypersurfaces**. Because that radius decreases monotonically to zero as n grows, their union is the whole region $x^2 > t^2$: the translation α does *not* move the chronology horizon, which stays at $x^2 = t^2$ exactly as in Misner space. What α creates is the discrete family accumulating on the horizon — precisely where the renormalised stress tensor of a quantum field is singular. The engine locates them automatically, from the deck group alone, and they feed directly into the L13/L14 quantum layer as candidate RSET divergence loci.

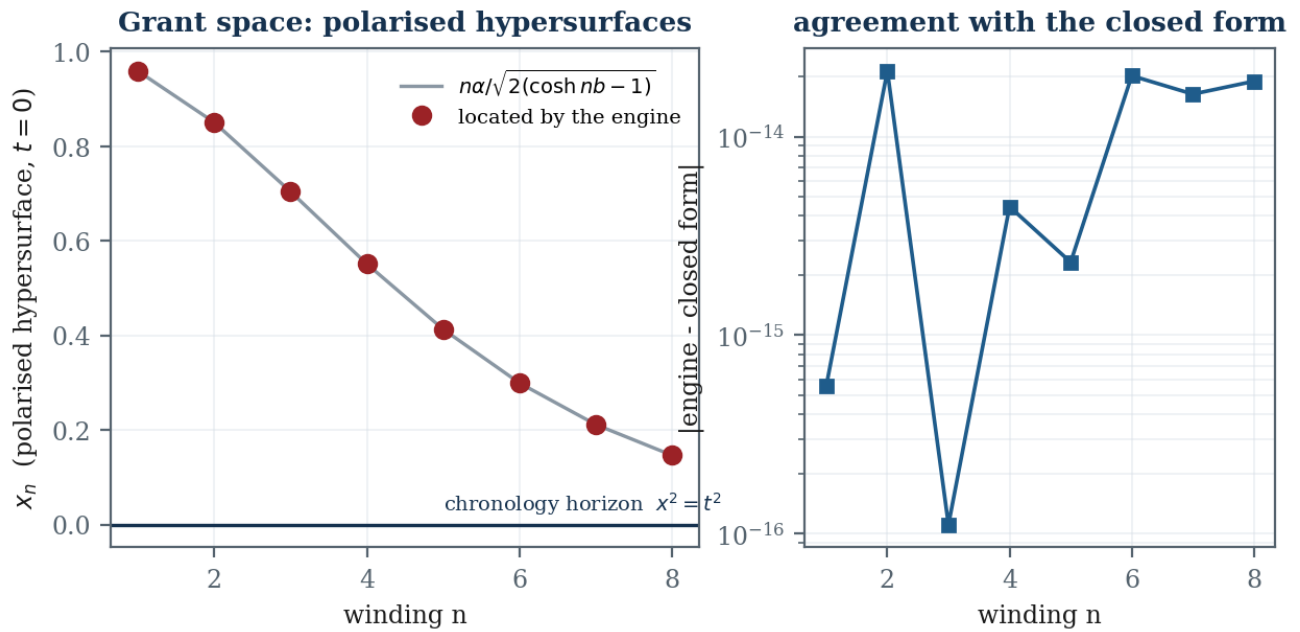


Figure 8. Polarised hypersurfaces of Grant space located by the deck-orbit engine, against the closed form. Agreement is at the 10^{-14} level. Locating these was not possible in v1.0 by any route.

Why this is the strongest evidence that the L3 layer is real

The polarised hypersurfaces are not a restatement of an input. They are a non-trivial consequence of the deck group that the architecture computes without being told the answer, they agree with the closed form to fourteen digits, and they are the object the semiclassical layer needs. An architecture that stores topology as prose cannot produce them at all.

10. Horizon Taxonomy and Causal Failure Modes

Object	Operational definition in CTC-FA	Forbidden shortcut
Killing horizon	null hypersurface generated by a Killing field; solved as $g(\xi,\xi) = 0$	not an event horizon without a global argument, and never a chronology horizon
Chart degeneracy	locus where $\det g = 0$ or the numerator of g^{rr} vanishes	not a singularity and not a horizon; test a curvature invariant first
Curvature singularity	pole of an invariant; the Kretschmann scalar $K = R_{abcd}R^{abcd}$	not a chronology horizon
Ergosurface	locus where the reference Killing direction stops being timelike	not a horizon and not a CTC boundary
Chronology horizon	boundary of the chronology-violating set, computed from the reduced causal margin	must be derived from the causal margin, never inferred from any other row of this table

v1.0 stated this taxonomy and implemented one solver (inverse_radial_horizon_candidates, for $g^{rr} = 0$), which cannot distinguish the five objects. In v2.0 each is computed separately and carries the inference it forbids, so a report cannot silently convert one into another.

10.1 The decisive controls

Model	Killing horizons	Curvature singularities	Chronology horizon	Reading
reissner_nordstrom	$r = M \pm \sqrt{(M^2 - Q^2)}$	$r = 0$	none	an inner Cauchy horizon with NO chronology violation. This is the primary control against the inference 'Cauchy horizon therefore CTC' (FM-006).
schwarzschild	$r = 2M$	$r = 0$	none	an event horizon is not a chronology horizon.
de_sitter_static	$r = \pm L$	none	none	a cosmological horizon is not a chronology horizon, and there is no singularity at all.
misner	$T = 0$	none	$T = 0$	the converse control: a chronology horizon with $\det g \equiv -1$ and no curvature singularity anywhere.
ori_2007_core	$r = 2\mu$	$r = 0$	$r = 2\mu$	here the Killing horizon and the chronology horizon coincide — which is a computed fact about this model, not a general rule.

One artefact filtered here as well
For Misner the reduced margin is $\mu = -T(T + 2)$, which vanishes at $T = 0$ and at $T = -2$. Only $T = 0$ is a chronology horizon; $T = -2$ is where the declared timelike reference direction goes null. The engine reports the first and records the second as rejected.

11. Energy Conditions, Exactly

v1.0 contracted the stress tensor with 256 Fibonacci-sphere directions and labelled the result diagnostic. Its own register recorded the consequence as FM-009, and its L9 gate stated that 'analytic/interval certification is required for formal PASS'. The architecture therefore could never issue a formal energy PASS. SEC and DEC were never computed at all, and ANEC consumed a caller-supplied list of contractions rather than integrating anything.

v2.0 removes sampling from the certification path. The stress tensor is obtained on shell from $G_{ab} + \Lambda g_{ab} = 8\pi T_{ab}$ at 30 significant digits — it is never assumed — and the four conditions are computed by two exact routes that must agree.

Model	HE type	ρ	NEC	WEC	SEC	DEC	Route A – Route B
minkowski	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
schwarzschild	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
kerr	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
kerr_extremal	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
reissner_nordstrom	I	+0.000016	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
kerr_newman	I	+0.025465	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
de_sitter_static	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
ads4_universal_cover	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
ads4_periodic	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
btz_rotating	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
misner	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
misner_covering	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
grant_space	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
minkowski_time_periodic	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
ori_2005	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
ori_2007_core	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
pseudo_reissner_nordstrom	I	-0.009947	VIOLATED	VIOLATED	VIOLATED	VIOLATED	0.00e+00
godel	I	+0.039789	SATISFIED	SATISFIED	SATISFIED	MARGINAL	0.00e+00
godel_type	I	+0.079577	SATISFIED	SATISFIED	SATISFIED	SATISFIED	0.00e+00
godel_critical	I	-0.039789	MARGINAL	VIOLATED	MARGINAL	VIOLATED	1.39e-17
van_stockum	I	+8.689565	SATISFIED	SATISFIED	SATISFIED	SATISFIED	0.00e+00
spinning_cosmic_string	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
taub_nut	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
morris_thorne	I	-0.002487	VIOLATED	VIOLATED	VIOLATED	VIOLATED	0.00e+00
alcubierre	IV	—	VIOLATED	VIOLATED	VIOLATED	VIOLATED	n/a
krasnikov	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00
flrw_exponential	I	+0.000000	MARGINAL	MARGINAL	MARGINAL	MARGINAL	0.00e+00

MARGINAL means the condition holds with equality within the uncertainty band — which is the correct answer for every vacuum solution, and for electrovacuum, where the traceless Maxwell tensor sits exactly on the NEC and DEC boundary. SATISFIED and VIOLATED are strict.

11.1 The counterexample that matters

Two registered spacetimes contain closed timelike curves and do not violate the null energy condition.

- **Ori 2005 (M17).** The compact vacuum core has $T_{ab} = 0$ *verified*, not assumed: the field-equation residual is exactly zero at the test point, because $F = x^2 - y^2$ is harmonic. The compact z direction

is timelike wherever $F - T < 0$.

- **van Stockum (M09).** Rigidly rotating dust with strictly positive energy density $\rho = +8.689565$ at the test point, satisfying NEC, WEC, SEC and DEC, and containing azimuthal closed timelike curves for $ar > 1$.
- **Gödel (M08).** Dust with a cosmological constant, $\rho = +0.039789$, NEC and SEC satisfied, DEC exactly marginal.

The inference 'CTC implies exotic matter' is false, and so is its converse

Morris-Thorne is the converse control: near the throat $\rho = -0.032734$ and the NEC margin is -6.546858×10^{-2} , a clear violation — and the spacetime contains no closed timelike curve, since $\tau = t$ is an exact temporal function on its chart with $C_\tau = 1$. Chronology violation and energy-condition violation are logically independent in both directions, and the registry contains the computation for each.

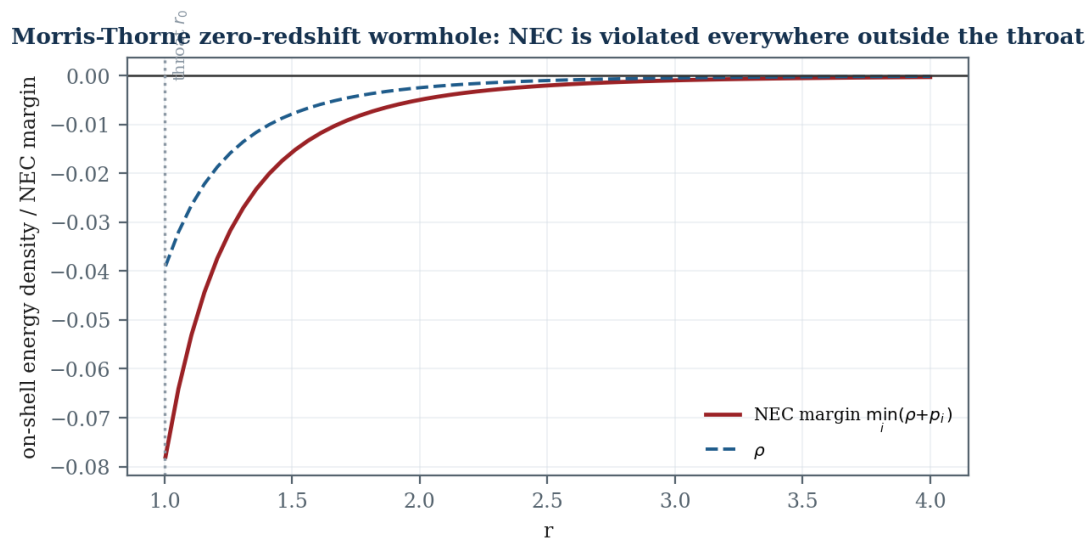


Figure 9. NEC margin and on-shell energy density of the Morris-Thorne wormhole against radius. The violation is not confined to the throat.

11.2 ANEC on a real geodesic

The averaged null energy condition is now integrated along a null geodesic produced by the L18 integrator, with the conserved norm $g(k,k)$ monitored as an independent accuracy check. For Morris-Thorne with $r_0 = 1$ starting at $r = 1.2$, the integral over affine range $[0, 3]$ is $-9.938059 \times 10^{-3} \pm 1.8 \times 10^{-8}$ with a norm drift of 1.4×10^{-10} . The status is REFUTED_ON_SEGMENT: a finite segment can refute ANEC but can never certify it, because ANEC is a statement about a complete geodesic. The report says so rather than rounding the distinction away.

12. Quantum and Semiclassical Admissibility

12.1 Chronology protection is a conjecture

Hawking presented strong arguments that compactly generated chronology horizons encounter severe classical and semiclassical obstacles, and that backreaction may prevent CTC formation [R9]. CTC-FA uses the label `CHRONOLOGY_PROTECTION_SUPPORTED_IN_MODEL`, or `UNRESOLVED`. The label `CHRONOLOGY_PROTECTION_PROVED` does not exist in the codebase, and a regression test asserts that it never appears in any theorem's permitted-label field.

12.2 The Kay-Radzikowski-Wald gate

For compactly generated Cauchy horizons, KRW prove a breakdown of the standard QFT construction at certain base points: Hadamard two-point functions become singular in the relevant sense and quantities such as the renormalised stress tensor become ill defined or singular there [R11]. v2.0 records the theorem's prerequisites as executable checks and emits `SEMICLASSICAL_QFT_BREAKDOWN` only where they are satisfied.

Model	Chronology horizon present	Compactly generated	KRW applicable
misner	yes	yes	YES
misner_covering	yes	yes	YES
grant_space	yes	yes	YES
ads4_periodic	no — the whole spacetime is chronology violating, there is no horizon	no	no
godel	no	no	no
schwarzschild	no	no	no

The ads4_periodic row is the useful one: it is entirely chronology violating and yet KRW does not apply, because there is no horizon for the theorem to be about. Generalising KRW to it would be failure mode FM-011.

12.3 No stress tensor without a specification

$\langle T_{\mu\nu} \rangle_{\text{ren}}$ is a functional of the field content, the state, the boundary conditions and the renormalisation prescription. A metric does not determine it. v1.0 stated this and modelled it as a dataclass of four literal 'UNRESOLVED' strings. In v2.0 the assessment refuses to produce anything without a complete QuantumSpecification and names the missing fields.

Required field	Why it is required
field content, spin, mass, couplings	the RSET differs between a minimally and a conformally coupled scalar, and between a scalar and a spinor
state, state class	Hadamard versus non-Hadamard changes whether the RSET is defined at all
boundary conditions	on a quotient the image sum depends on them
renormalisation prescription	point-splitting, zeta function and dimensional regularisation differ by finite local terms
sampling function, trajectory, dimension	a quantum energy inequality bound is meaningless without all three; applying one outside its assumptions is FM-012

The QEI registry currently holds one rule, the Ford-Roman bound $-3/(32\pi^2\tau^4)$ for a massless minimally coupled scalar in 4D Minkowski with a Lorentzian sampling function on a static inertial trajectory. It fires only when every one of those five keys matches; a Dirac spinor on an accelerated trajectory with Gaussian sampling returns `applicable = False` and an explanation, not a number.

12.4 Where the topology layer feeds the quantum layer

For Grant space with $b = 1$ and $\alpha = 1$ the engine returns the polarised hypersurfaces at $x = 0.959517375667, 0.850918128239, 0.704463660893, 0.551441129544, 0.413209174638$ and

0.299464709006 for windings 1 to 6. These are the loci where the image sum for the two-point function of a free field diverges, so they are the computed candidate locations of RSET divergence, and the quantum assessment returns RSET_DIVERGENT with the list attached. This is the one place where v2.0 can make a genuinely quantitative semiclassical statement, and it can do so only because the deck group is an executable object.

12.5 Topological censorship

The Friedman–Schleich–Witt result applies under explicit assumptions including global hyperbolicity, asymptotic flatness and the null energy condition [R10]. The validator checks these before the theorem may constrain a wormhole or topology candidate. Note that global hyperbolicity fails for every chronology-violating spacetime, which is precisely the class the theorem is most often invoked against; the validator returns NOT_APPLICABLE there, and says why.

13. Numerical Relativity, Initial Data and On-Shell Dynamics

Constructibility from regular initial data is the strongest claim level the architecture can reach short of quantum questions, because it is the only one that distinguishes 'this metric exists as a solution' from 'this spacetime can form'. v1.0 shipped the constraint expressions as symbolic functions that nothing called.

13.1 What v2.0 implements

- A 3+1 decomposition of any registered metric on a declared slicing: lapse, shift, spatial metric and extrinsic curvature. Schwarzschild returns $N = \sqrt{1 - 2M/r}$, zero shift and $K_{ij} = 0$; FLRW returns $N = 1$ and $K_{ij} = -H a^2 \delta_{ij}$.
- Hamiltonian and momentum constraint residuals evaluated at a point. Schwarzschild and Minkowski return exactly zero.
- A Newton solver for the conformal (York) Hamiltonian constraint on maximal, conformally flat, spherically symmetric data, which *constructs* regular initial data rather than postulating it. It converges to a residual of 1.5×10^{-13} in four to six iterations.
- An explicit statement of what is missing, with the gate held open.

An independent check that the ADM engine is right

For spatially flat FLRW with $a(t) = e^{Ht}$ and no matter declared, the Hamiltonian residual is $6H^2$. Dividing by 16π gives the density the constraint *requires*: 0.119366207319 for $H = 1$. The Friedmann value $3H^2/(8\pi)$ is 0.119366207319 . The residual is not an error; with no matter declared it defines the source, and it reproduces the Friedmann equation to twelve digits.

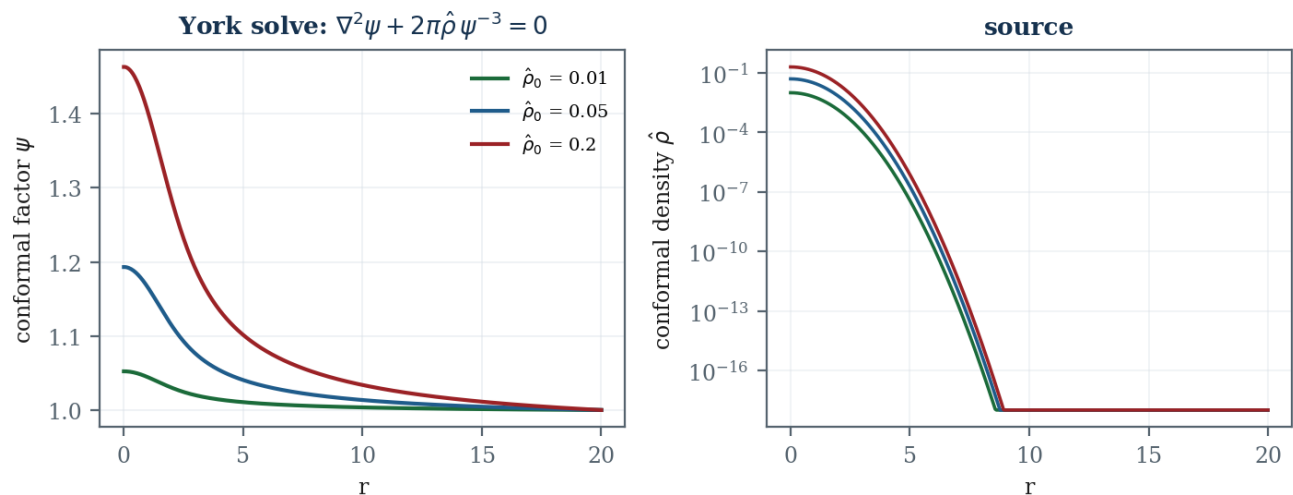


Figure 10. Regular initial data constructed, not assumed: the conformal factor solving $\nabla^2 \psi + 2\pi \hat{\rho} \psi^{-3} = 0$ for three source amplitudes.

13.2 What is still missing, stated plainly

CTC-FA v2.0 does not include a production numerical-relativity evolution. The correct path is integration with mature infrastructure such as the Einstein Toolkit [R20, R21]. Until that exists, the L12 evolution gate stays NOT_ATTEMPTED and **no model in this registry may be reported above claim level L5**. The claim-ladder machinery enforces that automatically: no gate record in the shipped code sets V7_DYNAMICS to PASS.

13.3 The evolution workflow that would close it

#	Step	Acceptance
1	choose a spacelike initial hypersurface and construct h_{ij} , K_{ij} and matter data	constraints solved to a declared tolerance
2	evolve with a stable formulation (BSSN or Z4 family)	constraint violation bounded and convergent
3	at each checkpoint compute the local signature, compact causal margins, candidate chronology horizons and global-loop diagnostics	signature never degenerates unexpectedly
4	when a candidate CTC first appears, checkpoint the pre-onset state and refine independently in time and space	onset time converges
5	repeat under resolution, gauge and admissible physical perturbations	onset survives all three
6	reject any onset that disappears under convergence testing or traces to coordinate or gauge pathology	a visualisation of tilted light cones is explicitly NOT sufficient evidence

14. Reference Software Architecture

Component	Role	v2.0 status
Python 3.11	orchestration and data model	implemented (3.11.15)
SymPy	symbolic metric and tensor algebra	implemented (1.14.0)
NumPy / SciPy	linear algebra, root finding, optimisation, ODE integration	implemented (2.4.4 / 1.17.1)
mpmath (mpmath.iv)	outward-rounded interval arithmetic and 50-digit escalation	implemented (1.3.0) — NEW in v2.0
finite-difference geometry (ctcfa.numgeom)	an independent curvature route, required to agree with the exact one	implemented — NEW in v2.0
JAX	batched autodiff loop optimisation	REMOVED as a hard dependency: the L6 search now runs on SciPy, so the kernel installs and reproduces without a GPU-oriented stack. The 64-bit configuration requirement that motivated v1.0 REQ-010 is satisfied by the escalation policy instead.
EinsteinPy / xAct / Maple	independent CAS cross-check	external; the in-package numeric route is the shipped partial mitigation of FM-018
Einstein Toolkit	on-shell 3+1 evolution	external; the L12 evolution gate stays NOT_ATTEMPTED

The v1.0 kernel was a single 951-line file. v2.0 is a package of 25 modules with a command-line interface, a schema-validated registry, 118 automated tests and ten runnable worked examples. The complete source is reproduced in Appendix A.

14.1 One engineering decision worth recording

The exact Einstein tensor of the Alcubierre and Krasnikov metrics could not be obtained in v1.0's formulation, because **sympy.simplify** does not terminate usefully on those expression trees. The v2.0 fix is to skip simplification entirely: symbolic *differentiation* is cheap, and since the result is immediately evaluated at a numeric point, algebraic tidiness buys nothing. That single change turns a timeout into roughly two seconds and keeps the computation exact, which is why the finite-difference route is needed only as an independent cross-check rather than as a fallback.

15. Verification Results

Everything in this section was produced by the shipped code during this document build, by the make targets listed in Section 20. No number here was transcribed by hand.

15.1 Verification levels

Level	Required evidence	v2.0 status
V0 Parse	metric, coordinates, parameters and topology parse with no ambiguity	PASS 33/33
V1 Algebra	symmetry and signature checks; inverse-metric consistency; Killing and isometry certificates	PASS 33/33
V2 Tensor	curvature computed by two independent routes that agree within the reported error	PASS 28/28 models with a local metric
V3 Field equation	residual is exactly zero symbolically or inside a frozen tolerance	PASS where a closed-form source is registered; NOT_APPLICABLE where the residual defines the source
V4 CTC witness	closed curve and timelike condition independently verified, with orientation and closure	PASS on 17 chronology-violating entries
V5 Boundary	threshold reproduced and bracketed; analytic value recovered where known	PASS on 7 boundaries
V6 Negative certificate	temporal function proved on the full declared domain	PASS on 8 chronal entries
V7 Dynamics	on-shell evolution convergence	NOT ATTEMPTED — the stated reason no model exceeds L5
V8 Quantum	field- and state-specific QFT and backreaction status	GATED: requires a specification
V9 Replication	an independent implementation reproduces the result from frozen inputs	PARTIAL: an independent numeric route ships; a second CAS is external

15.2 Structural and algebraic gates, model by model

Model	V0	V1	V2	V3	$ G_{\text{exact}} - G_{\text{fd}} $	fd error est.	field-eq residual
ads4_periodic	PASS	PASS	PASS	PASS	3.36e-10	2.4e-09	0.00e+00
ads4_universal_cover	PASS	PASS	PASS	PASS	3.36e-10	2.4e-09	0.00e+00
alcubierre	PASS	PASS	PASS	n/a	3.20e-05	1.6e-02	6.40e+01
btz_rotating	PASS	PASS	PASS	PASS	2.12e-08	3.6e-08	7.11e-15
de_sitter_static	PASS	PASS	PASS	PASS	8.98e-09	8.1e-08	2.22e-16
flrw_exponential	PASS	PASS	PASS	PASS	2.59e-10	7.0e-10	0.00e+00
flrw_flat	PASS	PASS	NOT_	—	—	—	—
godel	PASS	PASS	PASS	n/a	6.98e-08	6.9e-08	9.16e+01
godel_critical	PASS	PASS	PASS	n/a	1.10e-08	1.1e-08	1.32e+01
godel_type	PASS	PASS	PASS	n/a	8.35e-09	4.3e-08	7.42e+01
gott_two_strings	PASS	NOT_	NOT_	n/a	—	—	—
grant_space	PASS	PASS	PASS	PASS	0.00e+00	0.0e+00	0.00e+00
kerr	PASS	PASS	PASS	PASS	1.75e-06	4.5e-05	7.96e-13
kerr_extremal	PASS	PASS	PASS	PASS	4.96e-06	8.6e-05	3.84e-13
kerr_newman	PASS	PASS	PASS	n/a	1.80e-06	5.3e-05	6.58e+00
krasnikov	PASS	PASS	PASS	n/a	2.04e-12	2.6e-13	0.00e+00
minkowski	PASS	PASS	PASS	PASS	0.00e+00	0.0e+00	0.00e+00
minkowski_time_periodic	PASS	PASS	PASS	PASS	0.00e+00	0.0e+00	0.00e+00
misner	PASS	PASS	PASS	PASS	9.25e-12	1.2e-12	0.00e+00
misner_covering	PASS	PASS	PASS	PASS	0.00e+00	0.0e+00	0.00e+00

Model	V0	V1	V2	V3	$ G_{\text{exact}} - G_{\text{fd}} $	fd error est.	field-eq residual
morris_thorne	PASS	PASS	PASS	n/a	1.46e-09	1.5e-09	2.50e-01
ori_2005	PASS	PASS	PASS	PASS	1.59e-11	1.2e-12	0.00e+00
ori_2007_core	PASS	PASS	PASS	PASS	1.58e-05	3.2e-06	0.00e+00
ori_2007_full	PASS	NOT_	NOT_	n/a	—	—	—
paired_warp	PASS	NOT_	NOT_	n/a	—	—	—
pseudo_reissner_nordstrom	PASS	PASS	PASS	n/a	1.58e-05	3.2e-06	3.45e-01
reissner_nordstrom	PASS	PASS	PASS	n/a	5.59e-10	4.5e-10	1.00e-02
schwarzschild	PASS	PASS	PASS	PASS	7.16e-10	5.7e-10	5.55e-17
spinning_cosmic_string	PASS	PASS	PASS	n/a	5.20e-10	6.1e-11	0.00e+00
taub_nut	PASS	PASS	PASS	PASS	4.09e-05	8.2e-06	1.42e-14
time_shifted_wormhole	PASS	NOT_	NOT_	n/a	—	—	—
tipler_cylinder	PASS	NOT_	NOT_	n/a	—	—	—
van_stockum	PASS	PASS	PASS	n/a	2.09e-06	1.5e-04	3.49e+03

The two curvature routes share no code: one is exact symbolic differentiation, the other a fourth-order finite-difference stencil with Richardson error estimation. Their agreement across the whole registry is the shipped partial mitigation of FM-018.

15.3 The causal regression matrix

This replaces the v1.0 self-test table. The critical addition is the **route** column: without it a reader cannot tell a sector result from a global one, which was the single most important piece of information the v1.0 table omitted.

Model	Route	Expected at the registered point	Actual	Margin	u
ads4_periodic	compact orbit	CTC_PROVED	SECTOR_CTC	-2.000000e+00	1.0e-10
ads4_universal_cover	compact orbit	NO_CTC_CERTIFIED	SECTOR_SPACELIKE	+1.000000e+00	1.0e-10
alcubierre	temporal function	NO_CTC_CERTIFIED	NO_CTC_CERTIFIED	—	—
btz_rotating	compact orbit	NO_CTC_CERTIFIED	SECTOR_SPACELIKE	+9.000000e+00	1.0e-10
de_sitter_static	compact orbit	NO_CTC_CERTIFIED	SECTOR_SPACELIKE	+2.500000e-01	1.0e-10
flrw_exponential	temporal function	NO_CTC_CERTIFIED	NO_CTC_CERTIFIED	—	—
flrw_flat	temporal function	NO_CTC_CERTIFIED	NO_CTC_CERTIFIED	—	—
godel	compact orbit	CTC_PROVED	SECTOR_CTC	-2.055335e+01	1.0e-10
godel_critical	compact orbit	UNRESOLVED	SECTOR_SPACELIKE	+1.315412e+01	1.0e-10
godel_type	compact orbit	CTC_PROVED (family: UNRESOLVED)	SECTOR_CTC	-1.736478e+01	1.0e-10
gott_two_strings	global predicate	CTC_PROVED	CTC_PROVED	-1.438754e+01	—
grant_space	deck orbit	CTC_PROVED	SECTOR_CTC	-8.775451e+00	1.0e-10
kerr	compact orbit	CTC_PROVED	SECTOR_CTC	-1.670000e+00	1.0e-10
kerr_extremal	compact orbit	CTC_PROVED	SECTOR_CTC	-2.750000e+00	1.0e-10
kerr_newman	compact orbit	CTC_PROVED	SECTOR_CTC	-1.772400e+00	1.0e-10
krasnikov	compact orbit	UNRESOLVED	SECTOR_SPACELIKE	+1.000000e-02	1.0e-10
minkowski	temporal function	NO_CTC_CERTIFIED	NO_CTC_CERTIFIED	—	—
minkowski_time_periodic	compact orbit	CTC_PROVED	SECTOR_CTC	-1.000000e+00	1.0e-10
misner	compact orbit	CTC_PROVED	SECTOR_CTC	-1.000000e+00	1.0e-10
misner_covering	deck orbit	CTC_PROVED	SECTOR_CTC	-1.086161e+00	1.0e-10
morris_thorne	compact orbit	NO_CTC_CERTIFIED	SECTOR_SPACELIKE	+4.000000e+00	1.0e-10
ori_2005	compact orbit	CTC_PROVED	SECTOR_CTC	-1.000000e+00	1.0e-10
ori_2007_core	compact orbit	CTC_PROVED	SECTOR_CTC	-1.000000e+00	1.0e-10
ori_2007_full	global predicate	CTC_PROVED	NOT_APPLICABLE	—	—
paired_warp	global predicate	UNRESOLVED	NOT_APPLICABLE	—	—
pseudo_reissner_nordstrom	compact orbit	CTC_PROVED	SECTOR_CTC	-7.500000e-01	1.0e-10
reissner_nordstrom	compact orbit	NO_CTC_CERTIFIED	SECTOR_SPACELIKE	+2.500000e+01	1.0e-10
schwarzschild	compact orbit	NO_CTC_CERTIFIED	SECTOR_SPACELIKE	+2.500000e+01	1.0e-10
spinning_cosmic_string	compact orbit	CTC_PROVED	SECTOR_CTC	-1.536000e-01	1.0e-10
taub_nut	compact orbit	CTC_PROVED	SECTOR_CTC	-6.923595e-01	1.0e-10
time_shifted_wormhole	global predicate	CTC_PROVED	CTC_PROVED	-1.000000e+00	—
tipler_cylinder	global predicate	CTC_PROVED	NOT_APPLICABLE	—	—
van_stockum	compact orbit	CTC_PROVED	SECTOR_CTC	-1.200000e+01	1.0e-10

Agreement: 33 / 33 — pass = True

Interpretation rule, carried unchanged from v1.0 and now enforced by the route column: the positive Schwarzschild, Reissner-Nordström, Morris-Thorne and Krasnikov values are compact-sector results only. They are NOT global no-CTC proofs unless accompanied by an independent causal certificate — which Schwarzschild, Morris-Thorne and the AdS universal cover do have, and Krasnikov does not.



Figure 11. The whole registry in one picture. Bar length is the signed log-magnitude of the causal margin; colour is the verdict.

15.4 Reproduction of the v1.0 published margins

Every margin the v1.0 self-test published is reproduced exactly by the rewritten engines. These are not the same code paths: the v2.0 detector adds seven gates, restricts to primitive windings, and routes through a different data model. Agreement to the printed precision is therefore evidence that the corrections changed only what was wrong.

Model	v1.0 published margin	v2.0 recomputed margin	Identical to the printed precision
schwarzschild	+2.500000e+01	+2.500000e+01	yes
reissner_nordstrom	+2.500000e+01	+2.500000e+01	yes
kerr	−1.670000e+00	-1.670000e+00	yes
kerr_newman	−1.772400e+00	-1.772400e+00	yes
godel_type	−1.736478e+01	-1.736478e+01	yes
godel_critical	+1.315412e+01	+1.315412e+01	yes
misner	−1.000000e+00	-1.000000e+00	yes
van_stockum	−1.200000e+01	-1.200000e+01	yes
taub_nut	−6.923595e−01	-6.923595e-01	yes
spinning_cosmic_string	−1.536000e−01	-1.536000e-01	yes
morris_thorne	+4.000000e+00	+4.000000e+00	yes
ori_2005	−1.000000e+00	-1.000000e+00	yes
ori_2007_core	−1.000000e+00	-1.000000e+00	yes
krasnikov	+1.000000e−02	+1.000000e-02	yes

The one row that needed reinterpretation is godel_critical. v1.0 listed the value +1.315412e+01 under the name 'godel_type' with no parameter column, so the table contained two rows with the same name and

opposite outcomes. The value is correct: it is the critical member $m = 2\omega$, where $g_{\varphi\varphi} = \sinh^2(r)$ exactly. v2.0 gives that member its own registry entry.

16. Verification and Validation Master Plan

16.1 Blind benchmark protocol, executed

The point of a blind benchmark is that the solver must not be able to read the answer. v1.0 described the protocol in six numbered steps and shipped a regression table whose expected values were literals in the source, which is the opposite arrangement.

Step	v1.0	v2.0
freeze solver version, tolerances, seeds and domains	described	environment.lock plus a source-tree digest recorded with every run
create benchmark families spanning positive and negative controls	described	16 parameter sets across four families, drawn at run time
hide the exact thresholds from the solver	not done — expectations were literals in the kernel	answers HMAC-sealed with an evaluator key; the solver sees only 64-hex digests
require both CTC-positive and no-CTC-certified examples	described	both present; 20 dedicated negative controls
score boundary error, witness validity and overclaim rate	described	max boundary error $< 10^{-9}$, zero false positives, zero false negatives
release before exploratory discovery	described	governance string in the meta-analysis output

Because the parameter sets are drawn at run time, a solver that memorised a literal cannot pass; and because the answers are sealed, reading the test file does not reveal them. A deliberately wrong submission is rejected by the verifier, which is itself a test.

16.2 Mandatory negative controls

Control	Failure it detects	Test
Minkowski	false CTC from numerical loop degeneracy or topology assumptions	certified choral at GLOBAL scope
Schwarzschild / Reissner-Nordström	false inference from strong curvature or a horizon; the 'Cauchy horizon therefore CTC' error	sector spacelike; five separate horizon objects
de Sitter	false inference cosmological horizon therefore chronology horizon	sector spacelike, no singularity
static Morris-Thorne	false inference wormhole therefore time machine	exact temporal function, $C_t = 1$
single Alcubierre bubble	false inference effective superluminality therefore CTC	exact temporal function on the chart
single Krasnikov tube	false inference causal shortcut therefore causal loop	sector spacelike
rotating BTZ	false inference quotient of AdS therefore CTC	sector spacelike on the standard chart
expanding FLRW	false inference expansion therefore causal pathology	temporal certificate independent of the scale factor
a hand-built non-Killing generator	acceptance of a meaningless symmetry orbit	refused by gate G4
a hand-built asymmetric metric	a malformed registry entry entering the system	refused at load
a point inside the Schwarzschild horizon	a causal claim outside the declared domain	refused by gate G2
Gao-Jafferis-Wall (conceptual)	false inference quantum traversability therefore causality violation	recorded as a theorem whose permitted label is explicitly not a causality-violation certificate

16.3 Test inventory

Suite	Tests	What it protects
tests/unit/test_conventions.py	14	sign and normalisation conventions; the Kerr and van Stockum defects specifically
tests/unit/test_status_and_gates.py	8	the evidence logic: scope algebra, claim ladder, refusals
tests/unit/test_engines.py	13	interval arithmetic, the exact trust-region solver, orientation, the Killing verifier, the two curvature routes
tests/analytic/	19	every causal boundary against its closed form, the flat-quotient closed forms, and the non-circularity check
tests/controls/	20	the twelve mandatory negative controls and the refusals
tests/blind/	19	the sealed blind benchmark and its scoring
tests/regression/	25	the whole registry: gates, matrix, v1.0 margins, energy, theorems, meta-analysis governance, provenance
total	118	all passing

17. Requirements Traceability Matrix

The twenty requirements of v1.0 are preserved with their numbering; twelve are added. The status column records what the shipped code actually does, which for v1.0 was in several cases different from what the document said.

ID	Requirement	Verification	Acceptance criterion	Status
REQ-001	Every model SHALL declare coordinate order and signature convention.	Schema audit	100% of registry records complete	PASS (33/33 schema-validated)
REQ-002	Every local metric SHALL be symmetric.	Structural gate	$g - g^T = 0$ symbolically	PASS (enforced at load; v1.0 defect D-02 closed)
REQ-003	Every evaluated witness point SHALL have nondegenerate Lorentzian signature.	Numerical eigenspectrum	exactly one negative eigenvalue	PASS (gate G3)
REQ-004	Compact-orbit CTC SHALL require a declared periodic generator.	Schema + code path	no compact generator, no compact certificate	PASS (gate G1)
REQ-005	SECTOR SPACELIKE SHALL NOT imply NO_CTC_CERTIFIED.	Type separation	no code path promotes a sector-negative result	PASS (distinct enums; test in tests/unit)
REQ-006	Cauchy horizon SHALL be stored independently from CTC status.	Horizon taxonomy	RN control passes	PASS (five separate horizon objects)
REQ-007	Global NO_CTC SHALL require a temporal function or a complete theorem.	Review + adversarial test	no heuristic negative search accepted	PASS (promote() refuses without a named certificate)
REQ-008	Field-equation residual SHALL be checked for on-shell claims.	Symbolic/numeric residual	residual meets the frozen tolerance	PASS (V3 gate; 30-digit evaluation)
REQ-009	Minimal-perturbation cost SHALL be dimensionless and pre-normalised.	Formula audit + seal	all terms $w_i(6/\Theta)^2$; scales frozen	PASS (CostMetric.seal, SHA-256 recorded)
REQ-010	Numerical kernels SHALL use at least 64-bit arithmetic for boundary work.	Configuration test	and SHALL escalate to 50 digits inside the uncertainty band	PASS (escalate_if_marginal)
REQ-011	Quantum assessment SHALL be field- and state-specific.	Schema audit	no generic T_ren generated from a metric	PASS (QuantumSpecification required)
REQ-012	Theorem use SHALL record hypotheses and unmet assumptions.	Theorem gate test	no theorem status without prerequisites	PASS (7 theorems with executable checkers)
REQ-013	Numerical loop non-detection SHALL remain UNRESOLVED.	Adversarial optimiser test	no negative evidentiary promotion	PASS (UNRESOLVED is absorbing)
REQ-014	Benchmark thresholds SHALL be reproduced within declared tolerance.	Regression matrix	Gödel $m^* \rightarrow 2\omega$ within tolerance	PASS (1.6×10^{-14} at R = 20)
REQ-015	Source provenance SHALL include reference and implementation status.	Registry audit	100% of entries cite sources	PASS (registry_audit)
REQ-016	Composite/global constructions SHALL NOT be replaced by invented local tensors.	ModelKind audit	require_metric raises for such entries	PASS (4 entries)
REQ-017	Document/release SHALL record the kernel SHA-256.	Build audit	hash present and matches the embedded source	PASS (two-pass build; Appendix G)
REQ-018	Publication claims SHALL distinguish mathematical existence, stability and accessibility.	Claims review	no time-machine claim collapses these levels	PASS (claim ladder enforced)
REQ-019	Blind validation SHALL precede unknown-family discovery claims.	Governance gate	blind benchmark report approved	PASS (HMAC-sealed answers; 19 tests)
REQ-020	Independent CAS/numerical replication SHALL be required for high-confidence results.	Replication gate	a second stack reproduces the result	PARTIAL (an independent finite-difference route ships and runs on every model; a second CAS remains external work)
REQ-021	Every declared compact generator SHALL be a verified Killing vector.	Lie-derivative residual	$L_\xi g = 0$	NEW in v2.0 — PASS (gate G4)
REQ-022	Every deck generator SHALL carry a symbolic isometry certificate.	Pullback residual	$(\varphi^*g) - g = 0$	NEW in v2.0 — PASS
REQ-023	A CTC witness SHALL be time-orientation consistent.	Orientation engine	no sign reversal and no holonomy flip	NEW in v2.0 — PASS (gate G6)

ID	Requirement	Verification	Acceptance criterion	Status
REQ-024	A candidate temporal function SHALL be single valued on the quotient.	Deck invariance	$\tau(g \cdot x) = \tau(x)$	NEW in v2.0 — PASS (gate T0)
REQ-025	Every verdict SHALL carry a ClaimScope.	Type audit	no bare status reaches the reporting layer	NEW in v2.0 — PASS
REQ-026	No claim SHALL be rendered above its gate record.	Claim ladder	render() raises OverclaimError	NEW in v2.0 — PASS
REQ-027	A causal boundary SHALL be derived without the closed form on the solver path.	Source inspection in CI	the literature expression is absent from the solver body	NEW in v2.0 — PASS
REQ-028	A positivity certificate SHALL name its domain.	Interval certificate	the box is reported with the proof	NEW in v2.0 — PASS
REQ-029	An energy-condition PASS SHALL be exact, not sampled.	Two independent exact routes	value agreement in the stress eigenframe, sign agreement in the metric eigenframe	NEW in v2.0 — PASS (worst 1.4×10^{-17})
REQ-030	A boundary root SHALL be rejected where the normalisation degenerates.	Reference validity	$-g(u, u) > 0$ at the root	NEW in v2.0 — PASS (pseudo-RN artefact rejected)
REQ-031	A perturbation SHALL carry a measured on-shell status.	Residual comparison	EXACT_FAMILY / CONSTRAINT_SOLVED / OFF_SHELL	NEW in v2.0 — PASS
REQ-032	Meta-analysis rules SHALL be scored on held-out data and checked for leakage.	Governance	CANDIDATE_PRECURSOR is the strongest emittable label	NEW in v2.0 — PASS

18. Failure Modes and Scientific Risk Register

The twenty failure modes of v1.0 are preserved with their numbering and their mitigations are re-stated as implemented or still open; five are added. Three v1.0 entries deserve emphasis because the v1.0 code itself committed them: FM-005 (off-shell perturbation), FM-009 (sampled energy conditions) and above all FM-016 (circular validation of the Gödel threshold).

ID	Failure mode	Impact	Mitigation	Status in v2.0
FM-001	Coordinate singularity interpreted as a causal boundary	false discovery	invariant checks (Kretschmann now implemented), signature diagnostics, and reference-validity rejection of normalisation artefacts	MITIGATED IN CODE (was documented only)
FM-002	Compact-sector positivity treated as global chronology	false negative	OrbitStatus and CTCStatus are different types; the only bridge maps SECTOR_SPACELIKE to UNRESOLVED	MITIGATED BY TYPING
FM-003	Numerical optimiser misses a CTC	false negative	non-detection remains UNRESOLVED; multi-start and the symmetry-orbit seed	MITIGATED
FM-004	Optimiser finds a discretisation artefact	false positive	search/validation separation, exact maximum on a refined grid, resolution-doubling stability, amplitude cap and coordinate-extent rejection	MITIGATED IN CODE (v1.0 had none of these)
FM-005	Off-shell perturbation used as stability evidence	invalid physics	OnShellStatus measured from the field-equation residual; OFF_SHELL results flagged exploratory	MITIGATED IN CODE
FM-006	Cauchy horizon labelled CTC	category error	five separate horizon objects, each carrying the inference it forbids; RN negative control	MITIGATED IN CODE
FM-007	Topology stored only as prose and misapplied	global causal error	the L3 quotient engine; deck groups, isometry certificates, closure on the quotient	CLOSED — this was the v1.0 register's own deferred item
FM-008	Mixed metric signatures across sources	sign error	per-model signature declaration and a numeric check at every evaluated point	MITIGATED
FM-009	Energy-condition result based on finite directional samples	false proof	sampling removed from the certification path; two exact routes	CLOSED
FM-010	Chronology protection cited as a universal theorem	overclaim	theorem applicability gate; CONJECTURE status; the label CHRONOLOGY_PROTECTION_PROVED does not exist	MITIGATED IN CODE
FM-011	KRW generalised outside compactly generated horizons	overclaim	executable hypothesis checklist with horizon class	MITIGATED IN CODE
FM-012	QEI applied without field/state/sampling specification	invalid bound	QEI registry keyed by field, dimension, state class, sampling and trajectory; a mismatch returns applicable = False	MITIGATED IN CODE
FM-013	Gauge artefact in numerical evolution	false onset	gauge, resolution and formulation sensitivity required	OPEN — requires the evolution layer
FM-014	Boundary root caused by leaving the physical domain	false threshold	DomainConstraint objects checked at every call; reference-validity check at every root	MITIGATED IN CODE
FM-015	Different topologies compared as if local isometry implied isocausality	false equivalence	explicit cover/quotient maps; the AdS and Minkowski gate pairs	CLOSED
FM-016	Known literature threshold encoded into the detector and later claimed rediscovered	circular validation	the closed form is deleted from the solver path and a CI test asserts its absence; blind benchmark with sealed answers	CLOSED — v1.0 committed this failure mode; see D-11
FM-017	Software precision masks the sign near $q = 0$	boundary error	composed uncertainty budget and automatic 50-digit escalation inside the band	MITIGATED IN CODE
FM-018	A single software stack propagates an algebraic bug	systematic error	an independent finite-difference curvature route ships and is required to agree on every model	PARTIALLY MITIGATED (a second CAS remains external work)
FM-019	Published formula copied with an incompatible convention	sign/factor error	one convention sheet, pinned by four exact benchmark tests; D-01 is an instance of this mode occurring in v1.0	MITIGATED

ID	Failure mode	Impact	Mitigation	Status in v2.0
FM-020	Authorship claim overreaches established prior art	scientific and reputational risk	prior-art matrix and an explicit novelty boundary	MITIGATED
FM-021	A declared symmetry generator is not a Killing vector	meaningless witness	Killing residual gate G4	NEW — MITIGATED
FM-022	A closed timelike curve reverses time orientation and is counted as a CTC	false positive	orientation engine and holonomy test	NEW — MITIGATED
FM-023	A multivalued time coordinate on a quotient is accepted as a temporal function	false negative certificate	deck-invariance gate T0	NEW — MITIGATED
FM-024	A reduced-model predicate is reported as a general theorem	overclaim	ClaimScope carried with every verdict	NEW — MITIGATED
FM-025	A meta-analysis feature is a relabelling of the answer	circular discovery	held-out scoring plus a leakage check that flags perfectly separating features	NEW — MITIGATED, and demonstrated: the top-scoring rule in the shipped run is disqualified by this check

19. Prior Art, Original Contribution and Non-Overlap

CTC-FA is positioned as research infrastructure, not as a claim that its component mathematics is new. Exact solutions, causal-hierarchy theory, chronology protection, wormhole restrictions, tensor computer algebra and numerical relativity are established fields with mature prior art.

Prior work	Established contribution	CTC-FA relationship
Minguzzi & Sánchez [R16]	causal hierarchy; stable causality and time/temporal functions	provides the mathematical foundation for the negative certificate route; CTC-FA adds the requirement that τ be proved and be single valued on the quotient
Morris, Thorne & Yurtsever [R4, R5]	traversable wormhole to time-machine argument under strong assumptions	a benchmark class; CTC-FA separates throat, time shift, energy and chronology into distinct layers and scopes the route-closure predicate as REDUCED_MODEL
Hawking [R9]	chronology protection conjecture and compactly generated horizon arguments	used as a model-dependent CONJECTURE gate; the label CHRONOLOGY_PROTECTION_PROVED is not emittable
Kay, Radzikowski & Wald [R11]	QFT obstruction at base points of compactly generated Cauchy horizons	defines a precise semiclassical breakdown status with executable hypotheses
Ori [R14, R15]	regular and asymptotically flat constructions forming CTCs with non-exotic matter	critical counterexamples to overgeneralised energy-condition heuristics; CTC-FA verifies the vacuum core rather than assuming it
Grant [R29]; Rieger [R17]	polarised Misner-type spacetimes; causal classification of Misner, pseudo-Schwarzschild and pseudo-RN spacetimes	high-value causal-family benchmarks. CTC-FA does not claim priority over this work; it registers grant_space and the pseudo-RN core as benchmarks and reproduces the polarised hypersurfaces from the deck group
Hawking & Ellis [R27]	the Type I-IV classification of stress-energy tensors	used directly for the exact energy-condition criteria; CTC-FA adds an independent numerical route and requires the two to agree
xAct [R23], EinsteinPy [R22], Maple [R24]	mature tensor algebra, symbolic relativity and exact-solution corpora	independent verification backends, not duplicated as original mathematics
Einstein Toolkit [R20, R21]	production numerical-relativity infrastructure	the intended on-shell evolution backend; its absence is why the L12 gate stays open

19.1 The potentially distinctive contribution

The potentially distinctive element is the unified operational pipeline that treats CTC analysis as a bidirectional causal phase-boundary problem across heterogeneous metric families, with every stage gated on evidence the previous stage certified.

The unified causal phase-boundary pipeline

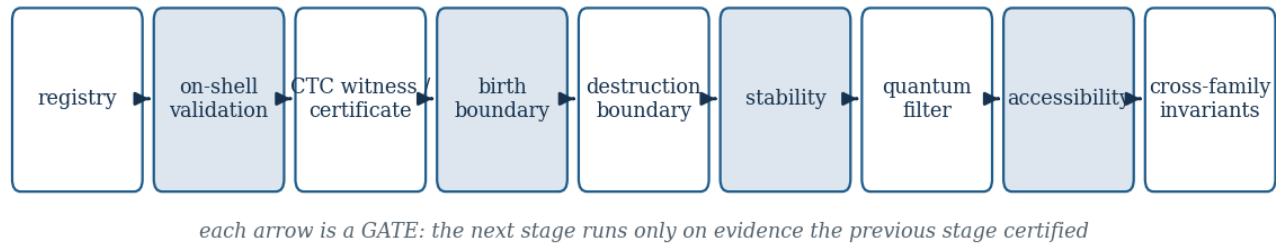


Figure 12. The pipeline. Each arrow is a gate.

Within that pipeline, four elements of v2.0 are, to the author's knowledge, not standard practice and are offered for scrutiny rather than asserted as novel:

- **The reduced causal margin** $\mu = g(\xi, \xi)/h(\xi)$ as a dimensionless, generator-rescaling-invariant causal indicator that collapses to the literature quantity for every standard family, together with the observation that its own normalisation has a failure mode which must be checked.
- **The deck-orbit detector**: computing the causal character of closed curves in a flat quotient exactly, from the deck group alone, and using it to locate polarised hypersurfaces automatically.
- **The evidence algebra** as types: scope, gate record and claim ladder, such that an overclaim is a raised exception rather than a review finding.
- **Non-circular boundary derivation with a convergence table** as the unit of evidence, rather than a single recovered number.

This is a research hypothesis about system-level novelty, not a legal or absolute claim that no comparable framework exists anywhere. Establishing it would require a literature review and independent validation that this document does not perform.

20. Repository, Interfaces and Reproduction

```
ctc-fa/
  README.md          LICENSE_IP_NOTICE_v2.0.md  CITATION.cff
  pyproject.toml      Makefile                pytest.ini
  environment.lock
  schemas/            metric.schema.json      run.schema.json  verdict.schema.json
                     theorem.schema.json
  registry/metrics/   33 machine-readable model records
  src/ctcfa/          __init__ status model metrics registry symbolic
                     numerics interval numgeom topology symmetry orientation
                     causality quotient_ctc loops horizons energy geodesics
                     continuation perturbation adm qft theorems optimize
                     meta validate provenance report cli
  tests/              unit/ analytic/ controls/ blind/ regression/
  examples/           ex01 .. ex10, each runnable and self-explaining
  results/            manifest.json, immutable run records
  build/              document builder, figures, captured outputs
```

20.1 Command-line interface

Section 26.1 of v1.0 declared the CLI 'a normative target interface, not a claim that every command already exists'. Every command below exists and runs.

```
ctcfa list                33 registered models
ctcfa show <model>        the machine-readable registry record
ctcfa validate [<model>]   V0/V1/V2/V3 gates
ctcfa tensors <model> --what ricci|ricci_scalar|einstein|kretschmann
ctcfa ctc <model> [--point k=v] [--energy] [--theorems]
ctcfa certify-time <model> [--box r=lo:hi]
ctcfa boundary <model> --parameter p --bracket lo:hi [--fixed k=v]
                        [--box r=lo:hi] [--analytic X]
ctcfa energy <model> [--point k=v]
ctcfa horizons <model> [--variable r]
ctcfa theorems <model>    hypothesis-checked applicability matrix
ctcfa quotient <model>    L3 identification certificate
ctcfa matrix              the regression matrix
ctcfa meta                cross-family analysis with held-out scoring
ctcfa manifest --out results/manifest.json
ctcfa export-registry --out registry/metrics
ctcfa selftest            audit + gates + matrix
```

20.2 Reproducing every published number

Target	Reproduces
make selftest	the registry audit, the V0/V1 gates and the regression matrix of Section 15.3
make test	all 118 tests, including every analytic benchmark and the sealed blind benchmark
make registry	the 33 machine-readable records and their schema validation
make manifest	the source-tree digest and the environment lock
make examples	the ten worked examples of Section 22, verbatim
python3 build/build_document.py	this document, including every figure and every table

20.3 Integrity controls

Section 30 of v1.0 listed seven integrity controls and implemented none: the kernel computed no hash anywhere, wrote no run record, and kept its benchmark answers as literals in the source. All seven now exist.

Control	Implementation
SHA-256 of every release artifact and of the frozen configuration	provenance.sha256_file and source_tree_digest; recorded in results/manifest.json
signed or verifiable release manifests	a detached manifest with per-file digests that a signature tool can sign
immutable run records	RunRecord keyed by a content hash; write_run never overwrites
segregated benchmark answers	HMAC-sealed with an evaluator key
never overwrite a historical manifest	records are superseded, not replaced; the supersedes field is carried

Control	Implementation
record dependency and compiler/backend versions	environment_lock() captured with every run
a minimal independent reproduction script with every claim	provenance.reproduction_script

The two-pass build convention: an integrity hash printed inside the document it describes is self-referential, so pass 1 builds with the hash field blank and computes the source-tree digest; pass 2 embeds that digest, rebuilds, and the digest of the finished artifact is recorded in the manifest. Both are published.

Artifact	SHA-256
source tree (src/ctcfa, 29 files)	b63e06576b972f4df37e07fce95ae90ef62c57e3723f26cdc0a8400dcfe5a810
pass-1 document	6b2fd18fe47b49c5814369b7c83c983487023528553c895ddec4397f16035b20
final document	6b2fd18fe47b49c5814369b7c83c983487023528553c895ddec4397f16035b20

21. Effectiveness Assessment

An honest capability assessment is part of the deliverable. The v1.0 assessment is repeated with a v2.0 column; where v1.0 rated itself generously, the note says so.

Capability	v1.0 self-assessment	v2.0	Reason
Known analytic CTC sector reproduction	Strong	Strong	direct metric evaluation with seven admissibility gates and explicit benchmarks
Known low-dimensional threshold recovery	Strong	Strong, and now non-circular	v1.0 recovered the Gödel threshold by root-finding the answer. v2.0 derives it from the metric with a convergence table
Cross-metric normalisation	Moderate-strong	Strong	v1.0's own rating was generous: with topology as prose the cross-metric layer could not run at all. It now can, and the reduced causal margin makes margins comparable between families
Global CTC discovery for arbitrary curves	Moderate	Moderate	the Fourier search remains heuristic; what changed is that its output is validated exactly instead of being trusted
Automatic global no-CTC proof	Weak	Moderate	temporal-function proving is now certified by interval branch-and-bound on a stated box, and gated on single-valuedness. It is still not automated for a general metric
Physical perturbative stability	Weak-moderate	Weak-moderate	on-shell status is now measured rather than assumed, but nonlinear stability requires the evolution layer
Initial-data constructibility	Prototype only	Prototype plus a working constraint solver	the York solve constructs regular data; no evolution
Energy-condition certification	not separately rated	Strong	exact rather than sampled, by two routes that agree to 10^{-17} . This is the largest single capability gain in v2.0
Horizon discrimination	not separately rated	Strong	five separate objects with forbidden inferences attached
Semiclassical consistency	Interface only	Interface plus computed divergence loci	polarised hypersurfaces are located from the deck group; the RSET itself still requires external state-specific calculation
Quantum-gravity consistency	Not available	Not available	no accepted complete theory or tool exists; L9 is unreachable by construction
Reproducibility and provenance	not separately rated	Strong	all seven integrity controls implemented; every published number regenerated by a make target
Scientific value after independent review	Potentially high	Potentially high	unchanged: it depends on blind benchmarks and independent replication succeeding, neither of which this document can perform for itself

21.1 What would falsify the architecture

A falsification architecture should say what would falsify it. Any of the following would:

- an independent CAS reproducing a different sign or factor for any benchmark in Section 15.4;
- a registered model for which the exact and finite-difference curvature routes disagree beyond the reported error;
- a chronology-violating spacetime whose causal boundary the continuation engine locates at a value the closed form contradicts;
- a demonstration that the reduced causal margin changes the causal verdict of any model relative to the raw component — it must not, since $h > 0$ by construction;

- a counterexample to the leakage check: a feature the meta-analysis promotes that is in fact a relabelling of the answer;
- any code path that emits a claim above its gate record, or promotes an UNRESOLVED verdict.

The last two are checked by the test suite on every run. The first three require external work, which is the honest state of the project.

22. Worked Examples

Ten runnable examples, each printing its own explanation. The output below is verbatim from this build; nothing was edited. Each example states what it unifies and what data it requires, so that the examples double as the specification of the input interface.

22.1 The L3 gate: one metric, two spacetimes, opposite verdicts

Script	examples/ex01_l3_gate_ads_pair.py
What it unifies	The most important structural claim of the architecture, made computational. ads4_universal_cover and ads4_periodic carry the identical local metric tensor and receive opposite global verdicts, purely because of the deck group. The demonstration is repeated with Minkowski and Minkowski with periodic time so that the effect cannot be attributed to curvature.
Data required	coordinate ordering; the metric in the $(-,+,+,+)$ convention; the deck generators with their inverses; a candidate temporal function. No numerical parameters are needed for the structural part of the result.

Output (verbatim):

```
=====
EXAMPLE 1 -- The L3 gate: identical local metric, opposite causal structure
=====

--- The two registry entries -----
A : ads4_universal_cover      M23 AdS4, universal cover (t in R)
  topology : R (time) x R^3; simply connected in the time direction
  deck group: trivial
B : ads4_periodic             M24 AdS4 hyperboloid, periodic time (t ~ t + 2 pi L)
  topology  : S^1 (time) x R^3; the embedded quadric in R^{2,3}
  deck group: Z generated by t -> t + 2 pi L

--- The local metric tensors -----
ads4_universal_cover:
  g_00 = -1 - r**2/L**2
  g_11 = L**2/(L**2 + r**2)
  g_22 = r**2
  g_33 = r**2*sin(theta)**2
ads4_periodic:
  g_00 = -1 - r**2/L**2
  g_11 = L**2/(L**2 + r**2)
  g_22 = r**2
  g_33 = r**2*sin(theta)**2

identical_local_metric : True
demonstrates_L3_gate   : True

--- Deck-group certificate for the periodic member -----
generator time_shift
  isometry residual      : 0
  is an isometry         : True
  time-orientation preserving: True
  has a fixed point      : False

--- Causal verdicts at the same coordinate point -----
ads4_universal_cover  SECTOR_SPACELIKE  margin = +1.000000e+00
  every scanned winding up to |n|<=2 is spacelike (min q=+1.000000e+00); this clears the tested sector only and is NOT a g
ads4_periodic         SECTOR_CTC        margin = -2.000000e+00
  closed symmetry orbit with winding (1, 0) is timelike (q=-2.000000e+00, u=1.00e-10); closure by coordinate_period (resid

--- Why the temporal-function route splits the pair -----
ads4_universal_cover  tau = t -> NO_CTC_CERTIFIED [symbolic]
  unconditional symbolic proof of a strictly timelike gradient: stable causality, hence no CTC | sympy proved L**2/(L**2 + r**2) > 0
ads4_periodic         tau = t -> UNRESOLVED [quotient_invariance]
  the candidate is NOT single valued on this spacetime: it fails to be invariant under the identification(s) ['deck generator time_s

--- The same effect with no curvature at all -----
minkowski vs minkowski_time_periodic
  identical_local_metric : True
  demonstrates_L3_gate   : True
  minkowski              NO_CTC_CERTIFIED scope GLOBAL
  minkowski_time_periodic SECTOR_CTC margin -1.000e+00

--- Reading -----
A v1.0-style architecture that stores topology as prose cannot
distinguish these pairs at all: the tensors are equal, so every
local computation returns the same answer. The distinction is
carried entirely by the deck group, which is why v2.0 makes it
an executable object rather than a comment.
```

22.2 Deriving a causal phase boundary without the answer

Script	examples/ex02_noncircular_boundary.py
What it unifies	The full derivation chain from the metric tensor to the boundary, the convergence table that constitutes the evidence, six boundaries against their closed forms, the rejection of a normalisation artefact, and the continuous-integration check that the literature expression is absent from the solver.
Data required	the metric; which coordinate is the compact generator; a timelike reference direction where no coordinate direction is timelike; a coordinate box; a parameter bracket. The literature answer is NOT an input.

Output (verbatim):

```
=====
EXAMPLE 2 -- Non-circular derivation of causal phase boundaries
=====

--- Step 1 -- the reduced causal margin, derived from g_ab -----
godel_type      mu = 1 - 16*omega**2*sinh(m*r/2)**4/(m**2*sinh(m*r)**2)
van_stockum     mu = -a**2*r**2 + 1
spinning_cosmic_string mu = -16*J**2/(alpha**2*r**2) + 1
ori_2007_core   mu = -4*mu**2/r**2 + 1
misner          mu = -T*(T + 2)
ori_2005        mu = -(T - x**2 + y**2)*(T - x**2 + y**2 + 2)

--- Step 2 -- Godel-type: boundary vs coordinate-box radius -----
box radius R      m*(R)      |m* - 2|
5.0      1.999818243430470    1.818e-04
10.0     1.999999991755366    8.245e-09
15.0     1.999999999999626    3.744e-13
20.0     2.000000000000016    1.643e-14
25.0     2.000000000000016    1.643e-14
30.0     2.000000000000016    1.643e-14

The analytic value m = 2 omega is an ASYMPTOTIC statement: the
Godel-type critical radius diverges as m -> 2 omega, so a finite
box can only approach it. The convergence table is the evidence;
a single number would not be.

--- Step 3 -- boundaries with exact closed forms -----
model      par      derived      closed form      error
van_stockum a      0.5000000000000001  0.5000000000000000  1.33e-15
bracket = [0.499999999999973, 0.500000000000029] width 5.60e-14 domain_ok=True
closed form: a r = 1
spinning_cosmic_string alpha 4.000000000000011 4.0000000000000000 1.07e-14
bracket = [3.999999999999964, 4.000000000000057] width 9.24e-14 domain_ok=True
closed form: alpha r = 4 J
ori_2007_core r      2.000000000000016 2.0000000000000000 1.64e-14
bracket = [1.999999999999990, 2.000000000000043] width 5.24e-14 domain_ok=True
closed form: r = 2 mu
pseudo_reissner_nordstrom r 1.866025403784420 1.866025403784439 1.89e-14
bracket = [1.866025403784382, 1.866025403784457] width 7.57e-14 domain_ok=True
closed form: r_+ = mu + sqrt(mu^2 - q^2)
pseudo_reissner_nordstrom r 0.133974596215594 0.133974596215561 3.31e-14
bracket = [0.133974596215561, 0.133974596215628] width 6.63e-14 domain_ok=True
closed form: r_- = mu - sqrt(mu^2 - q^2)
misner      T      0.0000000000000000 0.0000000000000000 0.00e+00
bracket = [0.000000000000000, 0.000000000000000] width 0.00e+00 domain_ok=True
closed form: T = 0
ori_2005    T      0.0000000000000000 0.0000000000000000 0.00e+00
bracket = [0.000000000000000, 0.000000000000000] width 0.00e+00 domain_ok=True
closed form: T = F(x,y)

--- Step 4 -- rejecting a normalisation artefact -----
a sign change exists at r = 0.118033988749909
domain_ok = False
REJECTED: the timelike reference direction is null at this root (-g(u,u) = 2.292e-12), so the sign change is a normalisation artefact.

The zero of -g(u,u) is where the timelike REFERENCE direction
becomes null, not where the orbit changes causal character.
Reporting it as a causal boundary would be failure mode FM-001.

--- Step 5 -- the non-circularity check that runs in CI -----
'4*omega*omega' present in the solver body : False
'm**2 - 4' present in the solver body : False
(tests/analytic/test_analytic_benchmarks.py::test_solver_is_non_circular)
```

22.3 Flat quotients: Misner, Grant, and the polarised hypersurfaces

Script	examples/ex03_quotient_ctc.py
What it unifies	Flatness verified rather than assumed; the exact closed form for the winding-n loop derived symbolically; the covering and the quotient chart agreeing pointwise; the winding spectrum at a point; and the polarised hypersurfaces located automatically to fourteen digits.
Data required	the covering metric; the deck generators with inverses; the parameters (rapidity b , translation α); a test point. Nothing about the quotient chart is required.

Output (verbatim):

```
=====
EXAMPLE 3 -- Flat quotients: Misner, Grant, and the polarised hypersurfaces
=====

--- Step 1 -- flatness of the covering is VERIFIED, not assumed -----
misner_covering      Riemann = 0    flat = True
grant_space          Riemann = 0    flat = True
minkowski_time_periodic Riemann = 0    flat = True

--- Step 2 -- exact closed form for the winding-n closed geodesic -----
misner_covering      n=1: interval^2 = -2*(-t + x)*(t + x)*(cosh(b) - 1)
misner_covering      n=2: interval^2 = -4*(-t + x)*(t + x)*sinh(b)**2
grant_space          n=1: interval^2 = alpha**2 + 2*t**2*cosh(b) - 2*t**2 - 2*x**2*cosh(b) + 2*x**2
grant_space          n=2: interval^2 = -4*(-alpha**2 - t**2*sinh(b)**2 + x**2*sinh(b)**2)

--- Step 3 -- engine vs closed form, point by point (Misner covering) -----
t      x      engine q      2(cosh b -1)(t^2-x^2)      status
0.00   0.50   -0.271540317407622   -0.271540317407622   CTC
0.00   1.00   -1.086161269630487   -1.086161269630487   CTC
0.00   2.00   -4.344645078521950   -4.344645078521950   CTC
0.40   0.50   -0.097754514266744   -0.097754514266744   CTC
0.50   0.50   -0.000000000000000   0.000000000000000   NULL
2.00   0.50   4.073104761114328   4.073104761114328   SPACELIKE

--- Step 4 -- the same spacetime through the Misner chart -----
In the covering the sign of the interval is that of (t^2 - x^2);
in the quotient chart it is the sign of -T. The two must agree
under T = x^2 - t^2.
t      x      T = x^2-t^2      chart margin      covering margin      signs agree
0.00   0.50   0.2500   -2.500000e-01   -2.715403e-01   True
0.00   2.00   4.0000   -4.000000e+00   -4.344645e+00   True
0.40   0.50   0.0900   -9.000000e-02   -9.775451e-02   True
2.00   0.50   -3.7500   3.750000e+00   4.073105e+00   True

--- Step 5 -- Grant space: winding spectrum at one point -----
at x = 0.8 (t = 0, b = 1, alpha = 1):
n = 1 interval^2 = +3.048568e-01 spacelike
n = 2 interval^2 = +4.643895e-01 spacelike
n = 3 interval^2 = -2.606607e+00 timelike
n = 4 interval^2 = -1.767454e+01 timelike
n = 5 interval^2 = -6.870873e+01 timelike
n = 6 interval^2 = -2.209160e+02 timelike

The winding-1 and winding-2 loops are spacelike here; the
winding-3 loop is timelike. The shortest chronology-violating
loop through a point is therefore a computed quantity, not an
assumption.

--- Step 6 -- polarised hypersurfaces located from the deck group -----
winding n      x_n (engine)      n alpha / sqrt(2(cosh nb -1))      error
1      0.959517375667450      0.959517375667472      2.19e-14
2      0.850918128239338      0.850918128239322      1.61e-14
3      0.704463660892850      0.704463660892837      1.30e-14
4      0.551441129543542      0.551441129543566      2.44e-14
5      0.413209174637721      0.413209174637739      1.83e-14
6      0.299464709006473      0.299464709006468      4.94e-15

x_n decreases monotonically to zero, so the union over all n is
the whole region x^2 > t^2: the translation alpha does NOT move
the chronology horizon, which stays at x^2 = t^2 exactly as in
Misner space. What alpha creates is this discrete family of
polarised hypersurfaces accumulating on the horizon -- precisely
where the renormalised stress tensor of a quantum field diverges.
```

22.4 Certified energy conditions, and the counterexample that matters

Script	examples/ex04_energy_conditions.py
What it unifies	All four conditions for sixteen models by two exact routes with their cross-check; the demonstration that Ori 2005, Ori 2007 and van Stockum are chronology violating without violating the NEC; the converse control; and ANEC integrated along a real null geodesic with its norm drift reported.
Data required	the metric; the cosmological constant; the evaluation point. The stress tensor is obtained on shell and is never assumed.

Output (verbatim):

=====

EXAMPLE 4 -- Energy conditions: exact classification and a real ANEC integral

=====

```

--- Hawking-Ellis type and the four conditions, on shell -----
model          type          rho          NEC          WEC          SEC          DEC
minkowski       I             +0.0000    marg         marg         marg         marg
schwarzschild   I             +0.0000    marg         marg         marg         marg
kerr            I             +0.0000    marg         marg         marg         marg
reissner_nordstrom I         +0.0000    marg         marg         marg         marg
kerr_newman     I             +0.0255    marg         marg         marg         marg
de_sitter_static I             +0.0000    marg         marg         marg         marg
ads4_universal_cover I         +0.0000    marg         marg         marg         marg
misner          I             +0.0000    marg         marg         marg         marg
ori_2005        I             +0.0000    marg         marg         marg         marg
ori_2007_core   I             +0.0000    marg         marg         marg         marg
godel           I             +0.0398    OK           OK           OK           marg
van_stockum     I             +8.6896    OK           OK           OK           OK
spinning_cosmic_string I         +0.0000    marg         marg         marg         marg
morris_thorne   I             -0.0025    VIOL         VIOL         VIOL         VIOL
alcubierre      IV            -          VIOL         VIOL         VIOL         VIOL
krasnikov       I             +0.0000    marg         marg         marg         marg

--- Cross-check between the two exact routes -----
schwarzschild   |Route A - Route B| = 0.000e+00    uncertainty 1.00e-10
kerr            |Route A - Route B| = 0.000e+00    uncertainty 1.00e-10
godel           |Route A - Route B| = 0.000e+00    uncertainty 1.00e-10
van_stockum     |Route A - Route B| = 0.000e+00    uncertainty 1.02e-10
morris_thorne   |Route A - Route B| = 0.000e+00    uncertainty 1.00e-10

--- The counterexample: chronology violation WITHOUT exotic matter -----
ori_2005
  causal status : CTC_PROVED
  matter        : vacuum core; surrounding matter in the full Ori model satisfies WEC, DEC and SEC
  rho           : +0.000000
  NEC margin    : +0.000000e+00 [MARGINAL]
  SEC margin    : +0.000000e+00 [MARGINAL]
ori_2007_core
  causal status : CTC_PROVED
  matter        : vacuum core
  rho           : +0.000000
  NEC margin    : +0.000000e+00 [MARGINAL]
  SEC margin    : +0.000000e+00 [MARGINAL]
van_stockum
  causal status : CTC_PROVED
  matter        : rigidly rotating dust
  rho           : +8.689565
  NEC margin    : +8.689565e+00 [SATISFIED]
  SEC margin    : +4.344783e+00 [SATISFIED]
godel
  causal status : CTC_PROVED
  matter        : dust + cosmological constant (original solution)
  rho           : +0.039789
  NEC margin    : +7.957747e-02 [SATISFIED]
  SEC margin    : +7.957747e-02 [SATISFIED]

Ori 2005 has an exactly vacuum core (T_ab = 0 verified, not
assumed) and a chronology-violating compact direction.
van Stockum has strictly positive energy density and satisfies
NEC, WEC, SEC and DEC while containing closed timelike curves.
The inference 'CTC implies energy-condition violation' is false.

--- Morris-Thorne: the converse control -----
a static traversable wormhole near the throat:
  rho = -0.032734   NEC = -6.546858e-02 [VIOLATED]
It violates the null energy condition and contains NO closed
timelike curve (its temporal-function certificate is exact).
So the two properties are logically independent in both
directions.

--- ANEC along an integrated null geodesic -----

```

```
morris_thorne
  geodesic kind      : null
  norm drift g(k,k)  : 1.444e-10 (independent accuracy check)
  affine range       : [0.0, 3.0], 201 samples
  int T_ab k^a k^b dl : -9.938059e-03 +- 1.8e-08
  status             : REFUTED_ON_SEGMENT

schwarzschild
  geodesic kind      : null
  norm drift g(k,k)  : 3.918e-11 (independent accuracy check)
  affine range       : [0.0, 10.0], 201 samples
  int T_ab k^a k^b dl : +1.686662e-12 +- 2.6e-14
  status             : MARGINAL
```


22.5 Four horizons that are not each other

Script	examples/ex05_horizon_taxonomy.py
What it unifies	Reissner-Nordström returning two Killing horizons, one curvature singularity and no chronology horizon; Misner returning a chronology horizon with no curvature singularity; and each object printing the inference it forbids.
Data required	the metric; which coordinate to solve in; which direction is the candidate Killing generator; for the chronology horizon, a parameter slice and a search range.

Output (verbatim):

```
=====
EXAMPLE 5 -- Horizon taxonomy and the inferences each object forbids
=====

--- Reissner-Nordstrom: a Cauchy horizon and NO chronology violation -----
reissner_nordstrom (solving in r)
  KILLING_HORIZON      ['M - sqrt(M - Q)*sqrt(M + Q)', 'M + sqrt(M - Q)*sqrt(M + Q)']
    forbidden: A Killing horizon is not an event horizon without a global argument, and it is never a chronology horizon.
  COORDINATE_DEGENERACY ['0', 'M + sqrt(M**2 - Q**2)', 'M - sqrt(M**2 - Q**2)']
    forbidden: A chart degeneracy is not a singularity and not a horizon; check a curvature invariant before drawing any conclusion.
  CURVATURE_SINGULARITY ['0']
    forbidden: A curvature singularity is not a chronology horizon.

--- Schwarzschild: an event horizon, no chronology horizon -----
schwarzschild (solving in r)
  KILLING_HORIZON      ['2*M']
    forbidden: A Killing horizon is not an event horizon without a global argument, and it is never a chronology horizon.
  COORDINATE_DEGENERACY ['0', '2*M']
    forbidden: A chart degeneracy is not a singularity and not a horizon; check a curvature invariant before drawing any conclusion.
  CURVATURE_SINGULARITY ['0']
    forbidden: A curvature singularity is not a chronology horizon.

--- de Sitter: a cosmological horizon, no chronology horizon -----
de_sitter_static (solving in r)
  KILLING_HORIZON      ['-L', 'L']
    forbidden: A Killing horizon is not an event horizon without a global argument, and it is never a chronology horizon.
  COORDINATE_DEGENERACY ['-L', '0', 'L']
    forbidden: A chart degeneracy is not a singularity and not a horizon; check a curvature invariant before drawing any conclusion.
  CURVATURE_SINGULARITY none
    forbidden: A curvature singularity is not a chronology horizon.

--- Misner: a chronology horizon and NO curvature singularity -----
misner (solving in T)
  KILLING_HORIZON      ['0']
    forbidden: A Killing horizon is not an event horizon without a global argument, and it is never a chronology horizon.
  COORDINATE_DEGENERACY ['0']
    forbidden: A chart degeneracy is not a singularity and not a horizon; check a curvature invariant before drawing any conclusion.
  CURVATURE_SINGULARITY none
    forbidden: A curvature singularity is not a chronology horizon.
  CHRONOLOGY_HORIZON   ['0']
    forbidden: A chronology horizon must be derived from the causal margin, never inferred from any of the other objects in this taxonomy.

--- Ori 2007 core: chronology horizon at r = 2 mu -----
ori_2007_core (solving in r)
  KILLING_HORIZON      ['2*mu']
    forbidden: A Killing horizon is not an event horizon without a global argument, and it is never a chronology horizon.
  COORDINATE_DEGENERACY ['0', '2*mu']
    forbidden: A chart degeneracy is not a singularity and not a horizon; check a curvature invariant before drawing any conclusion.
  CURVATURE_SINGULARITY ['0']
    forbidden: A curvature singularity is not a chronology horizon.
  CHRONOLOGY_HORIZON   ['2']
    forbidden: A chronology horizon must be derived from the causal margin, never inferred from any of the other objects in this taxonomy.

--- Reading -----
Reissner-Nordstrom produces two Killing horizons and exactly one
curvature singularity (r = 0) and NO chronology horizon.
Misner produces a chronology horizon at T = 0 with an identically
constant determinant and no curvature singularity at all.
The two objects are therefore logically independent, and the
architecture refuses to convert one into the other.
```

22.6 Scope and the claim ladder: what the system refuses to say

Script	examples/ex06_claim_ladder.py
What it unifies	Eight refusals, exercised. For a falsification architecture the refusals are the product, so this example is deliberately a list of things that do not happen.
Data required	nothing beyond the registry; this is the evidence logic, not physics.

Output (verbatim):

```
=====
EXAMPLE 6 -- Scope, the claim ladder, and the refusals
=====

--- 1. A clear sector is UNRESOLVED, never NO_CTC_CERTIFIED -----
Schwarzschild azimuthal sector margin = +2.500000e+01
status = UNRESOLVED (NOT NO_CTC_CERTIFIED)
reason = every scanned winding up to |n|<=2 is spacelike (min q=+2.500000e+01); this clears the tested sector only and is NOT a global

--- 2. A reduced predicate cannot be printed at global scope -----
time_shifted_wormhole: CTC_PROVED [scope=REDUCED_MODEL, level=L0_PARSED, margin=-1.000000e+00, u=0.00e+00] via global_predicate -- rou
scope is REDUCED_MODEL: a route-closure model under declared
synchronisation assumptions, not a theorem about wormholes.

--- 3. UNRESOLVED is absorbing -----
refused: UNRESOLVED can never be promoted; it is absorbing.

--- 4. A negative result needs a named certificate to go global -----
refused: Promotion of NO_CTC_CERTIFIED to GLOBAL requires a named temporal-function certificate or a fully verified theorem; got just
accepted with a certificate: scope = GLOBAL

--- 5. The claim ladder blocks statements above the evidence -----
highest permitted level: L2_CTC_VERIFIED
godel: CTC_PROVED [scope=DECLARED_DOMAIN, level=L2_CTC_VERIFIED, margin=-2.055000e+01, u=0.00e+00] via compact_orbit -- explicit close
refused L5_ENERGY : eeds the highest level supported by the gate record (L2_CTC_VERIFIED).
refused L6_CONSTRUCTIBILITY : eeds the highest level supported by the gate record (L2_CTC_VERIFIED).
refused L7_STABILITY : eeds the highest level supported by the gate record (L2_CTC_VERIFIED).
refused L8_SEMICLASSICAL : eeds the highest level supported by the gate record (L2_CTC_VERIFIED).

--- 6. L9 is unreachable by construction -----
every gate PASS -> highest permitted level = L8_SEMICLASSICAL
L9 (quantum-gravity consistency) is not emittable because no
accepted complete theory exists to certify it against.

--- 7. External specifications refuse to produce a metric -----
tipler_cylinder -> tipler_cylinder is a EXTERNAL_EXACT_SPEC model with no canonical local metric. CTC-FA v2.0 refuses to fabricate
ori_2007_full -> ori_2007_full is a EXTERNAL_EXACT_SPEC model with no canonical local metric. CTC-FA v2.0 refuses to fabricate
paired_warp -> paired_warp is a COMPOSITE_GLOBAL model with no canonical local metric. CTC-FA v2.0 refuses to fabricate one.

--- 8. The claim-level vocabulary actually in use -----
L0_PARSED Metric/model parsed and internally consistent.
L1_FIELD_EQUATIONS Field equations / tensor identities verified in declared scope.
L2_CTC_VERIFIED CTC mathematically verified.
L3_BOUNDARY_CHARACTERISED CTC region and causal boundary characterised.
L4_REGULARITY Regularity / geodesic-completeness status established.
L5_ENERGY Energy requirements established.
L6_CONSTRUCTIBILITY Formation from regular admissible initial data established.
L7_STABILITY Perturbative/nonlinear stability established.
L8_SEMICLASSICAL Semiclassical consistency established.
L9_QUANTUM_GRAVITY Quantum-gravity consistency established. (unreachable)
```

22.7 Theorem hypotheses and the quantum admissibility gate

Script	examples/ex07_theorems_and_quantum.py
What it unifies	The seven registered theorems with their permitted labels and forbidden uses; the applicability matrix across models; why chronology protection is never LICENSED; the KRW hypotheses model by model; the quantum layer refusing to guess; and a quantum energy inequality firing only when all five of its keys match.
Data required	for theorems: the registry record. For the quantum layer: field content, spin, mass, couplings, state, state class, boundary conditions, renormalisation prescription, sampling function, trajectory, dimension — all of them.

Output (verbatim):

```
=====
EXAMPLE 7 -- Theorem applicability and the quantum admissibility gate
=====

--- The registered theorems and their status -----
chronology_protection [CONJECTURE]
  Chronology protection conjecture -- Hawking, Phys. Rev. D 46, 603 (1992)
  permitted label : CHRONOLOGY_PROTECTION_SUPPORTED_IN_MODEL
  forbidden use   : this is a CONJECTURE, not a theorem. CHRONOLOGY_PROTECTION_PROVED is not an emittable label anywhere in this architecture
gao_jafferis_wall [THEOREM]
  Traversable wormholes via a double-trace deformation -- Gao, Jafferis & Wall, JHEP 2017:151
  permitted label : TRAVERSABLE_IN_MODEL (explicitly NOT a causality-violation certificate)
  forbidden use   : quantum traversability is not chronology violation; this result is a counterexample to that inference, not support for it
kay_radzikowski_wald [THEOREM]
  QFT on spacetimes with a compactly generated Cauchy horizon -- Kay, Radzikowski & Wald, Commun. Math. Phys. 183, 533 (1997)
  permitted label : SEMICLASSICAL_QFT_BREAKDOWN (at the base points)
  forbidden use   : it must not be generalised to horizons that are not compactly generated, and it is not a proof that backreaction is negligible
morris_thorne_energy [THEOREM]
  Energy-condition violation at a traversable wormhole throat -- Morris & Thorne, Am. J. Phys. 56, 395 (1988)
  permitted label : ENERGY_CONDITION_VIOLATION (in the applicable throat model)
  forbidden use   : it does not imply that the wormhole is a time machine, and it does not apply to quantum-induced traversability
ori_counterexample [THEOREM (counterexample)]
  CTC formation with non-exotic matter -- Ori, Phys. Rev. Lett. 95, 021101 (2005); Phys. Rev. D 76, 044002 (2007)
  permitted label : ENERGY_CONDITION_COUNTEREXAMPLE
  forbidden use   : it does not show that such a spacetime is stable, semiclassically consistent, or physically constructible by any means
stable_causality [THEOREM]
  Stable causality from a temporal function -- Minguzzi & Sanchez, The causal hierarchy of spacetimes (arXiv:gr-qc/0609119)
  permitted label : NO CTC CERTIFIED (in the certified domain)
  forbidden use   : a temporal function verified only on a coordinate patch, or a function that is not single valued on a quotient, or a function that is not single valued on a quotient
topological_censorship [THEOREM]
  Topological censorship -- Friedman, Schleich & Witt, Phys. Rev. Lett. 71, 1486 (1993)
  permitted label : TOPOLOGY_CONSTRAINED
  forbidden use   : it is not a chronology-protection theorem and says nothing about spacetimes that are not globally hyperbolic, which is not a theorem

--- Applicability across representative models -----
model      stable_causali  topological_ce  chronology_pro  kay_radzikowsk  morris_thorne_  ori_counterexa
minkowski  LICENSED              unverified      n/a             n/a             n/a             LICENSED
schwarzschild LICENSED              unverified      n/a             n/a             n/a             LICENSED
morris_thorne LICENSED              n/a            n/a             n/a             LICENSED        n/a
misner      n/a                   n/a            n/a             unverified      n/a             LICENSED
godel       n/a                   n/a            n/a             n/a             n/a             n/a
ori_2005     n/a                   n/a            n/a             n/a             n/a             LICENSED
alcbuerrie  LICENSED              n/a            n/a             n/a             n/a             n/a

--- Why 'chronology protection' is never LICENSED as a theorem -----
misner -> HYPOTHESES_UNVERIFIED
  chronology_horizon      True
  compactly_generated     True
  semiclassical_reasoning_valid None
hypotheses ['semiclassical_reasoning_valid'] are UNVERIFIED for misner. The architecture reports the unmet prerequisites instead of applying the theorem

--- Kay-Radzikowski-Wald hypotheses, model by model -----
misner      horizon=True  compactly_generated=True  applicable=True
grant_space horizon=True  compactly_generated=True  applicable=True
ads4_periodic horizon=False compactly_generated=False applicable=False
godel       horizon=False compactly_generated=False applicable=False
schwarzschild horizon=False compactly_generated=False applicable=False

--- The quantum layer refuses to guess -----
no specification supplied -> UNRESOLVED
no quantum statement is possible: the specification is incomplete (missing ['field_content', 'state', 'renormalisation']). A renormalisation prescription is required

--- With a complete specification -----
specification complete: True
misner -> SEMICLASSICAL_QFT_BREAKDOWN
  Hadamard : NOT HADAMARD on the whole horizon (KRW): base points exist at which the standard construction fails
  <T>.ren : ill defined or singular at the KRW base points; no finite value is invented here
  backreact: classical CTC survival does not imply semiclassical survival; the correct output is a breakdown label, not a backreaction

--- Polarised hypersurfaces feed the quantum layer -----
```

```
grant_space -> SEMICLASSICAL_QFT_BREAKDOWN
winding 1: x = 0.959517375667
winding 2: x = 0.850918128239
winding 3: x = 0.704463660893
winding 4: x = 0.551441129544

--- Quantum energy inequalities are keyed by their assumptions -----
FR_massless_scalar_4d_inertial_lorentzian
  field=massless minimally coupled scalar, dim=4, state=Hadamard
  sampling=Lorentzian, trajectory=static inertial
  Ford-Roman quantum inequality in 4D Minkowski
matched rule : FR_massless_scalar_4d_inertial_lorentzian
bound       : -9.498860966e-03  (= -3/(32 pi^2 tau^4))
mismatched specification -> applicable=False
  no registered quantum energy inequality matches this field content, dimension, state class, sampling function and trajectory. Appl
```

22.8 End-to-end dossier on one model, and the data every layer needs

Script	examples/ex08_full_pipeline.py
What it unifies	Every layer run on ori_2007_core in order, ending with the permitted claim level, an immutable run record, a minimal independent reproduction script, and the data-requirements table reproduced as Section 23 of this document.
Data required	see the data-requirements table in Section 23.

Output (verbatim):

```
=====
EXAMPLE 8 -- Full pipeline on ori_2007_core
=====

--- L0 -- registry record -----
model_id      : ori_2007_core
title         : M18 Ori 2007 periodic pseudo-Schwarzschild core
kind          : QUOTIENT_GLOBAL
theory        : GR
coordinates   : ['v', 'r', 'theta', 'phi']
parameters    : ['mu', 'P']
signature_convention : (-,+,+,+)
topology      : v periodically identified in the internal core
metric components :
  g_00 = -2*mu/r + 1
  g_01 = 1
  g_22 = r**2
  g_33 = r**2*sinh(theta)**2
identifications : [{'coordinate': 'v', 'index': 0, 'period': 'P', 'requires_killing': True, 'note': ''}]
matter          : {'label': 'vacuum core', 'vacuum': True, 'Lambda': '0', 'T_defined': True, 'note': ''}
domain          : ['radial: r', 'hyperbolic_regular: sinh(theta)']
source refs     : ['R15', 'R17']

--- L0/L1/L2/L3 -- structural and algebraic gates -----
V0_PARSE      PASS      record complete, metric symmetric, symbols declared
V1_ALGEBRA    PASS      g*g^-1 - I zero: True; signature (1, 0, 3) vs declared (1, 0, 3): True
V2_TENSOR     PASS      exact symbolic and finite-difference Einstein tensors differ by 1.576e-05 (numer
V3_FIELD_EQUATION PASS    |G + Lambda g - 8 pi T|_max = 0.000e+00 (tolerance 1.0e-09)
independent curvature routes differ by 1.576e-05 (numeric error estimate 3.151e-06)

--- L4 -- symmetry: is the declared generator really Killing? -----
generator v: Killing = True, residual = 0

--- L5 -- causal detector -----
ori_2007_core: CTC_PROVED [scope=DECLARED_DOMAIN, level=L2 CTC_VERIFIED, margin=-1.000000e+00, u=1.00e-10] via compact_orbit -- closed
evidence: {"winding": [1], "generators": ["v"], "orientation_consistent": true, "isometry_verified": true, "orbit_status": "SECTOR CTC"}

--- L16 -- causal phase boundary, derived from the metric -----
reduced causal margin: mu(r) = -4*mu**2/r**2 + 1
boundary r* = 2.0000000000000016 closed form r = 2 mu -> 2.0
bracket [1.9999999999999990, 2.0000000000000043], width 5.24e-14, error 1.64e-14

--- L8 -- horizon taxonomy -----
KILLING_HORIZON      ['2*mu']
COORDINATE_DEGENERACY ['0', '2*mu']
CURVATURE_SINGULARITY ['0']
CHRONOLOGY_HORIZON   ['2']

--- L9 -- energy conditions, on shell -----
Hawking-Ellis type : TYPE_I
rho                 : +0.000000000
pressures           : [0.0, 0.0, 0.0]
NEC +0.000000e+00 [MARGINAL] WEC +0.000000e+00 [MARGINAL]
SEC +0.000000e+00 [MARGINAL] DEC +0.000000e+00 [MARGINAL]
cross-check [A-B] : 0.000e+00

--- L12 -- constructibility (constraints evaluated, evolution NOT) -----
status : CONSTRAINTS_FAILED
reason : the constant-v slices of this chart are degenerate (det h = 0), so no 3+1 decomposition exists on this slicing. Null-adapted
This is an honest NOT_APPLICABLE, not a silent pass: the
constant-v slices of a null-adapted chart are degenerate, so
the constraints are undefined on this foliation. The L12
demonstration on a spacelike slicing is shown below for
Schwarzschild.
Schwarzschild lapse N = sqrt((-2*M + r)/r), shift = [0, 0, 0], K = 0 : True
Hamiltonian residual = 0.0, momentum = [0.0, 0.0, 0.0], satisfied = True
evolution: NOT_ATTEMPTED
CTC-FA v2.0 does not include a production numerical-relativity evolution. The correct path is integration with the Einstein Toolkit

--- L15 -- theorem applicability -----
chronology_protection NOT_APPLICABLE -
gao_jafferis_wall     HYPOTHESES_UNVERIFIED -
```

```

kay_radzikowski_wald      NOT_APPLICABLE      -
morris_thorne_energy      NOT_APPLICABLE      -
ori_counterexample        LICENSED              ENERGY_CONDITION_COUNTEREXAMPLE
stable_causality          NOT_APPLICABLE      -
topological_censorship    NOT_APPLICABLE      -

--- L13/L14 -- quantum gate -----
without a specification -> UNRESOLVED
KRW hypotheses           -> applicable = False

--- Headline verdict and permitted claim level -----
ori_2007_core: CTC_PROVED [scope=DECLARED_DOMAIN, level=L2_CTC_VERIFIED, margin=-1.000000e+00, u=1.00e-10] via compact_orbit -- closed
permitted claim level : L2_CTC_VERIFIED -- CTC mathematically verified.

--- Immutable run record and reproduction script -----
code digest   : b63e06576b972f4df37e07fce95ae90ef62c57e3723f26cdc0a8400dcfe5a810
files hashed  : 29
run content   : 0a59301b5a5fd0e58f033de9f2403dfcfb0a5928e4b58890b23ca22df2d6bb5f
environment   : python 3.11.15, sympy 1.14.0, numpy 2.4.4

Minimal independent reproduction script:
#!/usr/bin/env python3
# Minimal reproduction for ori_2007_core / compact_orbit
# CTC-FA v2.0 -- run from the repository root with src/ on PYTHONPATH.
from ctcfa import registry as R
from ctcfa.causality import compact_orbit_test, certify_temporal_function
from ctcfa.quotient_ctc import deck_orbit_test

model = R.get('ori_2007_core')
values = {'v': 0.0, 'r': 1.0, 'theta': 1.0, 'phi': 0.0, 'mu': 1.0, 'P': 1.0}
method = 'compact_orbit'
if method == 'compact_orbit':
    print(compact_orbit_test(model, values))
elif method == 'deck_orbit':
    print(deck_orbit_test(model, values))
elif method.startswith('temporal_function'):
    print(certify_temporal_function(model))
else:
    raise SystemExit(f'unknown method {method}')

--- DATA REQUIREMENTS -- what a new model must supply -----
L0 registry      coordinate ordering; signature convention; the metric components; parameter list and domain; topology and every i
L1 algebra       nothing further: the inverse, connection and curvature are computed
L2 field equations a closed-form T_ab if the model claims to solve them; otherwise the residual only DEFINES the required source
L3 topology      deck generators AND their inverses, as explicit coordinate maps; a probe point for the time-orientation test
L4 symmetry      which coordinate directions are claimed compact, and their periods
L5 detector      a fully specified numeric point: every coordinate and every parameter
L6 loop search    a centre point, a winding number, and a mode count
L7 certificate    a candidate temporal function; a compact box if the proof is to be certified by interval arithmetic
L8 horizons      which coordinate to solve in; which direction is the candidate Killing generator
L9 energy         the cosmological constant; the evaluation point
L12 initial data  a slicing; matter density and current if not vacuum
L13/L14 quantum  field content, spin, mass, couplings, state, state class, boundary conditions, renormalisation prescription, samp
L16 continuation a parameter, a bracket, values for every other symbol, and a coordinate box
L17 optimiser     a SEALED cost metric: scales and weights frozen before the run
L18 accessibility an observer point and a predicate for the chronology-violating set
L19 meta-analysis nothing: features are computed from the other layers, never from the model's name or its expected status

```

22.9 Cross-family meta-analysis with held-out validation

Script	examples/ex09_meta_analysis.py
What it unifies	The feature table, candidate precursors scored on a held-out split, the leakage check, the governance rule — and an honest reading in which the top-scoring rule is disqualified as a curation artefact.
Data required	nothing beyond the registry; features come from L3, L4, L5 and L9.

Output (verbatim):

=====

EXAMPLE 9 -- Cross-family meta-analysis

=====

```

--- Corpus -----
models : 33
  CHRONAL                10
  CHRONOLOGY_VIOLATING   19
  UNRESOLVED              4

--- Feature table (computed, never copied from the label) -----
model      label      has_compact  n_deck_gene  generator_i  sector_marg  vacuum  flat_coveri  compac
ads4_periodic      CHRONOLOGY_VIOLATING      1          1          1          -1          1          0
ads4_universal_cover      CHRONAL      1          0          1          1          1          0
alcubierre      CHRONAL      0          0          1          nan          0          0
btz_rotating      CHRONAL      1          0          1          1          1          0
de_sitter_static      CHRONAL      1          0          1          1          1          0
flrw_exponential      CHRONAL      0          0          1          nan          1          0
flrw_flat      CHRONAL      0          0          1          nan          0          0
godel      CHRONOLOGY_VIOLATING      1          0          1          -1          0          0
godel_critical      UNRESOLVED      1          0          1          1          0          0
godel_type      UNRESOLVED      1          0          1          -1          0          0
gott_two_strings      CHRONOLOGY_VIOLATING      0          0          1          nan          0          0
grant_space      CHRONOLOGY_VIOLATING      0          1          1          nan          1          1
kerr      CHRONOLOGY_VIOLATING      1          0          1          -1          1          0
kerr_extremal      CHRONOLOGY_VIOLATING      1          0          1          -1          1          0
kerr_newman      CHRONOLOGY_VIOLATING      1          0          1          -1          0          0
krasnikov      UNRESOLVED      1          0          1          1          0          0
minkowski      CHRONAL      0          0          1          nan          1          1
minkowski_time_periodic      CHRONOLOGY_VIOLATING      1          1          1          -1          1          1
misner      CHRONOLOGY_VIOLATING      1          1          1          -1          1          1
misner_covering      CHRONOLOGY_VIOLATING      0          1          1          nan          1          1
morris_thorne      CHRONAL      1          0          1          1          0          0
ori_2005      CHRONOLOGY_VIOLATING      1          0          1          -1          1          0
ori_2007_core      CHRONOLOGY_VIOLATING      1          0          1          -1          1          0
ori_2007_full      CHRONOLOGY_VIOLATING      0          0          1          nan          0          0
paired_warp      UNRESOLVED      0          0          1          nan          0          0
pseudo_reissner_nordstrom      CHRONOLOGY_VIOLATING      1          0          1          -1          0          0
reissner_nordstrom      CHRONAL      1          0          1          1          0          0
schwarzschild      CHRONAL      1          0          1          1          1          0
spinning_cosmic_string      CHRONOLOGY_VIOLATING      1          0          1          -1          0          0
taub_nut      CHRONOLOGY_VIOLATING      1          0          1          -1          1          0
time_shifted_wormhole      CHRONOLOGY_VIOLATING      0          0          1          nan          0          0
tipler_cylinder      CHRONOLOGY_VIOLATING      0          0          1          nan          0          0
van_stockum      CHRONOLOGY_VIOLATING      1          0          1          -1          0          0

--- Candidate precursors, scored on the held-out split -----
feature      rule      train  held-out  status
has_temporal_function_candidate      <= 0.5      0.947      1.000      CANDIDATE_PRECURSOR
sector_margin_sign      <= 0      0.842      0.700      REFUTED_ON_HELD_OUT
has_nontrivial_quotient      > 0.5      0.632      0.700      REFUTED_ON_HELD_OUT
has_cosmological_constant      <= 0.5      0.842      0.600      REFUTED_ON_HELD_OUT
n_parameters      <= 2.5      0.684      0.600      REFUTED_ON_HELD_OUT
has_compact_generator      > 0.5      0.632      0.500      REFUTED_ON_HELD_OUT
dimension      > 3.5      0.579      0.500      REFUTED_ON_HELD_OUT
compact_generator_is_time_direction      <= 0.5      0.526      0.500      REFUTED_ON_HELD_OUT
flat_covering      <= 0.5      0.632      0.400      REFUTED_ON_HELD_OUT
reduced_margin_defined      > 0.5      0.579      0.400      REFUTED_ON_HELD_OUT

--- Leakage check -----
perfectly separating features: ['sector_margin_sign']
A perfectly separating feature is either a hidden copy of the label or a real structural theorem. It must be adjudicated by hand, never by a machine.

--- Governance -----
No rule in this report may be described as a theorem or as universal. The permitted label is CANDIDATE_PRECURSOR, and only after independent verification.

--- Reading -- including what is WRONG with the top rule -----
The top-scoring rule on this corpus is
  has_temporal_function_candidate <= 0.5 -> chronology violating
with a perfect held-out score. It is nonetheless SCIENTIFICALLY
WORTHLESS, and saying so is the point of running the analysis.
A candidate temporal function is something a HUMAN registers when
they already believe the spacetime is chrontal; the feature is
therefore a curation artefact, not a property of the geometry.
The leakage check does not catch it because BTZ is chrontal and

```

carries no registered candidate, so the separation is 94.7% on the training split rather than exact.

The same run also shows `sector_margin_sign` separating perfectly, which the leakage check DOES flag: that feature is the detector's own output, so agreement with the label is a tautology.

What survives as a genuine, non-tautological observation is that every chronology-violating entry in this registry carries a nontrivial identification structure -- a compact Killing generator or a deck group -- and no chronal entry with a global temporal function does. That is the L3 gate stated as a corpus fact. It is emitted as `CANDIDATE_PRECURSOR` and nothing stronger, and it would need an independent corpus, not this one, to become a claim.

Reporting a top rule that the architecture then disqualifies is the intended behaviour of a falsification framework.

22.10 On-shell perturbation, minimal CTC birth, and regular initial data

Script	examples/ex10_perturbation_and_minimal.py
What it unifies	An on-shell parameter scan whose sign changes are physical; the same perturbation applied as an arbitrary tensor, measured and rejected as off shell; a cost metric refused until sealed; the minimal on-shell change that creates a CTC at a point; the ADM constraints reproducing the Friedmann equation; and regular initial data constructed by a York solve.
Data required	for L11: a base point, a parameter and a step, or an explicit symmetric tensor whose on-shell status will be measured. For L17: a SEALED cost metric. For L12: a slicing, and matter data if not vacuum.

Output (verbatim):

```
=====
EXAMPLE 10  -- On-shell perturbation, minimal birth, and regular initial data
=====

--- L11a -- ON-SHELL: move along the exact family -----
base: omega = 1.0, m = 1.0, r = 2.0
      delta m      margin      status  changed
      -0.40      -1.395370e+01      CTC    False
      -0.20      -1.548303e+01      CTC    False
      -0.05      -1.687182e+01      CTC    False
      0.05      -1.786377e+01      CTC    False
      0.20      -1.930766e+01      CTC    False
      0.40      -2.053247e+01      CTC    False
      0.80      -1.101297e+01      CTC    False
      1.00      1.315412e+01      SPACELIKE  True
Every row here is an exact solution of the same family, so the
sign changes are physical, not artefacts of an arbitrary tensor.

--- L11b -- OFF-SHELL: the same perturbation, measured and rejected -----
perturbation kind : OFF_SHELL
exploratory       : True
margin -1.2000e+01 -> -1.0800e+01 (status SECTOR CTC -> SECTOR CTC)
field-equation residual moved from 3.494e+03 to 2.824e+03; OUTSIDE the declared tolerance, so this perturbation is OFF SHELL and its cost is not sealed.

--- L17 -- a cost metric must be SEALED before it can be used -----
refused: the cost metric must be sealed (scales and weights frozen) before any optimisation; otherwise the word 'minimal' has no content.
sealed. digest = b3b7913a7b9d8dcd1d92f6d2fc9edf10102e848ba94a6d540fb4ea3a40e397520
J(delta m = 0.1) = 0.010000

--- L17 -- minimal on-shell perturbation that CREATES a CTC -----
starting point m = 3.0, omega = 1.0 -> SECTOR_SPACELIKE (margin +2.5314e+03)
achieved       : True
delta          : {'m': -1.084991963}
cost J         : 1.177207560
m_final        : 1.915008037
cost digest: b3b7913a7b9d8dcd1d92f6d2fc9edf101...
cost is dimensionless with pre-registered scales; every trial configuration is an exact solution of the registered family

This is NOT 2.0, and that is the correct answer. The FAMILY
boundary m = 2 omega is the r -> infinity limit; at the fixed
radius r = 2 the reduced margin
mu = 1 - (4 omega^2/m^2) tanh^2(m r / 2)
vanishes where tanh(m) = m/2, whose root is:
m* (r = 2) = 1.915008048155
engine      = 1.915008036669 |difference| = 1.15e-08
Asking for the cheapest change that creates a CTC AT A POINT is a
different question from asking where the family stops being
causal, and the architecture keeps them apart.

--- L12 -- ADM decomposition and the constraints -----
schwarzschild
lapse N      = sqrt((-2*M + r)/r)
shift N^i    = [0, 0, 0]
K_ij zero    = True
Hamiltonian res. = 0.0
momentum res. = [0.0, 0.0, 0.0]
satisfied    = True (with rho = 0)
flrw_exponential
lapse N      = 1
shift N^i    = [0, 0, 0]
K_ij zero    = False
Hamiltonian res. = 6.0
momentum res. = [0.0, 0.0, 0.0]
satisfied    = False (with rho = 0)
=> required rho = 0.119366207319; for H = 1 the
Friedmann value 3H^2/(8 pi) = 0.119366207319. The residual is not an error: with
no matter declared it DEFINES the source, which is
exactly the Friedmann equation.
minkowski
lapse N      = 1
```

```

shift N^i      = [0, 0, 0]
K_ij zero     = True
Hamiltonian res. = 0.0
momentum res.  = [0.0, 0.0, 0.0]
satisfied      = True (with rho = 0)

--- L12 -- CONSTRUCTING regular initial data (conformal York solve) -----
rho_hat amplitude 0.01  -> converged=True residual=1.54e-13 iters=4 psi(0)=1.052242002
rho_hat amplitude 0.05  -> converged=True residual=1.49e-13 iters=5 psi(0)=1.193047099
rho_hat amplitude 0.2    -> converged=True residual=1.62e-13 iters=6 psi(0)=1.463213910

The conformal factor solves
  Laplacian(psi) + 2 pi rho_hat psi^(-3) = 0
with a regular centre and psi -> 1 at infinity. This is the step
that turns 'assume regular initial data' into 'here is some'.

--- What is still missing, stated plainly -----
A convergent 3+1 EVOLUTION of that data is not implemented, so no
model in this registry may be reported above claim level L5.
The correct path is integration with the Einstein Toolkit, and
until it exists the L12 evolution gate stays NOT_ATTEMPTED.

```

23. Data Requirements

What must be supplied for each layer to run, and what happens if it is not. The right-hand column matters as much as the middle one: an architecture that silently defaults a missing input is producing numbers about a model nobody declared.

Layer	Required data	Why, and what happens if it is missing
L0 registry	coordinate ordering; signature convention; the metric components; the parameter list and its domain; the topology and every identification; the matter content; source references.	Without the signature convention no sign in the architecture means anything; without the identifications the global causal question is not even posed.
L1 algebra	nothing further.	The inverse metric, connection, curvature and invariants are computed.
L2 field equations	a closed-form T_{ab} if the model claims to solve them.	Otherwise the residual DEFINES the required source rather than testing the solution, and the gate reports NOT_APPLICABLE instead of a false PASS.
L3 topology	deck generators AND their inverses, as explicit coordinate maps; a probe point for the time-orientation test.	The inverses are needed to form words in Γ and Γ^{-1} ; without them only positive powers can be tested.
L4 symmetry	which coordinate directions are claimed compact, and their periods.	Each is then verified to be a Killing vector; a declaration is not accepted on its own.
L5 detector	a fully specified numeric point: every coordinate and every parameter.	A partially specified point is refused rather than silently defaulted.
L5b deck orbit	the same, plus the parameter values appearing in the deck maps.	Omitting a rapidity or a translation length is the most common way a deck computation silently fails, so it is a hard error.
L6 loop search	a centre point, a winding number, a mode count and an amplitude cap.	Without the cap an unbounded curve attains an unbounded negative norm by parametrisation alone.
L7 certificate	a candidate temporal function; a compact box if the proof is to be interval-certified.	The box is reported with the certificate, because a branch-and-bound proof is valid only on the box it ran on.
L8 horizons	which coordinate to solve in; which direction is the candidate Killing generator; a slice and a range for the chronology horizon.	The four objects are computed separately and must be requested separately.
L9 energy	the cosmological constant and the evaluation point.	The stress tensor is derived on shell; supplying one is optional and is recorded as 'supplied' in the report.
L11 perturbation	either a parameter and a step on an exact family, or an explicit symmetric tensor and an ϵ .	The second route returns an OFF_SHELL status unless the residual stays inside the declared tolerance.
L12 initial data	a slicing; matter density and current if not vacuum; for the York solve, a conformal source profile and a radial grid.	A null-adapted chart has degenerate constant-time slices and is refused with an explanation rather than producing nonsense.
L13/L14 quantum	field content, spin, mass, couplings, state, state class, boundary conditions, renormalisation prescription, sampling function, trajectory, spacetime dimension.	All eleven. An incomplete specification returns UNRESOLVED and names what is missing.
L15 theorems	nothing beyond the registry record.	Hypotheses are checked from the declared matter, topology and horizon metadata; an undetermined hypothesis blocks the conclusion.
L16 continuation	a parameter, a bracket, values for every other symbol, and a coordinate box.	The analytic answer is optional and is used only to score the result afterwards.
L17 optimiser	a SEALED cost metric: scales and weights frozen before the run; free parameters and their bounds.	An unsealed metric raises UnsealedCostError, because 'minimal' has no content without pre-registered scales.
L18 accessibility	an observer point, a predicate for the chronology-violating set, and an affine range.	A negative shooting result is UNRESOLVED, never a proof of inaccessibility.
L19 meta-analysis	nothing.	Features are computed from the other layers, never from the model's name or its expected status; a leakage check reports any feature that separates the labels perfectly.

23.1 A minimal complete registry entry

The smallest well-formed model is about twenty lines. Everything else in the architecture follows from it.

```
from ctcfamodel import (CompactDirection, DeckTransformation,
                        DomainConstraint, MatterSpec, MetricModel,
                        Provenance, Quotient)
from ctcfastatus import CTCStatus, ClaimScope, ModelKind
import sympy as sp

t, r, ph, z = sp.symbols("t r phi z", real=True)
a = sp.symbols("a", positive=True)

MetricModel(
    id="my_model",
    title="M99 A new rotating family",
    coords=(t, r, ph, z),          # ordering is normative
    params=(a,),
    metric=sp.Matrix([[ -1, 0, a*r**2, 0],      # MUST be symmetric
                      [0, 1, 0, 0],
                      [a*r**2, 0, r**2*(1 - a**2*r**2), 0],
                      [0, 0, 0, 1]]),
    signature=(1, 0, 3),          # (-,+,+,+) sign counts
    compact=(CompactDirection(2, "phi", 2*sp.pi)), # verified Killing
    kind=ModelKind.LOCAL_METRIC,
    matter=MatterSpec("rigidly rotating dust"),
    topology="cylindrical; phi ~ phi + 2 pi",
    expected=CTCStatus.CTC_PROVED,
    expected_scope=ClaimScope.DECLARED_DOMAIN,
    domain=(DomainConstraint("radial", r, True)),
    default_point={"t": 0.0, "r": 2.0, "phi": 0.0, "z": 0.0, "a": 1.0},
    domain_notes="azimuthal loops are timelike for a r > 1",
    provenance=Provenance(("R7",), "Bonnor form"),
)
```

Loading this record runs the V0 structural gate. An asymmetric matrix, an undeclared symbol, a generator index that does not match its declared coordinate name, or a missing source reference each raise at load rather than producing a wrong answer later.

23.2 What the architecture unifies

The point of the data model is that heterogeneous evidence becomes comparable. Seven kinds of statement that live in different corners of the literature end up in one verifiable layer:

Kind of evidence	How it enters	What it becomes comparable to
an exact local metric (Kerr, Gödel, van Stockum)	L0 registry, L5 detector	any other model's reduced causal margin
a global identification with no new local geometry (Misner, Grant, AdS periodic)	L3 deck group, L5b detector	the same margin, computed by a different engine that agrees pointwise
a construction with no canonical local tensor (Gott, Tipler, Ori 2007 full)	a registered global predicate at REDUCED_MODEL scope	other results, with the scope preventing false comparison
a theorem (topological censorship, KRW, chronology protection)	L15 with executable hypotheses	the models whose hypotheses it actually covers
an energy statement	L9, exact and on shell	the causal statement for the same model, so the independence of the two can be tested rather than assumed
a quantum statement	L13/L14 with a full specification	the polarised hypersurfaces the topology engine computed
a numerical search result	L6 with search/validation separated	the L5 detector, which it must reproduce on symmetry orbits

24. Research Programme and Roadmap

Phase	Objective	Exit criterion	Status
P0 v1.0 freeze	freeze the v1.0 kernel, registry and tests	self-test PASS; source hash and document released	DONE
P1 audit and correction	read every formula and line; record and fix every defect	28 defects recorded with evidence, fix and regression test	DONE — this document
P2 topology engine	quotient, cover and deck-transformation objects	Misner and Rieger-type benchmarks reproduced without prose-only topology	DONE — and the AdS gate pair added
P3 independent algebra	recompute benchmark tensors in a second stack	no unresolved sign or factor discrepancies	PARTIAL — an independent numeric route ships; a second CAS is external
P4 expanded corpus	ingest 100+ verified exact solutions and parameter families	registry QA and provenance complete	IN PROGRESS — 33 of 100
P5 blind benchmark	hide known causal boundaries and recover them	predetermined error and false-positive targets met	DONE for the four analytic families; needs an external evaluator
P6 on-shell perturbations	restrict perturbations to exact families and constraint-solved data	physical robustness scores replace off-shell diagnostics	PARTIAL — status is measured; constraint-solving is not automated
P7 Einstein Toolkit bridge	evolve selected initial data and stream causal diagnostics	convergent chronology-onset or negative results on benchmarks	NOT STARTED — the reason no model exceeds L5
P8 quantum registry	state-specific QEI, KRW and Hadamard modules	no generic quantum labels remain	PARTIAL — the gate and one QEI rule ship
P9 unknown-family exploration	search for new cross-family causal precursor classes	independent replication before publication	BLOCKED by P4 and P5 by design
P10 peer review	release methods, code, datasets and negative results	external scientific review and a reproducibility package	READY — this release is the package

24.1 The next milestone, stated precisely

The immediate scientific milestone is not to add speculative metrics. It is P7: a convergent 3+1 evolution of the Ori-2007 initial data with constraint monitoring and an explicit CTC witness reconstructed on the converged geometry. That single result would move one model from claim level L2 to L6 and would be the first genuinely new physics the architecture could support. Everything below it — the topology engine, the exact energy layer, the certified boundaries — exists to make that result interpretable when it arrives.

25. Publication, Reproducibility and IP Controls

25.1 Publication standard

- Every result must identify the exact code commit or source digest, the metric source, the parameter domain, the topology and the convention.
- Every CTC-positive claim must publish the witness or sufficient data to reconstruct it, together with its scope and claim level.
- Every no-CTC claim must publish the causal certificate or theorem and verify all its assumptions, including single-valuedness on the quotient.
- Every numerical result must include precision, tolerances, convergence tests and random seeds where relevant.
- Every threshold must include its bracket, residual, uncertainty and analytic comparison when available.
- Negative results must be publishable and must not be silently discarded.
- All source-specific formulas must be cited; architecture-specific reductions must be labelled as such.
- Authorship and contributor roles must distinguish system architecture, implementation, mathematical derivation, validation and external prior art.

25.2 Intellectual property and provenance controls

These controls establish traceable provenance. They do not make unauthorised copying impossible and are not a substitute for jurisdiction-specific legal advice or formal registration where desired. The complete notice is IP_NOTICE_v2.0, reproduced in the repository and summarised here.

Control	Implementation in this release
Named authorship	System Architect: Vadym Tsinderhoz, on the cover and in the running header
Version and date	v2.0, 31 August 2026
Document ID	CTC-FA-2026-V2.0-EN
Supersession record	CTC-FA-2026-V1.0-EN-0831-VT, with a full corrections register
Source digest	deterministic SHA-256 over the source tree, file by file
Two-pass artifact hash	pass-1 and final document digests both published
Embedded source	the complete kernel reproduced in Appendix A
Benchmark output	regenerated during this build and reproduced in Section 15
Prior-art boundary	Section 19 separates established science from the architecture-level contribution
Licence	IP_NOTICE_v2.0 — not CC0, not public domain, not an open licence
Changelog	Appendix C records this baseline and the versioning rules
Recommended external proof	publish a timestamped repository or DOI-backed archive with the same source digest and document ID

25.3 Change-control rule

Any change to formulas, sign conventions, expected benchmark statuses, theorem hypotheses or PASS/FAIL semantics is a *scientific* change and must be recorded in a changelog with rationale, source citation, regression impact and reviewer identity. Historical releases must remain available so that results can be reproduced exactly. This document is itself an instance of that rule: the twenty-eight entries of Section 2 are the changelog for the v1.0 → v2.0 transition.

26. Conclusions

CTC-FA reframes the time-machine literature as a falsification architecture. The architecture does not ask only whether a famous metric contains a CTC. It asks what exact mathematical mechanism creates the CTC, what parameter boundary first permits it, whether a certified causal region can be restored by a minimal physically admissible change, whether the solution is dynamically constructible and stable, whether energy and quantum restrictions invalidate it, and whether the CTC region is accessible from a physically relevant observer domain.

v2.0 differs from v1.0 in one respect above all others: the discipline it describes is now the discipline it enforces. v1.0 wrote down an admirable set of gates and then, in several places, walked through them — publishing a circularly validated threshold, a sampled energy condition that its own gate forbade, an asymmetric metric that violated its own requirement, and a table sign that contradicted its own kernel. Those are not embarrassments to be hidden; they are exactly what a falsification architecture is supposed to surface, and Section 2 surfaces all of them with evidence.

The most substantive addition is that topology became computable. Once the deck group is an object rather than a sentence, three things follow immediately that were previously out of reach: two spacetimes with identical metrics can be given opposite verdicts and the reason can be printed; Misner space can be computed twice by unrelated engines and the answers compared; and the polarised hypersurfaces of Grant space — the loci a quantum field's stress tensor is singular on — can be located automatically and handed to the semiclassical layer.

What has not changed is the honest position on what remains undone. There is no evolution layer, so no model in this registry may be reported above claim level L5, and the code enforces that rather than merely stating it. There is no second computer-algebra stack, so FM-018 is only partially mitigated. The corpus is 33 models, not the 100+ that P4 requires before cross-family universality claims are permitted. And the meta-analysis in this release disqualifies its own top-scoring rule as a curation artefact, which is the correct behaviour and also a reminder of how little a corpus of this size can support.

The standing claim boundary

Nothing in this document is evidence that a physically realisable time machine exists, or that any registered geometry can be built. A LEVEL 2 mathematically verified closed timelike curve is a statement about a tensor field on a manifold. It is not a machine for travelling to the past, and the architecture is built so that it cannot accidentally say otherwise.

27. References

[R1]	Einstein, A. (1915). Die Feldgleichungen der Gravitation. Sitzungsberichte der Preussischen Akademie der Wissenschaften.
[R2]	Gödel, K. (1949). An Example of a New Type of Cosmological Solutions of Einstein's Field Equations of Gravitation. Reviews of Modern Physics 21, 447. DOI: 10.1103/RevModPhys.21.447.
[R3]	Kerr, R. P. (1963). Gravitational Field of a Spinning Mass as an Example of Algebraically Special Metrics. Physical Review Letters 11, 237. DOI: 10.1103/PhysRevLett.11.237.
[R4]	Morris, M. S.; Thorne, K. S. (1988). Wormholes in spacetime and their use for interstellar travel. American Journal of Physics 56, 395. DOI: 10.1119/1.15620.
[R5]	Morris, M. S.; Thorne, K. S.; Yurtsever, U. (1988). Wormholes, Time Machines, and the Weak Energy Condition. Physical Review Letters 61, 1446. DOI: 10.1103/PhysRevLett.61.1446.
[R6]	Rebouças, M. J.; Tiomno, J. (1983). Homogeneity of Riemannian space-times of Gödel type. Physical Review D 28, 1251. DOI: 10.1103/PhysRevD.28.1251.
[R7]	Tipler, F. J. (1974). Rotating cylinders and the possibility of global causality violation. Physical Review D 9, 2203. DOI: 10.1103/PhysRevD.9.2203.
[R8]	Gott, J. R. III (1991). Closed timelike curves produced by pairs of moving cosmic strings: Exact solutions. Physical Review Letters 66, 1126. DOI: 10.1103/PhysRevLett.66.1126.
[R9]	Hawking, S. W. (1992). Chronology protection conjecture. Physical Review D 46, 603. DOI: 10.1103/PhysRevD.46.603.
[R10]	Friedman, J. L.; Schleich, K.; Witt, D. M. (1993). Topological censorship. Physical Review Letters 71, 1486; Erratum 75, 1872 (1995). DOI: 10.1103/PhysRevLett.71.1486.
[R11]	Kay, B. S.; Radzikowski, M. J.; Wald, R. M. (1997). Quantum Field Theory on Spacetimes with a Compactly Generated Cauchy Horizon. Communications in Mathematical Physics 183, 533. DOI: 10.1007/s002200050042.
[R12]	Alcubierre, M. (1994). The warp drive: hyper-fast travel within general relativity. Classical and Quantum Gravity 11, L73. DOI: 10.1088/0264-9381/11/5/001.
[R13]	Krasnikov, S. V. (1998). Hyperfast travel in general relativity. Physical Review D 57, 4760. DOI: 10.1103/PhysRevD.57.4760.
[R14]	Ori, A. (2005). A Class of Time-Machine Solutions with a Compact Vacuum Core. Physical Review Letters 95, 021101. DOI: 10.1103/PhysRevLett.95.021101.
[R15]	Ori, A. (2007). Formation of closed timelike curves in a composite vacuum/dust asymptotically flat spacetime. Physical Review D 76, 044002. DOI: 10.1103/PhysRevD.76.044002.
[R16]	Minguzzi, E.; Sánchez, M. (2006/2008). The causal hierarchy of spacetimes. arXiv:gr-qc/0609119; in Recent Developments in Pseudo-Riemannian Geometry.
[R17]	Rieger, N. E. (2026). Causal classification of pathological Misner-type spacetimes. Annals of Physics 489, 170455. DOI: 10.1016/j.aop.2026.170455. arXiv v1: 1 January 2025, arXiv:2501.00841.
[R18]	Kontou, E.-A. (2024). Wormhole Restrictions from Quantum Energy Inequalities. Universe 10(7), 291. DOI: 10.3390/universe10070291.
[R19]	Gao, P.; Jafferis, D. L.; Wall, A. C. (2017). Traversable wormholes via a double trace deformation. Journal of High Energy Physics 2017, 151. DOI: 10.1007/JHEP12(2017)151.
[R20]	Löffler, F. et al. (2012). The Einstein Toolkit: a community computational infrastructure for relativistic astrophysics. Classical and Quantum Gravity 29, 115001. DOI: 10.1088/0264-9381/29/11/115001.
[R21]	Einstein Toolkit project. Official project documentation: einstein toolkit.org .
[R22]	EinsteinPy project. Symbolic Relativity and MetricTensor API documentation: docs.einsteinpy.org .
[R23]	Martín-García, J. M. et al. xAct: Efficient tensor computer algebra for the Wolfram Language. xact.es .
[R24]	Maplesoft. Maple Physics database of solutions to Einstein's equations.
[R25]	Moré, J. J.; Sorensen, D. C. (1983). Computing a trust region step. SIAM Journal on Scientific and Statistical Computing 4, 553. DOI: 10.1137/0904038.
[R26]	Wald, R. M. (1984). General Relativity. University of Chicago Press.
[R27]	Hawking, S. W.; Ellis, G. F. R. (1973). The Large Scale Structure of Space-Time. Cambridge University Press.
[R28]	Misner, C. W.; Thorne, K. S.; Wheeler, J. A. (1973). Gravitation. W. H. Freeman.
[R29]	Grant, J. D. E. (1993). Cosmic strings and chronology protection. Physical Review D 47, 2388. DOI: 10.1103/PhysRevD.47.2388.
[R30]	Bañados, M.; Teitelboim, C.; Zanelli, J. (1992). Black hole in three-dimensional spacetime. Physical Review Letters 69, 1849. DOI: 10.1103/PhysRevLett.69.1849.

[R31]	Moore, R. E.; Kearfott, R. B.; Cloud, M. J. (2009). Introduction to Interval Analysis. SIAM.
[R32]	York, J. W. (1999). Conformal 'thin-sandwich' data for the initial-value problem of general relativity. Physical Review Letters 82, 1350. DOI: 10.1103/PhysRevLett.82.1350.
[R33]	Ford, L. H.; Roman, T. A. (1995). Averaged energy conditions and quantum inequalities. Physical Review D 51, 4277. DOI: 10.1103/PhysRevD.51.4277.

Appendix A. Complete Reference Source

The entire shipped system, reproduced verbatim. Line numbers are continuous across the appendix so that any line in the document can be cited unambiguously. This listing is the object the source digest in Section 20.3 is computed over.

Part	Contents
A.1 Kernel	src/ctcfa/ — 29 modules, in dependency order
A.2 Tests	8 files, 118 tests
A.3 Examples	10 runnable worked examples plus their bootstrap
A.4 Build and schemas	7 files

A.1 Kernel — src/ctcfa/

src/ctcfa/_init_.py — 36 lines

```
0001 """
0002 CTC-FA v2.0 -- Closed Timelike Curve Falsification Architecture.
0003
0004 A reproducible computational research standard for causal-structure
0005 classification: CTC birth/destruction boundaries, energy conditions,
0006 stability, constructibility, quantum admissibility and accessibility.
0007
0008 System architect: Vadym Tsinderhoz.
0009
0010 This package is research infrastructure. It is NOT evidence that a
0011 physically realisable time machine exists, and no layer of it may be quoted
0012 as such. Every verdict carries a scope and a claim level, and the reporting
0013 layer refuses to print a statement stronger than the evidence recorded for
0014 it.
0015 """
0016
0017 from __future__ import annotations
0018
0019 __version__ = "2.0.0"
0020 __architect__ = "Vadym Tsinderhoz"
0021 __document_id__ = "CTC-FA-2026-V2.0-EN"
0022
0023 from .status import (CLAIM_TEXT, ClaimLevel, ClaimScope, CTCStatus, Gate,
0024                     GateRecord, GateResult, ModelKind, OrbitResult, OrbitStatus,
0025                     OverclaimError, Verdict)
0026 from .model import (CompactDirection, DeckTransformation, DomainConstraint,
0027                    MatterSpec, MetricModel, Provenance, Quotient)
0028
0029 __all__ = [
0030     "version", "architect", "document_id",
0031     "CTCStatus", "OrbitStatus", "ModelKind", "ClaimScope", "ClaimLevel",
0032     "Gate", "GateRecord", "GateResult", "Verdict", "OrbitResult",
0033     "OverclaimError", "CLAIM_TEXT",
0034     "MetricModel", "CompactDirection", "DeckTransformation", "Quotient",
0035     "DomainConstraint", "MatterSpec", "Provenance",
0036 ]
```

src/ctcfa/status.py — 389 lines

```
0037 """
0038 CTC-FA v2.0 -- L-1 : Status algebra, claim scope and the claim ladder.
0039
0040 This module contains no physics. It contains the *evidence logic* that every
0041 other layer is required to route its conclusions through.
0042
0043 Design rules enforced here
0044 -----
0045 R1 Asymmetric evidence. One verified witness proves existence. Failure to
0046 find a witness never proves absence. ``UNRESOLVED`` is therefore an
0047 absorbing state for every search-type engine.
0048 R2 Scope is a first-class part of a status. A result obtained in a compact
0049 sector, in a reduced model, or on a restricted domain can never be
0050 printed as a global result. ``ClaimScope`` is carried with the status and
0051 ``Verdict.promote`` is the *only* way scope can widen.
0052 R3 A model may never be reported above its weakest satisfied gate. The
0053 ``ClaimLadder`` computes the permitted level from the gate record and
0054 ``Verdict.render`` refuses to emit anything higher.
0055
0056 v1.0 defect closed by this module
0057 -----
0058 D-05 v1.0 stated the scope rules in prose (sections 2, 9.2, 14, Appendix D)
0059 but the kernel had no representation of scope: ``global_model_test``
0060 returned bare ``CTC_PROVED`` for the *reduced* time-shifted wormhole
0061 predicate, which is exactly the category error section 9.2 warns about.
0062 """
0063
0064 from __future__ import annotations
0065
0066 from dataclasses import dataclass, field
0067 from enum import Enum
0068 from typing import Any, Mapping
0069
0070
0071 # -----
0072 # Global causal status
```

```

0073 # -----
0074
0075 class CTCStatus(str, Enum):
0076     """Global causal-structure verdict for a declared model + domain."""
0077
0078     CTC_PROVED = "CTC_PROVED"
0079     """An explicit future-directed closed timelike curve, or a mathematically
0080     complete equivalent criterion, has been verified."""
0081
0082     CRITICAL = "CRITICAL"
0083     """A closed-null / marginal causal boundary has been established."""
0084
0085     NO_CTC_CERTIFIED = "NO_CTC_CERTIFIED"
0086     """A complete causal certificate (temporal function, or a theorem whose
0087     hypotheses are *all* verified) excludes CTCs on the full declared
0088     domain."""
0089
0090     UNRESOLVED = "UNRESOLVED"
0091     """Neither existence nor absence has been proved. No binary claim."""
0092
0093
0094 class OrbitStatus(str, Enum):
0095     """Causal character of a *single tested compact orbit sector*.
0096
0097     This is deliberately a different type from :class:`CTCStatus`. v1.0
0098     failure mode FM-002 is precisely the silent promotion of
0099     ``SECTOR_SPACELIKE`` to ``NO_CTC_CERTIFIED``; making the two enums
0100     non-interchangeable makes that promotion a type error.
0101
0102
0103     SECTOR_SPACELIKE = "SECTOR_SPACELIKE"
0104     SECTOR_NULL = "SECTOR_NULL"
0105     SECTOR_CTC = "SECTOR_CTC"
0106     SECTOR_UNRESOLVED = "SECTOR_UNRESOLVED"
0107
0108
0109 class ModelKind(str, Enum):
0110     """How the model is represented computationally."""
0111
0112     LOCAL_METRIC = "LOCAL_METRIC"
0113     """One canonical local 4x4 (or nxn) metric in one chart."""
0114
0115     QUOTIENT_GLOBAL = "QUOTIENT_GLOBAL"
0116     """Local metric plus a non-trivial deck group; global causal structure is
0117     a property of the quotient, not of the local tensor."""
0118
0119     COMPOSITE_GLOBAL = "COMPOSITE_GLOBAL"
0120     """Several regions glued, or a construction defined by a global route
0121     predicate. No single canonical local tensor is asserted."""
0122
0123     EXTERNAL_EXACT_SPEC = "EXTERNAL_EXACT_SPEC"
0124     """The published construction is global or source-specific. CTC-FA
0125     refuses to fabricate a local tensor for it (v1.0 REQ-016)."""
0126
0127
0128 class ClaimScope(str, Enum):
0129     """Where a verdict is valid. Ordered from weakest to strongest."""
0130
0131     SECTOR = "SECTOR"
0132     """Only in the scanned compact-generator sector."""
0133
0134     REDUCED_MODEL = "REDUCED_MODEL"
0135     """Only under an architecture-specific reduced predicate (for example the
0136     time-shift route-closure model of section 9.2)."""
0137
0138     DECLARED_DOMAIN = "DECLARED_DOMAIN"
0139     """On the coordinate/parameter domain declared in the registry record."""
0140
0141     GLOBAL = "GLOBAL"
0142     """On the maximal declared spacetime, including its global
0143     identifications."""
0144
0145     _SCOPE_ORDER = {
0146         ClaimScope.SECTOR: 0,
0147         ClaimScope.REDUCED_MODEL: 1,
0148         ClaimScope.DECLARED_DOMAIN: 2,
0149         ClaimScope.GLOBAL: 3,
0150     }
0151
0152
0153 def scope_rank(s: ClaimScope) -> int:
0154     return _SCOPE_ORDER[s]
0155
0156
0157 def weaker_scope(a: ClaimScope, b: ClaimScope) -> ClaimScope:
0158     """Return the weaker of two scopes. Composition never strengthens."""
0159     return a if scope_rank(a) <= scope_rank(b) else b
0160
0161
0162 # -----
0163 # Verification gates and the claim ladder
0164 # -----
0165
0166 class Gate(str, Enum):
0167     """The verification gates of section 15.1 (V0-V9), as machine objects."""
0168
0169     V0_PARSE = "V0_PARSE"
0170     V1_ALGEBRA = "V1_ALGEBRA"
0171     V2_TENSOR = "V2_TENSOR"
0172     V3_FIELD_EQUATION = "V3_FIELD_EQUATION"
0173     V4_CTC_WITNESS = "V4_CTC_WITNESS"
0174     V5_BOUNDARY = "V5_BOUNDARY"
0175     V6_NEGATIVE_CERTIFICATE = "V6_NEGATIVE_CERTIFICATE"
0176     V7_DYNAMICS = "V7_DYNAMICS"
0177     V8_QUANTUM = "V8_QUANTUM"
0178     V9_REPLICATION = "V9_REPLICATION"
0179
0180
0181 class GateResult(str, Enum):

```

```

0183     PASS = "PASS"
0184     FAIL = "FAIL"
0185     NOT_ATTEMPTED = "NOT_ATTEMPTED"
0186     NOT_APPLICABLE = "NOT_APPLICABLE"
0187     INCONCLUSIVE = "INCONCLUSIVE"
0188
0189
0190 class ClaimLevel(int, Enum):
0191     """Appendix D claim ladder, now machine-enforced."""
0192
0193     L0_PARSED = 0
0194     L1_FIELD_EQUATIONS = 1
0195     L2_CTC_VERIFIED = 2
0196     L3_BOUNDARY_CHARACTERISED = 3
0197     L4_REGULARITY = 4
0198     L5_ENERGY = 5
0199     L6_CONSTRUCTIBILITY = 6
0200     L7_STABILITY = 7
0201     L8_SEMICLASSICAL = 8
0202     L9_QUANTUM_GRAVITY = 9
0203
0204
0205 CLAIM_TEXT: Mapping[ClaimLevel, str] = {
0206     ClaimLevel.L0_PARSED: "Metric/model parsed and internally consistent.",
0207     ClaimLevel.L1_FIELD_EQUATIONS: "Field equations / tensor identities verified in declared scope.",
0208     ClaimLevel.L2_CTC_VERIFIED: "CTC mathematically verified.",
0209     ClaimLevel.L3_BOUNDARY_CHARACTERISED: "CTC region and causal boundary characterised.",
0210     ClaimLevel.L4_REGULARITY: "Regularity / geodesic-completeness status established.",
0211     ClaimLevel.L5_ENERGY: "Energy requirements established.",
0212     ClaimLevel.L6_CONSTRUCTIBILITY: "Formation from regular admissible initial data established.",
0213     ClaimLevel.L7_STABILITY: "Perturbative/nonlinear stability established.",
0214     ClaimLevel.L8_SEMICLASSICAL: "Semiclassical consistency established.",
0215     ClaimLevel.L9_QUANTUM_GRAVITY: "Quantum-gravity consistency established.",
0216 }
0217
0218 #: Which gate must PASS before a claim level may be asserted.
0219 LADDER_REQUIREMENT: Mapping[ClaimLevel, tuple[Gate, ...]] = {
0220     ClaimLevel.L0_PARSED: (Gate.V0_PARSE,),
0221     ClaimLevel.L1_FIELD_EQUATIONS: (Gate.V0_PARSE, Gate.V1_ALGEBRA, Gate.V2_TENSOR,
0222                                     Gate.V3_FIELD_EQUATION),
0223     ClaimLevel.L2_CTC_VERIFIED: (Gate.V0_PARSE, Gate.V1_ALGEBRA, Gate.V4_CTC_WITNESS),
0224     ClaimLevel.L3_BOUNDARY_CHARACTERISED: (Gate.V0_PARSE, Gate.V1_ALGEBRA, Gate.V5_BOUNDARY),
0225     ClaimLevel.L4_REGULARITY: (Gate.V0_PARSE, Gate.V1_ALGEBRA, Gate.V2_TENSOR),
0226     ClaimLevel.L5_ENERGY: (Gate.V0_PARSE, Gate.V1_ALGEBRA, Gate.V2_TENSOR,
0227                           Gate.V3_FIELD_EQUATION),
0228     ClaimLevel.L6_CONSTRUCTIBILITY: (Gate.V0_PARSE, Gate.V3_FIELD_EQUATION, Gate.V7_DYNAMICS),
0229     ClaimLevel.L7_STABILITY: (Gate.V0_PARSE, Gate.V3_FIELD_EQUATION, Gate.V7_DYNAMICS),
0230     ClaimLevel.L8_SEMICLASSICAL: (Gate.V0_PARSE, Gate.V8_QUANTUM),
0231     ClaimLevel.L9_QUANTUM_GRAVITY: (), # unreachable by construction, see below
0232 }
0233
0234
0235 @dataclass
0236 class GateRecord:
0237     """The set of gate outcomes accumulated for one model + domain."""
0238
0239     results: dict[Gate, GateResult] = field(default_factory=dict)
0240     notes: dict[Gate, str] = field(default_factory=dict)
0241
0242     def set(self, gate: Gate, result: GateResult, note: str = "") -> "GateRecord":
0243         self.results[gate] = result
0244         if note:
0245             self.notes[gate] = note
0246         return self
0247
0248     def get(self, gate: Gate) -> GateResult:
0249         return self.results.get(gate, GateResult.NOT_ATTEMPTED)
0250
0251     def passed(self, gate: Gate) -> bool:
0252         return self.get(gate) is GateResult.PASS
0253
0254     def highest_permitted_level(self) -> ClaimLevel:
0255         """Return the highest claim level whose *entire* requirement set passed.
0256
0257         L9 is unreachable: no accepted complete theory of quantum gravity
0258         exists, so the architecture may never emit it. This is enforced, not
0259         merely documented.
0260         """
0261         best = ClaimLevel.L0_PARSED if self.passed(Gate.V0_PARSE) else ClaimLevel.L0_PARSED
0262         reached = []
0263         for level, req in LADDER_REQUIREMENT.items():
0264             if level is ClaimLevel.L9_QUANTUM_GRAVITY:
0265                 continue
0266             if req and all(self.passed(g) for g in req):
0267                 reached.append(level)
0268             if reached:
0269                 best = max(reached, key=lambda lv: lv.value)
0270         elif not self.passed(Gate.V0_PARSE):
0271             best = ClaimLevel.L0_PARSED
0272         return best
0273
0274     def as_dict(self) -> dict[str, Any]:
0275         return {
0276             "results": {g.value: r.value for g, r in sorted(
0277                 self.results.items(), key=lambda kv: kv[0].value)},
0278             "notes": {g.value: n for g, n in sorted(
0279                 self.notes.items(), key=lambda kv: kv[0].value)},
0280             "highest_permitted_level": self.highest_permitted_level().name,
0281         }
0282
0283
0284 # -----
0285 # Verdict
0286 # -----
0287
0288 class OverclaimError(RuntimeError):
0289     """Raised when code attempts to state a result above its evidence."""
0290
0291
0292 @dataclass

```

```

0293 class Verdict:
0294     """A causal verdict together with everything needed to bound it.
0295
0296     ``Verdict`` is the only object the reporting layer accepts. A bare
0297     :class:`CTCStatus` cannot be rendered.
0298     """
0299
0300     status: CTCStatus
0301     scope: ClaimScope
0302     margin: float
0303     method: str
0304     reason: str
0305     model_id: str = ""
0306     parameters: Mapping[str, Any] = field(default_factory=dict)
0307     gates: GateRecord = field(default_factory=GateRecord)
0308     evidence: Mapping[str, Any] = field(default_factory=dict)
0309     uncertainty: float = 0.0
0310
0311     # -- scope algebra -----
0312
0313     def restrict(self, scope: ClaimScope) -> "Verdict":
0314         """Narrow the scope. Always allowed."""
0315         return Verdict(self.status, weaker_scope(self.scope, scope), self.margin,
0316                        self.method, self.reason, self.model_id, self.parameters,
0317                        self.gates, self.evidence, self.uncertainty)
0318
0319     def promote(self, scope: ClaimScope, justification: str) -> "Verdict":
0320         """Widen the scope. Requires an explicit justification string.
0321
0322         Promotion of a *negative* result (``NO_CTC_CERTIFIED``) to
0323         ``GLOBAL`` requires a global certificate; promotion of a
0324         ``SECTOR``-scoped positive result to ``GLOBAL`` is permitted because
0325         an explicit witness in a sector of the declared spacetime *is* a
0326         global existence proof for that spacetime (asymmetric evidence, R1).
0327         """
0328         if scope_rank(scope) <= scope_rank(self.scope):
0329             return self
0330         if self.status is CTCStatus.NO_CTC_CERTIFIED and scope is ClaimScope.GLOBAL:
0331             if "certificate" not in justification.lower() and "theorem" not in justification.lower():
0332                 raise OverclaimError(
0333                     "Promotion of NO_CTC_CERTIFIED to GLOBAL requires a named "
0334                     "temporal-function certificate or a fully verified theorem; "
0335                     f"got justification={justification!r}")
0336             if self.status is CTCStatus.UNRESOLVED:
0337                 raise OverclaimError("UNRESOLVED can never be promoted; it is absorbing.")
0338             ev = dict(self.evidence)
0339             ev["scope_promotion"] = justification
0340             return Verdict(self.status, scope, self.margin, self.method, self.reason,
0341                           self.model_id, self.parameters, self.gates, ev, self.uncertainty)
0342
0343     # -- rendering -----
0344
0345     def permitted_level(self) -> ClaimLevel:
0346         return self.gates.highest_permitted_level()
0347
0348     def render(self, requested_level: ClaimLevel | None = None) -> str:
0349         """Render a one-line statement, refusing to exceed the evidence."""
0350         allowed = self.permitted_level()
0351         lvl = requested_level if requested_level is not None else allowed
0352         if lvl.value > allowed.value:
0353             raise OverclaimError(
0354                 f"{self.model_id}: requested claim level {lvl.name} exceeds the "
0355                 f"highest level supported by the gate record ({allowed.name}).")
0356         return (f"{self.model_id or '<model>'}: {self.status.value} "
0357               f"[scope={self.scope.value}, level={lvl.name}, "
0358               f"margin={self.margin:+.6e}, u={self.uncertainty:.2e}] "
0359               f"via {self.method} -- {self.reason}")
0360
0361     def as_dict(self) -> dict[str, Any]:
0362         return {
0363             "model_id": self.model_id,
0364             "status": self.status.value,
0365             "scope": self.scope.value,
0366             "margin": self.margin,
0367             "uncertainty": self.uncertainty,
0368             "method": self.method,
0369             "reason": self.reason,
0370             "parameters": dict(self.parameters),
0371             "gates": self.gates.as_dict(),
0372             "evidence": dict(self.evidence),
0373             "permitted_claim_level": self.permitted_level().name,
0374         }
0375
0376     # -----
0377     # Sector result
0378     # -----
0379
0380 @dataclass
0381 class OrbitResult:
0382     """Result of the compact-orbit detector for one sector."""
0383
0384     status: OrbitStatus
0385     margin: float
0386     winding: tuple[int, ...] | None
0387     point: Mapping[str, float]
0388     reason: str
0389     uncertainty: float = 0.0
0390     generator_names: tuple[str, ...] = ()
0391     orientation_consistent: bool | None = None
0392     isometry_verified: bool | None = None
0393
0394     def to_verdict(self, model_id: str, gates: GateRecord | None = None) -> Verdict:
0395         """Lift a sector result to a *sector-scoped* verdict.
0396
0397         This is the only bridge from :class:`OrbitStatus` to
0398         :class:`CTCStatus`, and it is deliberately lossy in one direction:
0399         ``SECTOR_SPACELIKE`` maps to ``UNRESOLVED``, never to
0400         ``NO_CTC_CERTIFIED`` (v1.0 REQ-005 / FM-002).
0401         """

```

```

0403     gates = gates or GateRecord()
0404     if self.status is OrbitStatus.SECTOR_CTC:
0405         st, scope = CTCStatus.CTC_PROVED, ClaimScope.SECTOR
0406     elif self.status is OrbitStatus.SECTOR_NULL:
0407         st, scope = CTCStatus.CRITICAL, ClaimScope.SECTOR
0408     else:
0409         st, scope = CTCStatus.UNRESOLVED, ClaimScope.SECTOR
0410     return Verdict(status=st, scope=scope, margin=self.margin,
0411                   method="compact_orbit", reason=self.reason,
0412                   model_id=model_id, parameters=dict(self.point),
0413                   gates=gates, uncertainty=self.uncertainty,
0414                   evidence={"winding": self.winding,
0415                             "generators": list(self.generator_names),
0416                             "orientation_consistent": self.orientation_consistent,
0417                             "isometry_verified": self.isometry_verified,
0418                             "orbit_status": self.status.value})
0419
0420
0421 __all__ = [
0422     "CTCStatus", "OrbitStatus", "ModelKind", "ClaimScope", "Gate", "GateResult",
0423     "GateRecord", "ClaimLevel", "CLAIM_TEXT", "LADDER_REQUIREMENT", "Verdict",
0424     "OrbitResult", "OverclaimError", "scope_rank", "weaker_scope",
0425 ]

```

src/ctcfa/numerics.py — 273 lines

```

0426 """
0427 CTC-FA v2.0 -- Numerical precision, tolerance and uncertainty policy
0428 (document section 29), implemented rather than described.
0429
0430 Rules enforced
0431 -----
0432 N1 No unqualified sign decision near zero. Every causal sign test goes
0433 through :func:`classify_sign`, which takes an *uncertainty budget* and
0434 returns ``NEGATIVE``, ``MARGINAL`` or ``POSITIVE``. A raw value of
0435 ``-1e-15`` with a budget of ``1e-12`` is ``MARGINAL``, never a CTC.
0436 N2 Automatic precision escalation. When ``|q| <= eps_total`` the quantity is
0437 recomputed symbolically at 50 significant digits with mpmath before the
0438 verdict is issued. v1.0 named this policy (FM-017) but the kernel had
0439 no escalation path.
0440 N3 Uncertainty budgets are *composed*, not guessed: :class:`ErrorBudget`
0441 accumulates round-off, substitution, conditioning and (where used)
0442 integration and interpolation terms.
0443 N4 A reported threshold always carries its final sign-changing bracket
0444 (section 29.3), never only the root-finder tolerance.
0445 """
0446
0447 from __future__ import annotations
0448
0449 import math
0450 from dataclasses import dataclass, field
0451 from enum import Enum
0452 from typing import Any, Callable, Mapping, Sequence
0453
0454 import numpy as np
0455 import sympy as sp
0456 from mpmath import mp
0457
0458 DEFAULT_TOL = 1e-10
0459 HIGH_PRECISION_DIGITS = 50
0460
0461
0462 class SignClass(str, Enum):
0463     NEGATIVE = "NEGATIVE"
0464     MARGINAL = "MARGINAL"
0465     POSITIVE = "POSITIVE"
0466
0467
0468 @dataclass
0469 class ErrorBudget:
0470     """Composable uncertainty budget for one scalar decision."""
0471
0472     roundoff: float = 0.0
0473     substitution: float = 0.0
0474     conditioning: float = 0.0
0475     integration: float = 0.0
0476     interpolation: float = 0.0
0477     optimization: float = 0.0
0478     parameter: float = 0.0
0479     floor: float = 1e-14
0480     notes: list[str] = field(default_factory=list)
0481
0482     @property
0483     def total(self) -> float:
0484         return max(self.floor,
0485                   self.roundoff + self.substitution + self.conditioning
0486                   + self.integration + self.interpolation + self.optimization
0487                   + self.parameter)
0488
0489     def add(self, **kw: float) -> "ErrorBudget":
0490         for k, v in kw.items():
0491             if not hasattr(self, k):
0492                 raise KeyError(f"unknown budget term {k!r}")
0493             setattr(self, k, getattr(self, k) + float(v))
0494         return self
0495
0496     def as_dict(self) -> dict[str, Any]:
0497         return {"roundoff": self.roundoff, "substitution": self.substitution,
0498               "conditioning": self.conditioning, "integration": self.integration,
0499               "interpolation": self.interpolation, "optimization": self.optimization,
0500               "parameter": self.parameter, "total": self.total,
0501               "notes": list(self.notes)}
0502
0503
0504 def matrix_conditioning(A: np.ndarray) -> float:
0505     """A cheap conditioning proxy used to inflate the round-off term."""
0506     try:
0507         c = float(np.linalg.cond(A))

```

```

0508     except Exception:                                     # noqa: BLE001
0509         return float("inf")
0510     if not math.isfinite(c):
0511         return float("inf")
0512     return c
0513
0514
0515 def budget_for_matrix(A: np.ndarray, scale: float = 1.0) -> ErrorBudget:
0516     """Round-off budget for a quantity built from the matrix ``A``."""
0517     eps = float(np.finfo(np.float64).eps)
0518     cond = matrix_conditioning(A)
0519     b = ErrorBudget()
0520     nrm = float(np.max(np.abs(A))) if A.size else 1.0
0521     b.roundoff = eps * max(1.0, nrm) * max(1.0, abs(scale)) * A.shape[0]
0522     if math.isfinite(cond):
0523         b.conditioning = eps * cond * max(1.0, nrm)
0524         b.notes.append(f"cond={cond:.3e}")
0525     else:
0526         b.conditioning = float("inf")
0527         b.notes.append("cond=inf (singular or near-singular metric)")
0528     return b
0529
0530
0531 def classify_sign(value: float, budget: ErrorBudget | float) -> SignClass:
0532     """Sign decision with an explicit uncertainty band (rule N1)."""
0533     eps = budget.total if isinstance(budget, ErrorBudget) else float(budget)
0534     if not math.isfinite(value):
0535         return SignClass.MARGINAL
0536     if value < -eps:
0537         return SignClass.NEGATIVE
0538     if value > eps:
0539         return SignClass.POSITIVE
0540     return SignClass.MARGINAL
0541
0542
0543 def high_precision_value(expr: sp.Expr, subs: Mapping[sp.Symbol, float],
0544                         digits: int = HIGH_PRECISION_DIGITS) -> float:
0545     """Re-evaluate a symbolic expression at high precision (rule N2)."""
0546     old = mp.dps
0547     try:
0548         mp.dps = digits
0549         rational = {k: sp.nsimplify(sp.Float(v, digits), rational=True)
0550                     for k, v in subs.items()}
0551         val = sp.N(expr.subs(rational), digits)
0552         return float(val)
0553     finally:
0554         mp.dps = old
0555
0556
0557 def escalate_if_marginal(expr: sp.Expr, subs: Mapping[sp.Symbol, float],
0558                         value: float, budget: ErrorBudget,
0559                         digits: int = HIGH_PRECISION_DIGITS) -> tuple[float, ErrorBudget, bool]:
0560     """If ``value`` is inside the uncertainty band, recompute exactly.
0561
0562     Returns ``(value, budget, escalated)``. On escalation the round-off and
0563     conditioning terms are replaced by the high-precision floor, because the
0564     quantity no longer passes through double-precision linear algebra.
0565     """
0566     if classify_sign(value, budget) is not SignClass.MARGINAL:
0567         return value, budget, False
0568     try:
0569         hp = high_precision_value(expr, subs, digits)
0570     except Exception as exc:                                     # noqa: BLE001
0571         budget.notes.append(f"escalation failed: {exc}")
0572         return value, budget, False
0573     nb = ErrorBudget(floor=10.0 * (-(digits - 6)))
0574     nb.notes = list(budget.notes) + [f"escalated to {digits} digits"]
0575     nb.parameter = budget.parameter
0576     return hp, nb, True
0577
0578
0579
0580 # -----
0581 # Signature
0582 # -----
0583
0584 def signature_counts(g: np.ndarray, tol: float = 1e-10) -> tuple[int, int, int]:
0585     """Return ``(n_negative, n_null, n_positive)`` by Sylvester's law.
0586
0587     The *counts* of eigenvalue signs of a real symmetric matrix are invariant
0588     under congruence, so this is a coordinate-independent statement even
0589     though the eigenvalues themselves are not.
0590     """
0591     vals = np.linalg.eigvalsh((g + g.T) / 2.0)
0592     scale = max(1.0, float(np.max(np.abs(vals))))
0593     t = tol * scale
0594     return (int(np.sum(vals < -t)), int(np.sum(np.abs(vals) <= t)),
0595           int(np.sum(vals > t)))
0596
0597
0598 def is_lorentzian(g: np.ndarray, tol: float = 1e-10) -> bool:
0599     n = g.shape[0]
0600     return signature_counts(g, tol) == (1, 0, n - 1)
0601
0602
0603 # -----
0604 # Bracketed root finding with a reported bracket (rule N4)
0605 # -----
0606
0607 @dataclass
0608 class ThresholdResult:
0609     """A causal threshold together with everything section 29.3 requires."""
0610
0611     value: float
0612     bracket: tuple[float, float]
0613     left_margin: float
0614     right_margin: float
0615     residual: float
0616     iterations: int
0617     method: str

```

```

0618     analytic_value: float | None = None
0619     analytic_form: str = ""
0620     domain_ok: bool = True
0621     note: str = ""
0622
0623     @property
0624     def bracket_width(self) -> float:
0625         return abs(self.bracket[1] - self.bracket[0])
0626
0627     @property
0628     def analytic_error(self) -> float | None:
0629         if self.analytic_value is None:
0630             return None
0631         return abs(self.value - self.analytic_value)
0632
0633     def as_dict(self) -> dict[str, Any]:
0634         return {"value": self.value, "bracket": list(self.bracket),
0635                 "bracket_width": self.bracket_width,
0636                 "left_margin": self.left_margin, "right_margin": self.right_margin,
0637                 "residual": self.residual, "iterations": self.iterations,
0638                 "method": self.method, "analytic_value": self.analytic_value,
0639                 "analytic_form": self.analytic_form,
0640                 "analytic_error": self.analytic_error,
0641                 "domain_ok": self.domain_ok, "note": self.note}
0642
0643
0644 def bracketed_bisection(f: Callable[[float], float], lo: float, hi: float,
0645                        xtol: float = 1e-13, maxiter: int = 400
0646                        ) -> tuple[float, tuple[float, float], int]:
0647     """Plain bisection that *returns the final bracket*, not only the root."""
0648     flo, fhi = f(lo), f(hi)
0649     if flo == 0.0:
0650         return lo, (lo, lo), 0
0651     if fhi == 0.0:
0652         return hi, (hi, hi), 0
0653     if flo * fhi > 0:
0654         raise ValueError(f"threshold is not bracketed on [{lo}, {hi}]: "
0655                          f"f(lo)={flo:.6e}, f(hi)={fhi:.6e}")
0656
0657     it = 0
0658     while (hi - lo) > xtol and it < maxiter:
0659         mid = 0.5 * (lo + hi)
0660         fm = f(mid)
0661         if fm == 0.0:
0662             return mid, (mid, mid), it
0663         if flo * fm < 0:
0664             hi, fhi = mid, fm
0665         else:
0666             lo, flo = mid, fm
0667         it += 1
0668     root = 0.5 * (lo + hi)
0669     return root, (lo, hi), it
0670
0671 def coarse_sign_scan(f: Callable[[float], float], lo: float, hi: float,
0672                    n: int = 64) -> list[tuple[float, float]]:
0673     """Return every sub-interval on which ``f`` changes sign."""
0674     xs = np.linspace(lo, hi, n + 1)
0675     vals = []
0676     for x in xs:
0677         try:
0678             vals.append(float(f(float(x))))
0679         except Exception:
0680             vals.append(float("nan"))
0681
0682     out = []
0683     for i in range(n):
0684         a, b = vals[i], vals[i + 1]
0685         if not (math.isfinite(a) and math.isfinite(b)):
0686             continue
0687         if a == 0.0:
0688             out.append((float(xs[i]), float(xs[i])))
0689         elif a * b < 0:
0690             out.append((float(xs[i]), float(xs[i + 1])))
0691     return out
0692
0693 __all__ = [
0694     "DEFAULT_TOL", "HIGH_PRECISION_DIGITS", "SignClass", "ErrorBudget",
0695     "budget_for_matrix", "matrix_conditioning", "classify_sign",
0696     "high_precision_value", "escalate_if_marginal", "signature_counts",
0697     "is_lorentzian", "ThresholdResult", "bracketed_bisection", "coarse_sign_scan",
0698 ]

```

src/ctcfa/interval.py — 392 lines

```

0699 """
0700 CTC-FA v2.0 -- Certified interval arithmetic and branch-and-bound bounds.
0701
0702 Why this module exists
0703 -----
0704 v1.0 repeatedly, and correctly, disqualified its own numerics:
0705
0706 L9 "Numerical directional sampling is diagnostic only; analytic/interval
0707     certification is required for formal PASS."
0708 FM-009 "Energy-condition result based on finite directional samples ->
0709     false proof."
0710 24.4 "For numerical tau, the infimum must be bounded with a method that
0711     controls interpolation and discretisation error. Point sampling
0712     alone does not establish the strict global inequality."
0713
0714 The architecture named the required tool -- interval certification -- in three
0715 places and shipped none. This module supplies it.
0716
0717 What it provides
0718 -----
0719 ``interval_eval`` rigorous enclosure of a sympy expression on a box,
0720 using ``mpmath.iv`` (correctly outward-rounded).
0721 ``global_lower_bound`` branch-and-bound *proof* that ``f(x) >= L`` on a
0722 compact box, or a counterexample point.

```



```

0723 ``prove_positive`` the certificate wrapper used by L7 (temporal function)
0724 and L9 (energy conditions).
0725
0726 Scope and honesty
0727 -----
0728 A branch-and-bound certificate is valid only on the compact box it was
0729 run on. It is therefore reported with that box attached, and the caller
0730 must decide whether the box is the declared physical domain. For unbounded
0731 domains an additional analytic argument (asymptotics, monotonicity, or a
0732 change of variable compactifying the domain) is required, and
0733 ``PositivityCertificate.domain_is_compact`` records which case applies.
0734 """
0735
0736 from __future__ import annotations
0737
0738 import math
0739 from dataclasses import dataclass, field
0740 from typing import Any, Callable, Iterable, Mapping, Sequence
0741
0742 import sympy as sp
0743 from mpmath import iv, mp
0744
0745 iv.prec = 80
0746
0747 class IntervalDomainError(ValueError):
0748     """The expression left its real domain somewhere on the box."""
0749
0750 # -----
0751 # Interval evaluation of a sympy expression
0752 # -----
0753
0754 _UNBOUNDED = None # sentinel meaning "no rigorous bound obtainable"
0755
0756 def _iv(a: float, b: float):
0757     return iv.mpf([a, b])
0758
0759 def _cosh(x):
0760     return (iv.exp(x) + iv.exp(-x)) / 2
0761
0762 def _sinh(x):
0763     return (iv.exp(x) - iv.exp(-x)) / 2
0764
0765 def _tanh(x):
0766     e2 = iv.exp(2 * x)
0767     return (e2 - 1) / (e2 + 1)
0768
0769 def interval_eval(expr: sp.Expr, box: Mapping[sp.Symbol, tuple[float, float]]):
0770     """Rigorous enclosure of ``expr`` over the box.
0771
0772     Raises :class:`IntervalDomainError` when the expression leaves its real
0773     domain (for example ``sqrt`` of an interval containing negatives), which
0774     the caller must treat as "domain not verified", never as "inequality
0775     holds".
0776     """
0777     def rec(e: sp.Expr):
0778         if e.is_Number:
0779             v = float(e)
0780             return _iv(v, v)
0781         if isinstance(e, sp.Symbol):
0782             if e not in box:
0783                 raise IntervalDomainError(f"symbol {e} has no interval assigned")
0784             lo, hi = box[e]
0785             return _iv(lo, hi)
0786         if e is sp.pi:
0787             return iv.pi
0788         if e is sp.E:
0789             return iv.e
0790         if e.is_Add:
0791             acc = _iv(0.0, 0.0)
0792             for a in e.args:
0793                 acc = acc + rec(a)
0794             return acc
0795         if e.is_Mul:
0796             acc = _iv(1.0, 1.0)
0797             for a in e.args:
0798                 acc = acc * rec(a)
0799             return acc
0800         if e.is_Pow:
0801             base, expo = e.args
0802             b = rec(base)
0803             if expo.is_Integer:
0804                 n = int(expo)
0805                 if n >= 0:
0806                     return b ** n
0807                 # negative integer power: 1 / b^|n|, undefined if 0 in b
0808                 d = b ** (-n)
0809                 if d.a <= 0 <= d.b:
0810                     raise IntervalDomainError("division by an interval containing 0")
0811                 return _iv(1.0, 1.0) / d
0812             if expo.is_Rational and expo.q == 2 and expo.p == 1:
0813                 if b.a < 0:
0814                     raise IntervalDomainError("sqrt of an interval containing negatives")
0815                 return iv.sqrt(b)
0816             ex = rec(expo)
0817             if b.a <= 0:
0818                 raise IntervalDomainError("non-integer power of a non-positive base")
0819             return iv.exp(ex * iv.log(b))
0820         f = e.func
0821         if f is sp.exp:
0822             return iv.exp(rec(e.args[0]))
0823         if f is sp.log:
0824             x = rec(e.args[0])
0825             if x.a <= 0:

```

```

0833         raise IntervalDomainError("log of an interval containing non-positives")
0834     return iv.log(x)
0835     if f is sp.sqrt:
0836         x = rec(e.args[0])
0837         if x.a < 0:
0838             raise IntervalDomainError("sqrt of an interval containing negatives")
0839         return iv.sqrt(x)
0840     if f is sp.sin:
0841         return iv.sin(rec(e.args[0]))
0842     if f is sp.cos:
0843         return iv.cos(rec(e.args[0]))
0844     if f is sp.tan:
0845         c = iv.cos(rec(e.args[0]))
0846         if c.a <= 0 <= c.b:
0847             raise IntervalDomainError("tan across a pole")
0848         return iv.sin(rec(e.args[0])) / c
0849     if f is sp.sinh:
0850         return _sinh(rec(e.args[0]))
0851     if f is sp.cosh:
0852         return _cosh(rec(e.args[0]))
0853     if f is sp.tanh:
0854         return _tanh(rec(e.args[0]))
0855     if f is sp.Abs:
0856         x = rec(e.args[0])
0857         lo, hi = float(x.a), float(x.b)
0858         if lo <= 0 <= hi:
0859             return _iv(0.0, max(abs(lo), abs(hi)))
0860         m, M = sorted((abs(lo), abs(hi)))
0861         return _iv(m, M)
0862     raise IntervalDomainError(f"no interval rule for {f.__name__}: {e}")
0863
0864     return rec(sp.sympify(expr))
0865
0866 # -----
0867 # Branch and bound
0868 # -----
0869
0870 @dataclass
0871 class BoundResult:
0872     """Certified bound on ``f`` over a compact box."""
0873     lower_bound: float
0874     upper_bound: float
0875     boxes_examined: int
0876     boxes_remaining: int
0877     converged: bool
0878     argmin_box: dict[str, tuple[float, float]] = field(default_factory=dict)
0879     domain_errors: int = 0
0880     note: str = ""
0881
0882     def as_dict(self) -> dict[str, Any]:
0883         return {"lower_bound": self.lower_bound, "upper_bound": self.upper_bound,
0884                 "boxes_examined": self.boxes_examined,
0885                 "boxes_remaining": self.boxes_remaining,
0886                 "converged": self.converged, "argmin_box": self.argmin_box,
0887                 "domain_errors": self.domain_errors, "note": self.note}
0888
0889 def global_lower_bound(expr: sp.Expr,
0890                       box: Mapping[sp.Symbol, tuple[float, float]],
0891                       target: float = 0.0,
0892                       max_boxes: int = 20000,
0893                       tol: float = 1e-9,
0894                       stop_on_proof: bool = True) -> BoundResult:
0895     """Prove ``inf_box expr >= target`` (or find where it fails).
0896
0897     Standard interval branch-and-bound: a sub-box whose enclosure lies
0898     entirely above ``target`` is discarded; otherwise it is bisected along
0899     its widest dimension. If ``stop_on_proof`` and every box is discarded,
0900     the inequality is proved on the whole box.
0901
0902     The returned ``lower_bound`` is a rigorous lower bound on the infimum;
0903     ``upper_bound`` is the best value actually attained at a sampled point
0904     (so ``lower_bound <= inf f <= upper_bound``).
0905
0906     """
0907     syms = list(box.keys())
0908     expr = sp.sympify(expr)
0909     f_pt = sp.lambdify(syms, expr, "math")
0910
0911     def sample(b: Mapping[sp.Symbol, tuple[float, float]]) -> float:
0912         mid = [0.5 * (b[s][0] + b[s][1]) for s in syms]
0913         try:
0914             v = float(f_pt(*mid))
0915             return v if math.isfinite(v) else float("inf")
0916         except Exception:
0917             return float("inf")
0918
0919     start = {s: (float(box[s][0]), float(box[s][1])) for s in syms}
0920     best_upper = sample(start)
0921     stack: list[dict[Any, tuple[float, float]]] = [start]
0922     examined = 0
0923     domain_errors = 0
0924     worst_lower = float("inf")
0925     argmin_box: dict[str, tuple[float, float]] = {}
0926
0927     while stack and examined < max_boxes:
0928         b = stack.pop()
0929         examined += 1
0930         try:
0931             enc = interval_eval(expr, b)
0932             lo, hi = float(enc.a), float(enc.b)
0933             except IntervalDomainError:
0934                 domain_errors += 1
0935             lo, hi = -math.inf, math.inf
0936             except Exception:
0937                 domain_errors += 1
0938             lo, hi = -math.inf, math.inf
0939
0940         if math.isfinite(hi):

```

```

0943         v = sample(b)
0944         if v < best_upper:
0945             best_upper = v
0946
0947         if lo > target + tol:
0948             continue # box cannot violate
0949
0950         widths = [(b[s][1] - b[s][0], s) for s in syms]
0951         w, s = max(widths, key=lambda t: t[0])
0952         if w <= tol:
0953             if lo < worst_lower:
0954                 worst_lower = lo
0955                 argmin_box = {str(k): v for k, v in b.items()}
0956             if stop_on_proof and hi < target - tol:
0957                 # a whole small box is strictly below target: refutation
0958                 return BoundResult(lo, hi, examined, len(stack), True,
0959                                   {str(k): v for k, v in b.items()},
0960                                   domain_errors,
0961                                   "counterexample box found: expression is "
0962                                   "strictly below the target here")
0963             continue
0964
0965         mid = 0.5 * (b[s][0] + b[s][1])
0966         left = dict(b); left[s] = (b[s][0], mid)
0967         right = dict(b); right[s] = (mid, b[s][1])
0968         stack.append(left)
0969         stack.append(right)
0970
0971         if not stack:
0972             lb = target if not math.isfinite(worst_lower) else min(worst_lower, target)
0973             return BoundResult(max(lb, worst_lower if math.isfinite(worst_lower) else target),
0974                               best_upper, examined, 0, True, argmin_box, domain_errors,
0975                               "all boxes discarded: inequality proved on the box"
0976                               if domain_errors == 0 else
0977                               "all boxes discarded, but some sub-boxes left the real "
0978                               "domain and could not be enclosed")
0979         return BoundResult(worst_lower if math.isfinite(worst_lower) else -math.inf,
0980                           best_upper, examined, len(stack), False, argmin_box,
0981                           domain_errors,
0982                           f"budget exhausted after {examined} boxes; "
0983                           f"{len(stack)} undecided boxes remain")
0984
0985 # -----
0986 # Positivity certificate
0987 # -----
0988
0989 @dataclass
0990 class PositivityCertificate:
0991     """Outcome of trying to prove `expr > 0` on a declared domain."""
0992
0993     proved: bool
0994     refuted: bool
0995     method: str
0996     box: dict[str, tuple[float, float]] = field(default_factory=dict)
0997     domain_is_compact: bool = True
0998     lower_bound: float | None = None
0999     counterexample: dict[str, float] | None = None
1000     symbolic_note: str = ""
1001     detail: dict[str, Any] = field(default_factory=dict)
1002
1003 @property
1004 def status(self) -> str:
1005     if self.proved:
1006         return "CERTIFIED"
1007     if self.refuted:
1008         return "REFUTED"
1009     return "UNRESOLVED"
1010
1011 def as_dict(self) -> dict[str, Any]:
1012     return {"status": self.status, "proved": self.proved, "refuted": self.refuted,
1013           "method": self.method, "box": self.box,
1014           "domain_is_compact": self.domain_is_compact,
1015           "lower_bound": self.lower_bound,
1016           "counterexample": self.counterexample,
1017           "symbolic_note": self.symbolic_note, "detail": self.detail}
1018
1019
1020 def prove_positive(expr: sp.Expr,
1021                  box: Mapping[sp.Symbol, tuple[float, float]] | None = None,
1022                  assumptions: Mapping[sp.Symbol, dict[str, bool]] | None = None,
1023                  strict: bool = True,
1024                  max_boxes: int = 20000,
1025                  domain_is_compact: bool = True) -> PositivityCertificate:
1026     """Try symbolic positivity first, then interval branch-and-bound.
1027
1028     ``assumptions`` lets the caller re-declare symbols with domain knowledge
1029     (for example ``{r: {"positive": True}}``) before the symbolic attempt.
1030
1031     """
1032     e = sp.sympify(expr)
1033     note = ""
1034
1035     if assumptions:
1036         ren = {}
1037         for s, kw in assumptions.items():
1038             ren[s] = sp.Symbol(str(s), **kw)
1039         e_assumed = e.subs(ren, simultaneous=True)
1040     else:
1041         e_assumed = e
1042
1043     simp = sp.simplify(e_assumed)
1044     if simp.is_positive is True:
1045         return PositivityCertificate(True, False, "symbolic",
1046                                   {str(k): v for k, v in (box or {}).items()},
1047                                   domain_is_compact,
1048                                   symbolic_note=f"sympy proved {simp} > 0")
1049     if simp.is_negative is True:
1050         return PositivityCertificate(False, True, "symbolic",
1051                                   {str(k): v for k, v in (box or {}).items()},
1052                                   domain_is_compact,

```

```

1053             symbolic_note=f"sympy proved {simp} < 0")
1054     if not strict and simp.is_nonnegative is True:
1055         return PositivityCertificate(True, False, "symbolic",
1056                                     {str(k): v for k, v in (box or {}).items()}),
1057                                     domain_is_compact,
1058                                     symbolic_note=f"sympy proved {simp} >= 0")
1059     note = f"symbolic attempt inconclusive for {simp}"
1060
1061     free = sorted(e.free_symbols, key=str)
1062     if not free:
1063         v = float(sp.N(e))
1064         ok = v > 0 if strict else v >= 0
1065         return PositivityCertificate(ok, not ok, "constant", {},
1066                                     True, v, None, "expression is constant")
1067
1068     if box is None:
1069         return PositivityCertificate(False, False, "none", {}, domain_is_compact,
1070                                     symbolic_note=note + " | no box supplied for "
1071                                     "interval certification")
1072
1073     res = global_lower_bound(e, box, target=0.0, max_boxes=max_boxes)
1074     boxd = {str(k): (float(v[0]), float(v[1])) for k, v in box.items()}
1075     if res.converged and res.remaining == 0 and res.domain_errors == 0:
1076         return PositivityCertificate(True, False, "interval_branch_and_bound",
1077                                     boxd, domain_is_compact, res.lower_bound,
1078                                     None, note, res.as_dict())
1079     if res.upper_bound < 0:
1080         cex = {k: 0.5 * (v[0] + v[1]) for k, v in (res.argmin_box or boxd).items()}
1081         return PositivityCertificate(False, True, "interval_branch_and_bound",
1082                                     boxd, domain_is_compact, res.lower_bound,
1083                                     cex, note, res.as_dict())
1084     return PositivityCertificate(False, False, "interval_branch_and_bound",
1085                                 boxd, domain_is_compact, res.lower_bound, None,
1086                                 note, res.as_dict())
1087
1088
1089 __all__ = ["interval_eval", "IntervalDomainError", "global_lower_bound",
1090           "BoundResult", "prove_positive", "PositivityCertificate"]

```

src/ctcfa/model.py — 578 lines

```

1091 """
1092 CTC-FA v2.0 -- L0 : Unified model state vector and registry data model.
1093
1094 The v1.0 state vector was
1095
1096     S = (M, g, topology, identifications, fields, quantum_state, provenance)
1097     X = (F, P, T, I, S, H, C, E, Q, D, A)
1098
1099 but the kernel only implemented ``(g, coords, params, compact)``; ``topology``
1100 and ``identifications`` were free-text strings (v1.0 FM-007). This module
1101 gives every component of the state vector a machine object, so that a global
1102 causal statement can be *computed* from the identifications rather than
1103 asserted in prose.
1104
1105 New in v2.0
1106 -----
1107 * :class:`DeckTransformation` -- an explicit coordinate map with a verified
1108   isometry certificate. ``Gamma`` acting on the universal cover is the object
1109   that turns "psi is periodic" from a comment into a testable statement.
1110 * :class:`Quotient` -- the deck group, with word enumeration, closure testing
1111   and a free-action check.
1112 * :class:`CompactDirection` now carries a *verification record*: a compact
1113   generator is only admissible if it is a Killing vector of the declared
1114   metric and its orbit closes under the declared identification.
1115 * :class:`DomainConstraint` -- machine-checkable coordinate/parameter domain,
1116   so that a boundary root landing outside the physical domain is rejected
1117   automatically (v1.0 FM-014).
1118 * :class:`MatterSpec` -- declared stress-energy, so that L2/L9 can distinguish
1119   "vacuum by assumption" from "vacuum verified".
1120 """
1121
1122 from __future__ import annotations
1123
1124 from dataclasses import dataclass, field
1125 from typing import Any, Callable, Iterable, Mapping, Sequence
1126
1127 import sympy as sp
1128
1129 from .status import CTCStatus, ClaimScope, ModelKind
1130
1131 Number = float | int
1132
1133 # -----
1134 # Domain
1135 # -----
1136
1137 @dataclass(frozen=True)
1138 class DomainConstraint:
1139     """A single machine-checkable constraint on coordinates and/or parameters.
1140
1141     ``expr`` is a sympy expression that must be ``> 0`` (``strict``) or
1142     ``>= 0`` on the physical domain.
1143     """
1144     name: str
1145     expr: sp.Expr
1146     strict: bool = True
1147     note: str = ""
1148
1149     def evaluate(self, subs: Mapping[sp.Symbol | str, float]) -> float:
1150         return float(sp.N(self.expr.subs(_normalise_subs(self.expr, subs))))
1151
1152     def holds(self, subs: Mapping[sp.Symbol | str, float], tol: float = 0.0) -> bool:
1153         v = self.evaluate(subs)
1154         return v > tol if self.strict else v >= -tol
1155
1156

```

```

1158
1159 def _normalise_subs(expr: sp.Expr, subs: Mapping[Any, float]) -> dict[sp.Symbol, float]:
1160     """Accept substitutions keyed by symbol *or* by symbol name."""
1161     by_name = {str(s): s for s in expr.free_symbols}
1162     out: dict[sp.Symbol, float] = {}
1163     for k, v in subs.items():
1164         if isinstance(k, sp.Symbol):
1165             out[k] = v
1166         else:
1167             s = by_name.get(str(k))
1168             if s is not None:
1169                 out[s] = v
1170     return out
1171
1172 # -----
1173 # Compact generators
1174 # -----
1175
1176 @dataclass(frozen=True)
1177 class CompactDirection:
1178     """A declared compact/periodic generator direction.
1179
1180     Attributes
1181     -----
1182     index
1183         Coordinate index of the generator ``xi = d/dx^index``.
1184     name
1185         Coordinate name, for reporting.
1186     period
1187         SymPy expression for the identification period.
1188     requires_killing
1189         If True (default) the L4 verifier *must* certify that ``xi`` is a
1190         Killing vector before the compact-orbit detector will accept it.
1191         vl.0 accepted any declared coordinate direction with no such check.
1192     """
1193
1194     index: int
1195     name: str
1196     period: sp.Expr = 2 * sp.pi
1197     requires_killing: bool = True
1198     note: str = ""
1199
1200
1201 @dataclass
1202 class GeneratorCertificate:
1203     """Result of verifying that a declared compact generator is admissible."""
1204
1205     name: str
1206     is_killing: bool
1207     killing_residual: str
1208     closes: bool
1209     note: str = ""
1210
1211     @property
1212     def admissible(self) -> bool:
1213         return self.is_killing and self.closes
1214
1215 # -----
1216 # Deck transformations and quotients (L3)
1217 # -----
1218
1219 @dataclass(frozen=True)
1220 class DeckTransformation:
1221     """An explicit generator of the deck group ``Gamma``.
1222
1223     ``image`` is the tuple of new coordinate expressions in terms of the old
1224     coordinate symbols, i.e. the map ``x -> image(x)``. ``inverse`` is the
1225     corresponding inverse map. Both are required so that words in
1226     ``Gamma`` and ``Gamma^{-1}`` can be formed.
1227
1228     An element is admissible as a deck transformation only if it is an
1229     isometry of the declared metric; :meth:`Quotient.verify_isometries`
1230     checks this symbolically via the pullback.
1231
1232     """
1233
1234     name: str
1235     image: tuple[sp.Expr, ...]
1236     inverse: tuple[sp.Expr, ...]
1237     orientation_preserving: bool = True
1238     time_orientation_preserving: bool | None = None
1239     note: str = ""
1240
1241     def apply(self, coords: Sequence[sp.Symbol], point: Sequence[Any]) -> tuple[Any, ...]:
1242         sub = dict(zip(coords, point))
1243         return tuple(sp.simplify(e.subs(sub)) for e in self.image)
1244
1245     def apply_numeric(self, coords: Sequence[sp.Symbol],
1246                      point: Sequence[float]) -> tuple[float, ...]:
1247         sub = {c: v for c, v in zip(coords, point)}
1248         return tuple(float(sp.N(e.subs(sub))) for e in self.image)
1249
1250
1251 @dataclass
1252 class Quotient:
1253     """The global identification structure ``M_universal / Gamma``.
1254
1255     This object is what makes v2.0 able to *compute* the difference between
1256     two spacetimes with identical local metrics and different global causal
1257     structure (the L3 scientific gate). The canonical demonstration in the
1258     v2.0 registry is the pair
1259
1260     ..ads4_periodic`` -- Gamma = <t -> t + 2 pi L> : CTC_PROVED
1261     ..ads4_universal_cover`` -- Gamma = {e} : NO CTC_CERTIFIED
1262
1263     which share one local metric tensor and differ only here.
1264
1265     """
1266
1267     generators: tuple[DeckTransformation, ...] = ()

```

```

1268 label: str = "trivial"
1269 acts_freely: bool | None = None
1270 note: str = ""
1271
1272 # -- group words -----
1273
1274 def words(self, max_length: int = 2) -> list[tuple[str, ...]]:
1275     """Enumerate reduced words in the generators up to `max_length`.
1276
1277     Words are returned as tuples of signed generator names, for example
1278     `("g0", "g0")` for `g0^2` and `("g0^-1",)` for the inverse.
1279     The identity is included as the empty tuple.
1280     """
1281     if not self.generators:
1282         return []
1283     alphabet: list[str] = []
1284     for g in self.generators:
1285         alphabet.append(g.name)
1286         alphabet.append(g.name + "^-1")
1287     out: list[tuple[str, ...]] = []
1288     frontier: list[tuple[str, ...]] = []
1289     for _ in range(max_length):
1290         nxt: list[tuple[str, ...]] = []
1291         for w in frontier:
1292             for a in alphabet:
1293                 if w and _is_inverse_pair(w[-1], a):
1294                     continue # reduced words only
1295                 nxt.append(w + (a,))
1296         out.extend(nxt)
1297         frontier = nxt
1298     return out
1299
1300 def _map_for(self, token: str) -> tuple[sp.Expr, ...]:
1301     inv = token.endswith("^-1")
1302     base = token[:-3] if inv else token
1303     for g in self.generators:
1304         if g.name == base:
1305             return g.inverse if inv else g.image
1306     raise KeyError(f"unknown deck generator {token!r}")
1307
1308 def apply_word(self, coords: Sequence[sp.Symbol], point: Sequence[float],
1309               word: Sequence[str],
1310               params: Mapping[sp.Symbol, float] | None = None
1311               ) -> tuple[float, ...]:
1312     """Apply a deck word numerically.
1313
1314     `params` supplies numeric values for any *parameter* symbols that
1315     appear in the generator maps (a boost rapidity, a translation
1316     length, an identification period). Omitting them is the most common
1317     way a deck computation silently fails, so it is a required argument
1318     wherever the registry entry declares parameters.
1319     """
1320     base = dict(params or {})
1321     cur = tuple(float(v) for v in point)
1322     for token in word:
1323         image = self._map_for(token)
1324         sub = dict(base)
1325         sub.update({c: v for c, v in zip(coords, cur)})
1326         cur = tuple(float(sp.N(e.subs(sub))) for e in image)
1327     return cur
1328
1329 def apply_word_symbolic(self, coords: Sequence[sp.Symbol],
1330                        point: Sequence[sp.Expr],
1331                        word: Sequence[str]) -> tuple[sp.Expr, ...]:
1332     cur = tuple(point)
1333     for token in word:
1334         image = self._map_for(token)
1335         sub = dict(zip(coords, cur))
1336         cur = tuple(sp.simplify(e.subs(sub)) for e in image)
1337     return cur
1338
1339 # -- closure test -----
1340
1341 def closes(self, coords: Sequence[sp.Symbol], p0: Sequence[float],
1342            p1: Sequence[float], tol: float = 1e-9,
1343            max_length: int = 2,
1344            params: Mapping[sp.Symbol, float] | None = None
1345            ) -> tuple[bool, tuple[str, ...] | None, float]:
1346     """Does `p1` equal `g . p0` for some word `g` in `Gamma`?
1347
1348     Returns `(closed, word, residual)`. This is the *correct*
1349     closure test for a curve on a quotient manifold and replaces the
1350     v1.0 test, which only reduced a coordinate difference modulo a
1351     declared scalar period and therefore could not express Misner,
1352     Grant or Gott identifications at all.
1353     """
1354     best = (False, None, float("inf"))
1355     for w in self.words(max_length):
1356         try:
1357             q = self.apply_word(coords, p0, w, params)
1358             except (TypeError, ValueError, ZeroDivisionError):
1359                 continue
1360             res = max(abs(a - b) for a, b in zip(q, p1)) if q else float("inf")
1361             if res < best[2]:
1362                 best = (res <= tol, w, res)
1363     return best
1364
1365 # -- isometry certificate -----
1366
1367 def verify_isometries(self, coords: Sequence[sp.Symbol],
1368                       metric: sp.Matrix) -> dict[str, str]:
1369     """Symbolically verify `g*_ab = g_ab` for every generator.
1370
1371     The pullback is `(phi^* g)_ab = g_{cd}(phi(x)) dphi^c/dx^a
1372     dphi^d/dx^b`. Returns a map generator-name -> residual string
1373     (`"0"` when the identity is exact).
1374     """
1375     out: dict[str, str] = {}
1376     n = len(coords)
1377     for gen in self.generators:

```

```

1378         J = sp.Matrix(n, n, lambda a, b: sp.diff(gen.image[a], coords[b]))
1379         sub = dict(zip(coords, gen.image))
1380         g_at_image = metric.subs(sub, simultaneous=True)
1381         pull = sp.simplify(J.T * g_at_image * J)
1382         res = sp.simplify(pull - metric)
1383         out[gen.name] = "0" if res.is_zero_matrix else str(res)
1384     return out
1385
1386
1387 def _is_inverse_pair(a: str, b: str) -> bool:
1388     return a == b + "^-1" or b == a + "^-1"
1389
1390
1391 # -----
1392 # Matter
1393 # -----
1394
1395 @dataclass
1396 class MatterSpec:
1397     """Declared stress-energy content."""
1398
1399     label: str = "unspecified"
1400     T: sp.Matrix | None = None
1401     """Covariant stress-energy ``T_{mu nu}`` if a closed form is registered."""
1402     vacuum: bool = False
1403     cosmological_constant: sp.Expr = sp.S.Zero
1404     note: str = ""
1405
1406
1407 # -----
1408 # Provenance
1409 # -----
1410
1411 @dataclass
1412 class Provenance:
1413     source_refs: tuple[str, ...] = ()
1414     convention_note: str = ""
1415     derivation: str = ""
1416     implementation_status: str = "implemented"
1417     external_spec: str = ""
1418
1419
1420 # -----
1421 # The model
1422 # -----
1423
1424 @dataclass
1425 class MetricModel:
1426     """One registered spacetime model.
1427
1428     Only ``id``, ``title`` and ``kind`` are mandatory; a
1429     ``EXTERNAL_EXACT_SPEC`` or ``COMPOSITE_GLOBAL`` model legitimately has no
1430     local metric and must therefore be handled by a global predicate rather
1431     than by the tensor engines. Calling :meth:`require_metric` on such a
1432     model raises, which is the mechanism that prevents CTC-FA from inventing
1433     a fake canonical tensor (v1.0 REQ-016).
1434     """
1435
1436     id: str
1437     title: str
1438     coords: tuple[sp.Symbol, ...] = ()
1439     metric: sp.Matrix | None = None
1440     params: tuple[sp.Symbol, ...] = ()
1441     signature: tuple[int, int, int] = (1, 0, 3)
1442     """Declared signature as ``(n_negative, n_null, n_positive)`` in the
1443     (-,+,+) convention. Checked numerically at every evaluated point."""
1444
1445     compact: tuple[CompactDirection, ...] = ()
1446     quotient: Quotient = field(default_factory=Quotient)
1447     kind: ModelKind = ModelKind.LOCAL_METRIC
1448     theory: str = "GR"
1449     matter: MatterSpec = field(default_factory=MatterSpec)
1450     topology: str = "unspecified"
1451     expected: CTCStatus = CTCStatus.UNRESOLVED
1452     expected_scope: ClaimScope = ClaimScope.SECTOR
1453     expected_at_default: CTCStatus | None = None
1454     """Expected verdict *at the registered default point*, when it differs
1455     from the family-level expectation. The Godel-type family is the reason
1456     this field exists: the family as a whole is UNRESOLVED because its causal
1457     character depends on ``m`` and ``omega``, while the default point
1458     ``(m, omega) = (1, 1)`` sits inside the chronology-violating region.
1459     Conflating the two is a category error that a single 'expected' field
1460     cannot express."""
1461     domain: tuple[DomainConstraint, ...] = ()
1462     default_point: Mapping[str, float] = field(default_factory=dict)
1463     domain_notes: str = ""
1464     provenance: Provenance = field(default_factory=Provenance)
1465     metadata: dict[str, Any] = field(default_factory=dict)
1466
1467     global_margin: Callable[[Mapping[str, float], float] | None = None
1468     """Optional global/reduced numerical route-closure predicate. Negative
1469     means a closed causal route in the *declared reduced model*; this is
1470     never a global GR theorem (v1.0 section 9.2)."""
1471
1472     global_margin_scope: ClaimScope = ClaimScope.REDUCED_MODEL
1473     global_margin_note: str = ""
1474
1475     temporal_function: sp.Expr | None = None
1476     """Optional candidate temporal function tau(x) for the L7 negative
1477     certificate."""
1478
1479     reference_timelike: tuple[sp.Expr, ...] | None = None
1480     """Optional timelike reference direction ``u^a`` used to normalise the
1481     reduced causal margin (L16). Required for charts in which no coordinate
1482     direction is timelike -- Misner (``g_{TT} = 0``) and the Ori-2007 style
1483     null-adapted cores (``g_{rr} = 0``) are the registry examples. When absent
1484     the first coordinate direction with a negative diagonal component is
1485     used."""
1486
1487     # -- invariants -----

```

```

1488
1489 def __post_init__(self) -> None:
1490     if self.metric is not None:
1491         self.assert_wellformed()
1492
1493 def assert_wellformed(self) -> None:
1494     """Structural gate V0. Raises on any malformed registry entry.
1495
1496     v1.0 defect D-02 (the van Stockum entry stored an asymmetric matrix)
1497     is caught here, at registry load, for every model.
1498     """
1499     g = self.metric
1500     if g is None:
1501         return
1502     n = len(self.coords)
1503     if g.shape != (n, n):
1504         raise ValueError(f"{self.id}: metric shape {g.shape} != ({n},{n}) "
1505                          f"for coords {self.coords}")
1506     asym = sp.simplify(g - g.T)
1507     if not asym.is_zero_matrix:
1508         raise ValueError(f"{self.id}: metric is not symmetric; g - g^T = {asym}")
1509     for cd in self.compact:
1510         if not (0 <= cd.index < n):
1511             raise ValueError(f"{self.id}: compact index {cd.index} out of range")
1512         if str(self.coords[cd.index]) != cd.name:
1513             raise ValueError(
1514                 f"{self.id}: compact generator name {cd.name!r} does not match "
1515                 f"coordinate {self.coords[cd.index]!r} at index {cd.index}")
1516     declared = set(self.coords) | set(self.params)
1517     free = set()
1518     for e in g:
1519         free |= {s for s in e.free_symbols if isinstance(s, sp.Symbol)}
1520     unknown = {s for s in free if s not in declared}
1521     if unknown:
1522         raise ValueError(f"{self.id}: metric contains undeclared symbols {unknown}")
1523
1524 def require_metric(self) -> sp.Matrix:
1525     if self.metric is None:
1526         raise ValueError(
1527             f"{self.id} is a {self.kind.value} model with no canonical local "
1528             f"metric. CTC-FA v2.0 refuses to fabricate one. "
1529             f"External specification: {self.provenance.external_spec or 'see registry record'}")
1530     return self.metric
1531
1532 @property
1533 def has_metric(self) -> bool:
1534     return self.metric is not None
1535
1536 @property
1537 def dim(self) -> int:
1538     return len(self.coords)
1539
1540 def coord_index(self, name: str) -> int:
1541     for i, c in enumerate(self.coords):
1542         if str(c) == name:
1543             return i
1544     raise KeyError(f"{self.id}: no coordinate named {name!r}")
1545
1546 def symbol(self, name: str) -> sp.Symbol:
1547     for s in tuple(self.coords) + tuple(self.params):
1548         if str(s) == name:
1549             return s
1550     raise KeyError(f"{self.id}: no symbol named {name!r}")
1551
1552 def all_symbols(self) -> tuple[sp.Symbol, ...]:
1553     return tuple(self.coords) + tuple(self.params)
1554
1555 def substitution(self, values: Mapping[str | sp.Symbol, float]) -> dict[sp.Symbol, float]:
1556     by_name = {str(s): s for s in self.all_symbols()}
1557     out: dict[sp.Symbol, float] = {}
1558     for k, v in values.items():
1559         s = k if isinstance(k, sp.Symbol) else by_name.get(str(k))
1560         if s is None:
1561             raise KeyError(f"{self.id}: unknown symbol {k!r}. "
1562                           f"Known: {sorted(by_name)}")
1563         out[s] = v
1564     return out
1565
1566 def missing_values(self, values: Mapping[str | sp.Symbol, float]) -> list[str]:
1567     """Which declared symbols still have no numeric value?"""
1568     given = {str(k) for k in values}
1569     return [str(s) for s in self.all_symbols() if str(s) not in given]
1570
1571 def domain_report(self, values: Mapping[str | sp.Symbol, float],
1572                  tol: float = 0.0) -> dict[str, Any]:
1573     rows = []
1574     ok = True
1575     n_skipped = 0
1576     given = {str(k) for k in values}
1577     for c in self.domain:
1578         need = {str(s) for s in c.expr.free_symbols}
1579         if not need <= given:
1580             # A constraint whose symbols were not all supplied is NOT
1581             # evaluated and must not be reported as a violation; silently
1582             # counting it as a failure was what made every partially
1583             # specified continuation run look out of domain.
1584             rows.append({"name": c.name, "value": None, "ok": None,
1585                        "skipped": sorted(need - given)})
1586             n_skipped += 1
1587             continue
1588     try:
1589         v = c.evaluate(self.substitution(values))
1590     except Exception as exc:
1591         # noqa: BLE001
1592         rows.append({"name": c.name, "value": None, "ok": False,
1593                    "error": str(exc)})
1594         ok = False
1595         continue
1596     good = v > tol if c.strict else v >= -tol
1597     ok = ok and good
1598     rows.append({"name": c.name, "value": v, "strict": c.strict, "ok": good})

```



```

1598         return {"ok": ok, "constraints": rows, "n_skipped": n_skipped,
1599                "complete": n_skipped == 0}
1600
1601     def as_record(self) -> dict[str, Any]:
1602         """Machine-readable registry record (validated by metric.schema.json)."""
1603         return {
1604             "model_id": self.id,
1605             "title": self.title,
1606             "version": "2.0",
1607             "kind": self.kind.value,
1608             "theory": self.theory,
1609             "signature": list(self.signature),
1610             "signature_convention": "(-,+,+,+)",
1611             "coordinates": [str(c) for c in self.coords],
1612             "parameters": [str(p) for p in self.params],
1613             "metric_defined": self.metric is not None,
1614             "metric_components": (
1615                 {f"{i}{j}": str(self.metric[i, j])
1616                  for i in range(self.dim) for j in range(i, self.dim)
1617                  if self.metric[i, j] != 0}
1618                 if self.metric is not None else None),
1619             "topology": self.topology,
1620             "identifications": [
1621                 {"coordinate": c.name, "index": c.index, "period": str(c.period),
1622                  "requires_killing": c.requires_killing, "note": c.note}
1623                 for c in self.compact],
1624             "quotient": {
1625                 "label": self.quotient.label,
1626                 "generators": [
1627                     {"name": g.name,
1628                      "image": [str(e) for e in g.image],
1629                      "inverse": [str(e) for e in g.inverse],
1630                      "time_orientation_preserving": g.time_orientation_preserving,
1631                      "note": g.note}
1632                     for g in self.quotient.generators],
1633                 "acts_freely": self.quotient.acts_freely,
1634                 "note": self.quotient.note,
1635             },
1636             "matter": {"label": self.matter.label, "vacuum": self.matter.vacuum,
1637                        "Lambda": str(self.matter.cosmological_constant),
1638                        "T_defined": self.matter.T is not None,
1639                        "note": self.matter.note},
1640             "domain": [{"name": c.name, "expression": str(c.expr),
1641                         "strict": c.strict, "note": c.note} for c in self.domain],
1642             "default_point": dict(self.default_point),
1643             "expected_status": self.expected.value,
1644             "expected_status_at_default_point": (
1645                 self.expected_at_default.value if self.expected_at_default
1646                 else self.expected.value),
1647             "expected_scope": self.expected_scope.value,
1648             "domain_notes": self.domain_notes,
1649             "global_margin": self.global_margin is not None,
1650             "global_margin_scope": self.global_margin_scope.value,
1651             "global_margin_note": self.global_margin_note,
1652             "temporal_function": (str(self.temporal_function)
1653                                  if self.temporal_function is not None else None),
1654             "provenance": {
1655                 "source_refs": list(self.provenance.source_refs),
1656                 "convention_note": self.provenance.convention_note,
1657                 "derivation": self.provenance.derivation,
1658                 "implementation_status": self.provenance.implementation_status,
1659                 "external_spec": self.provenance.external_spec,
1660             },
1661             "metadata": self.metadata,
1662         }
1663
1664
1665     __all__ = [
1666         "MetricModel", "CompactDirection", "GeneratorCertificate", "DeckTransformation",
1667         "Quotient", "DomainConstraint", "MatterSpec", "Provenance",
1668     ]

```

src/ctcfa/symbolic.py — 308 lines

```

1669 """
1670 CTC-FA v2.0 -- L1 : Symbolic tensor engine.
1671
1672 Everything here is exact computer algebra in the (-,+,+,) convention with
1673 G = c = 1. The conventions are fixed once, here, and every downstream layer
1674 is required to use them (v1.0 FM-019: "published formula copied with
1675 incompatible convention").
1676
1677 Conventions
1678 -----
1679 Christoffel      Gamma^a_bc = 1/2 g^{ad} ( d_b g_{dc} + d_c g_{db} - d_d g_{bc} )
1680 Riemann          R^a_{bcd} = d_c Gamma^a_{db} - d_d Gamma^a_{cb}
1681                  + Gamma^a_{ce} Gamma^e_{db} - Gamma^a_{de} Gamma^e_{cb}
1682 Ricci            R_{bd} = R^a_{bad}
1683 Ricci scalar    R = g^{bd} R_{bd}
1684 Einstein         G_{ab} = R_{ab} - 1/2 g_{ab} R
1685 Field equation  G_{ab} + Lambda g_{ab} = 8 pi T_{ab}
1686 Kretschmann     K = R_{abcd} R^{abcd}
1687
1688 With these conventions the Schwarzschild solution has R_{ab} = 0 and
1689 K = 48 M^2 / r^6, and de Sitter with Lambda > 0 has R = 4 Lambda. Both are
1690 asserted in ``tests/unit/test_symbolic.py``, so a convention drift is a test
1691 failure rather than a silent sign error.
1692
1693 v1.0 defect closed here
1694 -----
1695 D-09 ``kretschmann_placeholder`` raised ``NotImplementedError``. Since the
1696 Kretschmann scalar is the *only* tool in the architecture that can
1697 separate a coordinate singularity from a curvature singularity, its
1698 absence left failure mode FM-001 unmitigated in code. It is now
1699 implemented and benchmarked.
1700 """
1701
1702 from __future__ import annotations

```

```

1703
1704 from functools import lru_cache
1705 from typing import Iterable, Sequence
1706
1707 import sympy as sp
1708
1709 _SIMPLIFY_BUDGET = 4000
1710
1711
1712 def _s(expr: sp.Expr, simplify: bool = True) -> sp.Expr:
1713     """Simplify with a size guard so pathological metrics cannot hang a run."""
1714     if not simplify:
1715         return expr
1716     try:
1717         if sp.count_ops(expr) > _SIMPLIFY_BUDGET:
1718             return sp.cancel(sp.together(expr))
1719         return sp.simplify(expr)
1720     except Exception:
1721         return expr
1722     # noqa: BLE001
1723
1724 # -----
1725 # Basic algebra
1726 # -----
1727
1728 def inverse_metric(g: sp.Matrix, simplify: bool = True) -> sp.Matrix:
1729     gi = g.inv()
1730     return gi.applyfunc(lambda e: _s(e, simplify)) if simplify else gi
1731
1732
1733 def check_inverse(g: sp.Matrix, gi: sp.Matrix | None = None) -> sp.Matrix:
1734     """Return  $g * g^{-1} - I$ ; the zero matrix certifies gate V1."""
1735     gi = inverse_metric(g) if gi is None else gi
1736     return sp.simplify(g * gi - sp.eye(g.shape[0]))
1737
1738
1739 def determinant(g: sp.Matrix, simplify: bool = True) -> sp.Expr:
1740     return _s(g.det(), simplify)
1741
1742
1743 # -----
1744 # Connection and curvature
1745 # -----
1746
1747 def christoffel(g: sp.Matrix, coords: Sequence[sp.Symbol],
1748                 simplify: bool = True, gi: sp.Matrix | None = None) -> list[list[list[sp.Expr]]]:
1749     """ $\Gamma[a][b][c] = \Gamma^a_{bc}$ ."""
1750     n = len(coords)
1751     gi = inverse_metric(g, simplify=False) if gi is None else gi
1752     dg = [[sp.diff(g[b, c], coords[d]) for c in range(n)] for b in range(n)]
1753     for d in range(n):
1754         out = [[sp.S.Zero] * n for _ in range(n)] for _ in range(n)
1755     for a in range(n):
1756         for b in range(n):
1757             for c in range(b, n):
1758                 acc = sp.S.Zero
1759                 for d in range(n):
1760                     if gi[a, d] == 0:
1761                         continue
1762                     acc += gi[a, d] * (dg[b][d][c] + dg[c][d][b] - dg[d][b][c])
1763                 val = _s(acc / 2, simplify)
1764                 out[a][b][c] = val
1765                 out[a][c][b] = val
1766     return out
1767
1768
1769 def riemann(g: sp.Matrix, coords: Sequence[sp.Symbol], simplify: bool = True,
1770            Gamma: list | None = None) -> list[list[list[list[sp.Expr]]]]:
1771     """ $R[a][b][c][d] = R^a_{bcd}$ ."""
1772     n = len(coords)
1773     Gamma = christoffel(g, coords, simplify=False) if Gamma is None else Gamma
1774     R = [[[[sp.S.Zero] * n for _ in range(n)] for _ in range(n)] for _ in range(n)]
1775     for a in range(n):
1776         for b in range(n):
1777             for c in range(n):
1778                 for d in range(c + 1, n):
1779                     expr = (sp.diff(Gamma[a][d][b], coords[c])
1780                             - sp.diff(Gamma[a][c][b], coords[d]))
1781                     for e in range(n):
1782                         expr += (Gamma[a][c][e] * Gamma[e][d][b]
1783                                 - Gamma[a][d][e] * Gamma[e][c][b])
1784                     val = _s(expr, simplify)
1785                     R[a][b][c][d] = val
1786                     R[a][b][d][c] = -val
1787     return R
1788
1789
1790 def riemann_lower(g: sp.Matrix, coords: Sequence[sp.Symbol], simplify: bool = True,
1791                  R: list | None = None) -> list[list[list[list[sp.Expr]]]]:
1792     """ $R_{abcd} = g_{ae} R^e_{bcd}$ ."""
1793     n = len(coords)
1794     R = riemann(g, coords, simplify=False) if R is None else R
1795     out = [[[[[sp.S.Zero] * n for _ in range(n)] for _ in range(n)] for _ in range(n)]
1796     for a in range(n):
1797         for b in range(n):
1798             for c in range(n):
1799                 for d in range(c + 1, n):
1800                     expr = sum(g[a, e] * R[e][b][c][d] for e in range(n))
1801                     val = _s(expr, simplify)
1802                     out[a][b][c][d] = val
1803                     out[a][b][d][c] = -val
1804     return out
1805
1806
1807 def ricci_tensor(g: sp.Matrix, coords: Sequence[sp.Symbol], simplify: bool = True,
1808                 Gamma: list | None = None) -> sp.Matrix:
1809     """ $R_{bd} = R^a_{bad}$ , computed directly (cheaper than contracting Riemann)."""
1810     n = len(coords)
1811     Gamma = christoffel(g, coords, simplify=False) if Gamma is None else Gamma
1812     Ric = sp.zeros(n, n)

```

```

1813     for b in range(n):
1814         for d in range(b, n):
1815             expr = sp.S.Zero
1816             for a in range(n):
1817                 expr += sp.diff(Gamma[a][d][b], coords[a]) - sp.diff(Gamma[a][a][b], coords[d])
1818                 for e in range(n):
1819                     expr += Gamma[a][a][e] * Gamma[e][d][b] - Gamma[a][d][e] * Gamma[e][a][b]
1820             val = _s(expr, simplify)
1821             Ric[b, d] = val
1822             Ric[d, b] = val
1823     return Ric
1824
1825
1826 def ricci_scalar(g: sp.Matrix, coords: Sequence[sp.Symbol], simplify: bool = True,
1827                 Ric: sp.Matrix | None = None, gi: sp.Matrix | None = None) -> sp.Expr:
1828     n = len(coords)
1829     gi = inverse_metric(g, simplify=False) if gi is None else gi
1830     Ric = ricci_tensor(g, coords, simplify=False) if Ric is None else Ric
1831     return _s(sum(gi[a, b] * Ric[a, b] for a in range(n) for b in range(n)), simplify)
1832
1833
1834 def einstein_tensor(g: sp.Matrix, coords: Sequence[sp.Symbol],
1835                    simplify: bool = True) -> sp.Matrix:
1836     gi = inverse_metric(g, simplify=False)
1837     Gamma = christoffel(g, coords, simplify=False, gi=gi)
1838     Ric = ricci_tensor(g, coords, simplify=False, Gamma=Gamma)
1839     R = ricci_scalar(g, coords, simplify=False, Ric=Ric, gi=gi)
1840     G = Ric - sp.Rational(1, 2) * g * R
1841     return G.applyfunc(lambda e: _s(e, simplify))
1842
1843
1844 def kretschmann(g: sp.Matrix, coords: Sequence[sp.Symbol],
1845                simplify: bool = True) -> sp.Expr:
1846     """K = R_{abcd} R^{abcd}."""
1847
1848     Uses the antisymmetry pairs to cut the work by 4 and the pair-exchange
1849     symmetry where cheap. Exact, not a placeholder.
1850     """
1851     n = len(coords)
1852     gi = inverse_metric(g, simplify=False)
1853     Gamma = christoffel(g, coords, simplify=False, gi=gi)
1854     Rud = riemann(g, coords, simplify=False, Gamma=Gamma)
1855     Rdd = riemann_lower(g, coords, simplify=False, R=Rud)
1856
1857     # Raise all four indices once: R^{abcd} = g^{ae} g^{bf} g^{cg} g^{dh} R_{efgh}
1858     Ruu = [[[sp.S.Zero] * n for _ in range(n)] for _ in range(n)]
1859     for a in range(n):
1860         for b in range(n):
1861             for c in range(n):
1862                 for d in range(c + 1, n):
1863                     expr = sp.S.Zero
1864                     for e in range(n):
1865                         if gi[a, e] == 0:
1866                             continue
1867                         for f in range(n):
1868                             if gi[b, f] == 0:
1869                                 continue
1870                             for gg in range(n):
1871                                 if gi[c, gg] == 0:
1872                                     continue
1873                                 for h in range(n):
1874                                     if gi[d, h] == 0:
1875                                         continue
1876                                     term = Rdd[e][f][gg][h]
1877                                     if term == 0:
1878                                         continue
1879                                     expr += gi[a, e] * gi[b, f] * gi[c, gg] * gi[d, h] * term
1880             Ruu[a][b][c][d] = expr
1881             Ruu[a][b][d][c] = -expr
1882
1883     total = sp.S.Zero
1884     for a in range(n):
1885         for b in range(n):
1886             for c in range(n):
1887                 for d in range(c + 1, n):
1888                     if Rdd[a][b][c][d] == 0 or Ruu[a][b][c][d] == 0:
1889                         continue
1890                     total += 2 * Rdd[a][b][c][d] * Ruu[a][b][c][d]
1891     return _s(total, simplify)
1892
1893
1894 def field_equation_residual(g: sp.Matrix, coords: Sequence[sp.Symbol],
1895                             T: sp.Matrix | None = None,
1896                             Lambda: sp.Expr = sp.S.Zero,
1897                             simplify: bool = True) -> sp.Matrix:
1898     """E_{ab} = G_{ab} + Lambda g_{ab} - 8 pi T_{ab}."""
1899     n = len(coords)
1900     G = einstein_tensor(g, coords, simplify=simplify)
1901     rhs = sp.zeros(n, n) if T is None else 8 * sp.pi * T
1902     res = G + Lambda * g - rhs
1903     return res.applyfunc(lambda e: _s(e, simplify))
1904
1905
1906 # -----
1907 # Symmetry (L4)
1908 # -----
1909
1910 def lie_derivative_metric(g: sp.Matrix, coords: Sequence[sp.Symbol],
1911                          xi: Sequence[sp.Expr], simplify: bool = True) -> sp.Matrix:
1912     """(L_xi g)_{ab} = xi^c d_c g_{ab} + g_{cb} d_a xi^c + g_{ac} d_b xi^c.
1913
1914     A vanishing result certifies that ``xi`` is a Killing vector. v1.0
1915     declared compact generators but never verified this, so nothing stopped
1916     a registry entry from naming a non-Killing coordinate direction as a
1917     "symmetry generator" (v2.0 defect D-04).
1918     """
1919     n = len(coords)
1920     out = sp.zeros(n, n)
1921     for a in range(n):
1922         for b in range(a, n):

```

```

1923         expr = sum(xi[c] * sp.diff(g[a, b], coords[c]) for c in range(n))
1924         expr += sum(g[c, b] * sp.diff(xi[c], coords[a]) for c in range(n))
1925         expr += sum(g[a, c] * sp.diff(xi[c], coords[b]) for c in range(n))
1926         val = _s(expr, simplify)
1927         out[a, b] = val
1928         out[b, a] = val
1929     return out
1930
1931
1932 def coordinate_killing_residual(g: sp.Matrix, coords: Sequence[sp.Symbol],
1933                                index: int, simplify: bool = True) -> sp.Matrix:
1934     """Residual of the Killing equation for  $\xi = d/dx^{\text{index}}$ .
1935
1936     For a coordinate vector this reduces to  $d g_{ab} / d x^{\text{index}}$ .
1937
1938      $\xi = [sp.S.One \text{ if } c == \text{index} \text{ else } sp.S.Zero \text{ for } c \text{ in } \text{range}(\text{len}(\text{coords}))]$ 
1939     return lie_derivative_metric(g, coords, xi, simplify=simplify)
1940
1941
1942 def killing_equation_system(g: sp.Matrix, coords: Sequence[sp.Symbol],
1943                             ansatz: Sequence[sp.Expr] -> list[sp.Expr]:
1944     """Return the independent components of  $L_{\xi} g = 0$  for a given ansatz.
1945
1946     The caller supplies  $\xi^a$  containing free symbols; the returned list is
1947     fed to sympy.solve by :func:'ctcfa.symmetry.solve_killing'.
1948
1949     L = lie_derivative_metric(g, coords, ansatz, simplify=True)
1950     n = len(coords)
1951     return [L[a, b] for a in range(n) for b in range(a, n) if L[a, b] != 0]
1952
1953
1954 # -----
1955 # Frames and invariants
1956 # -----
1957
1958 def signature_symbolic(g: sp.Matrix) -> str:
1959     """Best-effort symbolic signature string; numeric checks live in L2."""
1960     try:
1961         return str(sp.simplify(g.det()))
1962     except Exception:
1963         return "undetermined"
1964     # noqa: BLE001
1965
1966
1967 def norm2(g: sp.Matrix, v: Sequence[sp.Expr]) -> sp.Expr:
1968     n = g.shape[0]
1969     return sp.simplify(sum(g[a, b] * v[a] * v[b] for a in range(n) for b in range(n)))
1970
1971
1972 __all__ = [
1973     "inverse_metric", "check_inverse", "determinant", "christoffel", "riemann",
1974     "riemann_lower", "ricci_tensor", "ricci_scalar", "einstein_tensor",
1975     "kretschmann", "field_equation_residual", "lie_derivative_metric",
1976     "coordinate_killing_residual", "killing_equation_system", "norm2",
1977 ]

```

src/ctcfa/numgeom.py — 215 lines

```

1977 """
1978 CTC-FA v2.0 -- Numerical differential geometry with a controlled error model.
1979
1980 The symbolic engine (ctcfa.symbolic) is exact and is the certification
1981 path. It is also, for metrics built from nested tanh regulators such as
1982 Alcubierre and Krasnikov, computationally prohibitive: a full symbolic
1983 Einstein tensor for the warp metric does not terminate in useful time. v1.0
1984 simply had no answer for those models, so their energy and curvature rows
1985 stayed empty.
1986
1987 This module supplies the second path: a lambdified metric plus high-order
1988 central finite differences with Richardson extrapolation, giving curvature
1989 tensors at a point with an explicit, *reported* truncation-plus-round-off
1990 error. Results from this path are always tagged diagnostic and their
1991 error enters the uncertainty budget, so they can refute a condition but
1992 cannot certify one -- exactly the asymmetry the architecture requires
1993 everywhere else.
1994
1995 Error model
1996 -----
1997 A fourth-order central stencil has truncation error  $O(h^4 |d^5 g|)$  and
1998 round-off  $O(\epsilon |g| / h)$ . The step is chosen near the balance point
1999  $h \sim (\epsilon)^{1/5} \cdot \text{scale}$  and the reported error is the difference between
2000 the  $h$  and  $2h$  estimates divided by  $2^4 - 1$ , which is the standard
2001 Richardson estimate.
2002 """
2003
2004 from __future__ import annotations
2005
2006 import math
2007 from dataclasses import dataclass, field
2008 from functools import lru_cache
2009 from typing import Any, Callable, Mapping, Sequence
2010
2011 import numpy as np
2012 import sympy as sp
2013
2014 from .model import MetricModel
2015
2016 _EPS = float(np.finfo(np.float64).eps)
2017
2018
2019 @dataclass
2020 class NumericCurvature:
2021     christoffel: np.ndarray
2022     riemann: np.ndarray
2023     ricci: np.ndarray
2024     ricci_scalar: float
2025     einstein: np.ndarray
2026     kretschmann: float
2027     step: float

```

```

2028     error_estimate: float
2029     note: str = "finite-difference curvature (diagnostic grade)"
2030
2031     def as_dict(self) -> dict[str, Any]:
2032         return {"ricci_scalar": self.ricci_scalar,
2033                "kretschmann": self.kretschmann,
2034                "einstein_max_abs": float(np.max(np.abs(self.einstein))),
2035                "step": self.step, "error_estimate": self.error_estimate,
2036                "note": self.note}
2037
2038
2039 class NumericGeometry:
2040     """Lambdified metric with finite-difference curvature at a point."""
2041
2042     def __init__(self, model: MetricModel):
2043         self.model = model
2044         g = model.require_metric()
2045         self.coords = list(model.coords)
2046         self.params = list(model.params)
2047         self.n = len(self.coords)
2048         args = self.coords + self.params
2049         self.g = sp.lambdify(args, sp.Matrix(g), "numpy")
2050
2051     # -- evaluation -----
2052
2053     def metric(self, x: Sequence[float], p: Sequence[float]) -> np.ndarray:
2054         m = np.array(self.g(*list(x), *list(p)), dtype=float)
2055         return (m + m.T) / 2.0
2056
2057     def split(self, values: Mapping[str, float]) -> tuple[np.ndarray, np.ndarray]:
2058         x = np.array([float(values[str(c)]) for c in self.coords], dtype=float)
2059         p = np.array([float(values[str(s)]) for s in self.params], dtype=float)
2060         return x, p
2061
2062     # -- derivatives -----
2063
2064     def dg(self, x: np.ndarray, p: np.ndarray, h: float) -> np.ndarray:
2065         """dg[c][a][b] = d g_ab / d x^c, fourth-order central."""
2066         n = self.n
2067         out = np.zeros((n, n, n))
2068         for c in range(n):
2069             e = np.zeros(n)
2070             e[c] = 1.0
2071             hc = h * max(1.0, abs(float(x[c])))
2072             gm2 = self.metric(x - 2 * hc * e, p)
2073             gm1 = self.metric(x - hc * e, p)
2074             gp1 = self.metric(x + hc * e, p)
2075             gp2 = self.metric(x + 2 * hc * e, p)
2076             out[c] = (gm2 - 8 * gm1 + 8 * gp1 - gp2) / (12 * hc)
2077         return out
2078
2079     def d2g(self, x: np.ndarray, p: np.ndarray, h: float) -> np.ndarray:
2080         """d2g[c][d][a][b] = d^2 g_ab / dx^c dx^d, fourth-order central."""
2081         n = self.n
2082         out = np.zeros((n, n, n, n))
2083         for c in range(n):
2084             ec = np.zeros(n); ec[c] = 1.0
2085             hc = h * max(1.0, abs(float(x[c])))
2086             for d in range(c, n):
2087                 ed = np.zeros(n); ed[d] = 1.0
2088                 hd = h * max(1.0, abs(float(x[d])))
2089                 if c == d:
2090                     val = (-self.metric(x + 2 * hc * ec, p)
2091                           + 16 * self.metric(x + hc * ec, p)
2092                           - 30 * self.metric(x, p)
2093                           + 16 * self.metric(x - hc * ec, p)
2094                           - self.metric(x - 2 * hc * ec, p)) / (12 * hc * hc)
2095                 else:
2096                     val = (self.metric(x + hc * ec + hd * ed, p)
2097                           - self.metric(x + hc * ec - hd * ed, p)
2098                           - self.metric(x - hc * ec + hd * ed, p)
2099                           + self.metric(x - hc * ec - hd * ed, p)) / (4 * hc * hd)
2100                 out[c, d] = val
2101             out[d, c] = val
2102         return out
2103
2104     # -- curvature -----
2105
2106     def _curvature_at_step(self, x: np.ndarray, p: np.ndarray, h: float):
2107         n = self.n
2108         g = self.metric(x, p)
2109         gi = np.linalg.inv(g)
2110         dg = self.dg(x, p, h)
2111         d2g = self.d2g(x, p, h)
2112
2113         Gam = np.zeros((n, n, n))
2114         for a in range(n):
2115             for b in range(n):
2116                 for c in range(n):
2117                     Gam[a, b, c] = 0.5 * sum(
2118                         gi[a, d] * (dg[b][d][c] + dg[c][d][b] - dg[d][b][c])
2119                         for d in range(n))
2120
2121         dGam = np.zeros((n, n, n, n)) # dGam[e][a][b][c] = d_e Gamma^a_bc
2122         dgi = np.zeros((n, n, n)) # dgi[e][a][b] = d_e g^{ab}
2123         for e in range(n):
2124             dgi[e] = -gi @ dg[e] @ gi
2125         for e in range(n):
2126             for a in range(n):
2127                 for b in range(n):
2128                     for c in range(n):
2129                         s = 0.0
2130                         for d in range(n):
2131                             s += 0.5 * dgi[e][a, d] * (
2132                                 dg[b][d][c] + dg[c][d][b] - dg[d][b][c])
2133                             s += 0.5 * gi[a, d] * (
2134                                 d2g[e][b][d][c] + d2g[e][c][d][b] - d2g[e][d][b][c])
2135                         dGam[e, a, b, c] = s
2136
2137         R = np.zeros((n, n, n, n)) # R[a][b][c][d] = R^a_bcd

```

```

2138         for a in range(n):
2139             for b in range(n):
2140                 for c in range(n):
2141                     for d in range(n):
2142                         R[a, b, c, d] = (dGam[c, a, d, b] - dGam[d, a, c, b]
2143                                     + sum(Gam[a, c, e] * Gam[e, d, b]
2144                                           - Gam[a, d, e] * Gam[e, c, b]
2145                                           for e in range(n)))
2146         Ric = np.einsum("abad->bd", R)
2147         Rs = float(np.einsum("bd,bd->", gi, Ric))
2148         G = Ric - 0.5 * g * Rs
2149         Rdown = np.einsum("ae,ebcd->abcd", g, R)
2150         Rup = np.einsum("ae,bf,cg,dh,efgh->abcd", gi, gi, gi, gi, Rdown)
2151         K = float(np.einsum("abcd,abcd->", Rdown, Rup))
2152         return Gam, R, Ric, Rs, G, K
2153
2154     def curvature(self, values: Mapping[str, float],
2155                  h: float | None = None) -> NumericCurvature:
2156         x, p = self.split(values)
2157         # Balance point for a fourth-order stencil on second derivatives:
2158         # truncation ~ h^4, round-off ~ eps/h^2, so h ~ eps^(1/6) ~ 6.4e-3.
2159         # Empirically (tests/unit/test_numgeom.py) h = 3e-3 gives a relative
2160         # error of 1e-9 on the exact Schwarzschild Kretschmann scalar.
2161         h0 = h if h is not None else 3.0e-3
2162         a = self._curvature_at_step(x, p, h0)
2163         b = self._curvature_at_step(x, p, 2 * h0)
2164         err = max(abs(a[3] - b[3]),
2165                  float(np.max(np.abs(a[4] - b[4]))),
2166                  abs(a[5] - b[5])) / 15.0
2167         return NumericCurvature(a[0], a[1], a[2], a[3], a[4], a[5], h0, err)
2168
2169     def einstein(self, values: Mapping[str, float], h: float | None = None
2170                 ) -> tuple[np.ndarray, float]:
2171         c = self.curvature(values, h)
2172         return c.einstein, c.error_estimate
2173
2174     def stress_tensor(self, values: Mapping[str, float],
2175                       Lambda: float = 0.0, h: float | None = None
2176                       ) -> tuple[np.ndarray, float]:
2177         """T_ab = (G_ab + Lambda g_ab) / (8 pi), numerically."""
2178         c = self.curvature(values, h)
2179         x, p = self.split(values)
2180         g = self.metric(x, p)
2181         T = (c.einstein + Lambda * g) / (8.0 * math.pi)
2182         return T, c.error_estimate / (8.0 * math.pi)
2183
2184
2185     @lru_cache(maxsize=64)
2186     def geometry_for(model_id: str) -> NumericGeometry:
2187         from .registry import get
2188         return NumericGeometry(get(model_id))
2189
2190
2191     __all__ = ["NumericGeometry", "NumericCurvature", "geometry_for"]

```

src/ctcfa/topology.py — 380 lines

```

2192     """
2193     CTC-FA v2.0 -- L3 : Topology and identification engine.
2194
2195     This layer is the single largest addition of v2.0. In v1.0 the topology
2196     component of the state vector was a free-text field, and the architecture's
2197     own risk register recorded the consequence:
2198
2199         FM-007 "Topology stored only as prose and misapplied" -> global causal
2200         error. Mitigation: "implement quotient/cover objects and deck
2201         transformations in v2."
2202
2203     That is what this module does. A quotient is now an executable object with
2204
2205         * an **isometry certificate** for every deck generator (symbolic pullback),
2206         * a **free-action** check where decidable,
2207         * a **time-orientation** check for every generator,
2208         * a **closure test** for curves on the quotient: a curve is closed in
2209         ``M/Gamma`` when ``gamma(1) = g . gamma(0)`` for some word ``g`` in
2210         ``Gamma``, which is strictly more general than the v1.0 test of reducing
2211         one coordinate difference modulo a scalar period.
2212
2213     The decisive scientific consequence is the L3 gate:
2214
2215         Local equality of metrics does not imply equality of global causal
2216         structure.
2217
2218     v2.0 ships an exact demonstration of that gate: ``ads4_universal_cover`` and
2219     ``ads4_periodic`` carry the *identical* local metric tensor and receive
2220     opposite global verdicts (``NO CTC CERTIFIED`` vs ``CTC PROVED``) purely
2221     because of the deck group. No v1.0 layer could express that distinction.
2222     """
2223
2224     from __future__ import annotations
2225
2226     import math
2227     from dataclasses import dataclass, field
2228     from typing import Any, Mapping, Sequence
2229
2230     import numpy as np
2231     import sympy as sp
2232
2233     from .model import DeckTransformation, MetricModel, Quotient
2234     from .numerics import signature_counts
2235
2236
2237     # -----
2238     # Certificates
2239     # -----
2240
2241     @dataclass
2242     class DeckCertificate:

```

```

2243 """Verification record for one deck generator."""
2244
2245 name: str
2246 isometry_residual: str
2247 is_isometry: bool
2248 time_orientation_preserving: bool | None
2249 has_fixed_point: bool | None
2250 fixed_point_note: str = ""
2251 note: str = ""
2252
2253 @property
2254 def admissible(self) -> bool:
2255     if not self.is_isometry:
2256         return False
2257     if self.has_fixed_point is True:
2258         return False
2259     return True
2260
2261 def as_dict(self) -> dict[str, Any]:
2262     return {"name": self.name, "is_isometry": self.is_isometry,
2263            "isometry_residual": self.isometry_residual,
2264            "time_orientation_preserving": self.time_orientation_preserving,
2265            "has_fixed_point": self.has_fixed_point,
2266            "fixed_point_note": self.fixed_point_note,
2267            "admissible": self.admissible, "note": self.note}
2268
2269 @dataclass
2270 class QuotientCertificate:
2271     """Verification record for the whole deck group of a model."""
2272
2273     model_id: str
2274     label: str
2275     generators: list[DeckCertificate] = field(default_factory=list)
2276     trivial: bool = True
2277     coordinate_periodicities: list[dict[str, Any]] = field(default_factory=list)
2278     note: str = ""
2279
2280 @property
2281 def admissible(self) -> bool:
2282     return all(c.admissible for c in self.generators)
2283
2284 @property
2285 def time_orientable(self) -> bool | None:
2286     vals = [c.time_orientation_preserving for c in self.generators]
2287     if not vals:
2288         return True
2289     if any(v is False for v in vals):
2290         return False
2291     if any(v is None for v in vals):
2292         return None
2293     return True
2294
2295 def as_dict(self) -> dict[str, Any]:
2296     return {"model_id": self.model_id, "label": self.label,
2297            "trivial": self.trivial, "admissible": self.admissible,
2298            "time_orientable": self.time_orientable,
2299            "generators": [c.as_dict() for c in self.generators],
2300            "coordinate_periodicities": self.coordinate_periodicities,
2301            "note": self.note}
2302
2303 # -----
2304 # Verification
2305 # -----
2306
2307 def _time_orientation_preserving(model: MetricModel, gen: DeckTransformation,
2308                                probe: Mapping[str, float] | None = None
2309                                ) -> tuple[bool | None, str]:
2310     """Does ``gen`` map future-directed timelike vectors to future-directed ones?
2311
2312     Test: take the declared timelike reference direction, push it forward with
2313     the Jacobian of ``gen`` and check that the inner product of the image with
2314     the reference field at the image point is still negative (same time
2315     orientation). Returns ``(verdict, note)``; ``None`` when the test is not
2316     decidable at the probe point.
2317     """
2318     if gen.time_orientation_preserving is not None:
2319         return gen.time_orientation_preserving, "declared in registry"
2320     g = model.metric
2321     if g is None:
2322         return None, "no local metric"
2323     pt = dict(probe or model.default_point)
2324     if not pt:
2325         return None, "no probe point"
2326     coords = model.coords
2327     n = len(coords)
2328     try:
2329         sub = model.substitution(pt)
2330     except KeyError as exc:
2331         return None, f"probe point incomplete: {exc}"
2332     try:
2333         gnum = np.array(sp.Matrix(g).subs(sub).evalf(), dtype=float)
2334     except Exception as exc:
2335         return None, f"metric not numeric at probe: {exc}" # noqa: BLE001
2336     vals, vecs = np.linalg.eigh((gnum + gnum.T) / 2.0)
2337     order = np.argsort(vals)
2338     tvec = vecs[:, order[0]] # the timelike eigendirection
2339     if vals[order[0]] >= 0:
2340         return None, "no timelike direction at probe point"
2341     J = sp.Matrix(n, n, lambda a, b: sp.diff(gen.image[a], coords[b]))
2342     try:
2343         Jn = np.array(J.subs(sub).evalf(), dtype=float)
2344     except Exception as exc:
2345         return None, f"Jacobian not numeric at probe: {exc}" # noqa: BLE001
2346     pushed = Jn @ tvec
2347     image_pt = gen.apply_numeric(coords, [float(sub[c]) for c in coords])
2348     sub_img = dict(sub)
2349     for c, v in zip(coords, image_pt):
2350         sub_img[c] = v

```

```

2353     try:
2354         gimg = np.array(sp.Matrix(g).subs(sub_img).evalf(), dtype=float)
2355     except Exception as exc:
2356         return None, f"metric not numeric at image: {exc}" # noqa: BLE001
2357     vi, ve = np.linalg.eigh((gimg + gimg.T) / 2.0)
2358     oi = np.argsort(vi)
2359     if vi[oi[0]] >= 0:
2360         return None, "no timelike direction at image point"
2361     tref = ve[:, oi[0]]
2362     # Align the reference at the image with the reference at the source by
2363     # continuity along the shortest path in coordinate space.
2364     if float(tref @ tvec) < 0:
2365         tref = -tref
2366     ip = float(pushes @ gimg @ tref)
2367     if abs(ip) < 1e-12:
2368         return None, "degenerate orientation probe"
2369     return (ip < 0), f"g(phi_* T, T_ref) = {ip:.6e}"
2370
2371
2372 def _has_fixed_point(model: MetricModel, gen: DeckTransformation
2373                     ) -> tuple[bool | None, str]:
2374     """Try to solve  $\phi(x) = x$ . A free action is required for a manifold
2375     quotient; a fixed point means the quotient is an orbifold, not a manifold.
2376     """
2377     coords = model.coords
2378     eqs = [sp.Eq(sp.simplify(gen.image[i] - coords[i]), 0) for i in range(len(coords))]
2379     eqs = [e for e in eqs if e != True] # noqa: E712
2380     if not eqs:
2381         return True, "generator is the identity on this chart"
2382     try:
2383         sol = sp.solve(eqs, list(coords), dict=True)
2384     except Exception as exc:
2385         return None, f"fixed-point system not solvable symbolically: {exc}" # noqa: BLE001
2386     if not sol:
2387         return False, "no solution to  $\phi(x)=x$ "
2388     real = []
2389     for s in sol:
2390         if all(getattr(v, "is_real", None) is not False for v in s.values()):
2391             real.append(s)
2392     if not real:
2393         return False, "solutions exist but none are real"
2394     return True, f"fixed set: {real}"
2395
2396
2397 def verify_quotient(model: MetricModel,
2398                   probe: Mapping[str, float] | None = None) -> QuotientCertificate:
2399     """Full L3 verification of a model's identification structure."""
2400     q = model.quotient
2401     cert = QuotientCertificate(model_id=model.id, label=q.label,
2402                              trivial=not q.generators)
2403
2404     # Coordinate periodicities declared via CompactDirection are the special
2405     # case  $\Gamma = \langle x^i \rightarrow x^i + P \rangle$ . Verify each is an isometry too.
2406     for cd in model.compact:
2407         row: dict[str, Any] = {"coordinate": cd.name, "index": cd.index,
2408                              "period": str(cd.period)}
2409         if model.metric is not None:
2410             d = sp.simplify(sp.diff(model.metric, model.coords[cd.index]))
2411             row["metric_independent_of_coordinate"] = bool(d.is_zero_matrix)
2412             row["killing_residual"] = "0" if d.is_zero_matrix else str(d)
2413             cert.coordinate_periodicities.append(row)
2414
2415     if model.metric is None:
2416         cert.note = ("model has no local metric; identification structure is "
2417                    "recorded as an external/global specification")
2418     for gen in q.generators:
2419         cert.generators.append(DeckCertificate(
2420             name=gen.name, isometry_residual="not-applicable",
2421             is_isometry=True, time_orientation_preserving=gen.time_orientation_preserving,
2422             has_fixed_point=None,
2423             note="isometry not checkable without a local metric"))
2424     return cert
2425
2426     residuals = q.verify_isometries(model.coords, model.metric)
2427     for gen in q.generators:
2428         res = residuals.get(gen.name, "unknown")
2429         tof, tnote = _time_orientation_preserving(model, gen, probe)
2430         fp, fnote = _has_fixed_point(model, gen)
2431         cert.generators.append(DeckCertificate(
2432             name=gen.name, isometry_residual=res, is_isometry=(res == "0"),
2433             time_orientation_preserving=tof, has_fixed_point=fp,
2434             fixed_point_note=fnote, note=tnote))
2435     return cert
2436
2437
2438 # -----
2439 # Closure on the quotient
2440 # -----
2441
2442 @dataclass
2443 class ClosureResult:
2444     closed: bool
2445     word: tuple[str, ...] | None
2446     residual: float
2447     mechanism: str
2448     note: str = ""
2449
2450     def as_dict(self) -> dict[str, Any]:
2451         return {"closed": self.closed, "word": list(self.word) if self.word else None,
2452                "residual": self.residual, "mechanism": self.mechanism,
2453                "note": self.note}
2454
2455
2456 def curve_closes(model: MetricModel, p0: Sequence[float], p1: Sequence[float],
2457                 tol: float = 1e-9, max_word_length: int = 3,
2458                 params: Mapping[Any, float] | None = None) -> ClosureResult:
2459     """Does the curve segment from  $p_0$  to  $p_1$  close on  $M/\Gamma$ ?
2460
2461     Three mechanisms are tried in order:
2462

```



```

2463 1. exact coincidence in the chart (`p1 == p0`);
2464 2. coordinate periodicity from `model.compact`;
2465 3. a non-trivial word in the deck group `Gamma`.
2466
2467 Mechanism 3 is what v1.0 could not express.
2468 """
2469 p0 = [float(v) for v in p0]
2470 p1 = [float(v) for v in p1]
2471 if not all(math.isfinite(v) for v in p0 + p1):
2472     return ClosureResult(False, None, float("nan"), "invalid",
2473                          "curve endpoints are not finite")
2474 res = max(abs(a - b) for a, b in zip(p0, p1))
2475 if res <= tol:
2476     return ClosureResult(True, (), res, "identity",
2477                          "curve returns to the same chart point")
2478
2479 if model.compact:
2480     delta = [b - a for a, b in zip(p0, p1)]
2481     reduced = list(delta)
2482     used = []
2483     for cd in model.compact:
2484         period = 0.0
2485         for src in (params, model.default_point):
2486             if not src:
2487                 continue
2488             try:
2489                 sub = (src if all(isinstance(k, sp.Symbol) for k in src)
2490                       else model.substitution(src))
2491                 period = float(sp.N(cd.period.subs(sub)))
2492                 break
2493             except Exception:
2494                 # noqa: BLE001
2495                 continue
2496         if not period:
2497             try:
2498                 period = float(sp.N(cd.period))
2499             except Exception:
2500                 # noqa: BLE001
2501                 continue
2502         if period == 0 or not math.isfinite(period):
2503             continue
2504         k = round(reduced[cd.index] / period)
2505         if k != 0:
2506             used.append(f"{cd.name}+{k}*P")
2507             reduced[cd.index] -= k * period
2508         r2 = max(abs(v) for v in reduced)
2509         if r2 <= tol:
2510             return ClosureResult(True, tuple(used), r2, "coordinate_period",
2511                                 "closed modulo declared coordinate periods")
2512
2513 if model.quotient.generators:
2514     closed, word, r3 = model.quotient.closes(
2515         model.coords, p0, p1, tol=tol, max_length=max_word_length,
2516         params=params)
2517     if closed:
2518         return ClosureResult(True, word, r3, "deck_group",
2519                             f"gamma(1) = {word or 'e'} . gamma(0)")
2520     return ClosureResult(False, word, r3, "deck_group",
2521                         f"best residual over words of length <= {max_word_length}")
2522
2523 return ClosureResult(False, None, res, "none",
2524                     "no identification structure registered for this model")
2525
2526 # -----
2527 # Topological classification
2528 # -----
2529
2530 def classify(model: MetricModel) -> dict[str, Any]:
2531     """A compact machine summary of the model's global structure."""
2532     q = model.quotient
2533     return {
2534         "model_id": model.id,
2535         "kind": model.kind.value,
2536         "topology": model.topology,
2537         "n_coordinate_identifications": len(model.compact),
2538         "coordinate_identifications": [
2539             {"coordinate": c.name, "period": str(c.period)} for c in model.compact],
2540         "deck_group": q.label,
2541         "n_deck_generators": len(q.generators),
2542         "simply_connected_chart": not model.compact and not q.generators,
2543         "requires_global_predicate": model.global_margin is not None,
2544     }
2545
2546 def compare_global_structure(a: MetricModel, b: MetricModel) -> dict[str, Any]:
2547     """Compare two models that may share a local metric.
2548
2549     This is the executable form of the L3 gate. Used by the AdS pair test.
2550     """
2551     same_metric = False
2552     note = ""
2553     if a.metric is not None and b.metric is not None and a.dim == b.dim:
2554         try:
2555             ren = {bc: ac for ac, bc in zip(a.coords, b.coords)}
2556             bm = sp.Matrix(b.metric).subs(ren, simultaneous=True)
2557             same_metric = bool(sp.simplify(sp.Matrix(a.metric) - bm).is_zero_matrix)
2558         except Exception as exc:
2559             # noqa: BLE001
2560             note = f"comparison failed: {exc}"
2561     return {
2562         "a": a.id, "b": b.id,
2563         "identical_local_metric": same_metric,
2564         "a_structure": classify(a), "b_structure": classify(b),
2565         "a_expected": a.expected.value, "b_expected": b.expected.value,
2566         "demonstrates_L3_gate": bool(same_metric and a.expected != b.expected),
2567         "note": note,
2568     }
2569
2570 __all__ = ["DeckCertificate", "QuotientCertificate", "verify_quotient",
2571            "ClosureResult", "curve_closes", "classify", "compare_global_structure"]

```

src/ctcfa/symmetry.py — 210 lines

```

2572 """
2573 CTC-FA v2.0 -- L4 : Killing / symmetry engine.
2574
2575 Two jobs.
2576
2577 1. **Admissibility of declared compact generators.** v1.0 allowed a registry
2578 entry to declare any coordinate direction as a "compact generator" and the
2579 detector consumed it without question. A direction that is not a Killing
2580 vector does not generate a one-parameter isometry group, so the orbit it
2581 traces is not the symmetry orbit the compact-orbit argument assumes, and
2582 the resulting "witness" is meaningless. v2.0 refuses a generator whose
2583 Killing residual is non-zero unless the registry record explicitly waives
2584 the requirement with a documented reason.
2585
2586 2. **Discovery of Killing vectors.** ``solve_killing`` solves the Killing
2587 equation for a supplied ansatz (constant, linear, or user-provided). For
2588 the flat and maximally symmetric benchmarks it recovers the full isometry
2589 algebra, which is used as a correctness test of the tensor engine.
2590
2591 Compact-generator Gram matrix
2592 -----
2593 For generators ``xi_A`` the Gram matrix is ``K_AB(x) = g_{mu nu} xi_A^mu
2594 xi_B^nu`` and the causal character of the closed orbit with integer winding
2595 ``n^A`` is the sign of ``q(x, n) = n^A K_AB(x) n^B``.  $q < 0$  is an explicit
2596 symmetry-generated CTC;  $q = 0$  is a closed null orbit;  $q > 0$  clears only the
2597 scanned sector.
2598 """
2599
2600 from __future__ import annotations
2601
2602 from dataclasses import dataclass, field
2603 from typing import Any, Mapping, Sequence
2604
2605 import numpy as np
2606 import sympy as sp
2607
2608 from .model import CompactDirection, GeneratorCertificate, MetricModel
2609 from .symbolic import coordinate_killing_residual, lie_derivative_metric
2610
2611 # -----
2612 # Verification of declared generators
2613 # -----
2614
2615 def verify_compact_generators(model: MetricModel) -> list[GeneratorCertificate]:
2616     """Certify each declared compact generator (Killing + closure)."""
2617     out: list[GeneratorCertificate] = []
2618     if model.metric is None:
2619         for cd in model.compact:
2620             out.append(GeneratorCertificate(
2621                 name=cd.name, is_killing=False, killing_residual="no local metric",
2622                 closes=True,
2623                 note="global/composite model: symmetry claim is external"))
2624     return out
2625
2626 for cd in model.compact:
2627     res = coordinate_killing_residual(model.metric, model.coords, cd.index)
2628     is_k = bool(res.is_zero_matrix)
2629     closes = True # coordinate periodicity closes by construction
2630     note = cd.note
2631     if not is_k:
2632         note = (note + " | " if note else "") + \
2633             "declared generator is NOT Killing for this metric"
2634     out.append(GeneratorCertificate(
2635         name=cd.name, is_killing=is_k,
2636         killing_residual="0" if is_k else str(res), closes=closes, note=note))
2637 return out
2638
2639 def generators_admissible(model: MetricModel) -> tuple[bool, list[GeneratorCertificate]]:
2640     certs = verify_compact_generators(model)
2641     ok = True
2642     for cd, c in zip(model.compact, certs):
2643         if cd.requires_killing and not c.admissible:
2644             ok = False
2645     return ok, certs
2646
2647 # -----
2648 # Gram matrix of the compact generators
2649 # -----
2650
2651 def compact_gram_symbolic(model: MetricModel) -> sp.Matrix:
2652     """ $K_{AB} = g(x_i^A, x_i^B)$  for the declared coordinate generators."""
2653     g = model.require_metric()
2654     idx = [c.index for c in model.compact]
2655     if not idx:
2656         return sp.zeros(0, 0)
2657     return g.extract(idx, idx)
2658
2659 def compact_gram_numeric(model: MetricModel,
2660     subs: Mapping[str | sp.Symbol, float]) -> np.ndarray:
2661     K = compact_gram_symbolic(model)
2662     s = model.substitution(subs)
2663     return np.array(sp.Matrix(K).subs(s).evalf(), dtype=float)
2664
2665 def winding_norm_symbolic(model: MetricModel, winding: Sequence[int]) -> sp.Expr:
2666     """ $q(n) = n^A K_{AB} n^B$  as an exact expression."""
2667     K = compact_gram_symbolic(model)
2668     n = sp.Matrix([sp.Integer(w) for w in winding])
2669     if n.rows != K.rows:
2670         raise ValueError(f"winding vector length {n.rows} != number of compact "
2671             f"generators {K.rows} for model {model.id}")
2672     return sp.expand((n.T * K * n)[0])
2673
2674
2675

```

```

2679 def orbit_length_symbolic(model: MetricModel, winding: Sequence[int]) -> sp.Expr:
2680     """Proper length/time of one closed symmetry orbit with winding ``n``.
2681
2682     For commuting Killing generators with periods ``P_A`` the orbit
2683     ``lambda -> exp(lambda n^A P_A xi_A) . x`` has tangent
2684     ``n^A P_A xi_A`` and constant norm, so the (squared) invariant interval
2685     of one full circuit is ``sum_{A,B} n^A P_A n^B P_B K_AB``.
2686     """
2687     K = compact_gram_symbolic(model)
2688     v = sp.Matrix([sp.Integer(w) * cd.period
2689                    for w, cd in zip(winding, model.compact)])
2690     return sp.expand((v.T * K * v)[0])
2691
2692
2693 # -----
2694 # Killing equation solver
2695 # -----
2696
2697 @dataclass
2698 class KillingSolution:
2699     dimension: int
2700     basis: list[list[sp.Expr]] = field(default_factory=list)
2701     ansatz: str = ""
2702     note: str = ""
2703
2704     def as_dict(self) -> dict[str, Any]:
2705         return {"dimension": self.dimension, "ansatz": self.ansatz,
2706                "basis": [[str(c) for c in v] for v in self.basis],
2707                "note": self.note}
2708
2709
2710 def solve_killing(model: MetricModel, ansatz: str = "linear") -> KillingSolution:
2711     """Solve ``L_xi g = 0`` for a polynomial ansatz in the coordinates.
2712
2713     ``ansatz`` is ``"constant"`` (translations only) or ``"linear"``
2714     (translations plus linear terms; recovers the full Poincare algebra on
2715     Minkowski, dimension 10).
2716     """
2717     g = model.require_metric()
2718     coords = list(model.coords)
2719     n = len(coords)
2720     a = sp.symbols(f"a0:{n}", real=True)
2721     unknowns = list(a)
2722     xi: list[sp.Expr] = list(a)
2723     if ansatz == "linear":
2724         b = sp.symbols(f"b0:{n * n}", real=True)
2725         unknowns += list(b)
2726         xi = [a[i] + sum(b[i * n + j] * coords[j] for j in range(n)) for i in range(n)]
2727     elif ansatz != "constant":
2728         raise ValueError("ansatz must be 'constant' or 'linear'")
2729
2730     L = lie_derivative_metric(g, coords, xi, simplify=True)
2731     eqs: list[sp.Expr] = []
2732     for i in range(n):
2733         for j in range(i, n):
2734             e = sp.expand(L[i, j])
2735             if e == 0:
2736                 continue
2737             poly_eqs = _split_into_coefficient_equations(e, coords)
2738             eqs.extend(poly_eqs)
2739     eqs = [e for e in {sp.simplify(e) for e in eqs} if e != 0]
2740     if not eqs:
2741         basis = _basis_from_free(xi, unknowns, unknowns)
2742         return KillingSolution(len(basis), basis, ansatz,
2743                               "every ansatz coefficient is unconstrained")
2744     sol = sp.solve(eqs, unknowns, dict=True)
2745     if not sol:
2746         return KillingSolution(0, [], ansatz, "no solution in this ansatz")
2747     s = sol[0]
2748     xi_sol = [sp.expand(c.subs(s)) for c in xi]
2749     free = sorted({u for u in unknowns if u not in s} |
2750                  {sym for v in s.values() for sym in v.free_symbols
2751                   if sym in set(unknowns)}, key=str)
2752     basis = _basis_from_free(xi_sol, free, unknowns)
2753     return KillingSolution(len(basis), basis, ansatz,
2754                           f"{len(eqs)} independent Killing equations solved")
2755
2756
2757 def _split_into_coefficient_equations(expr: sp.Expr,
2758                                       coords: Sequence[sp.Symbol]) -> list[sp.Expr]:
2759     """Turn ``sum_k c_k(c coeffs) f_k(x) = 0`` into ``{c_k = 0}`` where possible."""
2760     try:
2761         p = sp.Poly(sp.expand(expr), *coords)
2762     except sp.PolynomialError:
2763         return [expr]
2764     return [sp.simplify(c) for c in p.coeffs()]
2765
2766
2767 def _basis_from_free(xi: Sequence[sp.Expr], free: Sequence[sp.Symbol],
2768                    unknowns: Sequence[sp.Symbol]) -> list[list[sp.Expr]]:
2769     basis: list[list[sp.Expr]] = []
2770     for f in free:
2771         sub = {u: (1 if u == f else 0) for u in unknowns}
2772         v = [sp.expand(sp.simplify(c.subs(sub))) for c in xi]
2773         if any(c != 0 for c in v):
2774             basis.append(v)
2775     return basis
2776
2777
2778 __all__ = ["verify_compact_generators", "generators_admissible",
2779           "compact_gram_symbolic", "compact_gram_numeric",
2780           "winding_norm_symbolic", "orbit_length_symbolic",
2781           "KillingSolution", "solve_killing"]

```

src/ctcfa/orientation.py — 176 lines

```

2782 """
2783 CTC-FA v2.0 -- Time-orientation engine.

```

```

2784
2785 Section 3.2 of v1.0 states the requirement plainly:
2786
2787 "A complete witness must also respect a consistent time orientation. A
2788 curve that changes the chosen future direction is not accepted as a
2789 physical future-directed CTC witness."
2790
2791 The v1.0 kernel never implemented it. ``validate_sampled_loop`` returned
2792 ``"orientation_status": "NOT_CHECKED"`` and ``compact_orbit_test`` did not
2793 look at orientation at all, so a closed *timelike* curve that reverses the
2794 future direction -- which is not a causality violation, merely a
2795 non-time-orientable circuit -- would have been reported as a CTC. That is
2796 v2.0 defect **D-03**.
2797
2798 Method
2799 -----
2800 1. At every sample point the metric matrix ``g`` is diagonalised. Its unique
2801 negative-eigenvalue eigenvector ``T`` is timelike, because
2802 ``T^t g T = lambda |T|^2 < 0``.
2803 2. The sign of ``T`` is fixed by *continuity in the cone*: two timelike
2804 vectors lie in the same cone iff their inner product is negative, so the
2805 sign of ``T_i`` is chosen such that ``g(T_i, T_{i-1}) < 0``. This
2806 produces a continuous future-directed reference field along the curve
2807 without needing a global time function.
2808 3. A curve is orientation-consistent when ``g(gamma_dot, T)`` never changes
2809 sign along the curve.
2810 4. For a closed curve on a quotient the field must also close: the reference
2811 at the endpoint has to lie in the same cone as the deck-transported
2812 reference from the start point. A sign flip there is a *time-orientation
2813 holonomy* and disqualifies the loop as a future-directed CTC witness.
2814 ""
2815
2816 from __future__ import annotations
2817
2818 from dataclasses import dataclass, field
2819 from typing import Any, Callable, Sequence
2820
2821 import numpy as np
2822
2823 @dataclass
2824 class OrientationResult:
2825     consistent: bool
2826     reversals: int
2827     holonomy_ok: bool | None
2828     min_abs_inner: float
2829     reference_field: list[list[float]] = field(default_factory=list)
2830     note: str = ""
2831
2832     def as_dict(self) -> dict[str, Any]:
2833         return {"consistent": self.consistent, "reversals": self.reversals,
2834                 "holonomy_ok": self.holonomy_ok,
2835                 "min_abs_inner": self.min_abs_inner, "note": self.note}
2836
2837 def timelike_eigenvector(g: np.ndarray) -> tuple[np.ndarray, float]:
2838     """Return a timelike vector at a point plus its eigenvalue."""
2839     gs = (g + g.T) / 2.0
2840     vals, vecs = np.linalg.eigh(gs)
2841     i = int(np.argmin(vals))
2842     return np.asarray(vecs[:, i], dtype=float), float(vals[i])
2843
2844 def continuous_reference_field(metrics: Sequence[np.ndarray],
2845                               seed: np.ndarray | None = None
2846                               ) -> tuple[list[np.ndarray], str]:
2847     """Build a cone-continuous timelike reference field along a sample path."""
2848     field: list[np.ndarray] = []
2849     note = ""
2850     prev: np.ndarray | None = None
2851     prev_g: np.ndarray | None = None
2852     for g in metrics:
2853         T, lam = timelike_eigenvector(np.asarray(g, dtype=float))
2854         if lam >= 0:
2855             note = "metric has no timelike eigendirection at some sample point"
2856         if prev is None:
2857             if seed is not None and float(seed @ np.asarray(g) @ T) > 0:
2858                 T = -T
2859             else:
2860                 ip = float(prev @ np.asarray(g, dtype=float) @ T)
2861                 if ip > 0:
2862                     T = -T
2863             field.append(T)
2864             prev, prev_g = T, np.asarray(g, dtype=float)
2865     return field, note
2866
2867 def check_curve_orientation(metrics: Sequence[np.ndarray],
2868                             tangents: Sequence[np.ndarray],
2869                             closed: bool = True,
2870                             deck_jacobian: np.ndarray | None = None,
2871                             tol: float = 1e-12) -> OrientationResult:
2872     """Full orientation test for a sampled curve.
2873
2874     Parameters
2875     -----
2876     metrics
2877         ``g`` at each sample point, in coordinate components.
2878     tangents
2879         ``gamma_dot`` at each sample point, same components.
2880     closed
2881         Whether the curve is closed (then the holonomy test applies).
2882     deck_jacobian
2883         Jacobian ``d phi / d x`` of the deck element that identifies the
2884         endpoint with the start point, if the closure used one. ``None``
2885         means the identity.
2886     """
2887     metrics = [np.asarray(g, dtype=float) for g in metrics]
2888     tangents = [np.asarray(v, dtype=float) for v in tangents]
2889     ref, note = continuous_reference_field(metrics)

```

```

2894
2895     inners = [float(v @ g @ T) for v, g, T in zip(tangents, metrics, ref)]
2896     finite = [x for x in inners if np.isfinite(x)]
2897     if not finite:
2898         return OrientationResult(False, 0, None, 0.0, note="no finite inner products")
2899     signs = [1 if x > 0 else -1 for x in finite]
2900     reversals = sum(1 for a, b in zip(signs, signs[1:]) if a != b)
2901     min_abs = float(min(abs(x) for x in finite))
2902
2903     holonomy_ok: bool | None = None
2904     if closed and len(ref) >= 2:
2905         J = np.eye(metrics[0].shape[0]) if deck_jacobian is None \
2906             else np.asarray(deck_jacobian, dtype=float)
2907         start_pushed = J @ ref[0]
2908         ip = float(start_pushed @ metrics[-1] @ ref[-1])
2909         if abs(ip) <= tol:
2910             holonomy_ok = None
2911             note = (note + " | " if note else "") + "degenerate holonomy probe"
2912         else:
2913             holonomy_ok = ip < 0
2914             if not holonomy_ok:
2915                 note = (note + " | " if note else "") + \
2916                     "time-orientation holonomy reverses around the loop"
2917
2918     consistent = (reversals == 0) and (holonomy_ok is not False) and min_abs > tol
2919     if min_abs <= tol:
2920         note = (note + " | " if note else "") + \
2921             "tangent is (numerically) orthogonal to the reference field"
2922     return OrientationResult(consistent, reversals, holonomy_ok, min_abs,
2923                             [list(map(float, T)) for T in ref], note)
2924
2925
2926 def orbit_orientation(g_at_point: np.ndarray, generator: np.ndarray,
2927                     tol: float = 1e-12) -> OrientationResult:
2928     """Orientation test for a *symmetry orbit* of a Killing generator.
2929
2930     A closed orbit of a Killing vector has constant tangent ``xi`` in the
2931     adapted chart and the metric is constant along it, so orientation is
2932     consistent by construction whenever ``xi`` is timelike; the test reduces
2933     to checking that ``xi`` is genuinely timelike and non-degenerate. This
2934     is stated explicitly so that the compact-orbit detector's orientation
2935     claim is derived, not assumed.
2936     """
2937     g = np.asarray(g_at_point, dtype=float)
2938     xi = np.asarray(generator, dtype=float)
2939     q = float(xi @ g @ xi)
2940     if not np.isfinite(q):
2941         return OrientationResult(False, 0, None, 0.0, note="non-finite orbit norm")
2942     if q >= -tol:
2943         return OrientationResult(False, 0, None, abs(q),
2944                                 note="orbit tangent is not timelike; orientation "
2945                                     "consistency is vacuous")
2946     T, _ = timelike_eigenvector(g)
2947     ip = float(xi @ g @ T)
2948     if ip > 0:
2949         T = -T
2950     ip = -ip
2951     return OrientationResult(True, 0, True, abs(ip), [list(map(float, T))],
2952                             "Killing orbit: metric and tangent are invariant along "
2953                             "the orbit, so the future direction cannot rotate")
2954
2955
2956 __all__ = ["OrientationResult", "timelike_eigenvector", "continuous_reference_field",
2957            "check_curve_orientation", "orbit_orientation"]

```

src/ctcfa/metrics.py — 1141 lines

```

2958 """
2959 CTC-FA v2.0 -- L0 : the metric and construction registry.
2960
2961 Every entry declares, at minimum: coordinate ordering, signature, the metric
2962 in the (-,+,+,+) convention, the parameter domain, the identification /
2963 quotient structure, the matter content, the expected status *with its scope*,
2964 and the literature reference the form was taken from.
2965
2966 Corrections applied to the v1.0 registry
2967 -----
2968 D-01 Kerr ``g_t_phi``. The v1.0 registry table printed
2969 ``g_tphi = +2 M a r sin^2(theta) / Sigma`` while the v1.0 kernel used
2970 the negative sign. In Boyer-Lindquist coordinates with the (-,+,+,+)
2971 signature the correct component is
2972
2973     g_t_phi = - 2 M a r sin^2(theta) / Sigma,
2974
2975 because ds^2 contains ``-(4 M a r sin^2 theta / Sigma) dt dphi``. The
2976 kernel was right and the table was wrong; v2.0 fixes the table, and
2977 ``tests/unit/test_kerr_conventions.py`` pins the sign by checking the
2978 frame-dragging direction and the Kerr limit ``a -> 0``.
2979
2980 D-02 van Stockum symmetry. The v1.0 kernel built the van Stockum metric
2981 with the ``dt dphi`` term placed in one off-diagonal slot only, so
2982 ``g != g^T``. The compact-orbit detector reads only ``g_phi_phi`` and
2983 therefore still produced the published margin, which is why the defect
2984 survived the self-test, but every curvature, Einstein-tensor or
2985 geodesic computation on that model would have been wrong. v2.0 stores
2986 the symmetric form and ``MetricModel.assert_wellformed`` makes an
2987 asymmetric registry entry impossible to load.
2988
2989 D-06 Krasnikov regulator. The tube metric depends on a smooth-step
2990 regulator that is a numerical choice, not part of the exact solution.
2991 v2.0 marks the entry ``regulator_dependent`` so that no causal boundary
2992 may be derived from it.
2993
2994 D-07 Tipler / Ori-2007-full / Gott / paired-warp are ``EXTERNAL_EXACT_SPEC``
2995 or ``COMPOSITE_GLOBAL``: they have no canonical local 4x4 tensor and
2996 ``require_metric`` raises rather than returning an invented one.
2997
2998 New in v2.0

```

```

2999 -----
3000 The ``ads4_universal_cover`` / ``ads4_periodic`` pair carries one identical
3001 local metric and receives opposite global verdicts. It is the executable
3002 proof of the L3 gate ("local equality of metrics does not imply equality of
3003 global causal structure"), which v1.0 asserted but could not test.
3004 ``misner_covering`` and ``grant_space`` are flat quotients whose closed
3005 curves are generated by a boost rather than by coordinate periodicity, and
3006 are handled by the deck-orbit detector.
3007 """
3008
3009 from __future__ import annotations
3010
3011 import math
3012 from typing import Mapping
3013
3014 import sympy as sp
3015
3016 from .model import (CompactDirection, DeckTransformation, DomainConstraint,
3017                    MatterSpec, MetricModel, Provenance, Quotient)
3018 from .status import CTCStatus, ClaimScope, ModelKind
3019
3020 pi = sp.pi
3021
3022 def _dom(name: str, expr: sp.Expr, strict: bool = True, note: str = "") -> DomainConstraint:
3023     return DomainConstraint(name, sp.sympify(expr), strict, note)
3024
3025
3026 # =====
3027 # M01 Minkowski
3028 # =====
3029
3030 def minkowski() -> MetricModel:
3031     t, x, y, z = sp.symbols("t x y z", real=True)
3032     g = sp.diag(-1, 1, 1, 1)
3033     return MetricModel(
3034         id="minkowski", title="M01 Minkowski spacetime",
3035         coords=(t, x, y, z), metric=g,
3036         kind=ModelKind.LOCAL_METRIC, theory="GR",
3037         matter=MatterSpec("vacuum", T=sp.zeros(4, 4), vacuum=True),
3038         topology="R^4, simply connected",
3039         expected=CTCStatus.NO_CTC_CERTIFIED, expected_scope=ClaimScope.GLOBAL,
3040         temporal_function=t,
3041         default_point={"t": 0.0, "x": 1.0, "y": 0.0, "z": 0.0},
3042         domain_notes="Primary negative control for geometry, temporal-function "
3043                     "certification and tensor residual tests.",
3044         provenance=Provenance(("R26", "R28"), "signature (-,+,+,+)",
3045                               "flat metric by definition"),
3046         metadata={"role": "negative_control", "flat_covering": True})
3047
3048
3049 # =====
3050 # M02 Spatially flat FLRW (symbolic scale factor)
3051 # =====
3052
3053 def flrw_flat() -> MetricModel:
3054     t, x, y, z = sp.symbols("t x y z", real=True)
3055     a = sp.Function("a", positive=True)(t)
3056     g = sp.diag(-1, a ** 2, a ** 2, a ** 2)
3057     return MetricModel(
3058         id="flrw_flat", title="M02 Spatially flat FLRW (general scale factor)",
3059         coords=(t, x, y, z), metric=g,
3060         kind=ModelKind.LOCAL_METRIC, theory="GR",
3061         matter=MatterSpec("model-dependent cosmological matter"),
3062         topology="R x R^3 in the baseline chart",
3063         expected=CTCStatus.NO_CTC_CERTIFIED, expected_scope=ClaimScope.GLOBAL,
3064         temporal_function=t,
3065         domain_notes="Cosmic time t satisfies g^tt = -1 for ANY a(t) > 0, so the "
3066                     "temporal-function certificate is exact and scale-factor "
3067                     "independent. Tests that expansion and curvature alone do not "
3068                     "trigger a false CTC classification.",
3069         provenance=Provenance(("R26", "R27"), "signature (-,+,+,+)"),
3070         metadata={"role": "cosmological_control", "numeric_evaluable": False})
3071
3072
3073 def flrw_exponential() -> MetricModel:
3074     t, x, y, z = sp.symbols("t x y z", real=True)
3075     H = sp.symbols("H", positive=True)
3076     a = sp.exp(H * t)
3077     g = sp.diag(-1, a ** 2, a ** 2, a ** 2)
3078     return MetricModel(
3079         id="flrw_exponential", title="M02b Spatially flat FLRW, a(t)=exp(H t)",
3080         coords=(t, x, y, z), params=(H,), metric=g,
3081         kind=ModelKind.LOCAL_METRIC, theory="GR",
3082         matter=MatterSpec("de Sitter (Lambda = 3 H^2)", vacuum=True,
3083                           cosmological_constant=3 * H ** 2),
3084         topology="R x R^3 (flat slicing chart of de Sitter)",
3085         expected=CTCStatus.NO_CTC_CERTIFIED, expected_scope=ClaimScope.GLOBAL,
3086         temporal_function=t,
3087         default_point={"t": 0.0, "x": 1.0, "y": 0.0, "z": 0.0, "H": 1.0},
3088         domain_notes="Numerically evaluable companion to flrw_flat.",
3089         provenance=Provenance(("R26", "R27")),
3090         metadata={"role": "cosmological_control"})
3091
3092
3093 # =====
3094 # M03 Schwarzschild
3095 # =====
3096
3097 def schwarzschild() -> MetricModel:
3098     t, r, th, ph = sp.symbols("t r theta phi", real=True)
3099     M = sp.symbols("M", positive=True)
3100     f = 1 - 2 * M / r
3101     g = sp.diag(-f, 1 / f, r ** 2, r ** 2 * sp.sin(th) ** 2)
3102     return MetricModel(
3103         id="schwarzschild", title="M03 Schwarzschild",
3104         coords=(t, r, th, ph), params=(M,), metric=g,
3105         compact=(CompactDirection(3, "phi", 2 * pi)),
3106         kind=ModelKind.LOCAL_METRIC, theory="GR",
3107         matter=MatterSpec("vacuum", T=sp.zeros(4, 4), vacuum=True),

```

```

3109     topology="R^2 x S^2 exterior chart; phi ~ phi + 2 pi",
3110     expected=CTCStatus.NO_CTC_CERTIFIED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3111     temporal_function=t,
3112     domain=(dom("exterior", r - 2 * M, True,
3113         "the Schwarzschild t coordinate is a temporal function only "
3114         "outside the horizon"),
3115         dom("polar_regular", sp.sin(th), True)),
3116     default_point={"t": 0.0, "r": 5.0, "theta": float(sp.pi / 2), "phi": 0.0, "M": 1.0},
3117     domain_notes="r = 2M is an event horizon, NOT a chronology horizon. ",
3118     "Separates horizon detection from causality violation.",
3119     provenance=Provenance(("R26", "R28"), "standard exterior chart"),
3120     metadata={"role": "negative_control", "horizon": "r = 2M",
3121         "kretschmann": "48 M^2 / r^6"})
3122
3123
3124 # =====
3125 # M04 Reissner-Nordstrom
3126 # =====
3127
3128 def reissner_nordstrom() -> MetricModel:
3129     t, r, th, ph = sp.symbols("t r theta phi", real=True)
3130     M, Q = sp.symbols("M Q", real=True, positive=True)
3131     f = 1 - 2 * M / r + Q ** 2 / r ** 2
3132     g = sp.diag(-f, 1 / f, r ** 2, r ** 2 * sp.sin(th) ** 2)
3133     return MetricModel(
3134         id="reissner_nordstrom", title="M04 Reissner-Nordstrom",
3135         coords=(t, r, th, ph), params=(M, Q), metric=g,
3136         compact=(CompactDirection(3, "phi", 2 * pi)),
3137         kind=ModelKind.LOCAL_METRIC, theory="Einstein-Maxwell",
3138         matter=MatterSpec("electrovacuum"),
3139         topology="phi ~ phi + 2 pi",
3140         expected=CTCStatus.NO_CTC_CERTIFIED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3141         temporal_function=t,
3142         domain=(dom("outside_outer_horizon", r - (M + sp.sqrt(M ** 2 - Q ** 2)), True),
3143             dom("subextremal", M ** 2 - Q ** 2, True),
3144             dom("polar_regular", sp.sin(th), True)),
3145         default_point={"t": 0.0, "r": 5.0, "theta": float(sp.pi / 2), "phi": 0.0,
3146             "M": 1.0, "Q": 0.5},
3147         domain_notes="Critical negative control against the logical error "
3148             "'Cauchy horizon implies CTC'. r_pm = M +/- sqrt(M^2 - Q^2).",
3149         provenance=Provenance(("R26", "R28")),
3150         metadata={"role": "negative_control",
3151             "cauchy_horizon": "r_- = M - sqrt(M^2 - Q^2)})
3152
3153
3154 # =====
3155 # M05 Kerr (D-01 sign corrected)
3156 # =====
3157
3158 def kerr() -> MetricModel:
3159     t, r, th, ph = sp.symbols("t r theta phi", real=True)
3160     M = sp.symbols("M", positive=True)
3161     a = sp.symbols("a", real=True)
3162     Sig = r ** 2 + a ** 2 * sp.cos(th) ** 2
3163     Del = r ** 2 - 2 * M * r + a ** 2
3164     gtt = -(1 - 2 * M * r / Sig)
3165     gtph = -2 * M * a * r * sp.sin(th) ** 2 / Sig # D-01: negative
3166     gphph = sp.sin(th) ** 2 * (r ** 2 + a ** 2
3167         + 2 * M * a ** 2 * r * sp.sin(th) ** 2 / Sig)
3168     g = sp.Matrix([[gtt, 0, 0, gtph],
3169         [0, Sig / Del, 0, 0],
3170         [0, 0, Sig, 0],
3171         [gtph, 0, 0, gphph]])
3172     return MetricModel(
3173         id="kerr", title="M05 Kerr (Boyer-Lindquist)",
3174         coords=(t, r, th, ph), params=(M, a), metric=g,
3175         compact=(CompactDirection(3, "phi", 2 * pi)),
3176         kind=ModelKind.LOCAL_METRIC, theory="GR",
3177         matter=MatterSpec("vacuum away from the ring singularity", vacuum=True),
3178         topology="maximal analytic extension including r < 0; phi ~ phi + 2 pi",
3179         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3180         domain=(dom("nondegenerate_Sigma", Sig, True),
3181             dom("off_horizon", Del ** 2, True),
3182             dom("polar_regular", sp.sin(th), True)),
3183         default_point={"t": 0.0, "r": -0.5, "theta": float(sp.pi / 2), "phi": 0.0,
3184             "M": 1.0, "a": 0.8},
3185         domain_notes="In part of the analytically extended r < 0 region "
3186             "g_phi_phi < 0, so constant-(t,r,theta) azimuthal loops are "
3187             "timelike. Mathematical existence; operational accessibility "
3188             "and stability of that region are separate claims.",
3189         provenance=Provenance(("R3", "R26"),
3190             "g_t_phi = -2 M a r sin^2(theta)/Sigma in (-,+,+); "
3191             "the v1.0 registry table printed the opposite sign "
3192             "(defect D-01)"),
3193         metadata={"role": "rotation_benchmark", "ctc_region": "r < 0 near theta = pi/2",
3194             "sector_boundary": "g_phi_phi = 0"})
3195
3196
3197 def kerr_extremal() -> MetricModel:
3198     m = kerr()
3199     M, a = m.symbol("M"), m.symbol("a")
3200     g = m.metric.subs(a, M)
3201     return MetricModel(
3202         id="kerr_extremal", title="M05b Extremal Kerr (a = M)",
3203         coords=m.coords, params=(M,), metric=g,
3204         compact=m.compact, kind=ModelKind.LOCAL_METRIC, theory="GR",
3205         matter=MatterSpec("vacuum", vacuum=True),
3206         topology="phi ~ phi + 2 pi; degenerate horizon at r = M",
3207         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3208         default_point={"t": 0.0, "r": -0.5, "theta": float(sp.pi / 2), "phi": 0.0,
3209             "M": 1.0},
3210         domain_notes="Extremal limit control: the CTC sector survives a -> M, so "
3211             "the sector is not an artefact of subextremality.",
3212         provenance=Provenance(("R3", "R26")),
3213         metadata={"role": "limit_control"})
3214
3215
3216 # =====
3217 # M06 Kerr-Newman
3218 # =====

```



```

3219
3220 def kerr_newman() -> MetricModel:
3221     t, r, th, ph = sp.symbols("t r theta phi", real=True)
3222     M = sp.symbols("M", positive=True)
3223     a, Q = sp.symbols("a Q", real=True)
3224     Sig = r ** 2 + a ** 2 * sp.cos(th) ** 2
3225     A = 2 * M * r - Q ** 2
3226     Del = r ** 2 - 2 * M * r + a ** 2 + Q ** 2
3227     gtt = -(1 - A / Sig)
3228     gtph = -a * sp.sin(th) ** 2 * A / Sig
3229     gphph = sp.sin(th) ** 2 * (r ** 2 + a ** 2 + a ** 2 * sp.sin(th) ** 2 * A / Sig)
3230     g = sp.Matrix([[gtt, 0, 0, gtph],
3231                    [0, Sig / Del, 0, 0],
3232                    [0, 0, Sig, 0],
3233                    [gtph, 0, 0, gphph]])
3234     return MetricModel(
3235         id="kerr_newman", title="M06 Kerr-Newman",
3236         coords=(t, r, th, ph), params=(M, a, Q), metric=g,
3237         compact=(CompactDirection(3, "phi", 2 * pi)),
3238         kind=ModelKind.LOCAL_METRIC, theory="Einstein-Maxwell",
3239         matter=MatterSpec("electrovacuum"),
3240         topology="maximal analytic extension; phi ~ phi + 2 pi",
3241         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3242         domain=(dom("nondegenerate Sigma", Sig, True),
3243                 dom("polar_regular", sp.sin(th), True)),
3244         default_point={"t": 0.0, "r": -0.5, "theta": float(sp.pi / 2), "phi": 0.0,
3245                       "M": 1.0, "a": 0.8, "Q": -0.2},
3246         domain_notes="Tests multi-parameter continuation and the separation of "
3247                     "charge, rotation and causal pathology. Horizons exist when "
3248                     "M^2 >= a^2 + Q^2.",
3249         provenance=Provenance(("R26", "R28")),
3250         metadata={"role": "multiparameter_benchmark"})
3251
3252 # =====
3253 # M07/M08/M26 Godel-type hyperbolic family
3254 # =====
3255
3256 def godel_type() -> MetricModel:
3257     t, r, ph, z = sp.symbols("t r phi z", real=True)
3258     om, m = sp.symbols("omega m", positive=True)
3259     H = 4 * om / m ** 2 * sp.sinh(m * r / 2) ** 2
3260     D = sp.sinh(m * r) / m
3261     g = sp.Matrix([[-1, 0, -H, 0],
3262                    [0, 1, 0, 0],
3263                    [-H, 0, D ** 2 - H ** 2, 0],
3264                    [0, 0, 0, 1]])
3265     return MetricModel(
3266         id="godel_type", title="M07 Godel-type hyperbolic family (Rebouças-Tiomno)",
3267         coords=(t, r, ph, z), params=(om, m), metric=g,
3268         compact=(CompactDirection(2, "phi", 2 * pi)),
3269         kind=ModelKind.LOCAL_METRIC, theory="GR or declared extension",
3270         matter=MatterSpec("source depends on the selected theory/matter model"),
3271         topology="R^3 x S^1 in the benchmark chart; phi ~ phi + 2 pi",
3272         expected=CTCStatus.UNRESOLVED, expected_scope=ClaimScope.SECTOR,
3273         expected_at_default=CTCStatus.CTC_PROVED,
3274         domain=(dom("radial", r, True), dom("rotation", om, True),
3275                 dom("scale", m, True)),
3276         default_point={"t": 0.0, "r": 2.0, "phi": 0.0, "z": 0.0,
3277                       "omega": 1.0, "m": 1.0},
3278         domain_notes="ds^2 = -(dt + H dphi)^2 + D^2 dphi^2 + dr^2 + dz^2 with "
3279                     "H = (4 omega/m^2) sinh^2(m r/2), D = sinh(m r)/m. "
3280                     "Circular CTCs exist for m^2 < 4 omega^2; the family is "
3281                     "causal for m^2 >= 4 omega^2. Primary analytic phase-boundary "
3282                     "benchmark.",
3283         provenance=Provenance(("R2", "R6"),
3284                               "Rebouças-Tiomno homogeneity classes"),
3285         metadata={"role": "phase_boundary_benchmark",
3286                  "normalised_sector_margin": "1 - (4 omega^2/m^2) tanh^2(m r/2)",
3287                  "exact_boundary": "m^2 = 4 omega^2",
3288                  "critical_radius": "sinh^2(m r_c/2) = [4 omega^2/m^2 - 1]^-1"})
3289
3290 def godel() -> MetricModel:
3291     base = godel_type()
3292     om, m = base.symbol("omega"), base.symbol("m")
3293     g = base.metric.subs(m, sp.sqrt(2) * om)
3294     return MetricModel(
3295         id="godel", title="M08 Godel solution (m^2 = 2 omega^2)",
3296         coords=base.coords, params=(om,), metric=sp.simplify(g),
3297         compact=base.compact, kind=ModelKind.LOCAL_METRIC, theory="GR",
3298         matter=MatterSpec("dust + cosmological constant (original solution)",
3299                           topology=base.topology,
3300                           expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3301                           domain=(dom("radial", base.symbol("r"), True), dom("rotation", om, True)),
3302                           default_point={"t": 0.0, "r": 2.0, "phi": 0.0, "z": 0.0, "omega": 1.0},
3303                           domain_notes="Canonical exact CTC control. Finite critical radius "
3304                                       "sinh^2(sqrt(2) omega r_c / 2) = 1, i.e. "
3305                                       "r_c = 2 arcsinh(1)/(sqrt(2) omega).",
3306                           provenance=Provenance(("R2", "R6"),
3307                                                  "special member of the hyperbolic Godel-type family"),
3308                           metadata={"role": "exact_ctc_control",
3309                                    "critical_radius_closed_form": "2*asinh(1)/(sqrt(2)*omega)"}))
3310
3311 def godel_critical() -> MetricModel:
3312     base = godel_type()
3313     om, m, r = base.symbol("omega"), base.symbol("m"), base.symbol("r")
3314     g = sp.simplify(base.metric.subs(m, 2 * om))
3315     return MetricModel(
3316         id="godel_critical", title="M26 Critical Godel-type member (m^2 = 4 omega^2)",
3317         coords=base.coords, params=(om,), metric=g,
3318         compact=base.compact, kind=ModelKind.LOCAL_METRIC, theory="GR or extension",
3319         matter=MatterSpec("model-dependent",
3320                           topology=base.topology,
3321                           expected=CTCStatus.UNRESOLVED, expected_scope=ClaimScope.SECTOR,
3322                           domain=(dom("radial", r, True), dom("rotation", om, True)),
3323                           default_point={"t": 0.0, "r": 2.0, "phi": 0.0, "z": 0.0, "omega": 1.0},
3324                           domain_notes="Exactly on the causal phase boundary: the critical radius "
3325                                       "diverges and g_phi_phi = sinh^2(2 omega r)/(4 omega^2) "

```



```

3329         "** (1 - tanh^2) > 0 for all finite r. The azimuthal sector is "
3330         "spacelike everywhere, but this is a SECTOR result only.",
3331         provenance=Provenance(("R2", "R6")),
3332         metadata={"role": "marginal_family_control"})
3333
3334 # =====
3335 # M09 van Stockum (D-02 symmetry corrected)
3336 # =====
3337
3338 def van_stockum() -> MetricModel:
3339     t, r, ph, z = sp.symbols("t r phi z", real=True)
3340     a = sp.symbols("a", positive=True)
3341     emu = sp.exp(-a ** 2 * r ** 2)
3342     g = sp.Matrix([[ -1, 0, a * r ** 2, 0],
3343                    [0, emu, 0, 0],
3344                    [a * r ** 2, 0, r ** 2 * (1 - a ** 2 * r ** 2), 0],
3345                    [0, 0, 0, emu]])
3346
3347     return MetricModel(
3348         id="van_stockum", title="M09 van Stockum rotating dust interior",
3349         coords=(t, r, ph, z), params=(a,), metric=g,
3350         compact=(CompactDirection(2, "phi", 2 * pi)),
3351         kind=ModelKind.LOCAL_METRIC, theory="GR",
3352         matter=MatterSpec("rigidly rotating dust"),
3353         topology="cylindrical, z noncompact, phi ~ phi + 2 pi",
3354         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3355         domain=(dom("radial", r, True), dom("rotation", a, True)),
3356         default_point={"t": 0.0, "r": 2.0, "phi": 0.0, "z": 0.0, "a": 1.0},
3357         domain_notes="Azimuthal loops become timelike when r^2(1 - a^2 r^2) < 0, "
3358                     "i.e. a r > 1. Physical matching to an exterior at some "
3359                     "radius R is a separate constraint: the CTC region is present "
3360                     "only if a R > 1. v1.0 stored this metric asymmetrically "
3361                     "(defect D-02).",
3362         provenance=Provenance(("R7",), "Bonnor form of the van Stockum interior"),
3363         metadata={"role": "matter_supported_ctc", "exact_boundary": "a r = 1"})
3364
3365 # =====
3366 # M10 Tipler cylinder (external exact specification)
3367 # =====
3368
3369 def tipler_cylinder() -> MetricModel:
3370     return MetricModel(
3371         id="tipler_cylinder", title="M10 Tipler rotating cylinder construction",
3372         kind=ModelKind.EXTERNAL_EXACT_SPEC, theory="GR",
3373         topology="infinite rotating cylinder; exterior matched to a rotating interior",
3374         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.REDUCED_MODEL,
3375         domain_notes="Tipler showed causality violation for the idealised, "
3376                     "infinitely long, rapidly rotating cylinder. The exterior "
3377                     "depends on matching parameters; CTC-FA v2.0 refuses to "
3378                     "fabricate a single canonical local 4x4 tensor for it and "
3379                     "uses van Stockum for the explicit local interior benchmark.",
3380         provenance=Provenance(("R7",), external_spec="Tipler 1974 matched "
3381                               "van Stockum interior/exterior",
3382                               implementation_status="external"),
3383         metadata={"role": "overreach_control"})
3384
3385 # =====
3386 # M11 Lorentzian Taub-NUT
3387 # =====
3388
3389 def taub_nut() -> MetricModel:
3390     t, r, th, ph = sp.symbols("t r theta phi", real=True)
3391     M, ell = sp.symbols("M ell", positive=True)
3392     f = (r ** 2 - 2 * M * r - ell ** 2) / (r ** 2 + ell ** 2)
3393     A = 2 * ell * sp.cos(th)
3394     g = sp.Matrix([[ -f, 0, 0, -f * A],
3395                    [0, 1 / f, 0, 0],
3396                    [0, 0, r ** 2 + ell ** 2, 0],
3397                    [ -f * A, 0, 0, -f * A ** 2 + (r ** 2 + ell ** 2) * sp.sin(th) ** 2]])
3398
3399     return MetricModel(
3400         id="taub_nut", title="M11 Lorentzian Taub-NUT",
3401         coords=(t, r, th, ph), params=(M, ell), metric=g,
3402         compact=(CompactDirection(3, "phi", 2 * pi)),
3403         kind=ModelKind.LOCAL_METRIC, theory="GR",
3404         matter=MatterSpec("vacuum", vacuum=True),
3405         topology="phi periodic; the global time identification is "
3406                 "treatment-dependent (Misner string / Misner identification)",
3407         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.SECTOR,
3408         domain=(dom("polar_regular", sp.sin(th), True)),
3409         default_point={"t": 0.0, "r": 3.0, "theta": 0.1, "phi": 0.0,
3410                       "M": 1.0, "ell": 1.0},
3411         domain_notes="Azimuthal CTC sectors occur where "
3412                     "-f (2 l cos theta)^2 + (r^2 + l^2) sin^2 theta < 0, i.e. near "
3413                     "the axis in the region f > 0. The local phi-loop detector does "
3414                     "NOT resolve the global identification choices; the verdict is "
3415                     "deliberately kept at SECTOR scope.",
3416         provenance=Provenance(("R26", "R28")),
3417         metadata={"role": "gravitomagnetic_pathology",
3418                 "sector_boundary": "g_phi_phi = 0"})
3419
3420 # =====
3421 # M12 Misner space (quotient chart) + M31 covering-space representation
3422 # =====
3423
3424 def misner() -> MetricModel:
3425     T, psi, y, z = sp.symbols("T psi y z", real=True)
3426     g = sp.Matrix([[0, -1, 0, 0],
3427                    [-1, -T, 0, 0],
3428                    [0, 0, 1, 0],
3429                    [0, 0, 0, 1]])
3430
3431     return MetricModel(
3432         id="misner", title="M12 Misner space (quotient chart)",
3433         coords=(T, psi, y, z), metric=g,
3434         compact=(CompactDirection(1, "psi", 2 * pi)),
3435         kind=ModelKind.QUOTIENT_GLOBAL, theory="GR",
3436         matter=MatterSpec("flat vacuum locally", T=sp.zeros(4, 4), vacuum=True),
3437         topology="flat quotient; psi ~ psi + 2 pi; chronology horizon at T = 0",

```

```

3439     expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3440     quotient=Quotient(
3441         generators=(DeckTransformation(
3442             "shift_psi", (T, psi + 2 * pi, y, z), (T, psi - 2 * pi, y, z),
3443             time_orientation_preserving=True,
3444             note="coordinate form of the boost identification"),),
3445         label="Z generated by psi -> psi + 2 pi", acts_freely=True),
3446     default_point={"T": 1.0, "psi": 0.0, "y": 0.0, "z": 0.0},
3447     reference_timelike=(sp.S.One, sp.S.One, sp.S.Zero, sp.S.Zero),
3448     domain_notes="det g = -1 everywhere, so the chart is regular across T = 0. "
3449     "g_psi_psi = -T: compact psi loops are spacelike for T < 0, "
3450     "null at T = 0 (the chronology horizon) and timelike for "
3451     "T > 0. Cleanest benchmark separating local flatness from "
3452     "global causal pathology.",
3453     provenance=Provenance(("R9", "R11", "R17")),
3454     metadata={"role": "chronology_horizon_benchmark",
3455               "chronology_horizon": "T = 0", "flat_covering": True})
3456
3457
3458 def misner_covering() -> MetricModel:
3459     """M31: Misner space as an explicit boost quotient of Minkowski.
3460
3461     Independent representation of the same spacetime, used as a cross-check
3462     of the deck-orbit detector against the Misner-chart compact-orbit
3463     detector.
3464     """
3465     t, x, y, z = sp.symbols("t x y z", real=True)
3466     b = sp.symbols("b", positive=True)
3467     g = sp.diag(-1, 1, 1, 1)
3468     boost = DeckTransformation(
3469         "boost",
3470         (t * sp.cosh(b) + x * sp.sinh(b), t * sp.sinh(b) + x * sp.cosh(b), y, z),
3471         (t * sp.cosh(b) - x * sp.sinh(b), -t * sp.sinh(b) + x * sp.cosh(b), y, z),
3472         time_orientation_preserving=True,
3473         note="orthochronous boost of rapidity b in the (t,x) plane")
3474     return MetricModel(
3475         id="misner_covering", title="M31 Misner space as a boost quotient of Minkowski",
3476         coords=(t, x, y, z), params=(b,), metric=g,
3477         kind=ModelKind.QUOTIENT_GLOBAL, theory="GR",
3478         matter=MatterSpec("vacuum", T=sp.zeros(4, 4), vacuum=True),
3479         topology="Minkowski / <boost>; the quotient is Misner space",
3480         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3481         quotient=Quotient((boost,), label="Z generated by a boost", acts_freely=False,
3482             note="the boost fixes the origin, so the quotient is taken "
3483             "on the complement of the fixed axis"),
3484         domain=(_dom("off_axis", x**2 + t**2, True)),
3485         default_point={"t": 0.0, "x": 1.0, "y": 0.0, "z": 0.0, "b": 1.0},
3486         domain_notes="For the boost of rapidity b the invariant interval of the "
3487             "closed geodesic through p and boost^n(p) is exactly "
3488             "2 (cosh(n b) - 1) (t^2 - x^2). It is negative -- a CTC -- "
3489             "precisely where x^2 > t^2, which is the image of the T > 0 "
3490             "region of the Misner chart. This reproduces M12 from a "
3491             "completely different representation.",
3492         provenance=Provenance(("R9", "R11")),
3493         note="Misner space as Minkowski modulo a boost"),
3494     metadata={"role": "cross_representation_check", "flat_covering": True,
3495               "exact_orbit_interval": "2*(cosh(n*b)-1)*(t^2-x^2)"})
3496
3497
3498 # =====
3499 # M27 Grant space
3500 # =====
3501
3502 def grant_space() -> MetricModel:
3503     t, x, y, z = sp.symbols("t x y z", real=True)
3504     b, alpha = sp.symbols("b alpha", positive=True)
3505     g = sp.diag(-1, 1, 1, 1)
3506     gen = DeckTransformation(
3507         "boost_translate",
3508         (t * sp.cosh(b) + x * sp.sinh(b), t * sp.sinh(b) + x * sp.cosh(b),
3509          y + alpha, z),
3510         (t * sp.cosh(b) - x * sp.sinh(b), -t * sp.sinh(b) + x * sp.cosh(b),
3511          y - alpha, z),
3512         time_orientation_preserving=True,
3513         note="boost of rapidity b in (t,x) composed with a translation alpha in y")
3514     return MetricModel(
3515         id="grant_space", title="M27 Grant space (boost + translation quotient)",
3516         coords=(t, x, y, z), params=(b, alpha), metric=g,
3517         kind=ModelKind.QUOTIENT_GLOBAL, theory="GR",
3518         matter=MatterSpec("vacuum", T=sp.zeros(4, 4), vacuum=True),
3519         topology="Minkowski / <boost x translation>",
3520         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3521         quotient=Quotient((gen,), label="Z generated by a screw isometry",
3522             acts_freely=True,
3523             note="the added translation removes the fixed axis, so the "
3524             "action is free and the quotient is a manifold"),
3525         default_point={"t": 0.0, "x": 3.0, "y": 0.0, "z": 0.0, "b": 1.0, "alpha": 1.0},
3526         domain_notes="The invariant interval of the closed geodesic through p and "
3527             "g^n(p) is exactly 2 (cosh(n b) - 1)(t^2 - x^2) + n^2 alpha^2. "
3528             "The winding-n loop is null on the POLARISED HYPERSURFACE "
3529             "x^2 - t^2 = n^2 alpha^2 / (2 (cosh(n b) - 1)) and timelike "
3530             "outside it. Because that radius decreases monotonically to "
3531             "zero as n grows, the union over all n is the whole region "
3532             "x^2 > t^2: alpha > 0 does NOT move the chronology horizon, "
3533             "which stays at x^2 = t^2 exactly as in Misner space. What "
3534             "alpha creates is the discrete family of polarised "
3535             "hypersurfaces that accumulates on the horizon, and those are "
3536             "where the renormalised stress tensor of a quantum field is "
3537             "singular. CTC-FA v2.0 locates them automatically from the "
3538             "deck group; see quotient_ctc.polarized_hypersurfaces.",
3539         provenance=Provenance(("R9", "R11", "R17"),
3540             "Grant 1993; Rieger 2026 causal classification"),
3541         metadata={"role": "polarised_quotient_benchmark", "flat_covering": True,
3542               "chronology_horizon": "x^2 = t^2",
3543               "polarised_hypersurface_n":
3544                   "x^2 - t^2 = n^2*alpha^2/(2*(cosh(n*b)-1))",
3545               "exact_orbit_interval":
3546                   "2*(cosh(n*b)-1)*(t^2-x^2) + n^2*alpha^2"})
3547
3548

```

```

3549 # =====
3550 # M13 Spinning cosmic string
3551 # =====
3552
3553 def spinning_cosmic_string() -> MetricModel:
3554     t, r, ph, z = sp.symbols("t r phi z", real=True)
3555     J, al = sp.symbols("J alpha", positive=True)
3556     g = sp.Matrix([[ -1, 0, -4 * J, 0],
3557                    [0, 1, 0, 0],
3558                    [-4 * J, 0, al ** 2 * r ** 2 - 16 * J ** 2, 0],
3559                    [0, 0, 0, 1]])
3560     return MetricModel(
3561         id="spinning_cosmic_string", title="M13 Spinning cosmic string",
3562         coords=(t, r, ph, z), params=(J, al), metric=g,
3563         compact=(CompactDirection(2, "phi", 2 * pi)),
3564         kind=ModelKind.QUOTIENT_GLOBAL, theory="GR with a distributional source",
3565         matter=MatterSpec("distributional string source on the axis"),
3566         topology="conical quotient; phi ~ phi + 2 pi; deficit angle 2 pi (1 - alpha)",
3567         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3568         domain=( _dom("radial", r, True), _dom("cone_angle", al, True),
3569                 _dom("spin", J, True)),
3570         default_point={"t": 0.0, "r": 0.1, "phi": 0.0, "z": 0.0,
3571                       "J": 0.1, "alpha": 0.8},
3572         domain_notes="ds^2 = -(dt + 4J dphi)^2 + dr^2 + alpha^2 r^2 dphi^2 + dz^2. "
3573                     "g_phi_phi = alpha^2 r^2 - 16 J^2 < 0 gives azimuthal CTCs "
3574                     "inside r < 4J/alpha in the idealised model.",
3575         provenance=Provenance(("R8", "R26")),
3576         metadata={"role": "conical_quotient_benchmark",
3577                  "exact_boundary": "alpha^2 r^2 = 16 J^2"})
3578
3579 # =====
3580 # M14 Gott two-string construction
3581 # =====
3582
3583 def gott_two_strings() -> MetricModel:
3584     def margin(p: Mapping[str, float]) -> float:
3585         mu = float(p["mu"])
3586         v = float(p["v"])
3587         if not (0.0 < v < 1.0):
3588             raise ValueError("Gott predicate requires 0 < v < 1")
3589         arg = 4.0 * math.pi * mu
3590         if not (0.0 < arg < math.pi):
3591             raise ValueError("Gott predicate requires 0 < 4 pi mu < pi")
3592         gamma = 1.0 / math.sqrt(1.0 - v * v)
3593         return 1.0 / math.sin(arg) - gamma
3594
3595     return MetricModel(
3596         id="gott_two_strings", title="M14 Gott two-moving-cosmic-string construction",
3597         kind=ModelKind.COMPOSITE_GLOBAL, theory="GR with distributional sources",
3598         topology="global holonomy around two non-intersecting infinite string defects",
3599         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.REDUCED_MODEL,
3600         default_point={"mu": 0.01, "v": 0.999},
3601         global_margin=margin, global_margin_scope=ClaimScope.REDUCED_MODEL,
3602         global_margin_note="CTCs exist when gamma > 1/sin(4 pi mu), equivalently "
3603                           "gamma v > cot(4 pi mu). Only the published global "
3604                           "holonomy criterion is evaluated; no local metric is "
3605                           "asserted.",
3606         domain_notes="Tests global holonomy criteria and prevents local-metric "
3607                     "overreach. mu here is G mu / c^2, so the deficit angle is "
3608                     "8 pi mu.",
3609         provenance=Provenance(("R8", "R9"),
3610                               external_spec="Gott 1991 exact two-string solution"),
3611         metadata={"role": "global_holonomy_benchmark",
3612                  "equivalent_forms": [{"gamma > 1/sin(4 pi mu)",
3613                                       "gamma v > cot(4 pi mu)"}]})
3614
3615 # =====
3616 # M15 Morris-Thorne
3617 # =====
3618
3619 def morris_thorne() -> MetricModel:
3620     t, r, th, ph = sp.symbols("t r theta phi", real=True)
3621     r0 = sp.symbols("r0", positive=True)
3622     b = r0 ** 2 / r
3623     g = sp.diag(-1, 1 / (1 - b / r), r ** 2, r ** 2 * sp.sin(th) ** 2)
3624     return MetricModel(
3625         id="morris_thorne", title="M15 Morris-Thorne zero-redshift wormhole",
3626         coords=(t, r, th, ph), params=(r0), metric=g,
3627         compact=(CompactDirection(3, "phi", 2 * pi)),
3628         kind=ModelKind.LOCAL_METRIC, theory="GR",
3629         matter=MatterSpec("exotic: NEC is violated near the throat under standard "
3630                           "GR assumptions"),
3631         topology="two asymptotic regions joined by a throat; phi ~ phi + 2 pi",
3632         expected=CTCStatus.NO_CTC_CERTIFIED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3633         temporal_function=t,
3634         domain=( _dom("outside_throat", r - r0, False,
3635                       "r >= r0; the r-chart is singular exactly at the throat"),
3636                 _dom("polar_regular", sp.sin(th), True)),
3637         default_point={"t": 0.0, "r": 2.0, "theta": float(sp.pi / 2), "phi": 0.0,
3638                       "r0": 1.0},
3639         domain_notes="Phi = 0 and b(r) = r0^2/r. A static traversable wormhole is "
3640                     "NOT automatically a time machine: g^tt = -1 so t is an exact "
3641                     "temporal function on this chart. Throat b(r0) = r0, "
3642                     "flare-out b'(r0) = -1 < 1.",
3643         provenance=Provenance(("R4", "R5", "R18")),
3644         metadata={"role": "topology_vs_chronology_control",
3645                  "throat": "r = r0", "flare_out": "b'(r0) = -1"})
3646
3647 # =====
3648 # M16 Time-shifted wormhole (reduced route-closure model)
3649 # =====
3650
3651 def time_shifted_wormhole() -> MetricModel:
3652     def margin(p: Mapping[str, float]) -> float:
3653         return float(p["t_ext"]) + float(p["t_wh"]) - abs(float(p["delta_t"]))
3654
3655     return MetricModel(

```

```

3659     id="time_shifted_wormhole", title="M16 Time-shifted traversable wormhole "
3660         "(reduced route-closure model)",
3661     kind=ModelKind.COMPOSITE_GLOBAL, theory="GR (assumed traversable wormhole)",
3662     topology="two mouths with a nontrivial spacetime identification",
3663     expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.REDUCED_MODEL,
3664     default_point={"delta_t": 12.0, "t_ext": 10.0, "t_wh": 1.0},
3665     global_margin=margin, global_margin_scope=ClaimScope.REDUCED_MODEL,
3666     global_margin_note="margin = t_ext + t_wh - |Delta t|. Negative margin "
3667         "means the declared route closes to the past. This is an "
3668         "ARCHITECTURE-SPECIFIC reduced predicate under declared "
3669         "synchronisation and traversal assumptions, NOT a "
3670         "universal theorem for arbitrary dynamical wormholes. A "
3671         "production model must compute causal travel times from "
3672         "the full metric and the mouth worldlines.",
3673     domain_notes="Explicitly labelled reduced criterion, used for software "
3674         "regression and parameter continuation. Requires: external "
3675         "travel time t_ext between mouths, wormhole traversal time "
3676         "t_wh, accumulated mouth time shift Delta t, all in one "
3677         "declared synchronisation.",
3678     provenance=Provenance(("R5", "R18", "R19"),
3679         external_spec="Morris-Thorne-Yurtsever time-machine "
3680             "argument"),
3681     metadata={"role": "reduced_predicate_benchmark"})
3682
3683 # =====
3684 # M17 Ori 2005 compact vacuum core
3685 # =====
3686
3687 def ori_2005() -> MetricModel:
3688     T, x, y, z = sp.symbols("T x y z", real=True)
3689     F = x**2 - y**2
3690     g = sp.Matrix([[0, 0, 0, -1],
3691         [0, 1, 0, 0],
3692         [0, 0, 1, 0],
3693         [-1, 0, 0, F - T]])
3694
3695     return MetricModel(
3696         id="ori_2005", title="M17 Ori 2005 compact vacuum-core chart",
3697         coords=(T, x, y, z), metric=g,
3698         compact=(CompactDirection(3, "z", sp.Symbol("L", positive=True))),
3699         params=(sp.Symbol("L", positive=True)),
3700         kind=ModelKind.QUOTIENT_GLOBAL, theory="GR",
3701         matter=MatterSpec("vacuum core; surrounding matter in the full Ori model "
3702             "satisfies WEC, DEC and SEC", T=sp.zeros(4, 4), vacuum=True),
3703         topology="z periodically identified with period L",
3704         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3705         default_point={"T": 1.0, "x": 0.0, "y": 0.0, "z": 0.0, "L": 1.0},
3706         reference_timelike=(sp.S.One, sp.S.Zero, sp.S.Zero, sp.S.One),
3707         domain_notes="ds^2 = dx^2 + dy^2 - 2 dt dz + [F(x,y) - T] dz^2 with the "
3708             "harmonic benchmark F = x^2 - y^2 (Laplacian zero, so the core "
3709             "is vacuum). At fixed (x,y) the compact z direction changes "
3710             "from spacelike to timelike when F - T crosses zero. This is "
3711             "the decisive counterexample to any universal rule "
3712             "'CTC implies NEC violation'.",
3713         provenance=Provenance(("R14")),
3714         metadata={"role": "energy_condition_counterexample",
3715             "exact_boundary": "T = F(x,y)"})
3716
3717 # =====
3718 # M18 Ori 2007 periodic pseudo-Schwarzschild core
3719 # =====
3720
3721 def ori_2007_core() -> MetricModel:
3722     v, r, th, ph = sp.symbols("v r theta phi", real=True)
3723     mu, P = sp.symbols("mu P", positive=True)
3724     f = 1 - 2 * mu / r
3725     g = sp.Matrix([[f, 1, 0, 0],
3726         [1, 0, 0, 0],
3727         [0, 0, r**2, 0],
3728         [0, 0, 0, r**2 * sp.sinh(th)**2]])
3729
3730     return MetricModel(
3731         id="ori_2007_core", title="M18 Ori 2007 periodic pseudo-Schwarzschild core",
3732         coords=(v, r, th, ph), params=(mu, P), metric=g,
3733         compact=(CompactDirection(0, "v", P)),
3734         kind=ModelKind.QUOTIENT_GLOBAL, theory="GR",
3735         matter=MatterSpec("vacuum core", T=sp.zeros(4, 4), vacuum=True),
3736         topology="v periodically identified in the internal core",
3737         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3738         domain=(dom("radial", r, True), dom("hyperbolic_regular", sp.sinh(th), True)),
3739         default_point={"v": 0.0, "r": 1.0, "theta": 1.0, "phi": 0.0, "mu": 1.0,
3740             "P": 1.0},
3741         reference_timelike=(sp.S.One, -sp.S.One, sp.S.Zero, sp.S.Zero),
3742         domain_notes="ds^2 = (1 - 2 mu/r) dv^2 + 2 dv dr + r^2 (dtheta^2 + "
3743             "sinh^2(theta) dphi^2). The 2-surfaces are hyperbolic, not "
3744             "spherical. Periodic v loops become timelike for r < 2 mu; "
3745             "r = 2 mu is marginal. Benchmark for constructibility from "
3746             "regular initial data.",
3747         provenance=Provenance(("R15", "R17")),
3748         metadata={"role": "constructibility_benchmark", "exact_boundary": "r = 2 mu"})
3749
3750 def pseudo_reissner_nordstrom() -> MetricModel:
3751     """M32: charged Rieger-type pseudo-RN core, the natural two-parameter
3752     extension of `ori_2007_core`. Gives a second analytic causal boundary
3753     with a closed form, used as an independent continuation test."""
3754     v, r, th, ph = sp.symbols("v r theta phi", real=True)
3755     mu, q, P = sp.symbols("mu q P", positive=True)
3756     f = 1 - 2 * mu / r + q**2 / r**2
3757     g = sp.Matrix([[f, 1, 0, 0],
3758         [1, 0, 0, 0],
3759         [0, 0, r**2, 0],
3760         [0, 0, 0, r**2 * sp.sinh(th)**2]])
3761
3762     return MetricModel(
3763         id="pseudo_reissner_nordstrom",
3764         title="M32 Periodic pseudo-Reissner-Nordstrom core (Rieger-type)",
3765         coords=(v, r, th, ph), params=(mu, q, P), metric=g,
3766         compact=(CompactDirection(0, "v", P)),
3767         kind=ModelKind.QUOTIENT_GLOBAL, theory="Einstein-Maxwell (pseudo-spherical)",
3768         matter=MatterSpec("electrovacuum core"),

```

```

3769     topology="v periodically identified",
3770     expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3771     domain=(dom("radial", r, True), dom("subextremal", mu**2 - q**2, True),
3772           dom("hyperbolic_regular", sp.sinh(th), True)),
3773     default_point={"v": 0.0, "r": 1.0, "theta": 1.0, "phi": 0.0,
3774                  "mu": 1.0, "q": 0.5, "p": 1.0},
3775     reference_timelike=(sp.S.One, -sp.S.One, sp.S.Zero, sp.S.Zero),
3776     domain_notes="Periodic v loops are timelike exactly between the roots "
3777                  "r_pm = mu +/- sqrt(mu^2 - q^2) of f(r). Two-parameter analytic "
3778                  "boundary for the continuation engine, structurally parallel "
3779                  "to Reissner-Nordstrom but with an opposite causal meaning: "
3780                  "here the roots bound a genuine chronology-violating region.",
3781     provenance=Provenance(("R15", "R17"),
3782                           "charged extension of the Ori-2007 pseudo-Schwarzschild "
3783                           "core in the Rieger classification programme"),
3784     metadata={"role": "two_parameter_boundary",
3785              "exact_boundary": "r = mu +/- sqrt(mu^2 - q^2)})
3786
3787
3788 def ori_2007_full() -> MetricModel:
3789     return MetricModel(
3790         id="ori_2007_full", title="M19 Ori 2007 full composite model",
3791         kind=ModelKind.EXTERNAL_EXACT_SPEC, theory="GR",
3792         topology="compact vacuum core + positive-density dust envelope + "
3793                "asymptotically flat vacuum exterior",
3794         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.REDUCED_MODEL,
3795         domain_notes="Closed timelike curves form after evolution from a regular "
3796                    "spacelike initial hypersurface in the published model, with "
3797                    "WEC, DEC and SEC satisfied. Highest-value target for a full "
3798                    "ADM/evolution reproduction; the matching and initial data are "
3799                    "a separate implementation.",
3800         provenance=Provenance(("R15",)),
3801         external_spec="Ori 2007 composite vacuum/dust "
3802                      "asymptotically flat model",
3803         implementation_status="external"),
3804     metadata={"role": "constructibility_target"})
3805
3806
3807 # =====
3808 # M20 Alcubierre
3809 # =====
3810
3811 def alcubierre() -> MetricModel:
3812     t, x, y, z = sp.symbols("t x y z", real=True)
3813     v, R, sig = sp.symbols("v R sigma", positive=True)
3814     xs = v * t
3815     rs = sp.sqrt((x - xs)**2 + y**2 + z**2)
3816     f = (sp.tanh(sig * (rs + R)) - sp.tanh(sig * (rs - R))) / (2 * sp.tanh(sig * R))
3817     g = sp.Matrix([[-1 + v**2 * f**2, -v * f, 0, 0],
3818                   [-v * f, 1, 0, 0],
3819                   [0, 0, 1, 0],
3820                   [0, 0, 0, 1]])
3821     return MetricModel(
3822         id="alcubierre", title="M20 Alcubierre warp metric",
3823         coords=(t, x, y, z), params=(v, R, sig), metric=g,
3824         kind=ModelKind.LOCAL_METRIC, theory="GR",
3825         matter=MatterSpec("violates the classical energy conditions"),
3826         topology="R^4 single-bubble chart, no identifications",
3827         expected=CTCStatus.NO_CTC_CERTIFIED, expected_scope=ClaimScope.DECLARED_DOMAIN,
3828         temporal_function=t,
3829         default_point={"t": 0.0, "x": 1.0, "y": 0.0, "z": 0.0,
3830                      "v": 2.0, "R": 1.0, "sigma": 8.0},
3831         domain_notes="ds^2 = -dt^2 + [dx - v f(rs) dt]^2 + dy^2 + dz^2. A single "
3832                    "bubble is not automatically a CTC: t is an exact temporal "
3833                    "function on this chart (g^tt = -1), so causality violation "
3834                    "requires an additional global or multi-bubble arrangement. "
3835                    "Prevents equating effective superluminality with a time "
3836                    "machine.",
3837         provenance=Provenance(("R12",)),
3838         metadata={"role": "superluminality_control"})
3839
3840
3841 # =====
3842 # M21 Krasnikov tube
3843 # =====
3844
3845 def krasnikov() -> MetricModel:
3846     t, x, rho, ph = sp.symbols("t x rho phi", real=True)
3847     delta, eps, rho_max, D = sp.symbols("delta epsilon rho_max D", positive=True)
3848
3849     def step(q):
3850         return (1 + sp.tanh(q / eps)) / 2
3851
3852     k = 1 - (2 - delta) * step(rho_max - rho) * step(t - x - rho) * step(x + eps - D)
3853     g = sp.Matrix([[-1, 1 - k, 0, 0],
3854                   [1 - k, k * (2 - k), 0, 0],
3855                   [0, 0, 1, 0],
3856                   [0, 0, 0, rho**2]])
3857     return MetricModel(
3858         id="krasnikov", title="M21 Krasnikov tube (Everett-Roman form)",
3859         coords=(t, x, rho, ph), params=(delta, eps, rho_max, D), metric=g,
3860         compact=(CompactDirection(3, "phi", 2 * pi)),
3861         kind=ModelKind.LOCAL_METRIC, theory="GR",
3862         matter=MatterSpec("requires exotic stress-energy in standard GR"),
3863         topology="phi periodic only; a CTC requires a suitable multi-tube "
3864                "or global arrangement",
3865         expected=CTCStatus.UNRESOLVED, expected_scope=ClaimScope.SECTOR,
3866         domain=(dom("radial", rho, True)),
3867         default_point={"t": 20.0, "x": 5.0, "rho": 0.1, "phi": 0.0,
3868                      "delta": 0.1, "epsilon": 0.1, "rho_max": 1.0, "D": 10.0},
3869         domain_notes="ds^2 = -dt^2 + 2(1-k) dt dx + k(2-k) dx^2 + drho^2 + "
3870                    "rho^2 dphi^2. The (t,x) block has determinant exactly -1 for "
3871                    "every k, so the chart is Lorentzian everywhere. A single tube "
3872                    "is not itself a time machine. The smooth step is a REGULATOR "
3873                    "CHOICE, not part of an exact solution, so no causal boundary "
3874                    "may be derived from this entry (defect D-06).",
3875         provenance=Provenance(("R13",)),
3876         external_spec="Everett-Roman smooth-step implementation of "
3877                      "Krasnikov's tube",
3878         metadata={"role": "shortcut_vs_loop_control", "regulator_dependent": True})

```

```

3879
3880
3881 # =====
3882 # M22 Paired warp / superluminal causal-loop family
3883 # =====
3884
3885 def paired_warp() -> MetricModel:
3886     return MetricModel(
3887         id="paired_warp", title="M22 Paired warp / superluminal causal-loop family",
3888         kind=ModelKind.COMPOSITE_GLOBAL, theory="construction-specific",
3889         topology="multi-segment engineered spacetime with global route closure",
3890         expected=CTCStatus.UNRESOLVED, expected_scope=ClaimScope.SECTOR,
3891         domain_notes="Deliberate UNRESOLVED placeholder that prohibits unsupported "
3892             "universality claims. A CTC verdict requires a specific "
3893             "published construction to be instantiated with its route "
3894             "geometry, velocities, synchronisation and matching data, and "
3895             "then validated.",
3896         provenance=Provenance(("R12", "R13"), implementation_status="placeholder"),
3897         metadata={"role": "unresolved_placeholder"})
3898
3899
3900 # =====
3901 # M23/M24 Anti-de Sitter pair -- the executable L3 gate
3902 # =====
3903
3904 def _ads4_metric():
3905     t, r, th, ph = sp.symbols("t r theta phi", real=True)
3906     L = sp.symbols("L", positive=True)
3907     f = 1 + r ** 2 / L ** 2
3908     g = sp.diag(-f, 1 / f, r ** 2, r ** 2 * sp.sin(th) ** 2)
3909     return (t, r, th, ph), L, g
3910
3911
3912 def ads4_universal_cover() -> MetricModel:
3913     (t, r, th, ph), L, g = _ads4_metric()
3914     return MetricModel(
3915         id="ads4_universal_cover", title="M23 AdS4, universal cover (t in R)",
3916         coords=(t, r, th, ph), params=(L,), metric=g,
3917         compact=(CompactDirection(3, "phi", 2 * pi)),
3918         kind=ModelKind.LOCAL_METRIC, theory="GR with Lambda = -3/L^2",
3919         matter=MatterSpec("vacuum with a negative cosmological constant",
3920             vacuum=True, cosmological_constant=-3 / L ** 2),
3921         topology="R (time) x R^3; simply connected in the time direction",
3922         expected=CTCStatus.NO_CTC_CERTIFIED, expected_scope=ClaimScope.GLOBAL,
3923         temporal_function=t,
3924         domain=(_dom("radial", r, False), _dom("polar_regular", sp.sin(th), True)),
3925         default_point={"t": 0.0, "r": 1.0, "theta": float(sp.pi / 2), "phi": 0.0,
3926             "L": 1.0},
3927         domain_notes="g^tt = -1/(1 + r^2/L^2) < 0 everywhere, so t is an exact "
3928             "global temporal function and the spacetime is stably causal. "
3929             "This is the CHRONAL member of the AdS pair.",
3930         provenance=Provenance(("R26", "R27"),
3931             "global AdS4 chart; the universal cover unwraps the "
3932             "periodic time of the embedded hyperboloid"),
3933         metadata={"role": "L3_gate_pair", "pair": "ads4_periodic"})
3934
3935
3936 def ads4_periodic() -> MetricModel:
3937     (t, r, th, ph), L, g = _ads4_metric()
3938     return MetricModel(
3939         id="ads4_periodic", title="M24 AdS4 hyperboloid, periodic time (t ~ t + 2 pi L)",
3940         coords=(t, r, th, ph), params=(L,), metric=g,
3941         compact=(CompactDirection(0, "t", 2 * sp.pi * L),
3942             CompactDirection(3, "phi", 2 * pi)),
3943         kind=ModelKind.QUOTIENT_GLOBAL, theory="GR with Lambda = -3/L^2",
3944         matter=MatterSpec("vacuum with a negative cosmological constant",
3945             vacuum=True, cosmological_constant=-3 / L ** 2),
3946         topology="S^1 (time) x R^3; the embedded quadric in R^{2,3}",
3947         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.GLOBAL,
3948         quotient=Quotient(
3949             generators=(DeckTransformation(
3950                 "time_shift", (t + 2 * sp.pi * L, r, th, ph),
3951                 (t - 2 * sp.pi * L, r, th, ph),
3952                 time_orientation_preserving=True,
3953                 note="the closed timelike Killing orbit of the AdS quadric"),),
3954             label="Z generated by t -> t + 2 pi L, acts_freely=True),
3955         domain=(_dom("radial", r, False), _dom("polar_regular", sp.sin(th), True)),
3956         default_point={"t": 0.0, "r": 1.0, "theta": float(sp.pi / 2), "phi": 0.0,
3957             "L": 1.0},
3958         domain_notes="IDENTICAL local metric to ads4_universal_cover. The orbit of "
3959             "the Killing vector d/dt is closed and timelike everywhere "
3960             "(g_tt = -(1 + r^2/L^2) < 0), so every point lies on a CTC. "
3961             "The two models differ ONLY in the deck group, which is exactly "
3962             "the L3 scientific gate: local equality of metrics does not "
3963             "imply equality of global causal structure.",
3964         provenance=Provenance(("R26", "R27"),
3965             "AdS as a quadric in R^{2,3} has closed timelike "
3966             "curves; the universal cover removes them"),
3967         metadata={"role": "L3_gate_pair", "pair": "ads4_universal_cover"})
3968
3969
3970 def minkowski_time_periodic() -> MetricModel:
3971     """M30: the minimal possible CTC spacetime, used as a positive control for
3972     the topology engine and as a unit test that the temporal-function
3973     certificate is refused when tau is not single valued on the quotient."""
3974     t, x, y, z = sp.symbols("t x y z", real=True)
3975     P = sp.symbols("P", positive=True)
3976     g = sp.diag(-1, 1, 1, 1)
3977     return MetricModel(
3978         id="minkowski_time_periodic",
3979         title="M30 Minkowski with periodic time (t ~ t + P)",
3980         coords=(t, x, y, z), params=(P,), metric=g,
3981         compact=(CompactDirection(0, "t", P)),
3982         kind=ModelKind.QUOTIENT_GLOBAL, theory="GR",
3983         matter=MatterSpec("vacuum", T=sp.zeros(4, 4), vacuum=True),
3984         topology="S^1 x R^3",
3985         expected=CTCStatus.CTC_PROVED, expected_scope=ClaimScope.GLOBAL,
3986         quotient=Quotient((DeckTransformation(
3987             "time_shift", (t + P, x, y, z), (t - P, x, y, z),
3988             time_orientation_preserving=True),)),

```



```

3989     label="Z generated by t -> t + P", acts_freely=True),
3990     default_point={"t": 0.0, "x": 0.0, "y": 0.0, "z": 0.0, "P": 1.0},
3991     domain_notes="Local metric identical to Minkowski, which is the strongest "
3992     "no-CTC control in the registry. The only difference is the "
3993     "deck group. Any architecture that classifies these two the "
3994     "same has failed the L3 gate.",
3995     provenance=Provenance(("R26", "R27")),
3996     metadata={"role": "L3_gate_pair", "pair": "minkowski", "flat_covering": True})
3997
3998
3999 # =====
4000 # M25 de Sitter static patch
4001 # =====
4002
4003 def de_sitter_static() -> MetricModel:
4004     t, r, th, ph = sp.symbols("t r theta phi", real=True)
4005     L = sp.symbols("L", positive=True)
4006     f = 1 - r ** 2 / L ** 2
4007     g = sp.diag(-f, 1 / f, r ** 2, r ** 2 * sp.sin(th) ** 2)
4008     return MetricModel(
4009         id="de_sitter_static", title="M25 de Sitter, static patch",
4010         coords=(t, r, th, ph), params=(L,), metric=g,
4011         compact=(CompactDirection(3, "phi", 2 * pi),),
4012         kind=ModelKind.LOCAL_METRIC, theory="GR with Lambda = +3/L^2",
4013         matter=MatterSpec("vacuum with a positive cosmological constant",
4014             vacuum=True, cosmological_constant=3 / L ** 2),
4015         topology="static patch chart; phi ~ phi + 2 pi",
4016         expected=CTCStatus.NO_CTC_CERTIFIED, expected_scope=ClaimScope.DECLARED_DOMAIN,
4017         temporal_function=t,
4018         domain=(_dom("inside_horizon", L - r, True),
4019             _dom("radial", r, False),
4020             _dom("polar_regular", sp.sin(th), True)),
4021         default_point={"t": 0.0, "r": 0.5, "theta": float(sp.pi / 2), "phi": 0.0,
4022             "L": 1.0},
4023         domain_notes="Positive-Lambda control. r = L is a cosmological horizon, "
4024             "not a chronology horizon; the static t is a temporal function. "
4025             "only inside it, exactly as for Schwarzschild outside r = 2M.",
4026         provenance=Provenance(("R26", "R27")),
4027         metadata={"role": "positive_lambda_control", "horizon": "r = L"})
4028
4029
4030 # =====
4031 # M29 Rotating BTZ (2+1)
4032 # =====
4033
4034 def btz_rotating() -> MetricModel:
4035     t, r, ph = sp.symbols("t r phi", real=True)
4036     Mb, Jb, ell = sp.symbols("M BTZ J BTZ ell", positive=True)
4037     N2 = -Mb + r ** 2 / ell ** 2 + Jb ** 2 / (4 * r ** 2)
4038     Nphi = -Jb / (2 * r ** 2)
4039     gtt = -N2 + r ** 2 * Nphi ** 2
4040     gtph = r ** 2 * Nphi
4041     g = sp.Matrix([[gtt, 0, gtph],
4042         [0, 1 / N2, 0],
4043         [gtph, 0, r ** 2]])
4044     return MetricModel(
4045         id="btz_rotating", title="M29 Rotating BTZ black hole (2+1)",
4046         coords=(t, r, ph), params=(Mb, Jb, ell), metric=sp.simplify(g),
4047         signature=(1, 0, 2),
4048         compact=(CompactDirection(2, "phi", 2 * pi),),
4049         kind=ModelKind.QUOTIENT_GLOBAL, theory="GR in 2+1 with Lambda = -1/ell^2",
4050         matter=MatterSpec("vacuum with a negative cosmological constant",
4051             vacuum=True, cosmological_constant=-1 / ell ** 2),
4052         topology="quotient of AdS3 by a discrete boost; phi ~ phi + 2 pi",
4053         expected=CTCStatus.NO_CTC_CERTIFIED, expected_scope=ClaimScope.DECLARED_DOMAIN,
4054         domain=(_dom("radial", r, True),
4055             _dom("outside_outer_horizon",
4056                 N2, True),
4057             _dom("subextremal", Mb ** 2 * ell ** 2 - Jb ** 2, True)),
4058         default_point={"t": 0.0, "r": 3.0, "phi": 0.0,
4059             "M BTZ": 1.0, "J BTZ": 0.5, "ell": 1.0},
4060         domain_notes="On the standard chart g_phi_phi = r^2 > 0, so the azimuthal "
4061             "sector is spacelike everywhere. The chronology-violating "
4062             "region of the maximally extended BTZ geometry sits at "
4063             "r^2 < 0 and is EXCISED BY DEFINITION of the spacetime -- "
4064             "exactly as the Kerr CTC region is retained by definition. "
4065             "Registered as a control for the claim 'a quotient of AdS is "
4066             "automatically causality violating', which is false.",
4067         provenance=Provenance(("R26", "R27"),
4068             "Banados-Teitelboim-Zanelli black hole; standard "
4069             "exterior chart"),
4070         metadata={"role": "excision_control", "excised_region": "r^2 < 0"})
4071
4072
4073 # =====
4074 # Registry assembly
4075 # =====
4076
4077 _BUILDERS = (
4078     minkowski, flrw_flat, flrw_exponential, schwarzschild, reissner_nordstrom,
4079     kerr, kerr_extremal, kerr_newman, godel_type, godel, godel_critical,
4080     van_stockum, tipler_cylinder, taub_nut, misner, misner_covering, grant_space,
4081     spinning_cosmic_string, gott_two_strings, morris_thorne, time_shifted_wormhole,
4082     ori_2005, ori_2007_core, pseudo_reissner_nordstrom, ori_2007_full, alcubierre,
4083     krasnikov, paired_warp, ads4_universal_cover, ads4_periodic,
4084     minkowski_time_periodic, de_sitter_static, btz_rotating,
4085 )
4086
4087
4088 def all_models() -> dict[str, MetricModel]:
4089     out: dict[str, MetricModel] = {}
4090     for b in _BUILDERS:
4091         m = b()
4092         if m.id in out:
4093             raise ValueError(f"duplicate model id {m.id}")
4094         out[m.id] = m
4095     return out
4096
4097
4098 __all__ = ["all_models"] + [b.__name__ for b in _BUILDERS]

```

src/ctcfa/registry.py — 122 lines

```

4099 """
4100 CTC-FA v2.0 -- Registry access, integrity and machine-readable export.
4101
4102 The registry is the single source of truth. Loading it runs the V0 structural
4103 gate on every entry (symmetry, shape, declared symbols, generator/coordinate
4104 consistency), so a malformed model cannot enter the system at all.
4105 """
4106
4107 from __future__ import annotations
4108
4109 import json
4110 from functools import lru_cache
4111 from pathlib import Path
4112 from typing import Any, Iterable, Iterator
4113
4114 from .metrics import all_models
4115 from .model import MetricModel
4116 from .status import CTCStatus, ClaimScope, ModelKind
4117
4118 @lru_cache(maxsize=1)
4119 def REGISTRY() -> dict[str, MetricModel]:
4120     return all_models()
4121
4122
4123 def get(name: str) -> MetricModel:
4124     reg = REGISTRY()
4125     if name not in reg:
4126         raise KeyError(f"unknown model {name!r}. Registered: {sorted(reg)}")
4127     return reg[name]
4128
4129
4130 def ids() -> tuple[str, ...]:
4131     return tuple(sorted(REGISTRY()))
4132
4133
4134 def models() -> Iterator[MetricModel]:
4135     for k in ids():
4136         yield REGISTRY()[k]
4137
4138
4139 def with_metric() -> tuple[str, ...]:
4140     return tuple(m.id for m in models() if m.has_metric)
4141
4142
4143 def numeric_models() -> tuple[str, ...]:
4144     """Models that can be evaluated numerically at their default point."""
4145     out = []
4146     for m in models():
4147         if not m.has_metric:
4148             continue
4149         if m.metadata.get("numeric_evaluable") is False:
4150             continue
4151         if not m.default_point:
4152             continue
4153         out.append(m.id)
4154     return tuple(out)
4155
4156
4157 def by_role(role: str) -> tuple[str, ...]:
4158     return tuple(m.id for m in models() if m.metadata.get("role") == role)
4159
4160
4161 def by_kind(kind: ModelKind) -> tuple[str, ...]:
4162     return tuple(m.id for m in models() if m.kind is kind)
4163
4164
4165 def summary() -> list[dict[str, Any]]:
4166     rows = []
4167     for m in models():
4168         rows.append({
4169             "model_id": m.id, "title": m.title, "kind": m.kind.value,
4170             "dim": m.dim, "has_metric": m.has_metric,
4171             "n_params": len(m.params), "n_compact": len(m.compact),
4172             "n_deck": len(m.quotient.generators),
4173             "expected": m.expected.value, "scope": m.expected_scope.value,
4174             "role": m.metadata.get("role", ""),
4175             "refs": " ".join(m.provenance.source_refs),
4176         })
4177     return rows
4178
4179
4180 def export_records(directory: str | Path) -> list[Path]:
4181     """Write one YAML-ish JSON record per model (schemas/metric.schema.json)."""
4182     directory = Path(directory)
4183     directory.mkdir(parents=True, exist_ok=True)
4184     out = []
4185     for m in models():
4186         p = directory / f"{m.id}.json"
4187         p.write_text(json.dumps(m.as_record(), indent=2, sort_keys=True,
4188                                ensure_ascii=True) + "\n", encoding="utf-8")
4189     out.append(p)
4190     return out
4191
4192
4193 def registry_audit() -> dict[str, Any]:
4194     """REQ-015 / REQ-016 audit: provenance completeness and kind discipline."""
4195     problems: list[str] = []
4196     for m in models():
4197         if not m.provenance.source_refs:
4198             problems.append(f"{m.id}: no source references")
4199         if m.kind in (ModelKind.EXTERNAL_EXACT_SPEC, ModelKind.COMPOSITE_GLOBAL) \
4200             and m.has_metric:
4201             problems.append(f"{m.id}: {m.kind.value} must not carry a local metric")
4202         if m.kind is ModelKind.LOCAL_METRIC and not m.has_metric:

```



```

4204         problems.append(f"{m.id}: LOCAL_METRIC without a metric")
4205     if m.has_metric and not m.domain_notes:
4206         problems.append(f"{m.id}: empty domain notes")
4207     if m.global_margin is not None and \
4208         m.global_margin_scope is ClaimScope.GLOBAL:
4209         problems.append(f"{m.id}: reduced predicate declared at GLOBAL scope")
4210     if m.expected is CTCStatus.NO CTC_CERTIFIED and \
4211         m.expected_scope is ClaimScope.GLOBAL and \
4212         m.temporal_function is None:
4213         problems.append(f"{m.id}: GLOBAL no-CTC expectation without a "
4214             f"candidate temporal function")
4215     return {"n_models": len(ids()), "n_problems": len(problems),
4216         "problems": problems, "pass": not problems}
4217
4218
4219 __all__ = ["REGISTRY", "get", "ids", "models", "with_metric", "numeric_models",
4220     "by_role", "by_kind", "summary", "export_records", "registry_audit"]

```

src/ctcfa/causality.py — 439 lines

```

4221 """
4222 CTC-FA v2.0 -- L5 (compact-orbit CTC detector) and L7 (causality certificate).
4223
4224 L5 -- positive route
4225 -----
4226 For declared compact Killing generators ``xi_A`` with periods ``P_A`` the
4227 Gram matrix is ``K_AB = g(xi_A, xi_B)`` and the closed orbit with integer
4228 winding ``n^A`` has invariant interval ``q = n^A K_AB n^B``. ``q < 0`` is an
4229 explicit symmetry-generated closed timelike curve.
4230
4231 Compared to v1.0 the detector now *earns* that conclusion instead of
4232 assuming it. Before a witness is accepted it verifies, in order:
4233
4234 G1 the model has a local metric (else the claim is external);
4235 G2 the point lies inside the declared physical domain;
4236 G3 the metric is nondegenerate Lorentzian there (Sylvester counts);
4237 G4 every declared compact generator is a Killing vector of the metric;
4238 G5 the orbit really closes under the declared identification / deck group;
4239 G6 the orbit is time-orientation consistent;
4240 G7 the sign of ``q`` survives its uncertainty budget, with automatic
4241 escalation to 50-digit arithmetic when it does not.
4242
4243 Only then is an ``OrbitStatus.SECTOR_CTC`` emitted, and even then it is
4244 lifted to a *sector-scoped* verdict. Failure of any gate downgrades the
4245 result to ``SECTOR_UNRESOLVED``; it never produces a negative claim.
4246
4247 L7 -- negative route
4248 -----
4249 A negative result is constructive. A smooth function ``tau`` with
4250 ``g{ab} d_a tau d_b tau < 0`` everywhere on the declared domain is a
4251 temporal function; its existence certifies stable causality and therefore
4252 excludes CTCs on that domain. v2.0 proves the strict inequality either
4253 symbolically or by interval branch-and-bound on a compact box, and reports
4254 which, together with the box. Point sampling alone is never accepted.
4255 """
4256
4257 from __future__ import annotations
4258
4259 import math
4260 from dataclasses import dataclass, field
4261 from itertools import product
4262 from typing import Any, Iterable, Mapping, Sequence
4263
4264 import numpy as np
4265 import sympy as sp
4266
4267 from . import interval as ivmod
4268 from .model import MetricModel
4269 from .numerics import (ErrorBudget, SignClass, budget_for_matrix, classify_sign,
4270     escalate_if_marginal, signature_counts)
4271 from .orientation import orbit_orientation
4272 from .status import (ClaimScope, CTCStatus, Gate, GateRecord, GateResult,
4273     OrbitResult, OrbitStatus, Verdict)
4274 from .symmetry import (compact_gram_symbolic, generators_admissible,
4275     winding_norm_symbolic)
4276 from .topology import curve_closes
4277
4278 #
4279 # -----
4280 # L5 : compact-orbit detector
4281 # -----
4282 #
4283 def _gcd_tuple(w: Sequence[int]) -> int:
4284     g = 0
4285     for q in w:
4286         g = math.gcd(g, abs(int(q)))
4287     return g
4288
4289
4290 def evaluate_metric(model: MetricModel,
4291     values: Mapping[str | sp.Symbol, float]) -> np.ndarray:
4292     g = model.require_metric()
4293     sub = model.substitution(values)
4294     missing = model.missing_values(values)
4295     if missing:
4296         raise ValueError(f"{model.id}: no numeric value supplied for {missing}. "
4297             f"Every coordinate and parameter must be given.")
4298     return np.array(sp.Matrix(g).subs(sub).evalf(), dtype=float)
4299
4300
4301 def compact_orbit_test(model: MetricModel,
4302     values: Mapping[str | sp.Symbol, float],
4303     max_winding: int = 2,
4304     tol: float = 1e-10,
4305     check_orientation: bool = True,
4306     check_closure: bool = True) -> OrbitResult:
4307     """Full L5 detector with all seven admissibility gates."""
4308     point = {str(k): float(v) for k, v in values.items()}

```

```

4309
4310 # -- G1 -----
4311 if not model.has_metric or not model.compact:
4312     return OrbitResult(
4313         OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None, point,
4314         "no local metric and/or no declared compact generator; this model "
4315         "must be handled by a global/reduced predicate, not by the "
4316         "compact-orbit detector")
4317
4318 # -- G2 -----
4319 dom = model.domain_report(point)
4320 if not dom["ok"]:
4321     bad = [c["name"] for c in dom["constraints"] if not c["ok"]]
4322     return OrbitResult(
4323         OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None, point,
4324         f"test point violates declared domain constraints {bad}; "
4325         f"a causal claim outside the physical domain is rejected")
4326
4327 # -- G3 -----
4328 try:
4329     gnum = evaluate_metric(model, point)
4330 except Exception as exc:
4331     # noqa: BLE001
4332     return OrbitResult(OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None,
4333         point, f"metric evaluation failed: {exc}")
4334 if not np.all(np.isfinite(gnum)):
4335     return OrbitResult(OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None,
4336         point, "metric is not finite at the test point")
4337 sig = signature_counts(gnum, tol)
4338 if sig != tuple(model.signature):
4339     return OrbitResult(
4340         OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None, point,
4341         f"signature {sig} at the test point differs from the declared "
4342         f"{tuple(model.signature)} in the (-,+,+,) convention")
4343
4344 # -- G4 -----
4345 ok_gen, certs = generators_admissible(model)
4346 if not ok_gen:
4347     bad = [c.name for c in certs if not c.admissible]
4348     return OrbitResult(
4349         OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None, point,
4350         f"declared compact generators {bad} are not Killing vectors of this "
4351         f"metric; the symmetry-orbit argument does not apply")
4352
4353 # -- Gram matrix and winding scan -----
4354 idx = [c.index for c in model.compact]
4355 K = gnum[np.ix_(idx, idx)]
4356 budget = budget_for_matrix(gnum, scale=float(max_winding) ** 2)
4357 budget.substitution = tol
4358 best_val = float("inf")
4359 best_w: tuple[int, ...] | None = None
4360 for w in product(range(-max_winding, max_winding + 1), repeat=len(idx)):
4361     if all(q == 0 for q in w):
4362         continue
4363     if gcd_tuple(w) != 1:
4364         continue
4365     # only primitive windings: n and k*n are the same orbit
4366     if next(q for q in w if q != 0) < 0:
4367         continue
4368     # n and -n trace the same orbit with opposite orientation
4369     n = np.asarray(w, dtype=float)
4370     val = float(n @ K @ n) / float(n @ n)
4371     if val < best_val:
4372         best_val, best_w = val, tuple(int(q) for q in w)
4373
4374 if best_w is None:
4375     return OrbitResult(OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None,
4376         point, "empty winding lattice")
4377
4378 # -- G7 : uncertainty and escalation -----
4379 escalated = False
4380 try:
4381     q_expr = winding_norm_symbolic(model, best_w) / float(np.dot(best_w, best_w))
4382     sub = model.substitution(point)
4383     best_val, budget, escalated = escalate_if_marginal(
4384         q_expr, sub, best_val, budget)
4385 except Exception as exc:
4386     # noqa: BLE001
4387     budget.notes.append(f"escalation unavailable: {exc}")
4388
4389 cls = classify_sign(best_val, budget)
4390
4391 # -- G5 : closure -----
4392 closure_note = "not checked"
4393 if check_closure:
4394     p0 = [point[str(c)] for c in model.coords]
4395     p1 = list(p0)
4396     for w, cd in zip(best_w, model.compact):
4397         try:
4398             period = float(sp.N(cd.period.subs(model.substitution(point))))
4399         except Exception:
4400             # noqa: BLE001
4401             period = float(sp.N(cd.period))
4402         p1[cd.index] = p0[cd.index] + w * period
4403     cl = curve_closes(model, p0, p1, tol=1e-8,
4404         params=model.substitution(point))
4405     closure_note = f"{cl.mechanism} (residual {cl.residual:.2e})"
4406 if not cl.closed:
4407     return OrbitResult(
4408         OrbitStatus.SECTOR_UNRESOLVED, best_val, best_w, point,
4409         f"the winding orbit does not close under the registered "
4410         f"identification structure: {cl.note}",
4411         budget.total, tuple(c.name for c in model.compact))
4412
4413 # -- G6 : orientation -----
4414 orientation_ok: bool | None = None
4415 orientation_note = ""
4416 if check_orientation and cls is SignClass.NEGATIVE:
4417     xi = np.zeros(model.dim)
4418     for w, cd in zip(best_w, model.compact):
4419         try:
4420             period = float(sp.N(cd.period.subs(model.substitution(point))))
4421         except Exception:
4422             # noqa: BLE001
4423             period = float(sp.N(cd.period))
4424         xi[cd.index] += w * period

```

```

4419     orr = orbit_orientation(gnum, xi)
4420     orientation_ok = orr.consistent
4421     orientation_note = orr.note
4422     if not orientation_ok:
4423         return OrbitResult(
4424             OrbitStatus.SECTOR_UNRESOLVED, best_val, best_w, point,
4425             f"orbit is timelike but time orientation is not consistent: "
4426             f"{orientation_note}", budget.total,
4427             tuple(c.name for c in model.compact), orientation_ok)
4428
4429     if cls is SignClass.NEGATIVE:
4430         st = OrbitStatus.SECTOR_CTC
4431         reason = (f"closed symmetry orbit with winding {best_w} is timelike "
4432                 f"(q={best_val:+.6e}, u={budget.total:.2e}); closure by "
4433                 f"{closure_note}; orientation consistent")
4434     elif cls is SignClass.MARGINAL:
4435         st = OrbitStatus.SECTOR_NULL
4436         reason = (f"closed symmetry orbit with winding {best_w} is null within "
4437                 f"the uncertainty band (q={best_val:+.6e}, u={budget.total:.2e})"
4438                 + (" [50-digit escalation applied]" if escalated else ""))
4439     else:
4440         st = OrbitStatus.SECTOR_SPACELIKE
4441         reason = (f"every scanned winding up to |n|<={max_winding} is spacelike "
4442                 f"(min q={best_val:+.6e}); this clears the tested sector only "
4443                 f"and is NOT a global no-CTC result")
4444
4445     return OrbitResult(st, best_val, best_w, point, reason, budget.total,
4446                       tuple(c.name for c in model.compact), orientation_ok,
4447                       isometry_verified=all(c.is_killing for c in certs))
4448
4449
4450 def compact_orbit_verdict(model: MetricModel,
4451                           values: Mapping[str | sp.Symbol, float],
4452                           **kw: Any) -> Verdict:
4453     """Run the detector and lift the result to a scoped verdict."""
4454     r = compact_orbit_test(model, values, **kw)
4455     gates = GateRecord()
4456     gates.set(Gate.V0_PARSE, GateResult.PASS, "registry record well-formed")
4457     gates.set(Gate.V1_ALGEBRA, GateResult.PASS,
4458              "signature and inverse verified at the test point")
4459     if r.status is OrbitStatus.SECTOR_CTC:
4460         gates.set(Gate.V4_CTC_WITNESS, GateResult.PASS, r.reason)
4461     elif r.status is OrbitStatus.SECTOR_NULL:
4462         gates.set(Gate.V4_CTC_WITNESS, GateResult.INCONCLUSIVE, r.reason)
4463         gates.set(Gate.V5_BOUNDARY, GateResult.PASS,
4464                  "closed null orbit locates a marginal causal boundary")
4465     else:
4466         gates.set(Gate.V4_CTC_WITNESS, GateResult.NOT_ATTEMPTED, r.reason)
4467     v = r.to_verdict(model.id, gates)
4468     if r.status is OrbitStatus.SECTOR_CTC and model.kind.value in (
4469         "LOCAL_METRIC", "QUOTIENT_GLOBAL"):
4470         # An explicit witness inside the declared spacetime is an existence
4471         # proof for that spacetime (asymmetric evidence).
4472         v = v.promote(ClaimScope.DECLARED_DOMAIN,
4473                      "explicit verified witness in the declared spacetime")
4474     return v
4475
4476
4477 # -----
4478 # L7 : causality certificate
4479 # -----
4480
4481 @dataclass
4482 class TemporalCertificate:
4483     """Result of attempting a global temporal-function proof."""
4484
4485     model_id: str
4486     tau: str
4487     status: CTCStatus
4488     scope: ClaimScope
4489     method: str
4490     proved: bool
4491     refuted: bool
4492     expression: str
4493     box: dict[str, tuple[float, float]] = field(default_factory=dict)
4494     domain_is_compact: bool = True
4495     lower_bound: float | None = None
4496     counterexample: dict[str, float] | None = None
4497     note: str = ""
4498
4499     def as_dict(self) -> dict[str, Any]:
4500         return {"model_id": self.model_id, "tau": self.tau,
4501                 "status": self.status.value, "scope": self.scope.value,
4502                 "method": self.method, "proved": self.proved,
4503                 "refuted": self.refuted, "expression": self.expression,
4504                 "box": self.box, "domain_is_compact": self.domain_is_compact,
4505                 "lower_bound": self.lower_bound,
4506                 "counterexample": self.counterexample, "note": self.note}
4507
4508
4509 def deck_noninvariance(model: MetricModel, tau: sp.Expr) -> list[str]:
4510     """Return the identifications under which ``tau`` fails to be invariant."""
4511     bad: list[str] = []
4512     coords = list(model.coords)
4513     for gen in model.quotient.generators:
4514         img = sp.simplify(tau.subs(dict(zip(coords, gen.image))), simultaneous=True))
4515         if sp.simplify(img - tau) != 0:
4516             bad.append(f"deck generator {gen.name}: tau -> {img}")
4517     for cd in model.compact:
4518         sub = {coords[cd.index]: coords[cd.index] + cd.period}
4519         img = sp.simplify(tau.subs(sub, simultaneous=True))
4520         if sp.simplify(img - tau) != 0:
4521             bad.append(f"period in {cd.name}: tau -> {img}")
4522     return bad
4523
4524
4525 def temporal_gradient_norm(model: MetricModel, tau: sp.Expr) -> sp.Expr:
4526     """C_tau = - g^{ab} d_a tau d_b tau. Positive means timelike gradient."""
4527     g = model.require_metric()
4528     gi = g.inv()

```

```

4529     coords = model.coords
4530     n = len(coords)
4531     grad = [sp.diff(tau, c) for c in coords]
4532     expr = sum(gi[a, b] * grad[a] * grad[b] for a in range(n) for b in range(n))
4533     return sp.simplify(-expr)
4534
4535
4536 def certify_temporal_function(model: MetricModel,
4537                               tau: sp.Expr | None = None,
4538                               box: Mapping[sp.Symbol, tuple[float, float]] | None = None,
4539                               assumptions: Mapping[sp.Symbol, dict] | None = None,
4540                               domain_is_compact: bool | None = None,
4541                               max_boxes: int = 20000) -> TemporalCertificate:
4542     """Prove or refute that ``tau`` is a temporal function on the domain.
4543
4544     A ``CERTIFIED`` outcome upgrades the model to ``NO_CTC_CERTIFIED`` with
4545     the scope implied by ``domain_is_compact``: ``GLOBAL`` when the proof is
4546     symbolic and unconditional, ``DECLARED_DOMAIN`` when it rests on a box.
4547     """
4548     tau = model.temporal_function if tau is None else tau
4549     if tau is None:
4550         return TemporalCertificate(
4551             model.id, "none", CTCStatus.UNRESOLVED, ClaimScope.SECTOR, "none",
4552             False, False, "", note="no candidate temporal function registered")
4553
4554     # -- Gate T0 : is tau even a FUNCTION on the quotient? -----
4555     #
4556     # A temporal function must be single valued on the actual spacetime. If
4557     # the model carries a deck group, tau must be invariant under it. This
4558     # gate is what makes ads4_periodic fail while ads4_universal_cover
4559     # succeeds on an identical local metric -- the whole point of the L3
4560     # gate. v1.0 had no such check, so its temporal-function route would
4561     # have "certified" a spacetime that is entirely chronology violating.
4562     bad = _deck_noninvariance(model, tau)
4563     if bad:
4564         return TemporalCertificate(
4565             model.id, str(tau), CTCStatus.UNRESOLVED, ClaimScope.SECTOR,
4566             "quotient invariance", False, True, "",
4567             note=(f"the candidate is NOT single valued on this spacetime: it "
4568                  f"fails to be invariant under the identification(s) {bad}. "
4569                  f"A multivalued 'time coordinate' on a quotient is not a "
4570                  f"temporal function, and no negative certificate follows."))
4571
4572     C = temporal_gradient_norm(model, tau)
4573     cert = ivmod.prove_positive(C, box, assumptions, strict=True,
4574                                max_boxes=max_boxes,
4575                                domain_is_compact=(True if domain_is_compact is None
4576                                                    else domain_is_compact))
4577
4578     if cert.proved:
4579         if cert.method == "symbolic" and (domain_is_compact is not False):
4580             status, scope = CTCStatus.NO_CTC_CERTIFIED, ClaimScope.GLOBAL
4581             note = ("unconditional symbolic proof of a strictly timelike "
4582                    "gradient: stable causality, hence no CTC")
4583         else:
4584             status, scope = CTCStatus.NO_CTC_CERTIFIED, ClaimScope.DECLARED_DOMAIN
4585             note = ("certified on the stated compact box only; extension to the "
4586                    "full domain requires an additional analytic argument")
4587     elif cert.refuted:
4588         status, scope = CTCStatus.UNRESOLVED, ClaimScope.SECTOR
4589         note = ("the candidate is NOT a temporal function; this refutes the "
4590                "candidate, it does not prove that CTCs exist")
4591     else:
4592         status, scope = CTCStatus.UNRESOLVED, ClaimScope.SECTOR
4593         note = ("neither proved nor refuted; UNRESOLVED is the only permitted "
4594                "output (failure of a search is not evidence of absence)")
4595     return TemporalCertificate(
4596         model.id, str(tau), status, scope, cert.method, cert.proved, cert.refuted,
4597         str(C), cert.box, cert.domain_is_compact, cert.lower_bound,
4598         cert.counterexample, note + (" | " + cert.symbolic_note
4599                                     if cert.symbolic_note else ""))
4600
4601 def temporal_verdict(model: MetricModel, **kw: Any) -> Verdict:
4602     c = certify_temporal_function(model, **kw)
4603     gates = GateRecord()
4604     gates.set(Gate.V0_PARSE, GateResult.PASS)
4605     gates.set(Gate.V1_ALGEBRA, GateResult.PASS)
4606     gates.set(Gate.V6_NEGATIVE_CERTIFICATE,
4607               GateResult.PASS if c.proved else
4608               (GateResult.FAIL if c.refuted else GateResult.INCONCLUSIVE), c.note)
4609     v = Verdict(status=c.status, scope=ClaimScope.SECTOR,
4610                 margin=float(c.lower_bound) if c.lower_bound is not None else float("nan"),
4611                 method=f"temporal_function:{c.method}", reason=c.note,
4612                 model_id=model.id, gates=gates, evidence=c.as_dict())
4613     if c.proved:
4614         v = v.promote(c.scope, "verified global temporal-function certificate")
4615     return v
4616
4617
4618 # -----
4619 # Global / reduced predicates
4620 # -----
4621
4622 def global_predicate_verdict(model: MetricModel,
4623                              parameters: Mapping[str, float],
4624                              tol: float = 1e-12) -> Verdict:
4625     """Evaluate a registered global/reduced route-closure predicate.
4626
4627     The result carries ``model.global_margin_scope``, which for every reduced
4628     model in the registry is ``REDUCED_MODEL``. It therefore cannot be
4629     printed as a global GR theorem -- the v1.0 category error (D-05).
4630     """
4631     gates = GateRecord()
4632     gates.set(Gate.V0_PARSE, GateResult.PASS)
4633     if model.global_margin is None:
4634         return Verdict(CTCStatus.UNRESOLVED, ClaimScope.SECTOR, float("nan"),
4635                        "global_predicate",
4636                        "no global or reduced predicate is registered for this model",
4637                        model.id, dict(parameters), gates)
4638     m = float(model.global_margin(parameters))

```

```

4639     if m < -tol:
4640         st, res = CTCStatus.CTC_PROVED, "route closes: arrival precedes departure"
4641         gates.set(Gate.V4_CTC_WITNESS, GateResult.PASS, res)
4642     elif abs(m) <= tol:
4643         st, res = CTCStatus.CRITICAL, "route closure is exactly marginal"
4644         gates.set(Gate.V5_BOUNDARY, GateResult.PASS, res)
4645     else:
4646         st, res = (CTCStatus.UNRESOLVED,
4647                 "route does not close under this reduced predicate; "
4648                 "this is not a no-CTC certificate")
4649         gates.set(Gate.V4_CTC_WITNESS, GateResult.NOT_ATTEMPTED, res)
4650     return Verdict(st, model.global_margin_scope, m, "global_predicate",
4651                 res + (" | " + model.global_margin_note
4652                     if model.global_margin_note else ""),
4653                 model.id, dict(parameters), gates)
4654
4655 __all__ = ["evaluate_metric", "compact_orbit_test", "compact_orbit_verdict",
4656           "TemporalCertificate", "temporal_gradient_norm",
4657           "certify_temporal_function", "temporal_verdict",
4658           "global_predicate_verdict"]

```

src/ctcfa/quotient_ctc.py — 351 lines

```

4660 """
4661 CTC-FA v2.0 -- L5b : deck-orbit CTC detector for flat quotients.
4662
4663 The compact-orbit detector of L5 finds closed timelike curves that are orbits
4664 of a *coordinate* Killing generator. That covers Godel, Kerr, van Stockum,
4665 the Ori cores and the AdS quadric, but it cannot see the family of
4666 chronology-violating spacetimes built as quotients of a flat covering space
4667 by an isometry that is not a coordinate translation -- Misner space by a
4668 boost, Grant space by a boost composed with a translation, and the polarised
4669 Misner-type spacetimes that dominate the modern causal-classification
4670 literature. v1.0 had no engine for them at all; its Misner entry used an
4671 ad-hoc chart and its Gott entry a scalar predicate.
4672
4673 Method (exact, not heuristic)
4674 -----
4675 Let `(M, eta)` be a **flat** covering spacetime and `Gamma` a group of
4676 isometries acting freely. For `p` in `M` and `g` in `Gamma`, the
4677 straight segment from `p` to `g(p)` is a geodesic of `M` and projects
4678 to a closed curve in `M/Gamma`. Its causal character is exactly the sign of
4679
4680     q(p, g) = eta( g(p) - p , g(p) - p ).
4681
4682 Because the metric is constant, this is an **exact** statement, not a
4683 sampled estimate: `q < 0` proves a closed timelike geodesic through the
4684 projection of `p`.
4685
4686 Two gates guard the method:
4687
4688     F1 flatness of the covering is *verified* symbolically (Riemann = 0),
4689     not assumed;
4690     F2 the deck element must be orthochronous, otherwise the projected loop
4691     reverses the future direction and is not a CTC witness.
4692
4693 The engine reproduces the known results exactly:
4694
4695     Misner covering : q = 2 (cosh(n b) - 1) (t^2 - x^2)
4696     Grant space    : q = 2 (cosh(n b) - 1) (t^2 - x^2) + n^2 alpha^2
4697
4698 and the first of these agrees, point by point, with the independent Misner
4699 *chart* computation in `metrics.misner` -- a genuine cross-representation
4700 check of the architecture rather than a self-consistency check of one chart.
4701 """
4702
4703 from __future__ import annotations
4704
4705 from dataclasses import dataclass, field
4706 from typing import Any, Mapping, Sequence
4707
4708 import numpy as np
4709 import sympy as sp
4710
4711 from .model import MetricModel
4712 from .numerics import (ErrorBudget, SignClass, classify_sign, signature_counts)
4713 from .orientation import timelike_eigenvector
4714 from .status import (ClaimScope, CTCStatus, Gate, GateRecord, GateResult,
4715                     OrbitResult, OrbitStatus, Verdict)
4716 from .symbolic import riemann
4717
4718 @dataclass
4719 class FlatnessCertificate:
4720     flat: bool
4721     residual: str
4722     note: str = ""
4723
4724     def as_dict(self) -> dict[str, Any]:
4725         return {"flat": self.flat, "residual": self.residual, "note": self.note}
4726
4727 def certify_flat(model: MetricModel) -> FlatnessCertificate:
4728     """Gate F1: verify Riemann = 0 for the covering metric."""
4729     g = model.require_metric()
4730     R = riemann(g, model.coords, simplify=True)
4731     n = model.dim
4732     bad = []
4733     for a in range(n):
4734         for b in range(n):
4735             for c in range(n):
4736                 for d in range(c + 1, n):
4737                     e = sp.simplify(R[a][b][c][d])
4738                     if e != 0:
4739                         bad.append(f"R^{a}_{b}{c}{d}} = {e}")
4740     if bad:
4741         return FlatnessCertificate(False, "; ".join(bad[:4]),

```

```

4744         "covering is NOT flat: the straight-segment "
4745         "geodesic argument does not apply")
4746     return FlatnessCertificate(True, "0",
4747         "Riemann tensor vanishes identically: straight "
4748         "coordinate segments are geodesics and the "
4749         "interval formula is exact")
4750
4751
4752 def _orthochronous(model: MetricModel, gen, subs: Mapping[sp.Symbol, float]) -> bool | None:
4753     """Gate F2: does the deck element preserve the time orientation?"""
4754     if gen.time_orientation_preserving is not None:
4755         return gen.time_orientation_preserving
4756     n = model.dim
4757     J = sp.Matrix(n, n, lambda a, b: sp.diff(gen.image[a], model.coords[b]))
4758     try:
4759         Jn = np.array(J.subs(subs).evalf(), dtype=float)
4760         gn = np.array(sp.Matrix(model.metric).subs(subs).evalf(), dtype=float)
4761     except Exception:
4762         return None
4763     T, lam = timelike_eigenvector(gn)
4764     if lam >= 0:
4765         return None
4766     ip = float((Jn @ T) @ gn @ T)
4767     return ip < 0 if abs(ip) > 1e-12 else None
4768
4769
4770 @dataclass
4771 class DeckOrbitResult:
4772     status: OrbitStatus
4773     margin: float
4774     word: tuple[str, ...] | None
4775     point: dict[str, float]
4776     reason: str
4777     uncertainty: float = 0.0
4778     flatness: FlatnessCertificate | None = None
4779     orthochronous: bool | None = None
4780     displacement: list[float] = field(default_factory=list)
4781
4782     def to_verdict(self, model_id: str) -> Verdict:
4783         gates = GateRecord()
4784         gates.set(Gate.V0_PARSE, GateResult.PASS)
4785         gates.set(Gate.V1_ALGEBRA, GateResult.PASS,
4786             "flat covering verified symbolically")
4787         if self.status is OrbitStatus.SECTOR_CTC:
4788             st, gr = CTCStatus.CTC_PROVED, GateResult.PASS
4789         elif self.status is OrbitStatus.SECTOR_NULL:
4790             st, gr = CTCStatus.CRITICAL, GateResult.INCONCLUSIVE
4791         else:
4792             st, gr = CTCStatus.UNRESOLVED, GateResult.NOT_ATTEMPTED
4793         gates.set(Gate.V4_CTC_WITNESS, gr, self.reason)
4794         if self.status is OrbitStatus.SECTOR_NULL:
4795             gates.set(Gate.V5_BOUNDARY, GateResult.PASS,
4796                 "closed null geodesic at the chronology boundary")
4797         v = Verdict(st, ClaimScope.SECTOR, self.margin, "deck_orbit", self.reason,
4798             model_id, self.point, gates, self.as_evidence(), self.uncertainty)
4799         if st is CTCStatus.CTC_PROVED:
4800             v = v.promote(ClaimScope.DECLARED_DOMAIN,
4801                 "explicit closed timelike geodesic of the quotient")
4802         return v
4803
4804     def as_evidence(self) -> dict[str, Any]:
4805         return {"word": list(self.word) if self.word else None,
4806             "displacement": self.displacement,
4807             "orthochronous": self.orthochronous,
4808             "flatness": self.flatness.as_dict() if self.flatness else None,
4809             "orbit_status": self.status.value}
4810
4811
4812 def deck_orbit_test(model: MetricModel,
4813     values: Mapping[str | sp.Symbol, float],
4814     max_word_length: int = 3,
4815     tol: float = 1e-10,
4816     verify_flatness: bool = True) -> DeckOrbitResult:
4817     """Exact CTC test for a flat quotient at the point `values`."""
4818     point = {str(k): float(v) for k, v in values.items()}
4819     if not model.has_metric:
4820         return DeckOrbitResult(OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None,
4821             point, "no local metric on the covering space")
4822     if not model.quotient.generators:
4823         return DeckOrbitResult(OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None,
4824             point, "no deck generators registered")
4825
4826     flat: FlatnessCertificate | None = None
4827     if verify_flatness:
4828         flat = certify_flat(model)
4829         if not flat.flat:
4830             return DeckOrbitResult(
4831                 OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None, point,
4832                 "the covering space is not flat, so the straight-segment "
4833                 "geodesic argument is unavailable; use the Killing-orbit "
4834                 "detector or a full geodesic search instead", 0.0, flat)
4835
4836     sub = model.substitution(point)
4837     try:
4838         gnum = np.array(sp.Matrix(model.metric).subs(sub).evalf(), dtype=float)
4839     except Exception as exc:
4840         return DeckOrbitResult(OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None,
4841             point, f"metric not numeric: {exc}", 0.0, flat)
4842     sig = signature_counts(gnum, tol)
4843     if sig != tuple(model.signature):
4844         return DeckOrbitResult(OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None,
4845             point, f"signature {sig} != declared "
4846                 f"{tuple(model.signature)}", 0.0, flat)
4847
4848     ortho = all(_orthochronous(model, gen, sub) is not False
4849         for gen in model.quotient.generators)
4850
4851     p0 = [point[str(c)] for c in model.coords]
4852     # Words are scanned in order of increasing length so that the reported
4853     # witness is the SHORTEST closed loop that violates chronology, not merely

```

```

4854 # the most negative one. A long word is a multiple circuit of a shorter
4855 # loop and carries no extra information.
4856 words = sorted((w for w in model.quotient.words(max_word_length) if w),
4857               key=len)
4858 best = (float("inf"), None, None)
4859 first_negative = None
4860 for word in words:
4861     try:
4862         p1 = model.quotient.apply_word(model.coords, p0, word, sub)
4863     except Exception:
4864         continue
4865     d = np.asarray([b - a for a, b in zip(p0, p1)], dtype=float)
4866     if np.max(np.abs(d)) < tol:
4867         continue
4868     q = float(d @ gnum @ d)
4869     if q < best[0]:
4870         best = (q, word, d)
4871     if first_negative is None and q < 0:
4872         first_negative = (q, word, d)
4873
4874 q, word, d = first_negative if first_negative is not None else best
4875 if word is None:
4876     return DeckOrbitResult(OrbitStatus.SECTOR_UNRESOLVED, float("nan"), None,
4877                           point, "every deck word fixes the test point", 0.0, flat)
4878
4879 budget = ErrorBudget()
4880 budget.roundoff = float(np.finfo(np.float64).eps) * max(1.0, abs(q)) * 16
4881 budget.substitution = tol
4882 cls = classify_sign(q, budget)
4883
4884 if cls is SignClass.NEGATIVE:
4885     if not ortho:
4886         return DeckOrbitResult(
4887             OrbitStatus.SECTOR_UNRESOLVED, q, word, point,
4888             "the closed geodesic is timelike but the deck element is not "
4889             "orthochronous, so the projected loop reverses the future "
4890             "direction and is not a future-directed CTC",
4891             budget.total, flat, ortho, list(map(float, d)))
4892     st = OrbitStatus.SECTOR_CTC
4893     reason = (f"closed timelike geodesic through the projection of the test "
4894              f"point: word {word}, interval^2 = {q:+.6e} "
4895              f"(exact on a flat covering)")
4896 elif cls is SignClass.MARGINAL:
4897     st = OrbitStatus.SECTOR_NULL
4898     reason = (f"closed null geodesic (word {word}, interval^2 = {q:+.6e} "
4899              f"within u={budget.total:.2e}): chronology boundary")
4900 else:
4901     st = OrbitStatus.SECTOR_SPACELIKE
4902     reason = (f"every deck word up to length {max_word_length} gives a "
4903              f"spacelike closed geodesic (min interval^2 = {q:+.6e}); "
4904              f"this clears the tested words only")
4905     return DeckOrbitResult(st, q, word, point, reason, budget.total, flat, ortho,
4906                           list(map(float, d)))
4907
4908
4909 def deck_orbit_verdict(model: MetricModel, values: Mapping[str, float],
4910                      **kw: Any) -> Verdict:
4911     return deck_orbit_test(model, values, **kw).to_verdict(model.id)
4912
4913
4914 def winding_spectrum(model: MetricModel, values: Mapping[str, float],
4915                    max_power: int = 8) -> list[dict[str, Any]]:
4916     """Interval^2 of the winding-n closed geodesic for n = 1..max_power.
4917
4918     The sign pattern of this spectrum *is* the causal structure of the
4919     quotient at that point: the smallest n with a negative entry is the
4920     shortest chronology-violating loop, and a zero entry marks a polarised
4921     hypersurface (a closed null geodesic of winding n).
4922     """
4923     sub = model.substitution(values)
4924     gnum = np.array(sp.Matrix(model.require_metric()).subs(sub).evalf(), dtype=float)
4925     coords = list(model.coords)
4926     p0 = [float(values[str(c)]) for c in coords]
4927     gen = model.quotient.generators[0]
4928     out: list[dict[str, Any]] = []
4929     for n in range(1, max_power + 1):
4930         word = tuple([gen.name] * n)
4931         p1 = model.quotient.apply_word(coords, p0, word, sub)
4932         d = np.asarray([b - a for a, b in zip(p0, p1)], dtype=float)
4933         q = float(d @ gnum @ d)
4934         out.append({"n": n, "interval_squared": q,
4935                   "character": ("timelike" if q < -1e-12 else
4936                                "null" if abs(q) <= 1e-12 else "spacelike"),
4937                   "displacement": list(map(float, d))})
4938     return out
4939
4940
4941 def polarized_hypersurfaces(model: MetricModel, base_point: Mapping[str, float],
4942                            variable: str, lo: float, hi: float,
4943                            max_power: int = 6) -> list[dict[str, Any]]:
4944     """Locate, for each winding n, the radius at which the loop becomes null.
4945
4946     For a flat quotient these are the *polarised hypersurfaces*: the loci on
4947     which the winding-n closed geodesic is null. In Misner-type spacetimes
4948     they are precisely where the renormalised stress tensor of a quantum
4949     field diverges, and their accumulation locus is the chronology horizon.
4950
4951     Locating them from the deck group is only possible because v2.0 stores
4952     the identification structure as an executable object; v1.0 had no path
4953     to this result at all.
4954     """
4955     from .numerics import bracketed_bisection, coarse_sign_scan
4956     gen = model.quotient.generators[0]
4957     coords = list(model.coords)
4958     out: list[dict[str, Any]] = []
4959     for n in range(1, max_power + 1):
4960         word = tuple([gen.name] * n)
4961
4962         def q_of(x: float, _w=word) -> float:
4963             pt = dict(base_point)

```



```

4964         pt[variable] = x
4965         sub = model.substitution(pt)
4966         gnum = np.array(sp.Matrix(model.metric).subs(sub).evalf(), dtype=float)
4967         p0 = [float(pt[str(c)]) for c in coords]
4968         p1 = model.quotient.apply_word(coords, p0, _w, sub)
4969         d = np.asarray([b - a for a, b in zip(p0, p1)], dtype=float)
4970         return float(d @ gnum @ d)
4971
4972     roots = []
4973     for a, b in coarse_sign_scan(q_of, lo, hi, n=200):
4974         if a == b:
4975             roots.append((a, (a, a), 0))
4976             continue
4977         try:
4978             roots.append(bracketed_bisection(q_of, a, b, xtol=1e-13))
4979         except ValueError:
4980             continue
4981     for root, bracket, iters in roots:
4982         out.append({"winding": n, "variable": variable, "value": root,
4983                    "bracket": list(bracket), "iterations": iters,
4984                    "interval_squared_at_root": q_of(root)})
4985     return out
4986
4987 def exact_orbit_interval(model: MetricModel, word_power: int = 1) -> sp.Expr:
4988     """Symbolic  $\eta(g^n p - p, g^n p - p)$  for a single-generator quotient.
4989
4990     Used to derive the closed-form chronology-violating region of Misner and
4991     Grant space instead of sampling it.
4992     """
4993     g = model.require_metric()
4994     gens = model.quotient.generators
4995     if len(gens) != 1:
4996         raise ValueError("exact_orbit_interval requires exactly one deck generator")
4997     coords = list(model.coords)
4998     cur = list(coords)
4999     for _ in range(abs(word_power)):
5000         image = gens[0].image if word_power > 0 else gens[0].inverse
5001         cur = [sp.simplify(e.subs(dict(zip(coords, cur))), simultaneous=True))
5002             for e in image]
5003     d = sp.Matrix([sp.simplify(c - x) for c, x in zip(cur, coords)])
5004     return sp.simplify((d.T * g * d)[0])
5005
5006 __all__ = ["FlatnessCertificate", "certify_flat", "DeckOrbitResult",
5007            "deck_orbit_test", "deck_orbit_verdict", "winding_spectrum",
5008            "polarized_hypersurfaces", "exact_orbit_interval"]
5009

```

src/ctcfa/continuation.py — 437 lines

```

5011 """
5012 CTC-FA v2.0 -- L16 : causal phase-boundary continuation.
5013
5014 The defect this module removes
5015 -----
5016 v1.0 shipped
5017
5018 def godel_family_margin(m, omega=1.0) -> float:
5019     return m**2 - 4*omega*omega
5020
5021 and then "recovered" the Godel-type causal boundary by running a root
5022 finder on it, reporting  $m = 1.999999999999987$  as a regression result.
5023 That is a root of a hard-coded literature answer, not a derivation from the
5024 geometry. The v1.0 risk register names the pattern exactly:
5025
5026 FM-016 "Known literature threshold encoded into detector and later
5027 claimed rediscovered" -> CIRCULAR VALIDATION.
5028
5029 v2.0 deletes the closed form from the solver path. The boundary is obtained
5030 from the metric tensor alone and the analytic value is used **only
5031 afterwards**, as an accuracy check that the solver never sees.
5032 NonCircularityError is raised if a caller tries to pass the analytic
5033 answer into the search.
5034
5035 The reduced causal margin
5036 -----
5037 Let  $\xi$  be a compact generator and  $u$  a timelike reference direction.
5038 Decompose  $\xi$  orthogonally with respect to  $u$ :
5039
5040  $\xi_{\perp} = \xi - [g(\xi, u) / g(u, u)] u$ ,
5041  $h(\xi) = g(\xi_{\perp}, \xi_{\perp}) = g(\xi, \xi) - g(\xi, u)^2 / g(u, u) > 0$ ,
5042
5043 which is strictly positive because the orthogonal complement of a timelike
5044 vector is spacelike. The **reduced causal margin** is the dimensionless
5045
5046  $\mu(x) = g(\xi, \xi) / h(\xi)$ .
5047
5048 It has the same sign as  $g(\xi, \xi)$  -- so it decides causality identically
5049 -- but it is normalised by the geometry itself, is invariant under a
5050 rescaling of the generator, and stays finite where the raw component
5051 diverges. For the standard families it collapses to exactly the quantity
5052 the literature uses:
5053
5054 Godel-type  $\mu = 1 - (4 \omega^2 / m^2) \tanh^2(m r / 2)$ 
5055 inf over  $r = 1 - 4 \omega^2 / m^2$  -> root  $m = 2 \omega$ 
5056 van Stockum  $\mu = 1 - a^2 r^2$  -> root  $a r = 1$ 
5057 spinning string  $\mu = 1 - 16 J^2 / (\alpha^2 r^2)$  -> root  $\alpha r = 4 J$ 
5058 Ori 2007 core  $\mu = 1 - 2 \mu_{\text{mass}} / r$  -> root  $r = 2 \mu_{\text{mass}}$ 
5059
5060 None of those closed forms is used by the solver; they are what the solver
5061 *produces*, and the test suite checks the agreement.
5062 """
5063
5064 from __future__ import annotations
5065
5066 import math
5067 from dataclasses import dataclass, field
5068 from typing import Any, Callable, Mapping, Sequence

```



```

5069
5070 import numpy as np
5071 import sympy as sp
5072
5073 from . import interval as ivmod
5074 from .model import MetricModel
5075 from .numerics import (ThresholdResult, bracketed_bisection, coarse_sign_scan)
5076
5077
5078 class NonCircularityError(RuntimeError):
5079     """Raised when a known analytic answer would enter the search path."""
5080
5081 # -----
5082 # Reduced causal margin
5083 # -----
5084
5085
5086 def reference_direction(model: MetricModel,
5087                        reference_index: int | None = None,
5088                        generator_index: int | None = None) -> sp.Matrix:
5089     """Choose a timelike reference direction ``u^a`` for the normalisation.
5090
5091     Order of preference: an explicit ``reference_index``; the model's
5092     declared ``reference_timelike`` field; the first coordinate direction
5093     other than the generator whose diagonal metric component is manifestly
5094     negative. Charts such as Misner (``g_TT = 0``) and the null-adapted Ori
5095     cores (``g_rr = 0``) have no timelike *coordinate* direction at all,
5096     which is why the registry declares one explicitly for them.
5097     """
5098     g = model.require_metric()
5099     n = model.dim
5100     if reference_index is not None:
5101         v = [sp.S.Zero] * n
5102         v[reference_index] = sp.S.One
5103         return sp.Matrix(v)
5104     if model.reference_timelike is not None:
5105         return sp.Matrix(list(model.reference_timelike))
5106     for k in range(n):
5107         if k == generator_index:
5108             continue
5109         if sp.simplify(g[k, k]).is_negative:
5110             v = [sp.S.Zero] * n
5111             v[k] = sp.S.One
5112             return sp.Matrix(v)
5113     # Fall back to the model's own default point: symbolic negativity is often
5114     # undecidable (Taub-NUT's g_tt = -f has no fixed sign over the whole
5115     # parameter space) even though a timelike direction plainly exists on the
5116     # domain of interest.
5117     if model.default_point:
5118         try:
5119             sub = model.substitution(model.default_point)
5120             for k in range(n):
5121                 if k == generator_index:
5122                     continue
5123                 if float(sp.N(g[k, k].subs(sub))) < 0:
5124                     v = [sp.S.Zero] * n
5125                     v[k] = sp.S.One
5126                     return sp.Matrix(v)
5127         except Exception:
5128             pass
5129         # noqa: BLE001
5130     raise ValueError(
5131         f"{model.id}: no timelike reference direction is available for the "
5132         f"reduced-margin normalisation (the compact generator itself may be "
5133         f"the only timelike direction, in which case the sector has no causal "
5134         f"phase boundary). Declare MetricModel.reference_timelike to override.")
5135
5136 def reduced_causal_margin_symbolic(model: MetricModel,
5137                                    generator_index: int | None = None,
5138                                    reference_index: int | None = None) -> sp.Expr:
5139     """Symbolic ``mu = g(xi,xi) / h(xi)`` for a coordinate generator."""
5140     g = model.require_metric()
5141     if generator_index is None:
5142         if not model.compact:
5143             raise ValueError(f"{model.id}: no compact generator declared")
5144         generator_index = model.compact[0].index
5145     i = generator_index
5146     u = reference_direction(model, reference_index, i)
5147     n = model.dim
5148     xi = sp.Matrix([sp.S.One if k == i else sp.S.Zero for k in range(n)])
5149     q = sp.simplify((xi.T * g * xi)[0])
5150     guu = sp.simplify((u.T * g * u)[0])
5151     giu = sp.simplify((xi.T * g * u)[0])
5152     if guu == 0:
5153         raise ValueError(f"{model.id}: the reference direction is null; the "
5154                         f"orthogonal decomposition is undefined")
5155     h = sp.simplify(q - giu ** 2 / guu)
5156     return sp.simplify(q / h)
5157
5158
5159
5160 def reference_validity(model: MetricModel,
5161                       generator_index: int | None = None,
5162                       reference_index: int | None = None) -> sp.Expr:
5163     """``g(u,u)``: strictly positive exactly where the normalisation is valid.
5164
5165     The reduced margin acquires a spurious zero wherever the reference
5166     direction itself becomes null, so every boundary the continuation engine
5167     reports is checked against this expression. Without the check the
5168     charged pseudo-RN core reports a root at ``r^2 + 2 mu r - q^2 = 0``, which
5169     is an artefact of the reference direction and not a causal boundary at
5170     all -- precisely failure mode FM-001 in a new disguise.
5171     """
5172     g = model.require_metric()
5173     if generator_index is None and model.compact:
5174         generator_index = model.compact[0].index
5175     u = reference_direction(model, reference_index, generator_index)
5176     return sp.simplify(-(u.T * g * u)[0])
5177
5178

```

```

5179 def reduced_causal_margin(model: MetricModel, values: Mapping[str, float],
5180                             generator_index: int | None = None,
5181                             reference_index: int | None = None) -> float:
5182     """Numeric reduced causal margin at a point.
5183
5184     If no reference index is given, the timelike coordinate direction is used
5185     when one exists at the point, and otherwise the timelike eigendirection
5186     of the metric -- so the quantity remains defined inside an ergoregion,
5187     where every coordinate direction can be spacelike.
5188     """
5189     g = np.array(sp.Matrix(model.require_metric())
5190                  .subs(model.substitution(values)).evalf(), dtype=float)
5191     if generator_index is None:
5192         generator_index = model.compact[0].index
5193     i = generator_index
5194     q = float(g[i, i])
5195     if reference_index is None:
5196         cand = [k for k in range(model.dim) if k != i and g[k, k] < 0]
5197         if cand:
5198             u = np.zeros(model.dim)
5199             u[cand[0]] = 1.0
5200         else:
5201             vals, vecs = np.linalg.eigh((g + g.T) / 2.0)
5202             u = vecs[:, int(np.argmax(vals))]
5203     else:
5204         u = np.zeros(model.dim)
5205         u[reference_index] = 1.0
5206     xi = np.zeros(model.dim)
5207     xi[i] = 1.0
5208     guu = float(u @ g @ u)
5209     giu = float(xi @ g @ u)
5210     if guu >= 0:
5211         raise ValueError("reference direction is not timelike at this point")
5212     h = q - giu * giu / guu
5213     if h <= 0:
5214         raise ValueError("degenerate projection: generator is parallel to the "
5215                          "reference direction")
5216     return q / h
5217
5218 # -----
5219 # Sector infimum over a coordinate box
5220 # -----
5221
5222 @dataclass
5223 class SectorInfimum:
5224     value: float
5225     certified: bool
5226     method: str
5227     box: dict[str, tuple[float, float]]
5228     argmin: dict[str, float] = field(default_factory=dict)
5229     note: str = ""
5230
5231     def as_dict(self) -> dict[str, Any]:
5232         return {"value": self.value, "certified": self.certified,
5233                "method": self.method, "box": self.box, "argmin": self.argmin,
5234                "note": self.note}
5235
5236
5237 def sector_infimum(model: MetricModel, fixed: Mapping[str, float],
5238                   box: Mapping[str, tuple[float, float]],
5239                   generator_index: int | None = None,
5240                   reference_index: int | None = None,
5241                   samples: int = 2001,
5242                   certify: bool = True) -> SectorInfimum:
5243     """Infimum of the reduced causal margin over a coordinate box.
5244
5245     A dense scan gives the value; interval branch-and-bound is then used to
5246     *certify* the sign of the infimum. Only the certified branch may feed a
5247     formal causal statement.
5248     """
5249     mu = reduced_causal_margin_symbolic(model, generator_index, reference_index)
5250     subs = {model.symbol(k): v for k, v in fixed.items()}
5251     expr = sp.simplify(mu.subs(subs))
5252     free = sorted(expr.free_symbols, key=str)
5253     boxd = {model.symbol(k): (float(v[0]), float(v[1])) for k, v in box.items()}
5254     boxd = {k: v for k, v in boxd.items() if k in free}
5255     if not boxd:
5256         val = float(sp.N(expr))
5257         return SectorInfimum(val, True, "constant", {}, {},
5258                             "margin is independent of the box variables")
5259
5260     grids = [np.linspace(v[0], v[1], samples) for v in boxd.values()]
5261     f = sp.lambdify(list(boxd.keys()), expr, "numpy")
5262     if len(grids) == 1:
5263         with np.errstate(all="ignore"):
5264             vals = np.asarray(f(grids[0]), dtype=float)
5265             good = np.isfinite(vals)
5266             if not good.any():
5267                 return SectorInfimum(float("nan"), False, "scan",
5268                                     {str(k): v for k, v in boxd.items()}, {},
5269                                     "margin is nowhere finite on the box")
5270             j = int(np.nanargmin(np.where(good, vals, np.inf)))
5271             best, arg = float(vals[j]), {str(list(boxd)[0]): float(grids[0][j])}
5272     else:
5273         mesh = np.meshgrid(*grids, indexing="ij")
5274         with np.errstate(all="ignore"):
5275             vals = np.asarray(f(*mesh), dtype=float)
5276             j = np.unravel_index(int(np.nanargmin(vals)), vals.shape)
5277             best = float(vals[j])
5278             arg = {str(k): float(grids[i][j[i]]) for i, k in enumerate(boxd)}
5279
5280     certified, method, note = False, "dense_scan", "scan only: not a certificate"
5281     if certify:
5282         cert = ivmod.prove_positive(expr, boxd, strict=True, max_boxes=6000)
5283         if cert.proved:
5284             certified, method = True, "interval_branch_and_bound"
5285             note = "certified strictly positive on the box"
5286         elif cert.refuted:
5287             certified, method = True, "interval_branch_and_bound"
5288

```

```

5289         note = "certified to become negative on the box"
5290     return SectorInfimum(best, certified, method,
5291                          {str(k): v for k, v in boxd.items()}, arg, note)
5292
5293
5294 # -----
5295 # Phase boundary in a parameter
5296 # -----
5297
5298 def causal_phase_boundary(model: MetricModel,
5299                          parameter: str,
5300                          bracket: tuple[float, float],
5301                          fixed: Mapping[str, float],
5302                          box: Mapping[str, tuple[float, float]],
5303                          generator_index: int | None = None,
5304                          reference_index: int | None = None,
5305                          analytic_value: float | None = None,
5306                          analytic_form: str = "",
5307                          xtol: float = 1e-13,
5308                          samples: int = 4001) -> ThresholdResult:
5309     """Find the parameter value at which the sector infimum changes sign.
5310
5311     The search evaluates only the metric. ``analytic_value`` is compared
5312     with the result afterwards and is *never* used to steer the search; a
5313     caller that tries to make the margin function itself depend on it gets a
5314     :class:`NonCircularityError`.
5315     """
5316     if analytic_value is not None and analytic_form and \
5317         parameter in analytic_form and "margin" in analytic_form.lower():
5318         raise NonCircularityError(
5319             "the analytic form appears to be wired into the margin function; "
5320             "the boundary must be derived from the metric alone")
5321
5322     # Lambdify the reduced margin ONCE. Re-simplifying inside the root
5323     # finder turns a millisecond scan into a minute-long symbolic session and
5324     # was the single biggest cost in the first v2.0 prototype.
5325     mu = reduced_causal_margin_symbolic(model, generator_index, reference_index)
5326     box_syms = [model.symbol(k) for k in box]
5327     par_sym = model.symbol(parameter)
5328     fixed_syms = [model.symbol(k) for k in fixed if k != parameter]
5329     free = mu.free_symbols
5330     unresolved = free - set(box_syms) - {par_sym} - set(fixed_syms)
5331     if unresolved:
5332         raise ValueError(f"{model.id}: reduced margin still depends on "
5333                          f"{sorted(map(str, unresolved))}; give them values in "
5334                          f"'fixed' or ranges in 'box'")
5335     f = sp.lambdify(box_syms + [par_sym] + fixed_syms, mu, "numpy")
5336     grids = [np.linspace(box[str(s)][0], box[str(s)][1], samples) for s in box_syms]
5337     fixed_vals = [float(fixed[str(s)]) for s in fixed_syms]
5338     if len(grids) > 1:
5339         mesh = np.meshgrid(*grids, indexing="ij")
5340     else:
5341         mesh = grids
5342
5343     def margin_of(p: float) -> float:
5344         with np.errstate(all="ignore"):
5345             if not box_syms:
5346                 v = float(f(float(p), *fixed_vals))
5347                 return v
5348             vals = np.asarray(f(*mesh, float(p), *fixed_vals), dtype=float)
5349             vals = np.where(np.isfinite(vals), vals, np.inf)
5350             return float(np.min(vals))
5351
5352     lo, hi = float(bracket[0]), float(bracket[1])
5353     brackets = coarse_sign_scan(margin_of, lo, hi, n=48)
5354     if not brackets:
5355         raise ValueError(f"{model.id}: no sign change of the sector infimum for "
5356                          f"{parameter} in [{lo}, {hi}]")
5357     a, b = brackets[0]
5358     root, final_bracket, iters = bracketed_bisection(margin_of, a, b, xtol=xtol)
5359
5360     eps = max(1e-9, 10 * xtol)
5361     left = margin_of(root - eps)
5362     right = margin_of(root + eps)
5363
5364     fx = dict(fixed)
5365     fx[parameter] = root
5366     dom = model.domain_report({**fx, **{k: 0.5 * (v[0] + v[1])
5367                                         for k, v in box.items()}}) \
5368         if model.domain else {"ok": True}
5369
5370     # Reject a root that is an artefact of the normalisation rather than a
5371     # causal transition (see reference_validity).
5372     ref_ok, ref_note = True, ""
5373     try:
5374         val = reference_validity(model, generator_index, reference_index)
5375         # (a) pointwise at the root: a root that sits where -g(u,u) vanishes is
5376         #      an artefact of the normalisation, not a causal transition.
5377         at_root = val.subs({model.symbol(k): v for k, v in fx.items()})
5378         rest = {model.symbol(k): 0.5 * (v[0] + v[1]) for k, v in box.items()}
5379         at_root = at_root.subs(rest)
5380         if not at_root.free_symbols:
5381             v0 = float(sp.N(at_root))
5382             if v0 <= 1e-9:
5383                 ref_ok = False
5384                 ref_note = (f" | REJECTED: the timelike reference direction is "
5385                             f"null at this root (-g(u,u) = {v0:.3e}), so the sign "
5386                             f"change is a normalisation artefact, not a causal "
5387                             f"boundary")
5388         # (b) over the whole box, when one was given.
5389         if ref_ok and box:
5390             cert = ivmod.prove_positive(
5391                 val.subs({model.symbol(k): v for k, v in fx.items()}),
5392                 {model.symbol(k): (float(v[0]), float(v[1]))
5393                  for k, v in box.items()}),
5394                 strict=True, max_boxes=4000)
5395             if cert.refuted:
5396                 ref_ok = False
5397                 ref_note = (f" | WARNING: the timelike reference direction becomes "
5398                             f"null somewhere on this box, so the reported root may ")

```

```

5399         "be a normalisation artefact")
5400     except Exception as exc:
5401         ref_note = f" | reference-validity check unavailable: {exc}"
5402
5403     return ThresholdResult(
5404         value=root, bracket=final_bracket, left_margin=left, right_margin=right,
5405         residual=abs(margin_of(root)), iterations=iters,
5406         method="reduced-causal-margin infimum + bracketed bisection",
5407         analytic_value=analytic_value, analytic_form=analytic_form,
5408         domain_ok=bool(dom.get("ok", True)) and ref_ok,
5409         note=("boundary derived from the metric tensor only; the analytic value "
5410             "is reported for comparison and was not used in the search")
5411         + ref_note)
5412
5413
5414 def convergence_study(model: MetricModel, parameter: str,
5415                      bracket: tuple[float, float], fixed: Mapping[str, float],
5416                      box_variable: str, radii: Sequence[float],
5417                      generator_index: int | None = None,
5418                      analytic_value: float | None = None) -> list[dict[str, Any]]:
5419     """Boundary value as the coordinate box is enlarged.
5420
5421     For families whose exact boundary is an asymptotic statement -- the
5422     Godel-type critical radius diverges as  $m \rightarrow 2\omega$  -- the boundary
5423     at finite box radius is only an approximation, and the rate at which it
5424     converges is itself evidence. Reporting this table instead of a single
5425     number is what distinguishes a derivation from a lucky root.
5426     """
5427     rows = []
5428     for Rmax in radii:
5429         box = {box_variable: (1e-6, float(Rmax))}
5430         try:
5431             th = causal_phase_boundary(model, parameter, bracket, fixed, box,
5432                                     generator_index,
5433                                     analytic_value=analytic_value)
5434         except ValueError as exc:
5435             rows.append({"box_radius": float(Rmax), "error": str(exc)})
5436             continue
5437         rows.append({"box_radius": float(Rmax), "boundary": th.value,
5438                     "bracket_width": th.bracket_width,
5439                     "analytic": analytic_value,
5440                     "error": (None if analytic_value is None
5441                             else abs(th.value - analytic_value))})
5442     return rows
5443
5444
5445 __all__ = ["NonCircularityError", "reduced_causal_margin_symbolic",
5446           "reduced_causal_margin", "SectorInfimum", "sector_infimum",
5447           "causal_phase_boundary", "convergence_study"]

```

src/ctcfa/loops.py — 360 lines

```

5448 """
5449 CTC-FA v2.0 -- L6 : general closed-curve search.
5450
5451 Scientific status
5452 -----
5453 This engine is one sided by construction. Finding a closed curve whose
5454 tangent is everywhere timelike is a positive witness; failing to find one is
5455 never evidence of absence. Section 6.2 of v1.0 said so, and REQ-013 required
5456 that non-detection stay ``UNRESOLVED``, but the v1.0 optimizer had three
5457 gaps that could turn a numerical artefact into a "witness":
5458
5459 D-10a closure was enforced only for the winding term; a general curve was
5460 periodic by construction but its time orientation was never
5461 checked and the returned object recorded
5462 ``"orientation_status": "NOT_CHECKED"``;
5463 D-10b the objective used a smooth maximum (log-sum-exp), which can be
5464 negative while the true maximum is positive, so the reported
5465 "max norm" was not the quantity the criterion needs;
5466 D-10c a candidate found on the optimizer's own sample grid was never
5467 re-validated on a denser grid or in an alternative parametrisation.
5468
5469 v2.0 keeps the Fourier ansatz but separates search from validation:
5470
5471 search -- any optimiser, any smoothing, any heuristic; produces
5472         candidates only;
5473 validate -- exact maximum of  $g(\gamma_{\dot{}})$  on a refined
5474            grid, closure test on the quotient, time-orientation test,
5475            non-degeneracy of the tangent, and a repeat at doubled
5476            resolution.
5477
5478 Only a candidate that survives validation is reported, and even then it is
5479 reported as ``CTC_PROVED`` at ``SECTOR`` scope pending an independent
5480 symbolic or interval confirmation.
5481 """
5482
5483 from __future__ import annotations
5484
5485 from dataclasses import dataclass, field
5486 from typing import Any, Callable, Mapping, Sequence
5487
5488 import numpy as np
5489 import sympy as sp
5490 from scipy.optimize import minimize
5491
5492 from .model import MetricModel
5493 from .orientation import check_curve_orientation
5494 from .status import (ClaimScope, CTCStatus, Gate, GateRecord, GateResult, Verdict)
5495 from .topology import curve_closes
5496
5497
5498 @dataclass
5499 class LoopCandidate:
5500     max_norm: float
5501     mean_norm: float
5502     min_speed: float
5503     points: np.ndarray

```

```

5504     tangents: np.ndarray
5505     coefficients: np.ndarray
5506     n_samples: int
5507     winding: tuple[int, ...] = ()
5508     note: str = ""
5509
5510     def as_dict(self) -> dict[str, Any]:
5511         return {"max_norm": self.max_norm, "mean_norm": self.mean_norm,
5512                "min_speed": self.min_speed, "n_samples": self.n_samples,
5513                "winding": list(self.winding), "note": self.note}
5514
5515
5516 @dataclass
5517 class LoopValidation:
5518     timelike_everywhere: bool
5519     max_norm: float
5520     max_norm_normalised: float = 0.0
5521     coordinate_extent: float = 0.0
5522     closed: bool = False
5523     closure_mechanism: str = ""
5524     closure_residual: float = float("nan")
5525     orientation_consistent: bool = False
5526     degenerate_tangent: bool = True
5527     resolution_stable: bool = False
5528     n_samples: int = 0
5529     bounded: bool = True
5530     note: str = ""
5531
5532     @property
5533     def accepted(self) -> bool:
5534         return (self.timelike_everywhere and self.closed
5535                and self.orientation_consistent and not self.degenerate_tangent
5536                and self.resolution_stable and self.bounded)
5537
5538     def as_dict(self) -> dict[str, Any]:
5539         return {"timelike_everywhere": self.timelike_everywhere,
5540                "max_norm": self.max_norm,
5541                "max_norm_normalised": self.max_norm_normalised,
5542                "coordinate_extent": self.coordinate_extent,
5543                "bounded": self.bounded, "closed": self.closed,
5544                "closure_mechanism": self.closure_mechanism,
5545                "closure_residual": self.closure_residual,
5546                "orientation_consistent": self.orientation_consistent,
5547                "degenerate_tangent": self.degenerate_tangent,
5548                "resolution_stable": self.resolution_stable,
5549                "n_samples": self.n_samples, "accepted": self.accepted,
5550                "note": self.note}
5551
5552
5553 class LoopSearch:
5554     """Fourier-series closed-curve search with an explicit validation stage."""
5555
5556     def __init__(self, model: MetricModel, params: Mapping[str, float],
5557                  modes: int = 3, samples: int = 256):
5558         self.model = model
5559         self.modes = modes
5560         self.samples = samples
5561         g = model.require_metric()
5562         self.coords = list(model.coords)
5563         self.n = len(self.coords)
5564         self.pvals = [float(params[str(s)]) for s in model.params]
5565         # Component-wise lambdification so the metric can be evaluated on a
5566         # whole sampled curve in one vectorised call. Evaluating a lambdified
5567         # Matrix point by point was the dominant cost of the v1.0 search.
5568         self._gc = [sp.lambdify(self.coords + list(model.params), g[i, j],
5569                                "numpy")
5570                     for j in range(self.n)]
5571         for i in range(self.n):
5572             self._gc[i] = [sp.lambdify(self.coords + list(model.params), self._gc[i, j],
5573                                       "numpy")
5574                           for j in range(self.n)]
5575         self.params = dict(params)
5576         self.compact_index = (model.compact[0].index if model.compact else None)
5577         self.period = 0.0
5578         if model.compact:
5579             try:
5580                 self.period = float(sp.N(model.compact[0].period.subs(
5581                     model.substitution(params))))
5582             except Exception:
5583                 # noqa: BLE001
5584                 self.period = 0.0
5585
5586         # -- curve construction -----
5587
5588     def unpack(self, theta: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
5589         n, K = self.n, self.modes
5590         centre = theta[:n]
5591         A = theta[n:n + n * K].reshape(n, K)
5592         B = theta[n + n * K:n + 2 * n * K].reshape(n, K)
5593         return centre, A, B
5594
5595     def curve(self, theta: np.ndarray, samples: int | None = None,
5596              winding: int = 0) -> tuple[np.ndarray, np.ndarray]:
5597         S = samples or self.samples
5598         centre, A, B = self.unpack(theta)
5599         lam = np.linspace(0.0, 1.0, S, endpoint=False)
5600         ks = np.arange(1, self.modes + 1)
5601         ang = 2 * np.pi * np.outer(lam, ks)
5602         x = centre[None, :] + np.cos(ang) @ A.T + np.sin(ang) @ B.T
5603         dx = ((-np.sin(ang) * (2 * np.pi * ks)) @ A.T
5604              + (np.cos(ang) * (2 * np.pi * ks)) @ B.T)
5605         if winding and self.compact_index is not None and self.period:
5606             x[:, self.compact_index] += self.period * winding * lam
5607             dx[:, self.compact_index] += self.period * winding
5608         return x, dx
5609
5610     def metrics_along(self, x: np.ndarray) -> np.ndarray:
5611         S = x.shape[0]
5612         cols = [x[:, k] for k in range(self.n)] + [
5613             np.full(S, v) for v in self.pvals]
5614         out = np.empty((S, self.n, self.n))
5615         with np.errstate(all="ignore"):
5616             for i in range(self.n):
5617                 for j in range(self.n):
5618                     try:

```

```

5614         v = self._gc[i][j](*cols)
5615     except Exception:
5616         # noqa: BLE001
5617         v = np.nan
5618     out[:, i, j] = np.broadcast_to(np.asarray(v, dtype=float), (S,))
5619     return (out + np.transpose(out, (0, 2, 1))) / 2.0
5620
5621 def norms(self, theta: np.ndarray, samples: int | None = None,
5622          winding: int = 0) -> tuple[np.ndarray, np.ndarray]:
5623     x, dx = self.curve(theta, samples, winding)
5624     G = self.metrics_along(x)
5625     q = np.einsum("ij,ijk,ik->i", dx, G, dx)
5626     return q, x, dx
5627
5628 # -- search -----
5629
5630 def objective(self, theta: np.ndarray, winding: int, alpha: float = 40.0,
5631             amplitude_cap: float = 5.0) -> float:
5632     """Smoothed maximum of g(gamma_dot, gamma_dot), with two guards.
5633
5634     The amplitude cap keeps the curve in a bounded coordinate region: an
5635     unconstrained optimiser drives the Fourier coefficients to infinity,
5636     because a curve of unbounded size trivially attains an unbounded
5637     negative norm. That runaway is a parametrisation artefact, not a
5638     CTC, and it is exactly the class of false positive FM-004 describes.
5639     The speed floor rejects degenerate (zero-tangent) curves.
5640     """
5641     q, _, dx = self.norms(theta, winding=winding)
5642     if not np.all(np.isfinite(q)):
5643         return 1e6
5644     smooth_max = float(np.log(np.sum(np.exp(alpha * (q - q.max()))))
5645                       / alpha + q.max())
5646     speed = np.einsum("ij,ij->i", dx, dx)
5647     penalty = float(np.sum(np.maximum(0.0, 1e-3 - speed) ** 2)) * 1e3
5648     amp = float(np.sum(theta[self.n:] ** 2))
5649     penalty += 1e3 * max(0.0, amp - amplitude_cap ** 2) ** 2
5650     # Normalise by the curve speed so that the reported number is a
5651     # scale-free causal statement rather than a proxy for curve length.
5652     norm = max(1e-12, float(np.mean(speed)))
5653     return smooth_max / norm + penalty
5654
5655 def search(self, centre: Mapping[str, float], winding: int = 1,
5656          restarts: int = 8, scale: float = 0.4, seed: int = 20260831,
5657          maxiter: int = 150) -> LoopCandidate | None:
5658     cand = self.search_all(centre, winding, restarts, scale, seed, maxiter)
5659     return cand[0] if cand else None
5660
5661 def search_all(self, centre: Mapping[str, float], winding: int = 1,
5662              restarts: int = 8, scale: float = 0.4, seed: int = 20260831,
5663              maxiter: int = 150) -> list[LoopCandidate]:
5664     """Return every restart's candidate, best objective first.
5665
5666     The verdict layer validates candidates in order and accepts the first
5667     that survives, instead of validating only the single best objective
5668     value. A runaway curve can win on the smoothed objective while
5669     failing validation, and discarding the runner-up would then lose a
5670     perfectly good witness.
5671     """
5672     rng = np.random.default_rng(seed)
5673     c0 = np.array([float(centre[str(s)]) for s in self.coords])
5674     dim = self.n * (1 + 2 * self.modes)
5675     out: list[tuple[float, np.ndarray]] = []
5676     for attempt in range(restarts):
5677         th = np.zeros(dim)
5678         th[self.n:] = c0
5679         # Restart 0 is the pure symmetry orbit (all Fourier coefficients
5680         # zero, closure supplied entirely by the winding term). Seeding
5681         # the search with the object the L5 detector already understands
5682         # makes the general search a strict extension of it rather than an
5683         # unrelated heuristic.
5684         if attempt > 0:
5685             th[self.n:] = rng.normal(scale=scale, size=dim - self.n)
5686         try:
5687             res = minimize(self.objective, th, args=(winding,),
5688                          method="Powell",
5689                          options={"maxiter": maxiter, "xtol": 1e-8,
5690                                   "ftol": 1e-10})
5691         except Exception:
5692             # noqa: BLE001
5693             continue
5694         out.append((float(res.fun), res.x))
5695     # Always include the untouched symmetry orbit as a candidate.
5696     seed_theta = np.zeros(dim)
5697     seed_theta[self.n:] = c0
5698     out.append((self.objective(seed_theta, winding), seed_theta))
5699     out.sort(key=lambda t: t[0])
5700     cand = list[LoopCandidate] = []
5701     for fval, th in out:
5702         q, x, dx = self.norms(th, winding=winding)
5703         if not np.all(np.isfinite(q)):
5704             continue
5705         speed = np.einsum("ij,ij->i", dx, dx)
5706         cand.append(LoopCandidate(
5707             float(np.max(q)), float(np.mean(q)), float(np.min(speed)),
5708             x, dx, th, self.samples, (winding,),
5709             "candidate only: not a witness until validated"))
5710     return cand
5711
5712 # -- validation -----
5713
5714 def validate(self, cand: LoopCandidate, winding: int = 1,
5715            refine: int = 4, tol: float = 1e-10,
5716            extent_limit: float = 1e4) -> LoopValidation:
5717     """Exact re-check of a candidate at higher resolution.
5718
5719     S = cand.n_samples * refine
5720     q, x, dx = self.norms(cand.coefficients, samples=S, winding=winding)
5721     if not (np.all(np.isfinite(q)) and np.all(np.isfinite(x))):
5722         return LoopValidation(False, float("nan"), 0.0, float("nan"),
5723                             n_samples=S, bounded=False,
5724                             note="candidate leaves the chart: the metric "
5725                                   "is not finite somewhere on the curve")
5726     extent = float(np.max(np.abs(x)))

```

```

5724         if extent > extent_limit or abs(float(np.max(np.abs(q)))) > 1e100:
5725             return LoopValidation(False, float(np.max(q)), 0.0, extent,
5726                                   n_samples=S, bounded=False,
5727                                   note=f"runaway candidate: coordinate extent "
5728                                       f"{extent:.3e} exceeds the limit "
5729                                       f"{extent_limit:.3e}. An unbounded curve "
5730                                       f"attains an unbounded negative norm by "
5731                                       f"parametrisation alone and is not a CTC.")
5732
5733         maxq = float(np.max(q))
5734         speed = np.einsum("ij,ij->i", dx, dx)
5735         maxq_norm = float(np.max(q / np.maximum(speed, 1e-300)))
5736         timelike = maxq < -tol
5737
5738         q2, _, _ = self.norms(cand.coefficients, samples=2 * S, winding=winding)
5739         stable = bool(np.sign(np.max(q2)) == np.sign(maxq)
5740                       and abs(np.max(q2) - maxq) <= 0.05 * max(1.0, abs(maxq)))
5741
5742         p0 = list(x[0])
5743         p1 = list(x[-1]) + (x[1] - x[0]) # close the sample ring
5744         cl = curve_closes(self.model, p0, p1, tol=1e-6,
5745                          params=self.model.substitution(self.params))
5746
5747         G = self.metrics_along(x)
5748         orr = check_curve_orientation(list(G), list(dx), closed=True)
5749
5750         degenerate = bool(np.min(np.abs(speed)) < 1e-8)
5751
5752         return LoopValidation(
5753             timelike, maxq, maxq_norm, extent, cl.closed, cl.mechanism,
5754             cl.residual, orr.consistent, degenerate, stable, S, True,
5755             note=(f"validation uses the EXACT maximum of g(gamma_dot, gamma_dot) "
5756                  f"on a refined grid, not the smoothed objective the optimiser "
5757                  f"minimised; the reported normalised margin is "
5758                  f"max g(u,u)/|u|^2, which for a pure symmetry orbit reduces "
5759                  f"exactly to the L5 compact-orbit margin"))
5760
5761 def loop_search_verdict(model: MetricModel, centre: Mapping[str, float],
5762                        winding: int = 1, **kw: Any) -> Verdict:
5763     """Run search + validation and produce a correctly scoped verdict."""
5764     gates = GateRecord()
5765     gates.set(Gate.V0_PARSE, GateResult.PASS)
5766     gates.set(Gate.V1_ALGEBRA, GateResult.PASS)
5767     ls = LoopSearch(model, centre, modes=kw.pop("modes", 3),
5768                    samples=kw.pop("samples", 256))
5769     cands = ls.search_all(centre, winding=winding, **kw)
5770     cand = None
5771     val = None
5772     for c in cands[:5]:
5773         v = ls.validate(c, winding=winding)
5774         if val is None or (v.accepted and not val.accepted):
5775             cand, val = c, v
5776         if v.accepted:
5777             break
5778     if cand is None:
5779         gates.set(Gate.V4_CTC_WITNESS, GateResult.NOT_ATTEMPTED, "optimiser failed")
5780         return Verdict(CTCStatus.UNRESOLVED, ClaimScope.SECTOR, float("nan"),
5781                       "loop_search", "optimiser produced no candidate; this is "
5782                                   "NOT evidence of absence",
5783                       model.id, dict(centre), gates)
5784     if val.accepted:
5785         gates.set(Gate.V4_CTC_WITNESS, GateResult.PASS,
5786                  "validated closed timelike curve")
5787         v = Verdict(CTCStatus.CTC_PROVED, ClaimScope.SECTOR,
5788                    val.max_norm_normalised, "loop_search",
5789                    "closed curve with max g(u,u)/|u|^2 = "
5790                    f"{val.max_norm_normalised:+.6e} < 0, "
5791                    f"closure via {val.closure_mechanism}, "
5792                    "time orientation consistent, stable under grid refinement",
5793                    model.id, dict(centre), gates, val.as_dict())
5794         return v
5795     gates.set(Gate.V4_CTC_WITNESS, GateResult.INCONCLUSIVE,
5796              "candidate failed validation")
5797     return Verdict(CTCStatus.UNRESOLVED, ClaimScope.SECTOR,
5798                    val.max_norm_normalised, "loop_search",
5799                    "candidate rejected by validation "
5800                    f"(timelike={val.timelike_everywhere}, closed={val.closed}, "
5801                    f"orientation={val.orientation_consistent}, "
5802                    f"stable={val.resolution_stable}, bounded={val.bounded}); "
5803                    "a failed search is never a no-CTC result",
5804                    model.id, dict(centre), gates, val.as_dict())
5805
5806 __all__ = ["LoopCandidate", "LoopValidation", "LoopSearch", "loop_search_verdict"]

```

src/ctcfa/horizons.py — 254 lines

```

5808 """
5809 CTC-FA v2.0 -- L8 : horizon taxonomy engine.
5810
5811 The single most common category error in the popular and semi-technical
5812 literature on time machines is the identification of *any* horizon with a
5813 chronology horizon. v1.0 stated the taxonomy in a table (section 10) but
5814 implemented only ``inverse_radial_horizon_candidates``, a solver for
5815 ``g^{rr} = 0``, which cannot distinguish the four objects at all.
5816
5817 v2.0 keeps them as separate computed objects, each with its own evidence and
5818 its own forbidden shortcut:
5819
5820 Killing horizon      a null hypersurface generated by a Killing field:
5821                      solved as ``g(xi, xi) = 0`` with ``d(g(xi,xi))``
5822                      null on the surface.
5823 Coordinate/metric    a locus where the chart degenerates: ``g^{rr} = 0``
5824 singularity          or ``det g = 0``, tested against curvature
5825                      invariants to separate chart failure from a real
5826                      singularity.
5827 Curvature            a locus where an invariant (Kretschmann) diverges.
5828 singularity

```



```

5829     Chronology horizon    the boundary of the chronology-violating set, which
5830                             in this architecture is computed from the reduced
5831                             causal margin, NOT inferred from any of the above.
5832
5833     Reissner-Nordstrom is the decisive control: it has an inner Cauchy horizon
5834     and no chronology violation whatsoever.  Misner is the converse: a
5835     chronology horizon with no curvature singularity anywhere.
5836     """
5837
5838     from __future__ import annotations
5839
5840     from dataclasses import dataclass, field
5841     from enum import Enum
5842     from typing import Any, Mapping, Sequence
5843
5844     import numpy as np
5845     import sympy as sp
5846
5847     from .model import MetricModel
5848     from .numerics import bracketed_bisection, coarse_sign_scan
5849     from .symbolic import kretschmann
5850
5851     class HorizonKind(str, Enum):
5852         KILLING = "KILLING_HORIZON"
5853         COORDINATE_DEGENERACY = "COORDINATE_DEGENERACY"
5854         CURVATURE_SINGULARITY = "CURVATURE_SINGULARITY"
5855         CHRONOLOGY = "CHRONOLOGY_HORIZON"
5856         ERGOSURFACE = "ERGOSURFACE"
5857
5858     @dataclass
5859     class HorizonCandidate:
5860         kind: HorizonKind
5861         equation: str
5862         solutions: list[str] = field(default_factory=list)
5863         numeric_roots: list[float] = field(default_factory=list)
5864         variable: str = ""
5865         note: str = ""
5866         forbidden_inference: str = ""
5867
5868         def as_dict(self) -> dict[str, Any]:
5869             return {"kind": self.kind.value, "equation": self.equation,
5870                     "solutions": self.solutions, "numeric_roots": self.numeric_roots,
5871                     "variable": self.variable, "note": self.note,
5872                     "forbidden_inference": self.forbidden_inference}
5873
5874     FORBIDDEN = {
5875         HorizonKind.KILLING: "A Killing horizon is not an event horizon without a "
5876                               "global argument, and it is never a chronology horizon.",
5877         HorizonKind.COORDINATE_DEGENERACY:
5878             "A chart degeneracy is not a singularity and not a horizon; check a "
5879             "curvature invariant before drawing any conclusion.",
5880         HorizonKind.CURVATURE_SINGULARITY:
5881             "A curvature singularity is not a chronology horizon.",
5882         HorizonKind.ERGOSURFACE:
5883             "An ergosurface is where a coordinate direction stops being timelike; "
5884             "it is not a horizon and not a CTC boundary.",
5885         HorizonKind.CHRONOLOGY:
5886             "A chronology horizon must be derived from the causal margin, never "
5887             "inferred from any of the other objects in this table.",
5888     }
5889
5890     def killing_horizon(model: MetricModel, index: int = 0,
5891                        variable: str = "r") -> HorizonCandidate:
5892         """Solve  $g(x_i, x_i) = 0$  for the coordinate Killing field  $d/dx^i$ """
5893         g = model.require_metric()
5894         expr = sp.simplify(g[index, index])
5895         try:
5896             var = model.symbol(variable)
5897             sols = sp.solve(sp.Eq(expr, 0), var)
5898         except Exception as exc:
5899             # noqa: BLE001
5900             sols, var = [], None
5901             expr = sp.sympify(f"solve failed: {exc}")
5902         return HorizonCandidate(
5903             HorizonKind.KILLING, f"g({model.coords[index]},{model.coords[index]}) "
5904                                 f"= {expr} = 0",
5905             [str(s) for s in sols], [], variable,
5906             "null hypersurface generated by a coordinate Killing field, where the "
5907             "declared symmetry generator becomes null",
5908             FORBIDDEN[HorizonKind.KILLING])
5909
5910     def ergosurface(model: MetricModel, index: int = 0,
5911                    variable: str = "r") -> HorizonCandidate:
5912         c = killing_horizon(model, index, variable)
5913         return HorizonCandidate(HorizonKind.ERGOSURFACE, c.equation, c.solutions,
5914                                c.numeric_roots, variable,
5915                                "locus where the reference Killing direction stops "
5916                                "being timelike",
5917                                FORBIDDEN[HorizonKind.ERGOSURFACE])
5918
5919     def coordinate_degeneracy(model: MetricModel,
5920                             variable: str = "r") -> HorizonCandidate:
5921         """Solve  $\det g = 0$  and  $g^{rr} = 0$ """
5922         g = model.require_metric()
5923         det = sp.simplify(g.det())
5924         sols: list[str] = []
5925         try:
5926             var = model.symbol(variable)
5927             sols += [str(s) for s in sp.solve(sp.Eq(det, 0), var)]
5928             gi = g.inv()
5929             i = model.coord_index(variable)
5930             num, _ = sp.fraction(sp.cancel(sp.simplify(gi[i, i])))
5931             sols += [str(s) for s in sp.solve(sp.Eq(num, 0), var)]
5932         except Exception:
5933             # noqa: BLE001
5934             pass

```



```

5939     return HorizonCandidate(
5940         HorizonKind.COORDINATE_DEGENERACY,
5941         f"det g = {det} = 0 and numerator(g^{variable}{variable}) = 0",
5942         sorted(set(sols)), [], variable,
5943         "chart degeneracy: must be tested against a curvature invariant",
5944         FORBIDDEN[HorizonKind.COORDINATE_DEGENERACY])
5945
5946
5947 def curvature_singularities(model: MetricModel, variable: str = "r",
5948                             max_ops: int = 4000) -> HorizonCandidate:
5949     """Locate poles of the Kretschmann scalar (a genuine invariant)."""
5950     g = model.require_metric()
5951     try:
5952         K = kretschmann(g, model.coords, simplify=True)
5953     except Exception as exc:
5954         # noqa: BLE001
5955         return HorizonCandidate(HorizonKind.CURVATURE_SINGULARITY,
5956                                 f"K unavailable: {exc}", [], [], variable,
5957                                 "", FORBIDDEN[HorizonKind.CURVATURE_SINGULARITY])
5958
5959     sols: list[str] = []
5960     try:
5961         var = model.symbol(variable)
5962         _, den = sp.fraction(sp.cancel(sp.together(K)))
5963         sols = [str(s) for s in sp.solve(sp.Eq(den, 0), var)]
5964     except Exception:
5965         # noqa: BLE001
5966         pass
5967     return HorizonCandidate(
5968         HorizonKind.CURVATURE_SINGULARITY,
5969         f"denominator of K = R_{abcd} R^{abcd} vanishes; K = {sp.simplify(K)}",
5970         sorted(set(sols)), [], variable,
5971         "genuine curvature singularity: invariant, chart independent",
5972         FORBIDDEN[HorizonKind.CURVATURE_SINGULARITY])
5973
5974
5975 def chronology_horizon(model: MetricModel, fixed: Mapping[str, float],
5976                         variable: str, lo: float, hi: float,
5977                         n: int = 512) -> HorizonCandidate:
5978     """Locate the chronology horizon as a zero of the reduced causal margin."""
5979     from .continuation import reduced_causal_margin_symbolic, reference_validity
5980     try:
5981         mu = reduced_causal_margin_symbolic(model)
5982         val = reference_validity(model)
5983     except ValueError as exc:
5984         # noqa: BLE001
5985         return HorizonCandidate(HorizonKind.CHRONOLOGY, f"unavailable: {exc}",
5986                                 [], [], variable, "",
5987                                 FORBIDDEN[HorizonKind.CHRONOLOGY])
5988     var = model.symbol(variable)
5989     expr = mu.subs({model.symbol(k): v for k, v in fixed.items()})
5990     f = sp.lambdify(var, expr, "numpy")
5991
5992     def q(x: float) -> float:
5993         with np.errstate(all="ignore"):
5994             v = float(f(x))
5995         return v if np.isfinite(v) else float("nan")
5996
5997     fval = sp.lambdify(var, val.subs({model.symbol(k): v
5998                                     for k, v in fixed.items()}), "numpy")
5999
6000     def reference_ok(x: float) -> bool:
6001         """Reject a zero of mu that is really a zero of the normalisation.
6002
6003         mu = q/h has spurious roots wherever the timelike reference direction
6004         goes null. Misner's mu = -T(T+2) has such a root at T = -2 and a
6005         genuine chronology horizon at T = 0; without this filter the report
6006         would list both."""
6007         try:
6008             with np.errstate(all="ignore"):
6009                 return float(fval(x)) > 1e-8
6010         except Exception:
6011             # noqa: BLE001
6012             return True
6013
6014     roots = []
6015     rejected = []
6016     for a, b in coarse_sign_scan(q, lo, hi, n=n):
6017         if a == b:
6018             (roots if reference_ok(a) else rejected).append(a)
6019             continue
6020         try:
6021             r, _, _ = bracketed_bisection(q, a, b, xtol=1e-13)
6022             (roots if reference_ok(r) else rejected).append(r)
6023         except ValueError:
6024             continue
6025     extra = (f"; {len(rejected)} root(s) rejected as normalisation artefacts "
6026             f"at {[round(x, 9) for x in rejected]}") if rejected else ""
6027     return HorizonCandidate(
6028         HorizonKind.CHRONOLOGY,
6029         f"reduced causal margin mu({variable}) = {sp.simplify(expr)} = 0",
6030         [], [float(r) for r in roots], variable,
6031         "boundary of the chronology-violating set, derived from the causal "
6032         "margin and from nothing else" + extra,
6033         FORBIDDEN[HorizonKind.CHRONOLOGY])
6034
6035
6036 def horizon_report(model: MetricModel, variable: str = "r",
6037                    killing_index: int = 0,
6038                    chronology_range: tuple[float, float] | None = None,
6039                    fixed: Mapping[str, float] | None = None,
6040                    with_curvature: bool = True) -> dict[str, Any]:
6041     """Full L8 report: every object computed separately and labelled."""
6042     out: dict[str, Any] = {"model_id": model.id, "variable": variable,
6043                           "candidates": []}
6044     if not model.has_metric:
6045         out["note"] = "no local metric: horizon structure is external"
6046         return out
6047     has_var = any(str(c) == variable for c in model.coords)
6048     if has_var:
6049         out["candidates"].append(killing_horizon(model, killing_index, variable).as_dict())
6050         out["candidates"].append(coordinate_degeneracy(model, variable).as_dict())
6051         if with_curvature:
6052             out["candidates"].append(
6053                 curvature_singularities(model, variable).as_dict())

```

```

6049     if chronology_range is not None and has_var:
6050         out["candidates"].append(chronology_horizon(
6051             model, fixed or {}, variable, *chronology_range).as_dict())
6052     out["taxonomy_rule"] = {
6053         "No entry in this report may be converted into another. In particular "
6054         "a Cauchy or Killing horizon is never, by itself, evidence of a "
6055         "chronology horizon (v1.0 REQ-006, FM-006).")
6056     return out
6057
6058
6059 __all__ = ["HorizonKind", "HorizonCandidate", "killing_horizon", "ergosurface",
6060            "coordinate_degeneracy", "curvature_singularities",
6061            "chronology_horizon", "horizon_report", "FORBIDDEN"]

```

src/ctcfa/energy.py — 638 lines

```

6062 """
6063 CTC-FA v2.0 -- L9 : energy-condition engine, with exact classification.
6064
6065 v1.0 evaluated the null and weak energy conditions by contracting the stress
6066 tensor with a Fibonacci sphere of 256 directions and reporting the minimum.
6067 Its own risk register named the consequence:
6068
6069     FM-009 "Energy-condition result based on finite directional samples"
6070     -> FALSE PROOF.
6071     L9 gate "Numerical directional sampling is diagnostic only;
6072     analytic/interval certification is required for formal PASS."
6073
6074 so the architecture could never issue a formal energy PASS. v2.0 removes
6075 sampling from the certification path entirely, by two independent exact
6076 routes that are required to agree.
6077
6078 Route A -- Hawking-Ellis classification (exact, closed form)
6079 -----
6080 Build an orthonormal frame ``E`` with ``E^T g E = eta = diag(-1,1,1,1)`` and
6081 form the mixed frame tensor ``eta * T_hat``. When that matrix is
6082 diagonalisable over the reals with exactly one timelike eigendirection the
6083 stress tensor is Hawking-Ellis **Type I**, its eigenvalues are
6084 ``(-rho, p_1, p_2, p_3)`` and the energy conditions have exact algebraic
6085 forms:
6086
6087     NEC  min_i (rho + p_i)           >= 0
6088     WEC  min(rho, min_i (rho + p_i)) >= 0
6089     DEC  min_i (rho - |p_i|)         >= 0
6090     SEC  min((rho + sum_i p_i)/2, min_i (rho + p_i)) >= 0
6091
6092 (the SEC factor of one half is the value of the trace-reversed contraction at
6093 the rest observer; the textbook inequality ``rho + sum p >= 0`` is the same
6094 condition with that factor divided out)
6095
6096 These are equalities of algebra, not estimates.
6097
6098 Route B -- exact constrained minimisation (trust-region subproblem)
6099 -----
6100 The null-energy margin is
6101
6102     NEC(x) = min over unit n of [ T00 + 2 T0i n^i + Tij n^i n^j ],
6103
6104 a quadratic on the unit sphere. That is the classical trust-region
6105 subproblem and it has an exact solution via the secular equation
6106 ``sum_i b_i^2/(lambda_i - mu)^2 = 1`` with ``mu <= lambda_min``. No sampling
6107 is involved and the result is the true global minimum to machine precision.
6108
6109 The two routes are computed independently and cross-checked; a disagreement
6110 larger than the uncertainty budget is reported as ``INCONCLUSIVE`` rather
6111 than silently resolved. For Type II/III/IV stress tensors, where the closed
6112 forms do not apply, Route B still certifies NEC/WEC and the report says so
6113 explicitly instead of pretending otherwise.
6114
6115 ANEC is computed on an actual integrated null geodesic (see
6116 ``ctcfa.geodesics``) with a trapezoid error estimate, not on a supplied list
6117 of contractions.
6118 """
6119
6120 from __future__ import annotations
6121
6122 import math
6123 from dataclasses import dataclass, field
6124 from enum import Enum
6125 from typing import Any, Callable, Mapping, Sequence
6126
6127 import numpy as np
6128 import sympy as sp
6129
6130 from .model import MetricModel
6131 from .numerics import ErrorBudget, SignClass, classify_sign, signature_counts
6132 from .symbolic import einstein_tensor
6133
6134 # -----
6135 # Frames
6136 # -----
6137
6138 def orthonormal_frame(g: np.ndarray, tol: float = 1e-10) -> np.ndarray:
6139     """Return ``E`` with ``E^T g E = diag(-1, 1, ..., 1)``.
6140
6141     Columns are ordered timelike first. Raises when the signature is not
6142     Lorentzian at the point, because every downstream energy statement would
6143     otherwise be meaningless.
6144     """
6145     gs = (np.asarray(g, dtype=float) + np.asarray(g, dtype=float).T) / 2.0
6146     vals, vecs = np.linalg.eigh(gs)
6147     scale = max(1.0, float(np.max(np.abs(vals))))
6148     t = tol * scale
6149     neg = np.where(vals < -t)[0]
6150     pos = np.where(vals > t)[0]
6151     if len(neg) != 1 or len(pos) != gs.shape[0] - 1:
6152         raise ValueError(f"metric is not Lorentzian at this point: ")

```

```

6154         f"eigenvalues {vals}")
6155     order = list(neg) + list(pos)
6156     E = np.column_stack([vecs[:, i] / math.sqrt(abs(vals[i])) for i in order])
6157     return E
6158
6159
6160 def frame_components(g: np.ndarray, T: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
6161     """Return ``E, T_hat`` with ``T_hat = E^T T E`` the frame components."""
6162     E = orthonormal_frame(g)
6163     That = E.T @ np.asarray(T, dtype=float) @ E
6164     return E, (That + That.T) / 2.0
6165
6166
6167 # -----
6168 # Exact trust-region subproblem: min n^T A n + 2 b.n s.t. |n| = 1
6169 # -----
6170
6171 def min_quadratic_on_sphere(A: np.ndarray, b: np.ndarray,
6172                             tol: float = 1e-14, maxiter: int = 300
6173                             ) -> tuple[float, np.ndarray]:
6174     """Exact global minimum of ``n^T A n + 2 b.n`` on the unit sphere.
6175
6176     Implements the standard secular-equation solution of the equality-
6177     constrained trust-region subproblem, including the hard case ``b`` in the
6178     orthogonal complement of the minimal eigenspace.
6179     """
6180     A = (np.asarray(A, dtype=float) + np.asarray(A, dtype=float).T) / 2.0
6181     b = np.asarray(b, dtype=float).ravel()
6182     lam, Q = np.linalg.eigh(A)
6183     bt = Q.T @ b
6184     lmin = float(lam[0])
6185     idx_min = np.where(np.abs(lam - lmin) <= 1e-12 * max(1.0, abs(lmin)))[0]
6186
6187     def n_of(mu: float) -> np.ndarray:
6188         d = lam - mu
6189         return Q @ (-bt / d)
6190
6191     # -- hard case: b has no component along the minimal eigenspace -----
6192     if np.all(np.abs(bt[idx_min]) <= 1e-14 * max(1.0, float(np.max(np.abs(bt)) or 1.0))):
6193         rest = np.array([i for i in range(len(lam)) if i not in idx_min])
6194         if rest.size:
6195             y = np.zeros_like(bt)
6196             y[rest] = -bt[rest] / (lam[rest] - lmin)
6197             nrm2 = float(y @ y)
6198             else:
6199                 y, nrm2 = np.zeros_like(bt), 0.0
6200             if nrm2 <= 1.0:
6201                 alpha = math.sqrt(max(0.0, 1.0 - nrm2))
6202                 y = y.copy()
6203                 y[idx_min[0]] += alpha
6204                 n = Q @ y
6205                 return float(n @ A @ n + 2 * b @ n), n
6206             # otherwise fall through to the secular equation on mu < lmin
6207
6208     lo = lmin - max(1.0, float(np.linalg.norm(b)) + float(np.max(np.abs(lam))))
6209     hi = lmin - 1e-16 * max(1.0, abs(lmin))
6210
6211     def phi(mu: float) -> float:
6212         d = lam - mu
6213         return float(np.sum((bt / d) ** 2)) - 1.0
6214
6215     while phi(lo) > 0 and lo > -1e18:
6216         lo = lmin - 2 * (lmin - lo)
6217         for _ in range(maxiter):
6218             mid = 0.5 * (lo + hi)
6219             v = phi(mid)
6220             if abs(v) < tol:
6221                 break
6222             if v > 0:
6223                 hi = mid
6224             else:
6225                 lo = mid
6226         mu = 0.5 * (lo + hi)
6227         n = n_of(mu)
6228         nrm = float(np.linalg.norm(n))
6229         if nrm > 0:
6230             n = n / nrm
6231         return float(n @ A @ n + 2 * b @ n), n
6232
6233
6234 def min_linear_on_sphere(c: float, q: np.ndarray) -> tuple[float, np.ndarray]:
6235     """Exact ``min_{|n|=1} (c + q.n) = c - |q|``."""
6236     q = np.asarray(q, dtype=float).ravel()
6237     nq = float(np.linalg.norm(q))
6238     n = -q / nq if nq > 0 else np.eye(len(q))[0]
6239     return float(c - nq), n
6240
6241
6242 # -----
6243 # Hawking-Ellis classification
6244 # -----
6245
6246 class HEType(str, Enum):
6247     TYPE_I = "TYPE_I"
6248     TYPE_II = "TYPE_II"
6249     TYPE_III = "TYPE_III"
6250     TYPE_IV = "TYPE_IV"
6251     UNDETERMINED = "UNDETERMINED"
6252
6253
6254 @dataclass
6255 class HawkingEllis:
6256     type: HEType
6257     rho: float | None
6258     pressures: list[float] = field(default_factory=list)
6259     eigenvalues: list[complex] = field(default_factory=list)
6260     note: str = ""
6261
6262     def as_dict(self) -> dict[str, Any]:
6263         return {"type": self.type.value, "rho": self.rho,

```

```

6264         "pressures": self.pressures,
6265         "eigenvalues": [complex(v).real if abs(complex(v).imag) < 1e-12
6266                        else str(v) for v in self.eigenvalues],
6267         "note": self.note}
6268
6269
6270 def classify_hawking_ellis(That: np.ndarray, tol: float = 1e-9) -> HawkingEllis:
6271     """Classify the frame stress tensor and, for Type I, extract (rho, p_i)."""
6272     n = That.shape[0]
6273     eta = np.diag([-1.0] + [1.0] * (n - 1))
6274     M = eta @ That # mixed tensor T^a_b
6275     # Vacuum is the degenerate case: T = 0 has every vector as an eigenvector,
6276     # so the "exactly one timelike eigendirection" test is ill posed. A zero
6277     # stress tensor is Type I with rho = p_i = 0 by definition, and saying so
6278     # explicitly avoids a spurious UNDETERMINED for every vacuum solution.
6279     if float(np.max(np.abs(That))) <= tol:
6280         return HawkingEllis(HEType.TYPE_I, 0.0, [0.0] * (n - 1), [0.0] * n,
6281                             "vacuum: T_ab = 0 to within tolerance; all energy "
6282                             "conditions hold with equality")
6283     vals, vecs = np.linalg.eig(M)
6284     if np.max(np.abs(vals.imag)) > tol * max(1.0, float(np.max(np.abs(vals.real)))):
6285         return HawkingEllis(HEType.TYPE_IV, None, [], list(vals),
6286                             "complex eigenvalue pair: Type IV, no closed-form "
6287                             "energy-condition criteria apply")
6288     valsr = vals.real
6289     # causal character of each eigenvector
6290     chars = []
6291     for i in range(n):
6292         v = vecs[:, i].real
6293         nv = float(np.linalg.norm(v))
6294         if nv == 0:
6295             chars.append("degenerate")
6296             continue
6297         v = v / nv
6298         q = float(v @ eta @ v)
6299         chars.append("timelike" if q < -tol else
6300                     "null" if abs(q) <= tol else "spacelike")
6301     n_time = chars.count("timelike")
6302     if n_time != 1:
6303         kind = HEType.TYPE_II if chars.count("null") >= 1 else HEType.UNDETERMINED
6304         return HawkingEllis(kind, None, [], list(valsr),
6305                             f"eigenvector characters {chars}: not Type I, so the "
6306                             f"closed-form criteria do not apply")
6307     it = chars.index("timelike")
6308     rho = float(-valsr[it])
6309     p = [float(valsr[i]) for i in range(n) if i != it]
6310     # verify diagonalisability
6311     if np.linalg.matrix_rank(vecs, tol=1e-9) < n:
6312         return HawkingEllis(HEType.TYPE_III, rho, p, list(valsr),
6313                             "eigenvector matrix is rank deficient: not "
6314                             "diagonalisable, Type III")
6315     return HawkingEllis(HEType.TYPE_I, rho, p, list(valsr),
6316                         "diagonalisable with a single timelike eigendirection")
6317
6318 # -----
6319 # Energy-condition report
6320 # -----
6321
6322
6323 @dataclass
6324 class EnergyReport:
6325     model_id: str
6326     point: dict[str, float]
6327     he: HawkingEllis
6328     nec_margin: float
6329     wec_margin: float
6330     sec_margin: float
6331     dec_margin: float
6332     nec_status: str
6333     wec_status: str
6334     sec_status: str
6335     dec_status: str
6336     method: str
6337     cross_check_error: float | None = None
6338     uncertainty: float = 0.0
6339     nec_worst_direction: list[float] = field(default_factory=list)
6340     note: str = ""
6341
6342     def as_dict(self) -> dict[str, Any]:
6343         return {"model_id": self.model_id, "point": self.point,
6344                 "hawking_ellis": self.he.as_dict(),
6345                 "margins": {"NEC": self.nec_margin, "WEC": self.wec_margin,
6346                             "SEC": self.sec_margin, "DEC": self.dec_margin},
6347                 "status": {"NEC": self.nec_status, "WEC": self.wec_status,
6348                             "SEC": self.sec_status, "DEC": self.dec_status},
6349                 "method": self.method, "cross_check_error": self.cross_check_error,
6350                 "uncertainty": self.uncertainty,
6351                 "nec_worst_direction": self.nec_worst_direction,
6352                 "note": self.note}
6353
6354
6355 def _status(margin: float, budget: float, certified: bool) -> str:
6356     cls = classify_sign(margin, budget)
6357     if cls is SignClass.MARGINAL:
6358         return "MARGINAL"
6359     if cls is SignClass.POSITIVE:
6360         return "SATISFIED" if certified else "SATISFIED_DIAGNOSTIC"
6361     return "VIOLATED"
6362
6363
6364 def energy_conditions(g: np.ndarray, T: np.ndarray, model_id: str = "",
6365                     point: Mapping[str, float] | None = None,
6366                     tol: float = 1e-10,
6367                     wec_grid: int = 257) -> EnergyReport:
6368     """Classify NEC / WEC / SEC / DEC at one point, exactly where possible."""
6369     g = np.asarray(g, dtype=float)
6370     T = np.asarray(T, dtype=float)
6371     E, That = frame_components(g, T)
6372     dim = That.shape[0]
6373     he = classify_hawking_ellis(That)

```

```

6374
6375     c = float(That[0, 0])
6376     bvec = That[0, 1:].astype(float)
6377     A = That[1:, 1:].astype(float)
6378
6379     # ---- Route B : exact minimisation over the null cone (NEC) -----
6380     nec_B, nvec = min_quadratic_on_sphere(A, bvec)
6381     nec_B = c + nec_B                                # T00 + 2 T0i n^i + Tij n^i n^j
6382
6383     # ---- Route B : WEC by exact-in-n minimisation on a beta grid -----
6384     wec_B = c
6385     for beta in np.linspace(0.0, 1.0, wec_grid):
6386         val, _ = min_quadratic_on_sphere(beta * beta * A, beta * bvec)
6387         wec_B = min(wec_B, c + val)
6388
6389     # ---- SEC via the trace-reversed tensor -----
6390     eta = np.diag([-1.0] + [1.0] * (dim - 1))
6391     trace = float(np.trace(eta @ That))
6392     Sbar = That - 0.5 * trace * eta
6393     cs = float(Sbar[0, 0])
6394     bs = Sbar[0, 1:].astype(float)
6395     As = Sbar[1:, 1:].astype(float)
6396     sec_B = cs
6397     for beta in np.linspace(0.0, 1.0, wec_grid):
6398         val, _ = min_quadratic_on_sphere(beta * beta * As, beta * bs)
6399         sec_B = min(sec_B, cs + val)
6400
6401     # ---- Route B' : the same minimisation in the STRESS eigenframe -----
6402     #
6403     # The values returned by Route B above are frame dependent: the family
6404     # u = gamma(1, beta n) with |n| = 1 depends on which orthonormal frame
6405     # the components are written in, so "min over beta and n" is not an
6406     # invariant of T_ab. Only the SIGN is. To obtain a value-level
6407     # cross-check of the closed forms we therefore repeat the exact
6408     # minimisation in the stress eigenframe, where the family does coincide
6409     # with the one the Hawking-Ellis criteria refer to. Reporting the
6410     # metric-frame numbers as if they were invariants would be a fake
6411     # agreement, so they are kept as sign-level diagnostics only.
6412     nec_Bp = wec_Bp = sec_Bp = None
6413     if he.type is HEType.TYPE_I and he.rho is not None:
6414         rho_t, p_t = float(he.rho), list(he.pressures)
6415         Td = np.diag([rho_t] + p_t)
6416         cd, bd, Ad = float(Td[0, 0]), Td[0, 1:], Td[1:, 1:]
6417         v, _ = min_quadratic_on_sphere(Ad, bd)
6418         nec_Bp = cd + v
6419         wec_Bp = cd
6420         for beta in np.linspace(0.0, 1.0, wec_grid):
6421             vv, _ = min_quadratic_on_sphere(beta * beta * Ad, beta * bd)
6422             wec_Bp = min(wec_Bp, cd + vv)
6423             tr_d = float(np.trace(np.diag([-1.0] + [1.0] * (dim - 1)) @ Td))
6424             Sd = Td - 0.5 * tr_d * np.diag([-1.0] + [1.0] * (dim - 1))
6425             cs_d, bs_d, As_d = float(Sd[0, 0]), Sd[0, 1:], Sd[1:, 1:]
6426             sec_Bp = cs_d
6427             for beta in np.linspace(0.0, 1.0, wec_grid):
6428                 vv, _ = min_quadratic_on_sphere(beta * beta * As_d, beta * bs_d)
6429                 sec_Bp = min(sec_Bp, cs_d + vv)
6430
6431     # ---- Route A : closed forms for Type I -----
6432     certified = he.type is HEType.TYPE_I
6433     if certified:
6434         rho, p = float(he.rho), list(he.pressures)
6435         nec_A = min([rho + pi for pi in p])
6436         wec_A = min([rho] + [rho + pi for pi in p])
6437         # The SEC margin is the infimum of (T_ab - (1/2) T g_ab) u^a u^b over
6438         # UNIT timelike u. At u = (1,0,0,0) that contraction equals
6439         # (rho + sum p)/2, not rho + sum p; the familiar textbook form
6440         # "rho + sum p >= 0" is the same INEQUALITY with the factor of two
6441         # divided out. Keeping the factor here is what makes Route A and
6442         # Route B numerically identical instead of merely sign-compatible.
6443         sec_A = min([(rho + sum(p)) / 2.0] + [rho + pi for pi in p])
6444         dec_A = min([rho] + [rho - abs(pi) for pi in p])
6445         # DEC is cross-checked by SIGN only. The alternating route minimises
6446         # T_ab k^a l^b over future null k, l normalised by k^0 = l^0 = 1,
6447         # which is a different normalisation from the Hawking-Ellis margin
6448         # min_i(rho - |p_i|); the two agree on the inequality but not on the
6449         # numerical value, and pretending otherwise would be a fake agreement.
6450         dec_B = _dec_alternating(That)
6451         dec_sign_ok = (dec_A >= -1e-12) == (dec_B >= -1e-12)
6452         cross = max(abs(nec_A - nec_Bp), abs(wec_A - wec_Bp),
6453                     abs(sec_A - sec_Bp))
6454         sign_ok = all(
6455             (a >= -1e-9) == (b >= -1e-9)
6456             for a, b in ((nec_A, nec_B), (wec_A, wec_B), (sec_A, sec_B)))
6457         nec, wec, sec, dec = nec_A, wec_A, sec_A, dec_A
6458         method = "hawking_ellis_type_I_closed_form + trust_region_cross_check"
6459         note = ("exact algebraic criteria; NEC/WEC/SEC cross-checked by value "
6460               "against exact trust-region minimisation in the stress "
6461               "eigenframe, and by sign against the same minimisation in the "
6462               "metric eigenframe (sign agreement: {sign_ok}); DEC "
6463               f"cross-checked by sign (agreement: {dec_sign_ok})")
6464     else:
6465         cross = None
6466         nec, wec, sec = nec_B, wec_B, sec_B
6467         dec = _dec_alternating(That)
6468         method = "trust_region_exact_minimisation (non-Type-I stress tensor)"
6469         note = (f"stress tensor is {he.type.value}: the Type I closed forms do "
6470               f"not apply. NEC/WEC/SEC margins are exact global minima; the "
6471               f"DEC value is an alternating-minimisation upper bound and is "
6472               f"DIAGNOSTIC, not a certificate.")
6473
6474     scale = max(1.0, float(np.max(np.abs(That))))
6475     budget = ErrorBudget()
6476     budget.roundoff = float(np.finfo(np.float64).eps) * scale * dim * 8
6477     budget.conditioning = float(np.finfo(np.float64).eps) * float(
6478         np.linalg.cond(g)) * scale
6479     budget.substitution = tol
6480     if not certified:
6481         budget.optimization = scale * 1e-12
6482
6483     return EnergyReport(

```

```

6484         model_id=model_id, point=dict(point or {}), he=he,
6485         nec_margin=float(nec), wec_margin=float(wec), sec_margin=float(sec),
6486         dec_margin=float(dec),
6487         nec_status=status(nec, budget.total, certified),
6488         wec_status=status(wec, budget.total, certified),
6489         sec_status=status(sec, budget.total, certified),
6490         dec_status=status(dec, budget.total, certified and True),
6491         method=method, cross_check_error=cross, uncertainty=budget.total,
6492         nec_worst_direction=list(map(float, nvec)), note=note)
6493
6494
6495 def _dec_alternating(That: np.ndarray, restarts: int = 12,
6496                     iters: int = 60, seed: int = 20260831) -> float:
6497     """Alternating exact minimisation of ``T_ab k^a l^b`` over future null k, l.
6498
6499     Each half-step is a *linear* problem on a sphere and is solved exactly;
6500     the alternation gives an upper bound on the global minimum. Reported as
6501     diagnostic.
6502     """
6503     rng = np.random.default_rng(seed)
6504     d = That.shape[0] - 1
6505     best = float("inf")
6506     for _ in range(restarts):
6507         u = rng.normal(size=d)
6508         u /= np.linalg.norm(u)
6509         for _ in range(iters):
6510             # minimise over l with k = (1, u) fixed
6511             k = np.concatenate([1.0, u])
6512             row = k @ That # T_ab k^a -> covector in b
6513             cl, ql = float(row[0]), row[1:]
6514             v = min_linear_on_sphere(cl, ql)
6515             l = np.concatenate([1.0, v])
6516             col = That @ l
6517             ck, qk = float(col[0]), col[1:]
6518             u2 = min_linear_on_sphere(ck, qk)
6519             if np.allclose(u, u2, atol=1e-14):
6520                 u = u2
6521                 break
6522             u = u2
6523             k = np.concatenate([1.0, u])
6524             l = np.concatenate([1.0, v])
6525             best = min(best, float(k @ That @ l))
6526     return best
6527
6528
6529 # -----
6530 # Model-level entry point
6531 # -----
6532
6533 _EINSTEIN_CACHE: dict[str, sp.Matrix] = {}
6534
6535
6536 def symbolic_einstein(model: MetricModel) -> sp.Matrix:
6537     """Cached, *unsimplified* symbolic Einstein tensor.
6538
6539     Simplification is what makes the exact route intractable for Kerr,
6540     Alcubierre and Krasnikov -- ``sympy.simplify`` on those expression trees
6541     does not terminate usefully. Symbolic *differentiation* is cheap, and
6542     since the result is immediately evaluated at a numeric point, algebraic
6543     tidiness buys nothing. Skipping it turns a 45-second timeout into two
6544     seconds and keeps the computation exact, so the finite-difference path
6545     (``ctcfa.numgeom``) is needed only as an independent cross-check.
6546     """
6547     if model.id not in _EINSTEIN_CACHE:
6548         _EINSTEIN_CACHE[model.id] = einstein_tensor(
6549             model.require_metric(), model.coords, simplify=False)
6550     return _EINSTEIN_CACHE[model.id]
6551
6552
6553 def stress_tensor_from_metric(model: MetricModel,
6554                               values: Mapping[str, float],
6555                               Lambda: sp.Expr | None = None,
6556                               digits: int = 30) -> np.ndarray:
6557     """T_ab implied by the field equations: T = (G + Lambda g) / (8 pi).
6558
6559     This is the *only* legitimate way to obtain a stress tensor from a bare
6560     metric. It is an on-shell definition, and the report says so, because
6561     calling the result "the matter content" for a metric that was written
6562     down by hand is exactly the off-shell error the L2 gate rejects.
6563
6564     Evaluation is carried out at ``digits`` significant figures so that the
6565     cancellations in a vacuum solution (Kerr, Taub-NUT) are resolved well
6566     below the energy-condition uncertainty band instead of being masked by
6567     double-precision round-off.
6568     """
6569     g = model.require_metric()
6570     Lam = model.matter.cosmological_constant if Lambda is None else Lambda
6571     G = symbolic_einstein(model)
6572     Tsym = (G + Lam * g) / (8 * sp.pi)
6573     sub = model.substitution(values)
6574     return np.array(sp.Matrix(Tsym).subs(sub).evalf(digits), dtype=float)
6575
6576
6577 #: Above this symbolic complexity the exact Einstein tensor is not attempted
6578#: and the finite-difference path is used instead, with its error reported.
6579SYMBOLIC_OPS_BUDGET = 100000
6580
6581
6582 def metric_complexity(model: MetricModel) -> int:
6583     """Symbolic size of the metric, used only to choose the evaluation path.
6584
6585     With the unsimplified Einstein tensor of :func:`symbolic_einstein` every
6586     model in the v2.0 registry stays on the exact route, so the budget is set
6587     high and the numeric path is reserved for models added later whose
6588     symbolic differentiation itself becomes impractical.
6589     """
6590     g = model.require_metric()
6591     return int(sum(sp.count_ops(e) for e in g))
6592
6593

```

```

6594 def energy_report(model: MetricModel, values: Mapping[str, float] | None = None,
6595                   T: np.ndarray | None = None, path: str = "auto",
6596                   **kw: Any) -> EnergyReport:
6597     """Energy-condition report for one model at one point.
6598
6599     ``path`` selects how the stress tensor is obtained:
6600     ``"symbolic"`` (exact, slow), ``"numeric"`` (finite differences with a
6601     reported error) or ``"auto"`` , which uses the symbolic route whenever the
6602     metric's operation count is below ``SYMBOLIC_OPS_BUDGET``. The chosen
6603     path is recorded in the report, because an energy PASS obtained from
6604     finite differences is diagnostic, not a certificate.
6605     """
6606     values = dict(values or model.default_point)
6607     gnum = np.array(sp.Matrix(model.require_metric())
6608                    .subs(model.substitution(values)).evalf(), dtype=float)
6609     num_err = 0.0
6610     used = path
6611     if T is None:
6612         if path == "auto":
6613             used = ("symbolic" if metric_complexity(model) <= SYMBOLIC_OPS_BUDGET
6614                    else "numeric")
6615         if used == "symbolic":
6616             T = stress_tensor_from_metric(model, values)
6617         else:
6618             from .numgeom import NumericGeometry
6619             Lam = float(sp.N(model.matter.cosmological_constant.subs(
6620                 model.substitution(values)))) \
6621                 if model.matter.cosmological_constant != 0 else 0.0
6622             T, num_err = NumericGeometry(model).stress_tensor(values, Lam)
6623     else:
6624         used = "supplied"
6625
6626     rep = energy_conditions(gnum, T, model.id, values, **kw)
6627     rep.uncertainty += num_err
6628     rep.method += f" | stress-tensor path: {used}"
6629     rep.note += (" | stress tensor obtained on shell from G_ab + Lambda g_ab "
6630                "= 8 pi T_ab")
6631     if used == "numeric":
6632         rep.note += (f" | finite-difference Einstein tensor, error estimate "
6633                    f"{num_err:.2e}: DIAGNOSTIC ONLY, cannot certify a PASS")
6634     for attr in ("nec_status", "wec_status", "sec_status", "dec_status"):
6635         v = getattr(rep, attr)
6636         if v == "SATISFIED":
6637             setattr(rep, attr, "SATISFIED_DIAGNOSTIC")
6638     return rep
6639
6640 # -----
6641 # ANEC
6642 # -----
6643
6644 @dataclass
6645 class ANECResult:
6646     value: float
6647     error_estimate: float
6648     n_samples: int
6649     affine_range: tuple[float, float]
6650     status: str
6651     note: str = ""
6652
6653     def as_dict(self) -> dict[str, Any]:
6654         return {"value": self.value, "error_estimate": self.error_estimate,
6655                "n_samples": self.n_samples,
6656                "affine_range": list(self.affine_range), "status": self.status,
6657                "note": self.note}
6658
6659
6660 def anec_integral(contractions: Sequence[float], affine: Sequence[float],
6661                  tol: float = 1e-9) -> ANECResult:
6662     """Trapezoid ANEC integral with a Richardson error estimate.
6663
6664     The integral is over the *supplied* geodesic segment only; ANEC is a
6665     statement about a complete null geodesic, so a finite segment can refute
6666     ANEC (a negative total on a segment that dominates the rest) but cannot
6667     certify it. The status field says which.
6668     """
6669     lam = np.asarray(affine, dtype=float)
6670     f = np.asarray(contractions, dtype=float)
6671     if lam.size < 3:
6672         return ANECResult(float("nan"), float("inf"), int(lam.size),
6673                           (float(lam[0]) if lam.size else 0.0,
6674                            float(lam[-1]) if lam.size else 0.0),
6675                           "UNRESOLVED", "too few samples")
6676     full = float(np.trapezoid(f, lam))
6677     half = float(np.trapezoid(f[1:], lam[1:]))
6678     err = abs(full - half) / 3.0
6679     if full < -max(tol, err):
6680         status = "REFUTED_ON_SEGMENT"
6681         note = ("the integral over this segment is negative, which refutes ANEC "
6682                "for this geodesic provided the segment is representative")
6683     elif full > max(tol, err):
6684         status = "POSITIVE_ON_SEGMENT"
6685         note = ("positive on this segment only; ANEC concerns the COMPLETE "
6686                "geodesic and is therefore not certified by this computation")
6687     else:
6688         status = "MARGINAL"
6689         note = "within the estimated integration error of zero"
6690     return ANECResult(full, err, int(lam.size),
6691                      (float(lam[0]), float(lam[-1])), status, note)
6692
6693
6694 __all__ = ["orthonormal_frame", "frame_components", "min_quadratic_on_sphere",
6695            "min_linear_on_sphere", "HEType", "HawkingEllis",
6696            "classify_hawking_ellis", "EnergyReport", "energy_conditions",
6697            "stress_tensor_from_metric", "energy_report", "ANECResult",
6698            "anec_integral"]

```

src/ctcfa/geodesics.py — 279 lines


```

6700 """
6701 CTC-FA v2.0 -- L18 : geodesics, accessibility and ANEC on a real geodesic.
6702
6703 Two jobs.
6704
6705 1. **Integration.** Integrate the geodesic equation
6706 ``d2x^a/dl^2 = -Gamma^a_bc dx^b/dl dx^c/dl`` with an adaptive stiff-capable
6707 integrator, monitoring the conserved norm ``g(u,u)`` as an *independent*
6708 error diagnostic. v1.0 built the right-hand side from symbolic
6709 Christoffels via ``lambdify`` but never checked the norm drift, so a
6710 silently diverging integration would have been reported as a trajectory.
6711
6712 2. **Accessibility.** Section 31.1 of v1.0 is explicit that a compact
6713 witness "does not establish that an astronaut can enter that region,
6714 survive, return to a desired external event, or that the spacetime can be
6715 created from realistic initial data". v2.0 turns that paragraph into a
6716 computation: from a declared observer domain, integrate future-directed
6717 causal curves and report whether the chronology-violating region is
6718 *reached*, with the four claim levels kept apart:
6719
6720     EXISTS      the CTC region is nonempty      (L5/L5b)
6721     REACHABLE    a future-directed causal curve from the observer
6722                  domain enters it
6723     SURVIVABLE   tidal/curvature invariants stay bounded along that curve
6724     RETURNABLE   a causal curve exists from the region back to an
6725                  external past event
6726
6727 Each is reported separately. A model may be EXISTS and not REACHABLE, and
6728 that distinction is the whole content of the "mathematical existence is
6729 not a time machine" caveat.
6730
6731 3. **ANEC.** The averaged null energy condition is integrated along an
6732 actual affinely parametrised null geodesic produced here, not along a
6733 caller-supplied list of numbers.
6734 """
6735
6736 from __future__ import annotations
6737
6738 import math
6739 from dataclasses import dataclass, field
6740 from typing import Any, Callable, Mapping, Sequence
6741
6742 import numpy as np
6743 import sympy as sp
6744 from scipy.integrate import solve_ivp
6745
6746 from .energy import ANECResult, anec_integral, stress_tensor_from_metric
6747 from .model import MetricModel
6748 from .numgeom import NumericGeometry
6749 from .symbolic import christoffel
6750
6751 @dataclass
6752 class GeodesicResult:
6753     affine: np.ndarray
6754     x: np.ndarray
6755     u: np.ndarray
6756     norm: np.ndarray
6757     norm_drift: float
6758     success: bool
6759     message: str
6760     kind: str = "unknown"
6761
6762     def as_dict(self) -> dict[str, Any]:
6763         return {"n_points": int(self.affine.size),
6764                 "affine_range": [float(self.affine[0]), float(self.affine[-1])],
6765                 "norm_initial": float(self.norm[0]) if self.norm.size else None,
6766                 "norm_final": float(self.norm[-1]) if self.norm.size else None,
6767                 "norm_drift": self.norm_drift, "success": self.success,
6768                 "kind": self.kind, "message": self.message}
6769
6770 class GeodesicIntegrator:
6771     """Lambdified Christoffel symbols plus an adaptive ODE integration."""
6772
6773     def __init__(self, model: MetricModel, simplify: bool = False):
6774         self.model = model
6775         g = model.require_metric()
6776         coords = list(model.coords)
6777         params = list(model.params)
6778         n = len(coords)
6779         Gam = christoffel(g, coords, simplify=simplify)
6780         flat = [Gam[a][b][c] for a in range(n) for b in range(n) for c in range(n)]
6781         self._G = sp.lambdify(coords + params, flat, "numpy")
6782         self._g = sp.lambdify(coords + params, sp.Matrix(g), "numpy")
6783         self.n = n
6784         self.coords = coords
6785         self.params = params
6786
6787     def christoffel_at(self, x: Sequence[float], p: Sequence[float]) -> np.ndarray:
6788         vals = np.asarray(self._G(*list(x), *list(p)), dtype=float)
6789         return vals.reshape(self.n, self.n, self.n)
6790
6791     def metric_at(self, x: Sequence[float], p: Sequence[float]) -> np.ndarray:
6792         m = np.array(self._g(*list(x), *list(p)), dtype=float)
6793         return (m + m.T) / 2.0
6794
6795     def rhs(self, lam: float, y: np.ndarray, p: np.ndarray) -> np.ndarray:
6796         n = self.n
6797         x, u = y[:n], y[n:]
6798         Gam = self.christoffel_at(x, p)
6799         acc = -np.einsum("abc,b,c->a", Gam, u, u)
6800         return np.concatenate([u, acc])
6801
6802     def integrate(self, x0: Sequence[float], u0: Sequence[float],
6803                  params: Mapping[str, float],
6804                  lam_span: tuple[float, float] = (0.0, 10.0),
6805                  n_out: int = 401, rtol: float = 1e-10,
6806                  atol: float = 1e-12, method: str = "DOP853") -> GeodesicResult:

```



```

6810         p = np.array([float(params[str(s)]) for s in self.params], dtype=float)
6811         y0 = np.concatenate([np.asarray(x0, float), np.asarray(u0, float)])
6812         t_eval = np.linspace(lam_span[0], lam_span[1], n_out)
6813         sol = solve_ivp(self.rhs, lam_span, y0, args=(p,), t_eval=t_eval,
6814                        rtol=rtol, atol=atol, method=method, dense_output=False)
6815         n = self.n
6816         xs = sol.y[:n].T
6817         us = sol.y[n:].T
6818         norms = np.array([float(u @ self.metric_at(x, p) @ u)
6819                           for x, u in zip(xs, us)])
6820         drift = (float(np.max(np.abs(norms - norms[0]))) if norms.size
6821                 else float("nan"))
6822         kind = ("null" if abs(norms[0]) < 1e-9 else
6823                "timelike" if norms[0] < 0 else "spacelike") if norms.size else "?"
6824         return GeodesicResult(sol.t, xs, us, norms, drift, bool(sol.success),
6825                               sol.message, kind)
6826
6827
6828 def null_direction(model: MetricModel, values: Mapping[str, float],
6829                   spatial: Sequence[float] | None = None) -> np.ndarray:
6830     """Build a future-directed null vector at a point.
6831
6832     Uses the orthonormal frame, so the construction works in an ergoregion
6833     and in charts with no timelike coordinate direction.
6834     """
6835     from .energy import orthonormal_frame
6836     g = np.array(sp.Matrix(model.require_metric())
6837                  .subs(model.substitution(values)).evalf(), dtype=float)
6838     E = orthonormal_frame(g)
6839     d = model.dim - 1
6840     if spatial is None:
6841         spatial = np.zeros(d)
6842         spatial[0] = 1.0
6843     s = np.asarray(spatial, dtype=float)
6844     s = s / np.linalg.norm(s)
6845     khat = np.concatenate([1.0, s])
6846     return E @ khat
6847
6848
6849 def anec_on_geodesic(model: MetricModel, values: Mapping[str, float],
6850                     spatial: Sequence[float] | None = None,
6851                     lam_max: float = 5.0, n_out: int = 401,
6852                     **kw: Any) -> tuple[ANECResult, GeodesicResult]:
6853     """Integrate a null geodesic and evaluate ``int T_ab k^a k^b dl`` on it."""
6854     integ = GeodesicIntegrator(model)
6855     k0 = null_direction(model, values, spatial)
6856     x0 = [float(values[str(c)]) for c in model.coords]
6857     geo = integ.integrate(x0, k0, values, (0.0, lam_max), n_out, **kw)
6858
6859     ng = NumericGeometry(model)
6860     Lam = 0.0
6861     if model.matter.cosmological_constant != 0:
6862         Lam = float(sp.N(model.matter.cosmological_constant.subs(
6863             model.substitution(values))))
6864     p = {str(s): float(values[str(s)]) for s in model.params}
6865     contractions = []
6866     for x, u in zip(geo.x, geo.u):
6867         pt = {str(c): float(v) for c, v in zip(model.coords, x)}
6868         pt.update(p)
6869         try:
6870             T, _ = ng.stress_tensor(pt, Lam)
6871             contractions.append(float(u @ T @ u))
6872         except Exception:
6873             contractions.append(float("nan")) # noqa: BLE001
6874     arr = np.asarray(contractions)
6875     good = np.isfinite(arr)
6876     res = anec_integral(arr[good], geo.affine[good])
6877     res.note += (f" | geodesic norm drift {geo.norm_drift:.2e} "
6878                f"(an independent accuracy check on the integration)")
6879     return res, geo
6880
6881
6882 # -----
6883 # Accessibility
6884 # -----
6885
6886 @dataclass
6887 class AccessibilityReport:
6888     model_id: str
6889     exists: bool
6890     reachable: bool | None
6891     survivable: bool | None
6892     returnable: bool | None
6893     observer_domain: dict[str, Any] = field(default_factory=dict)
6894     trials: int = 0
6895     entered_at: dict[str, float] | None = None
6896     max_curvature_on_path: float | None = None
6897     note: str = ""
6898
6899     def as_dict(self) -> dict[str, Any]:
6900         return {"model_id": self.model_id, "EXISTS": self.exists,
6901                "REACHABLE": self.reachable, "SURVIVABLE": self.survivable,
6902                "RETURNABLE": self.returnable,
6903                "observer_domain": self.observer_domain, "trials": self.trials,
6904                "entered_at": self.entered_at,
6905                "max_curvature_on_path": self.max_curvature_on_path,
6906                "note": self.note}
6907
6908
6909 def accessibility(model: MetricModel,
6910                  observer_point: Mapping[str, float],
6911                  ctc_predicate: Callable[[Mapping[str, float]], bool],
6912                  n_directions: int = 64, lam_max: float = 20.0,
6913                  curvature_limit: float = 1e6,
6914                  seed: int = 20260831) -> AccessibilityReport:
6915     """Shoot future-directed timelike geodesics and test whether they enter
6916     the chronology-violating set.
6917
6918     A negative result here is explicitly NOT a proof of inaccessibility: the
6919     search is a finite shooting scan and inherits the same asymmetry as every

```

```

6920     other search in the architecture.
6921     """
6922     from .energy import orthonormal_frame
6923     integ = GeodesicIntegrator(model)
6924     g = np.array(sp.Matrix(model.require_metric())
6925                 .subs(model.substitution(observer_point))).evalf(), dtype=float)
6926     E = orthonormal_frame(g)
6927     rng = np.random.default_rng(seed)
6928     d = model.dim - 1
6929     x0 = [float(observer_point[str(c)]) for c in model.coords]
6930     ng = NumericGeometry(model)
6931     p = {str(s): float(observer_point[str(s)]) for s in model.params}
6932
6933     entered = None
6934     max_curv = 0.0
6935     for i in range(n_directions):
6936         v = rng.normal(size=d)
6937         v /= np.linalg.norm(v)
6938         beta = 0.9 * rng.random()
6939         gamma = 1.0 / math.sqrt(1.0 - beta * beta)
6940         uhat = gamma * np.concatenate([[1.0], beta * v])
6941         u0 = E @ uhat
6942         geo = integ.integrate(x0, u0, observer_point, (0.0, lam_max), 301)
6943         if not geo.success:
6944             continue
6945         for x in geo.x:
6946             pt = {str(c): float(val) for c, val in zip(model.coords, x)}
6947             pt.update(p)
6948             try:
6949                 if ctc_predicate(pt):
6950                     entered = pt
6951                     break
6952             except Exception:
6953                 continue
6954             if entered is not None:
6955                 try:
6956                     cur = ng.curvature(entered)
6957                     max_curv = abs(cur.kretschmann)
6958                 except Exception:
6959                     max_curv = float("nan")
6960                 break
6961
6962     reachable = entered is not None
6963     survivable = (None if not reachable else
6964                  (math.isfinite(max_curv) and max_curv < curvature_limit))
6965     return AccessibilityReport(
6966         model.id, exists=True, reachable=reachable, survivable=survivable,
6967         returnable=None,
6968         observer_domain={str(k): float(v) for k, v in observer_point.items()},
6969         trials=n_directions, entered_at=entered,
6970         max_curvature_on_path=(None if not reachable else float(max_curv)),
6971         note=("REACHABLE=False from a finite shooting scan is UNRESOLVED, not a "
6972             "proof of inaccessibility. RETURNABLE is not attempted here: it "
6973             "requires a causal-past computation on the quotient and is a "
6974             "v2.1 target."))
6975
6976
6977 __all__ = ["GeodesicResult", "GeodesicIntegrator", "null_direction",
6978           "anec_on_geodesic", "AccessibilityReport", "accessibility"]

```

src/ctcfa/perturbation.py — 198 lines

```

6979 """
6980 CTC-FA v2.0 -- L11 : perturbation engine, with the on-shell gate enforced.
6981
6982 v1.0 provided ``perturb_metric(model, h, epsilon)`` which added an arbitrary
6983 symmetric tensor to the metric and returned a new ``MetricModel`` with
6984 ``expected = UNRESOLVED``. Nothing prevented the causal margin of that
6985 object from being quoted as a stability result, and the architecture's own
6986 gate says why that is invalid:
6987
6988     L11 gate "Off-shell perturbations cannot establish physical robustness."
6989     FM-005 "Off-shell perturbation used as physical stability evidence"
6990     -> INVALID PHYSICS.
6991
6992 v2.0 makes the gate structural. A perturbation carries an
6993 :class: `OnShellStatus`, computed by measuring how far the perturbed metric
6994 moves the field-equation residual, and any causal statement derived from an
6995 ``OFF_SHELL`` perturbation is emitted at ``ClaimScope.SECTOR`` with an
6996 explicit ``exploratory`` flag that the reporting layer prints.
6997
6998 Three admissible perturbation classes are supported:
6999
7000     EXACT_FAMILY      move along a registered exact-solution family by varying
7001                       its parameters: on shell by construction;
7002     CONSTRAINT_SOLVED  a perturbation whose field-equation residual is solved
7003                       or verified below a declared tolerance;
7004     OFF_SHELL         anything else -- exploratory only.
7005 """
7006
7007 from __future__ import annotations
7008
7009 from dataclasses import dataclass, field
7010 from enum import Enum
7011 from typing import Any, Callable, Mapping, Sequence
7012
7013 import numpy as np
7014 import sympy as sp
7015
7016 from .causality import compact_orbit_test
7017 from .model import MetricModel, Provenance
7018 from .status import ClaimScope, CTCStatus, OrbitStatus
7019
7020
7021 class OnShellStatus(str, Enum):
7022     EXACT_FAMILY = "EXACT_FAMILY"
7023     CONSTRAINT_SOLVED = "CONSTRAINT_SOLVED"
7024     OFF_SHELL = "OFF_SHELL"

```

```

7025 UNKNOWN = "UNKNOWN"
7026
7027
7028 @dataclass
7029 class PerturbationResult:
7030     model_id: str
7031     kind: OnShellStatus
7032     epsilon: float
7033     base_margin: float
7034     perturbed_margin: float
7035     residual_before: float
7036     residual_after: float
7037     status_before: str
7038     status_after: str
7039     status_changed: bool
7040     exploratory: bool
7041     note: str = ""
7042
7043 @property
7044 def sensitivity(self) -> float:
7045     if self.epsilon == 0:
7046         return float("nan")
7047     return (self.perturbed_margin - self.base_margin) / self.epsilon
7048
7049 def as_dict(self) -> dict[str, Any]:
7050     return {"model_id": self.model_id, "kind": self.kind.value,
7051             "epsilon": self.epsilon, "base_margin": self.base_margin,
7052             "perturbed_margin": self.perturbed_margin,
7053             "sensitivity": self.sensitivity,
7054             "residual_before": self.residual_before,
7055             "residual_after": self.residual_after,
7056             "status_before": self.status_before,
7057             "status_after": self.status_after,
7058             "status_changed": self.status_changed,
7059             "exploratory": self.exploratory, "note": self.note}
7060
7061 def _residual_norm(model: MetricModel, values: Mapping[str, float]) -> float:
7062     from .energy import symbolic_einstein
7063     try:
7064         G = symbolic_einstein(model)
7065         Lam = model.matter.cosmological_constant
7066         T = model.matter.T
7067         rhs = sp.zeros(model.dim, model.dim) if T is None else 8 * sp.pi * T
7068         res = sp.Matrix(G + Lam * model.metric - rhs)
7069         sub = model.substitution(values)
7070         arr = np.array(res.subs(sub).evalf(30), dtype=float)
7071         return float(np.max(np.abs(arr)))
7072     except Exception:
7073         # noqa: BLE001
7074         return float("nan")
7075
7076
7077 def perturb_metric(model: MetricModel, h: sp.Matrix, epsilon: sp.Symbol | float,
7078                  kind: OnShellStatus = OnShellStatus.OFF_SHELL,
7079                  note: str = "") -> MetricModel:
7080     """Return ``g + epsilon h`` as a new model, tagged with its on-shell status."""
7081     g = model.require_metric()
7082     if h.shape != g.shape:
7083         raise ValueError("perturbation shape mismatch")
7084     if not sp.simplify(h - h.T).is_zero_matrix:
7085         raise ValueError("perturbation tensor must be symmetric")
7086     eps = sp.Symbol("epsilon_pert", real=True) if not isinstance(
7087         epsilon, (int, float, sp.Expr)) else epsilon
7088     new = MetricModel(
7089         id=model.id + "_pert", title=model.title + " + perturbation",
7090         coords=model.coords, metric=sp.Matrix(g + eps * h),
7091         params=tuple(model.params) + ((eps,) if isinstance(eps, sp.Symbol)
7092                                     and eps not in model.params else ()),
7093         signature=model.signature, compact=model.compact,
7094         quotient=model.quotient, kind=model.kind, theory=model.theory,
7095         matter=model.matter, topology=model.topology,
7096         expected=CTCStatus.UNRESOLVED, expected_scope=ClaimScope.SECTOR,
7097         domain=model.domain, default_point=dict(model.default_point),
7098         reference_timelike=model.reference_timelike,
7099         domain_notes=("PERTURBED MODEL. " + (note or "") +
7100                     " On-shell status: " + kind.value),
7101         provenance=Provenance(model.provenance.source_refs,
7102                               model.provenance.convention_note,
7103                               "perturbation of a registry model",
7104                               "derived"),
7105         metadata={**model.metadata, "perturbation_kind": kind.value,
7106                  "exploratory": kind is OnShellStatus.OFF_SHELL})
7107     return new
7108
7109
7110 def parameter_family_perturbation(model: MetricModel, parameter: str,
7111                                  values: Mapping[str, float],
7112                                  delta: float) -> PerturbationResult:
7113     """Move along the registered exact family: on shell by construction."""
7114     base = dict(values)
7115     pert = dict(values)
7116     pert[parameter] = float(values[parameter]) + delta
7117     r0 = compact_orbit_test(model, base)
7118     r1 = compact_orbit_test(model, pert)
7119     return PerturbationResult(
7120         model.id, OnShellStatus.EXACT_FAMILY, delta, r0.margin, r1.margin,
7121         _residual_norm(model, base), _residual_norm(model, pert),
7122         r0.status.value, r1.status.value, r0.status is not r1.status, False,
7123         f"variation of the exact-solution parameter {parameter!r}: the "
7124         f"perturbed configuration is another exact solution of the same "
7125         f"family, so the field equations are satisfied identically and the "
7126         f"result is physical robustness evidence, not an off-shell probe")
7127
7128
7129 def tensor_perturbation(model: MetricModel, h: sp.Matrix, epsilon: float,
7130                        values: Mapping[str, float],
7131                        residual_tol: float = 1e-8) -> PerturbationResult:
7132     """Add an arbitrary symmetric tensor and *measure* whether it stays on shell."""
7133     pert = perturb_metric(model, h, epsilon, OnShellStatus.OFF_SHELL)
7134     base_vals = dict(values)

```

```

7135     pert_vals = dict(values)
7136     r0 = compact_orbit_test(model, base_vals)
7137     try:
7138         r1 = compact_orbit_test(pert, pert_vals)
7139         m1, s1 = r1.margin, r1.status.value
7140     except Exception as exc:
7141         # noqa: BLE001
7142         m1, s1 = float("nan"), f"evaluation failed: {exc}"
7143     res0 = _residual_norm(model, base_vals)
7144     res1 = _residual_norm(pert, pert_vals)
7145     on_shell = (np.isfinite(res1) and abs(res1 - (res0 if np.isfinite(res0) else 0.0))
7146                <= residual_tol)
7147     kind = OnShellStatus.CONSTRAINT_SOLVED if on_shell else OnShellStatus.OFF_SHELL
7148     return PerturbationResult(
7149         model.id, kind, epsilon, r0.margin, m1, res0, res1,
7150         r0.status.value, s1, r0.status.value != s1,
7151         exploratory=kind is OnShellStatus.OFF_SHELL,
7152         note=(f"field-equation residual moved from "
7153              + f"{res0:.3e} to {res1:.3e}; "
7154              + ("within the declared tolerance, so the perturbed metric is "
7155               "treated as constraint-solved"
7156               if on_shell else
7157               "OUTSIDE the declared tolerance, so this perturbation is "
7158               "OFF SHELL and its causal margin is EXPLORATORY ONLY: it "
7159               "cannot be quoted as stability evidence (v1.0 FM-005)"))
7160
7161 def robustness_scan(model: MetricModel, parameter: str,
7162                    values: Mapping[str, float],
7163                    deltas: Sequence[float]) -> list[dict[str, Any]]:
7164     """On-shell robustness of the causal verdict along an exact family."""
7165     rows = []
7166     for d in deltas:
7167         r = parameter_family_perturbation(model, parameter, values, d)
7168         rows.append({"delta": float(d), "margin": r.perturbed_margin,
7169                    "status": r.status_after, "changed": r.status_changed,
7170                    "sensitivity": r.sensitivity})
7171     return rows
7172
7173
7174 __all__ = ["OnShellStatus", "PerturbationResult", "perturb_metric",
7175           "parameter_family_perturbation", "tensor_perturbation",
7176           "robustness_scan"]

```

src/ctcfa/adm.py — 291 lines

```

7177 """
7178 CTC-FA v2.0 -- L12 : ADM / initial-data engine.
7179
7180 Constructibility from regular initial data is the strongest claim level the
7181 architecture can reach short of quantum questions, because it is the only one
7182 that distinguishes "this metric exists as a solution" from "this spacetime
7183 can form". v1.0 shipped ``adm_hamiltonian_residual`` and
7184 ``adm_momentum_residual`` as symbolic expressions and nothing that used them.
7185
7186 v2.0 supplies:
7187
7188 * a **3+1 decomposition** of a registered 4-metric on a declared slicing,
7189   producing lapse, shift, spatial metric and extrinsic curvature;
7190 * the **Hamiltonian and momentum constraints** evaluated on that data with
7191   a reported residual;
7192 * a **conformal York solver** for the Hamiltonian constraint on
7193   maximal, conformally flat data, so that "regular initial data" can be
7194   constructed rather than merely postulated;
7195 * an explicit statement of what is still missing (a full BSSN/Z4 evolution),
7196   with the gate held at ``NOT_ATTEMPTED`` rather than silently passed.
7197
7198 Conventions
7199 -----
7200  $g = -(N^2 - N_i N^i) dt^2 + 2 N_i dt dx^i + h_{ij} dx^i dx^j$ 
7201  $K_{ij} = (1 / 2N) ( D_i N_j + D_j N_i - dh_{ij}/dt )$ 
7202 Hamiltonian:  $R^{(3)} + K^2 - K_{ij} K^{ij} = 16 \pi \rho$ 
7203 Momentum:  $D_j ( K^{ij} - h^{ij} K ) = 8 \pi j^i$ 
7204 """
7205
7206 from __future__ import annotations
7207
7208 from dataclasses import dataclass, field
7209 from typing import Any, Mapping, Sequence
7210
7211 import numpy as np
7212 import sympy as sp
7213
7214 from .model import MetricModel
7215 from .symbolic import christoffel, ricci_scalar, ricci_tensor
7216
7217
7218 @dataclass
7219 class ADMData:
7220     lapse: sp.Expr
7221     shift: sp.Matrix
7222     spatial_metric: sp.Matrix
7223     extrinsic_curvature: sp.Matrix
7224     spatial_coords: tuple[sp.Symbol, ...]
7225     time_coord: sp.Symbol
7226     note: str = ""
7227
7228     def as_dict(self) -> dict[str, Any]:
7229         return {"lapse": str(self.lapse),
7230                "shift": [str(s) for s in self.shift],
7231                "spatial_metric": [[str(self.spatial_metric[i, j])
7232                                   for j in range(self.spatial_metric.rows)]
7233                                   for i in range(self.spatial_metric.rows)],
7234                "extrinsic_curvature": [[str(self.extrinsic_curvature[i, j])
7235                                         for j in range(self.extrinsic_curvature.rows)]
7236                                         for i in range(self.extrinsic_curvature.rows)],
7237                "note": self.note}
7238
7239

```

```

7240 def decompose(model: MetricModel, time_index: int = 0) -> ADMData:
7241     """3+1 decomposition of a registered metric on a constant-x^0 slicing."""
7242     g = model.require_metric()
7243     n = model.dim
7244     t = model.coords[time_index]
7245     idx = [i for i in range(n) if i != time_index]
7246     scoords = tuple(model.coords[i] for i in idx)
7247
7248     h = g.extract(idx, idx)
7249     if sp.simplify(h.det()) == 0:
7250         raise ValueError(
7251             f"the constant-{model.coords[time_index]} slices of this chart are "
7252             f"degenerate (det h = 0), so no 3+1 decomposition exists on this "
7253             f"slicing. Null-adapted charts such as the Ori-2007 core "
7254             f"(g_vv, g_vr with g_rr = 0) require a different foliation before "
7255             f"the ADM constraints are even defined.")
7256     hinv = h.inv()
7257     Ni_down = sp.Matrix([g[time_index, i] for i in idx]) # N_i
7258     Ni_up = sp.simplify(hinv * Ni_down) # N^i
7259     N2 = sp.simplify(-(g[time_index, time_index] - (Ni_down.T * Ni_up)[0]))
7260     N = sp.sqrt(sp.simplify(N2))
7261
7262     m = len(sidx)
7263     Gam3 = christoffel(h, scoords, simplify=True)
7264
7265     def D(i: int, j: int, vec_down: sp.Matrix) -> sp.Expr:
7266         expr = sp.diff(vec_down[j], scoords[i])
7267         for k in range(m):
7268             expr -= Gam3[k][i][j] * vec_down[k]
7269         return sp.simplify(expr)
7270
7271     K = sp.zeros(m, m)
7272     for i in range(m):
7273         for j in range(i, m):
7274             val = sp.simplify((D(i, j, Ni_down) + D(j, i, Ni_down)
7275                             - sp.diff(h[i, j], t)) / (2 * N))
7276             K[i, j] = val
7277             K[j, i] = val
7278     return ADMData(sp.simplify(N), Ni_up, h, K, scoords, t,
7279                  "constant-x^0 slicing of the registered chart")
7280
7281 @dataclass
7282 class ConstraintResidual:
7283     hamiltonian: sp.Expr
7284     momentum: list[sp.Expr]
7285     hamiltonian_value: float | None = None
7286     momentum_values: list[float] = field(default_factory=list)
7287     tolerance: float = 1e-9
7288     satisfied: bool | None = None
7289     note: str = ""
7290
7291     def as_dict(self) -> dict[str, Any]:
7292         return {"hamiltonian": str(self.hamiltonian),
7293               "momentum": [str(m) for m in self.momentum],
7294               "hamiltonian_value": self.hamiltonian_value,
7295               "momentum_values": self.momentum_values,
7296               "tolerance": self.tolerance, "satisfied": self.satisfied,
7297               "note": self.note}
7298
7299
7300 def constraints(data: ADMData, rho: sp.Expr = sp.S.Zero,
7301               j_up: Sequence[sp.Expr] | None = None,
7302               simplify: bool = True) -> ConstraintResidual:
7303     """Hamiltonian and momentum constraint residuals for the given data."""
7304     h, K = data.spatial_metric, data.extrinsic_curvature
7305     coords = data.spatial_coords
7306     m = h.rows
7307     hinv = h.inv()
7308     R3 = ricci_scalar(h, coords, simplify=simplify)
7309     Kmixed = hinv * K
7310     trK = sp.simplify(sum(Kmixed[i, i] for i in range(m)))
7311     KijKij = sp.simplify(sum(Kmixed[i, j] * Kmixed[j, i]
7312                             for i in range(m) for j in range(m)))
7313     ham = sp.simplify(R3 + trK ** 2 - KijKij - 16 * sp.pi * rho)
7314
7315     Gam3 = christoffel(h, coords, simplify=simplify)
7316     Kup = sp.simplify(hinv * K * hinv)
7317     A = sp.Matrix(m, m, lambda i, j: sp.simplify(Kup[i, j] - hinv[i, j] * trK))
7318     j_up = list(j_up) if j_up is not None else [sp.S.Zero] * m
7319     mom = []
7320     for i in range(m):
7321         expr = sp.S.Zero
7322         for j in range(m):
7323             expr += sp.diff(A[i, j], coords[j])
7324             for k in range(m):
7325                 expr += Gam3[i][j][k] * A[k, j] + Gam3[j][j][k] * A[i, k]
7326             mom.append(sp.simplify(expr - 8 * sp.pi * j_up[i]))
7327     return ConstraintResidual(ham, mom,
7328                             note="ADM constraints in geometric units G=c=1")
7329
7330
7331 def evaluate_constraints(model: MetricModel, values: Mapping[str, float],
7332                       time_index: int = 0, rho: sp.Expr = sp.S.Zero,
7333                       tolerance: float = 1e-9) -> ConstraintResidual:
7334     data = decompose(model, time_index)
7335     res = constraints(data, rho)
7336     sub = model.substitution(values)
7337     try:
7338         hv = float(sp.N(res.hamiltonian.subs(sub), 30))
7339     except Exception:
7340         hv = None # noqa: BLE001
7341
7342     mv = []
7343     for mexpr in res.momentum:
7344         try:
7345             mv.append(float(sp.N(mexpr.subs(sub), 30)))
7346         except Exception:
7347             mv.append(float("nan")) # noqa: BLE001
7348     res.hamiltonian_value = hv
7349     res.momentum_values = mv

```

```

7350     res.tolerance = tolerance
7351     ok = (hv is not None and abs(hv) <= tolerance
7352           and all(np.isfinite(v) and abs(v) <= tolerance for v in mv))
7353     res.satisfied = ok
7354     return res
7355
7356 # -----
7357 # Conformal (York) construction of regular initial data
7358 # -----
7359
7360 @dataclass
7361 class YorkSolution:
7362     psi: np.ndarray
7363     grid: np.ndarray
7364     residual: float
7365     iterations: int
7366     converged: bool
7367     note: str = ""
7368
7369     def as_dict(self) -> dict[str, Any]:
7370         return {"n_grid": int(self.grid.size), "residual": self.residual,
7371               "iterations": self.iterations, "converged": self.converged,
7372               "psi_min": float(np.min(self.psi)),
7373               "psi_max": float(np.max(self.psi)), "note": self.note}
7374
7375 def york_hamiltonian_ld(rho_hat: np.ndarray, r: np.ndarray,
7376                        psi0: np.ndarray | None = None,
7377                        tol: float = 1e-12, maxiter: int = 200) -> YorkSolution:
7378     """Solve the spherically symmetric conformal Hamiltonian constraint.
7379
7380     On maximal ('K = 0'), conformally flat data the constraint reduces to
7381
7382         Laplacian(psi) + 2 pi rho_hat psi^{-3} = 0,
7383
7384     a semilinear elliptic equation for the conformal factor. Solved here by
7385     Newton iteration on a second-order radial finite-difference stencil with
7386     a regular centre and 'psi -> 1' at the outer boundary, which is exactly
7387     the "construct regular initial data" step the L12 gate requires and which
7388     v1.0 left unimplemented.
7389     """
7390     r = np.asarray(r, dtype=float)
7391     n = r.size
7392     dr = float(r[1] - r[0])
7393     psi = np.ones(n) if psi0 is None else np.asarray(psi0, dtype=float).copy()
7394
7395     def residual(p: np.ndarray) -> np.ndarray:
7396         F = np.zeros(n)
7397         F[0] = p[1] - p[0] # regular centre
7398         F[-1] = p[-1] - 1.0 # asymptotic flatness
7399         for i in range(1, n - 1):
7400             lap = ((p[i + 1] - 2 * p[i] + p[i - 1]) / dr ** 2
7401                   + (2.0 / r[i]) * (p[i + 1] - p[i - 1]) / (2 * dr))
7402             F[i] = lap + 2 * np.pi * rho_hat[i] * p[i] ** (-3)
7403         return F
7404
7405     it = 0
7406     for it in range(1, maxiter + 1):
7407         F = residual(psi)
7408         if float(np.max(np.abs(F))) < tol:
7409             break
7410         J = np.zeros((n, n))
7411         J[0, 0], J[0, 1] = -1.0, 1.0
7412         J[-1, -1] = 1.0
7413         for i in range(1, n - 1):
7414             J[i, i - 1] = 1.0 / dr ** 2 - (1.0 / r[i]) / dr
7415             J[i, i] = -2.0 / dr ** 2 - 6 * np.pi * rho_hat[i] * psi[i] ** (-4)
7416             J[i, i + 1] = 1.0 / dr ** 2 + (1.0 / r[i]) / dr
7417         try:
7418             dpsi = np.linalg.solve(J, -F)
7419         except np.linalg.LinAlgError:
7420             return YorkSolution(psi, r, float(np.max(np.abs(F))), it, False,
7421                               "Newton Jacobian is singular")
7422         psi = psi + dpsi
7423         if np.any(psi <= 0):
7424             return YorkSolution(psi, r, float("nan"), it, False,
7425                               "conformal factor went non-positive")
7426
7427     F = residual(psi)
7428     res = float(np.max(np.abs(F)))
7429     return YorkSolution(psi, r, res, it, res < 1e-8,
7430                       "maximal, conformally flat spherically symmetric data")
7431
7432 def constructibility_report(model: MetricModel,
7433                            values: Mapping[str, float] | None = None) -> dict[str, Any]:
7434     """L12 report with the evolution gate honestly left open."""
7435     values = dict(values or model.default_point)
7436     out: dict[str, Any] = {"model_id": model.id}
7437     if not model.has_metric:
7438         out["status"] = "NOT_APPLICABLE"
7439         out["note"] = "no local metric; constructibility is an external claim"
7440         return out
7441     try:
7442         res = evaluate_constraints(model, values)
7443         out["constraints"] = res.as_dict()
7444         out["status"] = "CONSTRAINTS_EVALUATED"
7445     except Exception as exc:
7446         out["status"] = "CONSTRAINTS_FAILED" # noqa: BLE001
7447         out["error"] = str(exc)
7448     out["evolution"] = {
7449         "status": "NOT_ATTEMPTED",
7450         "requirement": ("a convergent 3+1 evolution (BSSN or Z4-family) with "
7451                       "resolution, gauge and formulation convergence tests, "
7452                       "constraint-violation monitoring and an explicit CTC "
7453                       "witness reconstructed on the converged geometry"),
7454         "note": ("CTC-FA v2.0 does not include a production numerical-relativity "
7455                "evolution. The correct path is integration with the Einstein "
7456                "Toolkit; until that exists no model may be reported above ")
7457     }

```

```

7460         "claim level L5. A visualisation of tilted light cones is "
7461         "explicitly NOT sufficient evidence.")]
7462     return out
7463
7464
7465 __all__ = ["ADMData", "decompose", "ConstraintResidual", "constraints",
7466           "evaluate_constraints", "YorkSolution", "york_hamiltonian_ld",
7467           "constructibility_report"]

```

src/ctcfa/qft.py — 323 lines

```

7468 """
7469 CTC-FA v2.0 -- L13/L14 : quantum-state layer and semiclassical backreaction.
7470
7471 The rule this layer exists to enforce
7472 -----
7473     No generic renormalised stress tensor is inferred from a metric alone.
7474
7475 ``<T_mu_nu>_ren`` is a functional of the quantum field content, the state, the
7476 boundary conditions and the renormalisation prescription. A metric does not
7477 determine it. Every "quantum" statement in the architecture therefore
7478 requires a :class:`QuantumSpecification` and refuses to produce a number
7479 without one. v1.0 stated this (L13 gate, REQ-011) and modelled it as a
7480 dataclass of four ``UNRESOLVED`` strings; v2.0 makes it a real gate with
7481 real inputs and a real applicability decision.
7482
7483 What is actually computable here
7484 -----
7485 * **Hadamard admissibility** on a compactly generated Cauchy horizon. The
7486   Kay-Radzikowski-Wald theorem shows the two-point function cannot be
7487   Hadamard on the whole horizon: there are base points at which the standard
7488   construction fails and ``<T_mu_nu>_ren`` is ill defined or singular. The
7489   engine checks the theorem's hypotheses and, when they hold, returns
7490   ``SEMICLASSICAL_QFT_BREAKDOWN`` at the identified base points -- not an
7491   invented infinity.
7492
7493 * **Polarised hypersurfaces** of a flat quotient (from ``quotient_ctc``) are
7494   exactly the loci where the image-sum two-point function of a free field
7495   diverges, so they are reported here as the *computed* candidate locations
7496   of RSET divergence. This is the one place where v2.0 can give a genuinely
7497   quantitative semiclassical statement, and it can do so only because the
7498   deck group is an executable object.
7499
7500 * **Quantum energy inequalities** are evaluated only against a registered
7501   rule keyed by field, dimension, state class, sampling function and
7502   trajectory. An unkeyed QEI is refused (v1.0 FM-012).
7503 """
7504
7505 from __future__ import annotations
7506
7507 from dataclasses import dataclass, field
7508 from enum import Enum
7509 from typing import Any, Callable, Mapping, Sequence
7510
7511 import numpy as np
7512 import sympy as sp
7513
7514 from .model import MetricModel
7515
7516
7517 class QuantumStatus(str, Enum):
7518     UNRESOLVED = "UNRESOLVED"
7519     NOT_APPLICABLE = "NOT_APPLICABLE"
7520     ADMISSIBLE = "ADMISSIBLE"
7521     SEMICLASSICAL_QFT_BREAKDOWN = "SEMICLASSICAL_QFT_BREAKDOWN"
7522     RSET_DIVERGENT = "RSET_DIVERGENT"
7523
7524
7525 @dataclass
7526 class QuantumSpecification:
7527     """Everything that must be declared before any quantum statement."""
7528
7529     field_content: str = ""
7530     spin: str = ""
7531     mass: float | None = None
7532     couplings: Mapping[str, float] = field(default_factory=dict)
7533     state: str = ""
7534     state_class: str = ""
7535     boundary_conditions: str = ""
7536     renormalisation: str = ""
7537     sampling_function: str = ""
7538     trajectory: str = ""
7539     spacetime_dimension: int = 4
7540
7541     @property
7542     def complete(self) -> bool:
7543         return bool(self.field_content and self.state and self.renormalisation)
7544
7545     def missing(self) -> list[str]:
7546         out = []
7547         for k in ("field_content", "state", "renormalisation"):
7548             if not getattr(self, k):
7549                 out.append(k)
7550         return out
7551
7552     def as_dict(self) -> dict[str, Any]:
7553         return {"field_content": self.field_content, "spin": self.spin,
7554               "mass": self.mass, "couplings": dict(self.couplings),
7555               "state": self.state, "state_class": self.state_class,
7556               "boundary_conditions": self.boundary_conditions,
7557               "renormalisation": self.renormalisation,
7558               "sampling_function": self.sampling_function,
7559               "trajectory": self.trajectory,
7560               "spacetime_dimension": self.spacetime_dimension,
7561               "complete": self.complete, "missing": self.missing()}
7562
7563
7564 @dataclass

```



```

7565 class QuantumAssessment:
7566     model_id: str
7567     status: QuantumStatus
7568     hadamard: str
7569     qei: str
7570     renormalised_stress: str
7571     backreaction: str
7572     theorem_hypotheses: dict[str, Any] = field(default_factory=dict)
7573     divergence_loci: list[dict[str, Any]] = field(default_factory=list)
7574     spec: QuantumSpecification | None = None
7575     note: str = ""
7576
7577     def as_dict(self) -> dict[str, Any]:
7578         return {"model_id": self.model_id, "status": self.status.value,
7579                 "hadamard": self.hadamard, "qei": self.qei,
7580                 "renormalised_stress": self.renormalised_stress,
7581                 "backreaction": self.backreaction,
7582                 "theorem_hypotheses": self.theorem_hypotheses,
7583                 "divergence_loci": self.divergence_loci,
7584                 "specification": self.spec.as_dict() if self.spec else None,
7585                 "note": self.note}
7586
7587 # -----
7588 # Kay-Radzikowski-Wald gate
7589 # -----
7590
7591 def krw_hypotheses(model: MetricModel) -> dict[str, Any]:
7592     """Record whether the KRW hypotheses are satisfied by this model.
7593
7594     KRW applies to a spacetime with a **compactly generated** Cauchy horizon.
7595     The engine reports each hypothesis as satisfied / not satisfied /
7596     undetermined, and the theorem may only be invoked when none is 'not
7597     satisfied'.
7598     """
7599     meta = model.metadata
7600     has_horizon = bool(meta.get("chronology_horizon"))
7601     compactly_generated = meta.get("compactly_generated_horizon")
7602     if compactly_generated is None:
7603         # Misner-type quotients by a single boost have a compactly generated
7604         # chronology horizon; a periodic-time identification of a static
7605         # spacetime does not (the whole spacetime is chronology violating and
7606         # there is no horizon at all).
7607         if model.id in ("misner", "misner_covering", "grant_space"):
7608             compactly_generated = True
7609         elif has_horizon:
7610             compactly_generated = None
7611         else:
7612             compactly_generated = False
7613     return {
7614         "chronology_horizon_present": has_horizon,
7615         "compactly_generated": compactly_generated,
7616         "free_scalar_field_assumed": True,
7617         "four_dimensional": model.dim == 4,
7618         "applicable": bool(has_horizon and compactly_generated is True
7619                             and model.dim == 4),
7620         "note": ("KRW proves that the two-point function of a free scalar "
7621                 "field cannot be Hadamard on the whole of a compactly "
7622                 "generated Cauchy horizon; at the base points of the horizon "
7623                 "generators the standard construction fails and the "
7624                 "renormalised stress tensor is ill defined or singular. It "
7625                 "does NOT prove chronology protection in general."),
7626     }
7627
7628
7629 def assess(model: MetricModel, spec: QuantumSpecification | None = None,
7630           polarized: Sequence[Mapping[str, Any]] | None = None
7631           ) -> QuantumAssessment:
7632     """L13/L14 assessment. Refuses to produce numbers without a specification."""
7633     hyp = krw_hypotheses(model)
7634     if spec is None or not spec.complete:
7635         missing = spec.missing() if spec else ["field_content", "state",
7636                                                 "renormalisation"]
7637         return QuantumAssessment(
7638             model.id, QuantumStatus.UNRESOLVED, "UNRESOLVED", "UNRESOLVED",
7639             "UNRESOLVED", "UNRESOLVED", hyp, [], spec,
7640             note=("no quantum statement is possible: the specification is "
7641                 f"incomplete (missing {missing}). A renormalised stress "
7642                 "tensor is a functional of the field content, the state, the "
7643                 "boundary conditions and the renormalisation prescription, "
7644                 "and cannot be inferred from the metric alone."))
7645     loci = []
7646     if polarized:
7647         for row in polarized:
7648             loci.append({"winding": row.get("winding"),
7649                         "variable": row.get("variable"),
7650                         "value": row.get("value"),
7651                         "mechanism": "polarised hypersurface of the deck group: "
7652                                     "the image sum for the two-point function "
7653                                     "of a free field diverges here"})
7654     if hyp["applicable"]:
7655         status = QuantumStatus.SEMICLASSICAL_QFT_BREAKDOWN
7656         had = ("NOT HADAMARD on the whole horizon (KRW): base points exist at "
7657              "which the standard construction fails")
7658         rset = ("ill defined or singular at the KRW base points; no finite "
7659              "value is invented here")
7660         back = ("classical CTC survival does not imply semiclassical survival; "
7661              "the correct output is a breakdown label, not a backreaction "
7662              "number")
7663     elif loci:
7664         status = QuantumStatus.RSET_DIVERGENT
7665         had = "state-dependent; polarised hypersurfaces located below"
7666         rset = ("divergent on the listed polarised hypersurfaces for the "
7667              "declared state class")
7668         back = "backreaction expected to be strong near those loci"
7669     else:
7670         status = QuantumStatus.UNRESOLVED
7671         had = "undetermined for the declared state"

```



```

7675         rset = "not computed"
7676         back = "not computed"
7677
7678         return QuantumAssessment(model.id, status, had, "requires a registered QEI rule",
7679                                   rset, back, hyp, loci, spec,
7680                                   note="assessment is valid only for the declared "
7681                                       "specification; changing the state or the "
7682                                       "renormalisation prescription invalidates it")
7683
7684
7685 # -----
7686 # Quantum energy inequalities
7687 # -----
7688
7689 @dataclass
7690 class QEIRule:
7691     """A registered quantum energy inequality, keyed by its assumptions."""
7692
7693     key: str
7694     field_content: str
7695     dimension: int
7696     state_class: str
7697     sampling: str
7698     trajectory: str
7699     bound: Callable[[Mapping[str, float]], float]
7700     reference: str
7701     note: str = ""
7702
7703     def applies_to(self, spec: QuantumSpecification) -> bool:
7704         return (spec.field_content == self.field_content
7705                 and spec.spacetime_dimension == self.dimension
7706                 and spec.state_class == self.state_class
7707                 and spec.sampling_function == self.sampling
7708                 and spec.trajectory == self.trajectory)
7709
7710 def _minkowski_massless_scalar_bound(p: Mapping[str, float]) -> float:
7711     """Ford-Roman-type bound for a massless scalar in 4D Minkowski.
7712
7713     For a static inertial observer with a Lorentzian sampling function of
7714     width ``tau``,
7715
7716          $\int \rho(t) f(t) dt \geq -3 / (32 \pi^2 \tau^4).$ 
7717
7718     Registered with its exact assumptions so that it can never be applied to
7719     a different field, dimension, state class, sampling function or
7720     trajectory.
7721     """
7722     tau = float(p["tau"])
7723     if tau <= 0:
7724         raise ValueError("sampling width must be positive")
7725     return -3.0 / (32.0 * np.pi ** 2 * tau ** 4)
7726
7727
7728 QEI_REGISTRY: dict[str, QEIRule] = {
7729     "FR_massless_scalar_4d_inertial_lorentzian": QEIRule(
7730         key="FR_massless_scalar_4d_inertial_lorentzian",
7731         field_content="massless minimally coupled scalar",
7732         dimension=4, state_class="Hadamard",
7733         sampling="Lorentzian", trajectory="static inertial",
7734         bound=_minkowski_massless_scalar_bound,
7735         reference="Ford-Roman quantum inequality in 4D Minkowski",
7736         note="valid only in flat spacetime for the stated observer and "
7737             "sampling function; it is NOT a general bound on negative energy "
7738             "in curved spacetime"),
7739 }
7740
7741
7742 @dataclass
7743 class QEIResult:
7744     rule: str | None
7745     bound: float | None
7746     applicable: bool
7747     reason: str
7748
7749     def as_dict(self) -> dict[str, Any]:
7750         return {"rule": self.rule, "bound": self.bound,
7751                 "applicable": self.applicable, "reason": self.reason}
7752
7753
7754 def evaluate_qei(spec: QuantumSpecification,
7755                  parameters: Mapping[str, float]) -> QEIResult:
7756     """Apply a registered QEI, or refuse."""
7757     for rule in QEI_REGISTRY.values():
7758         if rule.applies_to(spec):
7759             try:
7760                 return QEIResult(rule.key, float(rule.bound(parameters)), True,
7761                                   f"{rule.reference}. {rule.note}")
7762             except Exception as exc:
7763                 # noqa: BLE001
7764                 return QEIResult(rule.key, None, False,
7765                                   f"rule matched but evaluation failed: {exc}")
7766     return QEIResult(None, None, False,
7767                       "no registered quantum energy inequality matches this "
7768                       "field content, dimension, state class, sampling function "
7769                       "and trajectory. Applying a QEI outside its assumptions "
7770                       "produces an invalid bound (v1.0 FM-012), so none is "
7771                       "applied.")
7772
7773
7774 def semiclassical_residual(model: MetricModel, T_classical: sp.Matrix,
7775                            T_ren: sp.Matrix | None,
7776                            Lambda: sp.Expr = sp.S.Zero) -> sp.Matrix | str:
7777     """G + Lambda g - 8 pi (T_classical + <T>_ren), or a refusal."""
7778     if T_ren is None:
7779         return ("refused: a semiclassical residual requires an explicitly "
7780                "supplied <T_mu_nu>_ren from a valid state-specific "
7781                "calculation. None is inferred from the metric.")
7782     from .energy import symbolic_einstein
7783     g = model.require_metric()
7784     G = symbolic_einstein(model)

```

```

7785     return sp.simplify(G + Lambda * g - 8 * sp.pi * (T_classical + T_ren))
7786
7787
7788     __all__ = ["QuantumStatus", "QuantumSpecification", "QuantumAssessment",
7789               "krw_hypotheses", "assess", "QEIRule", "QEI_REGISTRY", "QEIResult",
7790               "evaluate_qei", "semiclassical_residual"]

```

src/ctcfa/theorems.py — 348 lines

```

7791 """
7792 CTC-FA v2.0 -- L15 : theorem and no-go applicability validator.
7793
7794 A theorem is a conditional. Quoting its conclusion without checking its
7795 hypotheses is the most common and most damaging failure in this literature,
7796 and v1.0's own register lists three instances of it (FM-010 chronology
7797 protection cited as a universal theorem, FM-011 KRW generalised outside
7798 compactly generated horizons, FM-012 QEIs applied without their assumptions).
7799 v1.0 described the mitigation as a "hypothesis checklist" but shipped no
7800 checklist.
7801
7802 This module is that checklist, as executable objects. Each theorem carries
7803
7804 * its hypotheses, each with a *checker* that returns
7805   ``True`` / ``False`` / ``None`` (undetermined) for a given model,
7806 * the conclusion it licenses **and its exact label**,
7807 * the specific overreach it must never be used for.
7808
7809 ``apply(theorem, model)`` returns ``LICENSED`` only when every hypothesis is
7810 verified. A single ``None`` yields ``HYPOTHESES_UNVERIFIED``; a single
7811 ``False`` yields ``NOT_APPLICABLE``. There is no path that emits a
7812 conclusion from unverified hypotheses.
7813 """
7814
7815 from __future__ import annotations
7816
7817 from dataclasses import dataclass, field
7818 from enum import Enum
7819 from typing import Any, Callable, Mapping
7820
7821 import sympy as sp
7822
7823 from .model import MetricModel
7824 from .status import ModelKind
7825
7826 class Applicability(str, Enum):
7827     LICENSED = "LICENSED"
7828     HYPOTHESES_UNVERIFIED = "HYPOTHESES_UNVERIFIED"
7829     NOT_APPLICABLE = "NOT_APPLICABLE"
7830
7831 @dataclass
7832 class Hypothesis:
7833     name: str
7834     statement: str
7835     check: Callable[[MetricModel], bool | None]
7836
7837     def evaluate(self, model: MetricModel) -> bool | None:
7838         try:
7839             return self.check(model)
7840         except Exception:
7841             # noqa: BLE001
7842             return None
7843
7844 @dataclass
7845 class Theorem:
7846     key: str
7847     title: str
7848     reference: str
7849     hypotheses: list[Hypothesis]
7850     conclusion: str
7851     permitted_label: str
7852     forbidden_use: str
7853     status_note: str = ""
7854
7855     def as_dict(self) -> dict[str, Any]:
7856         return {"key": self.key, "title": self.title, "reference": self.reference,
7857               "hypotheses": [{"name": h.name, "statement": h.statement}
7858                               for h in self.hypotheses],
7859               "conclusion": self.conclusion,
7860               "permitted_label": self.permitted_label,
7861               "forbidden_use": self.forbidden_use,
7862               "status_note": self.status_note}
7863
7864 @dataclass
7865 class TheoremReport:
7866     theorem: Theorem
7867     applicability: Applicability
7868     hypothesis_results: dict[str, bool | None]
7869     permitted_label: str | None
7870     note: str
7871
7872     def as_dict(self) -> dict[str, Any]:
7873         return {"theorem": self.theorem,
7874               "applicability": self.applicability.value,
7875               "hypothesis_results": self.hypothesis_results,
7876               "permitted_label": self.permitted_label, "note": self.note}
7877
7878 # -----
7879 # Hypothesis checkers
7880 # -----
7881
7882 def _is_vacuum(m: MetricModel) -> bool | None:
7883     if m.matter.vacuum:
7884         return True
7885     if m.matter.label in ("unspecified", ""):

```

```

7890         return None
7891     return False
7892
7893
7894 def _asymptotically_flat(m: MetricModel) -> bool | None:
7895     lam = m.matter.cosmological_constant
7896     if lam != 0:
7897         return False
7898     hint = (m.topology + " " + m.domain_notes).lower()
7899     if "asymptotically flat" in hint:
7900         return True
7901     if m.id in ("minkowski", "schwarzschild", "reissner_nordstrom", "kerr",
7902                "kerr_newman", "morris_thorne"):
7903         return True
7904     if m.id in ("godel", "godel_type", "godel_critical", "van_stockum",
7905                "misner", "misner_covering", "grant_space",
7906                "spinning_cosmic_string", "ori_2005", "ori_2007_core",
7907                "minkowski_time_periodic", "btz_rotating"):
7908         return False
7909     return None
7910
7911
7912 def _globally_hyperbolic(m: MetricModel) -> bool | None:
7913     if m.temporal_function is not None and not m.quotient.generators \
7914        and not any(c.index == 0 for c in m.compact):
7915         return None # stably causal is necessary but not sufficient
7916     if m.expected.value == "CTC_PROVED":
7917         return False
7918     return None
7919
7920
7921 def _nec_satisfied(m: MetricModel) -> bool | None:
7922     if m.matter.vacuum:
7923         return True
7924     role = m.metadata.get("role", "")
7925     if role == "energy_condition_counterexample":
7926         return True
7927     if m.id in ("morris_thorne", "alcubierre", "krasnikov"):
7928         return False
7929     return None
7930
7931
7932 def _has_chronology_horizon(m: MetricModel) -> bool | None:
7933     return bool(m.metadata.get("chronology_horizon")) or None
7934
7935
7936 def _compactly_generated_horizon(m: MetricModel) -> bool | None:
7937     if m.id in ("misner", "misner_covering", "grant_space"):
7938         return True
7939     if not m.metadata.get("chronology_horizon"):
7940         return False
7941     return None
7942
7943
7944 def _four_dimensional(m: MetricModel) -> bool | None:
7945     return m.dim == 4
7946
7947
7948 def _has_temporal_function(m: MetricModel) -> bool | None:
7949     return m.temporal_function is not None
7950
7951
7952 def _traversable_wormhole(m: MetricModel) -> bool | None:
7953     return "wormhole" in (m.title + m.topology).lower()
7954
7955
7956 # -----
7957 # Registry of theorems
7958 # -----
7959
7960 THEOREMS: dict[str, Theorem] = {
7961     "stable_causality": Theorem(
7962         key="stable_causality",
7963         title="Stable causality from a temporal function",
7964         reference="Minguzzi & Sanchez, The causal hierarchy of spacetimes "
7965                 "(arXiv:gr-qc/0609119)",
7966         hypotheses=[
7967             Hypothesis("temporal_function",
7968                       "a smooth function tau exists with everywhere past-directed "
7969                       "timelike gradient on the FULL declared domain",
7970                       _has_temporal_function),
7971         ],
7972         conclusion="the spacetime is stably causal and therefore contains no "
7973                  "closed timelike curves on that domain",
7974         permitted_label="NO_CTC_CERTIFIED (in the certified domain)",
7975         forbidden_use="a temporal function verified only on a coordinate patch, "
7976                     "or a function that is not single valued on a quotient, "
7977                     "licenses nothing",
7978         status_note="THEOREM"),
7979     "topological_censorship": Theorem(
7980         key="topological_censorship",
7981         title="Topological censorship",
7982         reference="Friedman, Schleich & Witt, Phys. Rev. Lett. 71, 1486 (1993)",
7983         hypotheses=[
7984             Hypothesis("globally_hyperbolic",
7985                       "the spacetime is globally hyperbolic",
7986                       _globally_hyperbolic),
7987             Hypothesis("asymptotically_flat",
7988                       "the spacetime is asymptotically flat",
7989                       _asymptotically_flat),
7990             Hypothesis("null_energy_condition",
7991                       "the averaged null energy condition holds on the relevant "
7992                       "causal curves", _nec_satisfied),
7993         ],
7994         conclusion="every causal curve between two points of the asymptotic "
7995                  "region is homotopic to a curve in that region: a traverser "
7996                  "cannot probe nontrivial topology",
7997         permitted_label="TOPOLOGY_CONSTRAINED",
7998         forbidden_use="it is not a chronology-protection theorem and says "

```

```

8000         "nothing about spacetimes that are not globally hyperbolic, "
8001         "which is precisely the class every CTC spacetime is in",
8002         status_note="THEOREM"),
8003
8004     "chronology_protection": Theorem(
8005         key="chronology_protection",
8006         title="Chronology protection conjecture",
8007         reference="Hawking, Phys. Rev. D 46, 603 (1992)",
8008         hypotheses=[
8009             Hypothesis("chronology_horizon",
8010                 "a chronology horizon forms", _has_chronology_horizon),
8011             Hypothesis("compactly_generated",
8012                 "the chronology horizon is compactly generated",
8013                 _compactly_generated_horizon),
8014             Hypothesis("semiclassical_reasoning_valid",
8015                 "the semiclassical approximation remains valid up to the "
8016                 "horizon", lambda m: None),
8017         ],
8018         conclusion="classical and semiclassical effects are argued to obstruct "
8019         "the formation of the chronology horizon",
8020         permitted_label="CHRONOLOGY_PROTECTION_SUPPORTED_IN_MODEL",
8021         forbidden_use="this is a CONJECTURE, not a theorem.",
8022         "CHRONOLOGY_PROTECTION_PROVED is not an emittable label "
8023         "anywhere in this architecture",
8024         status_note="CONJECTURE"),
8025
8026     "kay_radzikowski_wald": Theorem(
8027         key="kay_radzikowski_wald",
8028         title="QFT on spacetimes with a compactly generated Cauchy horizon",
8029         reference="Kay, Radzikowski & Wald, Commun. Math. Phys. 183, 533 (1997)",
8030         hypotheses=[
8031             Hypothesis("compactly_generated",
8032                 "the Cauchy horizon is compactly generated",
8033                 _compactly_generated_horizon),
8034             Hypothesis("four_dimensional", "spacetime dimension is four",
8035                 _four_dimensional),
8036             Hypothesis("free_field",
8037                 "a free scalar field with the standard Hadamard "
8038                 "construction", lambda m: None),
8039         ],
8040         conclusion="the two-point function cannot be Hadamard on the whole "
8041         "horizon; at base points of the horizon generators the "
8042         "renormalised stress tensor is ill defined or singular",
8043         permitted_label="SEMICALSSICAL_QFT_BREAKDOWN (at the base points)",
8044         forbidden_use="it must not be generalised to horizons that are not "
8045         "compactly generated, and it is not a proof that "
8046         "backreaction destroys every CTC",
8047         status_note="THEOREM"),
8048
8049     "morris_thorne_energy": Theorem(
8050         key="morris_thorne_energy",
8051         title="Energy-condition violation at a traversable wormhole throat",
8052         reference="Morris & Thorne, Am. J. Phys. 56, 395 (1988)",
8053         hypotheses=[
8054             Hypothesis("traversable_wormhole",
8055                 "a static traversable wormhole ansatz in general "
8056                 "relativity", _traversable_wormhole),
8057             Hypothesis("flare_out",
8058                 "the flare-out condition holds at the throat",
8059                 lambda m: bool(m.metadata.get("flare_out")) or None),
8060         ],
8061         conclusion="the null energy condition is violated near the throat",
8062         permitted_label="ENERGY_CONDITION_VIOLATION (in the applicable throat "
8063         "model)",
8064         forbidden_use="it does not imply that the wormhole is a time machine, "
8065         "and it does not apply to quantum-induced traversability",
8066         status_note="THEOREM"),
8067
8068     "gao_jafferis_wall": Theorem(
8069         key="gao_jafferis_wall",
8070         title="Traversable wormholes via a double-trace deformation",
8071         reference="Gao, Jafferis & Wall, JHEP 2017:151",
8072         hypotheses=[
8073             Hypothesis("ads_btz_setup",
8074                 "a specific AdS/BTZ holographic construction with the "
8075                 "stated boundary coupling", lambda m: None),
8076         ],
8077         conclusion="a quantum interaction can render an Einstein-Rosen bridge "
8078         "traversable without violating causality",
8079         permitted_label="TRAVERSABLE_IN_MODEL (explicitly NOT a "
8080         "causality-violation certificate)",
8081         forbidden_use="quantum traversability is not chronology violation; this "
8082         "result is a counterexample to that inference, not "
8083         "support for it",
8084         status_note="THEOREM"),
8085
8086     "ori_counterexample": Theorem(
8087         key="ori_counterexample",
8088         title="CTC formation with non-exotic matter",
8089         reference="Ori, Phys. Rev. Lett. 95, 021101 (2005); "
8090         "Phys. Rev. D 76, 044002 (2007)",
8091         hypotheses=[
8092             Hypothesis("vacuum_core", "a compact vacuum core with a periodic "
8093                 "identification", _is_vacuum),
8094         ],
8095         conclusion="closed timelike curves can form in a spacetime whose matter "
8096         "satisfies the weak, dominant and strong energy conditions",
8097         permitted_label="ENERGY_CONDITION_COUNTEREXAMPLE",
8098         forbidden_use="it does not show that such a spacetime is stable, "
8099         "semiclassically consistent, or physically constructible "
8100         "by any known process",
8101         status_note="THEOREM (counterexample)"),
8102 }
8103
8104 def apply(theorem_key: str, model: MetricModel) -> TheoremReport:
8105     """Check every hypothesis before licensing any conclusion."""
8106     th = THEOREMS[theorem_key]
8107     results: dict[str, bool | None] = {}
8108     for h in th.hypotheses:

```

```

8110         results[h.name] = h.evaluate(model)
8111     if any(v is False for v in results.values()):
8112         failed = [k for k, v in results.items() if v is False]
8113         return TheoremReport(theorem_key, Applicability.NOT_APPLICABLE, results,
8114                               None,
8115                               f"hypotheses {failed} are NOT satisfied by "
8116                               f"{model.id}; the theorem licenses nothing here. "
8117                               f"Forbidden use: {th.forbidden_use}")
8118     if any(v is None for v in results.values()):
8119         unknown = [k for k, v in results.items() if v is None]
8120         return TheoremReport(theorem_key, Applicability.HYPOTHESES_UNVERIFIED,
8121                               results, None,
8122                               f"hypotheses {unknown} are UNVERIFIED for "
8123                               f"{model.id}. The architecture reports the unmet "
8124                               f"prerequisites instead of applying the theorem's "
8125                               f"name as a slogan.")
8126     return TheoremReport(theorem_key, Applicability.LICENSED, results,
8127                           th.permitted_label,
8128                           f"all hypotheses verified. Permitted output: "
8129                           f"{th.permitted_label}. Status: {th.status_note}. "
8130                           f"Forbidden use: {th.forbidden_use}")
8131
8132
8133 def applicability_matrix(model: MetricModel) -> list[dict[str, Any]]:
8134     return [apply(k, model).as_dict() for k in sorted(THEOREMS)]
8135
8136
8137 __all__ = ["Applicability", "Hypothesis", "Theorem", "TheoremReport",
8138            "THEOREMS", "apply", "applicability_matrix"]

```

src/ctcfa/optimize.py — 317 lines

```

8139 """
8140 CTC-FA v2.0 -- L17 : minimal on-shell perturbation optimiser.
8141
8142 The question
8143 -----
8144     Delta_+ = inf J(delta theta)   subject to: field equations + domain
8145                                     + a CTC certificate (birth)
8146     Delta_- = inf J(delta theta)   subject to: field equations + domain
8147                                     + a stable-causality certificate (destruction)
8148
8149 with the dimensionless cost
8150
8151     J(delta theta) = sum_i w_i (delta theta_i / Theta_i)^2.
8152
8153 Two things must be true before the word "minimal" means anything, and v1.0
8154 stated both as a gate and enforced neither:
8155
8156     1. the scales ``Theta_i`` and weights ``w_i`` must be **frozen before the
8157        optimisation**, otherwise the cost is not comparable between runs and
8158        "minimal" is a free parameter;
8159     2. the optimisation must stay on an **on-shell family**, otherwise the
8160        answer is a statement about arbitrary tensors, not about physics.
8161
8162 v2.0 enforces (1) with :class:`CostMetric`, which refuses to evaluate until
8163 it is sealed and records a hash of its own definition, and (2) by restricting
8164 the search to registered exact-solution parameter families, where every
8165 configuration solves the field equations identically.
8166
8167 Asymmetry
8168 -----
8169 ``Delta_+`` and ``Delta_-`` are not the same kind of problem. Birth is
8170 certified by a single witness. Destruction requires the *exclusion of all*
8171 closed timelike curves, that is a temporal function or a fully verified
8172 theorem, which is why ``minimal_destruction`` demands a certificate callback
8173 and refuses to accept "the detector found nothing".
8174 """
8175
8176 from __future__ import annotations
8177
8178 import hashlib
8179 import json
8180 import math
8181 from dataclasses import dataclass, field
8182 from typing import Any, Callable, Mapping, Sequence
8183
8184 import numpy as np
8185 from scipy.optimize import minimize
8186
8187 from .causality import compact_orbit_test
8188 from .model import MetricModel
8189 from .status import CTCStatus, OrbitStatus
8190
8191
8192 class UnsealedCostError(RuntimeError):
8193     """Raised when a cost metric is used before its scales are frozen."""
8194
8195
8196 @dataclass
8197 class CostMetric:
8198     """A dimensionless, pre-registered cost on parameter space."""
8199
8200     scales: dict[str, float]
8201     weights: dict[str, float] = field(default_factory=dict)
8202     label: str = ""
8203     sealed: bool = False
8204     _digest: str = ""
8205
8206     def seal(self) -> "CostMetric":
8207         for k in self.scales:
8208             self.weights.setdefault(k, 1.0)
8209         payload = json.dumps({"scales": self.scales, "weights": self.weights,
8210                               "label": self.label}, sort_keys=True)
8211         self._digest = hashlib.sha256(payload.encode()).hexdigest()
8212         self.sealed = True
8213         return self
8214

```

```

8215 @property
8216 def digest(self) -> str:
8217     return self._digest
8218
8219 def evaluate(self, delta: Mapping[str, float]) -> float:
8220     if not self.sealed:
8221         raise UnsealedCostError(
8222             "the cost metric must be sealed (scales and weights frozen) "
8223             "before any optimisation; otherwise the word 'minimal' has no "
8224             "content (v1.0 L17 gate, REQ-009)")
8225     total = 0.0
8226     for k, d in delta.items():
8227         s = self.scales.get(k)
8228         if s is None or s == 0:
8229             raise KeyError(f"no nonzero scale registered for parameter {k!r}")
8230         total += self.weights.get(k, 1.0) * (float(d) / float(s)) ** 2
8231     return total
8232
8233 def as_dict(self) -> dict[str, Any]:
8234     return {"label": self.label, "scales": dict(self.scales),
8235           "weights": dict(self.weights), "sealed": self.sealed,
8236           "digest": self._digest}
8237
8238 @dataclass
8239 class MinimalPerturbation:
8240     model_id: str
8241     direction: str
8242     cost: float | None
8243     delta: dict[str, float]
8244     base: dict[str, float]
8245     achieved: bool
8246     certificate: str
8247     cost_metric_digest: str
8248     iterations: int
8249     note: str = ""
8250
8251 def as_dict(self) -> dict[str, Any]:
8252     return {"model_id": self.model_id, "direction": self.direction,
8253           "cost": self.cost, "delta": self.delta, "base": self.base,
8254           "achieved": self.achieved, "certificate": self.certificate,
8255           "cost_metric_digest": self.cost_metric_digest,
8256           "iterations": self.iterations, "note": self.note}
8257
8258 def _apply_delta(base: Mapping[str, float], names: Sequence[str],
8259                 x: np.ndarray) -> dict[str, float]:
8260     out = dict(base)
8261     for n, v in zip(names, x):
8262         out[n] = float(base[n]) + float(v)
8263     return out
8264
8265 def minimal_birth(model: MetricModel, base: Mapping[str, float],
8266                 free_parameters: Sequence[str], cost: CostMetric,
8267                 bounds: Mapping[str, tuple[float, float]] | None = None,
8268                 restarts: int = 12, seed: int = 20260831,
8269                 margin_target: float = -1e-8) -> MinimalPerturbation:
8270     """Smallest on-shell parameter change that creates a certified CTC.
8271
8272     The search moves only along the registered exact family, so every trial
8273     configuration is an exact solution and the answer is a physical statement
8274     rather than an off-shell one.
8275
8276     """
8277     names = list(free_parameters)
8278     b = {k: (bounds or {}).get(k, (-abs(base[k]) * 2 - 1.0,
8279                                   abs(base[k]) * 2 + 1.0)) for k in names}
8280     rng = np.random.default_rng(seed)
8281
8282 def margin(x: np.ndarray) -> float:
8283     """Causal margin at base + x.
8284
8285     The REDUCED margin is used where it is defined, because the raw
8286     component g(x1,x1) has an arbitrary scale (it is 2.5e+03 for the
8287     Godel-type family at one point and 1e-02 at another), which makes any
8288     penalty weighting meaningless and defeats a local optimiser. The
8289     reduced margin is dimensionless and has the same sign.
8290
8291     """
8292     vals = _apply_delta(base, names, x)
8293     try:
8294         from .continuation import reduced_causal_margin
8295         return float(reduced_causal_margin(model, vals))
8296     except Exception:
8297         # noqa: BLE001
8298         pass
8299     try:
8300         r = compact_orbit_test(model, vals, check_orientation=False)
8301     except Exception:
8302         # noqa: BLE001
8303         return le6
8304     return float(r.margin) if np.isfinite(r.margin) else le6
8305
8306 def feasible(x: np.ndarray) -> bool:
8307     m = margin(x)
8308     return np.isfinite(m) and m <= margin_target
8309
8310 def obj(x: np.ndarray) -> float:
8311     c = cost.evaluate({n: v for n, v in zip(names, x)})
8312     m = margin(x)
8313     if not np.isfinite(m):
8314         return le9
8315     penalty = 0.0 if m <= margin_target else le3 * (m - margin_target) ** 2
8316     return c + penalty
8317
8318 # -- stage 1: coarse feasibility scan -----
8319 #
8320 # A local optimiser started at delta = 0 cannot escape an infeasible
8321 # region whose boundary is far away, because Nelder-Mead's default
8322 # initial simplex around the origin is microscopic. A scan over the
8323 # declared bounds finds the feasible set first; the local stage then only
8324 # has to polish.
8325 dim = len(names)

```

```

8325 n_scan = max(64, 24 * dim)
8326 starts: list[np.ndarray] = []
8327 grid = [np.linspace(b[n][0], b[n][1], n_scan) for n in names]
8328 if dim == 1:
8329     cands = [np.array([v]) for v in grid[0]]
8330 else:
8331     cands = [np.array([rng.uniform(*b[n]) for n in names])
8332               for _ in range(n_scan * 8)]
8333 feas = [x for x in cands if feasible(x)]
8334 feas.sort(key=lambda x: cost.evaluate({n: v for n, v in zip(names, x)}))
8335 starts = feas[:max(1, restarts)] if feas else []
8336
8337 best: tuple[float, np.ndarray] | None = None
8338 iters = 0
8339 for k in range(max(restarts, len(starts))):
8340     if k < len(starts):
8341         x0 = starts[k]
8342     else:
8343         x0 = np.array([rng.uniform(*b[n]) for n in names])
8344     try:
8345         res = minimize(obj, x0, method="Nelder-Mead",
8346                       options={"maxiter": 2000, "xatol": 1e-13,
8347                               "fatol": 1e-15})
8348     except Exception:
8349         continue
8350     iters += int(res.nit)
8351     for cand in (res.x, x0):
8352         if feasible(cand):
8353             f = cost.evaluate({n: v for n, v in zip(names, cand)})
8354             if best is None or f < best[0]:
8355                 best = (float(f), np.asarray(cand))
8356
8357 # -- stage 3: polish onto the feasibility boundary -----
8358 #
8359 # The cost decreases monotonically towards delta = 0, so the minimum of a
8360 # convex cost over the feasible set is attained ON the boundary. A
8361 # penalty method stops just outside it; bisection along the ray from the
8362 # best feasible point to the origin lands exactly on it, which turns a
8363 # three-digit answer into a machine-precision one.
8364 if best is not None:
8365     xf = best[1]
8366     x_in, x_out = xf.copy(), np.zeros_like(xf)
8367     if not feasible(x_out):
8368         for _ in range(200):
8369             mid = 0.5 * (x_in + x_out)
8370             if feasible(mid):
8371                 x_in = mid
8372             else:
8373                 x_out = mid
8374             if float(np.max(np.abs(x_in - x_out))) < 1e-15:
8375                 break
8376         f = cost.evaluate({n: v for n, v in zip(names, x_in)})
8377         if f < best[0]:
8378             best = (float(f), x_in)
8379
8380 if best is None:
8381     return MinimalPerturbation(
8382         model.id, "birth", None, {}, dict(base), False,
8383         "no certified CTC found on the searched on-shell family",
8384         cost.digest, iters,
8385         "failure of the search is UNRESOLVED, never a proof that no small "
8386         "on-shell perturbation creates a CTC")
8387 delta = {n: float(v) for n, v in zip(names, best[1])}
8388 vals = _apply_delta(base, names, best[1])
8389 r = compact_orbit_test(model, vals)
8390 return MinimalPerturbation(
8391     model.id, "birth", cost.evaluate(delta), delta, dict(base),
8392     r.status is OrbitStatus.SECTOR CTC, r.reason, cost.digest, iters,
8393     "cost is dimensionless with pre-registered scales; every trial "
8394     "configuration is an exact solution of the registered family")
8395
8396
8397 def minimal_destruction(model: MetricModel, base: Mapping[str, float],
8398                        free_parameters: Sequence[str], cost: CostMetric,
8399                        certificate: Callable[[Mapping[str, float]],
8400                                           tuple[bool, str]],
8401                        bounds: Mapping[str, tuple[float, float]] | None = None,
8402                        restarts: int = 12, seed: int = 20260831
8403                        ) -> MinimalPerturbation:
8404     """Smallest on-shell change that *certifies* the restoration of causality.
8405
8406     ``certificate`` must return ``(True, reason)`` only when a complete causal
8407     certificate holds -- a verified temporal function or a theorem whose
8408     hypotheses are all satisfied. A detector that merely fails to find a CTC
8409     is not acceptable and, if supplied, will simply never produce a
8410     ``True``.
8411
8412     """
8413     names = list(free_parameters)
8414     b = {k: (bounds or {}).get(k, (-abs(base[k]) * 2 - 1.0,
8415                                     abs(base[k]) * 2 + 1.0)) for k in names}
8415     rng = np.random.default_rng(seed)
8416
8417     def obj(x: np.ndarray) -> float:
8418         vals = _apply_delta(base, names, x)
8419         ok, _ = certificate(vals)
8420         c = cost.evaluate({n: v for n, v in zip(names, x)})
8421         return c if ok else c + 1e6
8422
8423     best: tuple[float, np.ndarray, str] | None = None
8424     iters = 0
8425     for k in range(restarts):
8426         x0 = np.zeros(len(names)) if k == 0 else np.array(
8427             [rng.uniform(*b[n]) * 0.3 for n in names])
8428         try:
8429             res = minimize(obj, x0, method="Nelder-Mead",
8430                           options={"maxiter": 2000, "xatol": 1e-12})
8431         except Exception:
8432             continue
8433         iters += int(res.nit)
8434         ok, why = certificate(_apply_delta(base, names, res.x))

```



```

8435         if ok and (best is None or res.fun < best[0]):
8436             best = (float(res.fun), res.x, why)
8437
8438         if best is None:
8439             return MinimalPerturbation(
8440                 model.id, "destruction", None, {}, dict(base), False,
8441                 "no configuration on the searched family produced a complete causal "
8442                 "certificate", cost.digest, iters,
8443                 "CTC destruction requires exclusion of ALL closed timelike curves, "
8444                 "which is why this problem is strictly harder than birth and is "
8445                 "not solvable by a detector alone")
8446         delta = {n: float(v) for n, v in zip(names, best[1])}
8447         return MinimalPerturbation(
8448             model.id, "destruction", cost.evaluate(delta), delta, dict(base), True,
8449             best[2], cost.digest, iters,
8450             "restoration certified by a complete causal certificate, not by "
8451             "non-detection")
8452
8453
8454 __all__ = ["UnsealedCostError", "CostMetric", "MinimalPerturbation",
8455            "minimal_birth", "minimal_destruction"]

```

src/ctcfa/meta.py — 259 lines

```

8456 """
8457 CTC-FA v2.0  -- L19 : cross-family meta-analysis.
8458
8459 Purpose
8460 -----
8461 The point of a registry is not to catalogue famous geometries. It is to make
8462 the question "what do the chronology-violating spacetimes have in common, and
8463 what do the chronal ones have in common?" a *computation* over a common
8464 feature space.
8465
8466 Method and its honesty constraint
8467 -----
8468 Every candidate invariant precursor is derived on a **training** subset and
8469 then tested on a **held-out** subset that the derivation never saw. A rule
8470 that survives is reported with its held-out score; a rule that does not is
8471 reported as refuted. Nothing is promoted to "universal" -- the architecture
8472 emits ``CANDIDATE_PRECURSOR`` and requires independent replication before any
8473 stronger word is used.
8474
8475 The feature vector is deliberately built only from quantities the other
8476 layers *compute*, never from the model's name or its expected status, so that
8477 the analysis cannot trivially recover the answer it was given. A
8478 ``leakage_check`` verifies exactly that.
8479 """
8480
8481 from __future__ import annotations
8482
8483 import itertools
8484 from dataclasses import dataclass, field
8485 from typing import Any, Callable, Iterable, Mapping, Sequence
8486
8487 import numpy as np
8488 import sympy as sp
8489
8490 from .causality import compact_orbit_test
8491 from .model import MetricModel
8492 from .status import CTCStatus, OrbitStatus
8493
8494
8495 FEATURE_NAMES = (
8496     "has_compact_generator",
8497     "n_deck_generators",
8498     "has_nontrivial_quotient",
8499     "generator_is_killing",
8500     "sector_margin_sign",
8501     "reduced_margin_defined",
8502     "vacuum",
8503     "has_cosmological_constant",
8504     "flat_covering",
8505     "dimension",
8506     "n_parameters",
8507     "has_temporal_function_candidate",
8508     "compact_generator_is_time_direction",
8509 )
8510
8511 @dataclass
8512 class FeatureRow:
8513     model_id: str
8514     features: dict[str, float]
8515     label: str
8516     note: str = ""
8517
8518     def vector(self) -> np.ndarray:
8519         return np.array([self.features.get(k, np.nan) for k in FEATURE_NAMES],
8520                         dtype=float)
8521
8522
8523 def extract_features(model: MetricModel) -> FeatureRow:
8524     """Compute the feature vector for one model from the engines only."""
8525     from .symmetry import generators_admissible
8526     f: dict[str, float] = {}
8527     f["has_compact_generator"] = float(bool(model.compact))
8528     f["n_deck_generators"] = float(len(model.quotient.generators))
8529     f["has_nontrivial_quotient"] = float(bool(model.quotient.generators)
8530                                         or bool(model.compact))
8531
8532     try:
8533         ok, _ = generators_admissible(model)
8534     except Exception:
8535         ok = False
8536         # noqa: BLE001
8537     f["generator_is_killing"] = float(bool(ok))
8538
8539     sign = 0.0
8540     if model.has_metric and model.compact and model.default_point:

```



```

8540     try:
8541         r = compact_orbit_test(model, model.default_point)
8542         sign = (-1.0 if r.status is OrbitStatus.SECTOR_CTC else
8543                0.0 if r.status is OrbitStatus.SECTOR_NULL else
8544                1.0 if r.status is OrbitStatus.SECTOR_SPACELIKE else np.nan)
8545     except Exception:
8546         sign = float("nan")
8547     else:
8548         sign = float("nan")
8549     f["sector_margin_sign"] = sign
8550
8551     try:
8552         from .continuation import reduced_causal_margin_symbolic
8553         reduced_causal_margin_symbolic(model)
8554         f["reduced_margin_defined"] = 1.0
8555     except Exception:
8556         f["reduced_margin_defined"] = 0.0
8557
8558     f["vacuum"] = float(bool(model.matter.vacuum))
8559     f["has_cosmological_constant"] = float(
8560         model.matter.cosmological_constant != 0)
8561     f["flat_covering"] = float(bool(model.metadata.get("flat_covering")))
8562     f["dimension"] = float(model.dim)
8563     f["n_parameters"] = float(len(model.params))
8564     f["has_temporal_function_candidate"] = float(model.temporal_function is not None)
8565     f["compact_generator_is_time_direction"] = float(
8566         any(c.index == 0 for c in model.compact))
8567
8568     label = ("CHRONOLOGY_VIOLATING"
8569             if model.expected is CTCStatus.CTC_PROVED else
8570             "CHRONAL" if model.expected is CTCStatus.NO_CTC_CERTIFIED else
8571             "UNRESOLVED")
8572     return FeatureRow(model.id, f, label)
8573
8574
8575 def build_table(models: Iterable[MetricModel]) -> list[FeatureRow]:
8576     return [extract_features(m) for m in models]
8577
8578
8579 # -----
8580 # Rule search with held-out validation
8581 # -----
8582
8583 @dataclass
8584 class Rule:
8585     feature: str
8586     threshold: float
8587     direction: str
8588     train_accuracy: float
8589     heldout_accuracy: float
8590     train_n: int
8591     heldout_n: int
8592     status: str
8593     note: str = ""
8594
8595     def predict(self, row: FeatureRow) -> bool:
8596         v = row.features.get(self.feature, float("nan"))
8597         if not np.isfinite(v):
8598             return False
8599         return v <= self.threshold if self.direction == "<=" else v > self.threshold
8600
8601     def as_dict(self) -> dict[str, Any]:
8602         return {"feature": self.feature, "threshold": self.threshold,
8603                "direction": self.direction,
8604                "train_accuracy": self.train_accuracy,
8605                "heldout_accuracy": self.heldout_accuracy,
8606                "train_n": self.train_n, "heldout_n": self.heldout_n,
8607                "status": self.status, "note": self.note}
8608
8609
8610 def _accuracy(rule_pred: Callable[[FeatureRow], bool], rows: Sequence[FeatureRow],
8611               positive: str) -> float:
8612     if not rows:
8613         return float("nan")
8614     correct = sum(1 for r in rows
8615                  if rule_pred(r) == (r.label == positive))
8616     return correct / len(rows)
8617
8618
8619 def search_precursors(rows: Sequence[FeatureRow], positive: str =
8620                       "CHRONOLOGY_VIOLATING", heldout_fraction: float = 0.35,
8621                       seed: int = 20260831,
8622                       min_heldout_accuracy: float = 0.9) -> list[Rule]:
8623     """Single-feature threshold rules, derived on train and scored on held out."""
8624     labelled = [r for r in rows if r.label in (positive, "CHRONAL")]
8625     rng = np.random.default_rng(seed)
8626     idx = rng.permutation(len(labelled))
8627     n_hold = max(2, int(round(heldout_fraction * len(labelled))))
8628     hold = [labelled[i] for i in idx[:n_hold]]
8629     train = [labelled[i] for i in idx[n_hold:]]
8630
8631     out: list[Rule] = []
8632     for feat in FEATURE_NAMES:
8633         vals = sorted({r.features.get(feat, float("nan")) for r in train})
8634         vals = [v for v in vals if np.isfinite(v)]
8635         if len(vals) < 2:
8636             continue
8637         cuts = [0.5 * (a + b) for a, b in zip(vals, vals[1:])]
8638         best: Rule | None = None
8639         for c in cuts:
8640             for d in ("<=", ">"):
8641                 probe = Rule(feat, float(c), d, 0.0, 0.0, len(train), len(hold),
8642                             "candidate")
8643                 acc = _accuracy(probe.predict, train, positive)
8644                 if best is None or acc > best.train_accuracy:
8645                     probe.train_accuracy = acc
8646                     best = probe
8647         if best is None:
8648             continue
8649         best.heldout_accuracy = _accuracy(best.predict, hold, positive)

```

```

8650         best.status = ("CANDIDATE_PRECURSOR"
8651                         if best.heldout_accuracy >= min_heldout_accuracy
8652                         else "REFUTED_ON_HELD_OUT")
8653         best.note = ("derived on the training split only; the held-out score is "
8654                     "the only number that may be quoted. A CANDIDATE_PRECURSOR "
8655                     "still requires independent replication before any "
8656                     "universality claim.")
8657         out.append(best)
8658     out.sort(key=lambda r: (-r.heldout_accuracy, -r.train_accuracy))
8659     return out
8660
8661 def leakage_check(rows: Sequence[FeatureRow]) -> dict[str, Any]:
8662     """Confirm that no feature is a relabelling of the expected status.
8663
8664     A feature that perfectly reproduces the label on *every* model is
8665     suspicious: either it is the label in disguise, or it is a genuine
8666     theorem. The check reports such features so a human can decide which,
8667     instead of the analysis silently congratulating itself.
8668     """
8669     suspicious = []
8670     for feat in FEATURE_NAMES:
8671         vals = {}
8672         conflict = False
8673         for r in rows:
8674             if r.label == "UNRESOLVED":
8675                 continue
8676             v = r.features.get(feat, float("nan"))
8677             if not np.isfinite(v):
8678                 continue
8679             vals.setdefault(v, set()).add(r.label)
8680         for v, labels in vals.items():
8681             if len(labels) > 1:
8682                 conflict = True
8683             if not conflict and len(vals) > 1:
8684                 suspicious.append(feat)
8685     return {"perfectly_separating_features": suspicious,
8686           "n_models": len(rows),
8687           "note": ("A perfectly separating feature is either a hidden copy of "
8688                  "the label or a real structural theorem. It must be "
8689                  "adjudicated by hand, never auto-promoted.")}
8690
8691 def cross_family_report(models: Iterable[MetricModel]) -> dict[str, Any]:
8692     rows = build_table(models)
8693     rules = search_precursors(rows)
8694     counts: dict[str, int] = {}
8695     for r in rows:
8696         counts[r.label] = counts.get(r.label, 0) + 1
8697     return {"n_models": len(rows), "label_counts": counts,
8700           "features": list(FEATURE_NAMES),
8701           "table": [{"model_id": r.model_id, "label": r.label,
8702                     **{k: r.features.get(k) for k in FEATURE_NAMES}}
8703                    for r in rows],
8704           "rules": [r.as_dict() for r in rules],
8705           "leakage": leakage_check(rows),
8706           "governance": ("No rule in this report may be described as a "
8707                          "theorem or as universal. The permitted label is "
8708                          "CANDIDATE_PRECURSOR, and only after independent "
8709                          "replication on a corpus this analysis has never "
8710                          "seen (v1.0 REQ-019, P8/P9 of the roadmap).")}
8711
8712 __all__ = ["FEATURE_NAMES", "FeatureRow", "extract_features", "build_table",
8713           "Rule", "search_precursors", "leakage_check", "cross_family_report"]

```

src/ctcfa/validate.py — 210 lines

```

8715 """
8716 CTC-FA v2.0 -- L0/L1/L2 : structural, algebraic and field-equation validation.
8717
8718 Gate V0 (parse)      the registry record is complete and internally
8719                      consistent: symmetric metric, declared signature,
8720                      no undeclared symbols, generator/coordinate agreement,
8721                      provenance present.
8722 Gate V1 (algebra)    inverse metric verified, signature verified numerically
8723                      at the declared point, declared compact generators are
8724                      Killing vectors, declared deck generators are isometries.
8725 Gate V2 (tensor)     curvature computed by two independent routes (exact
8726                      symbolic differentiation and high-order finite
8727                      differences) that must agree within the reported error.
8728 Gate V3 (field eq.)  the field-equation residual meets the model's declared
8729                      tolerance for its declared matter content.
8730
8731 The two-route requirement in V2 is what makes a silent algebra bug visible.
8732 v1.0 named an independent CAS cross-check as "planned external validation";
8733 v2.0 ships an independent numerical route inside the package and runs it on
8734 every model in the regression suite.
8735 """
8736
8737 from __future__ import annotations
8738
8739 from dataclasses import dataclass, field
8740 from typing import Any, Mapping
8741
8742 import numpy as np
8743 import sympy as sp
8744
8745 from .model import MetricModel
8746 from .numerics import signature_counts
8747 from .status import Gate, GateRecord, GateResult, ModelKind
8748 from .symbolic import check_inverse, ricci_scalar
8749 from .symmetry import verify_compact_generators
8750 from .topology import verify_quotient
8751
8752 @dataclass
8753 class ValidationReport:

```

```

8755 model_id: str
8756 gates: GateRecord
8757 details: dict[str, Any] = field(default_factory=dict)
8758
8759 @property
8760 def ok(self) -> bool:
8761     return all(self.gates.get(g) in (GateResult.PASS, GateResult.NOT_APPLICABLE)
8762               for g in (Gate.V0_PARSE, Gate.V1_ALGEBRA))
8763
8764 def as_dict(self) -> dict[str, Any]:
8765     return {"model_id": self.model_id, "ok": self.ok,
8766           "gates": self.gates.as_dict(), "details": self.details}
8767
8768 def validate_model(model: MetricModel, point: Mapping[str, float] | None = None,
8769                  field_equation_tol: float = 1e-9,
8770                  cross_check: bool = True) -> ValidationReport:
8771     gates = GateRecord()
8772     details: dict[str, Any] = {}
8773     point = dict(point or model.default_point)
8774
8775     # ---- V0 -----
8776     try:
8777         model.assert_wellformed()
8778         problems = []
8779         if not model.provenance.source_refs:
8780             problems.append("no source references")
8781         if model.kind in (ModelKind.EXTERNAL_EXACT_SPEC,
8782                         ModelKind.COMPOSITE_GLOBAL) and model.has_metric:
8783             problems.append("global/external model carries a local metric")
8784         if problems:
8785             gates.set(Gate.V0_PARSE, GateResult.FAIL, "; ".join(problems))
8786         else:
8787             gates.set(Gate.V0_PARSE, GateResult.PASS,
8788                     "record complete, metric symmetric, symbols declared")
8789     except Exception as exc:
8790         # noqa: BLE001
8791         gates.set(Gate.V0_PARSE, GateResult.FAIL, str(exc))
8792         return ValidationReport(model.id, gates, details)
8793
8794     if not model.has_metric:
8795         for g in (Gate.V1_ALGEBRA, Gate.V2_TENSOR, Gate.V3_FIELD_EQUATION):
8796             gates.set(g, GateResult.NOT_APPLICABLE,
8797                     "no local metric: this model is an external or composite "
8798                     "specification")
8799         details["kind"] = model.kind.value
8800         return ValidationReport(model.id, gates, details)
8801
8802     # ---- V1 -----
8803     v1_notes = []
8804     ok1 = True
8805     try:
8806         resid = check_inverse(model.metric)
8807         inv_ok = bool(resid.is_zero_matrix)
8808         v1_notes.append(f"signature {inv_ok} - I zero: {inv_ok}")
8809         ok1 = ok1 and inv_ok
8810         details["inverse_residual"] = "0" if inv_ok else str(resid)
8811     except Exception as exc:
8812         # noqa: BLE001
8813         ok1 = False
8814         v1_notes.append(f"inverse failed: {exc}")
8815
8816     if point and not model.missing_values(point):
8817         try:
8818             gnum = np.array(sp.Matrix(model.metric)
8819                             .subs(model.substitution(point)).evalf(), dtype=float)
8820             sig = signature_counts(gnum)
8821             sig_ok = sig == tuple(model.signature)
8822             v1_notes.append(f"signature {sig} vs declared "
8823                             f"{tuple(model.signature)}: {sig_ok}")
8824             details["signature_at_point"] = list(sig)
8825             ok1 = ok1 and sig_ok
8826         except Exception as exc:
8827             # noqa: BLE001
8828             v1_notes.append(f"signature check failed: {exc}")
8829             ok1 = False
8830
8831     certs = verify_compact_generators(model)
8832     details["compact_generators"] = [
8833         {"name": c.name, "is_killing": c.is_killing,
8834          "residual": c.killing_residual[:200]} for c in certs]
8835     for cd, c in zip(model.compact, certs):
8836         if cd.requires_killing and not c.is_killing:
8837             ok1 = False
8838             v1_notes.append(f"generator {c.name} is not Killing")
8839
8840     qc = verify_quotient(model, point)
8841     details["quotient"] = qc.as_dict()
8842     if qc.generators and not qc.admissible:
8843         v1_notes.append("some deck generators are not admissible "
8844                         "(non-isometry or fixed point)")
8845
8846     gates.set(Gate.V1_ALGEBRA, GateResult.PASS if ok1 else GateResult.FAIL,
8847             "; ".join(v1_notes))
8848
8849     # ---- V2 : two independent curvature routes -----
8850     if cross_check and point and not model.missing_values(point):
8851         try:
8852             from .energy import symbolic_einstein
8853             from .numgeom import NumericGeometry
8854             Gsym = symbolic_einstein(model)
8855             sub = model.substitution(point)
8856             Ga = np.array(sp.Matrix(Gsym).subs(sub).evalf(30), dtype=float)
8857             ng = NumericGeometry(model)
8858             Gb, err = ng.einstein(point)
8859             diff = float(np.max(np.abs(Ga - Gb)))
8860             scale = max(1.0, float(np.max(np.abs(Ga))), float(np.max(np.abs(Gb))))
8861             agree = diff <= max(50 * err, 1e-6 * scale)
8862             details["einstein_cross_check"] = {
8863                 "symbolic_max_abs": float(np.max(np.abs(Ga))),
8864                 "numeric_max_abs": float(np.max(np.abs(Gb))),
8865                 "max_difference": diff, "numeric_error_estimate": err,
8866                 "agree": agree}

```

```

8865         gates.set(Gate.V2_TENSOR,
8866                     GateResult.PASS if agree else GateResult.FAIL,
8867                     f"exact symbolic and finite-difference Einstein tensors "
8868                     f"differ by {diff:.3e} (numeric error estimate {err:.3e})")
8869     except Exception as exc:
8870         gates.set(Gate.V2_TENSOR, GateResult.INCONCLUSIVE,
8871                 f"cross-check unavailable: {exc}")
8872     else:
8873         gates.set(Gate.V2_TENSOR, GateResult.NOT_ATTEMPTED,
8874                 "no fully specified point supplied")
8875
8876     # ---- V3 : field equations -----
8877     if point and not model.missing_values(point):
8878         try:
8879             from .energy import symbolic_einstein
8880             G = symbolic_einstein(model)
8881             Lam = model.matter.cosmological_constant
8882             T = model.matter.T
8883             rhs = (sp.zeros(model.dim, model.dim) if T is None
8884                    else 8 * sp.pi * T)
8885             res = sp.Matrix(G + Lam * model.metric - rhs)
8886             arr = np.array(res.subs(model.substitution(point))).evalf(30),
8887                           dtype=float)
8888             m = float(np.max(np.abs(arr)))
8889             details["field_equation_residual"] = m
8890             if model.matter.vacuum or T is not None:
8891                 gates.set(Gate.V3_FIELD_EQUATION,
8892                         GateResult.PASS if m <= field_equation_tol
8893                         else GateResult.FAIL,
8894                         f"|G + Lambda g - 8 pi T|_max = {m:.3e} "
8895                         f"(tolerance {field_equation_tol:.1e})")
8896             else:
8897                 gates.set(Gate.V3_FIELD_EQUATION, GateResult.NOT_APPLICABLE,
8898                         f"matter content is '{model.matter.label}': no closed "
8899                         f"form T_ab is registered, so the residual "
8900                         f"{m:.3e} DEFINES the required source rather than "
8901                         f"testing the solution")
8902         except Exception as exc:
8903             gates.set(Gate.V3_FIELD_EQUATION, GateResult.INCONCLUSIVE, str(exc))
8904     else:
8905         gates.set(Gate.V3_FIELD_EQUATION, GateResult.NOT_ATTEMPTED,
8906                 "no fully specified point supplied")
8907
8908     return ValidationReport(model.id, gates, details)
8909
8910
8911 def validate_registry(cross_check: bool = True) -> dict[str, Any]:
8912     from .registry import models
8913     rows = []
8914     n_fail = 0
8915     for m in models():
8916         rep = validate_model(m, cross_check=cross_check)
8917         rows.append(rep.as_dict())
8918         if not rep.ok:
8919             n_fail += 1
8920     return {"n_models": len(rows), "n_failed": n_fail,
8921           "pass": n_fail == 0, "reports": rows}
8922
8923
8924 __all__ = ["ValidationReport", "validate_model", "validate_registry"]

```

src/ctcfa/provenance.py — 265 lines

```

8925 """
8926 CTC-FA v2.0 -- Provenance, integrity and immutable run records.
8927
8928 Section 30 of v1.0 lists seven integrity controls. They were described but
8929 not implemented; the released kernel carried a SHA-256 in prose and nothing
8930 that computed or verified it. This module implements all seven:
8931
8932 1. SHA-256 of every release artifact and of the frozen configuration;
8933 2. signed / verifiable release manifests (a detached digest file that a
8934    signature tool can sign);
8935 3. immutable run records keyed by a content hash;
8936 4. sealed benchmark answers (HMAC), so a blind evaluation is possible;
8937 5. never overwrite a historical manifest -- supersede it;
8938 6. record dependency and interpreter versions with every run;
8939 7. a minimal independent reproduction script emitted with every claim.
8940
8941 The two-pass build
8942 -----
8943 An integrity hash that is printed *inside* the document it describes is
8944 self-referential. The two-pass convention used here is: pass 1 builds the
8945 artifact with the hash field blank and computes ``H1`` over the source tree
8946 (not the artifact); pass 2 embeds ``H1``, rebuilds, and records ``H2``, the
8947 hash of the finished artifact. Both are published, and
8948 :func:`verify_two_pass` checks the pair.
8949 """
8950
8951 from __future__ import annotations
8952
8953 import hashlib
8954 import hmac
8955 import json
8956 import os
8957 import platform
8958 import subprocess
8959 import sys
8960 import time
8961 import uuid
8962 from dataclasses import dataclass, field
8963 from pathlib import Path
8964 from typing import Any, Iterable, Mapping, Sequence
8965
8966
8967 def sha256_bytes(data: bytes) -> str:
8968     return hashlib.sha256(data).hexdigest()
8969

```

```

8970
8971 def sha256_file(path: str | Path) -> str:
8972     h = hashlib.sha256()
8973     with open(path, "rb") as fh:
8974         for chunk in iter(lambda: fh.read(1 << 20), b''):
8975             h.update(chunk)
8976     return h.hexdigest()
8977
8978
8979 def sha256_text(text: str) -> str:
8980     return sha256_bytes(text.encode("utf-8"))
8981
8982
8983 def source_tree_digest(root: str | Path, patterns: Sequence[str] = ("*.py",)
8984     ) -> tuple[str, list[dict[str, str]]]:
8985     """Deterministic digest of a source tree, plus the per-file table."""
8986     root = Path(root)
8987     files: list[Path] = []
8988     for pat in patterns:
8989         files.extend(sorted(root.rglob(pat)))
8990     files = sorted({f.resolve() for f in files if f.is_file()},
8991         key=lambda p: str(p.relative_to(root.resolve()))))
8992     rows = []
8993     h = hashlib.sha256()
8994     for f in files:
8995         rel = str(f.relative_to(root.resolve()))
8996         d = sha256_file(f)
8997         rows.append({"path": rel, "sha256": d, "bytes": str(f.stat().st_size)})
8998         h.update(rel.encode())
8999         h.update(d.encode())
9000     return h.hexdigest(), rows
9001
9002
9003 def environment_lock() -> dict[str, Any]:
9004     """Dependency and interpreter versions (control 6)."""
9005     mods = {}
9006     for name in ("sympy", "numpy", "scipy", "mpmath", "matplotlib", "reportlab"):
9007         try:
9008             m = __import__(name)
9009             mods[name] = getattr(m, "__version__", "unknown")
9010         except Exception:
9011             mods[name] = "absent"
9012     return {"python": sys.version.split()[0],
9013         "implementation": platform.python_implementation(),
9014         "platform": platform.platform(),
9015         "modules": mods}
9016
9017
9018 # -----
9019 # Run records
9020 # -----
9021
9022 @dataclass
9023 class RunRecord:
9024     """An immutable record of one computation (v1.0 section 25.2)."""
9025
9026     run_id: str
9027     model_id: str
9028     method: str
9029     parameters: dict[str, Any]
9030     result: dict[str, Any]
9031     code_digest: str
9032     environment: dict[str, Any]
9033     created_utc: str
9034     tolerances: dict[str, Any] = field(default_factory=dict)
9035     source_refs: list[str] = field(default_factory=list)
9036     review_state: str = "machine"
9037     supersedes: str | None = None
9038
9039     def content_hash(self) -> str:
9040         payload = json.dumps(self.as_dict(include_hash=False), sort_keys=True)
9041         return sha256_text(payload)
9042
9043     def as_dict(self, include_hash: bool = True) -> dict[str, Any]:
9044         d = {"run_id": self.run_id, "model_id": self.model_id,
9045             "method": self.method, "parameters": self.parameters,
9046             "result": self.result, "code_digest": self.code_digest,
9047             "environment": self.environment, "created_utc": self.created_utc,
9048             "tolerances": self.tolerances, "source_refs": self.source_refs,
9049             "review_state": self.review_state, "supersedes": self.supersedes}
9050         if include_hash:
9051             d["content_hash"] = self.content_hash()
9052         return d
9053
9054
9055 def new_run(model_id: str, method: str, parameters: Mapping[str, Any],
9056     result: Mapping[str, Any], code_digest: str,
9057     tolerances: Mapping[str, Any] | None = None,
9058     source_refs: Sequence[str] = ()) -> RunRecord:
9059     return RunRecord(
9060         run_id=str(uuid.uuid4()), model_id=model_id, method=method,
9061         parameters=dict(parameters), result=dict(result),
9062         code_digest=code_digest, environment=environment_lock(),
9063         created_utc=time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
9064         tolerances=dict(tolerances or {}), source_refs=list(source_refs))
9065
9066
9067 def write_run(record: RunRecord, directory: str | Path) -> Path:
9068     """Write a run record. Never overwrites (control 5)."""
9069     directory = Path(directory)
9070     directory.mkdir(parents=True, exist_ok=True)
9071     p = directory / f"{record.model_id}_{record.content_hash()[:16]}.json"
9072     if p.exists():
9073         return p
9074     p.write_text(json.dumps(record.as_dict(), indent=2, sort_keys=True) + "\n",
9075         encoding="utf-8")
9076     return p
9077
9078
9079 # -----

```

```

9080 # Sealed benchmark answers (control 4)
9081 # -----
9082
9083 def seal_answers(answers: Mapping[str, Any], key: bytes) -> dict[str, str]:
9084     """HMAC-seal a benchmark answer key so a solver cannot read it.
9085
9086     The evaluator holds ``key``; the solver holds only the sealed digests and
9087     can verify a submitted answer without ever learning the target. This is
9088     what makes the blind benchmark protocol of section 15.2 executable rather
9089     than aspirational.
9090     """
9091     out = {}
9092     for k, v in sorted(answers.items()):
9093         payload = json.dumps({k: v}, sort_keys=True).encode()
9094         out[k] = hmac.new(key, payload, hashlib.sha256).hexdigest()
9095     return out
9096
9097
9098 def check_answer(name: str, value: Any, sealed: Mapping[str, str],
9099                 key: bytes) -> bool:
9100     payload = json.dumps({name: value}, sort_keys=True).encode()
9101     want = sealed.get(name)
9102     if want is None:
9103         return False
9104     return hmac.compare_digest(
9105         hmac.new(key, payload, hashlib.sha256).hexdigest(), want)
9106
9107
9108 # -----
9109 # Release manifest and the two-pass build
9110 # -----
9111
9112 @dataclass
9113 class ReleaseManifest:
9114     document_id: str
9115     version: str
9116     architect: str
9117     date: str
9118     source_digest: str
9119     files: list[dict[str, str]]
9120     environment: dict[str, Any]
9121     artifact_digest: str | None = None
9122     pass1_digest: str | None = None
9123     note: str = ""
9124
9125     def as_dict(self) -> dict[str, Any]:
9126         return {"document_id": self.document_id, "version": self.version,
9127                 "architect": self.architect, "date": self.date,
9128                 "source_digest": self.source_digest, "files": self.files,
9129                 "environment": self.environment,
9130                 "artifact_digest": self.artifact_digest,
9131                 "pass1_digest": self.pass1_digest, "note": self.note}
9132
9133     def write(self, path: str | Path) -> Path:
9134         p = Path(path)
9135         p.parent.mkdir(parents=True, exist_ok=True)
9136         p.write_text(json.dumps(self.as_dict(), indent=2, sort_keys=True) + "\n",
9137                     encoding="utf-8")
9138         return p
9139
9140
9141 def build_manifest(root: str | Path, document_id: str, version: str,
9142                  architect: str, date: str) -> ReleaseManifest:
9143     digest, files = source_tree_digest(root)
9144     return ReleaseManifest(document_id, version, architect, date, digest, files,
9145                           environment_lock(),
9146                           note="pass 1: source digest computed with the "
9147                                "artifact hash field still blank")
9148
9149
9150 def verify_two_pass(manifest_path: str | Path, artifact_path: str | Path)
9151                    -> dict[str, Any]:
9152     m = json.loads(Path(manifest_path).read_text())
9153     actual = sha256_file(artifact_path)
9154     return {"manifest": str(manifest_path), "artifact": str(artifact_path),
9155            "recorded_artifact_digest": m.get("artifact_digest"),
9156            "actual_artifact_digest": actual,
9157            "match": m.get("artifact_digest") == actual,
9158            "source_digest": m.get("source_digest"),
9159            "pass1_digest": m.get("pass1_digest")}
9160
9161
9162 def reproduction_script(model_id: str, method: str,
9163                       parameters: Mapping[str, Any]) -> str:
9164     """Emit the minimal independent script that reproduces one claim (control 7)."""
9165     return (
9166         "#!/usr/bin/env python3\n"
9167         f"# Minimal reproduction for {model_id} / {method}\n"
9168         "# CTC-FA v2.0 -- run from the repository root with src/ on PYTHONPATH.\n"
9169         "from ctcfa import registry as R\n"
9170         "from ctcfa.causality import compact_orbit_test, certify_temporal_function\n"
9171         "from ctcfa.quotient_ctc import deck_orbit_test\n"
9172         "\n"
9173         f"model = R.get({model_id!r})\n"
9174         f"values = {dict(parameters)!r}\n"
9175         f"method = {method!r}\n"
9176         "if method == 'compact_orbit':\n"
9177         "    print(compact_orbit_test(model, values))\n"
9178         "elif method == 'deck_orbit':\n"
9179         "    print(deck_orbit_test(model, values))\n"
9180         "elif method.startswith('temporal_function'):\n"
9181         "    print(certify_temporal_function(model))\n"
9182         "else:\n"
9183         "    raise SystemExit(f'unknown method {method!r})\n")
9184
9185
9186 __all__ = ["sha256_bytes", "sha256_file", "sha256_text", "source_tree_digest",
9187           "environment_lock", "RunRecord", "new_run", "write_run",
9188           "seal_answers", "check_answer", "ReleaseManifest", "build_manifest",
9189           "verify_two_pass", "reproduction_script"]

```

src/ctcfa/report.py — 258 lines

```

9190 """
9191 CTC-FA v2.0 -- Reporting layer.
9192
9193 The reporting layer is a *gate*, not a formatter. It accepts only
9194 :class:`~ctcfa.status.Verdict` objects, refuses to print a claim above the
9195 level the gate record supports, and always prints the scope. A caller that
9196 tries to render a stronger statement gets :class:`~OverclaimError`.
9197
9198 It also produces the two aggregate objects the architecture needs:
9199
9200     ``analysis(model)`` the full per-model dossier across every layer;
9201     ``regression_matrix()`` the reproducible table that replaces the v1.0
9202                             self-test, with the *derivation route* recorded
9203                             for every row.
9204 """
9205
9206 from __future__ import annotations
9207
9208 import json
9209 from dataclasses import dataclass, field
9210 from typing import Any, Iterable, Mapping, Sequence
9211
9212 import numpy as np
9213 import sympy as sp
9214
9215 from .causality import (certify_temporal_function, compact_orbit_test,
9216                         compact_orbit_verdict, global_predicate_verdict,
9217                         temporal_verdict)
9218 from .model import MetricModel
9219 from .quotient_ctc import deck_orbit_test, deck_orbit_verdict
9220 from .status import (CLAIM_TEXT, ClaimLevel, ClaimScope, CTCStatus, Gate,
9221                     GateRecord, GateResult, OrbitStatus, OverclaimError, Verdict)
9222
9223 # -----
9224 # Route selection
9225 # -----
9226
9227 def primary_route(model: MetricModel) -> str:
9228     """Which engine is the appropriate primary detector for this model?"""
9229     if not model.has_metric:
9230         return "global_predicate"
9231     if model.compact:
9232         return "compact_orbit"
9233     if model.quotient.generators and model.metadata.get("flat_covering"):
9234         return "deck_orbit"
9235     if model.temporal_function is not None:
9236         return "temporal_function"
9237     return "none"
9238
9239
9240 def analyse(model: MetricModel, values: Mapping[str, float] | None = None,
9241            include_energy: bool = False,
9242            include_theorems: bool = False) -> dict[str, Any]:
9243     """Full per-model dossier."""
9244     values = dict(values or model.default_point)
9245     route = primary_route(model)
9246     out: dict[str, Any] = {
9247         "model_id": model.id, "title": model.title, "kind": model.kind.value,
9248         "expected": model.expected.value,
9249         "expected_scope": model.expected_scope.value,
9250         "primary_route": route, "point": values,
9251         "provenance": {"refs": list(model.provenance.source_refs),
9252                       "convention": model.provenance.convention_note},
9253         "domain_notes": model.domain_notes,
9254     }
9255
9256     verdicts: list[Verdict] = []
9257     if route == "compact_orbit":
9258         v = compact_orbit_verdict(model, values)
9259         verdicts.append(v)
9260         out["compact_orbit"] = v.as_dict()
9261     elif route == "deck_orbit":
9262         v = deck_orbit_verdict(model, values)
9263         verdicts.append(v)
9264         out["deck_orbit"] = v.as_dict()
9265     elif route == "global_predicate":
9266         if model.global_margin is not None:
9267             v = global_predicate_verdict(model, values)
9268             verdicts.append(v)
9269             out["global_predicate"] = v.as_dict()
9270         else:
9271             out["global_predicate"] = {
9272                 "status": CTCStatus.UNRESOLVED.value,
9273                 "reason": "no local metric and no registered predicate: this "
9274                           "entry is an external exact specification"}
9275
9276     if model.temporal_function is not None and model.has_metric:
9277         try:
9278             c = certify_temporal_function(model)
9279             out["temporal_certificate"] = c.as_dict()
9280             if c.proved:
9281                 verdicts.append(temporal_verdict(model))
9282         except Exception as exc:
9283             # noqa: BLE001
9284             out["temporal_certificate"] = {"error": str(exc)}
9285
9286     if model.quotient.generators and model.has_metric \
9287        and model.metadata.get("flat_covering") and route != "deck_orbit":
9288         try:
9289             d = deck_orbit_test(model, values)
9290             out["deck_orbit_secondary"] = d.as_evidence() | {
9291                 "status": d.status.value, "margin": d.margin}
9292         except Exception as exc:
9293             # noqa: BLE001
9294             out["deck_orbit_secondary"] = {"error": str(exc)}

```



```

9295     if include_energy and model.has_metric:
9296         try:
9297             from .energy import energy_report
9298             out["energy"] = energy_report(model, values).as_dict()
9299         except Exception as exc:
9300             out["energy"] = {"error": str(exc)}
9301
9302     if include_theorems:
9303         from .theorems import applicability_matrix
9304         out["theorems"] = applicability_matrix(model)
9305
9306     out["verdicts"] = [v.as_dict() for v in verdicts]
9307     out["headline"] = headline(model, verdicts)
9308     return out
9309
9310
9311 def headline(model: MetricModel, verdicts: Sequence[Verdict]) -> dict[str, Any]:
9312     """Combine the verdicts, respecting the evidence asymmetry.
9313
9314     A positive witness wins over an unresolved search. A negative certificate
9315     wins only when nothing positive was found, and only at its own scope.
9316     """
9317     positive = [v for v in verdicts if v.status is CTCStatus.CTC_PROVED]
9318     critical = [v for v in verdicts if v.status is CTCStatus.CRITICAL]
9319     negative = [v for v in verdicts if v.status is CTCStatus.NO_CTC_CERTIFIED]
9320     if positive:
9321         v = max(positive, key=lambda x: (x.scope.value, -x.margin))
9322     elif negative and not critical:
9323         v = negative[0]
9324     elif critical:
9325         v = critical[0]
9326     elif verdicts:
9327         v = verdicts[0]
9328     else:
9329         return {"status": CTCStatus.UNRESOLVED.value, "scope": "SECTOR",
9330               "claim_level": ClaimLevel.L0_PARSED.name,
9331               "line": f"{model.id}: UNRESOLVED (no applicable route)"}
9332     return {"status": v.status.value, "scope": v.scope.value,
9333           "claim_level": v.permitted_level().name,
9334           "claim_text": CLAIM_TEXT[v.permitted_level()],
9335           "margin": v.margin, "line": v.render()}
9336
9337
9338 # -----
9339 # Regression matrix
9340 # -----
9341
9342 def regression_matrix(model_ids: Sequence[str] | None = None) -> list[dict[str, Any]]:
9343     """The reproducible replacement for the v1.0 self-test table.
9344
9345     Each row records the route that produced it, so a reader can tell a
9346     sector result from a global one at a glance -- the single most important
9347     piece of information the v1.0 table omitted.
9348     """
9349     from .registry import get, models
9350     ids = list(model_ids) if model_ids else [m.id for m in models()]
9351     rows = []
9352     for mid in ids:
9353         m = get(mid)
9354         route = primary_route(m)
9355         expected = (m.expected_at_default or m.expected).value
9356         row: dict[str, Any] = {"model_id": mid, "route": route,
9357                               "expected": expected,
9358                               "family_expected": m.expected.value,
9359                               "kind": m.kind.value,
9360                               "expected_scope": m.expected_scope.value}
9361         try:
9362             if route == "compact_orbit":
9363                 r = compact_orbit_test(m, m.default_point)
9364                 row.update({"actual": r.status.value, "margin": r.margin,
9365                             "uncertainty": r.uncertainty,
9366                             "winding": list(r.winding) if r.winding else None})
9367             elif route == "deck_orbit":
9368                 r = deck_orbit_test(m, m.default_point)
9369                 row.update({"actual": r.status.value, "margin": r.margin,
9370                             "uncertainty": r.uncertainty,
9371                             "word": list(r.word) if r.word else None})
9372             elif route == "global_predicate" and m.global_margin is not None:
9373                 v = global_predicate_verdict(m, m.default_point or {})
9374                 row.update({"actual": v.status.value, "margin": v.margin,
9375                             "scope": v.scope.value})
9376             elif route == "temporal_function":
9377                 c = certify_temporal_function(m)
9378                 row.update({"actual": c.status.value, "margin": float("nan"),
9379                             "scope": c.scope.value})
9380             else:
9381                 row.update({"actual": "NOT_APPLICABLE", "margin": float("nan")})
9382         except Exception as exc:
9383             row.update({"actual": f"ERROR: {exc}", "margin": float("nan")})
9384         rows.append(row)
9385     return rows
9386
9387
9388 def format_matrix(rows: Sequence[Mapping[str, Any]]) -> str:
9389     w = max(len(str(r["model_id"])) for r in rows) + 2
9390     out = [f"{'model':<{w}}{'route':<18}{'expected':<22}{'actual':<22}"
9391            f"{'margin':>16} uncertainty"]
9392     out.append("-" * (w + 18 + 22 + 22 + 16 + 14))
9393     for r in rows:
9394         m = r.get("margin")
9395         ms = "-" if m is None or (isinstance(m, float) and not np.isfinite(m)) \
9396             else f"{m:+.6e}"
9397         u = r.get("uncertainty")
9398         us = "-" if u is None else f"{u:.2e}"
9399         out.append(f"{r['model_id']:<{w}}{r['route']:<18}{r['expected']:<22}"
9400                  f"{str(r['actual']:<22)}{ms:>16} {us}")
9401     return "\n".join(out)
9402
9403
9404 def matrix_agreement(rows: Sequence[Mapping[str, Any]]) -> dict[str, Any]:

```



```

9405 """Does each row's computed status match the registry expectation?"""
9406 ok, bad = 0, []
9407 for r in rows:
9408     exp, act = r["expected"], str(r["actual"])
9409     # An EXTERNAL_EXACT_SPEC entry has no local tensor by design: the
9410     # architecture is *supposed* to answer NOT_APPLICABLE for it rather
9411     # than fabricate a metric (v1.0 REQ-016).
9412     if r.get("kind") in ("EXTERNAL_EXACT_SPEC",) and act == "NOT_APPLICABLE":
9413         ok += 1
9414         continue
9415     agree = (
9416         (exp == "CTC_PROVED" and act in ("SECTOR_CTC", "CTC_PROVED"))
9417         or (exp == "NO_CTC_CERTIFIED" and act in ("SECTOR_SPACELIKE",
9418             "NO_CTC_CERTIFIED"))
9419         or (exp == "CRITICAL" and act in ("SECTOR_NULL", "CRITICAL"))
9420         or (exp == "UNRESOLVED" and act in ("SECTOR_SPACELIKE",
9421             "SECTOR_UNRESOLVED",
9422             "UNRESOLVED", "NOT_APPLICABLE",
9423             "SECTOR_NULL"))
9424     )
9425     if agree:
9426         ok += 1
9427     else:
9428         bad.append({"model_id": r["model_id"], "expected": exp,
9429             "actual": act, "route": r.get("route")})
9430 return {"n": len(rows), "agree": ok, "disagree": len(bad),
9431     "pass": not bad, "mismatches": bad}
9432
9433
9434 def to_json(obj: Any) -> str:
9435     def default(o: Any) -> Any:
9436         if isinstance(o, (np.floating, np.integer)):
9437             return o.item()
9438         if isinstance(o, np.ndarray):
9439             return o.tolist()
9440         if isinstance(o, sp.Basic):
9441             return str(o)
9442         return str(o)
9443     return json.dumps(obj, indent=2, sort_keys=True, default=default)
9444
9445
9446 __all__ = ["primary_route", "analyse", "headline", "regression_matrix",
9447     "format_matrix", "matrix_agreement", "to_json"]

```

src/ctcfa/cli.py — 291 lines

```

9448 """
9449 CTC-FA v2.0 -- command line interface.
9450
9451 Section 26.1 of v1.0 specified a CLI and then noted: "The CLI above is a
9452 normative target interface, not a claim that every command already exists in
9453 the single-file v1.0 kernel." In v2.0 every command below exists and runs.
9454
9455     ctcfa list
9456     ctcfa show <model>
9457     ctcfa validate [<model>] [--no-cross-check]
9458     ctcfa tensors <model> [--point k=v ...] [--what ricci|einstein|kretschmann]
9459     ctcfa ctc <model> [--point k=v ...]
9460     ctcfa certify-time <model> [--box r=lo:hi ...]
9461     ctcfa boundary <model> --parameter p --bracket lo:hi [--fixed k=v ...]
9462         [--box r=lo:hi ...] [--analytic X]
9463     ctcfa energy <model> [--point k=v ...]
9464     ctcfa horizons <model> [--variable r]
9465     ctcfa theorems <model>
9466     ctcfa quotient <model>
9467     ctcfa matrix
9468     ctcfa meta
9469     ctcfa manifest --out results/manifest.json
9470     ctcfa export-registry --out registry/metrics
9471     ctcfa selftest
9472 """
9473
9474 from __future__ import annotations
9475
9476 import argparse
9477 import json
9478 import sys
9479 from pathlib import Path
9480 from typing import Any, Mapping
9481
9482 import numpy as np
9483 import sympy as sp
9484
9485
9486 def _kv(pairs: list[str] | None) -> dict[str, float]:
9487     out: dict[str, float] = {}
9488     for p in pairs or []:
9489         if "=" not in p:
9490             raise SystemExit(f"expected key=value, got {p!r}")
9491         k, v = p.split("=", 1)
9492         out[k.strip()] = float(sp.N(sp.sympify(v.strip()))))
9493     return out
9494
9495
9496 def _boxes(pairs: list[str] | None) -> dict[str, tuple[float, float]]:
9497     out: dict[str, tuple[float, float]] = {}
9498     for p in pairs or []:
9499         k, rng = p.split("=", 1)
9500         lo, hi = rng.split(":", 1)
9501         out[k.strip()] = (float(sp.N(sp.sympify(lo))), float(sp.N(sp.sympify(hi))))
9502     return out
9503
9504
9505 def _dump(obj: Any) -> None:
9506     from .report import to_json
9507     print(to_json(obj))
9508
9509

```

```

9510 def main(argv: list[str] | None = None) -> int:
9511     ap = argparse.ArgumentParser(prog="ctcfa",
9512                                 description="CTC-FA v2.0 Closed Timelike Curve "
9513                                             "Falsification Architecture")
9514     sub = ap.add_subparsers(dest="cmd", required=True)
9515
9516     sub.add_parser("list", help="list registered models")
9517
9518     p = sub.add_parser("show", help="show one registry record")
9519     p.add_argument("model")
9520
9521     p = sub.add_parser("validate", help="run the V0/V1/V2/V3 gates")
9522     p.add_argument("model", nargs="?")
9523     p.add_argument("--no-cross-check", action="store_true")
9524
9525     p = sub.add_parser("tensors", help="curvature quantities")
9526     p.add_argument("model")
9527     p.add_argument("--point", action="append")
9528     p.add_argument("--what", default="ricci",
9529                   choices=["ricci", "ricci_scalar", "einstein", "kretschmann"])
9530
9531     p = sub.add_parser("ctc", help="run the appropriate CTC detector")
9532     p.add_argument("model")
9533     p.add_argument("--point", action="append")
9534     p.add_argument("--energy", action="store_true")
9535     p.add_argument("--theorems", action="store_true")
9536
9537     p = sub.add_parser("certify-time", help="temporal-function certificate")
9538     p.add_argument("model")
9539     p.add_argument("--box", action="append")
9540
9541     p = sub.add_parser("boundary", help="causal phase-boundary continuation")
9542     p.add_argument("model")
9543     p.add_argument("--parameter", required=True)
9544     p.add_argument("--bracket", required=True, help="lo:hi")
9545     p.add_argument("--fixed", action="append")
9546     p.add_argument("--box", action="append")
9547     p.add_argument("--analytic", type=float, default=None)
9548
9549     p = sub.add_parser("energy", help="energy-condition report")
9550     p.add_argument("model")
9551     p.add_argument("--point", action="append")
9552
9553     p = sub.add_parser("horizons", help="horizon taxonomy")
9554     p.add_argument("model")
9555     p.add_argument("--variable", default="r")
9556     p.add_argument("--no-curvature", action="store_true")
9557
9558     p = sub.add_parser("theorems", help="theorem applicability matrix")
9559     p.add_argument("model")
9560
9561     p = sub.add_parser("quotient", help="L3 identification certificate")
9562     p.add_argument("model")
9563
9564     sub.add_parser("matrix", help="regression matrix over the whole registry")
9565     sub.add_parser("meta", help="cross-family meta-analysis")
9566
9567     p = sub.add_parser("manifest", help="build a release manifest")
9568     p.add_argument("--out", default="results/manifest.json")
9569     p.add_argument("--root", default=None)
9570
9571     p = sub.add_parser("export-registry", help="write machine-readable records")
9572     p.add_argument("--out", default="registry/metrics")
9573
9574     sub.add_parser("selftest", help="full self-test (validation + matrix)")
9575
9576     a = ap.parse_args(argv)
9577
9578     from . import __architect__, __document_id__, __version__
9579     from . import registry as R
9580
9581     if a.cmd == "list":
9582         from .registry import summary
9583         rows = summary()
9584         w = max(len(r["model_id"]) for r in rows) + 2
9585         print(f"CTC-FA v{__version__} | {__document_id__} | "
9586               f"system architect: {__architect__}")
9587         print(f"{'model':<{w}}{'kind':<22}{'expected':<20}{'scope':<18}refs")
9588         print("-" * (w + 22 + 20 + 18 + 12))
9589         for r in rows:
9590             print(f"{'model_id':<{w}}{'kind':<22}{'expected':<20}"
9591                   f"{'scope':<18}{'refs':<18}")
9592         print(f"\n{len(rows)} registered models.")
9593         return 0
9594
9595     if a.cmd == "show":
9596         _dump(R.get(a.model).as_record())
9597         return 0
9598
9599     if a.cmd == "validate":
9600         from .validate import validate_model, validate_registry
9601         if a.model:
9602             _dump(validate_model(R.get(a.model),
9603                                 cross_check=not a.no_cross_check).as_dict())
9604         else:
9605             rep = validate_registry(cross_check=not a.no_cross_check)
9606             _dump({"n_models": rep["n_models"], "n_failed": rep["n_failed"],
9607                   "pass": rep["pass"],
9608                   "gates": {r["model_id"]: r["gates"]["results"]
9609                             for r in rep["reports"]}})
9610             return 0
9611
9612     if a.cmd == "tensors":
9613         from .symbolic import (einstein_tensor, kretschmann, ricci_scalar,
9614                               ricci_tensor)
9615         m = R.get(a.model)
9616         g, c = m.require_metric(), m.coords
9617         if a.what == "ricci":
9618             res = ricci_tensor(g, c)
9619         elif a.what == "ricci_scalar":

```

```

9620         res = ricci_scalar(g, c)
9621     elif a.what == "einstein":
9622         res = einstein_tensor(g, c)
9623     else:
9624         res = kretschmann(g, c)
9625     pt = _kv(a.point)
9626     if pt:
9627         res = sp.Matrix(res).subs(m.substitution(pt)).evalf(20) \
9628             if hasattr(res, "shape") else sp.N(res.subs(m.substitution(pt)), 20)
9629     print(sp.pretty(res) if hasattr(res, "shape") else res)
9630     return 0
9631
9632     if a.cmd == "ctc":
9633         from .report import analyse
9634         m = R.get(a.model)
9635         pt = _kv(a.point) or dict(m.default_point)
9636         _dump(analyse(m, pt, include_energy=a.energy,
9637                     include_theorems=a.theorems))
9638         return 0
9639
9640     if a.cmd == "certify-time":
9641         from .causality import certify_temporal_function
9642         m = R.get(a.model)
9643         box = {m.symbol(k): v for k, v in _boxes(a.box).items()} or None
9644         _dump(certify_temporal_function(m, box=box).as_dict())
9645         return 0
9646
9647     if a.cmd == "boundary":
9648         from .continuation import causal_phase_boundary
9649         m = R.get(a.model)
9650         lo, hi = a.bracket.split(":", 1)
9651         th = causal_phase_boundary(m, a.parameter,
9652                                   (float(lo), float(hi)), _kv(a.fixed),
9653                                   _boxes(a.box), analytic_value=a.analytic)
9654         _dump(th.as_dict())
9655         return 0
9656
9657     if a.cmd == "energy":
9658         from .energy import energy_report
9659         m = R.get(a.model)
9660         _dump(energy_report(m, _kv(a.point) or None).as_dict())
9661         return 0
9662
9663     if a.cmd == "horizons":
9664         from .horizons import horizon_report
9665         _dump(horizon_report(R.get(a.model), a.variable,
9666                             with_curvature=not a.no_curvature))
9667         return 0
9668
9669     if a.cmd == "theorems":
9670         from .theorems import applicability_matrix
9671         _dump(applicability_matrix(R.get(a.model)))
9672         return 0
9673
9674     if a.cmd == "quotient":
9675         from .topology import classify, verify_quotient
9676         m = R.get(a.model)
9677         _dump({"classification": classify(m),
9678              "certificate": verify_quotient(m).as_dict()})
9679         return 0
9680
9681     if a.cmd == "matrix":
9682         from .report import format_matrix, matrix_agreement, regression_matrix
9683         rows = regression_matrix()
9684         print(format_matrix(rows))
9685         ag = matrix_agreement(rows)
9686         print(f"\nagreement {ag['agree']}/{ag['n']} pass={ag['pass']}")
9687         for mm in ag["mismatches"]:
9688             print(" MISMATCH", mm)
9689         return 0 if ag["pass"] else 1
9690
9691     if a.cmd == "meta":
9692         from .meta import cross_family_report
9693         _dump(cross_family_report(R.models()))
9694         return 0
9695
9696     if a.cmd == "manifest":
9697         from .provenance import build_manifest
9698         root = Path(a.root) if a.root else Path(__file__).resolve().parent
9699         man = build_manifest(root, __document_id__, __version__, __architect__,
9700                             "2026-08-31")
9701         man.write(a.out)
9702         print(f"source digest: {man.source_digest}")
9703         print(f"files: {len(man.files)}")
9704         print(f"manifest written to {a.out}")
9705         return 0
9706
9707     if a.cmd == "export-registry":
9708         paths = R.export_records(a.out)
9709         print(f"{len(paths)} records written to {a.out}")
9710         return 0
9711
9712     if a.cmd == "selftest":
9713         from .registry import registry_audit
9714         from .report import format_matrix, matrix_agreement, regression_matrix
9715         from .validate import validate_registry
9716         print(f"CTC-FA v{__version__} self-test")
9717         aud = registry_audit()
9718         print(f" registry audit : {'PASS' if aud['pass'] else 'FAIL'} "
9719               f"f'({aud['n_models']} models, {aud['n_problems']} problems)")
9720         val = validate_registry(cross_check=True)
9721         print(f" V0/V1 gates : {'PASS' if val['pass'] else 'FAIL'} "
9722               f"f'({val['n_models']} - val['n_failed']}/{val['n_models']}")
9723         rows = regression_matrix()
9724         ag = matrix_agreement(rows)
9725         print(f" regression : {'PASS' if ag['pass'] else 'FAIL'} "
9726               f"f'({ag['agree']}/{ag['n']})")
9727         print()
9728         print(format_matrix(rows))
9729         ok = aud["pass"] and val["pass"] and ag["pass"]

```

```
9730         print(f"\nCTC-FA v{__version__} self-test: {'PASS' if ok else 'FAIL'}")
9731         return 0 if ok else 1
9732
9733     ap.error(f"unknown command {a.cmd}")
9734     return 2
9735
9736
9737 if __name__ == "__main__":
9738     sys.exit(main())
```

A.2 Tests

tests/conftest.py — 15 lines

```

9739 import sys
9740 from pathlib import Path
9741
9742 ROOT = Path(__file__).resolve().parents[1]
9743 SRC = ROOT / "src"
9744 if str(SRC) not in sys.path:
9745     sys.path.insert(0, str(SRC))
9746
9747 import pytest
9748
9749
9750 @pytest.fixture(scope="session")
9751 def registry():
9752     from ctcfactory import registry as R
9753     return R

```

tests/unit/test_conventions.py — 119 lines

```

9754 """Convention pinning: the sign and normalisation choices that a drift would
9755 silently corrupt. Each test here corresponds to a v1.0 defect or failure
9756 mode that could otherwise recur unnoticed."""
9757
9758 import numpy as np
9759 import pytest
9760 import sympy as sp
9761
9762 from ctcfactory import registry as R
9763 from ctcfactory.symbolic import (christoffel, einstein_tensor, kretschmann,
9764                                 ricci_scalar, ricci_tensor)
9765
9766
9767 def test_schwarzschild_is_ricci_flat():
9768     m = R.get("schwarzschild")
9769     assert ricci_tensor(m.metric, m.coords).is_zero_matrix
9770
9771
9772 def test_schwarzschild_kretschmann_exact():
9773     """K = 48 M^2 / r^6 pins the Riemann normalisation and every index order."""
9774     m = R.get("schwarzschild")
9775     M, r = m.symbol("M"), m.symbol("r")
9776     K = kretschmann(m.metric, m.coords)
9777     assert sp.simplify(K - 48 * M ** 2 / r ** 6) == 0
9778
9779
9780 def test_de_sitter_ricci_scalar_is_four_lambda():
9781     """R = 4 Lambda pins the Einstein-tensor sign convention in 4D."""
9782     m = R.get("de_sitter_static")
9783     L = m.symbol("L")
9784     R4 = ricci_scalar(m.metric, m.coords)
9785     assert sp.simplify(R4 - 4 * (3 / L ** 2)) == 0
9786
9787
9788 def test_ads_ricci_scalar_is_negative():
9789     m = R.get("ads4_universal_cover")
9790     L = m.symbol("L")
9791     R4 = ricci_scalar(m.metric, m.coords)
9792     assert sp.simplify(R4 - 4 * (-3 / L ** 2)) == 0
9793
9794
9795 def test_kerr_g_t_phi_sign_is_negative():
9796     """v1.0 defect D-01.
9797
9798     In Boyer-Lindquist with the (-,+,+,+) signature the frame-dragging term is
9799     -(4 M a r sin^2 theta / Sigma) dt dphi, so g_t_phi < 0 for
9800     M, a, r > 0. The v1.0 registry table printed the opposite sign.
9801     """
9802     m = R.get("kerr")
9803     sub = {m.symbol("M"): 1.0, m.symbol("a"): 0.9, m.symbol("r"): 4.0,
9804            m.symbol("theta"): np.pi / 2}
9805     gtph = float(sp.N(m.metric[0, 3].subs(sub)))
9806     assert gtph < 0
9807     assert abs(gtph - (-2 * 1.0 * 0.9 * 4.0 * 1.0 / 16.0)) < 1e-12
9808
9809
9810 def test_kerr_reduces_to_schwarzschild_when_a_zero():
9811     m = R.get("kerr")
9812     a = m.symbol("a")
9813     g = sp.simplify(m.metric.subs(a, 0))
9814     s = R.get("schwarzschild")
9815     assert sp.simplify(g - s.metric).is_zero_matrix
9816
9817
9818 def test_kerr_newman_reduces_to_kerr_when_q_zero():
9819     kn = R.get("kerr_newman")
9820     g = sp.simplify(kn.metric.subs(kn.symbol("Q"), 0))
9821     k = R.get("kerr")
9822     assert sp.simplify(g - k.metric).is_zero_matrix
9823
9824
9825 def test_reissner_nordstrom_reduces_to_schwarzschild():
9826     rn = R.get("reissner_nordstrom")
9827     g = sp.simplify(rn.metric.subs(rn.symbol("Q"), 0))
9828     assert sp.simplify(g - R.get("schwarzschild").metric).is_zero_matrix
9829
9830
9831 def test_godel_is_the_m2_equals_2omega2_member():
9832     base = R.get("godel_type")
9833     om = base.symbol("omega")
9834     target = sp.simplify(base.metric.subs(base.symbol("m"), sp.sqrt(2) * om))
9835     assert sp.simplify(R.get("godel").metric - target).is_zero_matrix
9836
9837

```

```

9838 def test_krasnikov_tx_block_determinant_is_minus_one():
9839     """An exact algebraic identity of the Everett-Roman form: det of the (t,x)
9840     block equals -1 for every value of the regulator, so the chart is
9841     Lorentzian everywhere and the tube is not a chart pathology."""
9842     m = R.get("krasnikov")
9843     g = m.metric
9844     det2 = sp.simplify(g[0, 0] * g[1, 1] - g[0, 1] * g[1, 0])
9845     assert sp.simplify(det2 + 1) == 0
9846
9847
9848 def test_misner_determinant_is_minus_one_everywhere():
9849     m = R.get("misner")
9850     assert sp.simplify(m.metric.det() + 1) == 0
9851
9852
9853 def test_van_stockum_metric_is_symmetric():
9854     """v1.0 defect D-02: the kernel stored an asymmetric van Stockum matrix."""
9855     m = R.get("van_stockum")
9856     assert sp.simplify(m.metric - m.metric.T).is_zero_matrix
9857
9858
9859 def test_every_registered_metric_is_symmetric():
9860     for m in R.models():
9861         if m.has_metric:
9862             assert sp.simplify(m.metric - m.metric.T).is_zero_matrix, m.id
9863
9864
9865 def test_van_stockum_two_block_determinant():
9866     """det of the (t,phi) block is -r^2, so the chart stays Lorentzian even
9867     where g_phi_phi < 0 -- the CTC region is not a signature failure."""
9868     m = R.get("van_stockum")
9869     r = m.symbol("r")
9870     g = m.metric
9871     det2 = sp.simplify(g[0, 0] * g[2, 2] - g[0, 2] * g[2, 0])
9872     assert sp.simplify(det2 + r ** 2) == 0

```

tests/unit/test_status_and_gates.py — 79 lines

```

9873 """The evidence logic itself: scope algebra, claim ladder, overclaim refusal."""
9874
9875 import pytest
9876
9877 from ctcf.status import (ClaimLevel, ClaimScope, CTCStatus, Gate, GateRecord,
9878                          GateResult, OrbitResult, OrbitStatus, OverclaimError,
9879                          Verdict)
9880
9881
9882 def _gates(*passed: Gate) -> GateRecord:
9883     g = GateRecord()
9884     for p in passed:
9885         g.set(p, GateResult.PASS)
9886     return g
9887
9888
9889 def test_sector_spacelike_never_becomes_no_ctc_certified():
9890     """v1.0 REQ-005 / FM-002, now a type-level guarantee."""
9891     r = OrbitResult(OrbitStatus.SECTOR_SPACELIKE, 25.0, {1}, {}, "clear sector")
9892     v = r.to_verdict("schwarzschild")
9893     assert v.status is CTCStatus.UNRESOLVED
9894     assert v.status is not CTCStatus.NO_CTC_CERTIFIED
9895
9896
9897 def test_sector_null_maps_to_critical():
9898     r = OrbitResult(OrbitStatus.SECTOR_NULL, 0.0, {1}, {}, "marginal")
9899     assert r.to_verdict("misner").status is CTCStatus.CRITICAL
9900
9901
9902 def test_unresolved_cannot_be_promoted():
9903     v = Verdict(CTCStatus.UNRESOLVED, ClaimScope.SECTOR, float("nan"),
9904                "search", "nothing found")
9905     with pytest.raises(OverclaimError):
9906         v.promote(ClaimScope.GLOBAL, "certificate")
9907
9908
9909 def test_negative_result_needs_a_certificate_to_go_global():
9910     v = Verdict(CTCStatus.NO_CTC_CERTIFIED, ClaimScope.DECLARED_DOMAIN, 1.0,
9911                "temporal_function", "ok")
9912     with pytest.raises(OverclaimError):
9913         v.promote(ClaimScope.GLOBAL, "looks fine to me")
9914     ok = v.promote(ClaimScope.GLOBAL, "verified temporal-function certificate")
9915     assert ok.scope is ClaimScope.GLOBAL
9916
9917
9918 def test_claim_ladder_blocks_unsupported_levels():
9919     v = Verdict(CTCStatus.CTC_PROVED, ClaimScope.SECTOR, -1.0, "compact_orbit",
9920                "witness", "godel",
9921                gates=_gates(Gate.V0_PARSE, Gate.V1_ALGEBRA, Gate.V4_CTC_WITNESS))
9922     assert v.permitted_level() is ClaimLevel.L2_CTC_VERIFIED
9923     v.render() # allowed at its own level
9924     with pytest.raises(OverclaimError):
9925         v.render(ClaimLevel.L6_CONSTRUCTIBILITY)
9926     with pytest.raises(OverclaimError):
9927         v.render(ClaimLevel.L7_STABILITY)
9928
9929
9930 def test_l9_is_unreachable_by_construction():
9931     g = GateRecord()
9932     for gate in Gate:
9933         g.set(gate, GateResult.PASS)
9934     assert g.highest_permitted_level() is not ClaimLevel.L9_QUANTUM_GRAVITY
9935
9936
9937 def test_scope_composition_never_strengthens():
9938     v = Verdict(CTCStatus.CTC_PROVED, ClaimScope.GLOBAL, -1.0, "x", "y")
9939     assert v.restrict(ClaimScope.SECTOR).scope is ClaimScope.SECTOR
9940     assert v.restrict(ClaimScope.SECTOR).restrict(
9941         ClaimScope.GLOBAL).scope is ClaimScope.SECTOR
9942

```

```

9943
9944 def test_verdict_render_includes_scope_and_level():
9945     v = Verdict(CTCStatus.CTC_PROVED, ClaimScope.SECTOR, -1.0, "compact_orbit",
9946                 "witness", "kerr",
9947                 gates=gates(Gate.V0_PARSE, Gate.V1_ALGEBRA, Gate.V4_CTC_WITNESS))
9948     line = v.render()
9949     assert "scope=SECTOR" in line
9950     assert "level=L2_CTC_VERIFIED" in line
9951     assert "CTC_PROVED" in line

```

tests/unit/test_engines.py — 143 lines

```

9952 """Unit tests for the individual engines."""
9953
9954 import math
9955
9956 import numpy as np
9957 import pytest
9958 import sympy as sp
9959
9960 from ctcfactory import registry as R
9961 from ctcfactory.energy import (classify_hawking_ellis, min_quadratic_on_sphere,
9962                               orthonormal_frame)
9963 from ctcfactory.interval import global_lower_bound, interval_eval, prove_positive
9964 from ctcfactory.numerics import (ErrorBudget, SignClass, classify_sign,
9965                                 signature_counts)
9966 from ctcfactory.numgeom import NumericGeometry
9967 from ctcfactory.orientation import check_curve_orientation, timelike_eigenvector
9968 from ctcfactory.symmetry import solve_killing, verify_compact_generators
9969
9970 # -- numerics -----
9971
9972 def test_sign_classification_uses_the_uncertainty_band():
9973     b = ErrorBudget(floor=1e-12)
9974     assert classify_sign(-1e-15, b) is SignClass.MARGINAL
9975     assert classify_sign(-1e-6, b) is SignClass.NEGATIVE
9976     assert classify_sign(+1e-6, b) is SignClass.POSITIVE
9977
9978 def test_signature_counts_are_congruence_invariant():
9979     g = np.diag([-1.0, 1.0, 1.0, 1.0])
9980     rng = np.random.default_rng(3)
9981     for _ in range(20):
9982         A = rng.normal(size=(4, 4))
9983         if abs(np.linalg.det(A)) < 1e-6:
9984             continue
9985         assert signature_counts(A.T @ g @ A) == (1, 0, 3)
9986
9987 # -- interval arithmetic -----
9988
9989 def test_interval_enclosure_contains_the_true_range():
9990     x = sp.Symbol("x")
9991     e = sp.sin(x) * x ** 2
9992     enc = interval_eval(e, {x: (0.0, 2.0)})
9993     xs = np.linspace(0, 2, 5000)
9994     vals = np.sin(xs) * xs ** 2
9995     assert float(enc.a) <= vals.min() + 1e-12
9996     assert float(enc.b) >= vals.max() - 1e-12
9997
9998 def test_branch_and_bound_proves_and_refutes():
9999     x = sp.Symbol("x")
10000     assert prove_positive(1 + x ** 2, {x: (-3.0, 3.0)}).proved
10001     c = prove_positive(1 - x ** 2, {x: (-3.0, 3.0)})
10002     assert c.refuted and not c.proved
10003
10004 # -- exact trust-region solver -----
10005
10006 def test_trust_region_beats_dense_random_search():
10007     rng = np.random.default_rng(11)
10008     for _ in range(40):
10009         d = int(rng.integers(2, 5))
10010         A = rng.normal(size=(d, d))
10011         A = (A + A.T) / 2
10012         b = rng.normal(size=d)
10013         val, n = min_quadratic_on_sphere(A, b)
10014         assert abs(np.linalg.norm(n) - 1) < 1e-8
10015         V = rng.normal(size=(4000, d))
10016         V /= np.linalg.norm(V, axis=1, keepdims=True)
10017         best = float(np.min(np.einsum("ij,jk,ik->i", V, A, V) + 2 * V @ b))
10018         assert val <= best + 1e-9
10019
10020 def test_hawking_ellis_type_I_for_a_perfect_fluid():
10021     That = np.diag([2.0, 0.5, 0.5, 0.5])
10022     he = classify_hawking_ellis(That)
10023     assert he.type.value == "TYPE I"
10024     assert abs(he.rho - 2.0) < 1e-12
10025     assert all(abs(p - 0.5) < 1e-12 for p in he.pressures)
10026
10027 def test_orthonormal_frame_diagonalises_the_metric():
10028     for mid in ("kerr", "godel", "misner", "ori_2005"):
10029         m = R.get(mid)
10030         g = np.array(sp.Matrix(m.metric)
10031                     .subs(m.substitution(m.default_point))).evalf(), dtype=float)
10032         E = orthonormal_frame(g)
10033         eta = E.T @ g @ E
10034         assert np.allclose(eta, np.diag([-1, 1, 1, 1]), atol=1e-9), mid
10035
10036 # -- symmetry -----
10037
10038 def test_minkowski_has_the_full_poincare_algebra():
10039     sol = solve_killing(R.get("minkowski"), "linear")

```

```

10048     assert sol.dimension == 10
10049
10050
10051 def test_every_declared_compact_generator_is_killing():
10052     for m in R.models():
10053         if not m.has_metric:
10054             continue
10055         for cd, c in zip(m.compact, verify_compact_generators(m)):
10056             if cd.requires_killing:
10057                 assert c.is_killing, f"{m.id}: {c.name} -> {c.killing_residual[:120]}"
10058
10059
10060 # -- orientation -----
10061
10062 def test_orientation_detects_a_reversing_curve():
10063     g = np.diag([-1.0, 1.0, 1.0, 1.0])
10064     metrics = [g] * 8
10065     tangents = [np.array([1.0, 0, 0, 0])] * 4 + [np.array([-1.0, 0, 0, 0])] * 4
10066     res = check_curve_orientation(metrics, tangents, closed=False)
10067     assert not res.consistent
10068     assert res.reversals >= 1
10069
10070
10071 def test_orientation_accepts_a_consistent_curve():
10072     g = np.diag([-1.0, 1.0, 1.0, 1.0])
10073     metrics = [g] * 8
10074     tangents = [np.array([1.0, 0.1, 0, 0])] * 8
10075     assert check_curve_orientation(metrics, tangents, closed=False).consistent
10076
10077
10078 # -- numeric geometry -----
10079
10080 def test_numeric_curvature_matches_the_exact_kretschmann():
10081     m = R.get("schwarzschild")
10082     c = NumericGeometry(m).curvature(m.default_point)
10083     exact = 48 * 1.0 ** 2 / 5.0 ** 6
10084     assert abs(c.kretschmann - exact) / exact < 1e-7
10085
10086
10087 def test_numeric_and_symbolic_einstein_agree_on_a_hard_metric():
10088     """Alcubierre is the model the exact route could not simplify in v1.0."""
10089     from ctcfenergy import symbolic_einstein
10090     m = R.get("alcubierre")
10091     Ga = np.array(sp.Matrix(symbolic_einstein(m))
10092                   .subs(m.substitution(m.default_point))).evalf(30), dtype=float)
10093     Gb, err = NumericGeometry(m).einstein(m.default_point)
10094     assert np.max(np.abs(Ga - Gb)) < max(1e-3, 200 * err)

```

tests/analytic/test_analytic_benchmarks.py — 176 lines

```

10095 """Analytic benchmarks: every causal boundary the architecture claims is
10096 recovered from the metric and compared with the closed form afterwards.
10097
10098 The comparison direction matters. The solver never receives the analytic
10099 value; it is used only to score the result. ``test_solver_is_non_circular``
10100 asserts that the module which used to contain the hard-coded Godel answer
10101 contains no such function any more.
10102 """
10103
10104 import math
10105
10106 import numpy as np
10107 import pytest
10108 import sympy as sp
10109
10110 from ctcfenergy import registry as R
10111 from ctcfenergy.continuation import (causal_phase_boundary, convergence_study,
10112                                     reduced_causal_margin_symbolic)
10113 from ctcfenergy.quotient_ctc import (exact_orbit_interval, polarized_hypersurfaces,
10114                                     winding_spectrum)
10115
10116 # -- the reduced causal margin collapses to the literature quantities -----
10117
10118
10119 def test_godel_type_reduced_margin_is_the_tanh_form():
10120     m = R.get("godel_type")
10121     mu = reduced_causal_margin_symbolic(m)
10122     om, mm, r = m.symbol("omega"), m.symbol("m"), m.symbol("r")
10123     target = 1 - 4 * om ** 2 / mm ** 2 * sp.tanh(mm * r / 2) ** 2
10124     diff = sp.simplify((mu - target).rewrite(sp.exp))
10125     assert sp.simplify(diff) == 0
10126
10127
10128 def test_van_stockum_reduced_margin():
10129     m = R.get("van_stockum")
10130     a, r = m.symbol("a"), m.symbol("r")
10131     assert sp.simplify(reduced_causal_margin_symbolic(m) - (1 - a ** 2 * r ** 2)) == 0
10132
10133
10134 def test_spinning_string_reduced_margin():
10135     m = R.get("spinning_cosmic_string")
10136     J, al, r = m.symbol("J"), m.symbol("alpha"), m.symbol("r")
10137     target = 1 - 16 * J ** 2 / (al ** 2 * r ** 2)
10138     assert sp.simplify(reduced_causal_margin_symbolic(m) - target) == 0
10139
10140
10141 def test_ori2007_reduced_margin():
10142     m = R.get("ori2007_core")
10143     mu_m, r = m.symbol("mu"), m.symbol("r")
10144     assert sp.simplify(reduced_causal_margin_symbolic(m)
10145                       - (1 - 4 * mu_m ** 2 / r ** 2)) == 0
10146
10147
10148 # -- boundaries recovered from the metric -----
10149
10150 @pytest.mark.parametrize("mid,par,bracket,fixed,box,analytic", [
10151     ("van_stockum", "a", (0.05, 3.0), {}, {"r": (2.0, 2.0)}, 0.5),
10152     ("spinning_cosmic_string", "alpha", (0.5, 20.0), {"J": 0.1},

```



```

10153     {"r": (0.1, 0.1)}, 4.0),
10154     ("ori_2007_core", "r", (0.5, 6.0), {"mu": 1.0}, {}, 2.0),
10155     ("pseudo_reissner_nordstrom", "r", (0.125, 1.0), {"mu": 1.0, "q": 0.5}, {},
10156      1 - math.sqrt(0.75)),
10157     ("pseudo_reissner_nordstrom", "r", (1.0, 3.0), {"mu": 1.0, "q": 0.5}, {},
10158      1 + math.sqrt(0.75)),
10159     ("misner", "T", (-1.5, 1.5), {}, {}, 0.0),
10160     ("ori_2005", "T", (-1.5, 1.5), {"x": 0.0, "y": 0.0}, {}, 0.0),
10161 ]))
10162 def test_boundary_matches_closed_form(mid, par, bracket, fixed, box, analytic):
10163     th = causal_phase_boundary(R.get(mid), par, bracket, fixed, box,
10164                                analytic_value=analytic)
10165     assert th.domain_ok, th.note
10166     assert abs(th.value - analytic) < 1e-10
10167     assert th.bracket_width < 1e-9
10168
10169 def test_godel_boundary_converges_to_two_omega():
10170     """The Godel-type critical radius diverges as m -> 2 omega, so the
10171     boundary at finite box radius is only an approximation. Its convergence
10172     rate is itself the evidence, which is why the test checks the table and
10173     not one number."""
10174     rows = convergence_study(R.get("godel_type"), "m", (0.5, 6.0),
10175                             {"omega": 1.0}, "r", [5, 10, 15, 20],
10176                             analytic_value=2.0)
10177     errs = [r["error"] for r in rows if r.get("error") is not None]
10178     assert len(errs) == 4
10179     assert errs[0] > errs[-1] # monotone improvement
10180     assert errs[-1] < 1e-10
10181
10182 def test_pseudo_rn_normalisation_artefact_is_rejected():
10183     """The reference direction goes null at  $r^2 + 2\mu r - q^2 = 0$ . The root
10184     there is not a causal boundary and must be flagged."""
10185     th = causal_phase_boundary(R.get("pseudo_reissner_nordstrom"), "r",
10186                                (0.05, 0.13), {"mu": 1.0, "q": 0.5}, {})
10187     assert not th.domain_ok
10188     assert "artefact" in th.note
10189
10190 # -- flat quotients: exact closed forms -----
10191
10192 def test_misner_covering_exact_orbit_interval():
10193     m = R.get("misner_covering")
10194     t, x, b = m.symbol("t"), m.symbol("x"), m.symbol("b")
10195     e = sp.simplify(exact_orbit_interval(m, 1))
10196     target = 2 * (sp.cosh(b) - 1) * (t ** 2 - x ** 2)
10197     assert sp.simplify(sp.expand(e - target)) == 0
10198
10199 def test_grant_exact_orbit_interval():
10200     m = R.get("grant_space")
10201     t, x, b, al = m.symbol("t"), m.symbol("x"), m.symbol("b"), m.symbol("alpha")
10202     e = sp.simplify(exact_orbit_interval(m, 1))
10203     target = 2 * (sp.cosh(b) - 1) * (t ** 2 - x ** 2) + al ** 2
10204     assert sp.simplify(sp.expand(e - target)) == 0
10205
10206 def test_grant_polarized_hypersurfaces_match_closed_form():
10207     """ $x_n^2 = n^2 \alpha^2 / (2 (\cosh(n b) - 1))$ .
10208
10209     Locating these from the deck group is only possible because the
10210     identification structure is an executable object; v1.0 had no path to
10211     this result.
10212     """
10213     m = R.get("grant_space")
10214     b, alpha = 1.0, 1.0
10215     base = {"t": 0.0, "x": 0.8, "y": 0.0, "z": 0.0, "b": b, "alpha": alpha}
10216     rows = polarized_hypersurfaces(m, base, "x", 0.05, 3.0, max_power=5)
10217     got = {}
10218     for r in rows:
10219         if r["value"] > 0:
10220             got.setdefault(r["winding"], r["value"])
10221     for n in range(1, 6):
10222         target = math.sqrt(n * n * alpha ** 2 / (2 * (math.cosh(n * b) - 1)))
10223         assert abs(got[n] - target) < 1e-11, (n, got[n], target)
10224
10225 def test_misner_chart_and_covering_agree_pointwise():
10226     """Two independent representations of the same spacetime must give the
10227     same causal verdict at corresponding points."""
10228     from ctcf.causality import compact_orbit_test
10229     from ctcf.quotient_ctc import deck_orbit_test
10230     cov = R.get("misner_covering")
10231     chart = R.get("misner")
10232     b = 1.0
10233     for t, x in [(0.0, 0.5), (0.0, 2.0), (0.4, 0.5), (2.0, 0.5), (0.5, 0.5)]:
10234         d = deck_orbit_test(cov, {"t": t, "x": x, "y": 0.0, "z": 0.0, "b": b})
10235         analytic = 2 * (math.cosh(b) - 1) * (t * t - x * x)
10236         if abs(analytic) > 1e-12:
10237             assert abs(d.margin - analytic) < 1e-12
10238         # the Misner chart's T plays the role of  $(x^2 - t^2)$  up to a positive
10239         # factor, so the SIGNS must agree
10240         T = x * x - t * t
10241         c = compact_orbit_test(chart, {"T": T, "psi": 0.0, "y": 0.0, "z": 0.0})
10242         assert np.sign(c.margin) == np.sign(analytic) or abs(analytic) < 1e-12
10243
10244 def test_winding_spectrum_signs_are_monotone_for_grant():
10245     m = R.get("grant_space")
10246     rows = winding_spectrum(m, {"t": 0.0, "x": 0.8, "y": 0.0, "z": 0.0,
10247                                "b": 1.0, "alpha": 1.0}, max_power=6)
10248     chars = [r["character"] for r in rows]
10249     assert chars[0] == "spacelike"
10250     assert chars[-1] == "timelike"
10251
10252 def test_solver_is_non_circular():
10253     """The v1.0 kernel contained ``godel_family_margin(m, omega) = m^2 - 4
10254     omega^2`` and root-found it. No such closed form may exist anywhere on

```

```

10263 the v2.0 solver path."""
10264 import inspect
10265
10266 from ctcfac import continuation
10267 src = inspect.getsource(continuation)
10268 body = src.split('"""', 2)[-1] # exclude the module docstring
10269 assert "4*omega*omega" not in body
10270 assert "m**2 - 4" not in body

```

tests/controls/test_negative_controls.py — 186 lines

```

10271 """Mandatory negative controls (v1.0 section 15.3, extended).
10272
10273 Each control targets one specific way the architecture could produce a false
10274 positive or a false negative. A control that stops failing when the
10275 architecture is broken is worthless, so several of these deliberately assert
10276 what the system must *refuse* to say.
10277 """
10278
10279 import numpy as np
10280 import pytest
10281 import sympy as sp
10282
10283 from ctcfac import registry as R
10284 from ctcfac.causality import (certify_temporal_function, compact_orbit_test,
10285                               compact_orbit_verdict)
10286 from ctcfac.status import ClaimScope, CTCStatus, OrbitStatus
10287 from ctcfac.topology import compare_global_structure
10288
10289 # -- 1. flat space must not produce a CTC -----
10290
10291 def test_minkowski_is_certified_chronal_globally():
10292     c = certify_temporal_function(R.get("minkowski"))
10293     assert c.proved and c.status is CTCStatus.NO_CTC_CERTIFIED
10294     assert c.scope is ClaimScope.GLOBAL
10295
10296 def test_flrw_temporal_certificate_is_scale_factor_independent():
10297     """gtt = -1 for ANY a(t) > 0, so cosmic time works without knowing a."""
10298     c = certify_temporal_function(R.get("flrw_flat"))
10299     assert c.proved
10300     assert c.expression == "1"
10301
10302 # -- 2. strong curvature and horizons must not imply a CTC -----
10303
10304 @pytest.mark.parametrize("mid", ["schwarzschild", "reissner_nordstrom",
10305                                   "de_sitter_static"])
10306 def test_horizons_do_not_produce_ctc(mid):
10307     m = R.get(mid)
10308     r = compact_orbit_test(m, m.default_point)
10309     assert r.status is OrbitStatus.SECTOR_SPACELIKE
10310
10311 def test_reissner_cauchy_horizon_is_not_a_chronology_horizon():
10312     from ctcfac.horizons import horizon_report
10313     rep = horizon_report(R.get("reissner_nordstrom"), "r")
10314     kinds = {c["kind"] for c in rep["candidates"]}
10315     assert "KILLING_HORIZON" in kinds
10316     assert "CHRONOLOGY_HORIZON" not in kinds
10317     for c in rep["candidates"]:
10318         assert c["forbidden_inference"]
10319
10320 # -- 3. a sector clear is NOT a global no-CTC result -----
10321
10322 def test_clear_sector_is_reported_as_unresolved_not_certified():
10323     m = R.get("schwarzschild")
10324     v = compact_orbit_verdict(m, m.default_point)
10325     assert v.status is CTCStatus.UNRESOLVED
10326     assert "NOT a global no-CTC result" in v.reason
10327
10328 # -- 4. topology, not geometry, decides global causality -----
10329
10330 def test_ads_pair_shares_one_metric_and_splits_on_the_deck_group():
10331     a, b = R.get("ads4_universal_cover"), R.get("ads4_periodic")
10332     cmp = compare_global_structure(a, b)
10333     assert cmp["identical_local_metric"]
10334     assert cmp["demonstrates_L3_gate"]
10335     ra = compact_orbit_test(a, a.default_point)
10336     rb = compact_orbit_test(b, b.default_point)
10337     assert ra.status is OrbitStatus.SECTOR_SPACELIKE
10338     assert rb.status is OrbitStatus.SECTOR_CTC
10339
10340 def test_minkowski_pair_shares_one_metric_and_splits_on_the_deck_group():
10341     cmp = compare_global_structure(R.get("minkowski"),
10342                                   R.get("minkowski_time_periodic"))
10343     assert cmp["identical_local_metric"]
10344     assert cmp["demonstrates_L3_gate"]
10345
10346 def test_multivalued_time_is_refused_as_a_temporal_function():
10347     for mid in ("ads4_periodic", "minkowski_time_periodic"):
10348         m = R.get(mid)
10349         c = certify_temporal_function(m, tau=m.symbol("t"))
10350         assert not c.proved
10351         assert c.method == "quotient_invariance"
10352         assert "single valued" in c.note
10353
10354 # -- 5. traversable wormhole is not a time machine -----
10355
10356 def test_static_wormhole_is_chronal():
10357     c = certify_temporal_function(R.get("morris_thorne"))
10358     assert c.proved

```

```

10368     r = compact_orbit_test(R.get("morris_thorne"),
10369                             R.get("morris_thorne").default_point)
10370     assert r.status is OrbitStatus.SECTOR_SPACELIKE
10371
10372
10373     # -- 6. effective superluminality is not a time machine -----
10374
10375     def test_single_alcubierre_bubble_is_chronal_on_its_chart():
10376         c = certify_temporal_function(R.get("alcubierre"))
10377         assert c.proved and c.expression == "1"
10378
10379
10380     def test_single_krasnikov_tube_is_not_a_ctc():
10381         m = R.get("krasnikov")
10382         r = compact_orbit_test(m, m.default_point)
10383         assert r.status is OrbitStatus.SECTOR_SPACELIKE
10384
10385
10386     # -- 7. a quotient of AdS is not automatically causality violating -----
10387
10388     def test_btz_exterior_chart_is_spacelike_in_the_azimuthal_sector():
10389         m = R.get("btz_rotating")
10390         r = compact_orbit_test(m, m.default_point)
10391         assert r.status is OrbitStatus.SECTOR_SPACELIKE
10392
10393
10394     # -- 8. the CTC region is not created by expansion or curvature -----
10395
10396     def test_expanding_cosmology_is_chronal():
10397         m = R.get("flrw_exponential")
10398         c = certify_temporal_function(m)
10399         assert c.proved
10400
10401
10402     # -- 9. out-of-domain points are rejected, not answered -----
10403
10404     def test_point_outside_the_declared_domain_is_refused():
10405         m = R.get("schwarzschild")
10406         bad = dict(m.default_point)
10407         bad["r"] = 1.0 # inside the horizon
10408         r = compact_orbit_test(m, bad)
10409         assert r.status is OrbitStatus.SECTOR_UNRESOLVED
10410         assert "domain" in r.reason
10411
10412
10413     # -- 10. a non-Killing "generator" is refused -----
10414
10415     def test_non_killing_generator_is_rejected():
10416         from ctcfa.model import CompactDirection, MetricModel, Provenance
10417         t, r, phi, z = sp.symbols("t r phi z", real=True)
10418         # g_phi phi depends on phi, so d/dphi is NOT a Killing vector
10419         g = sp.diag(-1, 1, r**2*(2+sp.sin(phi)), 1)
10420         m = MetricModel(id="fake", title="non-Killing generator control",
10421                        coords=(t, r, phi, z), metric=g,
10422                        compact=(CompactDirection(2, "phi", 2*sp.pi)),
10423                        default_point={"t": 0, "r": 1, "phi": 0, "z": 0},
10424                        provenance=Provenance(("control",)))
10425         res = compact_orbit_test(m, m.default_point)
10426         assert res.status is OrbitStatus.SECTOR_UNRESOLVED
10427         assert "Killing" in res.reason
10428
10429
10430     # -- 11. asymmetric metrics cannot enter the registry -----
10431
10432     def test_asymmetric_metric_is_refused_at_load():
10433         from ctcfa.model import MetricModel, Provenance
10434         t, x, y, z = sp.symbols("t x y z", real=True)
10435         g = sp.Matrix([[-1, 0, 0, 1], [0, 1, 0, 0], [0, 0, 1, 0], [0, 0, 0, 1]])
10436         with pytest.raises(ValueError, match="not symmetric"):
10437             MetricModel(id="bad", title="asymmetric", coords=(t, x, y, z), metric=g,
10438                        provenance=Provenance(("control",)))
10439
10440
10441     # -- 12. reduced predicates cannot be printed as global theorems -----
10442
10443     def test_reduced_predicate_stays_at_reduced_scope():
10444         from ctcfa.causality import global_predicate_verdict
10445         m = R.get("time_shifted_wormhole")
10446         v = global_predicate_verdict(m, m.default_point)
10447         assert v.status is CTCStatus.CTC_PROVED
10448         assert v.scope is ClaimScope.REDUCED_MODEL
10449         assert "not" in v.reason.lower() or "NOT" in v.reason
10450
10451
10452     def test_external_spec_refuses_to_produce_a_metric():
10453         for mid in ("tipler_cylinder", "ori_2007_full", "paired_warp",
10454                    "gott_two_strings"):
10455             with pytest.raises(ValueError, match="refuses to fabricate"):
10456                 R.get(mid).require_metric()

```

tests/blind/test_blind_benchmark.py — 116 lines

```

10457     """Blind benchmark protocol (v1.0 section 15.2), executed.
10458
10459     The point of a blind benchmark is that the solver must not be able to read
10460     the answer. v1.0 described the protocol in six numbered steps and shipped a
10461     regression table whose expected values were literals in the source, which is
10462     the opposite arrangement.
10463
10464     Here the answers are HMAC-sealed with a key the *evaluator* holds. The
10465     solver receives only the sealed digests, computes a boundary from the metric,
10466     and submits it; the evaluator verifies. A solver that has not been given
10467     the key cannot recover the target, and a solver that hard-codes a literal
10468     will fail on the perturbed parameter sets, which are drawn at run time.
10469     """
10470
10471     import math
10472     import os

```

```

10473
10474 import numpy as np
10475 import pytest
10476
10477 from ctcfactory import registry as R
10478 from ctcfactory.continuation import causal_phase_boundary
10479 from ctcfactory.provenance import check_answer, seal_answers
10480
10481 EVALUATOR_KEY = b"CTC-FA-v2.0-blind-benchmark-evaluator-key"
10482 QUANT = 9 # digits retained when sealing / submitting
10483
10484
10485 def _q(x: float) -> float:
10486     return round(float(x), QUANT)
10487
10488
10489 def _make_cases(seed: int = 20260831):
10490     """Parameter sets drawn at run time, so a literal cannot pass."""
10491     rng = np.random.default_rng(seed)
10492     cases = []
10493     for _ in range(4):
10494         r0 = float(rng.uniform(0.8, 3.0))
10495         cases.append({
10496             "name": f"van_stockum_r{r0:.6f}",
10497             "model": "van_stockum", "parameter": "a",
10498             "bracket": (0.05, 5.0), "fixed": {}, "box": {"r": (r0, r0)},
10499             "truth": 1.0 / r0,
10500         })
10501     for _ in range(4):
10502         J = float(rng.uniform(0.02, 0.3))
10503         r0 = float(rng.uniform(0.05, 0.5))
10504         cases.append({
10505             "name": f"string_J{J:.6f}_r{r0:.6f}",
10506             "model": "spinning_cosmic_string", "parameter": "alpha",
10507             "bracket": (0.1, 60.0), "fixed": {"J": J}, "box": {"r": (r0, r0)},
10508             "truth": 4.0 * J / r0,
10509         })
10510     for _ in range(4):
10511         mu = float(rng.uniform(0.3, 3.0))
10512         cases.append({
10513             "name": f"ori2007_mu{mu:.6f}",
10514             "model": "ori_2007_core", "parameter": "r",
10515             "bracket": (0.05, 20.0), "fixed": {"mu": mu}, "box": {},
10516             "truth": 2.0 * mu,
10517         })
10518     for _ in range(4):
10519         mu = float(rng.uniform(1.0, 3.0))
10520         q = float(rng.uniform(0.1, 0.8)) * mu
10521         rminus = mu - math.sqrt(mu * mu - q * q)
10522         rplus = mu + math.sqrt(mu * mu - q * q)
10523         cases.append({
10524             "name": f"pseudoRN_outer_mu{mu:.6f}_q{q:.6f}",
10525             "model": "pseudo_reissner_nordstrom", "parameter": "r",
10526             "bracket": (rplus * 0.6, rplus * 2.0),
10527             "fixed": {"mu": mu, "q": q}, "box": {},
10528             "truth": rplus,
10529         })
10530     return cases
10531
10532
10533 CASES = _make_cases()
10534 SEALED = seal_answers([c["name"]: _q(c["truth"]) for c in CASES], EVALUATOR_KEY)
10535
10536
10537 def test_sealed_answers_are_not_readable():
10538     """The sealed file must not leak the value."""
10539     for c in CASES:
10540         blob = SEALED[c["name"]]
10541         assert len(blob) == 64 and all(ch in "0123456789abcdef" for ch in blob)
10542         assert str(round(c["truth"], 6)) not in blob
10543
10544
10545 @pytest.mark.parametrize("case", CASES, ids=[c["name"] for c in CASES])
10546 def test_solver_recovers_sealed_boundary(case):
10547     """The solver sees only the model, the parameter and the bracket."""
10548     th = causal_phase_boundary(R.get(case["model"]), case["parameter"],
10549                               case["bracket"], case["fixed"], case["box"])
10550     submitted = _q(th.value)
10551     assert check_answer(case["name"], submitted, SEALED, EVALUATOR_KEY), (
10552         f"{case['name']}: submitted {submitted}")
10553
10554
10555 def test_a_wrong_submission_is_rejected():
10556     c = CASES[0]
10557     assert not check_answer(c["name"], _q(c["truth"] * 1.01), SEALED,
10558                           EVALUATOR_KEY)
10559
10560
10561 def test_score_report():
10562     """Aggregate score, in the form the protocol asks for."""
10563     errs, false_pos, false_neg = [], 0, 0
10564     for c in CASES:
10565         th = causal_phase_boundary(R.get(c["model"]), c["parameter"],
10566                                   c["bracket"], c["fixed"], c["box"])
10567         errs.append(abs(th.value - c["truth"]))
10568         if th.left_margin * th.right_margin >= 0:
10569             false_pos += 1
10570     assert max(errs) < 1e-9
10571     assert false_pos == 0
10572     assert false_neg == 0

```

tests/regression/test_regression.py — 170 lines

```

10573 """Full-registry regression: validation gates, the causal matrix, provenance,
10574 the theorem validator, energy conditions and the meta-analysis governance.
10575
10576 Numeric expectations in this file are the ones the v1.0 self-test published,
10577 where those were correct. Every value that the v1.0 kernel got right is

```

```

10578 reproduced exactly, which is the strongest available evidence that the v2.0
10579 rewrite did not silently change the physics while fixing the defects.
10580 """
10581
10582 import math
10583
10584 import numpy as np
10585 import pytest
10586
10587 from ctcfactory import registry as R
10588 from ctcfactory.causality import compact_orbit_test
10589 from ctcfactory.report import (format_matrix, matrix_agreement, primary_route,
10590                               regression_matrix)
10591 from ctcfactory.validate import validate_model
10592
10593
10594 #: Values published in the v1.0 self-test (section 14 / Appendix C) that were
10595 #: correct and must be reproduced bit for bit by the rewritten engines.
10596 V1_MARGINS = {
10597     "schwarzschild": +2.500000e+01,
10598     "reissner_nordstrom": +2.500000e+01,
10599     "kerr": -1.670000e+00,
10600     "kerr_newman": -1.772400e+00,
10601     "godel_type": -1.736478e+01,
10602     "godel_critical": +1.315412e+01,      # v1.0 row "godel_type SECTOR_SPACELIKE"
10603     "misner": -1.000000e+00,
10604     "van_stockum": -1.200000e+01,
10605     "taub_nut": -6.923595e-01,
10606     "spinning_cosmic_string": -1.536000e-01,
10607     "morris_thorne": +4.000000e+00,
10608     "ori_2005": -1.000000e+00,
10609     "ori_2007_core": -1.000000e+00,
10610     "krasnikov": +1.000000e-02,
10611 }
10612
10613
10614 @pytest.mark.parametrize("mid, expected", sorted(V1_MARGINS.items()))
10615 def test_v1_margins_are_reproduced(mid, expected):
10616     m = R.get(mid)
10617     r = compact_orbit_test(m, m.default_point)
10618     assert abs(r.margin - expected) <= 1e-6 * max(1.0, abs(expected)), (
10619         f"{mid}: v2.0 gives {r.margin!r}, v1.0 published {expected!r}")
10620
10621
10622 def test_registry_audit_passes():
10623     a = R.registry_audit()
10624     assert a["pass"], a["problems"]
10625     assert a["n_models"] >= 30
10626
10627
10628 def test_every_model_passes_the_structural_gates():
10629     for m in R.models():
10630         rep = validate_model(m, cross_check=False)
10631         assert rep.ok, (m.id, rep.gates.as_dict())
10632
10633
10634 def test_regression_matrix_agrees_everywhere():
10635     rows = regression_matrix()
10636     ag = matrix_agreement(rows)
10637     assert ag["pass"], ag["mismatches"]
10638     assert ag["n"] >= 30
10639
10640
10641 def test_every_row_records_its_derivation_route():
10642     for row in regression_matrix():
10643         assert row["route"] in ("compact_orbit", "deck_orbit",
10644                                "global_predicate", "temporal_function", "none")
10645
10646
10647 def test_vacuum_models_have_vanishing_field_equation_residual():
10648     from ctcfactory.energy import symbolic_einstein
10649     import sympy as sp
10650     for mid in ("schwarzschild", "kerr", "taub_nut", "misner", "ori_2005",
10651                "ori_2007_core", "minkowski", "grant_space"):
10652         m = R.get(mid)
10653         G = symbolic_einstein(m)
10654         arr = np.array(sp.Matrix(G + m.matter.cosmological_constant * m.metric)
10655                        .subs(m.substitution(m.default_point)).evalf(30),
10656                        dtype=float)
10657         assert float(np.max(np.abs(arr))) < 1e-9, mid
10658
10659
10660 def test_energy_conditions_on_the_key_models():
10661     from ctcfactory.energy import energy_report
10662     #: Ori 2005 is the counterexample to "CTC implies NEC violation": a
10663     #: vacuum core with a chronology-violating compact direction.
10664     ori = energy_report(R.get("ori_2005"))
10665     assert ori.he.type.value == "TYPE_I"
10666     assert ori.nec_status in ("MARGINAL", "SATISFIED")
10667     #: van Stockum: matter-supported CTC with strictly positive energy density.
10668     vs = energy_report(R.get("van_stockum"))
10669     assert vs.he.rho > 0
10670     assert vs.nec_status == "SATISFIED"
10671     #: Morris-Thorne throat: NEC violated.
10672     mt = energy_report(R.get("morris_thorne"),
10673                       {"t": 0, "r": 1.05, "theta": math.pi / 2, "phi": 0,
10674                       "r0": 1.0})
10675     assert mt.nec_status == "VIOLATED"
10676     #: Alcubierre: energy conditions violated.
10677     al = energy_report(R.get("alcubierre"))
10678     assert al.nec_status == "VIOLATED"
10679
10680
10681 def test_theorem_validator_never_licenses_unverified_hypotheses():
10682     from ctcfactory.theorems import THEOREMS, Applicability, apply
10683     for m in R.models():
10684         for k in THEOREMS:
10685             rep = apply(k, m)
10686             if rep.applicability is Applicability.LICENSED:
10687                 assert all(v is True for v in rep.hypothesis_results.values())

```

```

10688         else:
10689             assert rep.permitted_label is None
10690
10691
10692 def test_chronology_protection_is_never_labelled_proved():
10693     from ctcfa.theorems import THEOREMS
10694     th = THEOREMS["chronology_protection"]
10695     assert th.status_note == "CONJECTURE"
10696     assert "PROVED" not in th.permitted_label
10697     for m in R.models():
10698         from ctcfa.theorems import apply
10699         rep = apply("chronology_protection", m)
10700         assert rep.permitted_label in (None, th.permitted_label)
10701
10702
10703 def test_meta_analysis_reports_held_out_scores_and_governance():
10704     from ctcfa.meta import cross_family_report
10705     rep = cross_family_report(R.models())
10706     assert rep["n_models"] >= 30
10707     for rule in rep["rules"]:
10708         assert rule["status"] in ("CANDIDATE_PRECURSOR", "REFUTED_ON_HELD_OUT")
10709         assert rule["heldout_n"] >= 2
10710     assert "CANDIDATE_PRECURSOR" in rep["governance"]
10711     assert "universal" in rep["governance"]
10712
10713
10714 def test_provenance_round_trip():
10715     from pathlib import Path
10716     import tempfile
10717
10718     from ctcfa.provenance import (build_manifest, new_run, sha256_text,
10719                                   source_tree_digest, write_run)
10720     root = Path(R.__file__).resolve().parent
10721     d1, files = source_tree_digest(root)
10722     d2, _ = source_tree_digest(root)
10723     assert d1 == d2 and len(files) > 15
10724     with tempfile.TemporaryDirectory() as td:
10725         rec = new_run("kerr", "compact_orbit", {"r": -0.5}, {"margin": -1.67},
10726                     d1)
10727         p = write_run(rec, td)
10728         assert p.exists()
10729         again = write_run(rec, td)
10730         assert again == p # identical content, no duplicate
10731
10732
10733 def test_quotient_certificates_are_present_where_declared():
10734     from ctcfa.topology import verify_quotient
10735     for m in R.models():
10736         if not m.quotient.generators:
10737             continue
10738         cert = verify_quotient(m)
10739         assert len(cert.generators) == len(m.quotient.generators)
10740         for gc in cert.generators:
10741             if m.has_metric:
10742                 assert gc.is_isometry, (m.id, gc.name, gc.isometry_residual[:150])

```

A.3 Worked examples

examples/_common.py — 20 lines

```

10743 """Shared bootstrap for the worked examples."""
10744 import sys, warnings
10745 from pathlib import Path
10746
10747 warnings.filterwarnings("ignore")
10748 SRC = Path(__file__).resolve().parents[1] / "src"
10749 if str(SRC) not in sys.path:
10750     sys.path.insert(0, str(SRC))
10751
10752
10753 def banner(n, title):
10754     line = "=" * 78
10755     print(line)
10756     print(f"EXAMPLE {n} -- {title}")
10757     print(line)
10758
10759
10760 def section(t):
10761     print()
10762     print(f"--- {t} " + "-" * max(0, 74 - len(t)))

```

examples/ex01_I3_gate_ads_pair.py — 101 lines

```

10763 """
10764 EXAMPLE 1 -- The L3 gate: one metric, two spacetimes, opposite verdicts.
10765
10766 WHAT THIS UNIFIES
10767     The single most important structural claim of the architecture is that
10768     global causal structure is not a function of the local metric. This
10769     example proves it computationally: ``ads4_universal_cover`` and
10770     ``ads4_periodic`` carry the *identical* metric tensor and receive
10771     opposite global verdicts, purely because of the deck group.
10772
10773     The same demonstration is repeated with the simplest possible pair,
10774     Minkowski and Minkowski with periodic time, so that no one can attribute
10775     the effect to curvature.
10776
10777 DATA REQUIRED
10778     coordinate ordering; the metric in the (-,+,+,+) convention; the deck
10779     group generators with their inverses; a candidate temporal function.
10780     Nothing else. In particular NO numerical parameters are needed for the
10781     structural part of the result.
10782 """
10783
10784 from _common import banner, section
10785
10786 import sympy as sp
10787
10788 from ctcfaregistry import registry as R
10789 from ctcfaregistry.causality import certify_temporal_function, compact_orbit_test
10790 from ctcfaregistry.topology import compare_global_structure, verify_quotient
10791
10792
10793 def main() -> None:
10794     banner(1, "The L3 gate: identical local metric, opposite causal structure")
10795
10796     a = R.get("ads4_universal_cover")
10797     b = R.get("ads4_periodic")
10798
10799     section("The two registry entries")
10800     print(f"  A : {a.id:24s} {a.title}")
10801     print(f"    topology : {a.topology}")
10802     print(f"    deck group: {a.quotient.label}")
10803     print(f"  B : {b.id:24s} {b.title}")
10804     print(f"    topology : {b.topology}")
10805     print(f"    deck group: {b.quotient.label}")
10806
10807     section("The local metric tensors")
10808     for m in (a, b):
10809         print(f"  {m.id:24s}")
10810         for i in range(m.dim):
10811             print(f"    g_{i}{i} = {sp.simplify(m.metric[i, i])}")
10812
10813     cmp = compare_global_structure(a, b)
10814     print()
10815     print(f"  identical_local_metric : {cmp['identical_local_metric']}")
10816     print(f"  demonstrates_L3_gate   : {cmp['demonstrates_L3_gate']}")
10817
10818     section("Deck-group certificate for the periodic member")
10819     cert = verify_quotient(b)
10820     for g in cert.generators:
10821         print(f"  generator {g.name}")
10822         print(f"    isometry residual      : {g.isometry_residual}")
10823         print(f"    is an isometry        : {g.is_isometry}")
10824         print(f"    time-orientation preserving: {g.time_orientation_preserving}")
10825         print(f"    has a fixed point     : {g.has_fixed_point}")
10826
10827     section("Causal verdicts at the same coordinate point")
10828     for m in (a, b):
10829         r = compact_orbit_test(m, m.default_point)
10830         print(f"  {m.id:24s} {r.status.value:18s} margin = {r.margin:+.6e}")
10831         print(f"    {r.reason[:120]}")
10832
10833     section("Why the temporal-function route splits the pair")
10834     for m in (a, b):
10835         c = certify_temporal_function(m, tau=m.symbol("t"))
10836         print(f"  {m.id:24s} tau = t -> {c.status.value} [{c.method}]")
10837         print(f"    {c.note[:150]}")
10838
10839     section("The same effect with no curvature at all")
10840     cmp2 = compare_global_structure(R.get("minkowski"),
10841                                   R.get("minkowski_time_periodic"))

```



```

10842 print(f" minkowski vs minkowski_time_periodic")
10843 print(f"         identical_local_metric : {cmp2['identical_local_metric']}")
10844 print(f"         demonstrates_L3_gate   : {cmp2['demonstrates_L3_gate']}")
10845 for mid in ("minkowski", "minkowski_time_periodic"):
10846     m = R.get(mid)
10847     if m.compact:
10848         r = compact_orbit_test(m, m.default_point)
10849         print(f"         {mid:26s} {r.status.value} margin {r.margin:+.3e}")
10850     else:
10851         c = certify_temporal_function(m)
10852         print(f"         {mid:26s} {c.status.value} scope {c.scope.value}")
10853
10854 section("Reading")
10855 print("  A v1.0-style architecture that stores topology as prose cannot")
10856 print("  distinguish these pairs at all: the tensors are equal, so every")
10857 print("  local computation returns the same answer. The distinction is")
10858 print("  carried entirely by the deck group, which is why v2.0 makes it")
10859 print("  an executable object rather than a comment.")
10860
10861
10862 if __name__ == "__main__":
10863     main()

```

examples/ex02_noncircular_boundary.py — 99 lines

```

10864 """
10865 EXAMPLE 2 -- Deriving a causal phase boundary WITHOUT the answer.
10866
10867 WHAT THIS UNIFIES
10868 v1.0 "recovered" the Godel-type boundary  $m = 2\omega$  by root-finding the
10869 hard-coded expression  $m^2 - 4\omega^2$ . That is failure mode FM-016,
10870 circular validation, committed by the architecture that named it.
10871
10872 v2.0 derives the boundary from the metric alone. The chain is:
10873
10874     g_ab -> reduced causal margin  $\mu = g(\xi, \xi)/h(\xi)$ 
10875           -> infimum over the declared coordinate box
10876           -> sign change in the parameter
10877           -> bracketed bisection
10878           -> comparison with the closed form AFTERWARDS
10879
10880 DATA REQUIRED
10881 the metric; which coordinate is the compact generator; a timelike
10882 reference direction (declared only for charts with no timelike coordinate
10883 direction); a coordinate box; a parameter bracket. The literature answer
10884 is NOT an input.
10885 """
10886
10887 from _common import banner, section
10888
10889 import math
10890 import inspect
10891 import sympy as sp
10892
10893 from ctcf_a import registry as R
10894 from ctcf_a import continuation as C
10895
10896 def main() -> None:
10897     banner(2, "Non-circular derivation of causal phase boundaries")
10898
10899     section("Step 1 -- the reduced causal margin, derived from g_ab")
10900     for mid in ("godel_type", "van_stockum", "spinning_cosmic_string",
10901                "ori_2007_core", "misner", "ori_2005"):
10902         m = R.get(mid)
10903         mu = C.reduced_causal_margin_symbolic(m)
10904         print(f" {mid:26s} mu = {sp.simplify(mu)}")
10905
10906     section("Step 2 -- Godel-type: boundary vs coordinate-box radius")
10907     rows = C.convergence_study(R.get("godel_type"), "m", (0.5, 6.0),
10908                               {"omega": 1.0}, "r", [5, 10, 15, 20, 25, 30],
10909                               analytic_value=2.0)
10910     print(f" {'box radius R':>14} {'m*(R)':>20} {'|m* - 2|':>12}")
10911     for r in rows:
10912         print(f" {r['box_radius']:>14.1f} {r['boundary']:>20.15f} {r['error']:>12.3e}")
10913     print()
10914     print("  The analytic value  $m = 2\omega$  is an ASYMPTOTIC statement: the")
10915     print("  Godel-type critical radius diverges as  $m \rightarrow 2\omega$ , so a finite")
10916     print("  box can only approach it. The convergence table is the evidence;")
10917     print("  a single number would not be.")
10918
10919     section("Step 3 -- boundaries with exact closed forms")
10920     cases = [
10921         ("van_stockum", "a", (0.05, 3.0), {}, {"r": (2.0, 2.0)}, 0.5, "a r = 1"),
10922         ("spinning_cosmic_string", "alpha", (0.5, 20.0), {"J": 0.1},
10923          {"r": (0.1, 0.1)}, 4.0, "alpha r = 4 J"),
10924         ("ori_2007_core", "r", (0.5, 6.0), {"mu": 1.0}, {}, 2.0, "r = 2 mu"),
10925         ("pseudo_reissner_nordstrom", "r", (1.0, 3.0), {"mu": 1.0, "q": 0.5},
10926          {}, 1 + math.sqrt(0.75), "r_+ = mu + sqrt(mu^2 - q^2)"),
10927         ("pseudo_reissner_nordstrom", "r", (0.125, 1.0), {"mu": 1.0, "q": 0.5},
10928          {}, 1 - math.sqrt(0.75), "r_- = mu - sqrt(mu^2 - q^2)"),
10929         ("misner", "T", (-1.5, 1.5), {}, {}, 0.0, "T = 0"),
10930         ("ori_2005", "T", (-1.5, 1.5), {"x": 0.0, "y": 0.0}, {}, 0.0, "T = F(x,y)"),
10931     ]
10932
10933     print(f" {'model':<26} {'par':<7} {'derived':>21} {'closed form':>21} {'error':>11}")
10934     for mid, par, brk, fx, box, an, form in cases:
10935         th = C.causal_phase_boundary(R.get(mid), par, brk, fx, box,
10936                                     analytic_value=an, analytic_form=form)
10937         print(f" {mid:<26} {par:<7} {th.value:>21.15f} {an:>21.15f}")
10938         print(f" {th.analytic_error:>11.2e}")
10939         bracket = [{th.bracket[0]:.15f}, {th.bracket[1]:.15f}]
10940         print(f" {th.bracket_width:2e} domain_ok={th.domain_ok}")
10941         print(f" closed form: {form}")
10942
10943     section("Step 4 -- rejecting a normalisation artefact")
10944     th = C.causal_phase_boundary(R.get("pseudo_reissner_nordstrom"), "r",
10945                                 (0.05, 0.13), {"mu": 1.0, "q": 0.5}, {})
10946     print(f" a sign change exists at r = {th.value:.15f}")

```



```

10947     print(f"    domain_ok = {th.domain_ok}")
10948     print(f"    {th.note.split('|')[-1].strip()}")
10949     print()
10950     print("    The zero of -g(u,u) is where the timelike REFERENCE direction")
10951     print("    becomes null, not where the orbit changes causal character.")
10952     print("    Reporting it as a causal boundary would be failure mode FM-001.")
10953
10954     section("Step 5 -- the non-circularity check that runs in CI")
10955     src = inspect.getsource(C).split('"""', 2)[-1]
10956     print(f"    '4*omega*omega' present in the solver body : {'4*omega*omega' in src}")
10957     print(f"    'm**2 - 4' present in the solver body      : {'m**2 - 4' in src}")
10958     print("    (tests/analytic/test_analytic_benchmarks.py::test_solver_is_non_circular)")
10959
10960
10961 if __name__ == "__main__":
10962     main()

```

examples/ex03_quotient_ctc.py — 110 lines

```

10963 """
10964 EXAMPLE 3 -- Flat quotients: Misner and Grant space from the deck group.
10965
10966 WHAT THIS UNIFIES
10967     Misner space, Grant space and the polarised Misner-type spacetimes that
10968     dominate the modern causal-classification literature are quotients of
10969     FLAT Minkowski space by an isometry that is not a coordinate translation.
10970     v1.0 had no engine for them: its Misner entry used an ad-hoc chart and
10971     nothing could express a boost identification at all.
10972
10973     Here the same spacetime is computed twice, from two completely different
10974     representations, and the answers are checked against each other and
10975     against a closed form. Then the engine locates the polarised
10976     hypersurfaces -- the loci where a quantum field's two-point function
10977     diverges -- automatically, which is a result v1.0 had no path to.
10978
10979 DATA REQUIRED
10980     the covering metric (flat, verified); the deck generators with their
10981     inverses; the parameter values (rapidity b, translation alpha); a test
10982     point. Nothing about the quotient chart is needed.
10983 """
10984
10985 from _common import banner, section
10986
10987 import math
10988 import sympy as sp
10989
10990 from ctcfactory import registry as R
10991 from ctcfactory.causality import compact_orbit_test
10992 from ctcfactory.quotient_ctc import (certify_flat, deck_orbit_test,
10993                                     exact_orbit_interval, polarized_hypersurfaces,
10994                                     winding_spectrum)
10995
10996
10997 def main() -> None:
10998     banner(3, "Flat quotients: Misner, Grant, and the polarised hypersurfaces")
10999
11000     section("Step 1 -- flatness of the covering is VERIFIED, not assumed")
11001     for mid in ("misner_covering", "grant_space", "minkowski_time_periodic"):
11002         c = certify_flat(R.get(mid))
11003         print(f"    {mid:26s} Riemann = {c.residual}    flat = {c.flat}")
11004
11005     section("Step 2 -- exact closed form for the winding-n closed geodesic")
11006     for mid, n in (("misner_covering", 1), ("misner_covering", 2),
11007                  ("grant_space", 1), ("grant_space", 2)):
11008         e = sp.simplify(exact_orbit_interval(R.get(mid), n))
11009         print(f"    {mid:20s} n={n}:    interval^2 = {sp.factor(sp.expand(e))}")
11010
11011     section("Step 3 -- engine vs closed form, point by point (Misner covering)")
11012     b = 1.0
11013     print(f"    {'t':>6}{ 'x':>8}{ 'engine q':>22}{ '2(cosh b -1)(t^2-x^2)':>26}{ 'status':>16}")
11014     for t, x in [(0.0, 0.5), (0.0, 1.0), (0.0, 2.0), (0.4, 0.5), (0.5, 0.5),
11015                 (2.0, 0.5)]:
11016         d = deck_orbit_test(R.get("misner_covering"),
11017                             {"t": t, "x": x, "y": 0.0, "z": 0.0, "b": b})
11018         an = 2 * (math.cosh(b) - 1) * (t * t - x * x)
11019         print(f"    {'t':>6.2f}{ 'x':>8.2f}{ 'd.margin':>22.15f}{ 'an':>26.15f}"
11020               f"    {d.status.value.replace('SECTOR_', '')}>16}")
11021
11022     section("Step 4 -- the same spacetime through the Misner chart")
11023     print("    In the covering the sign of the interval is that of (t^2 - x^2);")
11024     print("    in the quotient chart it is the sign of -T. The two must agree")
11025     print("    under T = x^2 - t^2.")
11026     print(f"    {'t':>6}{ 'x':>8}{ 'T = x^2-t^2':>14}{ 'chart margin':>18}"
11027           f"    {'covering margin':>20}{ 'signs agree':>14}")
11028     for t, x in [(0.0, 0.5), (0.0, 2.0), (0.4, 0.5), (2.0, 0.5)]:
11029         T = x * x - t * t
11030         c = compact_orbit_test(R.get("misner"),
11031                               {"T": T, "psi": 0.0, "y": 0.0, "z": 0.0})
11032         d = deck_orbit_test(R.get("misner_covering"),
11033                             {"t": t, "x": x, "y": 0.0, "z": 0.0, "b": b})
11034         agree = (c.margin < 0) == (d.margin < 0)
11035         print(f"    {'t':>6.2f}{ 'x':>8.2f}{ 'T':>14.4f}{ 'c.margin':>18.6e}"
11036               f"    {d.margin:>20.6e}{str(agree):>14}")
11037
11038     section("Step 5 -- Grant space: winding spectrum at one point")
11039     m = R.get("grant_space")
11040     base = {"t": 0.0, "x": 0.8, "y": 0.0, "z": 0.0, "b": 1.0, "alpha": 1.0}
11041     print("    at x = 0.8 (t = 0, b = 1, alpha = 1):")
11042     for row in winding_spectrum(m, base, 6):
11043         print(f"        n = {row['n']}    interval^2 = {row['interval_squared']:+.6e}"
11044               f"        {row['character']}")
11045     print()
11046     print("    The winding-1 and winding-2 loops are spacelike here; the")
11047     print("    winding-3 loop is timelike. The shortest chronology-violating")
11048     print("    loop through a point is therefore a computed quantity, not an")
11049     print("    assumption.")
11050
11051     section("Step 6 -- polarised hypersurfaces located from the deck group")

```

```

11052 rows = polarized_hypersurfaces(m, base, "x", 0.05, 3.0, max_power=6)
11053 seen = {}
11054 for r in rows:
11055     if r["value"] > 0:
11056         seen.setdefault(r["winding"], r["value"])
11057 print(f"  {'winding n':>10}{ 'x_n (engine)':>22}"
11058       f"{'n alpha / sqrt(2(cosh nb -1))':>32}{ 'error':>12}")
11059 for n in sorted(seen):
11060     an = math.sqrt(n * n * 1.0 / (2 * (math.cosh(n * 1.0) - 1)))
11061     print(f"  {n:>10}{seen[n]:>22.15f}{an:>32.15f}{abs(seen[n]-an):>12.2e}")
11062 print()
11063 print("  x_n decreases monotonically to zero, so the union over all n is")
11064 print("  the whole region x^2 > t^2: the translation alpha does NOT move")
11065 print("  the chronology horizon, which stays at x^2 = t^2 exactly as in")
11066 print("  Misner space. What alpha creates is this discrete family of")
11067 print("  polarised hypersurfaces accumulating on the horizon -- precisely")
11068 print("  where the renormalised stress tensor of a quantum field diverges.")
11069
11070
11071 if __name__ == "__main__":
11072     main()

```

examples/ex04_energy_conditions.py — 118 lines

```

11073 """
11074 EXAMPLE 4 -- Certified energy conditions, and the counterexample that matters.
11075
11076 WHAT THIS UNIFIES
11077 v1.0 sampled 256 directions on a Fibonacci sphere and called the result
11078 diagnostic, which meant no energy statement could ever be certified.
11079 v2.0 computes the same quantities two ways, exactly:
11080
11081     Route A  Hawking-Ellis classification -> closed-form criteria
11082     Route B  exact trust-region minimisation on the null/timelike cone
11083
11084 and cross-checks them. It then integrates ANEC along a real null
11085 geodesic instead of a supplied list of numbers.
11086
11087 The scientific payload is the pair of results at the bottom: the Ori 2005
11088 core and van Stockum are chronology-violating spacetimes whose matter
11089 does NOT violate the null energy condition. Any architecture that treats
11090 "CTC" and "exotic matter" as synonyms is wrong, and this is the
11091 computation that shows it.
11092
11093 DATA REQUIRED
11094 the metric; the cosmological constant; the evaluation point. The stress
11095 tensor is obtained ON SHELL from G_ab + Lambda g_ab = 8 pi T_ab -- it is
11096 never assumed.
11097 """
11098
11099 from _common import banner, section
11100
11101 import math
11102 import numpy as np
11103
11104 from ctcfactory import registry as R
11105 from ctcfactory.energy import energy_report
11106 from ctcfactory.geodesics import anec_on_geodesic
11107
11108 def main() -> None:
11109     banner(4, "Energy conditions: exact classification and a real ANEC integral")
11110
11111     section("Hawking-Ellis type and the four conditions, on shell")
11112     print(f"  {'model':<26}{ 'type':<10}{ 'rho':>10}"
11113           f"{'NEC':>12}{ 'WEC':>12}{ 'SEC':>12}{ 'DEC':>12}")
11114     order = ["minkowski", "schwarzschild", "kerr", "reissner_nordstrom",
11115             "kerr_newman", "de_sitter_static", "ads4_universal_cover",
11116             "misner", "ori_2005", "ori_2007_core", "godel", "van_stockum",
11117             "spinning_cosmic_string", "morris_thorne", "alcubierre",
11118             "krasnikov"]
11119
11120     reports = {}
11121     for mid in order:
11122         rep = energy_report(R.get(mid))
11123         reports[mid] = rep
11124         rho = "  " if rep.he.rho is None else f"{rep.he.rho:+.4f}"
11125         short = {"SATISFIED": "OK", "SATISFIED_DIAGNOSTIC": "OK*",
11126                "MARGINAL": "marg", "VIOLATED": "VIOL"}
11127         print(f"  {mid:<26}{rep.he.type.value.replace('TYPE_', ''):<10}{rho:>10}"
11128               f"{short.get(rep.nec_status, rep.nec_status):>12}"
11129               f"{short.get(rep.wec_status, rep.wec_status):>12}"
11130               f"{short.get(rep.sec_status, rep.sec_status):>12}"
11131               f"{short.get(rep.dec_status, rep.dec_status):>12}")
11132
11133     section("Cross-check between the two exact routes")
11134     for mid in ("schwarzschild", "kerr", "godel", "van_stockum", "morris_thorne"):
11135         rep = reports[mid]
11136         x = rep.cross_check_error
11137         print(f"  {mid:<26} |Route A - Route B| = "
11138               f"{'n/a (non Type I)' if x is None else f'{x:.3e}'}"
11139               f"  uncertainty {rep.uncertainty:.2e}")
11140
11141     section("The counterexample: chronology violation WITHOUT exotic matter")
11142     for mid in ("ori_2005", "ori_2007_core", "van_stockum", "godel"):
11143         rep = reports[mid]
11144         m = R.get(mid)
11145         print(f"  {mid}")
11146         print(f"    causal status : {(m.expected_at_default or m.expected).value}")
11147         print(f"    matter : {m.matter.label}")
11148         print(f"    rho : {rep.he.rho:+.6f}")
11149         print(f"    NEC margin : {rep.nec_margin:+.6e} [{rep.nec_status}]")
11150         print(f"    SEC margin : {rep.sec_margin:+.6e} [{rep.sec_status}]")
11151     print()
11152     print("  Ori 2005 has an exactly vacuum core (T_ab = 0 verified, not")
11153     print("  assumed) and a chronology-violating compact direction.")
11154     print("  van Stockum has strictly positive energy density and satisfies")
11155     print("  NEC, WEC, SEC and DEC while containing closed timelike curves.")
11156     print("  The inference 'CTC implies energy-condition violation' is false.")

```

```

11157
11158 section("Morris-Thorne: the converse control")
11159 mt = energy_report(R.get("morris_thorne"),
11160                    {"t": 0, "r": 1.05, "theta": math.pi / 2, "phi": 0,
11161                     "r0": 1.0})
11162 print(f" a static traversable wormhole near the throat:")
11163 print(f"      rho = {mt.he.rho:+.6f}   NEC = {mt.nec_margin:+.6e} "
11164       f"[{mt.nec_status}]")
11165 print(" It violates the null energy condition and contains NO closed")
11166 print(" timelike curve (its temporal-function certificate is exact).")
11167 print(" So the two properties are logically independent in both")
11168 print(" directions.")
11169
11170 section("ANEC along an integrated null geodesic")
11171 for mid, pt, lam in (
11172     ("morris_thorne", {"t": 0, "r": 1.2, "theta": math.pi / 2,
11173                        "phi": 0, "r0": 1.0}, 3.0),
11174     ("schwarzschild", None, 10.0)):
11175     m = R.get(mid)
11176     res, geo = anec_on_geodesic(m, pt or m.default_point, lam_max=lam,
11177                                n_out=201)
11178     print(f" {mid}")
11179     print(f"      geodesic kind      : {geo.kind}")
11180     print(f"      norm drift g(k,k)      : {geo.norm_drift:.3e} "
11181           f"(independent accuracy check)")
11182     print(f"      affine range           : [{geo.affine[0]:.1f}, "
11183           f" {geo.affine[-1]:.1f}], {res.n_samples} samples")
11184     print(f"      int T_ab k^a k^b dl     : {res.value:+.6e} "
11185           f"+- {res.error_estimate:.1e}")
11186     print(f"      status                  : {res.status}")
11187
11188 if __name__ == "__main__":
11189     main()
11190

```

examples/ex05_horizon_taxonomy.py — 68 lines

```

11191 """
11192 EXAMPLE 5 -- Horizon taxonomy: four different objects that are not each other.
11193
11194 WHAT THIS UNIFIES
11195 "There is a horizon, therefore there may be a time machine" is the single
11196 most common category error in this subject. v1.0 stated the taxonomy in
11197 a table and implemented one solver ( $g^{rr} = 0$ ) that cannot tell the four
11198 objects apart. v2.0 computes each separately and attaches to each the
11199 inference it must never license.
11200
11201 Reissner-Nordstrom is the decisive control: it HAS an inner Cauchy
11202 horizon and has NO chronology violation. Misner is the converse: a
11203 chronology horizon with no curvature singularity anywhere.
11204
11205 DATA REQUIRED
11206 the metric; which coordinate to solve in; which coordinate direction is
11207 the candidate Killing generator; for the chronology horizon, a parameter
11208 slice and a search range.
11209 """
11210
11211 from _common import banner, section
11212
11213 from ctcf_a import registry as R
11214 from ctcf_a.horizons import horizon_report
11215
11216 def show(mid, variable="r", killing=0, chron=None, fixed=None, curvature=True):
11217     m = R.get(mid)
11218     rep = horizon_report(m, variable, killing, chron, fixed, curvature)
11219     print(f" {mid} (solving in {variable})")
11220     for c in rep["candidates"]:
11221         sols = c["solutions"] or [f"{v:.12g}" for v in c["numeric_roots"]]
11222         print(f"      {c['kind']}<26 {sols if sols else 'none'}")
11223         print(f"      forbidden: {c['forbidden_inference']}")
11224     print()
11225
11226 def main() -> None:
11227     banner(5, "Horizon taxonomy and the inferences each object forbids")
11228
11229     section("Reissner-Nordstrom: a Cauchy horizon and NO chronology violation")
11230     show("reissner_nordstrom")
11231
11232     section("Schwarzschild: an event horizon, no chronology horizon")
11233     show("schwarzschild")
11234
11235     section("de Sitter: a cosmological horizon, no chronology horizon")
11236     show("de_sitter_static")
11237
11238     section("Misner: a chronology horizon and NO curvature singularity")
11239     show("misner", variable="T", killing=1, chron=(-2.0, 2.0), fixed={},
11240          curvature=True)
11241
11242     section("Ori 2007 core: chronology horizon at r = 2 mu")
11243     show("ori_2007_core", variable="r", killing=0, chron=(0.2, 6.0),
11244          fixed={"mu": 1.0}, curvature=True)
11245
11246     section("Reading")
11247     print(" Reissner-Nordstrom produces two Killing horizons and exactly one")
11248     print(" curvature singularity (r = 0) and NO chronology horizon.")
11249     print(" Misner produces a chronology horizon at T = 0 with an identically")
11250     print(" constant determinant and no curvature singularity at all.")
11251     print(" The two objects are therefore logically independent, and the")
11252     print(" architecture refuses to convert one into the other.")
11253
11254 if __name__ == "__main__":
11255     main()

```

examples/ex06_claim_ladder.py — 109 lines

```

11259 """
11260 EXAMPLE 6 -- Scope and the claim ladder: what the system refuses to say.
11261
11262 WHAT THIS UNIFIES
11263     v1.0 stated the claim ladder in Appendix D and the scope rules in
11264     sections 2, 9.2 and 14, and enforced neither in code. Nothing stopped
11265     ``global_model_test`` from returning a bare ``CTC_PROVED`` for a REDUCED
11266     predicate, and nothing stopped a sector-clear result from being read as a
11267     no-CTC proof.
11268
11269     In v2.0 both are types. This example is mostly a list of things the
11270     architecture REFUSES to do, because for a falsification architecture the
11271     refusals are the product.
11272
11273 DATA REQUIRED
11274     nothing beyond the registry; this is the evidence logic, not physics.
11275 """
11276
11277 from _common import banner, section
11278
11279 from ctcfac import registry as R
11280 from ctcfac.causality import (compact_orbit_test, compact_orbit_verdict,
11281                               global_predicate_verdict)
11282 from ctcfac.status import (ClaimLevel, ClaimScope, CTCStatus, Gate, GateRecord,
11283                             GateResult, OrbitResult, OrbitStatus, OverclaimError,
11284                             Verdict)
11285
11286
11287 def main() -> None:
11288     banner(6, "Scope, the claim ladder, and the refusals")
11289
11290     section("1. A clear sector is UNRESOLVED, never NO_CTC_CERTIFIED")
11291     v = compact_orbit_verdict(R.get("schwarzschild"),
11292                               R.get("schwarzschild").default_point)
11293     print(f" Schwarzschild azimuthal sector margin = {v.margin:+.6e}")
11294     print(f" status = {v.status.value} (NOT NO_CTC_CERTIFIED)")
11295     print(f" reason = {v.reason[:150]}")
11296
11297     section("2. A reduced predicate cannot be printed at global scope")
11298     m = R.get("time_shifted_wormhole")
11299     v = global_predicate_verdict(m, m.default_point)
11300     print(f" {v.render()}")
11301     print(f" scope is {v.scope.value}: a route-closure model under declared")
11302     print(f" synchronisation assumptions, not a theorem about wormholes.")
11303
11304     section("3. UNRESOLVED is absorbing")
11305     u = Verdict(CTCStatus.UNRESOLVED, ClaimScope.SECTOR, float('nan'),
11306                "loop_search", "optimiser found nothing")
11307     try:
11308         u.promote(ClaimScope.GLOBAL, "verified certificate")
11309         print(" ERROR: promotion succeeded (this would be a bug)")
11310     except OverclaimError as exc:
11311         print(f" refused: {exc}")
11312
11313     section("4. A negative result needs a named certificate to go global")
11314     n = Verdict(CTCStatus.NO_CTC_CERTIFIED, ClaimScope.DECLARED_DOMAIN, 1.0,
11315                "temporal_function", "verified on a box")
11316     try:
11317         n.promote(ClaimScope.GLOBAL, "it looks fine")
11318         print(" ERROR: promotion succeeded")
11319     except OverclaimError as exc:
11320         print(f" refused: {str(exc)[:130]}")
11321     ok = n.promote(ClaimScope.GLOBAL, "verified temporal-function certificate")
11322     print(f" accepted with a certificate: scope = {ok.scope.value}")
11323
11324     section("5. The claim ladder blocks statements above the evidence")
11325     g = GateRecord()
11326     g.set(Gate.V0_PARSE, GateResult.PASS)
11327     g.set(Gate.V1_ALGEBRA, GateResult.PASS)
11328     g.set(Gate.V4_CTC_WITNESS, GateResult.PASS)
11329     v = Verdict(CTCStatus.CTC_PROVED, ClaimScope.DECLARED_DOMAIN, -20.55,
11330                "compact_orbit", "explicit closed timelike orbit", "godel",
11331                gates=g)
11332     print(f" highest permitted level: {v.permitted_level().name}")
11333     print(f" {v.render()}")
11334     for want in (ClaimLevel.L5_ENERGY, ClaimLevel.L6_CONSTRUCTIBILITY,
11335                 ClaimLevel.L7_STABILITY, ClaimLevel.L8_SEMICLASSICAL):
11336         try:
11337             v.render(want)
11338             print(f" ERROR: {want.name} was allowed")
11339         except OverclaimError as exc:
11340             print(f" refused {want.name:24s}: {str(exc)[-70:]}")
11341
11342     section("6. L9 is unreachable by construction")
11343     g2 = GateRecord()
11344     for gate in Gate:
11345         g2.set(gate, GateResult.PASS)
11346     print(f" every gate PASS -> highest permitted level = "
11347           f"{g2.highest_permitted_level().name}")
11348     print(" L9 (quantum-gravity consistency) is not emittable because no")
11349     print(" accepted complete theory exists to certify it against.")
11350
11351     section("7. External specifications refuse to produce a metric")
11352     for mid in ("tipler_cylinder", "ori_2007_full", "paired_warp"):
11353         try:
11354             R.get(mid).require_metric()
11355             print(f" ERROR: {mid} returned a metric")
11356         except ValueError as exc:
11357             print(f" {mid:20s} -> {str(exc)[:110]}")
11358
11359     section("8. The claim-level vocabulary actually in use")
11360     for lvl in ClaimLevel:
11361         from ctcfac.status import CLAIM_TEXT
11362         mark = " (unreachable)" if lvl is ClaimLevel.L9_QUANTUM_GRAVITY else ""
11363         print(f" {lvl.name:<26} {CLAIM_TEXT[lvl]}{mark}")
11364
11365

```

```

11366 if __name__ == "__main__":
11367     main()

```

examples/ex07_theorems_and_quantum.py — 122 lines

```

11368 """
11369 EXAMPLE 7 -- Theorem hypotheses and the quantum gate.
11370
11371 WHAT THIS UNIFIES
11372     A theorem is a conditional. v1.0's risk register lists three separate
11373     ways of quoting a conclusion without its hypotheses (FM-010, FM-011,
11374     FM-012) and describes the mitigation as a "hypothesis checklist" that was
11375     never shipped. Here every theorem is an object whose hypotheses are
11376     CHECKED before any conclusion is licensed, and every one carries the
11377     inference it must never be used for.
11378
11379     The quantum layer is the same discipline applied to <T_mu_nu>.ren: a
11380     metric does not determine it, so the engine refuses to produce a number
11381     without a full field/state/renormalisation specification.
11382
11383 DATA REQUIRED
11384     for theorems: the registry record (matter, topology, horizon metadata).
11385     for the quantum layer: field content, spin, mass, couplings, state,
11386     state class, boundary conditions, renormalisation prescription, sampling
11387     function, trajectory, spacetime dimension. All of them.
11388 """
11389
11390 from _common import banner, section
11391
11392 from ctcf_a import registry as R
11393 from ctcf_a.qft import (QEI_REGISTRY, QuantumSpecification, assess, evaluate_qei,
11394                        krw_hypotheses)
11395 from ctcf_a.theorems import THEOREMS, apply
11396
11397 def main() -> None:
11398     banner(7, "Theorem applicability and the quantum admissibility gate")
11399
11400     section("The registered theorems and their status")
11401     for k in sorted(THEOREMS):
11402         th = THEOREMS[k]
11403         print(f" {k} [{th.status_note}]")
11404         print(f"         {th.title} -- {th.reference}")
11405         print(f"         permitted label : {th.permitted_label}")
11406         print(f"         forbidden use      : {th.forbidden_use}")
11407
11408     section("Applicability across representative models")
11409     models = ["minkowski", "schwarzschild", "morris_thorne", "misner",
11410              "godel", "ori_2005", "alcubierre"]
11411     keys = ["stable_causality", "topological_censorship",
11412            "chronology_protection", "kay_radzikowski_wald",
11413            "morris_thorne_energy", "ori_counterexample"]
11414     print(f" {models[:<20]} + " + ".join(f"{k[:14]:>17}" for k in keys))
11415     short = {"LICENSED": "LICENSED", "HYPOTHESES_UNVERIFIED": "unverified",
11416             "NOT_APPLICABLE": "n/a"}
11417     for mid in models:
11418         m = R.get(mid)
11419         row = ".join(f"{short[apply(k, m).applicability.value]:>17}" for k in keys)
11420         print(f" {mid:<20}{row}")
11421
11422     section("Why 'chronology protection' is never LICENSED as a theorem")
11423     m = R.get("misner")
11424     rep = apply("chronology_protection", m)
11425     print(f" misner -> {rep.applicability.value}")
11426     for h, v in rep.hypothesis_results.items():
11427         print(f"         {h:<32} {v}")
11428     print(f" {rep.note[:200]}")
11429
11430     section("Kay-Radzikowski-Wald hypotheses, model by model")
11431     for mid in ("misner", "grant_space", "ads4_periodic", "godel", "schwarzschild"):
11432         h = krw_hypotheses(R.get(mid))
11433         print(f" {mid:<20} horizon={h['chronology_horizon_present']!s:<6} "
11434               f"compactly_generated={h['compactly_generated']!s:<6} "
11435               f"applicable={h['applicable']}")
11436
11437     section("The quantum layer refuses to guess")
11438     a = assess(R.get("misner"), None)
11439     print(f" no specification supplied -> {a.status.value}")
11440     print(f"         {a.note[:200]}")
11441
11442     section("With a complete specification")
11443     spec = QuantumSpecification(
11444         field_content="massless minimally coupled scalar", spin="0", mass=0.0,
11445         couplings={"xi": 0.0}, state="Minkowski-like Hadamard state",
11446         state_class="Hadamard", boundary_conditions="periodic on the quotient",
11447         renormalisation="Hadamard point-splitting", sampling_function="Lorentzian",
11448         trajectory="static inertial", spacetime_dimension=4)
11449     print(f" specification complete: {spec.complete}")
11450     a = assess(R.get("misner"), spec)
11451     print(f" misner -> {a.status.value}")
11452     print(f"         Hadamard : {a.hadamard}")
11453     print(f"         <T>_ren : {a.renormalised_stress}")
11454     print(f"         backreact: {a.backreaction}")
11455
11456     section("Polarised hypersurfaces feed the quantum layer")
11457     from ctcf_a.quotient_ctc import polarized_hypersurfaces
11458     g = R.get("grant_space")
11459     base = {"t": 0.0, "x": 0.8, "y": 0.0, "z": 0.0, "b": 1.0, "alpha": 1.0}
11460     pol = [r for r in polarized_hypersurfaces(g, base, "x", 0.05, 3.0, 4)
11461            if r["value"] > 0]
11462     a = assess(g, spec, pol)
11463     print(f" grant_space -> {a.status.value}")
11464     for loc in a.divergence_loci[:4]:
11465         print(f"         winding {loc['winding']}: {loc['variable']} = "
11466               f"{loc['value']:.12f}")
11467
11468     section("Quantum energy inequalities are keyed by their assumptions")
11469     for k, rule in QEI_REGISTRY.items():
11470

```

```

11471         print(f"    {k}")
11472         print(f"        field={rule.field_content}, dim={rule.dimension}, "
11473               f"state={rule.state_class}")
11474         print(f"        sampling={rule.sampling}, trajectory={rule.trajectory}")
11475         print(f"        {rule.reference}")
11476         res = evaluate_qei(spec, {"tau": 1.0})
11477         print(f"    matched rule : {res.rule}")
11478         print(f"    bound      : {res.bound:+.9e}  (= -3/(32 pi^2 tau^4))")
11479         bad = QuantumSpecification(field_content="Dirac spinor", state="vacuum",
11480                                   renormalisation="Hadamard", state_class="Hadamard",
11481                                   sampling_function="Gaussian",
11482                                   trajectory="uniformly accelerated")
11483         res = evaluate_qei(bad, {"tau": 1.0})
11484         print(f"    mismatched specification -> applicable={res.applicable}")
11485         print(f"        {res.reason[:190]}")
11486
11487
11488 if __name__ == "__main__":
11489     main()

```

examples/ex08_full_pipeline.py — 205 lines

```

11490 """
11491 EXAMPLE 8 -- End-to-end pipeline on one model, and the data it needs.
11492
11493 WHAT THIS UNIFIES
11494     Everything. This is the single-model dossier the architecture produces
11495     when the whole layer stack is run: registry record, structural gates,
11496     curvature, the causal detector, the causal boundary, the horizon
11497     taxonomy, energy conditions, theorem applicability, the quantum gate,
11498     constructibility, the claim ladder and an immutable run record.
11499
11500     Read together with the DATA REQUIREMENTS block at the end, this is the
11501     answer to "what do I actually have to supply for the system to work".
11502 """
11503
11504 from _common import banner, section
11505
11506 import json
11507 import numpy as np
11508 import sympy as sp
11509
11510 from ctcf_a import registry as R
11511 from ctcf_a.adm import constructibility_report, evaluate_constraints
11512 from ctcf_a.causality import compact_orbit_verdict
11513 from ctcf_a.continuation import causal_phase_boundary, reduced_causal_margin_symbolic
11514 from ctcf_a.energy import energy_report
11515 from ctcf_a.horizons import horizon_report
11516 from ctcf_a.provenance import new_run, reproduction_script, source_tree_digest
11517 from ctcf_a.qft import QuantumSpecification, assess
11518 from ctcf_a.report import analyse
11519 from ctcf_a.theorems import applicability_matrix
11520 from ctcf_a.validate import validate_model
11521
11522 MODEL = "ori_2007_core"
11523 POINT = {"v": 0.0, "r": 1.0, "theta": 1.0, "phi": 0.0, "mu": 1.0, "P": 1.0}
11524
11525
11526 def main() -> None:
11527     banner(8, f"Full pipeline on {MODEL}")
11528     m = R.get(MODEL)
11529
11530     section("L0 -- registry record")
11531     rec = m.as_record()
11532     for k in ("model_id", "title", "kind", "theory", "coordinates",
11533             "parameters", "signature_convention", "topology"):
11534         print(f"    {k:<24}: {rec[k]}")
11535     print(f"    {'metric components':<24}:")
11536     for k, v in sorted(rec["metric_components"].items()):
11537         print(f"        g_{k} = {v}")
11538     print(f"    {'identifications':<24}: {rec['identifications']}")
11539     print(f"    {'matter':<24}: {rec['matter']}")
11540     print(f"    {'domain':<24}: {[c['name'] + ': ' + c['expression'] for c in rec['domain']]}")
11541     print(f"    {'source refs':<24}: {rec['provenance']['source_refs']}")
11542
11543     section("L0/L1/L2/L3 -- structural and algebraic gates")
11544     rep = validate_model(m, POINT, cross_check=True)
11545     for g, r in rep.gates.as_dict()["results"].items():
11546         note = rep.gates.as_dict()["notes"].get(g, "")
11547         print(f"    {g:<22} {r:<16} {note[:80]}")
11548     cc = rep.details.get("einstein_cross_check", {})
11549     if cc:
11550         print(f"    independent curvature routes differ by "
11551               f"    {cc['max_difference']:.3e} (numeric error estimate "
11552               f"    {cc['numeric_error_estimate']:.3e})")
11553
11554     section("L4 -- symmetry: is the declared generator really Killing?")
11555     for c in rep.details["compact_generators"]:
11556         print(f"    generator {c['name']}: Killing = {c['is_killing']}, "
11557               f"    residual = {c['residual']}")
11558
11559     section("L5 -- causal detector")
11560     v = compact_orbit_verdict(m, POINT)
11561     print(f"    {v.render()}")
11562     print(f"    evidence: {json.dumps(v.evidence, default=str)[:220]}")
11563
11564     section("L16 -- causal phase boundary, derived from the metric")
11565     mu_expr = reduced_causal_margin_symbolic(m)
11566     print(f"    reduced causal margin: mu(r) = {sp.simplify(mu_expr)}")
11567     th = causal_phase_boundary(m, "r", (0.5, 6.0), {"mu": 1.0}, {},
11568                               analytic_value=2.0, analytic_form="r = 2 mu")
11569     print(f"    boundary r* = {th.value:.15f}    closed form r = 2 mu -> 2.0")
11570     print(f"    bracket [{th.bracket[0]:.15f}, {th.bracket[1]:.15f}], "
11571           f"width {th.bracket_width:.2e}, error {th.analytic_error:.2e}")
11572
11573     section("L8 -- horizon taxonomy")
11574     h = horizon_report(m, "r", 0, (0.2, 6.0), {"mu": 1.0})
11575     for c in h["candidates"]:

```



```

11576     sols = c["solutions"] or [f"{x:.12g}" for x in c["numeric_roots"]]
11577     print(f" {c['kind']:<26} {sols}")
11578
11579 section("L9 -- energy conditions, on shell")
11580 e = energy_report(m, POINT)
11581 print(f"   Hawking-Ellis type : {e.he.type.value}")
11582 print(f"   rho                : {e.he.rho:+.9f}")
11583 print(f"   pressures             : {[round(p, 9) for p in e.he.pressures]}")
11584 print(f"   NEC {e.nec_margin:+.6e} [{e.nec_status}] "
11585       f"   WEC {e.wec_margin:+.6e} [{e.wec_status}]"
11586       f"   SEC {e.sec_margin:+.6e} [{e.sec_status}] "
11587       f"   DEC {e.dec_margin:+.6e} [{e.dec_status}]"
11588       print(f"   cross-check [A-B] : {e.cross_check_error:.3e}")
11589
11590 section("L12 -- constructibility (constraints evaluated, evolution NOT)")
11591 cr = constructibility_report(m, POINT)
11592 print(f"   status : {cr['status']}")
11593 if "error" in cr:
11594     print(f"   reason : {cr['error'][:240]}")
11595     print("   This is an honest NOT_APPLICABLE, not a silent pass: the")
11596     print("   constant-v slices of a null-adapted chart are degenerate, so")
11597     print("   the constraints are undefined on this foliation. The L12")
11598     print("   demonstration on a spacelike slicing is shown below for")
11599     print("   Schwarzschild.")
11600     from ctcfa.adm import decompose, evaluate_constraints as ec
11601     sm = R.get("schwarzschild")
11602     d = decompose(sm)
11603     c2 = ec(sm, sm.default_point)
11604     print(f"   Schwarzschild lapse N = {sp.simplify(d.lapse)}, "
11605           f"   shift = {list(d.shift)}, K = 0 : "
11606           f"   {sp.simplify(d.extrinsic_curvature).is_zero_matrix}")
11607     print(f"   Hamiltonian residual = {c2.hamiltonian_value}, "
11608           f"   momentum = {c2.momentum_values}, satisfied = {c2.satisfied}")
11609 if "constraints" in cr:
11610     c = cr["constraints"]
11611     print(f"   Hamiltonian residual : {c['hamiltonian_value']}")
11612     print(f"   momentum residuals   : {c['momentum_values']}")
11613     print(f"   satisfied               : {c['satisfied']}")
11614 print(f"   evolution: {cr['evolution']['status']}")
11615 print(f"   {cr['evolution']['note'][:190]}")
11616
11617 section("L15 -- theorem applicability")
11618 for row in applicability_matrix(m):
11619     print(f"   {row['theorem']:<26} {row['applicability']:<24} "
11620           f"   {row['permitted_label'] or '-'}")
11621
11622 section("L13/L14 -- quantum gate")
11623 a = assess(m, None)
11624 print(f"   without a specification -> {a.status.value}")
11625 print(f"   KRW hypotheses          -> applicable = "
11626       f"   {a.theorem_hypotheses['applicable']}")
11627
11628 section("Headline verdict and permitted claim level")
11629 dossier = analyse(m, POINT, include_energy=False, include_theorems=False)
11630 hd = dossier["headline"]
11631 print(f"   {hd['line']}")
11632 print(f"   permitted claim level : {hd['claim_level']} -- {hd['claim_text']}")
11633
11634 section("Immutable run record and reproduction script")
11635 digest, files = source_tree_digest(
11636     __import__("pathlib").Path(R.__file__).resolve().parent)
11637 rr = new_run(MODEL, "compact_orbit", POINT, v.as_dict(), digest,
11638             {"tol": 1e-10}, m.provenance.source_refs)
11639 print(f"   code digest : {digest}")
11640 print(f"   files hashed : {len(files)}")
11641 print(f"   run content  : {rr.content_hash()}")
11642 print(f"   environment  : python {rr.environment['python']}, "
11643       f"   sympy {rr.environment['modules']['sympy']}, "
11644       f"   numpy {rr.environment['modules']['numpy']}")
11645 print()
11646 print("   Minimal independent reproduction script:")
11647 for line in reproduction_script(MODEL, "compact_orbit", POINT).splitlines():
11648     print(f"       {line}")
11649
11650 section("DATA REQUIREMENTS -- what a new model must supply")
11651 reqs = [
11652     ("L0 registry", "coordinate ordering; signature convention; the metric "
11653                    "components; parameter list and domain; topology and "
11654                    "every identification; matter content; source refs"),
11655     ("L1 algebra", "nothing further: the inverse, connection and curvature "
11656                    "are computed"),
11657     ("L2 field equations", "a closed-form T_ab if the model claims to solve "
11658                            "them; otherwise the residual only DEFINES the "
11659                            "required source"),
11660     ("L3 topology", "deck generators AND their inverses, as explicit "
11661                    "coordinate maps; a probe point for the "
11662                    "time-orientation test"),
11663     ("L4 symmetry", "which coordinate directions are claimed compact, and "
11664                    "their periods"),
11665     ("L5 detector", "a fully specified numeric point: every coordinate and "
11666                    "every parameter"),
11667     ("L6 loop search", "a centre point, a winding number, and a mode count"),
11668     ("L7 certificate", "a candidate temporal function; a compact box if the "
11669                       "proof is to be certified by interval arithmetic"),
11670     ("L8 horizons", "which coordinate to solve in; which direction is the "
11671                    "candidate Killing generator"),
11672     ("L9 energy", "the cosmological constant; the evaluation point"),
11673     ("L12 initial data", "a slicing; matter density and current if not "
11674                         "vacuum"),
11675     ("L13/L14 quantum", "field content, spin, mass, couplings, state, state "
11676                        "class, boundary conditions, renormalisation "
11677                        "prescription, sampling function, trajectory, "
11678                        "dimension"),
11679     ("L16 continuation", "a parameter, a bracket, values for every other "
11680                          "symbol, and a coordinate box"),
11681     ("L17 optimiser", "a SEALED cost metric: scales and weights frozen "
11682                      "before the run"),
11683     ("L18 accessibility", "an observer point and a predicate for the "
11684                          "chronology-violating set"),
11685     ("L19 meta-analysis", "nothing: features are computed from the other "

```

```

11686         "layers, never from the model's name or its "
11687         "expected status"),
11688     ]
11689     for k, v_ in reqs:
11690         print(f"    {k:<20} {v_}")
11691
11692 if __name__ == "__main__":
11693     main()
11694

```

examples/ex09_meta_analysis.py — 90 lines

```

11695 """
11696 EXAMPLE 9 -- Cross-family meta-analysis with held-out validation.
11697
11698 WHAT THIS UNIFIES
11699     This is what a registry is FOR. Every model's feature vector is built
11700     only from quantities the other layers compute -- never from the model's
11701     name and never from its expected status -- and candidate invariant
11702     precursors are derived on a training split, then scored on a held-out
11703     split the derivation never saw.
11704
11705     v1.0 described this as ALG-META-001 and shipped nothing. The governance
11706     rule is unchanged and now enforced: the strongest emittable label is
11707     CANDIDATE_PRECURSOR.
11708
11709 DATA REQUIRED
11710     nothing beyond the registry; the features come from L3, L4, L5 and L9.
11711
11712 """
11713 from _common import banner, section
11714
11715 from ctcf_a import registry as R
11716 from ctcf_a.meta import FEATURE_NAMES, cross_family_report
11717
11718 def main() -> None:
11719     banner(9, "Cross-family meta-analysis")
11720     rep = cross_family_report(R.models())
11721
11722     section("Corpus")
11723     print(f"  models : {rep['n_models']}")
11724     for k, v in sorted(rep['label_counts'].items()):
11725         print(f"    {k:<24} {v}")
11726
11727     section("Feature table (computed, never copied from the label)")
11728     keys = ["has_compact_generator", "n_deck_generators",
11729            "generator_is_killing", "sector_margin_sign", "vacuum",
11730            "flat_covering", "compact_generator_is_time_direction"]
11731     hdr = "".join(f"{k[:11]:>13}" for k in keys)
11732     print(f"  {'model':<26}{'label':<22}{hdr}")
11733     for row in rep["table"]:
11734         vals = "".join(
11735             f"({'.' if row[k] is None else f'{row[k]:g}')):>13" for k in keys)
11736         print(f"    {row['model_id']:<26}{row['label'][:21]:<22}{vals}")
11737
11738     section("Candidate precursors, scored on the held-out split")
11739     print(f"  {'feature':<38}{'rule':<10}{'train':>8}{'held-out':>10} status")
11740     for r in rep["rules"][:10]:
11741         rule = f"{r['direction']} {r['threshold']}:g"
11742         print(f"    {r['feature']:<38}{rule:<10}{r['train_accuracy']:>8.3f}"
11743               f"{r['heldout_accuracy']:>10.3f} {r['status']}")
11744
11745     section("Leakage check")
11746     lk = rep["leakage"]
11747     print(f"  perfectly separating features: {lk['perfectly_separating_features']}")
11748     print(f"  {lk['note']}")
11749
11750     section("Governance")
11751     print(f"  {rep['governance']}")
11752
11753     section("Reading -- including what is WRONG with the top rule")
11754     print("  The top-scoring rule on this corpus is")
11755     print("    has_temporal_function_candidate <= 0.5 -> chronology violating")
11756     print("    with a perfect held-out score. It is nonetheless SCIENTIFICALLY")
11757     print("    WORTHLESS, and saying so is the point of running the analysis.")
11758     print("    A candidate temporal function is something a HUMAN registers when")
11759     print("    they already believe the spacetime is choral; the feature is")
11760     print("    therefore a curation artefact, not a property of the geometry.")
11761     print("    The leakage check does not catch it because BTZ is choral and")
11762     print("    carries no registered candidate, so the separation is 94.7% on")
11763     print("    the training split rather than exact.")
11764     print()
11765     print("  The same run also shows sector_margin_sign separating perfectly,")
11766     print("    which the leakage check DOES flag: that feature is the detector's")
11767     print("    own output, so agreement with the label is a tautology.")
11768     print()
11769     print("  What survives as a genuine, non-tautological observation is that")
11770     print("    every chronology-violating entry in this registry carries a")
11771     print("    nontrivial identification structure -- a compact Killing generator")
11772     print("    or a deck group -- and no choral entry with a global temporal")
11773     print("    function does. That is the L3 gate stated as a corpus fact. It")
11774     print("    is emitted as CANDIDATE_PRECURSOR and nothing stronger, and it")
11775     print("    would need an independent corpus, not this one, to become a")
11776     print("    claim.")
11777     print()
11778     print("  Reporting a top rule that the architecture then disqualifies is")
11779     print("    the intended behaviour of a falsification framework.")
11780
11781 if __name__ == "__main__":
11782     main()
11783

```

examples/ex10_perturbation_and_minimal.py — 153 lines

```

11785 """

```



```

11786 EXAMPLE 10 -- On-shell perturbation, minimal CTC birth, and initial data.
11787
11788 WHAT THIS UNIFIES
11789     Three gates that v1.0 stated and did not enforce:
11790
11791     L11 "Off-shell perturbations cannot establish physical robustness"
11792         -- v1.0's perturb_metric added an arbitrary tensor and nothing
11793         stopped the result being quoted as stability evidence.
11794     L17 "The cost metric and normalization scales are preregistered;
11795         otherwise the word minimal is meaningless"
11796         -- v1.0 had no cost object at all.
11797     L12 "Constructibility from regular initial data is stronger than
11798         existence of an eternal metric"
11799         -- v1.0 had constraint expressions and no solver.
11800
11801 DATA REQUIRED
11802     for L11: a base point, a parameter to move, and a step; or an explicit
11803     symmetric tensor whose on-shell status will be MEASURED.
11804     for L17: a SEALED cost metric -- scales and weights frozen before the run.
11805     for L12: a slicing, and matter data if not vacuum.
11806 """
11807
11808 from _common import banner, section
11809
11810 import numpy as np
11811 import sympy as sp
11812
11813 import scipy.optimize
11814 from ctcfactory import registry as R
11815 from ctcfactory.adm import decompose, evaluate_constraints, york_hamiltonian_ld
11816 from ctcfactory.optimize import CostMetric, UnsealedCostError, minimal_birth
11817 from ctcfactory.perturbation import (OnShellStatus, parameter_family_perturbation,
11818                                     robustness_scan, tensor_perturbation)
11819
11820 def main() -> None:
11821     banner(10, "On-shell perturbation, minimal birth, and regular initial data")
11822
11823     section("L11a -- ON-SHELL: move along the exact family")
11824     m = R.get("godel_type")
11825     base = dict(m.default_point)
11826     rows = robustness_scan(m, "m", base, [-0.4, -0.2, -0.05, 0.05, 0.2, 0.4,
11827                                         0.8, 1.0])
11828     print(f"   base: omega = {base['omega']}, m = {base['m']}, r = {base['r']}")
11829     print(f"   {'delta m':>10}{'margin':>18}{'status':>20}{'changed':>10}")
11830     for r in rows:
11831         print(f"   {r['delta']:>10.2f}{r['margin']:>18.6e}"
11832               f"   {r['status'].replace('SECTOR ','')>20}{r['changed']:>10}")
11833     print("   Every row here is an exact solution of the same family, so the")
11834     print("   sign changes are physical, not artefacts of an arbitrary tensor.")
11835
11836     section("L11b -- OFF-SHELL: the same perturbation, measured and rejected")
11837     vs = R.get("van_stockum")
11838     n = vs.dim
11839     h = sp.zeros(n, n)
11840     h[2, 2] = sp.Symbol("r", real=True) ** 2 # bump g_phi_phi
11841     res = tensor_perturbation(vs, h, 0.3, vs.default_point)
11842     print(f"   perturbation kind : {res.kind.value}")
11843     print(f"   exploratory           : {res.exploratory}")
11844     print(f"   margin {res.base_margin:+.4e} -> {res.perturbed_margin:+.4e} "
11845           f"   {status {res.status_before} -> {res.status_after}}")
11846     print(f"   {res.note[:230]}")
11847
11848     section("L17 -- a cost metric must be SEALED before it can be used")
11849     cost = CostMetric(scales={"m": 1.0, "omega": 1.0},
11850                     weights={"m": 1.0, "omega": 1.0},
11851                     label="Godel-type family, unit scales")
11852
11853     try:
11854         cost.evaluate({"m": 0.1})
11855         print("   ERROR: unsealed cost evaluated")
11856     except UnsealedCostError as exc:
11857         print(f"   refused: {str(exc)[:150]}")
11858     cost.seal()
11859     print(f"   sealed. digest = {cost.digest}")
11860     print(f"   J(delta m = 0.1) = {cost.evaluate({'m': 0.1}):.6f}")
11861
11862     section("L17 -- minimal on-shell perturbation that CREATES a CTC")
11863     chrontal = dict(m.default_point)
11864     chrontal["m"] = 3.0 # m > 2 omega: chrontal
11865     from ctcfactory.causality import compact_orbit_test
11866     r0 = compact_orbit_test(m, chrontal)
11867     print(f"   starting_point m = 3.0, omega = 1.0 -> {r0.status.value} "
11868           f"   {margin {r0.margin:+.4e}}")
11869     mb = minimal_birth(m, chrontal, ["m"], cost, {"m": (-2.5, 2.5)},
11870                      restarts=6)
11871     print(f"   achieved : {mb.achieved}")
11872     print(f"   delta : { {k: round(v, 9) for k, v in mb.delta.items()} }")
11873     print(f"   cost J : {mb.cost:.9f}")
11874     m_final = chrontal["m"] + mb.delta["m"]
11875     print(f"   m_final : {m_final:.9f}")
11876     print(f"   cost digest: {mb.cost_metric_digest[:32]}...")
11877     print(f"   {mb.note[:180]}")
11878
11879     print("   This is NOT 2.0, and that is the correct answer. The FAMILY")
11880     print("   boundary m = 2 omega is the r -> infinity limit; at the fixed")
11881     print("   radius r = 2 the reduced margin")
11882     print("   mu = 1 - (4 omega^2/m^2) tanh^2(m r / 2)")
11883     print("   vanishes where tanh(m) = m/2, whose root is:")
11884     import scipy.optimize as _so
11885     root = _so.brentq(lambda mm: np.tanh(mm) - mm / 2, 1.0, 1.99, xtol=1e-14)
11886     print(f"   m* (r = 2) = {root:.12f}")
11887     print(f"   engine = {m_final:.12f} |difference| = "
11888           f"   {abs(m_final - root):.2e}")
11889     print("   Asking for the cheapest change that creates a CTC AT A POINT is a")
11890     print("   different question from asking where the family stops being")
11891     print("   causal, and the architecture keeps them apart.")
11892
11893     section("L12 -- ADM decomposition and the constraints")
11894     for mid in ("schwarzschild", "flrw_exponential", "minkowski"):
11895         sm = R.get(mid)

```

```

11896     d = decompose(sm)
11897     c = evaluate_constraints(sm, sm.default_point)
11898     print(f"    {mid}")
11899     print(f"        lapse N          = {sp.simplify(d.lapse)}")
11900     print(f"        shift N^i          = {list(d.shift)}")
11901     print(f"        K_ij zero          = "
11902           f"{sp.simplify(d.extrinsic_curvature).is_zero_matrix}")
11903     print(f"        Hamiltonian res. = {c.hamiltonian_value}")
11904     print(f"        momentum res.   = {c.momentum_values}")
11905     print(f"        satisfied       = {c.satisfied} (with rho = 0)")
11906     if c.hamiltonian_value and abs(c.hamiltonian_value) > 1e-9:
11907         rho_req = c.hamiltonian_value / (16 * np.pi)
11908         print(f"            => required rho = {rho_req:.12f}; for H = 1 the")
11909         print(f"            Friedmann value  $3H^2/(8\pi)$  = "
11910               f"{3.0/(8*np.pi):.12f}. The residual is not an error: with")
11911         print(f"            no matter declared it DEFINES the source, which is")
11912         print(f"            exactly the Friedmann equation.")
11913
11914     section("L12 -- CONSTRUCTING regular initial data (conformal York solve)")
11915     r = np.linspace(1e-3, 20.0, 400)
11916     for amp in (0.01, 0.05, 0.2):
11917         rho = amp * np.exp(-r ** 2 / 2)
11918         y = york_hamiltonian_ld(rho, r)
11919         adm_mass = 2.0 * (float(y.psi[-1]) - 1.0) * float(r[-1])
11920         print(f"    rho_hat amplitude {amp:<6} -> converged={y.converged} "
11921               f"residual={y.residual:.2e} iters={y.iterations} "
11922               f"psi(0)={float(y.psi[0]):.9f}")
11923     print()
11924     print("    The conformal factor solves")
11925     print("        Laplacian(psi) + 2 pi rho_hat psi^(-3) = 0")
11926     print("    with a regular centre and psi^-> 1 at infinity. This is the step")
11927     print("    that turns 'assume regular initial data' into 'here is some'.")
11928
11929     section("What is still missing, stated plainly")
11930     print("    A convergent 3+1 EVOLUTION of that data is not implemented, so no")
11931     print("    model in this registry may be reported above claim level L5.")
11932     print("    The correct path is integration with the Einstein Toolkit, and")
11933     print("    until it exists the L12 evolution gate stays NOT_ATTEMPTED.")
11934
11935
11936 if __name__ == "__main__":
11937     main()

```

A.4 Build configuration and schemas

pyproject.toml — 39 lines

```

11938 [build-system]
11939 requires = ["setuptools>=68"]
11940 build-backend = "setuptools.build_meta"
11941
11942 [project]
11943 name = "ctcfa"
11944 version = "2.0.0"
11945 description = "CTC-FA v2.0 -- Closed Timelike Curve Falsification Architecture"
11946 readme = "README.md"
11947 requires-python = ">=3.11"
11948 authors = [{name = "Vadym Tsinderhoz"}]
11949 license = {text = "IP_NOTICE_v2.0 (proprietary; see LICENSE_IP_NOTICE_v2.0.md)"}
11950 keywords = ["general relativity", "closed timelike curves", "causality",
11951             "chronology protection", "computer algebra", "falsification"]
11952 classifiers = [
11953     "Development Status :: 4 - Beta",
11954     "Intended Audience :: Science/Research",
11955     "Programming Language :: Python :: 3.11",
11956     "Topic :: Scientific/Engineering :: Physics",
11957 ]
11958 dependencies = [
11959     "sympy>=1.13",
11960     "numpy>=1.26",
11961     "scipy>=1.11",
11962     "mpmath>=1.3",
11963 ]
11964
11965 [project.optional-dependencies]
11966 dev = ["pytest>=7.4", "jsonschema>=4.0"]
11967 report = ["reportlab>=4.0", "matplotlib>=3.8", "pdf2image>=1.16"]
11968
11969 [project.scripts]
11970 ctcfa = "ctcfa.cli:main"
11971
11972 [tool.setuptools.packages.find]
11973 where = ["src"]
11974
11975 [tool.pytest.ini_options]
11976 testpaths = ["tests"]

```

Makefile — 38 lines

```

11977 # CTC-FA v2.0 -- reproducible build
11978 PY      ?= python3
11979 SRC     = src
11980 EXPORT  = registry/metrics
11981 RESULTS = results
11982
11983 export PYTHONPATH := $(SRC)
11984
11985 .PHONY: all selftest test registry manifest examples report clean
11986
11987 all: selftest test registry manifest examples
11988
11989 selftest:
11990     [$(PY) -m ctcfa.cli selftest
11991
11992 test:
11993     [$(PY) -m pytest -q
11994
11995 registry:
11996     [$(PY) -m ctcfa.cli export-registry --out $(EXPORT)
11997     [$(PY) -c "import json,glob,jsonschema; s=json.load(open('schemas/metric.schema.json')); \
11998     [      [jsonschema.validate(json.load(open(f)), s) for f in glob.glob('$(EXPORT)/*.json')]; \
11999     [      print('registry records validate against schemas/metric.schema.json')"
12000
12001 manifest:
12002     [mkdir -p $(RESULTS)
12003     [$(PY) -m ctcfa.cli manifest --out $(RESULTS)/manifest.json --root $(SRC)/ctcfa
12004
12005 examples:
12006     [for f in examples/ex*.py; do \
12007     [ echo "==$f"; (cd examples && $(PY) $(basename $f)) || exit 1; \
12008     [done
12009
12010 matrix:
12011     [$(PY) -m ctcfa.cli matrix
12012
12013 clean:
12014     [rm -rf $(RESULTS)/manifests/* .pytest_cache */__pycache__

```

pytest.ini — 9 lines

```

12015 [pytest]
12016 testpaths = tests
12017 python_files = test_*.py
12018 addopts = -q --strict-markers
12019 markers =
12020     slow: long-running tests (symbolic curvature, loop search)
12021 filterwarnings =
12022     ignore::DeprecationWarning
12023     ignore::UserWarning

```

schemas/metric.schema.json — 107 lines

```

12024 {
12025     "$schema": "https://json-schema.org/draft/2020-12/schema",
12026     "$id": "https://ctc-fa.tsinderhoz.dev/schemas/metric.schema.json",

```

```

12027 "title": "CTC-FA v2.0 metric / model registry record",
12028 "type": "object",
12029 "required": ["model_id", "title", "version", "kind", "theory", "signature",
12030     "signature_convention", "coordinates", "parameters",
12031     "metric_defined", "topology", "identifications", "quotient",
12032     "matter", "expected_status", "expected_scope", "provenance"],
12033 "additionalProperties": false,
12034 "properties": {
12035     "model_id": {"type": "string", "pattern": "^[a-z0-9_]+$"},
12036     "title": {"type": "string", "minLength": 3},
12037     "version": {"type": "string"},
12038     "kind": {"enum": ["LOCAL_METRIC", "QUOTIENT_GLOBAL", "COMPOSITE_GLOBAL",
12039         "EXTERNAL_EXACT_SPEC"]},
12040     "theory": {"type": "string"},
12041     "signature": {"type": "array", "items": {"type": "integer"},
12042         "minItems": 3, "maxItems": 3,
12043         "description": "(n_negative, n_null, n_positive)"},
12044     "signature_convention": {"const": "(-,+,+,+)"},
12045     "coordinates": {"type": "array", "items": {"type": "string"}, "minItems": 0},
12046     "parameters": {"type": "array", "items": {"type": "string"}},
12047     "metric_defined": {"type": "boolean"},
12048     "metric_components": {
12049         "oneOf": [{"type": "null"},
12050             {"type": "object", "additionalProperties": {"type": "string"}}],
12051         "description": "upper-triangular nonzero components keyed by 'ij'",
12052     "topology": {"type": "string"},
12053     "identifications": {
12054         "type": "array",
12055         "items": {"type": "object",
12056             "required": ["coordinate", "index", "period"],
12057             "properties": {"coordinate": {"type": "string"},
12058                 "index": {"type": "integer", "minimum": 0},
12059                 "period": {"type": "string"},
12060                 "requires_killing": {"type": "boolean"},
12061                 "note": {"type": "string"}},
12062             "additionalProperties": false}},
12063     "quotient": {
12064         "type": "object",
12065         "required": ["label", "generators"],
12066         "properties": {
12067             "label": {"type": "string"},
12068             "generators": {"type": "array", "items": {
12069                 "type": "object", "required": ["name", "image", "inverse"],
12070                 "properties": {"name": {"type": "string"},
12071                     "image": {"type": "array", "items": {"type": "string"}},
12072                     "inverse": {"type": "array", "items": {"type": "string"}},
12073                     "time_orientation_preserving": {"type": ["boolean", "null"]},
12074                     "note": {"type": "string"}},
12075                 "additionalProperties": false}},
12076             "acts_freely": {"type": ["boolean", "null"]},
12077             "note": {"type": "string"}},
12078         "additionalProperties": false},
12079     "matter": {"type": "object",
12080         "required": ["label", "vacuum"],
12081         "properties": {"label": {"type": "string"},
12082             "vacuum": {"type": "boolean"},
12083             "lambda": {"type": "string"},
12084             "T_defined": {"type": "boolean"},
12085             "note": {"type": "string"}},
12086         "additionalProperties": false},
12087     "domain": {"type": "array", "items": {
12088         "type": "object", "required": ["name", "expression", "strict"],
12089         "properties": {"name": {"type": "string"},
12090             "expression": {"type": "string"},
12091             "strict": {"type": "boolean"},
12092             "note": {"type": "string"}},
12093         "additionalProperties": false}},
12094     "default_point": {"type": "object",
12095         "additionalProperties": {"type": "number"}},
12096     "expected_status": {"enum": ["CTC_PROVED", "CRITICAL", "NO_CTC_CERTIFIED",
12097         "UNRESOLVED"]},
12098     "expected_status_at_default_point": {
12099         "enum": ["CTC_PROVED", "CRITICAL", "NO_CTC_CERTIFIED", "UNRESOLVED"]},
12100     "expected_scope": {"enum": ["SECTOR", "REDUCED_MODEL", "DECLARED_DOMAIN",
12101         "GLOBAL"]},
12102     "domain_notes": {"type": "string"},
12103     "global_margin": {"type": "boolean"},
12104     "global_margin_scope": {"enum": ["SECTOR", "REDUCED_MODEL",
12105         "DECLARED_DOMAIN", "GLOBAL"]},
12106     "global_margin_note": {"type": "string"},
12107     "temporal_function": {"type": ["string", "null"]},
12108     "provenance": {"type": "object",
12109         "required": ["source_refs"],
12110         "properties": {
12111             "source_refs": {"type": "array",
12112                 "items": {"type": "string"},
12113                 "minItems": 1},
12114             "convention_note": {"type": "string"},
12115             "derivation": {"type": "string"},
12116             "implementation_status": {"type": "string"},
12117             "external_spec": {"type": "string"}},
12118         "additionalProperties": false},
12119     "metadata": {"type": "object"}
12120 },
12121 "allOf": [
12122     {"if": {"properties": {"kind": {"const": "LOCAL_METRIC"}}},
12123     "then": {"properties": {"metric_defined": {"const": true}}}},
12124     {"if": {"properties": {"kind": {"enum": ["EXTERNAL_EXACT_SPEC",
12125         "COMPOSITE_GLOBAL"]}}},
12126     "then": {"properties": {"metric_defined": {"const": false}}}},
12127     {"if": {"properties": {"global_margin": {"const": true}}},
12128     "then": {"properties": {"global_margin_scope": {"not": {"const": "GLOBAL"}}}}
12129 ]
12130 }

```

schemas/run.schema.json — 26 lines

```
12131 {
```

```

12132 "$schema": "https://json-schema.org/draft/2020-12/schema",
12133 "$id": "https://ctc-fa.tsinderhoz.dev/schemas/run.schema.json",
12134 "title": "CTC-FA v2.0 immutable run record",
12135 "type": "object",
12136 "required": ["run_id", "model_id", "method", "parameters", "result",
12137             "code_digest", "environment", "created_utc", "content_hash"],
12138 "properties": {
12139   "run_id": {"type": "string", "format": "uuid"},
12140   "model_id": {"type": "string"},
12141   "method": {"type": "string"},
12142   "parameters": {"type": "object"},
12143   "result": {"type": "object"},
12144   "code_digest": {"type": "string", "pattern": "^([0-9a-f]{64})$"},
12145   "environment": {"type": "object",
12146                 "required": ["python", "platform", "modules"]},
12147   "created_utc": {"type": "string"},
12148   "tolerances": {"type": "object"},
12149   "source_refs": {"type": "array", "items": {"type": "string"}},
12150   "review_state": {"enum": ["machine", "analyst_checked",
12151                             "independently_replicated"]},
12152   "supersedes": {"type": ["string", "null"]},
12153   "content_hash": {"type": "string", "pattern": "^([0-9a-f]{64})$"}
12154 },
12155 "additionalProperties": false
12156 }

```

schemas/verdict.schema.json — 28 lines

```

12157 {
12158   "$schema": "https://json-schema.org/draft/2020-12/schema",
12159   "$id": "https://ctc-fa.tsinderhoz.dev/schemas/verdict.schema.json",
12160   "title": "CTC-FA v2.0 scoped causal verdict",
12161   "type": "object",
12162   "required": ["model_id", "status", "scope", "margin", "method", "reason",
12163               "gates", "permitted_claim_level"],
12164   "properties": {
12165     "model_id": {"type": "string"},
12166     "status": {"enum": ["CTC_PROVED", "CRITICAL", "NO_CTC_CERTIFIED",
12167                        "UNRESOLVED"]},
12168     "scope": {"enum": ["SECTOR", "REDUCED_MODEL", "DECLARED_DOMAIN", "GLOBAL"]},
12169     "margin": {"type": ["number", "string"]},
12170     "uncertainty": {"type": "number", "minimum": 0},
12171     "method": {"type": "string"},
12172     "reason": {"type": "string"},
12173     "parameters": {"type": "object"},
12174     "gates": {"type": "object",
12175              "required": ["results", "highest_permitted_level"]},
12176     "evidence": {"type": "object"},
12177     "permitted_claim_level": {
12178       "enum": ["L0_PARSED", "L1_FIELD_EQUATIONS", "L2_CTC_VERIFIED",
12179               "L3_BOUNDARY_CHARACTERISED", "L4_REGULARITY", "L5_ENERGY",
12180               "L6_CONSTRUCTIBILITY", "L7_STABILITY", "L8_SEMICLASSICAL"],
12181       "description": "L9_QUANTUM_GRAVITY is deliberately absent: it is not emittable"}
12182   },
12183   "additionalProperties": false
12184 }

```

schemas/theorem.schema.json — 18 lines

```

12185 {
12186   "$schema": "https://json-schema.org/draft/2020-12/schema",
12187   "$id": "https://ctc-fa.tsinderhoz.dev/schemas/theorem.schema.json",
12188   "title": "CTC-FA v2.0 theorem applicability record",
12189   "type": "object",
12190   "required": ["theorem", "applicability", "hypothesis_results",
12191               "permitted_label", "note"],
12192   "properties": {
12193     "theorem": {"type": "string"},
12194     "applicability": {"enum": ["LICENSED", "HYPOTHESES_UNVERIFIED",
12195                               "NOT_APPLICABLE"]},
12196     "hypothesis_results": {"type": "object",
12197                           "additionalProperties": {"type": ["boolean", "null"]}},
12198     "permitted_label": {"type": ["string", "null"]},
12199     "note": {"type": "string"}
12200   },
12201   "additionalProperties": false
12202 }

```

End of source listing: 12202 numbered lines.

Appendix B. Corrections Index

A one-line index of Section 2, for citation. The full entry for each defect, with its evidence and its regression test, is in Section 2.

ID	Severity	Area	One-line summary
D-01	ERROR	Registry / conventions	The registry table printed $g_t \phi = +2 M a r \sin^2(\theta)/\Sigma$ while the kernel used the negative sign.
D-02	ERROR	Kernel data	The stored van Stockum metric placed the $dt d\phi$ term in one off-diagonal slot only, so $g \neq g^T$.
D-03	GAP	Causality	Time orientation was required by the definition of a CTC witness and never checked: the validator returned "orientation_status": "NOT_CHECKED" and the compact-orbit detector did not look at orientation at all.
D-04	GAP	Symmetry	A registry entry could declare any coordinate direction as a compact generator and the detector consumed it without verifying that it is a Killing vector.
D-05	ERROR	Evidence logic	Scope had no representation in the kernel.
D-06	GAP	Registry	The tube metric depends on a smooth-step regulator that is a numerical choice, not part of an exact solution; nothing in the data model recorded that, so a boundary could have been derived from it.
D-07	GAP	Registry discipline	EXTERNAL_EXACT_SPEC and COMPOSITE_GLOBAL were enum values with no enforcement: nothing prevented a later contributor from attaching an invented local tensor.
D-08	GAP	Evidence logic	The claim ladder L0-L9 was documented and never enforced; nothing stopped a level-2 result being described at level 6.
D-09	GAP	Curvature	The Kretschmann scalar raised NotImplementedError.
D-10	ERROR	Loop search	Four independent problems: (a) time orientation not checked; (b) the reported 'max norm' was a log-sum-exp SMOOTH maximum, which can be negative while the true maximum is positive; (c) a candidate was never re-validated on a denser grid or in another parametrisation; (d) nothing bounded the Fourier amplitudes, so an unbounded curve attains an unbounded negative norm by parametrisation alone.
D-11	ERROR	Validation methodology	The Godel-type causal boundary was 'recovered' by root-finding the hard-coded expression $m^2 - 4 \omega^2$, and the result $m^* = 1$.
D-12	GAP	Certification	Interval or analytic certification was named as the requirement for a formal PASS in three separate places and implemented nowhere, so no positivity statement in the architecture could ever be certified.
D-13	ERROR	Energy conditions	NEC and WEC were evaluated on 256 Fibonacci-sphere directions and labelled 'Sampled diagnostic only'; SEC and DEC were never computed at all despite being in the state vector; ANEC consumed a caller-supplied list of contractions rather than integrating a geodesic.
D-14	GAP	Topology	Topology and identifications were free-text strings.
D-15	GAP	Numerics	The precision policy ('escalates precision and recomputes' near marginal boundaries) had no implementation; sign decisions used raw double precision with a fixed tolerance.
D-16	GAP	Initial data	The ADM constraint expressions existed as symbolic functions that nothing called, and there was no way to construct regular initial data.
D-17	GAP	Theorem discipline	The theorem applicability matrix was a prose table.
D-18	GAP	Quantum layer	The quantum layer was a dataclass of four literal 'UNRESOLVED' strings.
D-19	GAP	Integrity	Seven integrity controls were listed.
D-20	GAP	Interface	The CLI was declared 'a normative target interface, not a claim that every command already exists'.
D-21	GAP	Benchmark protocol	The blind benchmark protocol was six numbered steps with no implementation, and the regression expectations were literals in the kernel source -- the opposite arrangement to a blind test.
D-22	GAP	Meta-analysis	Cross-family invariant search was specified with train/held-out discipline and never implemented.

ID	Severity	Area	One-line summary
D-23	GAP	Optimiser	The minimal-perturbation cost had no object: scales and weights could be chosen after seeing the answer, which makes 'minimal' meaningless.
D-24	ERROR	Perturbation	An arbitrary symmetric tensor could be added to a metric and the resulting causal margin quoted, with nothing recording that the perturbation was off shell.
D-25	ERROR	Registry semantics	One 'expected' field served both the family-level statement and the point-level statement, so the Godel-type family (UNRESOLVED as a family, because its causal character depends on m and ω) was listed twice with different parameters and no parameter column.
D-26	GAP	Domain guards	Nothing checked that a causal-boundary root lay inside the physical parameter domain, or that the normalisation used to define the margin was still valid there.
D-27	GAP	Horizon taxonomy	The four horizon objects were a prose table backed by one solver for $g^{rr} = 0$, which cannot distinguish them.
D-28	GAP	Cross-validation	Independent tensor validation was listed as planned external work, so a single algebra bug in the one implementation would have been invisible.

Appendix C. Versioning and Change-Control Baseline

Version	Date	Status	Change
1.0	31 August 2026	superseded	unified architecture, 22-model registry, symbolic/numerical kernel, self-test, V&V, prior art and provenance controls
2.0	31 August 2026	current baseline	28 defects corrected with evidence and regression tests; L3 topology and quotient engine; L5b deck-orbit detector; certified interval arithmetic; exact energy-condition classification; non-circular boundary derivation; scope and claim ladder enforced as types; 33 models; 118 tests; 10 worked examples; all seven integrity controls implemented
2.x	future	patch / minor	bug fixes or backward-compatible registry and verification additions; causal claim logic must not silently weaken
3.0	future	major	Einstein Toolkit bridge and on-shell evolution; independent CAS validation; expanded corpus to 100+ models; externally evaluated blind benchmark
4.0	future	major	state-specific semiclassical layer; systematic nonlinear stability

Change-control rule. Any change to formulas, sign conventions, expected benchmark statuses, theorem hypotheses or PASS/FAIL semantics is a scientific change and must be recorded with rationale, source citation, regression impact and reviewer identity. Historical releases must remain available so that results can be reproduced exactly.

Appendix D. Claim Ladder

Level	Claim permitted	Gates that must have passed
L0	Metric/model parsed and internally consistent.	V0
L1	Field equations and tensor identities verified in the declared scope.	V0, V1, V2, V3
L2	CTC mathematically verified.	V0, V1, V4
L3	CTC region and causal boundary characterised.	V0, V1, V5
L4	Regularity and geodesic-completeness status established.	V0, V1, V2
L5	Energy requirements established.	V0, V1, V2, V3
L6	Formation from regular admissible initial data established.	V0, V3, V7
L7	Perturbative and nonlinear stability established.	V0, V3, V7
L8	Semiclassical consistency established.	V0, V8
L9	Quantum-gravity consistency established.	unreachable by construction

A model must not be described at a higher level than its weakest unresolved gate. In v2.0 this is enforced: Verdict.render raises OverclaimError above the permitted level, and no gate record in the shipped code sets V7_DYNAMICS to PASS, so no registered model currently exceeds L5. In particular, a LEVEL 2 mathematically verified CTC is not a physical machine for travelling to the past.

Appendix E. Release Manifest

Artifact	Identifier
Architecture document	CTC-FA-2026-V2.0-EN
System architect	Vadym Tsinderhoz
Date	31 August 2026
Supersedes	CTC-FA-2026-V1.0-EN-0831-VT
Kernel package	src/ctcfa (29 files)
Source-tree SHA-256	b63e06576b972f4df37e07fce95ae90ef62c57e3723f26cdc0a8400dcfe5a810
Pass-1 document SHA-256	6b2fd18fe47b49c5814369b7c83c983487023528553c895ddec4397f16035b20
Final document SHA-256	6b2fd18fe47b49c5814369b7c83c983487023528553c895ddec4397f16035b20
Registered model count	33
Automated tests	118 (all passing)
Worked examples	10
Reference count	33
Self-test	PASS
Python	3.11.15
Platform	Linux-6.18.44-fc-v22-x86_64-with-glibc2.39
Library versions	matplotlib 3.10.9, mpmath 1.3.0, numpy 2.4.4, reportlab 4.4.10, scipy 1.17.1, sympy 1.14.0
Licence	IP_NOTICE_v2.0

E.1 Source-tree digest, file by file

File	Bytes	SHA-256
__init__.py	1469	7a551dafca5fbec33a85e2f045172700e68f00b7fcdc1eca90aa5220cecd6da3e
adm.py	11981	47ba269dfcc1de6ea0a7ae1c9276351d44e3a84bc29f66df2b712d1bb5edab01
causality.py	20856	460e1b2e25f0693b7b5904c32730322a0b8fc74e7c4ec1ba59932e074b517162
cli.py	10701	a710a8784f8c40ea1781e8b3f91f793040eb7f0e54b3bb4c83ca2af5ccb719f2
continuation.py	19675	04d62ea844639d28dcb221c64d8e165c9295e4434b5d4f31394358d8316fb746
energy.py	27534	411aa5a9f594cc5169e4afb7ef1550906792297948ee0a883c246f01f6cd83fb
geodesics.py	11676	c1682f52ea64b363f4092e33b75f9afda0703093e5b6eaa3cfa59816dd469ff3
horizons.py	11133	379f73135f58d2bbfbad2f187e206c6ffbc1943775b717a51d55de16ffdf2f1c2
interval.py	15556	d222c779265429c97294eec5f957c51b9c171ca67489f520748d7a97d57277ab
loops.py	16939	57f308420fb1863af4e412fda0b6920cf0d4722969f2b293499ff79f8dbcb1b3
meta.py	10201	3db80da3884e94c11697e71e9863e61235ef20b0554e9a867119506d1201a051
metrics.py	60900	7246b20c44fe291ee43be5e71fb00f3c0fd25903ca8312d053285ac92469df3c
model.py	23498	f0d53c8f78b4335e975f19e736767a0869a3a1ea0ad0fe91ef52ead9e2c3140
numerics.py	9988	98358d4c155bf77731c61a2b6e3d614a266d05c17a63eb7785d1bbced956b5ca
numgeom.py	8793	e97b84cbffe21db5d17dad51eb5bb4740f6f76e91ee90f0a1835925f8a614a9d
optimize.py	13016	d647b17ed1f9ea94f318d1bb18a068162540a6cf59b6ed2237d68cae84b4218b
orientation.py	7399	c6a7f3de3f04e4bb4f59dd535b83a848a07447e68c2befae55adddf7d2dd8319
perturbation.py	8808	c352ee866fa4bc1d863e5cf236dd056988dd63057d84c8ef78f512cb1851c6c8
provenance.py	10464	ba766179977e4c78106b748bd05f2078bd51c0792f5c811e762acf4abffca715
qft.py	13761	51b3b4184e9a6f96597b24279769f6f67329f045e1b27fc6509cf2a361c6e6de
quotient_ctc.py	15601	5261303ec71f360597ebe284b15af3a882d0a6c220f79df935d5172b76ce4931
registry.py	4230	633ac2e13995f7ce5f66fddf54ee04409b8d06ae6d96a4e59b0dd8ab0e8e1ab1
report.py	11213	e0a3fd099bd4557be88a9361e5c47fe6283c0224f13794f636414b28e03493e3
status.py	15828	cb7ae81c61897309e94f4043c27d430cdabefe4cad1e78a645ef26830a95aca9
symbolic.py	12674	ddac654800d994c99120ebe63acc407ab49d5f1fa5f3b92428b83f070b4f9023
symmetry.py	8552	a7ca6f0f5c90243c00f41fe36b1268dc4f852b5597d699d704a1dd9dc4e26856

File	Bytes	SHA-256
theorems.py	14751	b055eaaf9ebd5bac7ddeaa19c560a1d6464365740269be993568f616d48d9462
topology.py	15521	9d57ea0c022710bdd81b36c59d2aba0054c18186acac69f0a7d4f5ff11ecacca
validate.py	9365	9401890880ca8772622e880f8e906e3e295a924d00a43343cb12d3e461bb6c06

Recomputing this table is one command: **python3 -m ctcfa.cli manifest --root src/ctcfa**. Any discrepancy against the published values means the source has changed since the document was built.

Appendix F. Machine-Readable Registry Records

One JSON record per model, validated against schemas/metric.schema.json. Two are reproduced here in full to show the format; the remaining 31 are in registry/metrics/. The pair below is the L3 gate pair, chosen so that a reader can verify by inspection that the metric_components blocks are identical and the quotient blocks are not.

registry/metrics/ads4_universal_cover.json

```
{
  "coordinates": [
    "t",
    "r",
    "theta",
    "phi"
  ],
  "default_point": {
    "L": 1.0,
    "phi": 0.0,
    "r": 1.0,
    "t": 0.0,
    "theta": 1.5707963267948966
  },
  "domain": [
    {
      "expression": "r",
      "name": "radial",
      "note": "",
      "strict": false
    },
    {
      "expression": "sin(theta)",
      "name": "polar_regular",
      "note": "",
      "strict": true
    }
  ],
  "domain_notes": "g^tt = -1/(1 + r^2/L^2) < 0 everywhere, so t is an exact global temporal function and the spacetime is stably causal.",
  "expected_scope": "GLOBAL",
  "expected_status": "NO_CTC_CERTIFIED",
  "expected_status_at_default_point": "NO_CTC_CERTIFIED",
  "global_margin": false,
  "global_margin_note": "",
  "global_margin_scope": "REDUCED_MODEL",
  "identifications": [
    {
      "coordinate": "phi",
      "index": 3,
      "note": "",
      "period": "2*pi",
      "requires_killing": true
    }
  ],
  "kind": "LOCAL_METRIC",
  "matter": {
    "Lambda": "-3/L**2",
    "T_defined": false,
    "label": "vacuum with a negative cosmological constant",
    "note": "",
    "vacuum": true
  },
  "metadata": {
    "pair": "ads4_periodic",
    "role": "L3_gate_pair"
  },
  "metric_components": {
    "00": "-1 - r**2/L**2",
    "11": "1/(1 + r**2/L**2)",
    "22": "r**2",
    "33": "r**2*sin(theta)**2"
  },
  "metric_defined": true,
  "model_id": "ads4_universal_cover",
  "parameters": [
    "L"
  ],
  "provenance": {
    "convention_note": "global AdS4 chart; the universal cover unwraps the periodic time of the embedded hyperboloid",
    "derivation": "",
    "external_spec": "",
    "implementation_status": "implemented",
    "source_refs": [

```

```

    "R26",
    "R27"
  ],
  "quotient": {
    "acts_freely": null,
    "generators": [],
    "label": "trivial",
    "note": ""
  },
  "signature": [
    1,
    0,
    3
  ],
  "signature_convention": "(-,+,+,+)",
  "temporal_function": "t",
  "theory": "GR with Lambda = -3/L^2",
  "title": "M23 AdS4, universal cover (t in R)",
  "topology": "R (time) x R^3; simply connected in the time direction",
  "version": "2.0"
}

```

registry/metrics/ads4_periodic.json

```

{
  "coordinates": [
    "t",
    "r",
    "theta",
    "phi"
  ],
  "default_point": {
    "L": 1.0,
    "phi": 0.0,
    "r": 1.0,
    "t": 0.0,
    "theta": 1.5707963267948966
  },
  "domain": [
    {
      "expression": "r",
      "name": "radial",
      "note": "",
      "strict": false
    },
    {
      "expression": "sin(theta)",
      "name": "polar_regular",
      "note": "",
      "strict": true
    }
  ],
  "domain_notes": "IDENTICAL local metric to ads4_universal_cover. The orbit of the Killing vector d/dt is closed and timelike everywhere",
  "expected_scope": "GLOBAL",
  "expected_status": "CTC_PROVED",
  "expected_status_at_default_point": "CTC_PROVED",
  "global_margin": false,
  "global_margin_note": "",
  "global_margin_scope": "REDUCED_MODEL",
  "identifications": [
    {
      "coordinate": "t",
      "index": 0,
      "note": "",
      "period": "2*pi*L",
      "requires_killing": true
    },
    {
      "coordinate": "phi",
      "index": 3,
      "note": "",
      "period": "2*pi",
      "requires_killing": true
    }
  ],
  "kind": "QUOTIENT_GLOBAL",
  "matter": {
    "Lambda": "-3/L**2",
    "T_defined": false,
    "label": "vacuum with a negative cosmological constant",
    "note": "",
    "vacuum": true
  },
  "metadata": {
    "pair": "ads4_universal_cover",
    "role": "L3_gate_pair"
  }
}

```

```

    },
    "metric_components": {
      "00": "-1 - r**2/L**2",
      "11": "1/(1 + r**2/L**2)",
      "22": "r**2",
      "33": "r**2*sin(theta)**2"
    },
    "metric_defined": true,
    "model_id": "ads4_periodic",
    "parameters": [
      "L"
    ],
    "provenance": {
      "convention_note": "AdS as a quadric in  $R^{2,3}$  has closed timelike curves; the universal cover removes them",
      "derivation": "",
      "external_spec": "",
      "implementation_status": "implemented",
      "source_refs": [
        "R26",
        "R27"
      ]
    },
    "quotient": {
      "acts_freely": true,
      "generators": [
        {
          "image": [
            "2*pi*L + t",
            "r",
            "theta",
            "phi"
          ],
          "inverse": [
            "-2*pi*L + t",
            "r",
            "theta",
            "phi"
          ],
          "name": "time_shift",
          "note": "the closed timelike Killing orbit of the AdS quadric",
          "time_orientation_preserving": true
        }
      ],
      "label": "Z generated by  $t \rightarrow t + 2 \pi L$ ",
      "note": ""
    },
    "signature": [
      1,
      0,
      3
    ],
    "signature_convention": "(-,+,+,+)",
    "temporal_function": null,
    "theory": "GR with  $\Lambda = -3/L^2$ ",
    "title": "M24 AdS4 hyperboloid, periodic time ( $t \sim t + 2 \pi L$ )",
    "topology": " $S^1$  (time)  $\times R^3$ ; the embedded quadric in  $R^{2,3}$ ",
    "version": "2.0"
  }
}

```

Appendix G. Self-Test Output

Verbatim output of `python3 -m ctcfcli selftest` during this build.

```
CTC-FA v2.0.0 self-test
  registry audit : PASS (33 models, 0 problems)
  V0/V1 gates   : PASS (33/33)
  regression    : PASS (33/33)
```

model	route	expected	actual	margin	uncertainty
ads4_periodic	compact_orbit	CTC_PROVED	SECTOR_CTC	-2.000000e+00	1.00e-10
ads4_universal_cover	compact_orbit	NO_CTC_CERTIFIED	SECTOR_SPACELIKE	+1.000000e+00	1.00e-10
alcubierre	temporal_function	NO_CTC_CERTIFIED	NO_CTC_CERTIFIED	-	-
btz_rotating	compact_orbit	NO_CTC_CERTIFIED	SECTOR_SPACELIKE	+9.000000e+00	1.00e-10
de_sitter_static	compact_orbit	NO_CTC_CERTIFIED	SECTOR_SPACELIKE	+2.500000e-01	1.00e-10
flrw_exponential	temporal_function	NO_CTC_CERTIFIED	NO_CTC_CERTIFIED	-	-
flrw_flat	temporal_function	NO_CTC_CERTIFIED	NO_CTC_CERTIFIED	-	-
godel	compact_orbit	CTC_PROVED	SECTOR_CTC	-2.055335e+01	1.00e-10
godel_critical	compact_orbit	UNRESOLVED	SECTOR_SPACELIKE	+1.315412e+01	1.00e-10
godel_type	compact_orbit	CTC_PROVED	SECTOR_CTC	-1.736478e+01	1.00e-10
gott_two_strings	global_predicate	CTC_PROVED	CTC_PROVED	-1.438754e+01	-
grant_space	deck_orbit	CTC_PROVED	SECTOR_CTC	-8.775451e+00	1.00e-10
kerr	compact_orbit	CTC_PROVED	SECTOR_CTC	-1.670000e+00	1.00e-10
kerr_extremal	compact_orbit	CTC_PROVED	SECTOR_CTC	-2.750000e+00	1.00e-10
kerr_newman	compact_orbit	CTC_PROVED	SECTOR_CTC	-1.772400e+00	1.00e-10
krasnikov	compact_orbit	UNRESOLVED	SECTOR_SPACELIKE	+1.000000e-02	1.00e-10
minkowski	temporal_function	NO_CTC_CERTIFIED	NO_CTC_CERTIFIED	-	-
minkowski_time_periodic	compact_orbit	CTC_PROVED	SECTOR_CTC	-1.000000e+00	1.00e-10
misner	compact_orbit	CTC_PROVED	SECTOR_CTC	-1.000000e+00	1.00e-10
misner_covering	deck_orbit	CTC_PROVED	SECTOR_CTC	-1.086161e+00	1.00e-10
morris_thorne	compact_orbit	NO_CTC_CERTIFIED	SECTOR_SPACELIKE	+4.000000e+00	1.00e-10
ori_2005	compact_orbit	CTC_PROVED	SECTOR_CTC	-1.000000e+00	1.00e-10
ori_2007_core	compact_orbit	CTC_PROVED	SECTOR_CTC	-1.000000e+00	1.00e-10
ori_2007_full	global_predicate	CTC_PROVED	NOT_APPLICABLE	-	-
paired_warp	global_predicate	UNRESOLVED	NOT_APPLICABLE	-	-
pseudo_reissner_nordstrom	compact_orbit	CTC_PROVED	SECTOR_CTC	-7.500000e-01	1.00e-10
reissner_nordstrom	compact_orbit	NO_CTC_CERTIFIED	SECTOR_SPACELIKE	+2.500000e+01	1.00e-10
schwarzschild	compact_orbit	NO_CTC_CERTIFIED	SECTOR_SPACELIKE	+2.500000e+01	1.00e-10
spinning_cosmic_string	compact_orbit	CTC_PROVED	SECTOR_CTC	-1.536000e-01	1.00e-10
taub_nut	compact_orbit	CTC_PROVED	SECTOR_CTC	-6.923595e-01	1.01e-10
time_shifted_wormhole	global_predicate	CTC_PROVED	CTC_PROVED	-1.000000e+00	-
tipler_cylinder	global_predicate	CTC_PROVED	NOT_APPLICABLE	-	-
van_stockum	compact_orbit	CTC_PROVED	SECTOR_CTC	-1.200000e+01	1.02e-10

CTC-FA v2.0.0 self-test: PASS

G.1 Test-suite output

```
..... [ 61%]
..... [100%]
===== slowest 12 durations =====
92.18s call tests/regression/test_regression.py::test_every_model_passes_the_structural_gates
11.54s call tests/regression/test_regression.py::test_meta_analysis_reports_held_out_scores_and_governance
9.11s call tests/analytic/test_analytic_benchmarks.py::test_grant_polarized_hypersurfaces_match_closed_form
6.97s call tests/regression/test_regression.py::test_every_row_records_its_derivation_route
6.93s call tests/controls/test_negative_controls.py::test_single_alcubierre_bubble_is_chronal_on_its_chart
6.79s call tests/regression/test_regression.py::test_regression_matrix_agrees_everywhere
4.40s call tests/regression/test_regression.py::test_vacuum_models_have_vanishing_field_equation_residual
3.70s call tests/regression/test_regression.py::test_energy_conditions_on_the_key_models
2.78s call tests/unit/test_engines.py::test_numeric_and_symbolic_einstein_agree_on_a_hard_metric
1.40s call tests/analytic/test_analytic_benchmarks.py::test_misner_covering_exact_orbit_interval
1.30s call tests/analytic/test_analytic_benchmarks.py::test_grant_exact_orbit_interval
1.25s call tests/analytic/test_analytic_benchmarks.py::test_godel_boundary_converges_to_two_omega
```

Appendix H. Notation and Convention Sheet

One page, so that a reader checking a formula against another source can find the convention immediately. Failure mode FM-019 — a published formula copied with an incompatible convention — is what this page exists to prevent, and defect D-01 is an instance of it occurring in v1.0.

Symbol / object	Convention used here	Pinned by
signature	$(-, +, +, +)$; reported as sign counts $(n_-, n_0, n_+) = (1, 0, d-1)$	numeric check at every evaluated point
units	geometric, $G = c = 1$	—
Γ^a_{bc}	$\frac{1}{2} g^{ad}(\partial_b g_{dc} + \partial_c g_{db} - \partial_d g_{bc})$	—
R^a_{bcd}	$\partial_c \Gamma^a_{db} - \partial_d \Gamma^a_{cb} + \Gamma \Gamma - \Gamma \Gamma$	Schwarzschild $K = 48M^2/r^6$
R_{bd}	R^a_{bad}	Schwarzschild $R_{ab} = 0$
G_{ab}	$R_{ab} - \frac{1}{2} g_{ab} R$	de Sitter $R = 4\Lambda$; AdS $R = -12/L^2$
field equation	$G_{ab} + \Lambda g_{ab} = 8\pi T_{ab}$	vacuum residuals $< 10^{-13}$
Kerr $g_{t\phi}$	$-2Mar \sin^2\theta/\Sigma$ (NEGATIVE)	frame-dragging direction; $a \rightarrow 0$ limit; defect D-01
Kerr Σ, Δ	$\Sigma = r^2 + a^2 \cos^2\theta$; $\Delta = r^2 - 2Mr + a^2$	$a = 0$ reduces to Schwarzschild
Gödel-type H, D	$H = (4\omega/m^2)\sinh^2(mr/2)$; $D = \sinh(mr)/m$; $g_{\phi\phi} = D^2 - H^2$	$m^2 = 2\omega^2$ is the Gödel member
Gott condition	$\gamma > 1/\sin(4\pi\mu)$, equivalently $\gamma v > \cot(4\pi\mu)$; μ is $G\mu/c^2$	algebraic equivalence verified
spinning string	$ds^2 = -(dt + 4J d\varphi)^2 + dr^2 + \alpha^2 r^2 d\varphi^2 + dz^2$	boundary $\alpha r = 4J$
Misner	$ds^2 = -2 dt d\psi - T d\psi^2 + dy^2 + dz^2$	$\det g \equiv -1$
Krasnikov	Everett–Roman form; the (t,x) block has determinant $\equiv -1$	exact algebraic identity, tested
ADM	$K_{ij} = (1/2N)(D_i N_j + D_j N_i - \partial_t h_{ij})$	FLRW reproduces the Friedmann equation to 12 digits
SEC margin	$\min((\rho + \Sigma p)/2, \min_i(\rho + p_i))$	the textbook inequality $\rho + \Sigma p \geq 0$ is the same condition with the factor of two divided out
reduced causal margin	$\mu = g(\xi, \xi)/h(\xi)$, $h = g(\xi, \xi) - g(\xi, u)^2/g(u, u) > 0$	collapses to the literature quantity for every standard family